若有疑问，请发邮件至 ：support@deltatau-china.com 或致电：010 6439 2833
或在 QQ 群里找用户：1556419980 留言

用户手册

Turbo PMAC

可编程多轴控制器
3Ax-602264-TUxx
2008 年 9 月 12 日



单源机械控制 *功能强大//灵活//使用方便*

北京阜通东大街 6 号院方恒国际 C 座 1608

版权信息

©2014 北京泰道控制系统设备有限公司 版权所有，侵权必究。

本手册提供给泰道公司的客户使用。未经泰道公司的书面许可，不得用于其他用途。本手册中的信息会随着产品的改进而实时更新，所以不可能与之前版本的信息完全符合。

工作条件

泰道公司所有的运动控制器产品、配件、放大器都包含有静电敏感元件，不正确的操作会引起其损坏。当安装或操作泰道公司的产品时，避免与高度绝缘材料接触。只有合格人员才允许操作本设备。

在工业应用的情况下，我们希望我们的产品受到保护，应该避免接触危险的或导电的材料和（或）环境，因为这些因素会损害元器件或者造成短路从而引起控制器受到损坏。在工业环境中使用我们的产品时，应该将其安装到工业电气柜或工业计算机中，以保护他们免于接触过量的或腐蚀性的潮湿、异常的环境温度和导电材料。如果泰道公司的产品被暴露在危险或导电材料和（或）环境下，我们不能保证它们的正常工作。

目录

第一章 使用本手册.....16
1-1 本手册内容.....16
1-2 需使用的其他手册.....16
Turbo PMAC 软件参考手册.....16
硬件参考手册.....16
UMAC 快速参考指南.....17
PMAC 执行程序手册.....17
软件开发包手册.....17
第二章 TURBO PMAC 系列.....18
2-1 Turbo PMAC 和 Non-Turbo PMAC.....18
2-2 Turbo PMAC 和 Turbo PMAC2.....18
2-3 Turbo PMAC 是一台计算机.....18
2-4 Turbo PMAC 做什么.....19
执行运动程序.....19
执行 PLC 程序.....19
伺服环更新.....19
换相更新.....20
内务管理.....20
与主机通信.....20
任务优先级.....20
2-5 关键硬件组件.....20
CPU 部分.....21
机械接口 IC.....23
通信端口.....28
2-6 关键的软件组件.....29
操作固件.....29
引导固件.....29
电机、轴和坐标系.....29
用户程序.....30
2-7 Turbo PMAC 配置.....31
板级设计.....31
UMAC 机架式安装设计.....31
QMAC 箱体式设计.....32
第三章 TURBO PMAC/PMAC2 系统配置和自动配置.....34
3-1 CPU 时钟频率.....34
3-2 Turbo PMAC2 的系统时钟源.....35
默认时钟源.....35
IC 时钟发生器设备.....35
典型的时钟源集成电路.....36
外部时钟源.....36
时钟信号的分配.....37
丢失时钟信号.....37

重新初始化动作.....38
用户定制的时钟源规格.....39
正常复位操作.....40
3-3 MACRO IC 选型.....40
3-4 双端口 RAM（DPRAM）的 IC 选型.....41
3-5 系统配置状态报告.....41
伺服芯片配置.....41
MACRO 芯片配置.....42
双口 RAM 芯片配置.....42
CPU 区域的配置.....42
UBUS 配件板识别.....42
3-6 设置系统时钟频率.....43
设置 Turbo PMAC 的相位和伺服时钟频率.....43
设置 Turbo PMAC2 的相位和伺服时钟频率.....45
设置 I10 伺服更新时间参数.....47
3-7 在 MACRO 环上设置一个 Turbo PMAC2.....47
MACRO 环频率控制变量.....48
I7：相位周期延拓.....48
I6840：MACRO IC0 主控制器配置.....49
I6890/I6940/I6990：MACRO IC1/2/3 的主控制器配置.....49
I6841/I6891/I6941/I6991：MACRO IC0/1/2/3 节点激活控制.....49
I70/I72/I74/I76：MACRO IC0/1/2/3 节点辅助功能使能.....50
I71/I73/I75/I77：MACRO IC0/1/2/3 节点协议类型控制.....51
I78：MACRO 主/从辅助通讯超时.....51
I79：MACRO 主/主辅助通信超时.....51
I80、I81、I82：MACRO 环检查周期和限制.....52
MACRO 节点地址.....52
3-8 复位和重新初始化 Turbo PMAC.....55
复位的方法.....55
一个正常复位的动作.....56
重新初始化下的复位的动作.....56
用于固件重载的复位的动作.....57
重新初始化和清除指令.....57
第四章 与 TURBO PMAC 通讯.....58
4-1 通信端口.....58
串行通信端口.....58
总线通讯端口.....61
双端口 RAM.....63
4-2 给 Turbo PMAC 发送指令.....64
Turbo PMAC 的指令处理.....64
指令确认.....65
数据响应.....65
数据完整性.....65
数据响应格式.....65

4-3 在线（即时）指令 65
在线指令的类型 66
4-4 缓冲的（程序）指令 67
第五章 设置反馈和主位置传感器 69
5-1 设置正交编码器 69
信号格式 69
硬件设置 70
Turbo PMAC 硬件控制参数设置 72
转换表处理设置---Turbo PMAC 接口 74
转换表处理设置---MACRO 站接口 74
缩放反馈单位 74
5-2 设置数字霍尔传感器 74
信号格式 75
硬件设置 75
Turbo PMAC 硬件-控制参数设置 76
上电相位使用 76
转换表处理设置-Turbo PMAC 接口 77
转换表处理设置 - MACRO 站接口 77
缩放反馈单位 77
5-3 设置正弦编码器 77
硬件设置 78
Turbo PMAC 硬件控制参数设置 78
转换表处理设置 - Turbo PMAC 接口 79
转换处理设置---MACRO 站接口 80
缩放反馈单位 81
5-4 设置旋转变压器 81
硬件设置 82
Turbo PMAC 硬件控制参数设置 82
转换表处理设置 --Turbo PMAC 接口 82
转换表处理设置 - MACRO 站接口 82
设置为上电绝对位置 82
缩放反馈单位 83
5-5 设置 MLDTs 83
MLDT 接口类型 84
信号格式 84
硬件设置 85
Turbo PMAC 硬件控制参数设置 85
转换表处理设置 –Turbo PMAC 接口 88
转换表处理设置 - MACRO 站接口 88
设置为上电绝对位置 89
缩放反馈单位 89
5-6 设置 LVDT、RVDT 与其他模拟量 90
硬件设置 90
Turbo PMAC 硬件控制参数设置 90

转换表处理设置--Turbo PMAC 接口 91
转换表处理设置—MACRO 站接口 91
设置为上电绝对位置 92
缩放反馈单位 92
5-7 设置绝对编码器 92
第六章 基本电机设置 94
6-1 硬件设置 94
6-2 设置基本电机运行的参数 94
6-3 初始设置参数 95
激活电机：Ixx00 95
激活 PMAC 电机换相：Ixx01 96
6-4 电机地址设置参数 96
指令输出地址：Ixx02 97
位置环反馈地址：Ixx03 97
速度环反馈地址：Ixx04 97
标志地址：Ixx25，Ixx42，Ixx43 98
标志模式：Ixx24 99
绝对上电位置地址和格式：Ixx10 和 Ixx95 100
上电模式：Ixx80 100
6-5 Turbo PMAC 为该电机换相还是闭合电流环？ 101
6-6 设置 Turbo PMAC 的速度或转矩控制 101
硬件设置 101
Turbo PMAC 的参数设置 102
6-7 设置 Turbo PMAC2 为脉冲和方向控制 104
硬件设置 104
信号定时 105
Turbo PMAC 的参数设置 105
对设置进行测试 110
第七章 设立基于 TURBO PMAC 的换相和/或电流环 112
7-1 开始换相设置 112
换相启用：Ixx01 112
换相周期大小：Ixx70＆Ixx71 112
换相反馈地址：Ixx83 113
Turbo PMAC 的电流环或不是 114
7-2 设置为直接 PWM 控制 114
介绍 115
数字电流环的操作原理 115
Turbo PMAC 的参数设置 119
对于有刷电机的直接 PWM 控制的特别说明 125
测试 PWM 和电流反馈操作 126
位置反馈和极性测试 127
建立基本电流回路操作 131
7-3 设置 Turbo PMAC 为正弦波输出控制 135
硬件设置 136

Turbo PMAC 的参数设置 136
建立基本输出操作 139
7-4 完成设定 Turbo PMAC 换相（直接 PWM 波或正弦波），同步电动机 140
确认换相极性匹配 140
建立一个相位参考 141
7-5 完成设定 Turbo PMAC 换相（直接 PWM 波或正弦波），异步（感应）电机 153
计算 Ixx78 转差率常量 153
设置 Ixx77 磁化电流 155
7-6 电动机的直接微步进（开环换相） 155
设置 I-变量 156
7-7 下一步做什么 159
7-8 用户编写的相位算法 159
编写算法 160
第八章 设置编码器转换表 162
8-1 转换表执行 163
8-2 转换表结构 163
设置结构 163
结果结构 166
使用结果 167
8-3 转换方法 167
数字正交编码器 168
模拟量正弦/余弦编码器 169
ACC-28 模拟 - 数字转换器 170
并行反馈 171
指数滤波器 172
和与差 173
时基（微分） 174
第九章 设置伺服环 176
9-1 伺服更新率 176
改变伺服更新率的原因 176
更改伺服更新率的影响 177
9-2 放大器的类型 177
伺服产生速度指令的放大器 177
伺服产生转矩/力命令的放大器 178
9-3 PID/前馈/陷波伺服滤波器 179
PID 反馈滤波器 180
前馈滤波器 180
实际的 PID/前馈算法 181
陷波滤波器 182
伺服回路修改器 186
9-4 扩展的伺服算法 188
9-5 级联伺服环 191
选择使用的 Turbo PMAC 电机 191

内环常规设置.....192
外环常规设置.....192
连接环路.....193
调整外环.....194
编程外环电机.....194
设置示例.....194
改变控制模式.....195
扩展单环设置的特别说明.....196
9-6 用户编写的伺服算法.....197
开放式伺服编译算法.....197
汇编的用户编写的伺服算法.....209
第十章 电机补偿表和常数.....211
10-1 位置补偿表.....211
源、目标和分配的电机.....211
表的维度.....213
使用期望的与实际的位置.....214
每个电机多个表.....214
表的范围.....214
确定补偿值.....215
输入表格.....215
启用和禁用表.....217
修正的有效计算.....217
报告表格信息.....218
删除表格.....218
10-2 间隙补偿.....218
常数间隙参数.....219
间隙参与率.....219
间隙滞后.....219
间隙补偿表.....219
启用和禁用间隙.....220
间隙表示例.....220
10-3 转矩补偿表.....222
输入表格.....222
报告表格信息.....223
启用和禁用表格.....223
校正的有效计算.....223
删除表格.....223
第十一章 TURBO PMAC 通用 I/ O 的使用.....224
11-1 Turbo PMAC 的通用 I/ O（JOPTO）端口.....224
硬件特性.....224
软件访问.....225
11-2 Turbo PMAC 多路复用 I/ O（JTHW）端口.....225
11-3 Turbo PMAC 控制面板端口.....225
控制面板输入.....225

控制面板输出.....226
11-4 Turbo PMAC2 通用 I/ O（JIO）端口.....226
硬件特性.....226
建议的 M-变量.....227
方向控制.....227
反相控制.....228
备用用途.....228
11-5 Turbo PMAC2 多路复用 I/ O 端口（JTHW）.....228
复用器端口配件.....229
硬件特性.....229
建议的 M-变量.....229
方向控制.....230
反相控制.....230
备用用途.....230
11-6 Turbo PMAC 模拟输入（JANA）端口.....231
硬件特性.....231
复用原理.....231
解除复用的 I-变量.....232
11-7 UMAC 数字 I/ O 板卡.....233
寻址 UMAC I/O 板.....233
设置 UMAC　I/O 板.....234
第十二章 确保应用程序安全.....237
12-1 跟随误差极限.....237
故障跟随误差限制.....237
警告跟随误差极限.....237
跟随误差的积分保护.....238
12-2 位置（超程）限制.....238
硬件超程限位开关.....238
软件超程限位变量.....239
12-3 速度极限.....240
矢量速度限制.....240
电机速度限制.....241
12-4 加速度限制.....241
12-5 指令输出限制.....241
12-6 积分电流（I^2T）保护.....242
12-7 放大器启用和故障线.....244
12-8 编码器丢失检测.....244
12-9 用户编写的安全算法.....246
12-10 看门狗定时器.....246
看门狗定时器跳闸时的动作.....246
诊断看门狗定时器跳闸.....247
12-11 硬件停止指令输入.....247
12-12 主机产生的停止指令.....248
12-13 程序校验和.....249

固件校验和.....249
用户程序校验和.....249
12-14 通信数据完整性.....249
第十三章 执行电机的基本移动.....250
13-1 手动移动控制.....250
手动加速度.....250
手动速度.....251
手动命令.....252
13-2 触发的电机移动.....253
触发的移动的类型.....254
触发条件的类型.....254
捕获触发位置.....255
后触发移动.....256
回零搜索移动.....256
慢跑（手动）-直到-触发.....264
运动程序移动-直到-触发.....265
13-3 开环移动.....266
第十四章 TURBO PMAC 的计算功能.....267
14-1 计算优先级.....267
单字符的 I / O.....267
换相更新.....267
伺服更新.....268
实时中断任务.....268
VME 邮箱处理.....268
后台任务.....269
优先级的优化.....271
14-2 评估 Turbo PMAC 的计算负荷.....271
相位中断任务.....271
伺服中断任务.....272
实时中断任务.....272
总中断任务.....272
示例监控程序.....273
后台周期时间.....273
14-3 数字值.....273
内部格式.....274
接收值.....274
报告值.....275
14-4 地址.....275
14-5 变量.....275
I-变量.....276
P-变量.....277
Q-变量.....277
M-变量.....279
14-6 运算符.....281

算术运算符.....281
逻辑运算符.....281
14-7 函数.....282
14-8 表达式.....282
14-9 {数据}语法.....282
14-10 变量值赋值语句.....283
I-变量默认赋值.....283
同步的 M-变量赋值.....283
14-11 比较符.....286
14-12 条件.....286
简单条件.....286
复合条件.....287
单行条件操作.....287
多行条件.....287
14-13 定时器.....288
14-14 计算的注意事项.....288
第十五章 设置一个坐标系.....291
15-1 什么是坐标系？.....291
15-2 什么是轴？.....291
单-电机的轴.....291
多电机轴.....292
虚拟轴.....292
15-3 轴定义.....292
匹配电机到轴.....292
缩放和偏移.....292
轴类型.....293
从轴位置转换到电机位置.....295
从电机位置转换到轴位置.....295
15-4 坐标系统的运动学计算.....296
创建运动学程序缓冲区.....297
带运动学的坐标系统转换.....304
执行运动学程序.....305
15-5 坐标系统时基.....307
第十六章 编写和执行运动程序.....308
16-1 执行运动程序序列.....308
16-2 流程控制.....308
16-3 G 代码.....308
16-4 模态指令.....309
16-5 移动命令.....309
16-6 运动程序轨迹.....309
16-7 线性混合运动.....309
位置或距离说明.....310
进给率或移动时间说明.....310
加速度参数.....311

加速度限制.....313
最小运动时间.....315
最大运动时间.....315
混合功能.....315
16-8 圆弧混合运动.....320
指定插补平面.....320
圆弧模式.....320
圆心矢量.....321
半径尺寸说明.....322
非圆心说明.....323
矢量进给轴.....323
圆弧半径误差.....323
移动分段模式.....324
16-9 快速模式的移动.....324
运动时间.....325
运动路径.....325
无混合.....325
运动程序的移动--直到--触发.....325
改变目的地的移动.....327
16-10 样条曲线模式移动.....328
工作原理.....328
增加段.....329
位置调整量化.....329
5 点样条修正.....329
非均匀样条.....329
16-11 PVT 模式移动.....331
模式声明.....332
运动语句.....332
Turbo PMAC 的计算.....332
步进中的问题.....332
利用 PVT 创建任意轮廓.....332
PVT 在轮廓加工中的使用.....333
16-12 刀具半径补偿.....334
定义补偿平面.....335
定义补偿幅度（刀具半径）.....335
开启补偿.....336
关闭补偿.....336
Turbo PMAC 如何引入补偿.....336
补偿的内角处理.....338
外角处理.....339
完全反向的处理.....341
注意完整的圆.....342
补偿移动的速度.....343
补偿中的变化.....343

刀具补偿中的故障.....346
刀具补偿的模块缓冲.....348
在补偿中的单步执行.....348
16-13 三维刀具半径补偿.....349
定义 3D 补偿的幅度（刀具半径）.....349
开启 3D 补偿.....350
关闭 3D 补偿.....350
声明表面法向量.....350
声明刀具定向矢量.....351
如何进行 3D 补偿.....351
16-14 Turbo PMAC 的前瞻功能.....352
快速说明：设置前瞻.....354
详细说明：设置使用前瞻.....355
通过前瞻运行一个程序.....360
在前瞻中停止.....362
在前瞻中反转.....364
16-15 轴转换矩阵.....365
建立矩阵.....366
使用矩阵.....366
计算的影响.....367
示例.....367
16-16 输入一个运动程序.....368
学习一个运动程序.....369
运动程序结构.....369
运行一个运动程序.....373
16-17 实现一个机床类型的程序.....374
G、M、T 和 D-代码.....374
标准 G 代码.....374
主轴程序.....378
标准的 M 代码.....379
默认条件.....381
16-18 旋转运动程序缓冲区.....382
定义一个旋转缓冲区.....382
准备运行.....382
打开用于输入.....382
保持领先于执行行.....383
关闭和删除缓冲区.....384
16-19 PMAC 如何执行一个运动程序.....384
计算随后的移动.....385
如果没有提前计算.....386
第十七章 同步 TURBO PMAC 到外部事件.....389
17-1 位置跟随（电子齿轮）.....389
位置跟随 I-变量.....389
动态改变比率.....390

在编程移动中叠加跟随.....391
17-2 外部时基控制（电子凸轮）.....391
什么是时基控制？.....392
使用外部时基信号的说明.....393
时基示例.....394
触发的时基.....396
17-3 Turbo PMAC 与其他的 Turbo PMAC 同步.....399
时钟定时.....399
运动程序定时.....401
17-4 硬件位置捕捉功能.....402
硬件捕获需求.....402
设置触发条件.....402
自动移动--直到--触发功能.....403
手动使用捕捉功能.....403
转换到电机坐标和轴坐标.....406
17-5 使用 Turbo PMAC 上的位置-比较功能.....407
位置-比较寄存器的缩放和偏移.....408
在一个 PMAC 式伺服 IC 上设置.....408
在一个 PMAC2 式伺服 IC 上设置.....409
从电机坐标和轴坐标进行转换.....414
17-6 同步 M-变量赋值输出.....416
第十八章 编写和执行 PLC 程序.....417
18-1PLC 程序是什么?.....417
何时使用.....417
常见的用途.....417
64 个 PLC 程序.....417
18-2 输入一个 PLC 程序.....418
打开缓冲区.....418
下载程序.....418
关闭缓冲区.....419
删除程序.....419
例子.....419
18-3 PLC 的程序结构.....419
计算语句.....419
条件语句.....420
18-4 编译的 PLC 程序.....422
执行编译的 PLC.....423
编写编译的 PLC 程序.....423
速度的优化.....427
内存利用率 7.....428
编译 PLC.....428
运行编译的 PLC.....429
第十九章 写一个主机的通信程序.....430
19-1 Turbo PMAC 的命令/响应格式.....430

响应类型.....430
变型.....431
清除端口.....431
19-2 串口通信.....432
设置接口.....432
发送一个字符.....433
读取一个字符.....433
19-3 ISA/ PCI 主机端口通信.....433
主机端口结构.....433
寄存器功能.....434
查询简单通信的寄存器.....434
设置端口.....434
发送一个字符.....435
读取一个字符.....435
ISA / PCI 中断.....435
初始化中断控制器.....439
19-4 VME 总线通讯.....440
设置 VME 通讯.....440
VME 邮箱寄存器通讯.....440
19-5 双端口 RAM 通信.....446
物理配置和连接.....446
19-6 主机地址设置.....446
ISA 总线设置.....447
VME 总线设置.....448
内存地址映射.....449
DPRAM 自动功能.....450
DPRAM 数据格式.....451
DPRAM 电机数据报告缓冲区.....451
DPRAM 后台数据报告缓冲区.....453
DPRAM 的 ASCII 通讯.....454
DPRAM 通讯中断.....455
DPRAM 后台变量读取缓冲区.....456
DPRAM 后台变量数据写缓冲区.....458
双口 RAM 的二进制旋转程序传输缓冲区.....460
DPRAM 数据采集缓冲区.....462
19-7 Turbo PMAC 以太网协议.....462
PMAC 以太网协议命令数据包说明.....463
Turbo PMAC 的以太网协议的命令集.....464
19-8 数据采集.....471

第一章 使用本手册

这是 DeltaTau 旗下的 Turbo PMAC 系列的机械运动控制器的用户手册。Turbo PMAC 系列产品在多种配置中结合了功能、灵活性和易操作性，从而为机械制造商提供最佳的解决方案。

1-1 本手册内容

本手册包含了逐步的指导来建立 Turbo PMAC 的应用程序，从整个系统的配置开始，接着是各个电机的设置以及将电机组合成坐标系，其次是编写运动程序、PLC 程序和主机通信程序。

用户手册分为几部分，这几部分是按照大多数新用户在应用程序的基本设置过程中所需要的顺序来排列的。在开发的后期阶段，典型的用户将来回翻阅本手册，来获取关于他们感兴趣的特定主题的更多详细信息。

1-2 需使用的其他手册

在 Turbo PMAC 项目的开发过程中，应该参考其他手册。其中最重要的这些手册列表如下：

- Turbo PMAC 软件参考手册
- 硬件参考手册
- UMAC 快速参考指南
- PMAC 应用程序手册
- 软件开发包手册

Turbo PMAC 软件参考手册

Turbo PMAC 系列控制器的软件参考手册对所有命令和设置变量都有详细的说明，并且按照按字母或数字顺序排列以方便参考。它还包含了关于 Turbo PMAC 系列的一个详细的内存和 I/O 映射，以及建议的 M-变量指针定义和 Turbo PMAC 嵌入式固件的更新列表。

如果对一个特定的 Turbo PMAC 设置变量、命令或寄存器的操作有具体问题，请查阅《软件参考手册》的相应部分获取最快的答案。

硬件参考手册

对控制器来说，一个特定的 Turbo PMAC 硬件配置都有自己的《硬件参考手册》，而且任何配件也都有自己的《硬件参考（用户）手册》。他们将是最先被查阅的使用手册，因为他们解释如何配置和安装 Turbo PMAC。

UMAC 快速参考指南

对于新用户来说，Turbo PMAC 系列的 UMAC 机架安装配置要参考《UMAC 快速参考指南》。本指南为 UMAC 及其配件的硬件和软件设置提供了一个很好的概述，因此更详细的手册会较少使用。

PMAC 应用程序手册

实际上，几乎所有用户都会在计算机上利用 PMAC 执行程序进行开发，即使在最终应用中不会是一台计算机。截止本文写作时最常用的执行程序是 Pewin32 Pro（用于 32 位 Windows 的 PMAC 执行程序，Pro Suite）。由于这个程序集的安装和使用非常直观，并符合行业标准，因此请查阅关于此软件包的手册。

其实，Pewin32 Pro 软件包是一套软件程序，包括逐步的安装程序教程、调试程序（交互式和自动调整）和绘制程序。

该 Pewin32 Pro 软件集自动化或简化了本手册中许多用以详细解释低级别设置的设置步骤。尽可能使用 Pro Suite 工具来进行自动设置。

软件开发包手册

泰道公司提供了多种软件开发包，旨在能够快速轻松地开发前端软件，用于与 Turbo PMAC 进行通讯、与机器操作员进行接口。每个软件包都有自己的手册。

最常用的两种软件包是通信例程的 PComm32 Pro 库（32 位 Windows 的 PMAC 通讯库,Pro suite）和 PHMI（PMAC 人机界面）GUI(图形用户界面)软件开发包。

第二章 TURBO PMAC 系列

Turbo PMAC 系列控制器是泰道公司的最新一代的运动和机械控制器。它拥有广泛的可用配置，允许用户优化控制器的硬件和软件以适应特定的应用需求。本节简要概述 Turbo PMAC 的结构；这里提到的所有入口在用户手册的其它地方或相关的参考手册均有更多详细的解释信息。

2-1 Turbo PMAC 和 Non-Turbo PMAC

Turbo PMAC 系列的控制器是继原始 PMAC 系列控制器后的新一代运动控制器，不同之处只是在于 Turbo PMAC 有一个更快、更强大的 CPU 部分，但能够维护保持相同的接口电路。事实上，在某些配置中，可以从一个运动控制器中删除旧的 non-TURBO PMAC 的 CPU 板，然后再安装 TURBO PMAC 的 CPU 进行现场升级。

Turbo PMAC 系列运动控制器保留了与原始 PMAC 系列运动控制器相同的功能和编程风格，但同样也增加了重要的新功能。这些新功能全都罗列在了软件参考手册的一个章节之中，并且在用户手册和软件参考手册的相应章节有详细的信息介绍。下面列出的是 Turbo PMAC 较之于原始 PMAC 的亮点：

- 32 个电机的容量 VS. 8 个电机
- 16 个坐标系容量 VS. 8 个坐标系
- 前向和逆向运动学算法
- 标准的多步前瞻算法
- 32,768 个用户变量 VS. 4096 个
- 在多个端口进行同步通讯

2-2 Turbo PMAC 和 Turbo PMAC2

与原始 PMAC 系列控制器一样，Turbo PMAC 系列可以是 Turbo PMAC 或者 Turbo PMAC2 控制器。不同之处在于“核心”伺服接口 IC(集成电路)的性质和对它们进行设置的固件有几个方面的不同。从根本上说，PMAC2 式伺服集成电路允许对关键时钟信号的频率进行可编程配置，时钟信号用来驱动控制器上硬件和软件处理，而 PMAC 式的伺服集成电路不允许该配置。

不同于旧的 Non-Turbo PMAC，可以将 PMAC 型和 PMAC2 型伺服集成电路混合在一个 Turbo PMAC 系统中。Turbo PMAC 的 CPU 将自动识别出当前的 IC 类型，并相应地配置设置变量 I-变量。

在本文档中，Turbo PMAC 一般指的是 Turbo PMAC 和 Turbo PMAC2 控制器两者。而 Turbo PMAC 和 Turbo PMAC2 则是专指一种类型的控制器。如果一个特定的产品名称（例如 Turbo PMAC-PC）没有具体指定是 1 或者 2，那么它指的是一个 PMAC 控制器。

2-3 Turbo PMAC 是一台计算机

很重要的一点是要认识到，Turbo PMAC 本身就是一台完整的计算机，它能

够使用其自己的存储程序独立地工作。此外，它是一台实时的、多任务处理的计算机，它可以划分任务优先级，并可以抢先较低优先级的任务来处理更高优先级的任务（大多数个人计算机做不到这一点）。

即使在与主机一起使用时，二者之间的通信也应该被视为是从一台计算机到另一台的通信，而不是电脑与外围设备之间的通信。在这些应用中，Turbo PMAC 具有同时运行多个任务、适当地划分任务优先级的能力，这就使得它可以在处理器的处理时间和任务切换的复杂性这两方面，为电脑主机（和它的编程器）减去一个巨大的负担。

2-4 Turbo PMAC 做什么

Turbo PMAC 可以处理所有机械控制需要的任务，每秒钟可以不断地在不同的任务之间来回切换数千次。机械控制涉及的主要任务总结如下。

执行运动程序

Turbo PMAC 最明显的任务是执行一个运动程序赋给它的运动序列。当被告知要执行一个运动程序时，Turbo PMAC 执行程序时是一次执行一步运动，并将到这步运动命令（包括非运动任务）相关的所有计算完成，为运动的实际执行做准备。Turbo PMAC 始终工作在实际的运动之前，所以如果需要的话也可以适当地混合即将执行的运动。有关更多详细信息，参见《编写一个运动程序》 的章节。

执行 PLC 程序

运动程序的有序性很适合它指挥命令一系列的运动动作以及其他的协调动作，但是这些程序并不擅长处理那些与运动顺序不直接协调的运动动作。对于这一类型的任务，Turbo PMAC 为用户提供了编写“PLC 程序”的功能。这些都是以可编程逻辑控制器（PLC）命名的，因为他们以同样的方式工作，以在处理器允许的时间尽可能快速持续地扫描其操作。对于与运动序列异步运行的任务来说，这些程序是非常有用的。有关更多详细信息，参考《编写 PLC 程序》 的章节。

伺服环更新

在自动执行的任务中，对 Turbo PMAC 用户而言本质上是不可见的，Turbo PMAC 对于每个电机以一个固定的频率（通常大约 2 千赫兹）执行一次伺服更新。对于电动机的伺服更新包括：根据由运动程序或其他运动命令生成的方程组来递增指令位置（如果必要的话），将该位置与从反馈传感器中读取的实际位置作比较，并基于二者的差值计算一个输出指令。这个任务会自动发生而不需要任何明确的指令。了解更多详细信息，请参阅《设置伺服环》的章节。

换相更新

如果要求 Turbo PMAC 为一个多相电机执行换相，它会自动以一个固定的频率（通常大约为 9 kHz）进行换相的更新。对电动机的换相或相位更新包括：测量和（或）估计转子磁场定向，然后在电动机的不同相之间按比例分配由伺服更新计算的指令。该任务自动发生而不需要任何明确的指令。有关更多详细信息请参考《设定换相》的章节。

内务管理

Turbo PMAC 定期自动执行内务管理，确保系统处于良好的工作状态。这些任务包括安全检查，如跟随误差限制、硬件超程限制、软件超程限制、放大器故障以及一般的状态更新。它们还包括看门狗定时器的更新。如果硬件或软件的任何问题阻碍了这些任务的执行，那么看门狗定时器将跳闸，并且该卡将关闭。更多的细节请参阅《确保使应用程序安全》的章节。

与主机通信

Turbo PMAC 可以在任何时间，即使处于运动序列的运行之中，都可以与一个或多个主机进行通讯。Turbo PMAC 将接受一个指令，并采取相应的行动---将指令放在程序缓冲区中以备后续执行、为主机提供数据响应、启动电机运动等。如果指令是非法的，它会向主机报告错误。

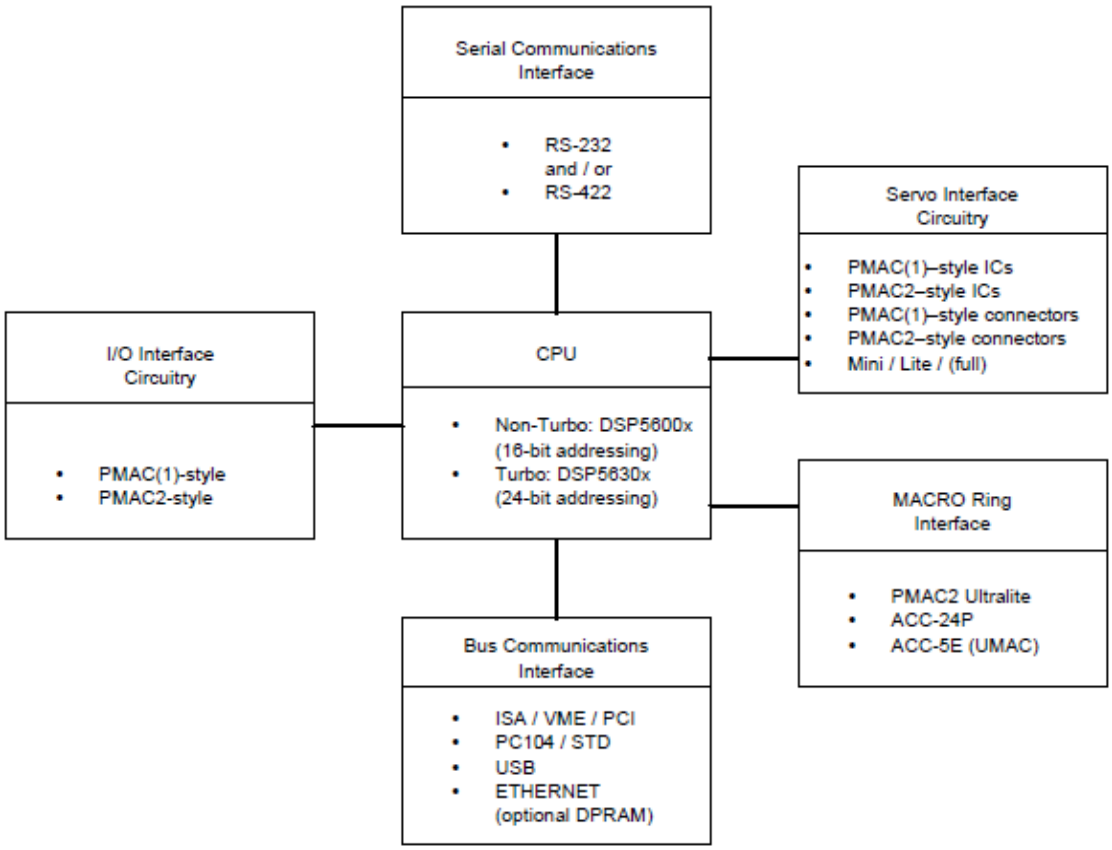
任务优先级

这些任务以优先级方案进行有序排列，优先级方案进行了优化以保持应用程序高效、安全地运行。虽然优先级是固定的，但是不同任务执行的频率是由用户控制的。更多细节请参见《计算的特征》 一节。

2-5 关键硬件组件

Turbo PMAC 控制器是将一个计算机处理器部分和用于运动、I/O 口、通讯的专门接口电路结合起来。Turbo PMAC 控制器下的不同配置为这些组件提供了不同的组合。下面的框图显示了 PMAC 和 Turbo PMAC 控制系统中的基本可能的概念组合。

PMAC 系列主要组件



CPU 部分

一个 Turbo PMAC 的计算机部分通常被称为 CPU（中央处理单元）。它由一个微处理器、存储器以及相关的支持电路组成。在大多数的 Turbo PMAC 的配置中，CPU 位于接口电路的一个独立电路板上---一个位于扩展槽控制器顶部的搭载板或一个 UMAC 控制器机架安装情况下的专用板。然而，在-Lite 版和-UltraLite 版控制器中，该电路被集成到主电路板上。

处理器

Turbo PMAC 控制器的处理器使用来自摩托罗拉 DSP56300 系列的数字信号处理器（DSP）。DSP 是一种特殊类型的微处理器，用来对在音频、视频和运动控制中常用的快速和重复的数学运算进行优化处理。与通用微处理器相比，它们为这些特定类型的计算提供了一种更经济高效的解决方案。

根据为 Turbo PMAC 选择的 CPU，使用 DSP56300 系列中不同的处理器。

- 选项 5Cx(x 指定外部存储器大小--见下文)提供的是 80MHz 的 DSP56303， 拥有 8K 的 24 位字的内存。这是默认的处理器。
- 选项 5Dx 提供一个 100 MHz 的 DSP56309，拥有 34K 的 24 位字的内存。
- 选项 5Ex 提供一个 160 MHz 的 DSP56311， 128K 的 24 位字的内存。这些选项要求的固件版本为 V1.939 或更高版本。
- 选项 5Fx（预计 2003 第三季度发行）提供一个 240 MHz 的 DSP56321， 192K 的 24 位字的内存。这些选项要求固件版本为 V1.940 或更高版本。

并非所有的这些选项都适用于所有的 Turbo PMAC 配置。

注意

仅仅是因为处理器能够在一个特定的频率上工作并不意味着它实际上是在该频率运行。上电或复位时，Turbo PMAC 的变量 I52 的值控制着处理器的工作频率为：10MHz*（I52+1）。对于 80 MHz 的操作，I52 的默认值是 7，而不管 CPU 的速度选项。**TYPE** 指令将显示 CPU 的实际运行频率。

要定位 CPU 部件的编号，发出 **CPU** 在线指令。Turbo PMAC 将响应部件编号（例如 56311）。在内部，全局状态位 **X:$000006** 21 位和 **Y:$000006** 21 位（全局状态查询字**???**的一部分）表示目前 CPU 是哪个类型。

活动内存

Turbo PMAC 使用静态 RAM（SRAM）芯片作为它的活动内存。这种类型的 RAM 比计算机存储器中大量的动态 RAM（DRAM）更快、更可靠。（SRAM 芯片用于计算机中的高速缓存存储器组。）

正如任何 RAM 芯片一样，当电源掉电后，这些 SRAM 芯片（如同 DSP 内部的存储器寄存器，和 ASIC 中的内存映射寄存器）中的内容会丢失。断电时应保留的设置必须首先使用 **SAVE** 指令将其复制到非易失性快闪存储器之中。

摩托罗拉的 DSP 采用了哈佛结构，该结构使用单独的内存组用于程序（编译的或汇编的机器代码指令）的存储和数据（其他一切，包括解释的程序指令）的存储。相比之下，在更常见的电脑使用的冯诺依曼体系结构之中，任何存储器均可用于程序或数据存储。

对于 Turbo PMAC（选项 5x0，其中“x”指定 CPU 的速度，如上所述）来说，除了 DSP 的内存之外，其标准内存配置还提供 128K 的 24 位字的程序内存和 128K 的 24 位字（可组织为 64K 的 48 位字）的数据内存。这是默认的配置。标准的内存配置下，总的可寻址内存是 DSP 内部的存储器与 SRAM 芯片中的外部存储器的总和，得到一个更多内存的 DSP 可以增加你的总存储器容量。

如果选择了扩展内存配置（选项 5x3），Turbo PMAC 一共有 512K 的 24 位字的程序存储器和 512K 的 24 位字（可组织为 256K 的 48 位字）的数据存储器。在扩展的内存配置下，总的可寻址内存被 DSP 的寻址空间限制，得到一个更多内存的 DSP 也不会增加总存储器容量（虽然它可以用更快的内存替代速度较慢的外部存储器）。

状态变量 I4908 包含了下一个寄存器的地址，超过了未保留的数据存储器的最后一个字。如果没有定义 UBUFFER，该地址比数据存储器的最后一个字大 1。标准的内存配置下，在重新初始化时不定义 UBUFFER。在扩展的内存配置下，一个 65,536 字的 UBUFFER--从 X/Y:$030000 到$03FFFF—将会自动定义在重新初始化时。

闪存

Turbo PMAC 的非易失性存储器是包含在一个闪存芯片中。闪存芯片会保留其内容，而无需外加电源。然而，他们在访问时间上相对比较慢，所以 Turbo PMAC 在正在进行的操作中不会使用闪存芯片。相反，在上电或复位时，它会将闪存中

的内容复制到快速 SRAM 之中，这样处理器就可以迅速访问它。只有遇到 SAVE 指令或当下载新的固件时，才可写入闪存。

闪存为泰道公司创建的固件和由用户创建的程序、变量值以及其他设置都提供了非易失性存储。闪存（IC）集成电路的大小取决于活动内存的容量，以便能够存储活动内存中的全部内容。状态变量 I4904 和 I4909 都包含了当前闪存大小的信息。

用 SAVE 指令将用户设置复制到闪存之中需要数秒的时间，在此期间，一些其他任务，其中包括一些安全检查，是不执行的；所以在机器实际工作中不应该做这件事。Turbo PMAC 在上电或复位时将闪存的存储数据复制到活动 SRAM 中，该过程中它会评估这些数据的校验和，并将其与在上一个 SAVE 操作过程中计算的校验和进行比较。如果有差异，它会将活动内存中的设置还原为出厂默认值，仿佛该卡已被重新初始化，并置位 Flash 读错误（又名 EAROM 错误）状态位在 X:$000006　21 位（全局状态的查询字???的一部分）。

可选电池-供电参数存储器

如果选择了选项 16A，Turbo CPU 部分会被提供一个 32k 的 24 位字（映射为 16K 的 48 位字）的数据-存储器 SRAM 附加组，该数据-存储器 SRAM 是电池供电（BBRAM）并能用于参数的存储。通常，该存储器组用于存储机器-状态的信息而无需一个冗长的 SAVE 指令。

低功耗 SRAM 可以用电池长时间供电，速度比用于主存储器的 SRAM 芯片要慢，所以用户应该知道，将该储存器组用于重要用途会产生一个明显的运算速度惩罚。P-变量和/或 Q-变量存储可以从快速主内存移动到电池-供电的存储器，通过改变 I46 的默认值 0。这提供了自动地保留变量值，代价是访问速度变慢。更常见的是，只用 M-变量作为指针，就可以访问 BBRAM 中的寄存器。BBRAM 的寄存器是从 **X/Y:$050000** 开始的。

作为给存储器供电的锂电池，虽然其预期平均电池寿命是五年或五年以上，但建议每年更换一次。有一个超级电容器，能够在无电池的情况下保持存储器中的内容几分钟，所以在控制器无电源供电时也可以更换电池。

Turbo PMAC 寄存器 X：$00003F 包含了 BBRAM 的末端寄存器的地址。如果没有 BBRAM 存在，它会报告一个值$050000。如果存在一个选项 16A BBRAM，它将报告一个值$054000。如果存在一个扩展 BBRAM，它将报告一个值$060000（Turbo PMAC 的设计支持这一点，但目前不出售扩展 BBRAM）。

机械接口 IC

Turbo PMAC CPU 通过几种类型的特殊 IC，与物理机器进行通讯，特殊 IC 有内存-映射的寄存器便于方便访问处理器、并有应用的特定电路用于机器的接口。其中最常见的是伺服集成电路、MACRO 集成电路、I/O 芯片以及专用集成电路（ASIC），由泰道公司设计、门阵列技术制造，以低耗高效的方式创建一个完整的功能集。

伺服集成电路

伺服集成电路包含所有的数字逻辑电路，以在 CPU 和运动（伺服或步进）通道之间提供接口。每个伺服 IC 提供到四个运动通道的接口。目前有两种类型的伺服集成电路，一种支持老版 PMAC 风格的接口（只有模拟放大器接口），另一种支持新版的 PMAC2 风格的接口（模拟或数字放大器接口）。

伺服通道：伺服通道在 Turbo PMAC 系统中是一个硬件结构，在伺服 IC 和周围系统中是一套接口和寄存器。一个通道包括接口和单个放大器、编码器、标志集的寄存器，。当这些通道被 Turbo PMAC 的电机、轴和坐标系（这是软件结构--见下文的关键软件组件 ）使用用于各种目的时，它们可以独立于任何那些结构而存在。

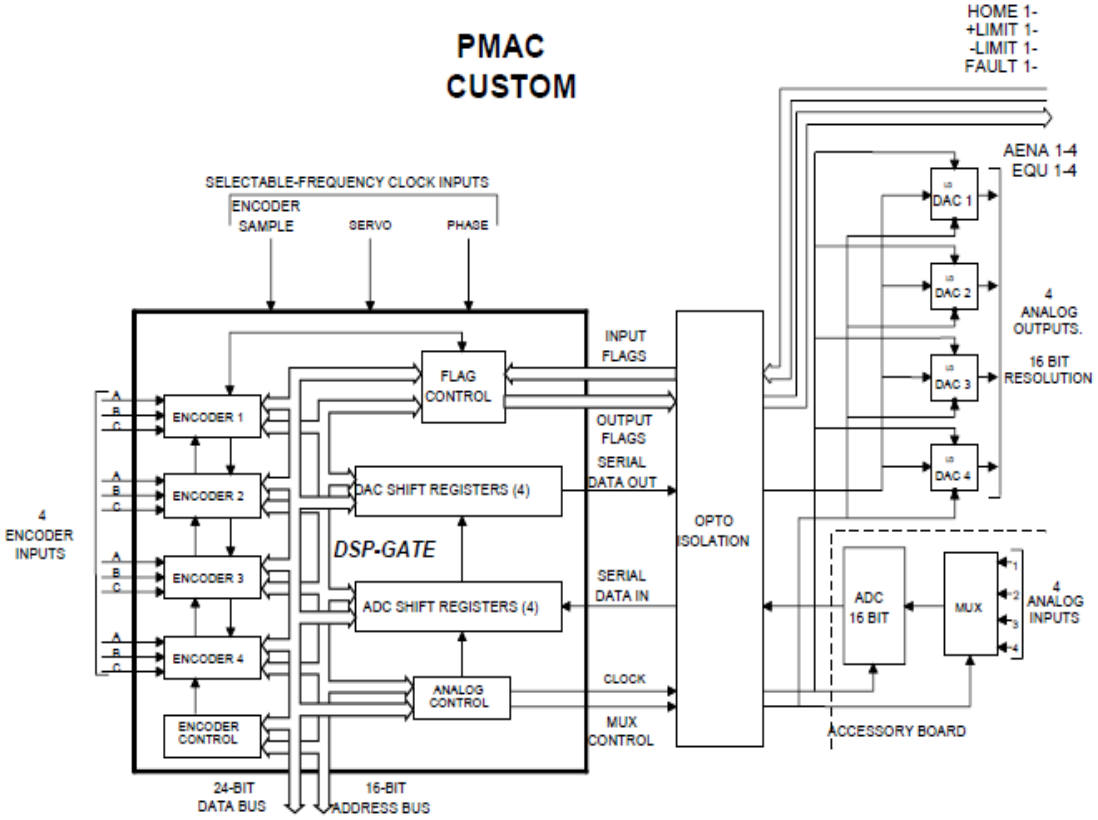
报告当前的伺服集成电路：Turbo PMAC 变量 I4900 报告当前有多少伺服集成电路以及他们在哪个地址。 I4901 报告每个伺服 IC 是哪种类型。下表显示了伺服集成电路的类型、它们的地址以及用于配置他们的 I-变量：

伺服 IC#	基地址	I 变量	伺服 IC#	基地址	I 变量
0	$078000	I7000 – I7049	5	$079300	I7500 – I7549
1	$078100	I7100 – I7149	6	$07A200	I7600 – I7649
2	$078200	I7200 – I7249	7	$07A300	I7700 – I7749
3	$078300	I7300 – I7349	8	$07B200	I7800 – I7849
4	$079200	I7400 – I7449	9	$07B300	I7900 – I7949

伺服集成电路 0 和 1 都是板载的，处于一个板级的 Turbo PMAC 上，或者在一个 3U 格式的栈中。伺服集成电路 2 到 9 是通过扩展端口访问的，或者是 UMAC 的 UBUS 背板扩展端口，或者是一个板级 Turbo PMAC 的 JEXP 电缆扩展端口，位于一个 Acc-24x 轴板上，或者一个 Acc-51x 插补器板上。

PMAC 式 DSPGATE 的 ASIC（专用集成电路）：该 PMAC 式伺服 IC 标记为 DSPGATE。这是一个 4-通道的配件，拥有 32 个内存映射寄存器的。每个通道都支持以下功能：

- 串行输出用于 16 位数-模转换器
- 输入用于数字正交或脉冲和方向反馈和索引
- 四个输入标志（回零，+/-极限，放大器故障），可以触发硬件编码器捕捉
- 放大器-使能输出
- 硬件位置-比较输出
- 来自 16 位模数转换器（从附件板）的输入



下面的 Turbo PMAC 产品提供 PMAC 式的 DSPGATE 集成电路：

- Turbo PMAC PC
- Turbo PMAC VME
- Turbo PMAC PCI
- Turbo PMAC PCI Lite
- ACC-24P ISA-格式的轴-扩展板
- ACC-51P ISA-格式的插补板

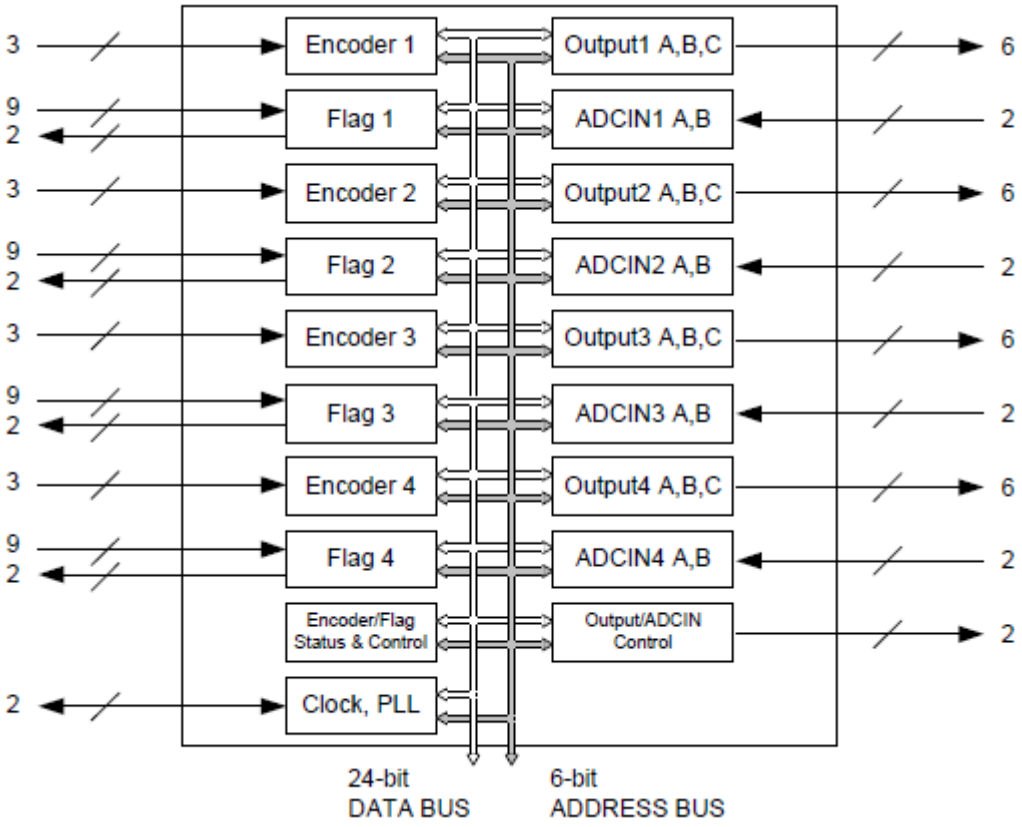
PMAC2 式 DSPGATE 的 ASIC(专用集成电路)：PMAC2 式伺服 IC 被标记为 DSPGATE1。这是一个 4-通道的配件，拥有 64 个内存映射寄存器。每个通道都支持以下功能：

- 三个输出指令信号集，可配置为下面二种之一：
  - 两个串行的数据流到高达 18 位的数-模转换器，和一个脉冲加方向对
  - 三个脉宽调制（PWM）的顶部和底部对
- 输入用于数字正交与索引，脉冲和方向，或 MLDT 反馈
- 四个输入标志（回零，+/-极限，用户），可以触发硬件编码器捕捉
- 放大器故障输入
- 四个辅助输入标志（T，U，V，W）用于霍尔换相传感器，分计数数据，故障代码， 或一般使用
- 放大器使能输出
  - 硬件位置-比较输出
  - 来自两个高达 18 位的模-数转换器（从放大器或附件板）的输入

该 DSPGATE1 集成电路还拥有板载的可配置软件的时钟发生电路。它可以为

整个的 Turbo PMAC 系统生成伺服和相位时钟（只有一个 IC 这么做，其他 IC 会接受这些作为输入）。它还会产生时钟信号用来驱动其自身的电路：DAC，ADC，PWM 和 PFM（脉冲频率调制）。

PMAC2 门阵列集成电路
“DSPGATE1”



下面的 Turbo PMAC 产品提供 PMAC2 式 DSPGATE1 集成电路：

- Turbo PMAC2 PC
- Turbo PMAC2 VME
- Turbo PMAC2 PCI
- Turbo PMAC2 PCI Lite
- ACC-24P2 ISA-格式的轴扩展板
- ACC-24E2 UMAC PWM 轴板
- ACC-24E2A UMAC 模拟轴板
- ACC-24E2S UMAC 编码器/步进板
- ACC-51E UMAC 插补板
- ACC-24C2 UMAC-CPCI PWM 轴板
- ACC-24C2A UMAC CPCI 模拟轴板
- ACC-51C UMAC CPCI 插补板
- ACC-2E3U-格式的栈轴板

MACRO ICs（集成电路）

Turbo PMAC 有两种类型的 ASIC（专用集成电路），可以支持在 MACRO 环上的通讯，该环是泰道公司开发的开放式结构、高速实时通信环，用来简化分布式控

制硬件的连接。

Turbo PMAC 变量 I4902 报告当前有多少 MACRO 集成电路存在以及位于哪个地址。I4903 报告每个 MACRO 集成电路是什么类型的，是 DSPGATE2 还是 MACROGATE（见下文）。变量 I20-I23 指定四个 MACRO 集成电路的地址，其中这四个电路自动用 I-变量配置。下表总结了这种配置：

MACRO IC#	I 变量	通常基地址	通常类型
0	I6800 – I6849	$078400	DSPGATE2
1	I6850 – I6899	$079400	MACROGATE
2	I6900 – I6949	$07A400	MACROGATE
3	I6950 – I6999	$07B400	MACROGATE

DSPGATE2 MACRO IC

该 DSPGATE2 IC 同时提供了 MACRO 环功能和通用 I/O 功能。I/O 功能将在下一节中介绍。对于 MACRO 环，该 DSPGATE2 提供了一个 16 节点的双向接口。这些节点中，其中的 8 个可被用作伺服节点，其中每一个节点都可以传送所有的用于电动机的伺服和换向的指令和反馈数据。6 个节点可用于通用 I/O，每个节点都支持每个方向上的 72 位硬实时 I/O;2 个节点是用于非实时通信，包括广播模式，其中的一个主控制器可以同时跟其所有的从属设备通讯。

Turbo PMAC2 UltraLite 控制器标准配置了一个 DSPGATE2 IC，用于 16 节点的 MACRO 接口。UMAC Acc-5E 也标配了一个 DSPGATE2 IC。

MACROGATE MACRO IC

和 DSPGATE2 IC 一样，该 MACROGATE IC 提供了相同的 16-节点的 MACRO-环接口，但它不具有 DSPGATE2 的 I/O 功能。，位于 Turbo PMAC2 UltraLite 的随后的可选 MACRO IC（选项 1U1、1U2、1U3）是 MACROGATE IC。

I/O IC

Turbo PMAC 的 CPU 通过多种 I/O 接口芯片访问通用数字 I/O。在 Turbo PMAC 板上，这些等同于现代化的经典的 8255 IC 设计，8255 IC 设计最初由英特尔提出。Turbo PMAC 的 CPU 寻址这些 IC 的地址在：Y:$078800 – Y:$078803

在 Turbo PMAC2 板和 UMAC I/O 板上，这些都是泰道公司设计的 I/O 的 ASIC，如下文所描述。

DSPGATE2 I/O IC

该 DSPGATE2 IC 同时提供 MACRO 环功能和通用 I/O 功能。MACRO 功能在上一节中已经描述过。每个板级的 Turbo PMAC2 板和 UMAC 的 Acc-5E 板使用 DSPGATE2 来支持非伺服 I/O 端口。这些端口是：

- JTHW 多路复用器端口
- JIO 通用 I/ O 端口
- JDISP 显示端口
- JHW 手轮端口

一个板级的 Turbo PMAC2 上的 DSPGATE2 IC 被寻址在：Y:$078400 – Y:$07843F。在默认配置下的地址拨码开关与 UMAC Acc-5E 板上，DSPGATE2 IC 被寻址在该相同的位置。

IOGATE I/O IC

在大多数 UMAC 的 I/O 板上，IOGATE IC 被用于访问通用数字 I/O 口。它提

供 48 个 I/O 点，寻址为 6 个字节在连续的寄存器里。不同的电路板使用 IOGATE 周围不同的缓冲区和驱动以提供所需的特定的 I/O 功能。而在 IOGATE 本身，每个 I/O 点都是单独选择方向，在大多数 I/O 板上，每个点的方向是由那点的外部电路固定的。IOGATE 必须在上电/复位时设置以支持它使用的板的特定方向配置。

潜在地，一个 UMAC 系统中最多可以安装 16 个配备 IOGATE 或类似芯片的板。下表显示了这些电路板的可能的地址

拨码开关	基地址	拨码开关	基地址	拨码开关	基地址	拨码开关	基地址
0	$078C00	4	$079C00	8	$07AC00	12	$07BC00
1	$078D00	5	$079D00	9	$07AD00	13	$07BD00
2	$078E00	6	$079E00	10	$07AE00	14	$07BE00
3	$078F00	7	$079F00	11	$07AF00	15	$07BF00

通信端口

Turbo PMAC 控制器提供了多个通信端口。这些端口可以同时使用，允许主端口用于实际应用中同时第二端口用于调试，或者第二端口用于一个操作员拉线开关。

总线端口

PMAC 的每个配置都支持一个总线端口用于快速并行通信。这些利用 DSP 的主机端口和可选的双端口 RAM 用于通过共享内存的通信。总线可以是传统的背板总线或者是较新的线型总线。

目前支持的背板总线是：

- ISA（或 PC/104 堆栈版本）
- PCI
- VME

目前支持的线型总线是：

- USB（1.1 或 2.0）
- 以太网（TCP/ IP 或 UDP/ IP 协议），10 Mbps

串行端口

Turbo PMAC 控制器还支持串口通信，通常是一个串行端口，可选的还可有第二个串行端口。

主串行端口

Turbo PMAC 控制器始终配备一个标准的串行通信端口。在大多数配置中，该端口使用 RS-232 信号格式或 RS-422 信号格式，可通过跳线选择。尽管两个信号格式都有单独的连接口，但这是一个单端口，所以一次只能使用一个连接口。

可选的辅助串行端口

如果选择了 Option 9T，则会提供一个第二个串行端口，通常为 RS-232 级。此端口可用于一般通信。可以将变量 I43 设置为 1 时，为该端口关闭命令解析器，这样 Turbo PMAC 就不会试图将收到的字符解译为命令，从而使用户可以如其所愿解析传入的数据。如果使用 IEC-1131 PMAC 梯形图程序，将需要对该端口进行设置和诊断。

2-6 关键的软件组件

在任何应用程序中，Turbo PMAC 都将同时拥有泰道公司及用户提供的软件组件。泰道的软件组件也就是固件（嵌入硬件的软件），这些不是应用程序特定的。用户软件组件--主要是运动程序和 PLC 程序--才是应用程序特有的。

操作固件

Turbo PMAC 的操作固件同时作为控制器的操作系统和 PMAC-语言解释器。作为一个操作系统时，它提供了一个硬实时、多任务环境，可以极快地切换任务，为诸如伺服环闭合的高优先级任务，提供极高水平的确定性。它还可以自动处理运动程序内的工作序列，因此逐步的工作序列不必由用户处理。

通常情况下，操作固件需要一年更新 1 到 2 次来增加新的功能或解决故障问题。固件进行现场升级很容易。只需上电 Turbo PMAC，且安装有固件重载跳线，然后启动 PC 上的 PMAC 执行程序。执行程序会自动发现 Turbo PMAC 准备好安装新固件以及查询要下载的文件。它会从指定的文件中自动下载新固件到 Turbo PMAC。

引导固件

引导固件的运行对几乎所有的 Turbo PMAC 用户是不可见的。它的功能非常类似于一个 PC 机的 BIOS，在硬件和操作系统之间提供一个链接，并允许操作系统加载。由于泰道公司会不时地升级引导固件（而且 CPU 的每一个新的硬件配置会需要不同的引导固件），所以泰道公司不支持引导固件的现场升级。

在正常的操作中，引导固件隐藏在操作固件下运行，每次上电或复位会自动将操作固件加载在它的上面。但是，如果Turbo PMAC在固件重载跳线为ON时通电或复位，操作固件将不从闪存中加载，而且主机可以直接与引导固件通讯（只通过主串行端口或使用CPU主机端口的总线）。如果操作固件尚未被加载，Turbo PMAC对？、？？或状态查询指令？？？的响应是BOOTSTRAP PROM。

电机、轴和坐标系

电机、轴和坐标系在一个 Turbo PMAC 系统中是软件结构。虽然他们通常利用硬件通道接口到外部世界，但也可以用它们作为虚拟结构，只为其计算功能。

电机

Turbo PMAC 的一个电动机包括必要的软件结构用于基本动作、指令-位置插补、伺服环闭合、换向和电流环闭合。每个 Turbo PMAC 都有能力执行 32 个电机的算法，即使是只提供了几个伺服通道。这些电机由数字（1～32）指定，它们的属性由原始“计数”、毫秒以及相关的单位来指定。

一个 Turbo PMAC 电动机通过设置 I-变量（I-变量包含了要使用的寄存器的地址）来访问硬件通道功能，是为了其各种输入和输出的需求。这种映射功能以一种简单的方式提供了极大的功能和灵活性。

轴

通常，一个 Turbo PMAC 系统中的运动是通过使用轴来指令的。Turbo PMAC 的一个轴包括了编程运动的软件结构。轴是由字母（A，B，C，U，V，W，X，Y 和 Z）指定，而他们的属性是按照用户指定的单位（例如毫米、英寸、度、秒、分钟）指定的。

轴是通过定义语句或运动子程序分配给电机的。虽然轴和电动机之间通常是 1 对 1 映射（例如电动机 1 分配到 X 轴，电机 2 分配到 Y 轴），但这并不必须是这种情况。多个电机可以被分配到一个轴，如一个龙门台架的配置；也可以不分配电机给一个轴，从而创建一个虚拟轴；或者在运动算法中可以有多个电机分配给多个轴的复杂关系。

轴线属于一个坐标系（见下文）。一个坐标系最多可以使用 9 个轴，一个字母表示一个轴。

坐标系

坐标系是 Turbo PMAC 的结构用以实现多台电机的紧密协调运动。Turbo PMAC 最多支持 16 个独立的坐标系统。在一个特定坐标系统中一个轴分配一个电动机。同一坐标系下，同一行运动程序指定的多个轴会自动地协调。

要使多台电机协调运动，将它们分配给同一坐标系中的轴。要使电机独立运动（即相对于彼此，在任意时间开始、停止或改变速度），将电机分配给不同坐标系的轴。

用户程序

Turbo PMAC 用户可以将多种类型的程序安装到控制器中，其中每种类型有一个特定的用途。

运动程序

对于同步于编程式运动的协调运动和执行任何计算来说，Turbo PMAC 的运动程序为二者提供了一种简单的方法来指定其顺序。运动程序语言结合了 RS-274 标准 G 代码机床编程语言和 BASIC 两者的功能，其中，RS-274 标准 G 代码机床编程语言可以很好地用于指定协调运动的序列，BASIC 可以很好地用于相关的数学和逻辑。

PLC 程序

Turbo PMAC 中的 PLC 程序的目的是用于那些异步于编程运动的动作和计算。PLC 程序以定期可编程逻辑控制器的方式反复扫描。它们可如专用 PLC 那样用于 I/O 控制，但是由于他们可以直接访问 Turbo PMAC 所有的寄存器，所以它们还有许多其他的用途。

Turbo PMAC 程序既可以用类似 BASIC 的文本语言编写，也可以用 IEC-1131 梯形逻辑或顺序功能图（需要扩展内存选项和特殊 PC 软件）来编写。文本 PLC 程序可以作为解译的代码运行，也可以作为编译代码运行以提高效率。更多细节参考 PLC 程序用户手册中的章节。

用户编写的伺服和相位算法

几乎所有用户都将能够利用 Turbo PMAC 内置的伺服环闭合和换相算法。然而，用户可以安装自定义算法（二者之一或者二者）来完成标准算法不能完成的任务。有些用户使用这些算法只是为了获得高速、高优先级的 I/O 或计算，通过

在其他未使用的电机上激活他们。

用户自编写的伺服算法可以用高级 PMAC 语言（使用方便）或 DSP56300 汇编语言（最大效率）来编写。用户编写的相位算法必须用汇编语言编写。更多详细信息见用户手册中关于换相和伺服环的章节。

2-7 Turbo PMAC 配置

Turbo PMAC 的控制器可以有各种不同的物理配置。从根本上说，这些配置可以分成三个不同的类型：板级、机架安装的和装箱的。每种类型的描述如下。

板级设计

Turbo PMAC 有多种的板级实现。这些设计可以被用作背板总线上的扩展卡，但它们不一定要安装在总线上，无论是初始设置，还是在实际的应用中。所有这些设计也有串行通信链接。

目前，Turbo PMAC 的设计有用于 ISA、VME 和 PCI 总线。这些设计配有 PMAC 风格或 PMAC2 风格的接口。Turbo PMAC 精简版的版本只有一个板载的伺服 IC，提供四个通道的伺服接口电路。UltraLite 版本的 Turbo PMAC2 用 MACRO-环接口取代了板载伺服接口电路；实际的伺服接口电路位于一个远程 MACRO 站中或嵌入到一个 MACRO 驱动中。

下表总结了截至本文写作时（2003 年中期）可用的板级 Turbo PMAC 实施：

产品	总线类型	最大#板载伺服 IC（通道）	板载 MACRO?
Turbo PMAC PC	ISA	2 (8)	否
Turbo PMAC VME	VME	2 (8)	否
Turbo PMAC PCI	PCI	2 (8)	否
Turbo PMAC PCI Lite	PCI	1 (4)	否
Turbo PMAC2 PC	ISA	2 (8)	否
Turbo PMAC2 PC Ultralite	ISA	0 (0)	是
Turbo PMAC2 VME	VME	2 (8)	否
Turbo PMAC2 VME Ultralite	VME	0 (0)	是
Turbo PMAC2 PCI	PCI	2 (8)	否
Turbo PMAC2 PCI Lite	PCI	1 (4)	否
Turbo PMAC2 PCI Ultralite	PCI	0 (0)	是

UMAC 机架式安装设计

Turbo PMAC 的机架式安装的实现被称为 UMAC（通用运动与自动化控制）。这些实现由一组模块化的 3U-格式（100 毫米 x160 毫米）欧洲卡组成，欧洲卡安装在欧洲机架上并装在一个共同的背板上。模块化结构允许控制器易于定制以用于用户的精密需求。机架安装易于安装和连接在工业机器上，而且布线到安装-电脑的控制器中不存在困难。高速电线通信链接，比如 USB 和以太网提供的通信速

度接近那些背板总线的速度。

UMAC Turbo

UMAC Turbo 是一个机架式的 Turbo PMAC 系统，其中的 3U 格式板跨接在背板上，背板上有一个物理格式的 VME 总线（96-针 DIN 连接口），但与 VME 总线没有电子或软件兼容。现场配线从卡片的顶部、底部、（有时候）前侧出来。

有各种各样的 UMAC 板，包括：

- Turbo CPU 板，有 PC/104 接口
- 轴板：PWM、模拟量、步进/编码器
- 传感器板：正弦编码器、SSI、绝对式编码器、旋转变压器
- 数字 I/O 板：TTL 和隔离的 24V、下沉式汇和吸入式源
- A/D 转换板：12 位和 16 位
- 高速通信板：USB 或以太网
- 现场总线接口：DeviceNet 和 Profibus，主站或从站
- 背板为 4 至 18 个板外加电源
- AC-输入（85-240VAC）和 DC-输入（24VDC）电源

Compact（紧凑型） UMAC Turbo

紧凑型 UMAC Turbo（以前称为 UMAC-CPCI Turbo）是一种安装在机架上的 Turbo PMAC 系统，其中 3U 格式的板跨接到背板上，背板上有物理格式的紧凑型 PCI（CPCI）总线（110 针高密度连接口），但和 CPCI 总线不电子或软件兼容。现场配线从背板后面的直通连接口出来走线。用户负责制定现场布线的分配方案。出于这个原因，紧凑型 UMAC 适用于大批量的用户，他们能够在一个应用程序的特定分配方案上负担得起投资。

有一个小型系列的紧凑型 UMAC 板。这个系列目前包括：

- Turbo CPU 板，有可选 USB 或以太网接口
- Acc-24C2 数字量（PWM/步进）4 轴接口板
- Acc-24C2A 模拟量 4 轴接口板
- Acc-11C48/96 隔离的数字 I/ O 板
- Acc-51C 正弦编码器插补板
- Acc-C8 8-槽背板（加上电源插槽）

QMAC 箱体式设计

QMAC 是一个箱体式 4 通道的 Turbo PMAC2,配有内置电源和前面板连接器板。它可用于脉冲和方向、模拟量、或直接-PWM 输出至放大器。

QMAC 具有以下标准功能：

- 4 通道基本轴-接口电路，每一个包括：
  - 正交编码器输入，有索引和霍尔换相传感器
  - 差分脉冲-方向步进输出
  - 四个隔离的输入标志（回零、+/-限制、用户）
  - 隔离的位置-比较输出标志
  - DB-15 连接口上的编码器/步进接口
  - 可拆卸接线端子上的标志接口
- DB-25 连接口上的两个补充通道，每个包括：

- 正交编码器输入
- 脉冲输出，可配置为 PWM 或 PFM

- 通用隔离的数字 I/O：8 个输出，16 个输入，位于可移动接线端子上
- RS-232 通讯口
- 多路复用-端口接口用于 I/O 附件
- 显示端口接口

QMAC 有以下可选功能：

- 四个通道的单路或双路模拟放大器接口
- 四个通道的直接-PWM 放大器接口
- USB 通讯接口
- 以太网通信接口

第三章 TURBO PMAC/PMAC2 系统配置和自动配置

Turbo PMAC，尤其是 Turbo PMAC2，其主板拥有丰富的功能，可以自动识别和自动配置系统。这对 UMAC Turbo 系统显得尤为重要，其具有各种各样的配置。这些功能为开始一个特定配置提供了易于使用和灵活性。

3-1 CPU 时钟频率

在任何的 Turbo PMAC 系统中，CPU 实际运行的时钟频率是由软件变量 I52 设定的，表示为系统的固定时钟晶体频率的乘法因子。

Turbo PMAC 的时钟晶体频率是 19.66MHz（有时也取 20MHz 作为近似值）。CPU 将它除半，得到 9.83 兆赫，然后将得到的频率乘以在一个称为锁相环（PLL）的电路中的一个整数，最后获得它的工作频率。CPU 工作频率的确切公式为：

CPU 频率（MHz）=9.83*（I52+1）

在近似计算中，公式为：

CPU 频率（MHz）= 10*（I52+1）

注意，CPU 的工作频率不一定等于 CPU 的最大额定频率。根据 CPU 选项的不同，则其最大额定频率也不同。下表显示了每个 CPU 选项下的最大额定频率，并显示了不超过这些频率时 I52 变量可使用的最大值。

CPU 选项	CPU 最大额定频率	I52 最大额定值	I52 最大值下的工作频率
5Cx	80MHz	7	78.64MHz
5Dx	100MHz	9	98.30MHz
5Ex	160MHz	15	157.28MHz
5Fx	240MHz	23	235.92MHz

I52 的出厂默认值是 7，因此产生 80MHz 的 CPU 频率，无论当前 CPU 选项是哪一个。I52 仅能在上电或复位时使用，所以要改变 CPU 频率，改变 I52 的活动值，然后用 SAVE 命令将该新的数值保存到非易失性闪存中，并将卡重置（通常是用**$$$**指令）。在上电或复位周期里，Turbo PMAC 将读取 I52 的保存值，设置 PLL 电路来产生合适的工作频率，并进行其他设置（例如波特率分频器、内存和 I/O 等待状态）以适应该频率。

通常情况下，设置 I52 以让 CPU 以最大额定频率运行。然而，也有一些情况，其中较低的频率会比较理想。如果需要使用主要或辅助串行端口以 115,200 波特率进行串口通信，那么 CPU 的频率必须是 30MHz（29.49MHz）的整数倍以产生足够准确的波特率。这样一来，选项 5Dx 100 MHz 的 CPU 需要工作在 90 MHz 才能做到这一点。

注意：

让 CPU 工作在比其额定频率更高的频率上是可以的，特别是在适度的环境温度下。但是，这些情况下，不能保证它安全工作，而且任何这样的操作完全是用户冒着自己的风险进行的。

系统时钟频率，例如相位和伺服时钟以及时钟频率驱动硬件电路，是由时钟晶体频率直接通过伺服 IC 或 MACRO IC 产生的，并且不依赖于 CPU 的工作频率。这些时钟频率被包括在下面的章节。

3-2 Turbo PMAC2 的系统时钟源

在一个 Turbo PMAC2 系统中，系统相位和伺服时钟（会中断处理器并为伺服系统锁存关键的输入和输出数据），来自于系统的一个（并且仅仅一个）伺服集成电路或 MACRO 集成电路中，或者也可能来自一个外部源。对于整个 Turbo PMAC2 系统，相位和伺服时钟必须有一个唯一的源。本节说明如何指定该时钟源。本节后面的部分，设置系统时钟频率，介绍了一旦时钟源确定后如何设置频率。

默认时钟源

这些时钟信号出厂默认时的源对几乎所有的应用程序都是适当的。只有在特殊情况下，使用其他源。Turbo 安装程序将引导您完成对选定源的频率设置。设置频率将在下一节讨论。

注意：

一个 Turbo PMAC 板采用固定的、离散逻辑来生成其相位和伺服时钟。如果附件板的伺服集成电路或 MACRO 芯片可以产生自己的时钟信号，并将附件板添加到 Turbo PMAC 中，它们必须设置成使用 Turbo PMAC 的相位和伺服时钟信号。Turbo PMAC 的系统也支持外部时钟源，其方式和 Turbo PMAC2 系统的方式是相同的。

IC 时钟发生器设备

每个 PMAC2 型的伺服集成电路（DSPGATE1 IC）和 DSPGATE2 型 MACRO 集成电路都有能力产生自己的相位和伺服时钟信号，或者接受外部的相位和伺服时钟信号（例外：MACROGATE 型 MACRO 集成电路可以产生他们自己的相位时钟，但不能

产生伺服时钟。因此，它们不能用于为整个系统产生时钟）。一个系统中最多有一个集成电路可以产生自己的时钟信号---如果信号来自外部源则一个也没有。

变量 I7m07 和 I7m57 分别控制伺服集成电路‘m’和‘m*’的时钟方向。变量 I6807、I6857、I6907 和 I6957 分别为 MACRO 集成电路 0、1、2 和 3 控制时钟的方向。如果变量值是 0，IC 会生成其自己的时钟信号并输出它们。如果变量值是 3，IC 会接受外部源给它的时钟信号。这些集成电路中最多有一个可以将变量值设为 0，其余的必须被设置为 3。

注意：

如果这些集成电路中有一个以上被设置为使用它自己的时钟信号并输出它们，处理器将被多个源中断而且不能正常工作---看门狗定时器有可能跳闸。（因为输出是集电极开路型，所以届时信号冲突不会造成硬件损坏，但系统软件的运行将大打折扣。）

典型的时钟源集成电路

在一个板级的 Turbo PMAC2 控制器上，且不是一个 UltraLite MACRO-only 控制器，通常系统时钟源是伺服 IC 0，即第一个板-载的伺服 IC。这意味着 I7007 被设置为 0，因此 I7000、I7001 和 I7002 控制系统时钟的频率。其他时钟方向 I-变量应该被设置为 3。对于 3U Turbo Stack 控制器也是如此。

在一个 Turbo PMAC2 UltraLite 控制器，或者一个使用 MACRO 环的 UMAC Turbo 系统中，通常情况下系统时钟源是 MACRO IC0，即第一个 MACRO IC。这意味着 I6807 被设置为 0，因此 I6800、I6801 和 I6802 控制系统时钟的频率。其他时钟方向 I-变量应该被设置为 3。

在一个不使用 MACRO 环的 UMAC Turbo 系统（大多数是 UMAC Turbo 系统）中，通常情况下系统时钟源是伺服 IC2，即 UBUS 背板上的第一个伺服 IC。这意味着 I7207 被设置为 0，因此 I7200、I7201 和 I7202 控制系统时钟的频率。其他时钟方向 I-变量应该被设置为 3。

外部时钟源

可以设置一个 Turbo PMAC 或 Turbo PMAC2 系统，使得该系统使用从主串行端口的额外引脚上输入的外部生成的相位与伺服时钟信号。有时候，这些时钟信号来自同一系统下其他的 Turbo PMAC，用来保持所有的控制器完全同步。有时候，他们可能来自一个视频系统，用来保证运动和视频帧完全同步。还有的时候，他们可能来自一个比 Turbo PMAC 自身时钟晶体更精确的参考时钟，这在望远系统的跟踪控制中很常见。

在一个 Turbo PMAC 系统中，如果任何 E40- E43 的跳线被删除，它会期待外部生成的相位和伺服时钟。在一个 Turbo PMAC2 系统中，这个功能是由板级 Turbo

PMAC 上的跳线 E1 和 UMAC 系统的 CPU 板上的跳线 E1A 和 E1B 决定的。

建议供给 Turbo PMAC 的每个伺服和相位时钟信号是差分的 5V 对（RS-422 级）。但是，单端 5V 信号可以被接受进入到“+”输入端；“-”的输入端应悬空并保持在内部电压 2.5V。相位时钟信号应具有 50%的占空比。伺服时钟必须和相位时钟同步，一个周期有 n 个相位时钟周期，其中 n 是一个正整数。SERVO+信号的下降沿必须和 PHASE+信号的下降沿一致，并且 SERVO+信号应为低电平最多 1 个相位时钟周期。

关于使用外部时钟信号的详细信息，请参阅本手册《同步 Turbo PMAC 到外部事件》的章节。

时钟信号的分配

不管相位和伺服时钟信号的源是谁，这些信号都必须是可用的对于处理器和所有伺服集成电路和 MACRO 集成电路以及任何其它使用这些信号工作的电路（例如 I/ O 卡，用于并行或串行反馈）。请注意，硬件时钟信号---DAC 时钟、ADC 时钟、编码器采样时钟和 PFM 时钟---是在各自的伺服集成电路和 MACRO 集成电路内部本地产生的，而且在集成电路之间不共享。

板级的 Turbo PMAC

在一个板级 Turbo PMAC 上，时钟源必须是板载伺服集成电路或 MACRO 集成电路其中的一个，并且时钟信号只在电路板上的电路迹线上传输，如果有一款搭载 CPU 板，让它连接到 CPU 板。伺服和相位时钟信号始终作为 JEXP 扩展端口的输出（永不输入）用于连接到这个端口的附件板。这些信号也可以作为缓冲的差分输出，在 RS-422 串行端口连接口的额外引脚上用于其他设备。

UMAC 系统

在 UMAC 系统（包括紧凑型 UMAC）中，相位和伺服时钟是共享的，跨过 UBUS 背板，共享在插入到该背板的不同 3U 格式卡之间。每张卡都有缓冲 IC 用于这些信号，在它们接口到背板时。在那些作为相位和伺服时钟信号潜在源的卡片上，如 Acc-24E/C 轴板或 Acc-5E MACRO 板，这些缓冲区可配置为输入或输出。

在每一个可作为时钟源的 UMAC 板上，都有一个跳线控制时钟方向缓冲区的配置。在一个设置，主板只能输入时钟信号。此设置是 UMAC MACRO 需要的，其中的时钟信号始终来自 MACRO 接口板。对于 UMAC Turbo 系统中不产生他们自己的相位和伺服时钟信号的电路板来说，这是允许的，但不推荐。对于生成系统相位和伺服时钟的 UMAC 板来说，此设置是不允许的。

在另一个设置中，时钟信号缓冲器的方向可以由 Turbo CPU 颠倒。此设置对于产生系统时钟的电路板是必需的，也推荐用于其它电路板（这样源就可以改变而无需移动任何缓冲区）。在上电或复位时，对于含有伺服 IC 或 MACRO IC 的电路板，其中由 I19 指定产生系统时钟，CPU 将配置缓冲区作为输出到 UBUS 背板中；它会配置所有其它板卡的缓冲区为来自 UBUS 背板上的输入。

丢失时钟信号

如果相位和/或伺服时钟信号不存在，这是一个严重的故障，并且 Turbo PMAC 系统将无法正常运作。本节介绍 Turbo PMAC 系统如何响应此类故障。

板级 Turbo PMAC

在板级 Turbo PMAC 上，如果 CPU 在任何时间没有收到相位和伺服时钟信号，看门狗定时器将立即跳闸并完全关闭系统。如果这种情况在上电或复位时由于时钟源设置的配置不当而即刻发生，那么安装重新初始化-跳线并再次为系统上电去恢复一个有效的时钟源，来自板载伺服 IC 或 MACRO IC 其中之一。

UMAC 系统

在一个 UMAC 系统（包括紧凑型 UMAC）中，如果在配置时钟源后，CPU 没有通过 UBUS 背板立即收到任何的相位或伺服时钟信号，它会进入一个特殊的模式，该模式下它分别以 9.03 kHz 和 2.26 kHz 的出厂默认频率产生自己的相位和伺服时钟。在这种情况下，它将未发现硬件时钟的全局状态位 X:$000006 3 位设置为 1。

这种模式是为了保持系统活动，允许用户为其设置来设置正确的时钟配置信息。它不适用于 UMAC 以这种模式实际控制。在固件版本 V1.940 或更新版本中，如果该错误位置位，那么将不能使动电机，而且如果 I6 设置为 1 或 3，Turbo PMAC 会报告一个 ERR018 给一个使动指令。

如果 UMAC CPU 自此之后停止接收相位或伺服时钟信号，看门狗定时器将跳闸并立即完全关闭系统。

重新初始化动作

在重新初始化一个 Turbo PMAC2 时（**$$$*****指令，或通过重新初始化跳线 E3 为 ON 时上电/复位），CPU 将搜索伺服 IC 和 MACRO IC 所有可能的位置，看哪个存在。如果系统没有设置为一个外部时钟源，它会决定用这些芯片中的哪个来生成系统的相位和伺服时钟，它将使用在下列表中找到的第一个 IC：

1.	Servo IC 0	(On-board or 3U Stack)	(I19=7007)
2.	MACRO IC 0	(On-board or Acc-5E)	(I19=6807)
3.	Servo IC 1	(On-board or 3U Stack)	(I19=7107)
4.	Servo IC 2	(Acc-24E2, 51E)	(I19=7207)
…			
11.	Servo IC 9	(Acc-24E2, 51E)	(I19=7907)
12.	Servo IC 2*	(Acc-24E2, 51E)	(I19=7257)
…			
19.	Servo IC 9*	(Acc-24E2, 51E)	(I19=7957)
20.	MACRO IC 1	(On-board or Acc-5E)	(I19=6857)
21.	MACRO IC 2	(On-board or Acc-5E)	(I19=6907)
22.	MACRO IC 3	(On-board or Acc-5E)	(I19=6957)

（MACRO 集成电路必须是 DSPGATE2 集成电路以用作一个时钟源。）

接下来，它会自动将 I19 设置为它找到的第一个 IC 的时钟-方向 I-变量的数字。例如，如果它找到了 Servo IC 0，它将 I19 设置成 7007。最后，将该时钟方向 I 变量设置为 0，并将它找到的所有其他伺服和 MACRO 集成电路的时钟方向 I-变量设置为 3。

然而，如果重新初始化时，它发现在 E1 跳线被配置为使用外部相和伺服时钟，那么 I19 会被设置为 0，所有的时钟方向 I-变量被设置为 3。

注意：

一旦系统设置为利用外部相位和伺服时钟信号，系统通电时这些信号必须始终存在，否则看门狗定时器将立即跳闸。

要更改一个系统的设置从外部时钟回到内部时钟，需要将它上电，而且：E3 重新初始跳线为 ON、E1 外部时钟跳线为 OFF、外部时钟信号存在。一旦内部时钟源的新配置建立，这些设置必须用 SAVE 指令存储到闪存中，要么通过自动重新初始化操作，或者通过用户设置的配置（见下文）。然后，系统可以断电，E3 重新-初始化跳线清除，外部时钟信号清除。最后，该系统就可再次正常通电了。

注意：

如果 Turbo PMAC2 处于一个 MACRO 环上，但它不是环控制器同步主站，那么需要设置成通过接收 MACRO 环的同步数据包来调整其相位时钟。具体设置是：将 I6840 的第 7 位设置为 1、将 I6841 的 16-19 位设置为同步数据包的节点号（通常是$F [=15]）。在这种情况下，尽管他们被锁定为接收该同步数据包，但相位和伺服时钟仍然是内部生成。这种类型的系统应将 I6807 设置为 0（I19 设为 6807）以使用 MACRO IC0 作为相位和伺服时钟的源。如果重新-初始化没有自动引起这种情况发生，就必须手动完成（见下文）。

用户定制的时钟源规格

对于 3U-格式的 Turbo PMAC2 系统和其他的 Turbo PMAC2 系统，改变时钟源的流程略有不同。

在一个 3U-格式的 Turbo PMAC2 系统（UMAC Turbo 或 3U Turbo Stack）中，要改变时钟源，只要简单地遵循这 3 个步骤：

1、将 I19 设置成（你想要作为源的）IC 的时钟-源 I-变量的数字。
2、用 SAVE 指令将这个值存储到非易失性闪存中。
3、正常复位系统（不是重新初始化）。

不要试图直接设置时钟-方向 I-变量。

在其他的 Turbo PMAC2 系统中，用单个指令（如 I6807=0 I7007=3）改变时钟-方向 I-变量本身。最好是将 I19 改变成设为 0 的 I-变量的数字（在这个例子中，I19=6807），但这不是必需的。使用 SAVE 指令将这些新值存储到非易失性闪存中。然后，在后续的每一个上电/复位时，他们将被自动使用。

在 UMAC Turbo 系统中，伺服和相位时钟线是双向的位于 UBUS 背板扩展端口上，所以这些信号可以来自或发往 CPU 板。然而，在 JEXP 扁平电缆扩展端口上，这些时钟线是单向的，所以只能从主 PMAC 板或 CPU 板输出。

如果 I19 设置为不正确的值时，在复位时看门狗定时器将立即跳闸。想要恢复，需要断电，并安装 E3 重新-初始化跳线，然后再通电。

最常见的更改默认设置的原因是将带有 Servo ICO 和/或 1 的 Turbo PMAC2 捆绑到一个它不是环控制器的 MACRO 环上。在这种情况下，MACRO ICO 应作为时钟源，但重新-初始化流程将在 Servo ICO 上决定。在这种情况下，将 I19 从 7007 改为 6807，保存（SAVE）和重置。

正常复位操作

在一个正常的上电或复位序列中，Turbo PMAC2 的 CPU 读取先前已经保存到闪存中的 I19 的值，然后将它找到其编号的 I-变量设置为 0，指定该集成电路采用自己的相位和伺服时钟，并将其输出到系统。例如，如果 I19 是 6807，I6807 将被设置为 0。然后，为在上电或复位时发现的所有其他伺服和 MACRO 集成电路，自动将时钟-方向 I-变量设置成 3，因此，这些集成电路接收伺服和相位时钟信号作为输入。

3-3 MACRO IC 选型

在 Turbo PMAC 固件 1.936 版本中开始时，I-变量 I20 - I23 必须设置指定用于自动固件功能的 MACRO 集成电路的地址。这不与旧的固件版本兼容。如果更新旧版本的应用程序，载入旧的 I-变量文件后，发出指令 I20..23=*，其次是 SAVE 指令，接着是$$$复位指令。

一些 Turbo PMAC2 系统（目前的 UMAC Turbo）最多可寻址 16 个 MACRO 集成电路。然而，在任何给定时间，只对这些集成电路中的四个有自动固件支持。这些集成电路被称为 MACRO IC 0、1、2、3。变量 I20 至 I23 分别指定 MACRO ICO 到 3 的基地址。这些变量必须正确设置以使用所需集成电路的任何自动固件功能。

MACRO ICO，由 I20 指定，有需自动固件支持的多种功能：

- 显示端口功能（可以动态改变）
- 多路复用端口功能（可以动态改变）
- I-变量 I6800-I6849（仅在上电或复位时自动分配值）
- MACRO 节点 0-15（可以动态改变）
- MACRO 类型 1 辅助通信，如果 I84=0

MACROIC1，通过 I21 指定，有需要自动固件支持的多种功能：

- I-变量 I6850-I6899（仅在上电或复位时自动分配值）
- MACRO 节点 16-31（可以动态改变）
- MACRO 类型 1 辅助通信，如果 I84=1

MACROIC2，通过 I22 指定，有需要自动固件支持的多种功能：

- I-变量 I6900-I6949（仅在上电或复位时自动分配值）
- MACRO 节点 32-47（可以动态改变）
- MACRO 类型 1 辅助通信，如果 I84=2

MACROIC3，通过 I23 指定的，有需要自动固件支持的多种功能：

- I-变量 I6950-I6999（仅在上电或复位时自动分配值）
- MACRO 节点 48-63（可以动态改变）
- MACRO 类型 1 辅助通信，如果 I84=3

在重新初始化时，Turbo PMAC2 用最低的基址搜索 MACRO 集成电路。I20 分配最低的基地址（如果找到一个）；I21 分配下一个（如果找到），等等。此外，这些变量被赋予缺省值（如 I20=*）时发生上述的同样操作。

3-4 双端口 RAM（DPRAM）的 IC 选型

在 Turbo PMAC 固件 1.936 版本上开始时，可以指定用于固件自动通信功能的双端口 RAM 芯片的基址。这允许支持新的配件，如 Acc-54E USB/以太网通讯板。在固件版本 V1.935 或者更老的版本上，“板载”双端口 RAM 基地址固定为 $060000。

新变量 I24 指定用于固件自动通信功能的双口 RAM 芯片的基址。为了向后兼容，如果 I24 设置为 0，双口 RAM 将被假定基址为$060000。

I24 仅用于上电或复位时。如果要使用不限于板载的双口 RAM 芯片，按照附件(如 Acc-54E)的指示,设置 I24 的值匹配附件的硬件设置,然后发出 SAVE 指令和**$$$**指令，此时附件可用于通信

在重新初始化时，I24 被设为任何双口 RAM 芯片发现的最低基地址。最常见的是，I24 将被设置为$060000 与背板总线通信一起使用，或设为$06C000 与双口 RAM 一起使用，在一个 UMAC Acc-54E USB/以太网通信板上。

3-5 系统配置状态报告

Turbo PMAC 系统可以自动检测并报告有关它们配置的重要信息。该功能通过处理器查询接口集成电路（伺服集成电路，MACRO 集成电路，双口 RAM 芯片，I/O 芯片）可能的地址位置得以实现。这个信息对 Turbo PMAC 系统的初始设置非常有用，并随后确认配置并没有改变。

伺服芯片配置

上电或复位时，Turbo PMAC 的 CPU 自动测试所有可能的伺服芯片的存在和类型，并将结果报告在 I4900 和 I4901 中。I4900 是一个 20 个独立位的集合，其中 0 到 9 位分别报告伺服芯片 0-9 的存在，10 到 19 位分别报告伺服集成电路 0* 到 9*的存在。位值为 1 表示该芯片存在；位值为 0 表示芯片不存在。

I4901 也是一个 20 个独立位的集合，其中 0 到 9 位分别报告伺服集成电路 0-9 的类型，10 到 19 位分别报告伺服集成电路 0* 到 9*的类型。位值为 1 表示是 PMAC2 型“DSPGATE1”集成电路；位值为 0 表示是 PMAC1 型的“DSPGATE”IC（或者如果 I4900 的相应位为 0，表示没有 IC 存在）。

MACRO 芯片配置

上电或复位时，Turbo PMAC 的 CPU 自动测试所有可能的 MACRO 芯片的存在和类型，并将结果报告在 I4902 和 I4903 中。I4902 是一个 16 个独立位的集合，每一位报告位于 16 个可能的位置之一的 MACRO 集成电路的存在。位值为 1 表示该 IC 是存在；位值为 0 表示 IC 不存在。

I4903 也是一个 16 个独立位的集合，每一位报告处于 16 个可能的位置之一的 MACRO 集成电路的类型。位值为 1 表示是 DSPGATE2 集成电路；位值为 0 表示是 MACROGATE IC（或者如果 I4900 的相应位为 0，表示没有 IC 存在）。

虽然一个 Turbo PMAC 系统中可最多安装 16 个 MACRO 集成电路，但其中只有四个可以在任何时候有自动固件功能的支持。I20-I23 包含了这四种芯片的基地址。当系统重新初始化时，这些变量被设置为以最低基址发现的 4 个芯片的值。

双口 RAM 芯片配置

上电或复位时，Turbo PMAC 的 CPU 自动测试所有可能的双端口 RAM 芯片的存在，并将结果报告在 I4904 中。I4904 是一个 8 个独立位的集合，每一位报告位于 8 个可能位置之一的双口 RAM 集成电路的存在。这些芯片中只有一个可以在任何时候固件有自动固件功能的支持。 I24 包含此芯片的基地址。

CPU 区域的配置

上电或复位时，Turbo 的 PMAC 自动测试自身 CPU 单元的配置，并将结果报告在 I4908 中。I4908 是一个 36 位的值，可以报告 CPU 的类型、活动内存的大小、双口 RAM 的大小、电池-供电 RAM 的大小、闪存的大小、辅助串行端口的存在、配件编号和供应商 ID。

UBUS 配件板识别

在 UMAC Turbo 系统中，Turbo PMAC 可以报告关于安装在 UBUS 扩展端口上的附件板的详细信息。此信息报告在变量 I4910 - I4965 中。每个变量都是 36 位的值，有下列内容：
供应商 ID：8 位
存在的选项：10 位
版本号：4 位
卡的 ID（部件号）：14 位

每个变量可以报告一部分信息或该消息的所有部分，这取决于 I39 的设置。如果 I39 设置为 5，变量报告附件板的基地址。

I4910-I4925 报告此信息给 16 个可能的配有伺服集成电路的附件板，如 Acc-24E2、24E2A、24E2S 和 51E。

I4926-I4941 报告此信息给 16 个可能的配有 MACRO 集成电路的附件板，如 Acc-5E。

I4942-I4949 报告此信息给 8 个可能的配有双口 RAM 芯片的附件板，如 Acc-54E USB/以太网接口。

I4950-I4965 报告该信息给 16 个可能的配有 I/O 芯片的附件板，如 Acc-14E、28E、36E、53E、59E（ACC-9E、10E、11E 和 12E 的 I/O 板目前无法提供此信息。）

3-6 设置系统时钟频率

相位时钟和伺服时钟信号为整个 Turbo PMAC 系统设置了动力，将硬件和软件的运行同步起来。虽然出厂默认频率--相位时钟的 9.04KHz 和伺服时钟的 2.26KHz--适用于大多数的应用，但是一些应用程序要么需要改变频率，要么可受益于改变其中一个或两个的频率。

由相位与伺服时钟信号驱动的硬件任务包括：

- 锁定编码器计数器
- 锁存并联反馈寄存器
- A/D 转换器的选通和锁存结果数据
- 输出到 D/A 转换器
- 输出到 PWM 电路
- 在 MACRO 环上通讯

由相位与伺服时钟信号驱动的软件任务包括：

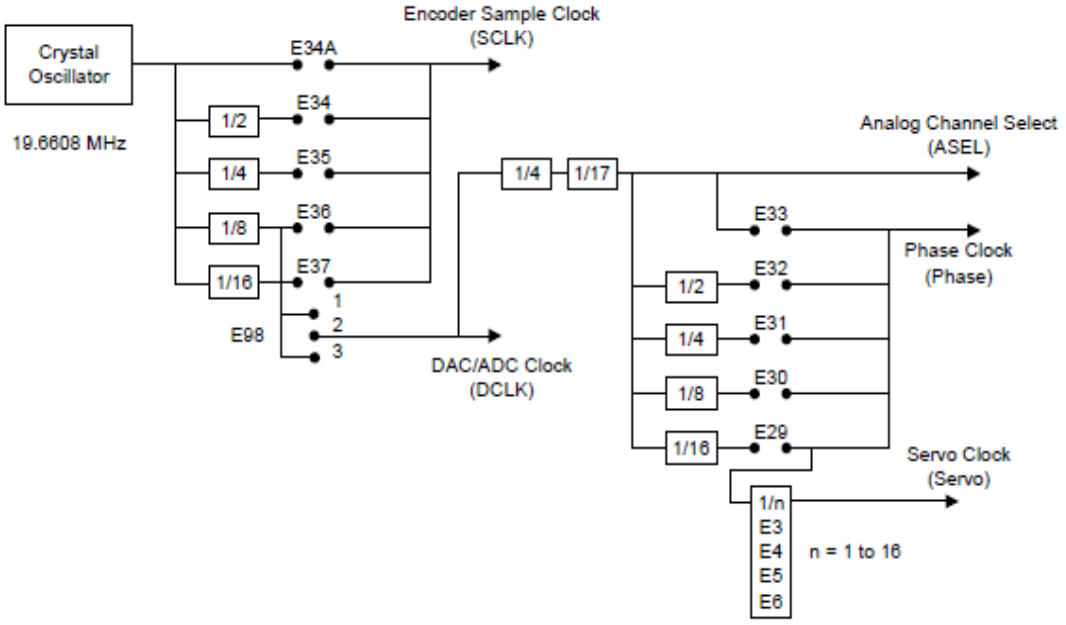
- 数字电流环闭合（相位时钟）
- 电机相位换相（相位时钟）
- 复用 A/D 转换器的多路分解（相位时钟）
- 编码器转换表预处理（伺服时钟）
- 轨迹插补（伺服时钟）
- 位置/速度环闭合（伺服时钟和 Ixx60）
- PLC0 和 PLCC0 执行（伺服时钟和 I8）
- 检查运动程序移动规划（伺服时钟和 I8）

换相和伺服回路闭合的章节将讨论这些信号的最佳频率的选择。一般情况下，更高的频率能带来更高的性能（虽然有收益递减点，而且其他的系统限制不管 Turbo PMAC 运行有多快，都会封顶性能），代价是增加处理器的使用。

设置 Turbo PMAC 的相位和伺服时钟频率

在一个 Turbo PMAC 控制器上，系统时钟频率由 E-点跳线的设置决定。虽然一些 E-点数针对特定的控制器是一定的，但是时钟频率跳线数对于所有的 Turbo PMAC 控制器是通用的，因此可以总结如下。请查阅硬件参考手册，针对特定的控制器，以获得电路板上这些跳线的位置。

PMAC（1）时钟分频电路



E98 DCLK 频率控制

E98 控制 DCLK 时钟信号的频率，其中 DCLK 时钟信号用于控制进入到板上所有 D/A 转换器的比特率，也控制连接到电路板上的所有 Acc-28 A/ D 转换器的输出比特率。出厂默认设置连接 E98 的引脚 1 和 2，产生 2.46MHz 的 DCLK 频率。这个设置应该保留，除非任何 Acc-28 A/D 转换器连接到电路板上。在这种情况下，跳线应该被移动来连接引脚 2 和 3，把频率降低到 1.23MHz。

E29-E33 相位时钟频率控制

跳线集 E29-E33 决定了相位时钟频率，控制其来自 E98 设置的 DCLK 信号的分频。该设置下，只有一个跳线可随时安装。最大的可能相位时钟频率是 DCLK 频率的 1/68---对于 2.46 MHz 的 DCLK，是 36.14KHz，或 1.23MHz DCLK 下的 18.07KHz。这是由 E33 选择的频率。下面的表显示了可选的可能的相位时钟频率：

跳线选择	2.46MHz 的 DCLK（E98 1-2）	1.23MHz 的 DCLK(E98 2-3)
E33	36.14KHz	18.07KHz
E32	18.07KHz	9.04KHz
E31(默认)	9.04KHz	4.52KHz
E30	4.52KHz	2.26KHz
E29	2.26KHz	1.13KHz

E3-E6 伺服时钟频率控制

跳线集 E3-E6 决定了伺服时钟频率，控制其对于 E29-E33 和 E98 设置的相位时钟信号的分频。这四个跳线创建一个二进制数 N 来指定相位时钟频率除以(N+1)获得伺服时钟频率。跳线接通（为 ON）时创建一个 0-位（即该位值为 0）；跳线断开（OFF）产生 1-位（即该位值为 1），E3 的位权最低。下表显示了可能的设置：

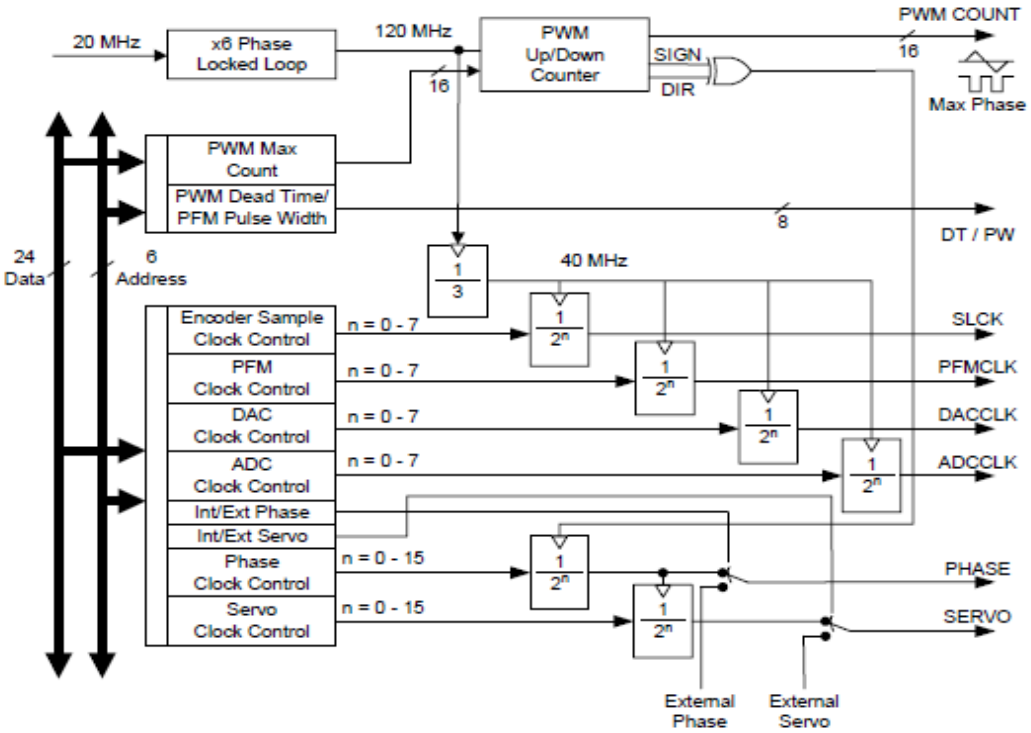
E3	E4	E5	E6	除数因子 N+1	E3	E4	E5	E6	除数因子 N+1
ON	ON	ON	ON	1	ON	ON	ON	ON	9
OFF	ON	ON	ON	2	OFF	ON	ON	OFF	10
ON	OFF	ON	ON	3	ON	OFF	ON	OFF	11
OFF	OFF	ON	ON	4*	OFF	ON	OFF	OFF	12
ON	ON	OFF	ON	5	ON	ON	OFF	OFF	13
OFF	ON	OFF	ON	6	OFF	ON	OFF	OFF	14
ON	OFF	OFF	ON	7	ON	OFF	OFF	OFF	15
OFF	OFF	OFF	ON	8	OFF	OFF	OFF	OFF	16
* 默认									

设置 Turbo PMAC2 的相位和伺服时钟频率

在一个 Turbo PMAC2 控制器（包括 UMAC 和 QMAC 系统）中，系统时钟频率是由伺服 IC 或 MACRO IC 的 I-变量的设置决定的，其中伺服集成电路或者 MACRO 集成电路已选作为系统的时钟源。这种集成电路的选择（通常是不带 MACRO 环的系统中第一个伺服集成电路，或者带有 MACRO 环的系统中第一个 MACRO 芯片）涵盖在上一节中，并且出厂默认的选择通常是可以接受的。

当选择 Servo IC m 作为时钟源时，I-变量 I7m00、I7m01 和 I7m02 控制这些系统时钟频率。当 MACRO IC0 被选中作为时钟源，就像在 Ultralite Turbo PMAC2 板一样，I-变量 I6800、I6801 和 I6802 控制这些系统时钟频率。

PMAC2 门阵列芯片时钟控制



I7m00/I6800 最大相位频率控制

选定作为系统时钟源的伺服电路的 Servo IC m，其 I7m00(或者 MACRO IC0 的 I6800) 设置内部最大相位时钟信号的频率，最大的可能的相位时钟频率的频率。它根据下式完成设置：

$$MaxPhase(kHz) = \frac{117,964.8}{2 * I7m00 + 3}$$

在 I7m00 默认值 6527 下，它为 MaxPhase（最大相位）设置一个 9.04 kHz 的频率。

I7m00 还为 Servo IC m 的所有通道设置了 PWM 输出频率；这个频率正好等于最大相位频率的一半。

I7m01/I6801 相位时钟频率控制

选定作为系统时钟源的伺服电路的 Servo IC m，其 I7m01(或者 MACRO IC1 的 I6801)设置系统的相位时钟信号的频率，控制它对 MaxPhase（最大相位）时钟频率的分频。它根据下式完成设置：

$$Phase(kHz) = \frac{MaxPhase(kHz)}{I7m01 + 1}$$

在 I7m01 默认值为 0（除数为 1）以及 MaxPhase 缺省频率为 9.04 kHz，据此设定的 9.04 kHz（周期为 110 微秒（μs））的相位时钟频率。

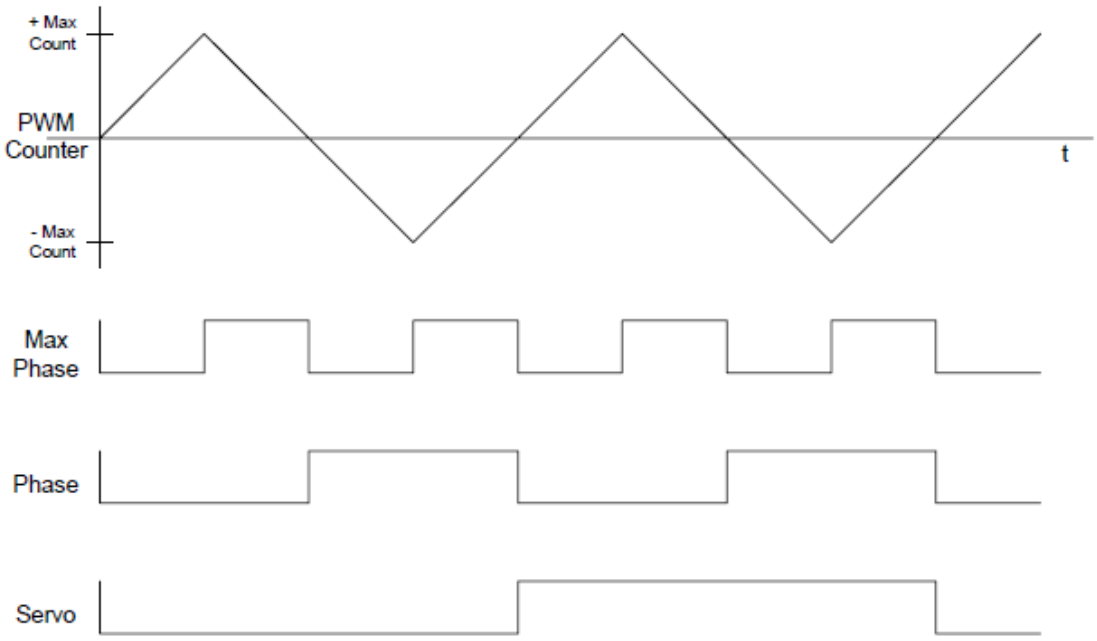
I7m02/I6802 伺服时钟频率控制

选定作为系统时钟源的伺服电路的 Servo IC m，其 I7m02(或者 MACRO IC1 的 I6802)设置系统的伺服时钟信号的频率，控制它对相位时钟频率的分频。它根据下式完成设置：

$$Servo(kHz) = \frac{Phase(kHz)}{I7m02 + 1}$$

I7m02 默认值为 3（除数为 4）以及缺省的相位时钟频率为 9.04 kHz，据此设定伺服时钟频率为 2.26 kHz（周期为 442 微秒（μs））。下图显示了 PWM 计数器(其频率由 I7m00/I6800 最大计数参数设定)、由此产生的最大相位(MaxPhase)时钟信号、相位和伺服时钟信号（从 MaxPhase 最大相位导出）之间的关系。

PMAC2 时钟信号示例



设置 I10 伺服更新时间参数

在任何的 Turbo PMAC 系统中，要设置 I10 伺服更新时间参数来匹配选择的伺服频率。I10 被插值算法用来决定每个伺服周期递增多远的位置，它必须被正确设置以适当的速度获得轨迹。I10 的缩放比例值是 2^{23}（8,388,608）来匹配 1 毫秒（1 kHz 的伺服时钟频率）的伺服更新类型。

将 I10 设置为伺服更新周期函数，方程是：

$$I10 = 8{,}388{,}608 * ServoUpdatePeriod(m\,sec) = \frac{8{,}388{,}608}{ServoUpdateFreq(kHz)}$$

I10 的默认值为 3,713,707，匹配默认的伺服周期 442 微秒，所以如果你保留这个默认周期，你不必改变 I10。请参阅软件参考手册中对 I10 的描述，获得 I10 设置的详细说明。要使得 I10 的新值生效，为所使用的每一个坐标系发出一个%100 的指令或 SAVE 指令并重置 Turbo PMAC。

3-7 在 MACRO 环上设置一个 Turbo PMAC2

Turbo PMAC2 控制器可以通过 MACRO 环命令轴和 I/O。最常见的是，这是通过一个 Ultralite 板级 Turbo PMAC2 控制器完成的，它被安装为主机计算机上的扩展卡，通过 MACRO 环通信到 MACRO 站、基于 MACRO 的驱动器和/或 MACRO 的外围设备。但是，对于配有 ACC-5E MACRO 接口的一个 UMAC Turbo 或者安装 MACRO 接口选项的一个 QMAC，通过 MACRO 环来命令装置也是可能的。

一些变量必须正确设置用于适当的环操作。通常情况下，这是通过使用 PC 上的 Turbo 设置程序自动完成的。下面的说明允许直接手动设置这些变量。

MACRO 环频率控制变量

MACRO 环更新频率是环的主控制器的相位时钟频率。如果在环上有多个 Turbo PMAC2 控制器，其中只有一个可以是环的主控制器（其他的是主控制器，但不是环的主控制器）。当然，如果在环上只有一个 Turbo PMAC2 控制器，那么它将是该环的主控制器。确定哪个 Turbo PMAC2（从技术上讲，是 Turbo PMAC2 上的哪个 MACRO 集成电路）作为环的主控制器，解释如下。

当环的主控制器有能力将环上其他设备的时钟产生强制成同步时，强烈建议，环上的所有设备，无论是其他的 Turbo PMAC2 控制器，还是任何从属设备，都设置为相同的相位时钟频率。确定哪个集成电路设置相位时钟频率以及实际设置相位时钟频率用于 Turbo PMAC2 控制器，已经在上文进行了说明。

对于一个 TurboPMAC2 驱动一个 MACRO 环，MACRO IC0 应该产生相位时钟信号。这意味着，I19 应设置为 6807（对于几乎任何能够驱动一个 MACRO 环的 Turbo PMAC2，这是默认的），并且 I6800 和 I6801 设定相位的时钟频率。

I7：相位周期延拓

在 Turbo PMAC2 板上，可以在相位更新软件的执行之间跳过硬件相位时钟周期。一个 Turbo PMAC2 板将在每（I7+1）个硬件相时钟周期都执行相位更新软件--换相和/或电流环闭合。I7 的默认值是 0，所以正常情况下，Turbo PMAC2 每一个硬件相位时钟周期都执行相位更新软件。

如果 Turbo PMAC2 板正在通过 MACRO 环闭合电流环用于直接 PWM 控制，最好是每个软件相位更新都有两个硬件环更新周期（这以硬件的相位时钟频率发生）。这消除了电流环中的一个环周期的延迟，它允许稍微高的增益和性能。要做到这一点，将 I7 设置为 1，这样相位更新软件将在每第二个硬件相时钟周期和环更新周期执行一次。

一般而言，最好以约 9kHz 的更新率（默认频率）闭合电流环。如果每个电流回路更新需要两个环更新，那么环更新频率需要被设置为 18KHz。如果环上的总有效节点不超过 40 个，这是可能的。要实现这一点，将 I6800 设置为默认值的一半，将 I6801 设置为默认值 0。

注意：

在作出这个改变时，先改变 TurboPMAC2 的 I6800 变量，然后是 MACRO 站的 MI992。只改变 MACRO 站的 MI992，以及接下来的 **MSSAVE** 命令和 **MS$$$**指令，会导致站的看门狗定时器跳闸。

I6840：MACRO IC0 主控制器配置

Turbo PMAC2 的任何 MACRO IC 跟 MACRO 站通讯必须被配置为环上的主控制器。为了 MACRO 协议的目的，每一个 MACRO IC 是一个单独的逻辑主控制器并有自己的主控号，即使在单个物理 Turbo PMAC2 上可能有多个 MACRO 集成电路。

每个环上必须有一个且只能有一个环控制器（同步主控），而且应该是环上的一个（且只能有一个）Turbo PMAC2 板上的 MACRO IC0。

在 Turbo PMAC2 上，将 I6840 设置为$30 从而使得该卡的 MACRO IC0 成为环控制器。这样就将该变量的第 4 位和第 5 位设置为 1。将第 4 位设置为 1 使得 IC 成为环上的主控；将第 5 位设置为 1 使得 IC 成为环控制器，通过自身启动每个环周期。

在 Turbo PMAC2 上，若想其 MACRO IC0 成为一个主控而不是环控制器，那么 I6840 应该设置为$90。这样就将变量的第 4 位和第 7 位设置为 1。将第 4 位设置为 1 使得 IC 成为环上的一个主控；将第 7 位设置为 1 将导致该 IC，每次只要接收到由 I6841 指定的一个环数据包，就被同步到环控制器 IC。

I6890/I6940/I6990：MACRO IC1/2/3 的主控制器配置

如果选项 1U1、1U2 和/或 1U3 被选定，一个 Turbo PMAC2 Ultralite 可能有额外的 MACRO 芯片。如果 Acc-5E 上的选项 1 被选定，或者如果有多个 Acc-5E 板被选定，一个 UMAC Turbo 系统会有额外的 MACRO 芯片。这些额外的芯片应该被设置为主控而不是环控制器，通过分别将 I6890、I6940 和 I6990 设置为$10。这样就将变量的第 4 位设置为 1，从而使 IC 构成环上的一个主控。这些 IC 不应该成为同步主控，而且因为他们不控制自己的板上的时钟信号，所以它们的内部时钟并不需要被同步到环上（仅 MACRO IC0 需要这么做）。

I6841/I6891/I6941/I6991：MACRO IC0/1/2/3 节点激活控制

Turbo PMAC2 上的 I6841、I6891、I6941、I6991 分别控制卡上的 MACRO IC0、1、2 和 3 中的 16 个 MACRO 节点中那些节点被激活。它们也控制各自相应 IC 的主站号，以及创建同步信号的数据包的节点号。这些 I-变量的位分布如下：

0 到 15 位：分别激活 MACRO 节点 0 到节点 15（1=激活，0=不激活）。这 16 位（通常读作为四个十六进制数）分别控制激活一个 Turbo PMAC2 上 MACRO IC 的 MACRO 节点。在匹配 MACRO 站上的每个激活的节点，无论是用于伺服、I/ O 或辅助通讯，都应该将它的节点激活位设置为 1。

当与泰道的 MACRO 站一起工作时，一个 Turbo PMAC2 上的每个 MACRO IC 的节点 15 必须被激活用以允许辅助通讯，所以如果 IC 用于和 MACRO 站进行通信，这个变量的第 15 位应始终设置为 1。

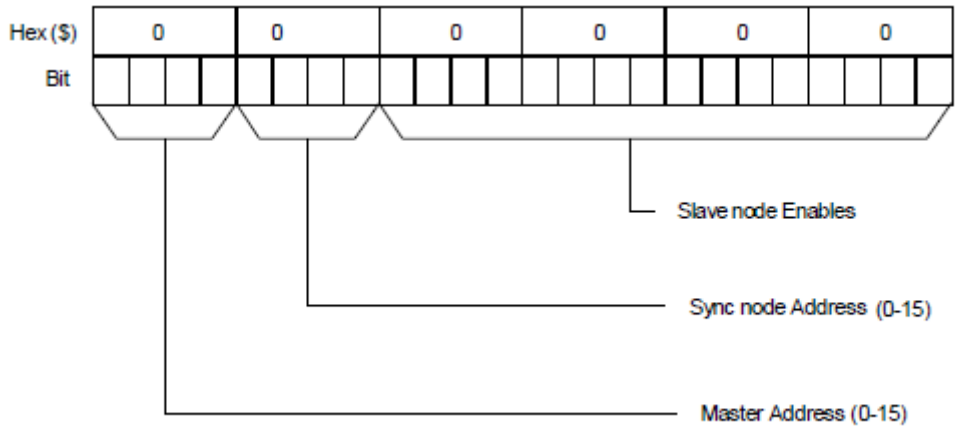
16-19 位：数据包同步节点从站号。这四位一起（通常理解为一个十六进制数字）组成数据包的从站号（0～15），数据包被 PMAC2 接收将置位 MACRO IC 的同步数据包接收的状态位。通常，这个数字被设置为$F（15），因为节点 15 始终是激活的。

Turbo PMAC2 必须看到该状态位定期置位，否则它将假定环有问题，并关闭

环上的伺服和 I/O 输出。在所有不是环控制器的 Turbo PMAC2 的 MACRO IC0 的 I6840 的第 7 位必须设置为 1，基于收到同步数据包，以使能他们的相位时钟同步到环控制器的相位时钟。

20-23 位：主控号码。这四位共同构成了 MACRO 环上 MACRO IC 的主控号（0 至 15）。每个作为环的一个主控的 MACRO IC，无论是在同一卡上还是不同的卡上，都必须有自己的主控号，并作为一个独立的主控站为了环协议的目的。此主控号和 PMAC2 通过 MACRO 环发送的每个数据包形成了一半的地址字节。

主控号可以和 MACRO IC 号是同一个数字（例如 MACRO IC0 具有主控号 0，MACRO IC1 具有主控号 1，等等），而且如果在环上只有一个 Turbo PMAC2，很可能就是这种情况。然而，这不是必需的。如果 1 型的主机-主机的辅助通讯要被使用，那么作为环控制器的 MACRO IC 必须有主控号 0。



注：Hex：十六进制；Slave node Enables：从站节点使能；
Sync node Address：同步节点地址；Master Address：主控地址

在上文部分和用于 3U MACRO 站的 SW1 开关设置的硬件参考手册中所示的表提供了一个起点用于 Turbo PMAC2 的 I6841/I6891/I6941/I6991 的值。如果 I/O 节点启用或者多个 3U MACRO 站被从一个 MACRO IC 上命令，这些 I-变量的附加位可能被设置为 1，

I70/I72/I74/I76：MACRO IC0/1/2/3 节点辅助功能使能

I70/I72/I74/I76 都是 16 位的 I-变量（0-15 位），对于匹配 MACRO IC 0/12/3 的位号的 MACRO 节点号，其中变量的每一位分别控制节点的辅助标记功能的启用或禁用。位值为 1 使辅助标记功能启动；位值为 0 禁用它。如果启用该功能，PMAC 分别自动复制信息在 MACRO 接口标志寄存器和 MACRO IC0/1/2/3 对应的 RAM 寄存器$00344n/$00345n/$00346n/和$00347n（其中 n 是 IC 的节点号 0-15）之间。

需要注意的是，Turbo PMAC 的 MACRO 节点号（与单个的 MACRO IC 节点号相反）从 0 到 63，其中 MACRO IC0 上板节点为 0-15，MACRO IC1 上板节点 16-31，MACRO IC2 上板节点 32-47，MACRO IC3 的板节点 48-63。

每个 MACRO 节点 n 用于伺服功能，而且应该将相应 I70、I72、I74、I76 的第 n 位设置为 1。使用节点 n 的电机 x 的 Ixx25 应该寻址$00344n、$00345n、

$00346n 或$00347n，而不是 MACRO 寄存器本身的地址（见下文）。如果一个 MACRO 节点 n 的寄存器 3 用于其他用途，如直接 I/O，相应的 I70、I72、I74、I76 的第 n 位应设置为 0，所以这个复制功能不会覆盖这些寄存器。

通常情况下，MACRO 站的非伺服 I/O 功能不涉及辅助标志功能，所以对于在 Turbo PMAC2 和 MACRO 站之间传输 I/O 的任何节点来说，这个标志复制功能应保持禁用。如果任何辅助通信是在 Turbo PMAC2 和 MACRO 站之间在节点 14 和/或 15 中完成的，那么这些变量的 14 位和 15 位必须被设置为 0。

示例：

```
I70=$3;          已启用，为 MACRO IC0 的节点 0 和 1
I72=$30;         已启用，为 MACRO IC1 的节点 4 和 5
I74=$3300;       已启用，为 MACRO IC2 的节点 8,9,12,13
I76=$3333;       已启用，为 MACRO IC3 的节点 0,1,4,5,8,9,12,13
```

I71/I73/I75/I77：MACRO IC0/1/2/3 节点协议类型控制

I71/I73/I75/I77 都是 16 位的 I-变量（位 0-15），分别对应于 MACRO 集成电路 0/1/2/3，其中变量的每一位控制 PMAC 使用用途 MACRO 0 型协议还是 MACRO1 型协议用于节点（节点号匹配位数），出于辅助伺服标志传输的目的。位值为 0 设置一个 0 型协议；位值为 1 设置 1 型协议。

所有的 3U MACRO 站节点都使用 1 型协议，所以与 MACRO 站一起的每个 MACRO 节点 n 要用于伺服用途就必须将 I1002 的 n 位设置为 1。一般来说 I71= I70，I73= I72，I75= I74 和 I77=I76，在一个与 MACRO 站通信的 Turbo PMAC 上。

请记住，如果用于多个 MACRO 站的伺服节点听从一个 MACRO IC 的命令，必须对各站的所有激活伺服节点选择协议。

I78：MACRO 主/从辅助通讯超时

如果 I78 设置大于 0，使用节点 15 的 MACRO 1 型主/从辅助通信协议将会启用。Turbo PMAC 实现使用该通信协议是通过 **MACROSLAVE（MS），MACROSLVREAD（MSR），以及 MACROSLVWRITE（MSW）**指令。

如果启用此功能，I78 设置 PMAC 伺服周期中的超时值。在这种情况下，如果 PMAC 在 I78 个伺服周期内没有得到一个对节点 15 辅助通讯指令的响应，它就会停止等待并注册一个 MACRO 辅助通信错误，将全局状态寄存器 X：$000006 的第 5 位置位。

如果需要与 MACRO 站进行任何辅助的通信，I78 必须被设定为大于 0。这会为 1 型辅助通信保留节点 15。 建议值为 32。如果 I78 被设定为大于 0，I70、I72、I74 以及 I76 的 15 位都必须被设置为 0，这样节点 15 也不会用于标志传输。

I79：MACRO 主/主辅助通信超时

如果 I79 设置为大于 0，使用节点 14 的 MACRO 1 型主/主辅助通信协议将启用。Turbo PMAC 实现该通信协议是通过使用 **MACROMASTER（MM）、MACROMSTREAD（MMR）、MACROMSTWRITE（MMW）**指令。只有作为“环控

制器”的 Turbo PMAC 才可以执行这些命令，其他的 Turbo PMAC 作为环上的主控，可以响应这些来自环控制器的命令。

如果启用此功能，I79 设置 PMAC 伺服周期中的“超时”值。在这种情况下，如果 Turbo PMAC 在 I79 个伺服周期内没有得到对节点 14 主/主辅助通信命令的响应，它就会停止等待，并注册一个“MACRO 辅助通信错误”，置位全局状态寄存器 X:$000006 的第 5 位。

如果需要与 MACRO 站进行任何辅助的通信，I79 必须被设定为大于 0。 建议值为 32。如果 I79 的值大于 0 已经被保存到 PMAC 的非易失性存储器中，那么在随后的上电/复位时，I70 的 14 位设定为 0，节点-14 广播位（I6840 的 14 位）设置为 1，节点 14 的激活位（I6841 的 14 位）被设置为 1，不管这些变量的保存值是多少。这就为 1 型主/主辅助通信保留了 MACRO IC0 的节点 14。

I80、I81、I82：MACRO 环检查周期和限制

如果 I80 设置为一个大于零的值，Turbo PMAC 将自动监测 MACRO 环断裂或重复的 MACRO 通信错误。一个非零值以 Turbo PMAC 伺服周期设置误差检测周期时间。Turbo PMAC 检查是为了看到“同步节点” 的数据包（见 I6840 和 I6841）被定期接收，而且没有出现过定期的通信错误。

这些检查的限制可以被变量 I81 和 I82 设置。如果在该时间间隔内小于 I82 同步节点的数据包被接收并被检测，或者如果在这个区间内 I81 或更多的环通信错误被检测到，Turbo PMAC 将假定一个主要的环问题，所有电机将被关闭。Turbo PMAC 将置位全局状态位“环错误”（X:$000006 的第 4 位）作为这个错误的一个指示。

Turbo PMAC 会在每一个实时中断（即每（I8+1）个伺服周期）查找一次同步节点数据包接收和环通信错误。该时间间隔由 I80 设置，必须足够大到 PMAC 的 I82 实时中断可以在时间间隔内执行，否则假的环错误将被检测到。请记住，长的运动程序的计算可能会引起在实时中断中跳跃。通常，使用 I80 的值设定约 20 毫秒的时间间隔。I80 可以根据下面的公式来设置：

I80=所需的周期时间（毫秒）*伺服更新频率（kHz）

例如，在 2.26 kHz 的缺省伺服更新频率，要获得 20 毫秒的环检查周期的时间间隔，I80 将被设置为 20 *2.26≅45。

MACRO 节点地址

MACRO 环工作时通过在整个环上高速复制寄存器。因此，环上的每个 Turbo PMAC2 主控制器与其从站通信需要通过读取和写入到其自己的地址空间中的寄存器。MACRO 硬件自动处理跨环的数据传输。

在 Turbo 固件 1.936 版本中开始时，多达 4 个 MACRO 集成电路的基地址，必须分别在 I20--I23 中为 MACRO 集成电路 0-3 指定。在此之前，基地址是固定的，分别为$078400/$079400/$07A400/$07B400。仅 UMAC Turbo 系统可以支持任何其他配置，并且另一个配置很少使用。

下表给出 Turbo PMAC2 控制器的 MACRO 环寄存器的地址。

注意：

在 UMAC Turbo 系统中有可能，虽然可能性不大，有其他的地址。在这些系统中，第四个数字不必是 4，它的值也可以是 5、6 和 7。

寄存器地址，用于 MACRO IC0，且 I20=$078400（默认）

	Turbo PMAC2	地址：	MACRO IC0	
节点#	**Reg. 0**	**Reg. 1**	**Reg. 2**	**Reg. 3**
0	Y:$078420	Y:$078421	Y:$078422	Y:$078423
1	Y:$078424	Y:$078425	Y:$078426	Y:$078427
2	X:$078420	X:$078421	X:$078422	X:$078423
3	X:$078424	X:$078425	X:$078426	X:$078427
4	Y:$078428	Y:$078429	Y:$07842A	Y:$07842B
5	Y:$07842C	Y:$07842D	Y:$07842E	Y:$07842F
6	X:$078428	X:$078429	X:$07842A	X:$07842B
7	X:$07842C	X:$07842D	X:$07842E	X:$07842F
8	Y:$078430	Y:$078431	Y:$078432	Y:$078433
9	Y:$078434	Y:$078435	Y:$078436	Y:$078437
10	X:$078430	X:$078431	X:$078432	X:$078433
11	X:$078434	X:$078435	X:$078436	X:$078437
12	Y:$078438	Y:$078439	Y:$07843A	Y:$07843B
13	Y:$078438	Y:$078439	Y:$07843A	Y:$07843B
14	X:$078438	X:$078439	X:$07843A	X:$07843B
15	X:$07843C	X:$07843D	X:$07843E	X:$07843F

寄存器地址，用于 MACRO IC1，且 I21=$079400（默认）

	Turbo PMAC2	地址：	MACRO IC1	
节点号	**Reg. 0**	**Reg. 1**	**Reg. 2**	**Reg. 3**
0	Y:$079420	Y:$079421	Y:$079422	Y:$079423
1	Y:$079424	Y:$079425	Y:$079426	Y:$079427
2	X:$079420	X:$079421	X:$079422	X:$079423
3	X:$079424	X:$079425	X:$079426	X:$079427
4	Y:$079428	Y:$079429	Y:$07942A	Y:$07942B
5	Y:$07942C	Y:$07942D	Y:$07942E	Y:$07942F
6	X:$079428	X:$079429	X:$07942A	X:$07942B
7	X:$07942C	X:$07942D	X:$07942E	X:$07942F
8	Y:$079430	Y:$079431	Y:$079432	Y:$079433
9	Y:$079434	Y:$079435	Y:$079436	Y:$079437
10	X:$079430	X:$079431	X:$079432	X:$079433
11	X:$079434	X:$079435	X:$079436	X:$079437
12	Y:$079438	Y:$079439	Y:$07943A	Y:$07943B
13	Y:$07943C	Y:$07943D	Y:$07943E	Y:$07943F
14	X:$079438	X:$079439	X:$07943A	X:$07943B
15	X:$07943C	X:$07943D	X:$07943E	X:$07943F

寄存器地址，用于 MACRO IC2，且 I22=$07A400（默认）

	Turbo PMAC2	地址：	MACRO IC2	
节点号	**Reg. 0**	**Reg. 1**	**Reg. 2**	**Reg. 3**
0	Y:$07A420	Y:$07A421	Y:$07A422	Y:$07A423
1	Y:$07A424	Y:$07A425	Y:$07A426	Y:$07A427
2	X:$07A420	X:$07A421	X:$07A422	X:$07A423
3	X:$07A424	X:$07A425	X:$07A426	X:$07A427
4	Y:$07A428	Y:$07A429	Y:$07A42A	Y:$07A42B
5	Y:$07A42C	Y:$07A42D	Y:$07A42E	Y:$07A42F
6	X:$07A428	X:$07A429	X:$07A42A	X:$07A42B
7	X:$07A42C	X:$07A42D	X:$07A42E	X:$07A42F
8	Y:$07A430	Y:$07A431	Y:$07A432	Y:$07A433
9	Y:$07A434	Y:$07A435	Y:$07A436	Y:$07A437
10	X:$07A430	X:$07A431	X:$07A432	X:$07A433
11	X:$07A434	X:$07A435	X:$07A436	X:$07A437
12	Y:$07A438	Y:$07A439	Y:$07A43A	Y:$07A43B
13	Y:$07A43C	Y:$07A43D	Y:$07A43E	Y:$07A43F
14	X:$07A438	X:$07A439	X:$07A43A	X:$07A43B
15	X:$07A43C	X:$07A43D	X:$07A43E	X:$07A43F

寄存器地址，用于 MACRO IC3，且 I23=$07B400（默认）

	Turbo PMAC2	地址：	MACRO IC2	
节点号	**Reg. 0**	**Reg. 1**	**Reg. 2**	**Reg. 3**
0	Y:$07B420	Y:$07B421	Y:$07B422	Y:$07B423
1	Y:$07B424	Y:$07B425	Y:$07B426	Y:$07B427
2	X:$07B420	X:$07B421	X:$07B422	X:$07B423
3	X:$07B424	X:$07B425	X:$07B426	X:$07B427
4	Y:$07B428	Y:$07B429	Y:$07B42A	Y:$07B42B
5	Y:$07B42C	Y:$07B42D	Y:$07B42E	Y:$07B42F
6	X:$07B428	X:$07B429	X:$07B42A	X:$07B42B
7	X:$07B42C	X:$07B42D	X:$07B42E	X:$07B42F
8	Y:$07B430	Y:$07B431	Y:$07B432	Y:$07B433
9	Y:$07B434	Y:$07B435	Y:$07B436	Y:$07B437
10	X:$07B430	X:$07B431	X:$07B432	X:$07B433
11	X:$07B434	X:$07B435	X:$07B436	X:$07B437
12	Y:$07B438	Y:$07B439	Y:$07B43A	Y:$07B43B
13	Y:$07B43C	Y:$07B43D	Y:$07B43E	Y:$07B43F
14	X:$07B438	X:$07B439	X:$07B43A	X:$07B43B
15	X:$07B43C	X:$07B43D	X:$07B43E	X:$07B43F

注意：

与 MACRO 站一起，只有那些映射到 Turbo PMAC2 Y 寄存器的节点（0，1，4，5，8，9，12 和 13）可用于伺服控制。这些节点在上面的表中没有被涂成阴影。那些映射到 X 寄存器中的节点（2，3，6，7，10，11 和 14）可用于 I／O 控制。节点 15 是为 1 型辅助通信保留的。节点 14 通常是为广播通信保留的。

3-8 复位和重新初始化 Turbo PMAC

了解一个 TurboPMAC 系统是如何重置或重新初始化的以及在每种情况下要采取哪些行为，这是很重要的。

复位的方法

基本上有三种方式，Turbo PMAC 系统可以重置：

1、重新上电给数字电路

2、硬件复位

3、软件复位

循环供电

去除 Turbo PMAC 的数字电路的电源，然后再重新供电，迫使 Turbo PMAC 经过它的复位周期。在大多数的 Turbo PMAC 中，电源是 5V 电源，根据需要为低电压部件如 CPU 和 RAM 调低电压。就本文来说，只有 UMAC CPCI 需要为 CPU 部分提供一个单独的 3.3V 电源。

为模拟电路去除和再应用+ /-12V 到+/-15V 的电源，不论这些电路是否与数字电路光隔离，都不会导致 Turbo PMAC 系统复位（虽然它可能会引起一个故障条件）。

硬件复位

一个 Turbo PMAC 系统的硬件复位是通过将一个特殊的数字输入线（INIT/）拉低，然后再释放它为高电平（它有一个内部上拉电阻）来实现的。这个信号有几种可能的来源。在板级的 Turbo PMAC 控制器上，该信号在 JPAN 控制面板端口（仅 Turbo PMAC）和 JTHW 复用器端口上是可用的。在模块化 UMAC 和 UMAC-CPCI 系统中，它是 CPU 板上的一个输入。

对于安装在 ISA 或 VME 总线的 Turbo PMAC 板，总线复位线可以通过在 Turbo PMAC 的安装跳线 E39 进而和 Turbo PMAC 的复位线绑在一起，这样，主计算机的复位也会迫使产生一个 Turbo PMAC 的硬件复位。

当硬件复位线为低电平时，CPU 停止运行，系统中的伺服、MACRO 和 I/O 的 ASIC 将被保持在他们的复位状态，这会迫使所有的离散量输出为关闭状态，所有的连续量输出为零状态。

软件复位

一个 Turbo PMAC 系统的软件复位是通过发出**$$$**在线复位指令来完成的。

一个正常复位的动作

当 Turbo PMAC 被配置为一个正常的复位时，如果使用任何上述方法复位，本节描述的标准动作会发生。当以下两个条件都为真时，Turbo PMAC 被配置为正常的复位：

1、重新初始化跳线（Turbo PMAC 上的 E51，Turbo PMAC2 上的 E3）未安装。
2、这四个用于 CPU 的引导模式跳线（E4-E7 在搭载的 Turbo CPU 板上，E20-E23 在 UMAC 和 UMAC-CPCI CPU 板上）是在其标准配置下（外部两个跳线断开，内部两个跳线接通）。

对于以这种方式配置一个的 Turbo PMAC，当应用一个数字电源或者硬件复位线被释放变为高电平或者**$$$**软件复位指令下达后，发生下面的动作：

1、安装的固件从闪存中加载到活动内存中。
2、上次保存的用户配置---变量的值和定义、用户程序、表格和缓冲区---从闪存中加载到活动内存和寄存器中。在该加载过程中，保存的数据的校验总和将被评估。如果保存的 I-变量的校验总和与数据不匹配，所有的活动内存中的 I-变量将被返回为出厂默认值。如果程序和缓冲区的校验总和不匹配数据，所有这些程序和缓冲区将完全从活动内存中清除。
3、系统的基本配置---存储器容量、ASIC 存在、位置和类型---被进行检查并记录。所有 ASIC 的计数器被清零。
4、Ixx80 的 0 位设置为 1，所有电机启用。
5、所有现有的 PLC 程序（其操作由 I5 的保存值所允许）被激活。

重新初始化下的复位的动作

当 Turbo PMAC 被配置用于重新初始化时，如果使用上述任何的方法复位时，在本节中所描述的动作会发生。同时满足以下两个条件，Turbo PMAC 被配置为重新初始化：

1、重新初始化跳线（Turbo PMAC 上的 E51，Turbo PMAC2 上的 E3）**被安装**。
2、这四个用于 CPU 的引导模式跳线（搭载 Turbo CPU 板上的 E4-E7， UMAC 和 UMAC-CPCI CPU 板上的 E20-E23）是在其标准配置下（外部两个跳线关闭，内部的两个跳线接通）。

对于以这种方式配置一个的 Turbo PMAC，当数字电源被应用或者硬件复位线被释放变为高电平或**$$$**软件复位指令下达后，发生以下动作：

1、安装的固件从闪存中加载到活动内存中。
2、出厂默认的 I-变量从固件中加载到活动内存和寄存器中。（闪存中上次保存的值不会丢失；他们只是不被使用。）上次保存的用户程序、表和缓冲区被加载到活动内存中，但没有一个会被激活因为默认的 I-变量设置。如果程序和缓冲区的校验总和不匹配数据，所有这些程序和缓冲区完全从活动内存中清

除。

3、系统的基本配置---存储器容量、ASIC 存在、位置和类型---被进行检查并记录。CPU 将基于配置信息做出一些关于默认的 I-变量值的决定。所有的 ASIC 的计数器被清零。

4、因为默认的 I-变量配置，无电机的启用，也没有程序被激活。

用于固件重载的复位的动作

当 Turbo PMAC 被配置为重新加载固件时，如果使用上述任何的复位方法时，将发生本节所描述的动作。Turbo PMAC 配置为固件重装时，四个用于 CPU 的引导模式跳线（搭载的 TurboCPU 板上的 E4 - E7，UMAC 和 UMAC-CPCI CPU 板上的 E20-E23）是在其固件装载配置下（第一的跳线关闭，后三个跳线接通）。

对于这种配置方式的 Turbo PMAC，当数字电源被应用或者硬件复位线被释放为高电平或者**$$$**软件复位指令下达时，只有引导固件被加载到活动内存中。在这一点上，它已准备好接受下载新的操作固件到它的闪存中通过主串口或者总线端口（操作通过 DSP（不是 VME 或 DPRAM）的主机端口进行）。

PMAC 的执行程序，当它建立与重置到这个模式的 Turbo PMAC 通信时，将自动检测 Turbo PMAC 是在引导模式并准备接受新的固件。它会要求含有下载固件的文件的名称。

更新固件会造成 Turbo PMAC 恢复到默认的 I-变量。在新的固件下载之前，确保配置备份。

如果写一个自定义的应用程序来执行此功能，通过发送？查询命令检测 Turbo PMAC 是否是在引导模式。在这种模式下，它会用字符 BOOTSTRAP PROM 来响应而不是一个十六进制状态字。发送<CTRL-O>字符是将使得 Turbo PMAC 准备下载二进制固件文件。发送完字符后，请等待至少 5 秒钟以确保闪存芯片已准备好接受数据。发送<CTRL-R>命令会将 Turbo PMAC 带出引导模式，并导致其采取正常复位下的动作。

重新初始化和清除指令

$$$***指令导致 Turbo PMAC 的复位和完全重新初始化。除了加载默认的 I-变量值，它也清除了所有的活动 RAM 的缓冲区：运动程序、PLC 程序、表格等。

有些用户总是设置该卡在复位周期重新初始化并清除，然后在每个周期后立即下载所有的参数设置和程序。这一战略背后的逻辑是，使用相同的启动操作顺序，即使新的替换板刚刚放进去。对于那些不希望以任何方式依赖 Turbo PMAC 自身的非挥发性闪存存储的应用，它也非常有用。

对于一个完整的 Turbo PMAC 的重新初始化到已知状态，下面的命令可添加：

```
P0..8191=0
Q0..8191=0
M0..8191->*
UNDEFINE ALL
```

请记住，这些命令仅仅直接影响活动内存（RAM）。要将新的设置复制到非易失性闪存中，使用 SAVE 命令。

第四章 与 TURBO PMAC 通讯

本节涵盖了从一台主机和 Turbo PMAC 通信的基本方面。在这个层面上，我们假设有一个程序，为主机处理这些通讯。 PMAC 执行程序（Pewin32 Pro）是这些程序中最常见的。

如果在最终应用中有一台主机，为主机编写自定义的通信例程，作为应用程序前端软件的一部分。这是一个更高级的话题，它包含在《编写一个主机通讯程序 》的章节。泰道公司提供的通信库在 Microsoft Windows 操作系统中使用。

在基本层面上，Turbo PMAC 可以在任何端口沟通到主机哑终端。通信大多是由来回传送的 ASCII 字符行组成。当然，大多数时候主机将成为一台电脑，有相当多的智能，但根源上会和卡通讯就好像它是一个终端。PMAC 执行电脑程序有一个终端仿真模式，可以直接做到这一点。

4-1 通信端口

每个版本的 Turbo PMAC 具有多个可用的通信端口：一个或两个串行端口、一个总线端口以及一个可选的双端口 RAM 的共享内存的通信接口。Turbo PMAC 的不同配置会有不同类型的这些接口。

Turbo PMAC 可以在多个端口同时通信，而且不同端口的通讯不会互相干扰（原始 PMAC 系列不具备这种能力）。这为用户提供了有用的功能，如同时采用一台主机和一个操作员挂件，或者使用一个端口用于应用程序的通信，另一个用于同步监测和调试。

串行通信端口

Turbo PMAC 控制器有一个或两个串行通信端口，可以提供一个简单的通信接口到主计算机、终端机或其他设备。根据具体的配置和端口，该端口可以使用 RS-232 或 RS-422 信号电平。

主串行端口

Turbo PMAC 的每一个配置都标配了串行通信端口。根据配置的不同，这可以只用于 RS-232 电平（单端+/-10V）、只用于 RS-422 电平（差分 5V），或用户可通过跳线在两个电平之间选择。Turbo PMAC 的 RS-422 电路能够直接连接到计算机的 RS-232 端口上，但噪声容限显著减少，因此通信可能会很容易受到电磁噪声。如果这两个电平都可用，要认识到这仅仅是一个端口，并且在任何给定时间只有一个电压电平可以使用，认识到这一点很重要。

主要串行端口的波特率是由变量 I54 设置。在上电复位时，Turbo PMAC 基于 I54 的设置和 I52 设置的 CPU 速度设置活动的波特率控制寄存器，而且波特率频率是从 CPU 的工作频率中分频的。出厂默认波特率为 38,400。该波特率在 Turbo PMAC 重新初始化时将自动选择，要么是通过使用重新初始化跳线或通过**$$$*** 指令。要更改波特率，改变 I54 的设置，用 SAVE 指令将这个新的设置复制到非易失性存储器中，然后重置 Turbo PMAC。在新的波特率重新建立通讯。

I54 的值和它们产生的波特率值是：

I54	波特率	I54	波特率	I54	波特率	I54	波特率
0	600	4	2400	8	9600	12	38,400
1	900	5	3600	9	14,400	13	57,600
2	1200	6	4800	10	19,200	14	76,800
3	1800	7	7200	11	28,800	15	115,200

由 I54 的奇数设置产生的波特率只有在 CPU 的频率是 30 MHz（技术上来说，是 29.4912 MHz）的整数倍时才精确。这是因为波特率是由 CPU 频率除以（256*N）产生的，其中 N 是从查找表中取的一个整数。该频率对 I54 的奇数设置和频率不是 30 MHz 的倍数的 CPU 来说不是精确匹配。对于这种类型的较低的波特率，误差不显著。然而，波特率为 115,200 的串口通信是可能的只有当 CPU 是在 30MHz 的整数倍（实际上是 29.4912 MHz 的整数倍）运行时。因此，要以这样的速度进行通信，让选项 5Cx 80 MHz 的 CPU 在 60 MHz 下运行、选项 5Dx 100 MHz 的 CPU 运行在 90 MHz、选项 5Ex 160 MHz 的 CPU 运行在 150 MHz 通过将 I52 设置成一个比 CPU 能够运行的更低的值。

可以通过主串口下载新的操作固件（请参阅《重置 Turbo PMAC》）。

辅助串行端口

如果为 Turbo PMAC 选择了 Option 9T，直接地或作为部分的选件包（作为 PMAC 梯形图编程环境），第二个串行端口会提供给 Turbo PMAC。此端口需要用来编程和监控 PMAC 梯形 IEC-1131 PLC 程序。

辅助端口波特率

用于辅助串行端口的波特率是由变量 I53 设置。在上电复位时，Turbo PMAC 基于 I53 的设置和 I52 设置的 CPU 速度来设置活动波特率-控制寄存器，因为波特率频率是从 CPU 的工作频率分频的。出厂默认波特率为 38,400。如果你想改变波特率，则必须更改 I53 的设置，用 SAVE 指令将这个新的设置复制到非易失性存储器中，然后重置 Turbo PMAC。然后，你必须在新的波特率重新建立通讯。I53 的值和它们产生的波特率是：

I53	波特率	I53	波特率	I53	波特率	I53	波特率
0	600	4	2400	8	9600	12	38,400
1	900	5	3600	9	14,400	13	57,600
2	1200	6	4800	10	19,200	14	76,800
3	1800	7	7200	11	28,800	15	115,200

由 I53 的奇数设置产生的波特率只有在 CPU 的频率是 30 MHz（技术上来说，是 29.4912 MHz）的整数倍时才精确。这是因为波特率是由 CPU 频率除以（256*N）产生的，其中 N 是从查找表中取的一个整数。该频率对 I53 的奇数设置和频率不是 30 MHz 的倍数的 CPU 来说不是精确匹配。对于这种类型的较低的波特率，误差不显著。然而，波特率为 115,200 的串口通信是可能的只有当 CPU 是在 30MHz 的整数倍（实际上是 29.4912 MHz 的整数倍）运行时。因此，要以这样的速度进行通信，让选项 5Cx 80 MHz 的 CPU 在 60 MHz 下运行、选项 5Dx 100 MHz 的 CPU 运行在 90 MHz、选项 5Ex 160 MHz 的 CPU 运行在 150 MHz 通过将 I52 设置成一个比 CPU 能够运行的更低的值。

辅助端口解析器禁用

将 I43 设置为 1 可以关闭辅助串行端口上的自动命令解析器。I43 设置为默认值 0 时，Turbo PMAC 将试图解释任何从辅助串口进来的并作为 Turbo PMAC 部分指令的字符。然而，I43 设置为 1 时，这些字符都只是放在旋转命令队列中位于 X:$1C00--X:$1CFF，在那里一个用户的应用程序可以解释它们。

不可能通过辅助串口下载新的操作固件。

信号格式

由于串行接口在系统之间会有所不同，所以 Turbo PMAC 提供了一个简单而灵活的接口。除了信号接地线，只需四个线（八个线，如果算上 RS-422 接口的补充线的话）：数据-发送、数据-接收、清除-发送、准备-发送。Turbo PMAC 的串口只是将 DSR 和 DTR 握手线短接在一起进而为那些需要的系统提供一个自动返回信号在此选通脉冲上。

用于数据和握手的发送和接收信号，被放置在位于数据终端设备（DTE）配置的连接器上（在 Turbo PMAC-PC 上，这可以通过使用跳线 E9-E16 改变）。这允许一个直-跨连接到配置为数据通讯设备（DCE）的串口上，这是在几乎所有现代计算机中都能找到的配置。但是，如果将串行端口连接到配置为 DTE 的另一个设备，诸如手持终端/垂饰，要跨穿通过数据和握手信号。

框头连接器的引出脚设计用于大多数 Turbo PMAC 的串行端口，因此扁平-电缆到另一端 D-sub 连接器的连接将提供适当的连接，如果另一端是处在 DCE 配置中。

数据格式和握手

每个字符的串行通信数据格式为 8 位，1 个起始位，1 位停止位，无奇偶校验。(未来的固件版本中可能有一个选项来允许奇偶校验。) 不支持字符的全双工应答。

RTS/CTS 握手是默认启用的。因为 Turbo PMAC 以非常高的速度处理通信，它永远不会延迟一个字符，除非它被锁定在复位或看门狗状态。如果 I1 设置为 1 或 3，Turbo PMAC 将忽略来自主机的、在主串行端口上的 CTS 握手，但在这种情况下，确保串行端口的高优先级的监控，以确保没有任何字符丢失（标准的 Microsoft Windows 串行驱动对于这个没有足够高的优先级）。

大多数 Turbo PMAC 串行端口将 DSR 和 DTR 线短路在一起来提供应答握手在那些需要的端口上。不支持任何 XON/XOFF 软件握手。

多-分支通讯

使用 RS-422 串行端口，有可能在一个多分支电缆上与多个菊花链式的 Turbo PMAC 进行通信。多达 16 个的 Turbo PMAC 可以寻址在一根电缆上。

I-变量启用

Turbo PMAC 的变量 I0 决定了一个多分支串行电缆上的控制器的软件地址。I0 的范围为 0～15（0 到 F，十六进制）。电缆上的每个控制器必须具有一个唯一的 I0 值。每个 Turbo PMAC 上的变量 I1 必须设置为 2 或 3 以使能串行软件寻址。

串行控制器寻址

一个控制器用@n 指令寻址，其中 n 是一个十六进制数字，大小为 0～F。当@n 指令通过串行电缆发送，其 I0 值等于 n 的控制器成为定址的控制器，所有其他未被编址。只有定址的控制器将响应命令，并确认他们，驱动其输出数据和握手线（在未编址控制器上为三态，以防止信号冲突）。

如果@@指令通过串行电缆发送，电缆上的所有控制器将响应行动指令。需要

一个数据响应的指令在此模式下不被允许。I0 值为 0 的 Turbo PMAC 将驱动其输出数据和握手线（所有其他的为三态）并确认主机指令。

可选时钟共享

可通过在 RS-422 多分支电缆的附加线共享伺服和相位时钟信号。这可以防止在连续运动中由于每个时钟晶振频率容差而导致控制器之间产生长期漂移。每个 Turbo PMAC 上的跳线决定控制器是产生自己的伺服和相位时钟信号并将其输出在串行端口上，还是期望从它的串行端口输入他们。在一个多分支电缆中只有一个控制器可以输出其伺服和相位时钟。只有该控制器上的频率控制变量或跳线会影响电缆上所有控制器的伺服和相位时钟的频率。

如果伺服和相位的时钟信号不在控制器之间共享，那么这些信号不应该在控制器之间通过电缆连接。

总线通讯端口

Turbo PMAC 的每一个配置都有一个总线端口，可提供比串口更快的通信。有传统的背板总线（ISA、PCI、VME），也有较新的导线总线（USB、以太网）。

大多数这些总线端口使用 Turbo PMAC 的 CPU 上的主机端口（VME 总线使用一组邮箱寄存器代替）。他们中的许多也使用可选的双端口 RAM（DPRAM）作为备用路径与 CPU 进行通信，虽然此通信使用相同的物理路径在主计算机和 Turbo PMAC 之间。

通过一个使用 Turbo PMAC 的 CPU（见《重置 Turbo PMAC》）上的主机端口的总线通讯口来下载新的操作固件是可能的。这不包括 VME 总线和只使用 DPRAM 通讯的任何线式总线接口。

背板总线

几种类型的背板总线在不同的 Turbo PMAC 配置中都是被支持的。这些总线提供最高速度和最低延迟的通信，归于其高原始数据速率（相对于简单的串行接口），并且没有中间的处理器（和导线总线相比）。

ISA（和 PC/104）总线

ISA 总线接口标准配置在板级的带 PC 后缀的 Turbo PMAC 控制器（如 Turbo PMAC-PC、Turbo PMAC2-PC）上。PC/104 总线接口，一个堆栈版本的 ISA 总线（相当于电气的和在软件中），标准配置在 3U-格式的 UMAC Turbo CPU 板上（这款主板之前为选项 2，现在始终包含）。

ISA 总线上的基本通信穿过 Turbo PMAC 的 CPU 上的主机端口。双端口 RAM 对该总线接口是一个可选的附加（见下文）。ISA 总线的 I/O 空间中的主机端口通信的地址由一组位于 Turbo PMAC 板上的跳线或 DIP 开关选定。出厂默认设置是用于基本 I/ O 地址 528（210，十六进制），而且接口界面占用 PC I/O 空间中的连续的 16 个地址。

下表显示了跳线（Turbo PMAC 板）或 DIP 开关（Turbo PMAC2）是怎样设置 ISA 或 PC/104 接口上的基址的。一个关闭（OFF）状态的 PMAC 的跳线或者打开（OPEN）状态的 PMAC2 DIP 开关将相关的位值加到基地址上；一个接通（ON）的跳线或者闭合（CLOSED）的 DIP 开关则不加任何值到基地址值上。

地址位#	15	14	13	12	11	10	9	8	7	6	5	4
PMAC1 跳线	x	x	x	x	E91	E92	E66	E67	E68	E69	E70	E71
PMAC2 开关	12	11	10	9	8	7	6	5	4	3	2	1
位值（DEC）	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16
位值（HEX）	$8000	$4000	$2000	$1000	$800	$400	$200	$100	$80	$40	$20	$10
默认跳线	–	–	–	–	ON	ON	OFF	ON	ON	ON	ON	OFF
默认开关	关	关	关	关	关	关	开	关	关	关	关	开

注：DEC---十进制；HEX---十六进制。

PCI 总线

PCI 总线接口标配在板级的带 PCI 后缀的 Turbo PMAC 控制器上（如 Turbo PMAC-PCI、Turbo PMAC2-PCI）。UMAC-CPCI Turbo 的 CPU 电路板被设计成接收一个子插件板--在那里其实现了紧凑型 PCI 接口，一种 PCI 总线（相当于电气的和在软件中）的机架式版本，但截至写这篇文章时，CPCI 子插件板还未实现。

在 PCI 总线上的基本通信穿过 Turbo PMAC 的 CPU 上的主机端口。双端口 RAM 是一个可选的附加对于该总线接口（见下文）。在 PCI 总线的 I/O 空间中的主机端口通信的地址由主机的操作系统选定。最近的微软 Windows 操作系统做到这一点是在一个即插即用的基础上。Turbo PMAC 上的 PCI 总线接口的设计被实现用来尽可能像旧的 ISA 总线接口那样，尽量减少用户从 ISA 上改变的任何过渡的问题。除了自动地址设定，到 Turbo PMAC 上 PCI 总线的软件接口与那些用于 ISA 总线的软件接口是相同的。

VME 总线

VME 总线接口标准配置在板级的带-VME 后缀的 Turbo PMAC 控制器上（如 Turbo PMAC-VME、Turbo PMAC2-VME）。它是总线上的一个从属接口，可以与总线上的 16、24、32-位寻址一起使用。地址、总线宽度和接口的其他细节是在上电或复位时通过 Turbo PMAC 变量 I90- I99 的值设置的。出厂默认设置是 24-位寻址，而且 VME 总线上的基地址为$7FA000。

VME 总线上最基本的通信穿过了一组位于 Turbo PMAC 的 VME 接口 IC 的 16 个邮箱寄存器，其中的每一个可容纳一个字符。请注意，不像其他的总线，这种通信不经过 Turbo CPU 的主机端口，所以不可能通过 VME 总线以引导模式下载新的操作固件到 Turbo PMAC 中。对于该总线接口，双端口 RAM 是一个可选的附加（见下文），但几乎始终使用。如果 DPRAM 存在，通常比较容易通过 DPRAM 来发送文本命令，且邮箱寄存器通常不使用。

下表显示了 I90-I99 如何设置以用于 24-位和 32-位寻址（16-位几乎从来没有使用过），带或不带双端口 RAM。表中的数值用于在$abcdef00 总线上的一个邮箱基地址，以及在$abg00000 的一个 DPRAM 基地址（a 和 b 在 24 位寻址中不使用）。

I 变量	I90	I91	I92	I93	I94	I95	I96	I97	I98	I99
24-位，无 DPRAM	$29	$04	*	$cd	$ef	**	**	*	$60	$10
24-位，含 DPRAM	$29	$04	*	$cd	$ef	**	**	$0g	$E0	$90
32-位，无 DPRAM	$39	$04	$ab	$cd	$ef	**	**	*	$60	$10
32-位，含 DPRAM	$39	$04	$ab	$cd	$ef	**	**	$0g	$E0	$90
*不关心，不使用 **系统相关的设置										

泰道公司用于 PC 的软件包，如执行和安装程序以及 PComm32 通信库，不支持 VME 总线通信，因为在 Microsoft Windows 操作系统下没有标准的 VME 通信方式，甚至是对于 PC-兼容的 VME 总线电脑。

导线总线

近年来，高速导线链接已经变得有成本效益且足够普遍，能够在主机与控制器之间提供经济但高带宽的连接。泰道目前支持以下两种链接：USB 和以太网。这些链接基本上匹配背板总线速度用于大型数据集在两个方向传送，但在对于短指令的响应只有大约一半的速度，归因于在链路的两端的源于处理器添加的延迟。

USB

一些 Turbo PMAC 配置支持使用通用串行总线（USB）通信。在最初的实施中使用 12M bit/sec（位/秒）的 USB1.1 的链接；较新的实施都使用 480M bit/sec 的 USB2.0 链接。请记住，USB1.1 设备可以在 100 Mb/s 的总线中使用。

以太网

一些 Turbo PMAC 配置支持使用以太网通信。最初的实现中使用 10M 位/秒的以太网链路，有计划升级到 100M 位/秒。记得 10M 位/秒的设备可以用在 100M 位/秒的网络中。

同时支持 TCP/IP 和 UDP/IP 协议。强烈推荐简单一点的 UDP/IP 协议，因为它更有效，而且这里使用的点到点通信不需要额外的数据包信息和 TCP/IP 的握手确认。和一台运行微软 Windows 操作系统的计算机使用 TCP/IP 协议特别低效，由于对每一个指令/响应事件都有操作系统超时。

双端口 RAM

总线通信端口，不管背板还是导线，都通过一个使用双端口 RAM 中的共享内存接口来提供可选的支持通信。ASCII 文本命令可以通过双口 RAM 发出，但双口 RAM 的主要优点是，它的二进制数据结构允许直接并行访问多个数据块。

这些结构包括数据-报告缓冲区，其中 Turbo PMAC 反复更新 DPRAM 中的固定寄存器，用通常期望的位置信息，诸如电机的位置；数据采集缓冲区，它允许同步地收集数据的实时上传；后台变量数据复制缓冲区，它这允许重复复制到用户指定的 Turbo PMAC 寄存器或者从其中复制；二进制旋转运动-程序下载缓冲区用于运动程序文件以最快的可能速度下载；以及用户自定义的结构。

对于 ISA 和 VME 总线，Turbo PMAC 变量 I93 和 I94 在上电或复位时设置总线的 DPRAM 的地址。对于 PCI 总线，主机的操作系统会自动设置这个地址。对于导线总线，DPRAM 并不直接映射到主机的内存空间；软件例程，如泰道的 PComm32 库，会创建一个双口 RAM 的虚拟影象在主机的内存中，使得双口 RAM 的运程对于

应用程序来说是透明的。

一些双口 RAM（即 DPRAM）的通信结构可分别通过 I-变量启用：

- DPRAM 的 ASCII 通讯：I58 和 I56
- DPRAM 电机数据前台报告：I48 和 I47
- DPRAM 电机数据后台报告：I57 和 I50
- DPRAM 坐标系统和全局后台数据报告：I49 和 I50
- DPRAM 后台变量缓冲区：I55
- DPRAM 二进制旋转型缓冲区前台传输：I45

在某些 Turbo PMAC 系统中可以有多个 DPRAM 集成电路，特别是在 UMAC 系统中。然而，这些集成电路中只有一个可以在任何给定时间使用与任何自动的数据结构。Turbo PMAC 变量 I24 决定了哪个 DPRAM 芯片支持这些自动功能，通过指定在 Turbo PMAC 自己的地址空间中该 IC 的基地址。在 Turbo PMAC 的重新初始化时，I24 被设置为发现的第一个 DPRAM IC（具有最低的基地址的那个）的地址。I4904 显示了已被 Turbo PMAC 的 CPU 发现的 DPRAM 芯片。

与主总线通信端口相关联的 DPRAM IC 有一个$060000 的基址在 Turbo PMAC 的内存映射中。由 Turbo PMAC 的 CPU 通过 JEXP 或 UBUS 扩展端口访问的 DPRAM 芯片，例如那些用于 UMAC 的在 Acc-54E 的 USB/以太网通讯卡上的芯片，可以有基地址$06C000，$06D000，$06E000，$06F000，$074000,$0750000,$76000，或者$077000。

4-2 给 Turbo PMAC 发送指令

Turbo PMAC 基本上是一个指令驱动的装置，不像其他控制器是由寄存器驱动的。如果发出 ASCII 命令文本字符串，Turbo PMAC 就可以做事，而且一般来说，Turbo PMAC 以 ASCII 文本字符串提供信息给主机。

> 注意：
>
> 如果选项 2 的双口 RAM 是存在的，可以通过写值到 DPRAM 中的指定寄存器中来命令 Turbo PMAC 工作，而且 Turbo PMAC 可以提供信息，通过将二进制的值放在这些寄存器中，但 ASCII 命令必须被发送到 Turbo PMAC 中，导致其当收到这些值时采取适当的行动，并把这些值放在这些寄存器中。

Turbo PMAC 的指令处理

当 Turbo PMAC 通过它的一个端口接收到一个字母数字文本字符，它什么都不做，但会将字符放置在命令队列中。它需要一个控制字符（ASCII 值 1 到 31），以使其采取一些实际行动。最常用的控制字符是回车（<CR>； ASCII 值 13），它会告诉 Turbo PMAC 去解释前述的字母数字字符集合为一个命令，并采取适当的

行动。

其他控制字符会导致 Turbo PMAC 采取一个独立于之前发送字母数字字符的行动。这些控制字符可被发送在一行字母数字命令字符的中间，而不会干扰该命令的流动。Turbo PMAC 将第一时间响应控制符命令，存储该文本字符串，直到接收到<CR>字符。

指令确认

Turbo PMAC 的指令确认和数据响应的确切性质是由 I-变量 I3、I4 和 I9 控制的，其中 I3 是最重要的。如果 I3 为 1，PMAC 确认一个有效的字母数字指令，是通过发送换行（<LF>；ASCII 值 10）字符返回给主机。如果 I3 为 2 或 3，它使用<ACK>字符（ASCII 值 6）来代替。如果 I3 是 0，它不提供任何的确认字符。无论 I3 的设定是什么，Turbo PMAC 通过返回<BELL>字符（ASCII 值 7），来响应一个无效的命令。

当采用一个哑终端模式和 Turbo PMAC 交互工作时，用<LF>作为确认往往很好，因为它会自动为命令隔开空间和在终端屏幕上响应

数据响应

当接收的指令需要一个数据响应时，如果 I3 设置为 1 或 3，Turbo PMAC 将在每一行的数据响应前加一个换行字符。如果 I3 设置为 0 或 2，它不会这么做。无论 I3 的设置是什么，Turbo PMAC 会用一个回车字符终止每一行的数据响应。对于这些命令，命令确认字符--<LF>或<ACK>--是在数据响应后发送的，作为一个结束传输的字符。对于计算机解析响应，用<ACK>作为一个独特的 EOT 字符是不错的。

数据完整性

变量 I4 决定在通信中由 Turbo PMAC 执行的一些数据完整性检查，其中最重要的是逐行的校验和。《编写一个主机通讯程序》的章节详细介绍了此功能。

数据响应格式

变量 I9 控制 Turbo PMAC 如何将数据发送到主机的一些方面。它的设置确定 Turbo PMAC 是以长形式还是短形式来列表程序行返回到主机，是否用完整的命令语句报告 I-变量值和 M-变量定义，以及寻址 I 变量值的报告是以十进制还是十六进制的形式。

4-3 在线（即时）指令

发给 Turbo PMAC 的大多数命令是在线命令，也就是说，它们由 Turbo PMAC 立即执行，去引起一些动作、改变一些变量或者报告一些信息反馈给主机。命令

在完成执行后本身会被扔掉（所以不能事后列出），虽然其效果可能还留在 Turbo PMAC 中。

有些命令，如 P1=1，如果没有打开的程序缓冲区，将会立即执行；如果有一个打开的，会被存储在缓冲区中。其他命令，如 X1000 Y1000，不能是在线命令；必须有一个打开的缓冲区--哪怕是用于立即执行的一个特殊的缓冲区。如果没有打开的缓冲区，这些命令将被 Turbo PMAC 拒绝执行（如果 I6 设置为 1 或 3，报告一个 ERR005）。还有其它的命令，如 J +，只是在线命令，并不能输入到程序缓冲区中（除非以例如 **CMD"J+"**的形式，）。

在线指令的类型

有四个基本类型的在线指令：

- 端口专用指令，这只会影响同一个端口上的后续命令的动作；
- 电机专用指令，这只会影响目前正在由主机寻址的马达；
- 坐标系统专用指令，这些命令只影响当前由主机寻址的坐标系；
- 全局命令，无论任何寻址模式都会影响卡。在本手册的参考部分，每个命令都被分为这些类型的其中之一，在范围描述下。

注意：

每一个从卡内使用 COMMAND 语句发出在线指令的程序，都有自己的电机和坐标系统的寻址，独立于主机寻址的电机和坐标系统。改变一个端口的寻址模式不会影响该程序的模式，反之亦然。

端口专用指令

为了在多个通信端口之间保持真正独立的通信，对某些影响后续命令的指令有必要只影响同一端口上的命令。出于这个原因，寻址指令--**#n** 用于电机和&n 用于坐标系---以及缓冲区 OPEN 和 CLOSE 命令，只影响同一端口上的后续指令。

电机专用指令

一个电动机是通过**#n** 命令被寻址为一个端口的，其中 n 是电机的编号，范围是 1-32，可兼的。该电机保持为该端口寻址的状态，直到卡在同一个端口接收到另一个**#n**。例如，命令行**#1J+#2J-**会告诉电机 1 在正方向微动和电机 2 在负方向微动（像大多数命令一样，微动命令不会生效直到接收到回车符，所以此情况下，两个坐标轴在大致相同的时间的命令下开始作用）。

目前只有少数种类的电机专用命令。这些包括微动命令，回零点命令，开环的命令，用于电机位置、速度、跟随误差和状态的请求。

注意：

如果定址的电机位于一个运行着运动程序的坐标系中，一个在线电机操作命令，如微动、回零、开环输出，是不允许的，即使运动程序不直接指挥任何分配给电机的轴。这样的命令将是一个错误被拒绝。

坐标系统专用指令

坐标系是通过一个&n 指令被寻址用于一个端口的，其中 n 是坐标系统的编号，范围是 1 到 16。这个坐标系统一直保持为定址的状态，直到卡在同一个端口收到另一个＆n 命令。例如，命令行＆1B6R＆2B8R 告诉坐标系统 1 运行运动程序 6，而坐标系统 2 运行运动程序 8。

有多种类型的坐标系统专用命令。轴定义语句作用在定址的坐标系统上，因为电动机要匹配到一个特定坐标系统中的轴上。由于它是一个运行着运动控制程序的坐标系，所有的程序控制指令都作用在定址的坐标系中。Q-变量赋值和查询命令也是坐标系统命令，因为 Q-变量本身就属于一个坐标系。

请注意，如果多台电机被分配到一个坐标系中，那么该坐标系统的命令可以影响多台电机。例如，如果电机 4 被分配到坐标系 1，对坐标系统 1 的用来执行一个运动程序的命令可以启动电动机 4 运动。

全局指令

一些在线命令不依赖哪个电机或坐标系已经被定址。例如，命令 P1=1 设定 P1 的值为 1 而无论是什么被定址。这些全局在线命令都是缓冲区管理命令。Turbo PMAC 具有多个缓冲区，一次可以打开其中一个。当一个缓冲区打开时，命令可以被输入到缓冲区中以便以后执行。

控制字符命令（那些的 ASCII 值为 0-31D）始终是全局命令。那些不需要数据响应的控制字符指令作用在一个串行菊花链上的所有卡上。这些字符包括回车<CR>、退格<BS>和几个特殊用途的字符。这允许，例如，在一行中发送命令到卡的多个位置，并在该行的末尾的回车<CR>处让它们同时生效（＆1R＆2R<CR>使得两个坐标系 1 和 2 都运行起来）。

4-4 缓冲的（程序）指令

正如其名称所暗示的，缓冲的指令不立即被执行生效，而是会保持待以后执行。发送一个缓冲的程序命令到 Turbo PMAC 只是导致该命令被加载到打开的程序缓冲区，该命令实际上不会被执行，直到该程序运行。

Turbo PMAC 有许多程序缓冲区---224 个常规运动程序缓冲区、16 个旋转运动程序缓冲区（一个坐标系一个）、16 个前向运动子程序缓冲区（每个坐标系一个）、16 个逆向运动学子程序缓冲区（每个坐标系一个）和 32 个未编译的 PLC 程序缓冲区。（用于编译的或汇编的程序---编译的 PLC、用户自定义的伺服算法和用户自定义的相位算法---的缓冲区不接受文本命令，相反，他们接受 DSP 的机器代码。）

在命令输入到一个缓冲区之前，该缓冲区必须打开（如 OPEN PROG3，OPEN PLC 7，&1 OPEN FORWARD）。需要注意的是缓冲指令只能通过发送 OPEN 指令的同

一个端口，来输入到缓冲区中。一次只有一个端口可以有一个打开的程序缓冲区。

在打开的缓冲区中，每个程序指令被添加到指令列表的末尾；如果你想替换现有的缓冲区，在进入新的之前，打开缓冲区后立即用 CLEAR 命令清除现有内容。完成输入程序语句后，使用 CLOSE 命令关闭打开的缓冲区。

一个坐标系的旋转运动程序缓冲区是一个特殊的程序缓冲，可以在执行运动程序的同时打开，用于从主机输入程序指令。如果一个打开的旋转程序缓冲区正在执行，但已经执行完发送给它的每一个命令，那么它几乎会立刻执行发送给它的下一个缓冲的程序命令。

请注意，如果在其被发送的端口上没有打开的缓冲区，某些命令会被当作在线指令，或者如果在这个端口上有打开的缓冲区，会被当作缓冲的程序指令。例如，如果端口上没有打开的缓冲区，命令 P1=1 将会被立即执行；但是如果端口上有打开的缓冲区，它会进入到一个程序缓冲区作为该程序的一部分并稍后执行。如果端口上有打开的或者没有打开的缓冲区，这些命令中的一些会有完全不同的含义。如果端口上没有打开的缓冲区，命令 M10 是查询 M10 变量的值；如果端口上有一个打开的运动程序缓冲区，它就是一个对 M 代码子程序的调用。

第五章 设置反馈和主位置传感器

Turbo PMAC 系统可以接口到各种位置传感器用于反馈和主控，无论是在主控板上还是通过各种辅助板。本部分总结了基本的硬件和软件设置问题；更多详细信息可以在相应的硬件参考手册和 Turbo PMAC 软件参考手册中找到。

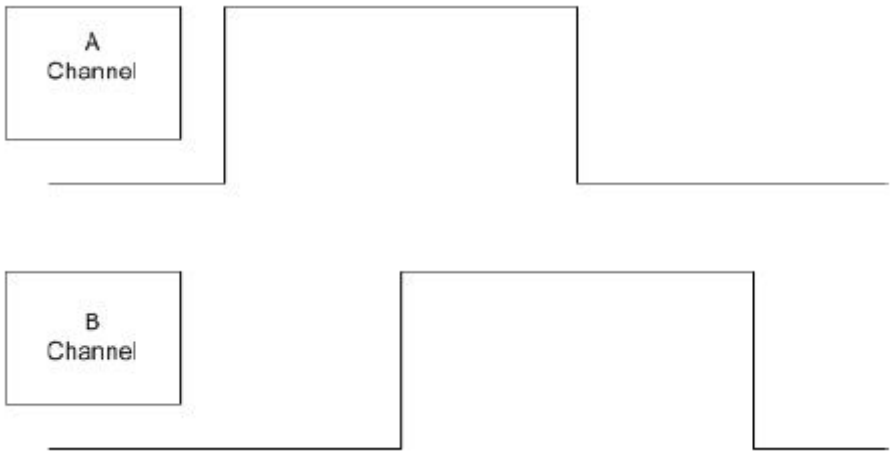
需要注意的是反馈或主传感器的初始设置独立于任何电机或坐标系。一个电动机或坐标系统可以使用从初始硬件和/或软件对位置信号的处理所得到的数值，但是这不是必需的。另外，用户程序或命令可直接访问这些位置值，而根本无需使用他们的电机或坐标系自动功能。

5-1 设置正交编码器

数字正交编码器是 Turbo PMAC 中最常用的位置传感器。这些编码器的接口电路标配在板级的 Turbo PMAC 控制器、UMAC 轴接口板以及 QMAC 控制箱上。

信号格式

正交编码器提供了两个数字信号，它们是编码器的位置的函数，每个名义上有 50%的占空比，而且名义上有四分之一周期的间隔。这种格式在每个信号周期或每个编码器线提供四个不同的状态。这两个信号的相位差允许译码电子设备辨别行进的方向，使用一路信号是不可能做到的。



通常情况下，这些信号是在 5V 的 TTL/ CMOS 电平，无论是单端或差分。输入电路是由控制器的主 5V 电源供电，但是他们可以接受每个差分对的信号之间高达+/-12V 的电压，以及信号和 GND 电压基准之间+/-12V 的电压。

差分的编码器信号可以通过提供共模噪声抑制来增强抗干扰能力。现代设计标准几乎强制其用于工业系统，特别是存在 PWM 功率放大器时，它会产生大量的电磁干扰。

硬件设置

本节笼统介绍 Turbo PMAC 编码器的硬件接口。请查阅硬件参考手册中有关您的特定配置的详细信息。

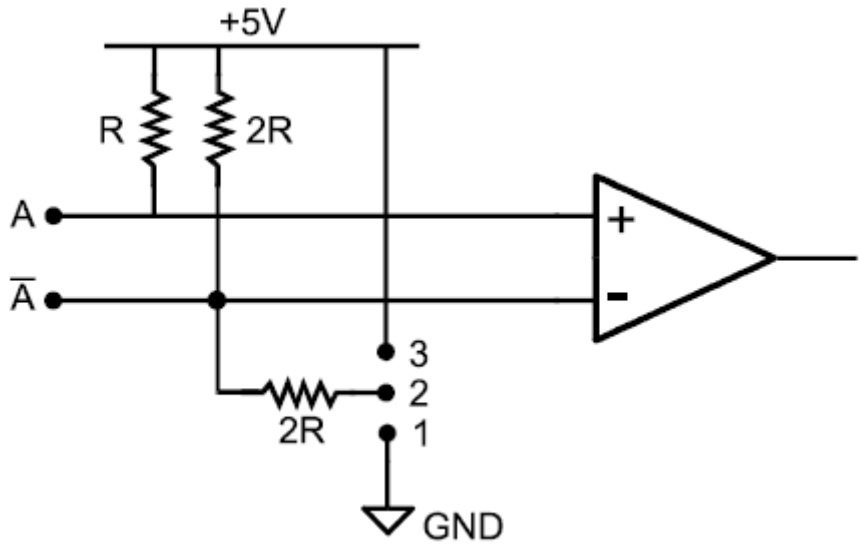
Turbo PMAC 的编码器接口电路采用差分线路接收器，但在出厂时已配置为接受单端或差分编码器。在此配置中，主（+）线拉至 5V，而互补（-）线用电压分压器连接到 2.5V。用一个单端编码器，每个通道的单个信号线会和该基准电压比较，在它在 0 和 5V 之间变化时。

注意：

当使用单端 TTL 电平数字编码器时，互补线路输入应悬空，不接地或接高；这对于 PMAC 的差分线路接收机的正常工作是必需的。

将互补线也拉到 5V 是可能的。在大多数的 Turbo PMAC 板上，这是通过改变编码器通道的一个 3-点跳线的设置来完成。在 Turbo PMAC2 板和系统上（包括 UMAC），这是通过由颠倒其插座的 SIP 电阻包来完成。

PMAC 编码器输入电路



- 连接端子 1 和 2，将差分线调压到+2.5 V
- 连接引脚 2 和 3，将差分线调压至+5 V（PMAC2 上可逆的插座式 SIP）
- 没有连接时，调压到+2.5 V
- 调压到+2.5 V，用于单端编码器
- 不要在意差分线驱动编码器
- 调压+5 V，用于互补集电极开路编码器（过时）
- 调压+5 V，支持编码器电路丢失的外部异或（XOR）

编码器丢失检测电路

拉高互补线的主要原因是为了启用编码器丢失检测电路，许多的 Turbo PMAC 和接口板都有它的存在。该电路采用异或（XOR）逻辑，要求每对信号是相反的逻辑状态来考虑编码器的存在。如果编码器出现故障或电缆断开，该对的输入均上拉为逻辑 1，电路指示编码器丢失。检查相应的硬件参考手册，看看你的设备有无编码器丢失检测以及它是如何实现的。

终端电阻

当通过一个较长的电缆（10 米以上）上驱动编码器的信号，在主线和互补线之间，可能需要添加终端电阻器以减少传递的振荡。 PMAC 为电阻包提供了插座用于这一目的。终端电阻的最佳值是与系统相关的；理想地匹配电缆的特性阻抗，但 330 欧姆是一个很好的起点。

电源和隔离

在一个 Turbo PMAC 的基本配置中，编码器电路是不和 Turbo PMAC 的数字电路隔离的，而且信号被参考到 Turbo PMAC 的数字公共电平 GND。典型地，在这种情况下，编码器从 PMAC 的+5 V 电源线供电以及一个返回在 GND 上。衡量 Turbo PMAC 的电源大小时必须考虑总的编码器电流消耗。

另外，对于非隔离的信号连接到 Turbo PMAC，编码器也可以使用单独的电源。在这种情况下，电源的返回应该被连接到 Turbo PMAC 的数字公共 GND 上，给信号一个公共基准。独立电源的+5 V 线不要绑在一起，因为它们会相互干扰来控制精确的电压水平。

隔离的编码器信号

在一些系统中，最好将编码器电路与 PMAC 的数字电路光学隔离起来。在编码器到控制器的距离较长（> 10M 或 30 英尺）的系统中和/或具有非常高强度的电磁噪声的系统中，这是常见的做法。使用 Acc-8D Opt 6 4-通道编码器隔离板可以实现隔离。有了一个隔离的编码器，编码器需要一个单独的电源，用于保持隔离，并且电源的返回不能连接到数字公共 GND，否则隔离会不起作用。

模拟的编码器信号

必须特别考虑那些系统，在无电刷电机放大器中由旋变-数字转换器提供给他们一个模拟的编码器信号。在这些系统中，编码器信号被引用到放大器的信号返回中，而反过来它又连接到 Turbo PMAC 的模拟公共 AGND。在这些情况下，最好的设置是用 ACC-8D Opt6 隔离板或类似的模块，将模拟的编码器信号和 PMAC 数字电路隔离起来。这将使得 Turbo PMAC 数字电路和放大器之间完全隔离。

如果模拟编码器信号的隔离是不可行的，Turbo PMAC 的数字电路和放大器信号电路（包括的 Turbo PMAC 的模拟量电路）必须很好地连接在一起，以提供一个共同的参考电压。要做到这一点，需要通过将跳线放在 Turbo PMAC 或接口板上（E-点 E85、E87、E88，在许多板子上），将 Turbo PMAC 数字和模拟电路连接在一起，因此模拟信号电路也这么做。必须避免的是，让模拟编码器电缆提供电路之间的唯一连接。这可能会导致从坏的参考中丢失信号，或甚至从接地回路中损坏部件。

接线技术

编码器的接线有几个重要的技术对噪音缓解很重要。首先，编码器电缆应尽所有可能与电机电源线保持物理分离。其次，这两种电缆应被屏蔽，电机电缆防止噪音出去，编码器电缆防止噪声进入。这些屏蔽只应在向内的一端接地，也就是说，连接到一个本身接地的设备。

第三个重要的消减噪音的技术是扭动配对的导线互相缠绕。有了这些双绞线，进入的噪音会在双绞线的另一半中趋于抵消自身。

Turbo PMAC 硬件控制参数设置

Turbo PMAC 的 ASIC 都默认设置为，接受正交反馈，但有些设置可能需要进行调整，以优化操作。

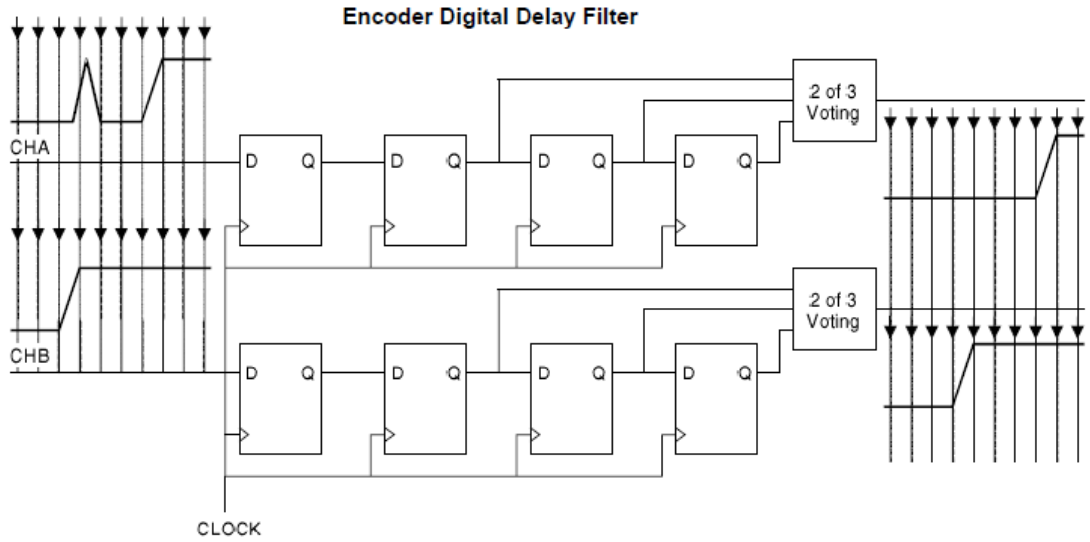
编码器的采样时钟频率：E34-E38，I7m03，I6803，MI903，MI907

通过差分线路接收机的前端处理后，正交编码器的输入被 Turbo PMAC 伺服 IC 或 MACRO IC 中的数字逻辑进行采样，采样速率由 SCLK 编码器的采样时钟”的频率（可由用户设置）决定。SCLK 频率越高，所允许的最大计数率越高；SCLK 频率越低，数字延迟噪声滤波器越有效。

每个编码器的输入通道有一个数字延迟滤波器，在每一线上包括三个级联的 D-触发器，关于触发器的输出是一个最好的 2/3 的投票表决方案。该触发器由 SCLK 信号计时。该滤波器不会通过一个只持续一个 SCLK 周期的状态变化；这小段的任何改变应该是一个噪声尖峰。在这一过程中，过滤器将实际过渡延迟了两个 SCLK 周期---几乎所有的系统中的一个微不足道的延迟。

如果 A 和 B 两个通道都在解码电路（后置滤波器）在同一个 SCLK 周期改变状态，将会对计数器值产生一个不可恢复的错误。ASIC 硬件指出这个问题通过在通道状态字中设置和锁定“编码器计数错误”位。这个问题也可以被检测到，通过捕获每转一圈在索引脉冲的计数值，看正确的计数是否已经过去。

编码器数字延迟滤波器



SLCK 频率必须至少比最大编码周期（线）频率输入高 4 倍，不管所使用的正交解码方法（最常见的 4 倍解码，SCLK 频率必须至少和计数率一样高）。在实际使用中，由于在输入信号中的缺陷，应该使用 20-25%的安全余量。

对于几乎所有应用程序，9.83 MHz 的默认 SCLK 频率是可以接受的。它可以接受高达约 2MHz 的（8 MHz 的计数率）编码器信号的周期频率---有安全余量---并仍然提供合适的数字滤波。该频率可以由两个因素改变，在大多数设计中最多

19.66 或 39.32MHz，或在大多数设计中下降到低至 1.25MHz 或 306kHz。在某些设计中，可以被提供外部 SCLK 信号。如果需要非常高的编码器计数率，SCLK 的频率可能必须要上调；如果需要更好的过滤以防止错误的计数，SCLK 的频率可能要降低。

Turbo PMAC 伺服 IC SCLK 频率控制

在一个 Turbo PMAC 板上，或在配有 PMAC 型伺服集成电路的 Acc-24x 上（Acc-24P 或 24V），SCLK 频率是通过跳线 E34-E38 的配置设置的。在任何给定的配置中这些跳线中只有一个可以是接通的。对于板上的所有伺服集成电路和通道，该 SCLK 频率是共同的。

Turbo PMAC2 伺服 IC SCLK 频率控制

在一个 Turbo PMAC2 板上，或在配有 PMAC2 式伺服 IC 或 MACRO IC 的附件板上（如 Acc-24P2，24E2，24E2A，24E2S，24C2，24C2A，5E），SCLK 频率是由伺服 IC 的一个 I-变量设置的。此变量为 IC 的所有通道设置频率，但每个芯片都有一个独立的设置。参考软件参考手册中对变量的描述，获得更详细的信息。

对于一个 PMAC2 式伺服 IC m，IC 变量 I7m03 决定 SCLK 频率。这个变量也为 IC 决定了其他三个时钟频率。如果 IC 是在一个 MACRO 站上，MI903 或 MI907 确定 SCLK 频率。

对于一个 Turbo PMAC2 板或 UMAC Acc-5E 板（用于手轮编码器）上的 MACRO 集成电路，I6803 以相同的方式对 IC 上的两个通道决定 SCLK 的频率。如果 IC 是在 MACRO 站上以及个 Acc-1E stack 2-轴板，MI993 决定 SCLK 频率。

所有这些变量以相同的方式工作。其他三个时钟频率每个都几乎从来没有改变过，所以下面的表用于设置 SCLK 频率，同时其他剩下的保持为默认频率：

变量值	SCLK 频率	变量值	SCLK 频率
2256	39.32 MHz	2260	2.46MHz
2257	19.66 MHz	2261	1.23MHz
2258*	9.83 MHz*	2262	612MHz
2259	4.92 MHz	2263	306MHz
*默认			

编码器解码控制：I7mn0，I68n0，MI910

编码器信号的解码，无论是分辨率还是方向，都是由一个通道专用的 I-变量决定的。对于 PMAC 型和 PMAC2 型的伺服集成电路，该变量是 I7mn0（用于伺服 IC m 通道 n）。对于一个 Turbo PMAC 上的 MACRO IC 0 通道 n*（一个“手轮”端口编码器），该变量是 I68n0。对于一个 MACRO 站上的编码器通道，该变量是节点专用的变量 MI910。

这个变量被设置为“4 倍”解码，每个信号周期得到 4 个计数，每个信号边沿一个。这就要求变量值为 3 或 7。这两个值之间的区别是方向感---哪个运动方向使计数器向上计数。记住，对于一个反馈编码器，该编码器的方向感应该匹配伺服环输出的方向感---一个正的伺服输出必须使计数器在正方向计数---否则当伺服回路关闭，会发生危险的失控状态。

转换表处理设置---Turbo PMAC 接口

数字正交编码器的处理是在转换表中以“1/T 扩展”方法（方法数字$0），采用与计数器相关联的定时器来计算分数计数信息，可提高运动平滑度。指定的源地址是通道的基地址的（例如$78200 用于伺服 IC 0 通道 1）；Turbo PMAC 将使用该通道的几个寄存器去自动组装增强的位置信息。

也可以在表格处理中禁用 1/T 扩展（方法位数$C）。关于设置编码器转换表来处理正交编码器的详细信息，请参阅《设置编码器转换表》和软件参考手册中的变量 I8000 -I8191 的规范的部分。

转换表处理设置---MACRO 站接口

如果正交编码器被连接到一个远程 MACRO 站，转换表的处理（通常为 1/T 扩展）在 MACRO 站中完成，而且得到的增强型位置信息被回传到 Turbo PMAC，在那里它被 Turbo PMAC 的编码器转换表处理作为未移位的并行数据（通常方法数字$2，模式开关位=1），因为它已经有分数计数信息，在被伺服使用之前。关于设置编码器转换表来处理正交编码器的详细信息，请参阅 MACRO 站手册，Turbo PMAC 用户手册中设置编码器转换表的章节，以及软件参考手册中关于变量 I8000 -I8191 的规范。

缩放反馈单位

您在伺服 IC 或 MACROIC 选择的解码方案决定一个计数是什么。例如，如果选择 4 倍解码，计数被定义为 1/4 的编码器线。如果编码器的每转有 2000 线，一个计数被定义为一转（圈）的 1/8000。所有随后的位置、速度和加速度的单位是基于一个计数的这个定义。

5-2 设置数字霍尔传感器

三相数字式霍尔效应位置传感器（或他们的同类）很流行用于换相反馈。它们也可以配用 Turbo PMAC 作为低分辨率的位置/速度传感器。作为换相位置传感器，通常他们只是被 Turbo PMAC 使用用于近似的上电相位位置；通常情况下，持续的相位位置是来自于同一个的用于伺服反馈的高分辨率编码器。（许多控制器和放大器使用这些霍尔传感器作为他们唯一的换相位置反馈，起始的和持续的，但那是一个低性能的技术。）

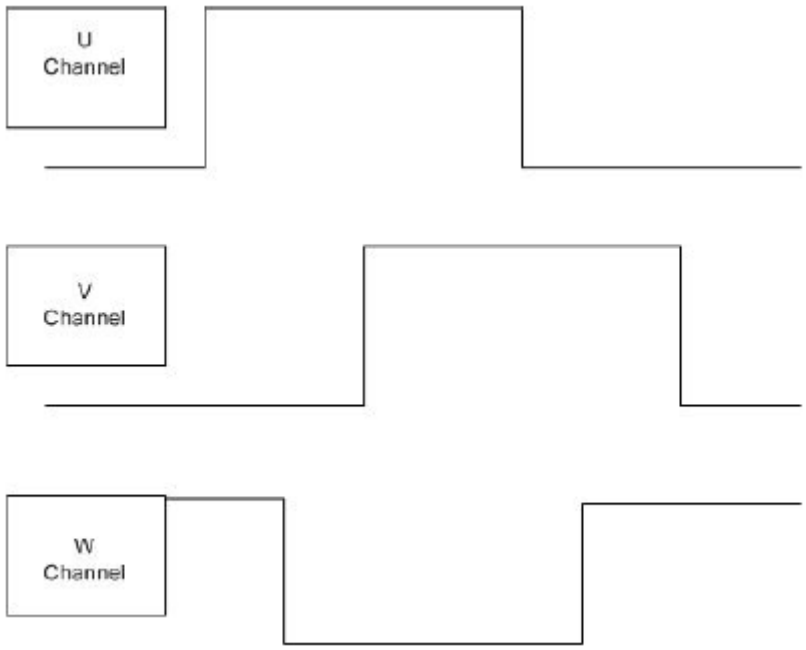
许多光学编码器有霍尔轨迹。这些换相轨迹提供的信号输出等同于磁性霍尔换向传感器的，但是用光学手段来创造信号。

注意：

这些数字式霍尔效应位置传感器不应该与模拟式霍尔效应电流传感器混淆，后者用在许多放大器中为电流环提供电流反馈数据。

信号格式

数字霍尔传感器提供三个数字信号，他们是电动机的位置的函数，每个名义上有 50%的占空比，并且名义上有三分之一周期的间隔。（这种格式通常被称为 120 度的间隔。Turbo PMAC 没有自动硬件或软件功能工作在 60° 间隔下）。这种格式在每个信号周期提供六个不同的状态。通常，该信号组的一个周期对应电动机的一个电子周期或极对。这些传感器，则可以提供的绝对的（如果分辨率低）信息，是关于电动机在其换向周期的位置，并消除了进行上电相位搜索操作的需要。



硬件设置

如果只是用于上电换向位置反馈，通常霍尔传感器被连接到 PMAC2 型的接口通道的 U，V 和 W 的辅助标志。这些都是单端的 5V 数字输入，在所有现有的硬件实现上。他们不是光学隔离的输入；如果需要隔离传感器，这必须在外部完成。

注意：

在磁性霍尔传感器的情况下，反馈信号经常返回到控制器中，在与电动机电源引线的同一电缆里。在这种情况下，必须考虑电机电源短路的可能性；安全性方面的考虑和工业设计的代码会不允许将信号直接连接到 Turbo PMAC TTL 输入中而不加隔离。

如果用于伺服位置和速度反馈（仅仅 PMAC2 式伺服芯片），3 个霍尔传感器要连接到 A、B 和 C 编码器输入，这样信号的边沿可被计算在内。正如正交编码器，这些输入可以是单端或差分的。他们不是光隔离的输入；如果传感器需要隔离，这必须在外部完成。有可能有这样的应用，在那里信号既连接到 U、V 和 W 输入（用于上电换向位置），又连接到 A、B 和 C 的输入（用于伺服反馈）。

Turbo PMAC 硬件-控制参数设置

霍尔传感器多路分配器控制：I7mn5，I68n5，MI915

如果霍尔传感器连接到通道的 U，V 和 W 输入端，你必须确保该通道的霍尔传感器多路分配器控制变量的 1 位设置为默认值 0。如果该位被设置为 1，通道的状态寄存器的 U、V 和 W 位的信息，会被从编码器输入的 C-通道中取消多路复用，基于 A 和 B 输入的状态，正如安川增量式编码器那样。对于伺服集成电路，这个控制变量是 I7mn5， MACRO IC 的是 I68n5，以及在 MACRO 站的节点专用变量 MI915。

编码器解码控制：I7mn0，I68n0，MI910

如果霍尔传感器被连接到编码器的输入端 A，B 和 C，该信号的解码是通过一个通道专用的 I-变量决定的。对于（PMAC2 式）伺服集成电路，它是 I7mn0（用于伺服 IC m 通道 n）。对于 Turbo PMAC 上的 MACRO IC0 通道 n*（一个手轮端口编码器），它是 I68n0。对于一个 MACRO 站上的编码器通道，它是节点专用的变量 MI910。

对于 3 相的霍尔传感器，解码必须设置为 6-倍解码，每个信号周期得到 6 个计数，每个信号边沿一个。这就要求变量值为 11 或 15。这两个值之间的差别是方向感---运动的哪个方向使计数器递增（向上）计数。请记住，对于一个反馈传感器，该传感器的方向感必须匹配伺服回路输出的方向感---一个正伺服输出必须使计数器在正方向计数---否则当伺服回路关闭，会发生危险的失控状态。

上电相位使用

最常见的使用这些霍尔传感器和 Turbo PMAC2 是为换相算法建立一个近似的绝对位置。在这种情况下，电机变量 Ixx81 包含了这些 U、V 和 W 输入的标志寄存器的地址。Ixx91 必须被设置为一个值，大小范围从$800000 至 FF0000；实际的值依赖于相对于电机换向周期的传感器的方向和定相。

转换表处理设置-Turbo PMAC 接口

如果霍尔传感器被连接到 Turbo PMAC 的编码器输入并因而连接到编码器计数器，关于正交编码器，计数信息应该在转换表中用 1/T 扩展方法（方法数字$0）进行处理。该方法的子计数估计在这里特别重要是因为该传感器的分辨率低。有关设置编码器转换表来处理用作编码器的霍尔传感器的详细信息，请参阅本手册中设置编码器转换表部分和软件参考手册中变量 I8000-I8191 的规范。

注意

如果霍尔传感器用于持续的换向反馈，那么最好使用来自转换表的扩展结果数据，而不是通常被用于编码器的原始计数器信息。这将提供更平滑的换向。在这种情况下，每一个换向周期将有 6 *32 =192 个计数，而不仅仅是 6 个。

转换表处理设置 - MACRO 站接口

如果霍尔传感器被连接到一个 MACRO 站的编码器输入端并因此连接到编码器计数器，计数信息应该用 1/T 扩展方法（方法数字$0）在站的转换表中处理，只是关于一个正交编码器。该方法的子计数估计在这里特别重要是因为该传感器的分辨率低。所得到的增强型位置信息被回传到 Turbo PMAC，在那里它被 Turbo PMAC 的编码器转换表处理为未移位的并行数据（通常的方法数字$2，模式开关位=1），因为它已经有分数计数信息，在 Turbo PMAC 的伺服和/或换相算法使用之前。

关于设置编码器转换表来处理用作编码器的霍尔传感器的详细信息，咨询 MACRO 站手册，本手册中设置编码器转换表的部分和软件参考手册中变量 I8000-I8191 的规范。

缩放反馈单位

为了电机的伺服回路和相关功能的目的，霍尔传感器将在每个电子周期提供六个计数；因此一个 4-极的电机在每一转将有 12 个计数。如果使用 1/T 扩展计数数据用于换向，换向周期的大小（Ixx71/Ixx70）将是 192 个计数。

5-3 设置正弦编码器

Turbo PMAC 系统可以接受来自正弦曲线（正弦/余弦）编码器的信号并产生分辨率非常高的位置数据。这是通过特殊的附件板和专用固件算法完成的。目前有两类插补器：那些低分辨率的每线产生 128 个或 256 个状态，那些高分辨率的

每线产生 4096 个状态。

> 注意
>
> 许多 Turbo PMAC 用户利用正弦编码器用于位置反馈，但工作在外部插补器模块的插值函数，其产生高频合成数字正交信号用于连接到 Turbo PMAC 的控制器。在这种情况下，Turbo PMAC 对待反馈就如同使用一个真正的数字正交编码器。

硬件设置

编码器的正弦和余弦通道可以作为单端或差分输入（强烈推荐差分）连接到插补器附件，根据对插补器的说明。可选地，索引通道也可以被连接。详细信息请参考特定附件的硬件参考手册。

如果连接一个低分辨率插补器到 Turbo PMAC 或配有 PMAC 型伺服集成电路的配件中，那么分数-计数数据会被带入到伺服集成电路中，在 TTL 电平，在那些编号比编码器信号本身编号高 1 的通道的标志信号上。通常，这些信号是 12-24V 隔离的标志以使用插补器，隔离器必须被移除并用导电分流器取代。

Turbo PMAC 硬件控制参数设置

编码器的采样时钟频率：E34-E38，I7m03，MI903，MI907

对于低分辨率的插器，Turbo PMAC 的 SCLK 编码器采样时钟驱动插补器上的模拟变换，如同在伺服 IC 中的编码器的功能。这个时钟必须被设置为 2.46MHz 的频率。如果插补器连接到 Turbo PMAC 或配有 PMAC 型伺服 IC 的附件，该频率通过 PMAC 或附件上的跳线设置：E-点 E34-E38， E36 上应该只有一个跳线。

如果低分辨率的插补器连接到 Turbo PMAC2 或有 PMAC2 式伺服集成电路的附件中，伺服 IC m 的变量 I7m03 设置这个频率。如果它被连接到一个 MACRO 站，那么站变量 MI903 或 MI907 设置这个频率。这些变量也设置其他的时钟频率，但如果其他频率保持在默认状态（它们通常是这样），将 SCLK 频率设置为 2.46 MHz 只需要将变量值从默认的 2258 改变为 2260。

对于一个 Acc-51X 高分辨率的插补器，SCLK 信号只驱动插补器自身伺服 IC 上的合成正交的采样。9.83 MHz 的默认频率几乎总是很好。在 Acc-51P 上，频率永久设置为这个值。在与 PMAC2 式伺服集成电路（例如 Acc-51E，Acc-51C）的高分辨率插补器上，如果插补器直接连接到一个 Turbo PMAC，IC 变量 I7m03 会设定这个频率，或者如果插补器被安装在一个 MACRO 站中，MACRO 站变量 MI903 或 MI907 设置这个频率。

编码器解码控制：I7mn0，MI910

这两种类型的插补器都从正弦/余弦输入中创建一个数字正交信号。这种数字信号进入编码器解码和计数电路来产生全-计数数据。要正确匹配全-计数和分数计数分辨率，解码-控制变量必须设置为 4 -倍正交解码。在 Turbo PMAC 系统

中这个变量是 I7mn0 用于伺服 IC m 通道 n。在 MACRO 站中，则是节点专用的变量 MI910。

此外，全-计数数据的方向感必须匹配分数-计数数据的方向感。对于低分辨率的插补器，这需要解码变量设置为 7。如果这个值不能产生所需要的方向感，那么更改编码器到附件的接线（交换正弦和余弦信号，或一个通道的正负线）。

对于高分辨率插补器，转换表将在上电或复位时检查编码器解码的方向感，并相应地调整分数数据的方向感。这允许通过将解码变量设置为 3 或 7，设定所需的方向感，但如果这个变量设置被改变，在分数方向感匹配之前，保存该设置并重置卡片。

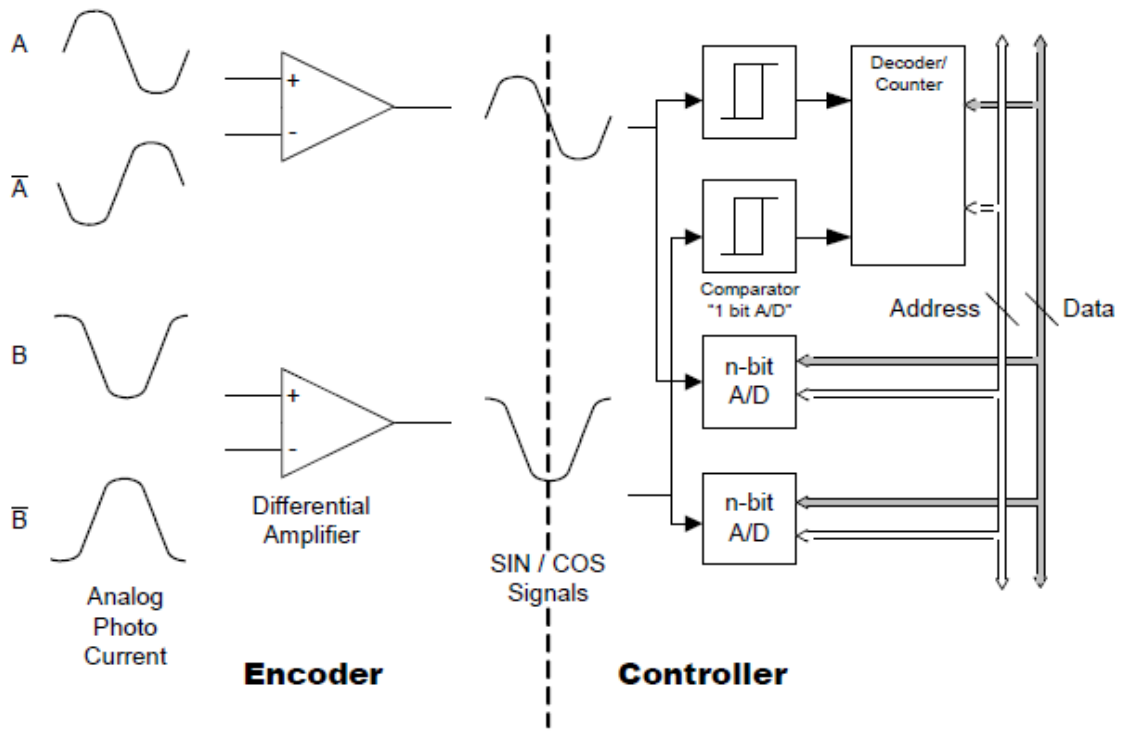
编码器的滤波器旁路：I7mn1（PMAC 式集成电路）

如果低分辨率的插补器连接到 PMAC 式伺服集成电路，编码器的数字延迟滤波器必须被旁路为了编码器的信号的两个通道，还有标记的通道，是为了保持全-计数和分数计数的数据的正确同步。同样的道理，如果使用 Acc-51P 高分辨率插补器和它自己的 PMAC 式伺服芯片在板上，也是这么做。延迟滤波器的旁路被选定是通过将通道变量 I7mn1 设置为 1。

转换表处理设置 - Turbo PMAC 接口

正弦编码器，接线到高分辨率（x4096）Acc-51x 插补器连接到 Turbo PMAC，是在 Turbo PMAC 的转换表中由一个高分辨率插补器入口（方法数字$ F）处理的，其中分数-计数数据是从正弦和余弦 A/D 转换器的读数中进行数学计算的。结果的最低有效位（LSB）表示一个编码器线的 1/4096。

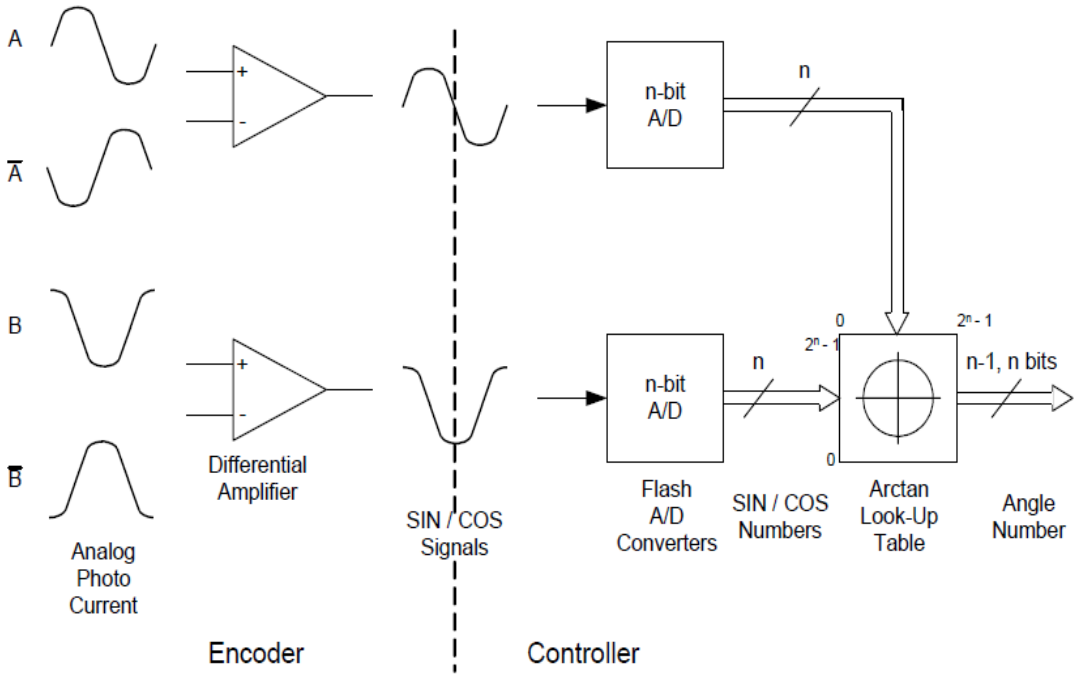
编码器插补：高分辨率的模拟“SIN / COS”编码器



正弦编码器，接线到一个低分辨率（x128/x256）的插补器连接到 Turbo PMAC，是在 Turbo PMAC 转换表中由一个并行扩展的增量编码器入口（方法数字$C）处理的，其中分数-计数数据已被外部计算在插补器的查找表中，并且只是通过转换表附加到全-计数数据中。结果的 LSB（最低有效位）代表一个编码器线的 1/128 或 1/256，这取决于插补器的分辨率跳线的设置。

关于设置编码器转换表来处理正弦编码器的详细信息，请参阅本手册中设置编码器转换表的章节和软件参考手册中关于变量 I8000-I8191 的说明。

编码器插补：模拟正弦/余弦编码器



转换处理设置---MACRO 站接口

正弦编码器，接线到一个高分辨率（x4096）Acc-51x 插补器并连接到 MACRO 站的 CPU，是在 MACRO 站 CPU 的转换表中由一个高分辨率插补器入口（方法数字$F）处理的，其中分数-计数数据是从正弦和余弦的 A/D 转换器的读数进行数学上的计算的。结果的最低有效位（LSB）表示一个编码器线的 1/4096。然后这个数据被通过 MACRO 环传送到 Turbo PMAC，在那里它被处理作为不移位的并行数据（方法数字$2）。

正弦编码器，连接到一个“低分辨率（x128/x256）插补器并连接到 MACRO 站 CPU，是在 MACRO 站 CPU 的转换表中由增量式编码器入口（方法数字$C）的并行扩展处理的，其中分数-计数数据已在插补器的查找表中被从外部计算，并且只是通过转换表附加到全-计数数据中。结果的 LSB（最低有效位）代表编码器线的 1/128 或 1/256，这取决于对插补器分辨率跳线的设置。然后这个数据通过 MACRO 环被传送到 Turbo PMAC 中，在那里它被处理为不移位的并行数据（方法数字$2）。

关于设置编码器转换表来处理正弦编码器的详细信息，请参阅 MACRO 站手册

和本手册中的设置编码器转换表的章节以及软件参考手册中关于变量 I8000-I8191 的说明。

缩放反馈单位

Turbo PMAC 会自动考虑它和 Ixx03、Ixx04、Ixx05 为伺服读取的数据，单位是以一个计数的 1/32（即有 5 位的小数-计数分辨率）。在转换表中处理来自一个正弦编码器插补器中的数据，必须非常小心明白一个计数是什么。

高分辨率（x4096）插补器

对于高分辨率插补器，插补算法产生数据而且 LSB（最低有效位）代表一个编码器线的 1/4096。由于 Turbo PMAC 电机软件考虑这些数据是以一个计数的 1/32 为单位，这意味着该软件认为每个编码器线有 128 个计数。其他电机和轴的位置、速度和加速度的值是基于这个关于一个计数的定义，称作一个软件计数。

请注意，伺服集成电路的高分辨率插补器的硬件计数器是在每个编码器线只包含 4 个计数。如果将编码器用于基于-Turbo PMAC 的换向，使用硬件计数器寄存器作为换向位置源，而不是插值结果，而且这个硬件计数分辨率应该是已知的。

低分辨率（x128/x256）插补器

对于低分辨率插补器，插补算法产生数据而且 LSB（最低有效位）代表一个编码器线的 1/128 或 1/256，并且通道的硬件计数器有数据而且 LSB（最低有效位）代表一个编码器线的 1/4 或 1/8，这取决于插补器分辨率跳线的设置。由于 Turbo PMAC 电机软件认为这些数据是以一个计数的 1/32 为单位，所以该软件认为每个编码器线会有四个或八个计数，匹配编码器计数器的分辨率。然而，相对于 1/T 基于定时器的子计数的估计，这种方法产生真正的分数-计数位置数据（即命令并保持最终位置到分辨率小于一个计数）。

5-4 设置旋转变压器

Turbo PMAC 系统可以通过一个 Acc-8D 选项 7 旋变-数字（R/D）转换器板直接接口到旋转变压器。此板可为多达四个旋转变压器器创建 AC 激励信号，接受来自这些旋转变压器调制的正弦和余弦信号，解调信号并从得到的信息推导出旋转变压器的位置，如果需要，以一个绝对意义的。

注意

许多 Turbo PMAC 用户会利用旋转变压器用于位置反馈，但旋变-数字转换的执行是在伺服放大器中，它使用所得到的位置信息用于它的换向功能。这种类型的放大器产生一个合成的数字正交信号用于控制器，其连接到 Turbo PMAC。在这种情况下，Turbo PMAC 对待反馈就如同使用一个真正的数字正交编码器。

硬件设置

硬件设置的细节都包含在 R/D 转换器板的硬件参考手册中。根本的是，R/D 板连接三个差分模拟信号对到每个旋转变压器中：一个单激励信号对，两个反馈信号对。它有两个不同的数字连接到 Turbo PMAC：一个用于串行的绝对位置到 JTHW 复用器端口（常见于电路板上所有通道，可选用），一个与合成数字正交用于每个通道，连接到一个正常的正交编码器接口。

Turbo PMAC 硬件控制参数设置

编码器解码控制：I7mn0，I68n0，MI910

要匹配从合成正交通过复用器端口读取的绝对位置所产生的持续的位置信息的分辨率和方向感，通道的编码器解码变量设置为 7（4-倍解码，逆时针）。如果不使用绝对位置，根据需要设置解码的分辨率和方向。对于 PMAC 型和 PMAC2 式二者的伺服集成电路，该变量是 I7mn0（伺服 IC m 通道 n）。对于 Turbo PMAC 上 的 MACRO 集成电路 0 通道 n*(一个手轮端口编码器)，它是 I68n0。对于 MACRO 站的编码器通道，它是节点特定变量 MI910。

转换表处理设置 --Turbo PMAC 接口

通常，数字正交信号，即使是合成的，是被处理在转换表中用 1/ T 的扩展方法（方法数字$0），它采用与计数器相关联的定时器来计算可提高运动的平滑度的分数计数信息。指定的源地址是通道的基地址的源地址（例如$78200 用于伺服 IC0 通道 1）；Turbo PMAC 将使用该通道的几个寄存器自动装配增强型位置信息。

关于设置编码器转换表来处理合成的正交编码器信号的详细信息，请参阅本手册中设置编码器转换表的章节以及软件参考手册中关于变量 I8000-I8191 的规范。

转换表处理设置 - MACRO 站接口

如果合成的正交信号被连接到一个远程 MACRO 站，转换表的处理（通常为 1/ T 扩展）是在 MACRO 站上完成的，而且得到的增强的位置信息被传递回 Turbo PMAC，在那里它被 Turbo PMAC 的编码器转换表处理作为未移位的并行数据（通常方法数字$2，模式开关位=1），因为它已经有分数计数信息，在伺服使用之前。关于设置编码器转换表来处理合成的正交编码器信号的细节，请参考 MACRO 站手册和本手册中设置编码器转换表的章节以及软件参考手册关于变量 I8000 -I8191 的规范。

设置为上电绝对位置

可以通过复用器端口直接从 ACC-8D 选项 7 中获得绝对位置。最常见的是，

这只是一个电机转（圈）甚至是一个换向周期内的绝对位置来建立换向相位参考位置而没有任何运动。本节总结了该技术的变量设置，请参阅软件参考手册了解详情。

绝对相位通电位置的地址和格式：Ixx81，Ixx91，MI11x

要从一个直接连接到 Turbo PMAC 的板上读取一个 R /D 转换器的绝对相位的位置，Ixx81 必须被设置为复用器端口上转换器电路板的地址（不直接在 Turbo PMAC 的地址空间中）。Ixx91 的 16-23 位必须被设置为在那个地址的板子上的转换器的位置。

如果 R/D 板被连接到一个 MACRO 站，那么 Ixx81 要被设置为 MACRO 节点号，并且 Ixx91 设为$730000。在 MACRO 站，设置 MI11x 的 0-7 位用来指定复用器端口上转换板的地址，相应地，设置 16-23 位用于指定转换器的位置。

电机相位偏移变量 Ixx75 包含了在绝对旋变位置和结果的相位角位置（如果有的话）之间的差值。

绝对伺服通电位置的地址和格式：Ixx10，Ixx95，MI11x

要从一个直接连接到 Turbo PMAC 的板中读取一个、两个或 3 个 R/ D 转换器的绝对相位位置，Ixx81 必须设置成多路复用器端口上的第一个转换电路板的地址（未直接在 Turbo PMAC 的地址空间中）。Ixx91 的 16-23 位必须设置为在该地址的板的第一个转换器的位置。

如果 R/D 板被连接到一个 MACRO 站，那么 Ixx81 要被设置为 MACRO 节点号，并且 Ixx91 设为$730000。在 MACRO 站，设置 MI11x 的 0-7 位用来指定转换复用器端口上板子的地址，设置 16-23 位用于指定转换器的位置。

如果第二个旋转变压器，从第一个变压器上降低了整数因子 n 倍，用于获取多圈的绝对位置，电机变量 Ixx99 必须被设置为这个 n 值。如果第三个旋转变压器，从第二个变压器上降低了整数因子 m 倍，用于扩展多圈的绝对位置的范围，电机变量 Ixx98 必须设置为这个 m 值。

电机偏移变量 Ixx26 包含了绝对旋变位置和结果的电动机的位置（如果有的话）之间的差值。

缩放反馈单位

ACC-8D Opt 7 的 R/D 转换器是一个 12 位转换器。它报告 4096 个独立状态在每个旋转变压器的电气周期（对于一个典型的 2 极旋转变压器是每转一周，对 4 极旋转变压器是每半周）。它报告这些为一个 12 位的二进制数，如果需要通过复用器端口查询绝对位置的话。对于持续的位置，它在每个电气周期会生成 1024 个数字正交周期，4-倍解码之后每个周期生成 4096 个计数。

5-5 设置 MLDTs

Turbo PMAC 配有 PMAC2 式 IC 能够提供直接接口到磁致伸缩线性位移传感器（即 MLDTs），如 MTS 公司的 Temposonics 品牌。MLDTs 可在恶劣环境中提供绝对位置信息；它们特别适用于液压应用。在此接口中，Turbo PMAC 提供了一种周期性的激励脉冲输出到 MLDT，然后接收传感器中以声速回传的脉冲，并且非常

精确地测量这些脉冲之间的时间，它直接正比于移动构件到传感器的固定底座的距离。因此，定时器包含一个位置测量。

MLDT 接口类型

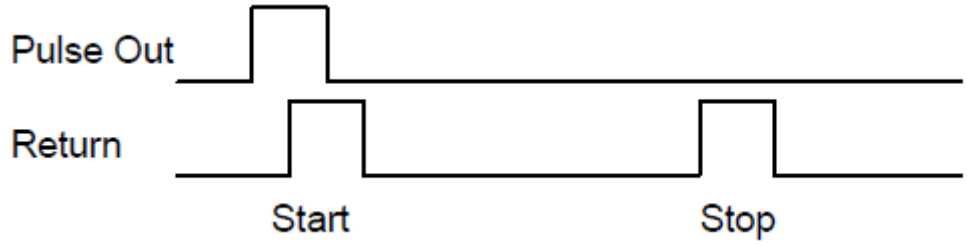
MLDTs 可用于几种不同的接口格式；对于这个接口，需要有外部激励的格式，因为 Turbo PMAC 提供了激励脉冲。通常，这种格式具有一个 RS-422 接口，因为激励和回声脉冲是在 RS-422 电平。Turbo PMAC MLDT 接口的输入和输出是在 RS-422 电平。

有些 MLDTs 配备了内部激励和位置计算，提供一个位置值以一种格式例如同步串行接口（SSI）或一个模拟电压。在这些情况下，设置 Turbo PMAC 接口仅取决于数据格式，而不是传感器的基本原理。详情请参阅相应的反馈格式章节。

信号格式

有两种常见的外部激励型的信号格式，MTS 称它们为 RPM 和 DPM。在 RPM 格式有 MLDT 返回的两个短脉冲：一个立即起始脉冲和一个延迟停止脉冲。

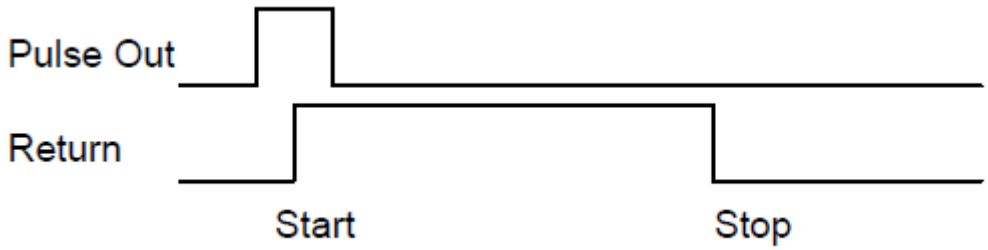
RPM 的信号格式



由于 Turbo PMAC 使用在输出脉冲的下降沿之后返回的第一个上升信号边缘去锁存定时器，这个格式的关键的设置问题是要确保输出的脉冲宽度足够大，能够使输出脉冲的下降沿发生在返回线的起始脉冲的上升沿之后（见下面的 PFM 脉冲宽度一节）。

在 DPM 格式，只有一个长脉冲从 MLDT 返回。

DPM 的信号格式



在 DPM 格式中的返回脉冲的上升沿等价于 RPM 格式的起始脉冲的上升沿。DPM 格式的返回脉冲的下降沿相当于 RPM 格式的停止脉冲的上升沿。因为 Turbo PMAC 想要一个上升的信号边沿来锁存计时器，在该信号格式下返回信号应该被反转，使得 MLDT 的“+”输出被连接到 Turbo PMAC 的“-”输入，反之亦然。

硬件设置

常用于命令步进驱动器的 PULSEn 输出用来作为 MLDT 的激励信号；一般地，编码器的反馈的部分的 CHAn 输入用于接受该响应。PULSEn 输出是一个 RS-422 式差分线路-驱动对。CHAn 输入是一个 RS-422 型差动线路接收器对。强烈鼓励输入和输出都采用差分对，使用它提供的共模噪声抑制。

在某些接口板（例如 Acc-24E2A，Acc-24E2S），PULSEn+/-信号是在线上的输出，否则是辅助标志输入，而且跳线必须安装，以启动这些线路上的输出。请参考主板用户手册，了解详情。

请记住，在 DPM 信号格式或其等价物（见上文），MLDT 的“+”输出应连接到 CHAn-输入，而 MLDT 的“-”输出应连接到 CHAn+输入。

Turbo PMAC 硬件控制参数设置

PFM 时钟频率：I7m03，I6803，MI903，MI907，MI993

脉冲输出使用 Turbo PMAC 的脉冲频率调制（PFM）功能。该 PFM 电路通过反复添加一个命令值到累加器中，进而产生周期性的输出脉冲。当累加器溢出时，产生一个输出脉冲。

将指令值加到累加器是每个 PFM 时钟（PFMCLK）周期执行一次。PFMCLK 频率的管理是通过伺服 IC m 上的通道的 I7m03、MACRO IC0 上辅助通道的 I6803、MACRO 站上的通道的 MI903/MI907/MI993 来完成的。所有通道的 PFMCLK 默认频率为 9.83 MHz；这个频率应该是适合所有的 MLDT 应用。

PFM 输出频率：Mxx07，MI926

一个通道的脉冲输出频率是同时由该通道的 PFMCLK 频率和该通道的 PFM 指令值控制的，也就是那个通道的 C-输出寄存器。当用于步进电机应用时，PFM 指令值是由瞬时指令速度和 Turbo PMAC 上的模拟伺服回路的增益决定的；Ixx02 告诉 Turbo PMAC 将这写到 PFM 命令寄存器中。

对于 MLDT 使用，我们将在上电或复位时用 M-变量写入 PFM 命令寄存器一次。（如果接口是在一个 MACRO 站上，该站的固件会自动执行此操作，使用节点专用变量 MI926 的保存值。）该寄存器的建议的 M 变量的定义是 Mxx07。下表显示了对于这些建议的定义的寄存器：

PFM 脉冲输出地址（Y 型寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Mxx07->Y:	**$078004**	**$07800C**	**$078014**	**$07801C**	**$078104**	**$07810C**	**$078114**	**$07811C**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Mxx07->Y:	**$078204**	**$07820C**	**$078214**	**$07821C**	**$078304**	**$07830C**	**$078314**	**$07831C**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Mxx07->Y:	**$079204**	**$07920C**	**$079214**	**$07921C**	**$079304**	**$07930C**	**$079314**	**$07931C**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Mxx07->Y:	**$07A204**	**$07A20C**	**$07A214**	**$07A21C**	**$07A304**	**$07A30C**	**$07A314**	**$07A31C**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Mxx07->Y:	**$07B204**	**$07B20C**	**$07B214**	**$07B21C**	**$07B304**	**$07B30C**	**$07B314**	**$07B31C**
IC#-通道#	M0 – 1*	M0 – 2*						
Mxx07->Y:	**$078414**	**$07841C**						

伺服 IC 0 和 1 是在 Turbo PMAC2 本身或在 Acc-2E3U-格式的堆栈板上。
伺服 IC 2-9 是在 Acc-24x2 板上。
在双-IC 板，用于奇数编号伺服 IC 的 IC 通道 1-4 是主板通道 5-8。
MACRO IC 通道在手轮端口。

一个示例的 M 变量的定义是：
M107->Y：$078004,8,16,S； 伺服 IC 0 通道 1 PFM（C）指令 值

注意

PFM 命令寄存器实际上是 24 位的寄存器；建议的定义只是使用高 16 位，这就可以为我们的目的提供足够的分辨率。如果您使用寄存器作为一个 24 位的值，例如 **M107->Y：$078004,0,24,S**，它的数值比这里所示的等式的高 256 倍。当使用 MACRO 站，MI926 是一个 24 位的而不是 16 位的值。

脉冲输出的频率应该产生一个周期只是稍微长于最长的预期响应时间对于回声脉冲。对于 MLDTs，响应时间为约 0.35 微秒/毫米（9 微秒/英寸）。在一个 MLDT 1500 毫米（约 60 英寸）长，最长响应时间大约为 540 微秒；对于这个设备脉冲输出之间的周期建议是 600 微秒，为 1667 赫兹的频率。

要从一个 Turbo PMAC IC 产生所需的脉冲输出频率，可用以下公式（假定一个 16 位的 M 变量定义）：

$$\text{输出频率(kHz)} = \frac{\text{Mxx07}}{65{,}536}\text{PFMCLK 频率(kHz)}$$

或者

$$\text{Mxx07} = 65{,}536 * \frac{\text{输出频率(kHz)}}{\text{PFMCLK 频率(kHz)}}$$

要产生 1.667 KHz 的脉冲输出频率，以 9.83 MHz 的默认 PFMCLK 频率，我们计算如下：

$$\text{Mxx07} = 65{,}536 * \frac{1.667}{9{,}830} \cong 11$$

要将这个值写入寄存器，建议一个上电的 PLC 程序；这也可以从主机上用在线命令完成。Turbo PMAC 固件不会自动将该值写入到寄存器。

要从 MACRO 站的一个通道产生所需的脉冲输出频率，下面的公式可以用于节点专用 24 位变量 MI926：

$$\text{输出频率(kHz)} = \frac{\text{MI926}}{16{,}777{,}216}\text{PFMCLK 频率}$$

或者

$$\text{MI926} = 16{,}777{,}216 * \frac{\text{输出频率(kHz)}}{\text{PFMCLK 频率}}$$

要产生 1.667kHz 的脉冲输出频率，以 9.83 MHz 的默认 PFMCLK 频率，我们这么计算：

$$\text{MI926} = 16{,}777{,}216 * \frac{1.667}{9{,}830} = 2{,}982$$

注意

对于使用 MLDT 的电动机，伺服更新时间应至少与此处设置的输出时间一样高（伺服频率应低至或低于输出频率）。

PFM 脉冲宽度：I7m04，I6804，MI904，MI908，MI994

输出脉冲的宽度的控制是通过 PFMCLK 频率与伺服 IC m 上的通道的 I7m04、用于 MACRO ICO 的通道的 I6804、用于 MACRO 站上 IC 的 MI904/MI908/MI994 来完成的。这个 I-变量指定脉冲宽度作为 PFMCLK 的周期数。在 9.83 MHz 的默认 PFMCLK 频率，默认值 15 会产生 1.5 微秒的输出脉冲宽度。这对大多数 MLDT 设备应该是满意的。当使用 RPM 格式或其等价物（见信号格式，上文），脉冲宽度必须足够大，以封闭返回的起始脉冲的上升沿---即，它必须大于在输出脉冲与返回的起始脉冲之间的延迟时间。

PFM 格式选择：I7mn6，I68n6，MI916

通道信号的输出格式的控制是通过用于伺服 IC m 通道 n 的变量 I7mn6、用于 MACRO IC 0 通道 n 的 I68n6、MACRO 站上节点特定变量 MI916 完成的。为了使通道 n 的 C-寄存器电路输出一个 PFM 脉冲序列而不是一个 PWM 脉冲序列，该变量必须设置为 2 或 3。最常见的是，它会被设置为 3，从而使通道 n 的 A 和 B 寄存器输出 DAC 信号而不是 PWM。

注意

Turbo PMAC 的一个通道不能同时用于电机的直接 PWM 控制和 MLDT 脉冲的产生。电机的直接 PWM 控制会在每个相位周期自动写入通道的 A、B 和 C 寄存器。

MLDT 反馈选择：I7mn0，I68n0，MI910

编码器输入信号的解码的控制是通过用于伺服 IC m 通道 n 的 I7mn0、MACRO IC 0 通道 n 的 I68n0、或者 MACRO 站上通道的节点专用变量 MI910 完成的。为了 MLDT 信号的正确解码，该变量必须被设置为 12。通过此设置，在输出脉冲的下降沿，脉冲定时器清零。然后它以 117.96 MHz 的频率向上计数，直到接收到返回脉冲的上升沿，此时定时器的值被锁存到内存映射的寄存器，处理器可以读取。该寄存器是 X-寄存器，在每个通道的基地址。

注意

MLDT 反馈使用的同一个电路用于那个信道上的正交编码器反馈，所以一个编码器和 MLDT 不能同时连接到 Turbo PMAC 同一信道的反馈。在这种模式下，是脉冲定时器被用作反馈的位置测量，而不是与编码器使用的脉冲计数器。计数器仍然登记返回的脉冲数，但此处并不代表一个位置测量。

转换表处理设置 –Turbo PMAC 接口

定时器的寄存器在转换表中处理为并行的反馈，代表位置，就像一个绝对型编码器那样。过滤后的并行读取（方法数字$3）应该被用来抑制因额外或丢失的回声脉冲导致的错误。咨询本手册中设置编码器转换表的章节和软件参考手册中的关于变量 I8000 -I8191 的规范。

转换表处理设置 - MACRO 站接口

如果 MLDT 被连接到 MACRO 站，数据必须同时通过站上和 Turbo PMAC 上的转换表进行处理（虽然 Turbo PMAC 的表实际上只是拷贝环中的数据）。MACRO 站的

转换表包含在设置变量 MI120 - MI151 中。此表应对定时器寄存器进行一个过滤的并行读取，滤波能够拒绝因额外或丢失的回声脉冲造成的错误。结果被通过环传递回 Turbo PMAC，在那里就像环的任何其他类型的反馈那样被处理。咨询本手册中设置编码器转换表的章节和软件参考手册中关于变量 I8000 -I8191 的规范以及 MACRO 站的手册。

设置为上电绝对位置

绝对通电位置的地址和格式：Ixx10，Ixx95，MI11x

因为 MLDTs 是绝对位置传感器，如果它们被用于位置反馈，则无需回零搜索运动。Turbo PMAC 变量 Ixx10（电机 xx 上电伺服位置地址）和 Ixx95（电机 xx 通电位置格式），可用于建立这种绝对位置。Turbo PMAC 软件参考手册中关于 Ixx10 的说明有一个完整的可能的地址表用于设置 MLDT 定时器寄存器，并且还可从 MACRO 站获取绝对位置。

对于一个板载 MLDT 定时器寄存器，Ixx95 应设置为$170000。它应该被设置为$740000 来从 MACRO 站指定通电位置；站变量 MI11x 应设置为$17nnnn，其中“nnnn”代表 MACRO 站上的定时寄存器的 Y 地址。

位置参考偏移：Ixx26

当使用读取绝对位置时，Ixx26 指定传感器的零位和电机零位之间的差值。有了 MLDT，传感器的零位置是在发射机/接收机模块，超过了传感器的可能的运动范围。如果期望该电机的零位是在行程的范围内，Ixx26 应该被用于定义传感器和电机零位之间的偏移量。 Ixx26 的单位是 1/16 计数，其中此处的一个计数是计时器的一个 LSB（最低有效位）。

上电模式：Ixx80

Ixx80 控制的是：为电机 xx 读取的绝对位置是在电源接通/复位时立即执行，还是延迟直到发出一个**$***或**$$***命令。因为当在上电时立刻完成一个读取时，用于 MLDT 的输出频率尚未建立，所以它应当被延迟和一个 MLDT。这需要 Ixx80 的 Bit 2 设置为 1，使得如果上电/复位时电动机不被立即启用，Ixx80 等于 4，或如果电机被立即启用，其值等于 5。

缩放反馈单位

电机单位

对于一个 MLDT 反馈设置的电动机，一个计数是定时器的一个增量（LSB）。这个增量的物理长度，也就是测量的分辨率，依赖于在 MLDT 的返回脉冲的速度和 Turbo PMAC 计时器的频率。返回脉冲（声音在金属中的速度）的速度在不同的设备上各不相同，，但始终大约是 0.35 微秒/毫米，或 9.0 微秒/英寸的倒数。计时器的频率为 117.96 兆赫。分辨率可以计算为：

$$\text{分辨率}\left(\frac{\text{长度}}{\text{增量}}\right)=\frac{1}{\text{定时器频率}}\left(\frac{\mu\,\text{sec}}{\text{增量}}\right)*\text{返回速度}\left(\frac{\text{长度}}{\mu\,\text{sec}}\right)$$

$$\cong\frac{1}{117.96}\frac{\mu\,\text{sec}}{\text{增量}}*\frac{1}{0.35}\frac{\text{mm}}{\mu\,\text{sec}}\cong 0.024\frac{\text{mm}}{\text{增量}}\left(4.13\frac{\text{增量}}{\text{mm}}\right)$$

$$\cong\frac{1}{117.96}\frac{\mu\,\text{sec}}{\text{增量}}*\frac{1}{9.0}\frac{\text{英寸}}{\mu\,\text{sec}}\cong 0.00094\frac{\text{英寸}}{\text{增量}}\left(1060\frac{\text{增量}}{\text{英寸}}\right)$$

轴的用户单位

通常使用 MLDT 作为反馈的电机，分配给它的轴将是毫米或英寸的工程单位。缩放是在轴定义语句来完成。与上面的例子中，一个创建毫米的轴的用户单位的轴定义语句是：

＃1 - >41.3X

轴的用户单位是英寸的轴定义语句是：

＃1 - >1060X

5-6 设置 LVDT、RVDT 与其他模拟量

Turbo PMAC 系统可以，从诸如线性变量位移传感器（LVDT）、旋转变量位移传感器（RVDT）、电容计和电位器的设备中，接受模拟电压，通过可选的模拟-数字转换器作为反馈。这些电压必须是直流 DC；也就是说，必须通过一个恒定的直流电压（而不是 AC 幅度）来表示一个恒定的位置。这意味着如果传感器基本上是载波频率用的交流(例如 LVDT 传感器,RVDT),该传感器信号必须被解调，以移除 AC 载波信号，在连接到 Turbo PMAC 系统之前。通常情况下，这些传感器的供应商可以提供这种信号调理。

硬件设置

模拟信号应连接到模拟配件或选项的连接器，根据相关手册的指示。如果提供差分输入并使用单端输入，物理上将互补输入接线到返回电压，通过差分接收机得到一个正确的读数。

Turbo PMAC 硬件控制参数设置

大部分的 12 位 A/ D 转换器---如在 Acc-36P、Acc-36V、Acc-36E、Acc-6E、选项 12---使用多路复用转换器。访问这些转换器用于伺服的的第一步是以固定的高速率对数据进行解除多路复用。在 Turbo PMAC，这个解除多路复用的步骤是通过变量 I5060 – I5096 控制的。每个相位中断，一对 ADC 中被读取并复制到存储器寄存器而且下一对被选择用于下一次转换。高达 16 对的模数转换器可以存在此环上。解除多路复用的数据被留在寄存器 Y：$003400-Y：$00341F，在那里它可以通过转换表中读取。

如果 12 位 A/ D 转换器是在 MACRO 站，这个解除复用的过程由站变量 MI987 和 MI989 控制的。它是从一个设备固定的对八对 ADC 解除复用。MI989 指定 ADC 的地址，并 MI987 被设置为 1 以启动该过程。由此产生的数据被放置在站寄存器 Y：$0200- Y：$0207，每对的 12 位值在一个 24 位寄存器中。

转换表处理设置--Turbo PMAC 接口

来自一个连接到 Turbo PMAC 的 Acc-28X 转换器的 16 位数据被处理用于伺服反馈，使用一个 Acc-28 A/D 入口（方法数字$1）在 Turbo PMAC 的转换表中。根本的是，这仅仅是将数据从 Y 一个寄存器复制到一个 X 寄存器（在那里伺服回路可以访问它）并移位数据，以便转换器的 LSB（最低有效位）出来被伺服视为一个计数。该转换表将所有的分数-计数位设置为零。不像并行数据的转换，它不会翻转和软件-扩展数据。集成的 Acc-28 A/D 入口（方法数字$5）也进行转移和移位，而且还将前一个的结果加到本周期的数据中。在这两种任一情况下，如果 A / D 数据是无符号的，如同来自 Acc-28B 或 28E 那样，位-19 模式开关必须设置为 1，妥善处理这些数据。

来自 12 位 A/ D 转换器的数据的处理使用的是并行数据转换（未滤波的方法数字$2，或滤波的$3）。源地址是解除多路复用的内存寄存器，而不是 ADC 本身的地址(除非您只使用了 ADC 的一个单通道)。入口的宽度/偏移设置字是$00C00C（宽度 12，偏移量 12），以使用寄存器的高 12 位的数据。这种转换将数据转移到一个 X 寄存器并移位，使得转换器的 LSB 输出来被伺服视为一个“计数”。该转换表将所有的“分数-计数”位均设置为零。

注意，此转换确实允许翻转和软件扩展，所以一定要确保在一个伺服周期不能有一个 11-位的变化（输入范围的一半），否则 Turbo PMAC 将翻转结果。

关于设置编码器转换表去处理模/数转换器的详细信息，咨询本手册中关于设置编码器转换表的章节和软件参考手册中关于变量 I8000-I8191 的规范。

转换表处理设置—MACRO 站接口

来自一个连接到 MACRO 站的 Acc-28x 转换器的 16 位数据被处理用于伺服反馈，使用一个 MACRO 站的转换表中的“Acc-28 A/D”入口（方法数字$1）。从根本上说，这只是将数据从一个 Y-寄存器复制到一个 X-寄存器（在那里伺服环可以访问它）并移位数据，这样使得转换器的 LSB（最低有效位）输出来被伺服视为一个计数。转换表将所有的分数-计数位都设置为零。不像并行数据的转换，它不会翻转和软件-扩展数据。集成的 Acc-28 A/D 入口（方法数字$5）也对数据进行转移和移位，而且还将前一个结果加到本周期的数据中。因为唯一一个可以连接到 MACRO 站的 16 位 ADC 是无符号的，位-19 模式开关必须设置为 1（使得第二个十六进制数字 8）来妥善处理数据。使用 MI10x 将产生的数据复制到 MACRO 环中，然后在 Turbo PMAC 中被视为任何的 MACRO 反馈数据那样，使用未移位的并行转换，将 MACRO 节点地址作为源。

来自 12 位 A/ D 转换器的数据是处理在 MACRO 站中使用并行数据转换（无滤波的方法数字$2，或滤波的$3）。源地址是解除多路复用的内存寄存器，而不是 ADC 本身的地址（除非您只使用了 ADC 的一个通道）。入口中的被用的位的掩码

设置字是$000FFF 来指定低 12 位，或者$FFF000 来使用高 12 位。这种转换将数据转移到一个 X 寄存器并移位，使得转换器的 LSB 输出来被伺服视为一个“计数”。该转换表将所有的“分数-计数”位都设置为零。

这种转换不允许翻转和软件扩展，因此你必须确保在一个伺服周期不能有一个 11-位（输入范围的一半）的改变，否则 Turbo PMAC 将翻转结果。

关于设置编码器转换表去处理模/数转换器的详细信息，咨询 MACRO 站手册和本手册中关于设置编码器转换表的章节以及软件参考手册中关于变量 I8000-I8191 的规范。

设置为上电绝对位置

绝对上电位置地址和格式：Ixx10，Ixx95，MI11x

一般来说，进入 Turbo PMAC 系统的位置数据作为一个模拟电压在本质上是绝对的，因此如果它被用于位置反馈，则不需要回零搜索运动。Turbo PMAC 变量 Ixx10（电机 xx 上电伺服位置地址）和 Ixx95（电机 xx 上电位置格式）可以用作建立这个绝对位置。在 Turbo PMAC 软件参考手册中的 Ixx10 的说明有一个完整的可能地址表用于 ADC 寄存器设置，还用于从一个 MACRO 站得到绝对位置。

Ixx95 应被设置为$B10000 用于来自 Acc-28A 的有符号的 16-位数据；对于来自 Acc-28B 的无符号的 16-位数据，它应设置为$310000。对于 12-位无符号的数据，它应被设置为$0C0000；对于 12-位有符号数据，它应被设置为$8C0000。它被设置为$740000 从 MACRO 站上来指定上电位置；站变量 MI11 应被设置为$17nnnn，其中“nnnn”代表 MACRO 站上 ADC 寄存器（或者内存中的解除-复用寄存器）的 Y-地址。

缩放反馈单位

如果遵循上述的说明，那么 ADC 的 LSB（最低有效位）的增量会被 Turbo PMAC 电机函数视为一个计数。所有后续的位置、速度、加速度单位都是基于该一个计数的定义。

5-7 设置绝对编码器

Turbo PMAC 可以通过各种接口板接受来自很多绝对编码器的反馈数据。关于为这些接口设置硬件和软件的说明，被详细阐述在特定接口板的手册中。下面的表格简要概括了这些设置。

编码器	接口板	转换表	绝对上电*
并行格式	Acc-14D	过滤的并行（$3）	Ixx95=$nn0000
	Acc-14V	过滤的并行（$3）	Ixx95=$nn0000
	Acc-14E	过滤的并行位（$F/$3）	Ixx95=$nn000b
Yaskawa Sigma I	Acc-8D 选项 9	1/T 编码器（$0）	Ixx95=$710000
SSI	Acc-53E	过滤的并行（$3）	Ixx95=$nn0000
Hiperface	Acc-51x 选项 2	高分辨率插补器（$F/$0）	Ixx95=$nn0000
Heidenhain EnDat	Acc-51x 选项 3	高分辨率插补器（$F/$0）	Ixx95=$nn0000
Mitsubishi	Acc-57E 选项 M	过滤的并行（$3）	Ixx95=$nn0000
Sanyo	Acc-49P	过滤的并行（$3）	Ixx95=$nn0000
Tamagawa	Acc-70P	过滤的并行（$3）	Ixx95=$nn0000
*在$nn0000 中的 nn 表示位的编号用作一个 16 进制的值。b 指定使用哪种类型的 24 位字：4 是用于低，5 是中，6 是高。			

对于这些数据格式的实际硬件和软件的设置详情，可以在适当的附件的用户手册中找到。

第六章 基本电机设置

Turbo PMAC 有很多模式用于控制电机。Turbo PMAC 的初始设置的主要部分是硬件和软件的配置来指定一个明确的运行模式。常用的运行模式是：

- 速度模式驱动的模拟量控制
- 扭矩模式驱动的模拟量控制
- 正弦波输入驱动的模拟量控制
- 电源组驱动的直接 PWM 控制
- 步进/步进-替代伺服驱动的脉冲加方向指令

这些模式中的任何一个都可直接在一个 Turbo PMAC 系统上或在一个 MACRO 环上应用。当使用 MACRO 环时，指令和反馈值作为二进制数值跨过环传送；指令信号的实际产生和反馈信号的处理是在远程 MACRO 节点上完成的。Turbo PMAC 中的电机算法都是相同的，不管是否使用 MACRO 环。对每一个电机来说，运行模式的选择都是独立的。

6-1 硬件设置

机械接口端口的硬件设置的细节是根据使用的 Turbo PMAC 系列的实际类型，以及经常是根据接口类型。扩大地说，有四类接口：

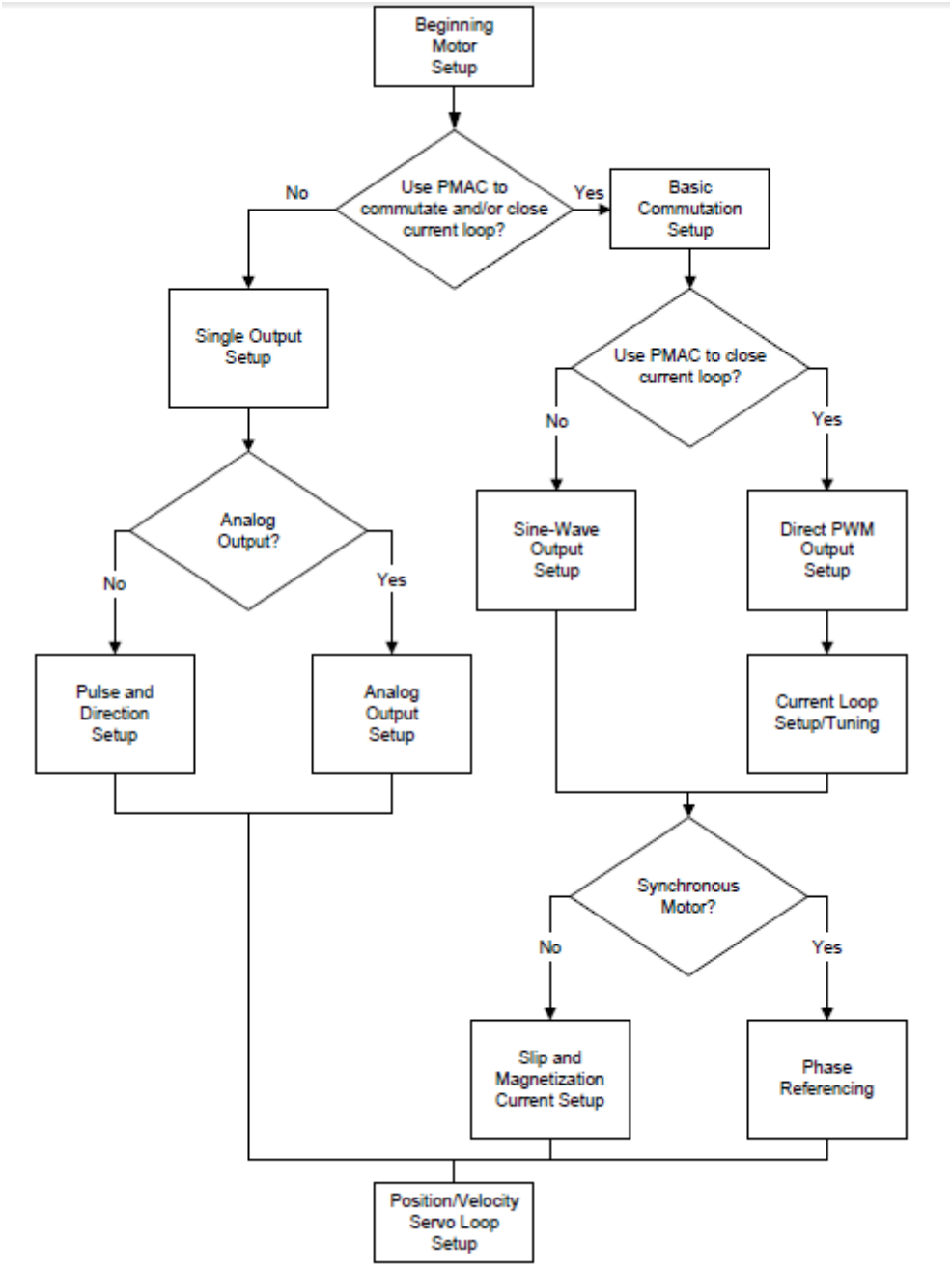
- Turbo PMAC 板
- Turbo PMAC2 板
- 3U-格式堆栈板（Turbo 堆栈和 MACRO 堆栈）
- 3U-格式 UMAC（包）板（UMAC Turbo 和 UMAC MACRO）

这部分概括了关于这几种板子的连接策略。关于连接和其他硬件设置问题，详情包含在各个板的硬件参考手册中。

6-2 设置基本电机运行的参数

每一个电机 xx 都有设置 I-变量，以允许指定该电机控制算法的软件配置。I-变量的百位和千位的数字指定配置哪个电机；例如，I1200 激活或者撤销电机 12。变量的通用参考使用字母 xx 用于电机数字；Ixx00 通常指的是电机 xx 的激活变量，其中 xx 是 1 到 32。

一个电机的软件配置大部分都涉及到为这些变量设置合适的值，就像下面所阐述的那样。这部分会快速概括关键变量以及那些所有模式常见的变量。根据电机如何设置，能够分枝出手册的其他部分。下面的流程图概括了电机设置的可能性。



Turbo PMAC 电机设置流程图

6-3 初始设置参数

激活电机：Ixx00

对于任何要在 Turbo PMAC 上使用的电机，Ixx00 电机激活参数必须设置为 1。对于那些不被使用的电机，它设置为 0，这样 Turbo PMAC 就不会在那些电机上浪费计算时间。一个激活的电机可以被启动或禁用；一个未激活的电机怎么着都不会被监测。

激活 PMAC 电机换相：Ixx01

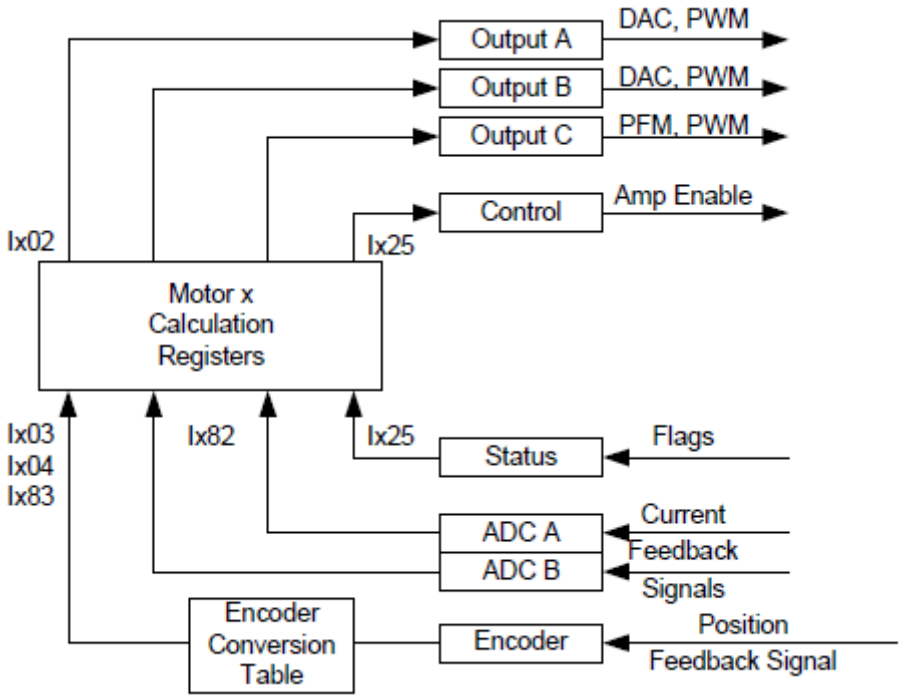
如果 Turbo PMAC 为电机执行换相和/或数字电流-环闭合，Ixx01 换相-使能参数的 0 位必须设置为 1。如果 Turbo PMAC 既不为电机执行换相也不执行数字电流环，那么它必须设置为 0。如果换相被启动，电机的变量 Ixx70-Ixx84 必须设置来指定换相如何完成。关于设置这些变量的说明在本手册的换相部分给出。

6-4 电机地址设置参数

每一个 Turbo PMAC 电机都有一些地址 I-变量，用来告诉电机对其输入和输出使用什么寄存器。这些变量的每一个都包含寄存器的 Turbo PMAC 地址用于特定功能。在电机计算寄存器和不同类型的用于物理接口的伺服 I/O 寄存器（编码器、D/A 转换器、标志等），这提供了一个映射。通过提供一个用户可设的映射，Turbo PMAC 使得对每一个电机利用不同类型的反馈和输出信号变得很简单。

下面的图显示了这种映射如何工作，对于一个电机直接访问伺服 I/O 寄存器（没有 MACRO）：

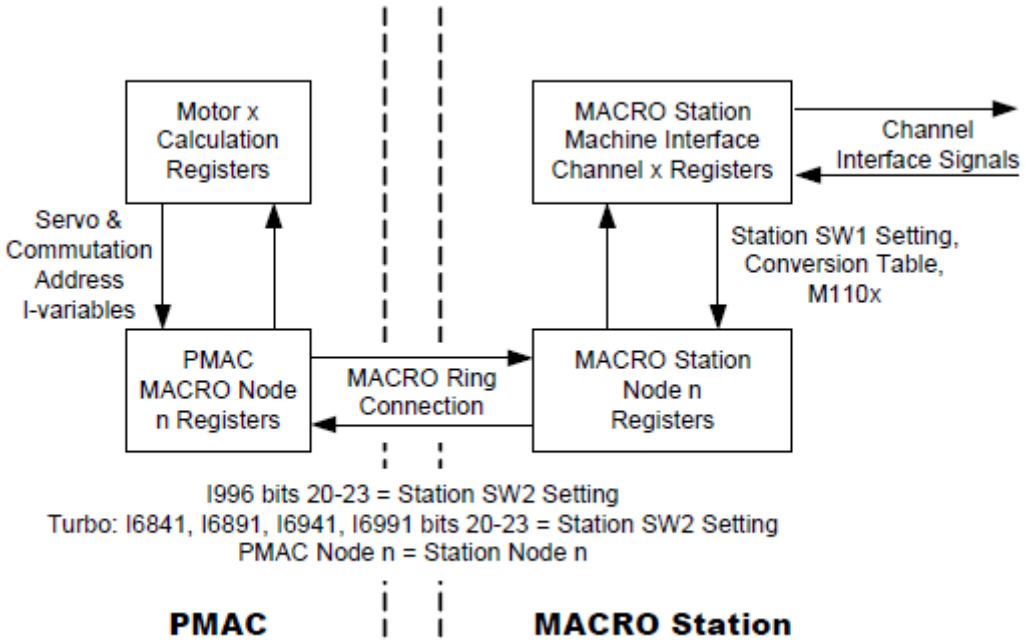
PMAC 直接寄存器映射



下一张图显示了映射的 3 步流程，是在电机计算寄存器和 MACRO 站上的伺服 I/O 寄存器之间，通过 MACRO 环进行。电机地址 I-变量仅仅指定第一步---在电机计算寄存器和 Turbo PMAC 自己的 MACRO 节点寄存器之间的映射。第二步---在 Turbo PMAC 的 MACRO 节点寄存器和 MACRO 站的 MACRO 节点寄存器之间的映射---包含在《基本设置》部分（和 MACRO 站手册中）。第三步---在 MACRO 站的 MACRO 节点寄存器之间的映射---包含在本手册相应章节（和 MACRO 手册）中。一旦映

射设置完毕，电机就要运行就像它有一个直接连接那样。

PMAC/MACRO 站寄存器映射



指令输出地址：Ixx02

Ixx02 指示 Turbo PMAC 将它的用于电机 xx 的输出指令值放置于何处，通过指定寄存器（或者如果使用多路输出，第一个寄存器）的地址。对于通常分配给电机的机械接口通道，Ixx02 的默认值使用输出寄存器；对于通常分配给电机的 MACRO 节点，使用指令寄存器（以寄存器 0 开始）。确切的设置依赖于使用的模式，而且包含在每一个控制模式的章节中。

位置环反馈地址：Ixx03

Ixx03 指定 Turbo PMAC 读取的寄存器的地址，用于电机的持续的位置环反馈。这是在编码器转换表中的一个处理的反馈数据寄存器的地址。默认的转换表下，Ixx03 的默认值指定处理的数据用于那些其通道匹配电机的编码器；这些值适用于大多数应用，不管指令输出模式是什么。更多信息参考本手册的编码器转换表部分。

速度环反馈地址：Ixx04

Ixx04 指定 Turbo PMAC 读取的寄存器的地址，用于电机的速度环反馈。它就像 Ixx03 那样工作，也包含同样的地址，除非电机使用双重反馈。

标志地址：Ixx25，Ixx42，Ixx43

Ixx25，可能还有 Ixx42 和 Ixx43，指定 Turbo PMAC 使用的寄存器的地址，用于它的标志信息用于电机 xx。标志有 3 种设置：

1、捕获输入标志（包括编码器索引），用于触发动作例如回零搜索运动

2、放大器标志（使能输出和缺省输入），用来和驱动握手

3、超行程极限输入标志，用来强迫进入电机位置范围

如果 Ixx42 和 Ixx43 被设为默认值 0，Ixx25 会设置所有 3 个标志的地址。这是典型的情况，而且是唯一一个允许在固件版本 1.939 和更早版本（到 2002）中使用的。但是，从固件版本 1.940 开始，新变量 Ixx42 和 Ixx43 允许拆分标志。当从一个 Acc-51 编码器插补器板卡上触发时，或者当使用 MACRO 环用于反馈和/或触发但不是放大器接口时，这特别有用。

如果 Ixx42 设为一个非零值，它会单独指定放大器标志的地址。如果 Ixx43 设置为一个非零值，它单独指定超行程极限标志的地址。Ixx25 始终指定捕获标志的地址。

下面的表格显示了，当通过 PMAC 型伺服 IC 访问标志时，这些变量可能的地址。

PMAC 型伺服 IC 标志地址

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx25/42/43	**$078000**	**$078004**	**$078008**	**$07800C**	**$078100**	**$078104**	**$078108**	**$07810C**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx25/42/43	**$078200**	**$078204**	**$078208**	**$07820C**	**$078300**	**$078304**	**$078308**	**$07830C**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx25/42/43	**$079200**	**$079204**	**$079208**	**$07920C**	**$079300**	**$079304**	**$079308**	**$07930C**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx25/42/43	**$07A200**	**$07A204**	**$07A208**	**$07A20C**	**$07A300**	**$07A304**	**$07A308**	**$07A30C**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx25/42/43	**$07B200**	**$07B204**	**$07B208**	**$07B20C**	**$07B300**	**$07B304**	**$07B308**	**$07B30C**

1、伺服 IC 0 和 1 都在 Turbo PMAC 本身中。

2、伺服 IC 2-9 都在 Acc-24P/V 或者 Acc-51P 板卡上。

3、奇数伺服 IC 的通道 1-4 是板上的通道 5-8。

下面的表格显示了，当通过 PMAC2 型伺服 IC 访问标志时，这些变量可能的地址。

PMAC2 型伺服 IC 标志地址

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx25/42/43	**$078000**	**$078008**	**$078010**	**$078018**	**$078100**	**$078108**	**$078110**	**$078118**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx25/42/43	**$078200**	**$078208**	**$078210**	**$078218**	**$078300**	**$078308**	**$078310**	**$078318**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx25/42/43	**$079200**	**$079208**	**$079210**	**$079218**	**$079300**	**$079308**	**$079310**	**$079318**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx25/42/43	**$07A200**	**$07A208**	**$07A210**	**$07A218**	**$07A300**	**$07A308**	**$07A310**	**$07A318**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx25/42/43	**$07B200**	**$07B208**	**$07B210**	**$07B218**	**$07B300**	**$07B308**	**$07B310**	**$07B318**

1、伺服 IC 0 和 1 都在 Turbo PMAC2 本身中或者在 Acc-2E 3U 堆栈板上。
2、伺服 IC 2-9 都在 Acc-24x2 或者 Acc-51E 板卡上。
3、奇数伺服 IC 的通道 1-4 是双-IC 板上的通道 5-8。

如果通过 MACRO 环访问标志，指定的地址是 MACRO 节点内存中一个专用映像寄存器的地址，而不是 MACRO 节点标志寄存器本身。下面的表格显示了，当通过 MACRO 环访问标志时，这些变量可能的地址。

MACRO 环标志位地址

IC#-节点#	0-0	0-1	0-4	0-5	0-8	0-9	0-12	0-13
Ixx25/42/43	**$003440**	**$003441**	**$003444**	**$003445**	**$003448**	**$003449**	**$00344C**	**$00344D**
IC#-节点#	1-0	1-1	1-4	1-5	1-8	1-9	1-12	1-13
Ixx25/42/43	**$003450**	**$003451**	**$003454**	**$003455**	**$003458**	**$003459**	**$00345C**	**$00345D**
IC#-节点#	2-0	2-1	2-4	2-5	2-8	2-9	2-12	2-13
Ixx25/42/43	**$003460**	**$003461**	**$003464**	**$003465**	**$003468**	**$003469**	**$00346C**	**$00346D**
IC#-节点#	3-0	3-1	3-4	3-5	3-8	3-9	3-12	3-13
Ixx25/42/43	**$003470**	**$003471**	**$003474**	**$003475**	**$003478**	**$003479**	**$00344C**	**$00347D**

标志模式：Ixx24

变量 Ixx24 是一个各个控制位的集合，决定在 Ixx25、Ixx42 和 Ixx43 中指定的标志如何使用（或者不被使用）。软件参考手册详细描述了每一位；这里给出一个快速总结。

0 位： 设为 0 用于 PMAC 型伺服 IC 的标志，设为 1 用于 PMAC2 型伺服 IC 的标志
11 位：设为 0 用正交编码器捕获，设为 1 用正弦编码器捕获通过 Acc-51 和高分辨率插补。
12 位：设为 0 以使用全-计数捕获，设为 1 以使用分数-计数捕获。

14 位：设为 0 停止在预期位置极限，设为 1 饱和在预期位置极限
15 位：设为 0 禁用预期位置极限，设为 1 是使能
16 位：设为 0 使用放大器使能输出标志，设为 1 不使用
17 位：设为 0 使用超行程极限标志，设为 1 不使用
18 位：设为 0 使用 Turbo PMAC 的捕获标志，设为 1 使用 MACRO 的捕获标志位
20 位：设为 0 使用放大器缺省输入标志，设为 1 不使用
20 位和 21 位：设为 00 以一个故障停止所有电机；设为 01 停止同一个坐标系下的电机；设为 10 只停止故障的电机。
23 位：设为 0 用于低-真的放大器故障（0 意味着故障），设为 1 用于高-真的放大器故障

绝对上电位置地址和格式：Ixx10 和 Ixx95

如果对电机有一个位置传感器在整个电机行程范围内都是绝对的，你可以使用电机变量 Ixx10 和 Ixx95 来分别告诉 Turbo PMAC 到哪里读取这个绝对位置数据，以及如何解译格式。这不必是同一个寄存器或者甚至同一个传感器通过 Ixx03 选定用于持续的位置反馈。如果 Ixx10 被保持为默认值 0，不会为电机执行读取绝对位置，而且电机的上电/复位位置被自动设置为 0。在这种情况下，必须使用回零搜索运动来为电机建立位置参考。

（如果位置传感器仅仅在一个电机转（圈）或者换相周期是绝对的，正如一个单-圈的绝对编码器或者旋变，且电机正由 Turbo PMAC 换相，使用类似变量 Ixx81 和 Ixx91 建立绝对相位位置，但是这种情况下 Ixx10 要设置为 0，而且仍然需要一个回零搜索运动。关于绝对相位位置的设置包含在换相部分。）

如果 Ixx80 的 2 位被设为默认值 0，由 Ixx10 和 Ixx95 定义的绝对位置读取将在上电/复位时自动执行。该绝对读取可以之后在**$***指令完成，不管它在上电/复位时是否完成。

用于绝对上电位置的传感器不一定和那些用于持续的位置反馈的是同一个传感器，而且不必使用相同的数据格式或通道。但是，为了通过 Ixx10 和 Ixx95 使用自动的绝对上电位置读取，必须和持续的位置反馈有同样的分辨率。如果绝对上电位置有一个不同的分辨率，你必须直接写到电机的绝对位置反馈寄存器中（建议的 M-变量 Mxx62），通常是从一个上电 PLC 程序中。如果电机 1 的绝对位置传感器在持续的位置反馈的每个计数时有 0.4 个计数，而且该传感器通过 M150 读取，上电 PLC 设置绝对位置的语句是：

```
M162=M150*0.4*I108*32
```

这必须在电机的环打开时完成。通常，上电模式变量 Ixx80（见下文）设置为 0 或 2，而且该语句执行完毕后环路要闭合。

上电模式：Ixx80

Ixx80 指定 PMAC 是在上电/复位时立刻控制电机（0 位=1）（必要的话执行一个相位搜索）；还是在上电/复位时将电机置为一个停止状态（0 位=0）并等待一个指令启动电机。如果当 Turbo PMAC 上好电时，对于控制不能确定一切准备就绪，将 Ixx80 的 0 位设置为 0（使 Ixx80 成为一个偶数）。通常，系统越复杂功

率越高，越可能延迟启动电机，以确保在电机启动之前一切就绪。

如果一个 PMAC-换相电机需要一个定相-搜索运动，，Ixx80 还控制完成什么类型的定相-搜索运动（这包含在《换相》部分），以及一个绝对位置读取是否在上电/复位时自动完成。

6-5 Turbo PMAC 为该电机换相还是闭合电流环？

所有电机的重要行程都需要在电机相位中进行换相（电流的反向），为了在电机运转时产生一致的扭矩/力。唯一的问题是这个换相在哪里以及如何完成。在一个直流电刷电机中换相在电机内部机械方式地执行。无刷电机的换相通常在驱动器内部电子方式执行。在这些情况下，Turbo PMAC 不执行换相。

几乎所有的现代伺服电机控制都采用电流环闭合用于高速响应、参数振动耐受性、电流过载保护。当这传统地执行在伺服驱动中时，配有 PMAC2 型伺服 IC 的 Turbo PMAC 可以为电机以数字化方式闭合电流环。

如果 Turbo PMAC 为电机或者是进行换相，或者闭合电流回路，或两者兼而有之，参考《设置基于 Turbo PMAC 的换向和/或电流环路》的部分作进一步的说明。如果 Turbo PMAC 对电机不做任务，继续本节下文。

在这一节随后有两个部分。第一部分涉及使用传统的模拟速度模式或转矩模式接口，仍然是最常见的伺服放大器接口。第二个涉及脉冲和方向接口，传统的以及仍是最常见的步进放大器接口，也可用于“步进替代”伺服放大器。

6-6 设置 Turbo PMAC 的速度或转矩控制

如果 Turbo PMAC 对于电动机不执行换向或电流回路，它为电机提供了一个单输出指令值。通常该输出代表一个速度指令或转矩（力，或电流幅值）命令，并且通常该输出被编码成一个模拟信号电压电平。而速度和转矩指令的伺服回路的整定是不同的，直到该点的设置对两种模式是相同的。

当通过一个比例阀或伺服阀驱动液压缸时，动力学对于 Turbo PMAC 来说，是一个速度指令到电动机的那些动力。

硬件设置

在 Turbo PMAC 板或 PMAC 型伺服 IC(如 Acc-24P 或 ACC-24V)的轴扩展板上，每个轴接口通道都有一个 16 位模拟输出。Breakout 板索性引导这个信号到适当的连接器。

板级的 Turbo PMAC2 控制器没有板载的模拟输出用于其伺服通道；一个 breakout 板与 D/A 转换器（DAC）如 Acc-8A 或 Acc-8E，必须被使用。在 UMAC 系统中，Acc- 24E2A 模拟轴接口板有 DAC 用于这一目的。UMAC-CPCI 系统中，Acc-24C2A 有 DAC 用于这一目的。在 QMAC 箱-液位控制器，如果选定了模拟选项，DAC 被安装在里面。在所有这些系统中，每个通道的两个 DAC 是标准的或可选的，但只需要一个 DAC 用于速度或转矩控制模式。泰道公司提供给 Turbo PMAC2 系统的 DAC 具有 18 位分辨率。

关于硬件设置和连接的详细信息，请查阅相应的硬件参考或配件手册。

Turbo PMAC 的参数设置

PMAC2 式芯片的硬件设置

如果通过一个 PMAC2 式伺服 IC 产生一个模拟输出，其也支持其他类型的输出，IC 的几个参数必须设置以实现模拟输出（如果模拟输出端通过一个 PMAC 式伺服集成电路生成，这不是必要的。）。

DAC 时钟频率控制：I7m03，MI903，MI907，MI993

一个 I-变量指定 DACCLK 信号的频率，它控制数据以什么样的速度移入到伺服 IC 的所有通道的串行 DAC。对于 Turbo PMAC 的伺服 IC m，这个变量是 I7m03。如果 IC 是一个 MACRO 站的一部分，该变量为 MI903、MI907 或 MI993。4.92 MHz 的 DACCLK 默认频率适用于泰道公司使用的所有 DAC。这个变量还为伺服 IC 控制其他时钟频率；如果你改变这个变量的值，请务必保持 DACCLK 频率相同。

DAC 选通控制：I7m05，MI905，MI909，MI999

一个 PMAC2 式伺服 IC 为 IC 的所有四个通道产生一个共同的 DAC 选通字。它的完成是通过在每个相位周期移出一个 24 位字，每个 DAC 时钟周期是 1 位，首先是最高有效位。I7m05 包含这个字用于 Turbo PMAC 的伺服 IC m 的通道。如果 IC 是一个 MACRO 站的一部分，该变量为 MI905、MI909 或 MI999。默认值$7FFFC0 适用于泰道使用的 18 位 DAC。

输出模式控制：I7m n6，MI916

I7mn6 必须设置为 1 或 3 来指定伺服 IC m 通道 n 的输出 A 和 B 是在 DAC 模式下，而不是 PWM。（在一个 MACRO 站，这是由节点的特定变量 MI916 控制）。设置为 1 将输出 C（在此模式下不用于伺服或换相的任务）放置在 PWM 模式；设置为 3 将输出 C 置于 PFM 模式。

输出反向控制：I7m n7，MI917

I7mn7 控制是否将到伺服 IC m 通道 n 的 DAC 的串行数据流反转。（在一个 MACRO 站，这是由节点的特定变量 MI917 控制）。默认值 0（非反向）是适用于泰道的任何模拟输出。反转串行数据流的位具有对 DAC 电压取反相的效果。在伺服算法中，如果系统被反转之前正常工作，这将改变输出和输入之间的极性匹配，从而会产生危险的失控状态。

电机参数设置

换相启用/禁用：Ixx01

Ixx01 的 0 位必须设置为 0 以禁用电机 XX 的换相算法。在这种操作模式下，换向在驱动器或电机中进行。Ixx01 的 1 位被设置为 0 来指定，那些在 Ixx02 中指定地址的输出寄存器是 Y 寄存器。几乎所有的在此模式下可使用的输出设备都位于 Y 寄存器中，所以平时 Ixx01 保留为默认值 0。

命令输出地址：Ixx02

Ixx02 通过指定地址来指示 Turbo PMAC 把电机 XX 的输出命令放在哪里。Ixx02 的默认值使用 DAC 寄存器用于匹配电机的机器接口通道。对于 DAC 应用，Ixx02 很少需要改变默认值。通常，所使用的值是：

PMAC 式伺服 IC 命令输出地址（Y-寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx02	**$078003**	**$078002**	**$07800B**	**$07800A**	**$078103**	**$078102**	**$07810B**	**$07810A**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx02	**$078203**	**$078202**	**$07820B**	**$07820A**	**$078303**	**$078302**	**$07830B**	**$07830A**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx02	**$079203**	**$079202**	**$07920B**	**$07920A**	**$079303**	**$079302**	**$07930B**	**$07930A**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx02	**$07A203**	**$07A202**	**$07A20B**	**$07A20A**	**$07A303**	**$07A302**	**$07A30B**	**$07A30A**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx02	**$07B203**	**$07B202**	**$07B20B**	**$07B20A**	**$07B303**	**$07B302**	**$07B30B**	**$07B30A**

伺服芯片 0 和 1 是在 Turbo PMAC 本身。
伺服芯片 2-9 是在 Acc- 24P/V 或 Acc-51P 板。
奇数伺服芯片的通道 1-4 是电路板上的通道 5-8。

PMAC2 式伺服 IC 命令输出地址（Y-寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx02	**$078002**	**$07800A**	**$078012**	**$07801A**	**$078102**	**$07810A**	**$078112**	**$07811A**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx02	**$078202**	**$07820A**	**$078212**	**$07821A**	**$078302**	**$07830A**	**$078312**	**$07831A**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx02	**$079202**	**$07920A**	**$079212**	**$07921A**	**$079302**	**$07930A**	**$079312**	**$07931A**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx02	**$07A202**	**$07A20A**	**$07A212**	**$07A21A**	**$07A302**	**$07A30A**	**$07A312**	**$07A31A**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx02	**$07B202**	**$07B20A**	**$07B212**	**$07B21A**	**$07B302**	**$07B30A**	**$07B312**	**$07B31A**

伺服芯片 0 和 1 是在 Turbo PMAC2 本身或者在 Acc-2E 3U-格式堆栈板上。
伺服芯片 2-9 是在 Acc- 24x2 或 Acc-51E 板上。
奇数伺服芯片的通道 1-4 是双-IC 板上的通道 5-8。

当 Turbo PMAC 运行电机 XX 在一个 MACRO 环上命令的速度模式或转矩模式下时，Ixx02 将为 MACRO 节点之一包含一个 MACRO 输出寄存器的地址。

当使用 Type 1 MACRO 协议，在多-轴 MACRO 组件上常见，如泰道 MACRO 站，应该使用下面的 Ixx02 值。典型地，这些在 Turbo PMAC2 UltraLite 是默认值。

MACRO 节点命令输出地址（Y-寄存器）

IC#-节点#	0-0	0-1	0-4	0-5	0-8	0-9	0-12	0-13
Ixx02	**$078420**	**$078424**	**$078428**	**$07842C**	**$078430**	**$078434**	**$078438**	**$07843C**
IC#-节点#	1-0	1-1	1-4	1-5	1-8	1-9	1-12	1-13
Ixx02	**$079420**	**$079424**	**$079428**	**$07942C**	**$079430**	**$079434**	**$079438**	**$07943C**
IC#-节点#	2-0	2-1	2-4	2-5	2-8	2-9	2-12	2-13
Ixx02	**$07A420**	**$07A424**	**$07A428**	**$07A42C**	**$07A430**	**$07A434**	**$07A438**	**$07A43C**
IC#-节点#	3-0	3-1	3-4	3-5	3-8	3-9	3-12	3-13
Ixx02	**$07B420**	**$07B424**	**$07B428**	**$07B42C**	**$07B430**	**$07B434**	**$07B438**	**$07B43C**

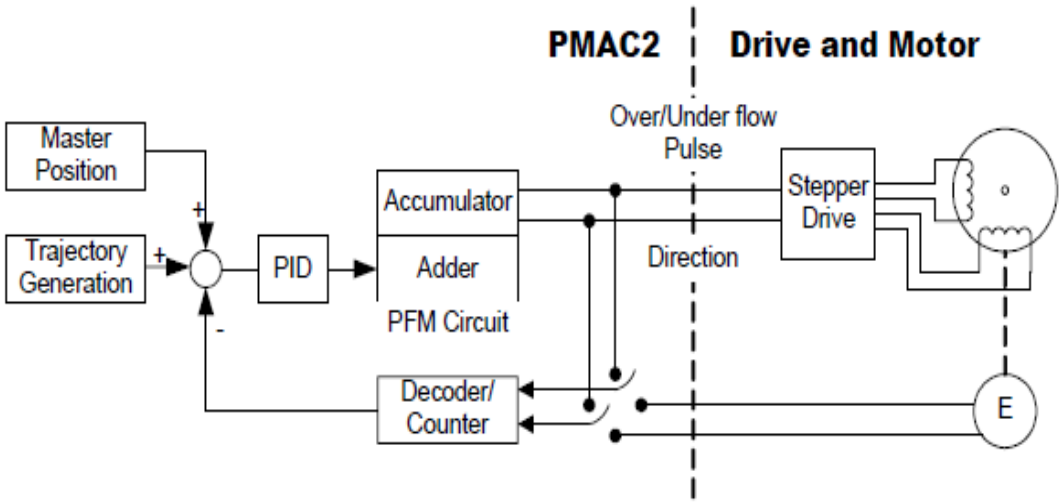
如果使用的是较旧的 Type 0 MACRO 协议，上表所示的值加 1（例如$078420 变成$078421）。

6-7 设置 Turbo PMAC2 为脉冲和方向控制

Turbo PMAC 与 PMAC2 式伺服芯片能够指挥那些需要脉冲和方向格式输入的步进电机驱动器，或需要此格式的“步进-替代”伺服驱动器。Turbo PMAC 可以命令这些驱动器，或者是在开环方式，在这种情况下，内部发送脉冲串到它自己的编码计数器来创建一个伪-闭环；或是在闭环方式，在这种情况下，外部反馈装置是连接到 Turbo PMAC 来创建一个真正的反馈回路。

一个 PMAC2 式伺服 IC 创建它的脉冲和方向输出信号是通过一个板载全数字脉冲频率调制（PFM）电路。该电路用高达 40 MHz 的可编程率重复将最新的指令频率值加到累加器中。当累加器溢出时，通过正方向信号产生一个输出脉冲；当累加器下溢时，通过负方向信号生成一个输出脉冲。这创造了一个脉冲序列，其频率是正比于指令值，几乎没有谐波失真，并且不存在任何影响模拟脉冲生成方案的偏移量的问题。

PMAC2 脉冲和方向步进系统



注意

一个 PMAC 式伺服 IC 的模拟输出，在标志-幅值模式（Ixx96=1）下使用，并且传送通过一个电压-频率转换器，可用于相同类型的操作。但是，强烈建议使用纯数字的 PMAC2 式伺服 IC 的方法来代替。

硬件设置

PMAC2 式伺服芯片，以及 DSPGATE2 MACRO IC，对 IC 的每个通道都有脉冲和方向输出。在接口和 breakout 硬件的大部分配置，这些信号都可以访问作为 RS-422 电平差分线性-驱动器输出对。由通道的输出寄存器 C 中的值驱动这些信号，脉冲频率正比于在该寄存器中的值。因为这个原因，这些输出在技术上称为脉冲频率调制（PFM）输出。

或者，C-输出寄存器中的信号可以被用作脉冲宽度调制（PWM）输出，并且通常形成无刷电动机的直接 PWM 输出的第三相位指令信号。请注意，如果使用通道用于电动机的直接 PWM 控制，PFM 输出将不会在同一通道上被看到。

对于板级的 Turbo PMAC2 控制器和 3U 格式的堆栈板（Acc-1E 和 2E），Acc-8S breakout 板通常是产生脉冲和方向信号的最有效的方法。这些信号也可用在 Acc-8A 和 ACC-8E 模拟 breakout 板以及 ACC-8F PWM breakout 板（作为第三相 PWM）。

脉冲和方向信号是可用的，在 QMAC 盒装控制器、UMAC Acc-24E2S、Acc- 24E2A、ACC-24E2 轴接口板的编码器连接器上。输出使用与通道的 T、U、V、W 标志输入相同的引脚；必须安装跳线以启动这些引脚的输出。

信号定时

PULSEn 和 DIRn 信号是从内部 PFMCLK 信号驱动的，其频率由 I7m03 控制（见下文）。该脉冲的宽度是由 PFMCLK 频率和 I7m04（见下文）来控制。PULSEn 的输出可以是高-真（脉冲期间高，否则低）或低-真，由 I7mn7 所控制；默认值是高-真。DIRn 信号的极性是由 I7mn8 控制。

PULSEn 和 DIRn 信号只可以在 PFMCLK 上升沿改变。如果 DIRn 在一个脉冲上改变，它会同时改变 PULSEn 的前端。一些步进驱动器需要在 PULSEn 的上升沿之前，在 DIRn 线上设定时间；这些系统可以被容纳，通过 I7mn7 反相 PULSEn 信号。

DIRn 信号被锁存在这个状态至少直到下一个脉冲的前端。PULSEn 信号保持为真，保持时间是由 I7m04 设置的 PFMCLK 周期的数量。然后，它变成假，并最少保持与这个时间相同。这保证了脉冲占空比不超过 50%；脉冲信号可以用 I7mn7 反转，而不违反最小脉冲宽度规格。

Turbo PMAC 的参数设置

PMAC2 式芯片的硬件设置

PFM 时钟频率：I7m03，I6803，MI903，MI907，MI993

一个 I-变量控制指令值加到累加器中的频率，通过设置一个所谓 PFMCLK 的时钟信号的频率。每个 PFMCLK 周期期间进行一次加法，所以加法频率等于 PFMCLK 频率。对于一个给定的指令值，脉冲频率正比于该加法频率。虽然默认频率适用于几乎所有的应用程序，但那些要求非常高或非常低脉冲频率的应用可能需要更改此时钟频率。

该 PFMCLK/加法频率对可产生的脉冲频率设置了一个上限---PFMCLK/加法频率的 1/4 的绝对限制。根据高速下可以容忍的最坏情况下的频率失真，大多数人会将他们的最大脉冲频率限制在 PFMCLK/加法频率的 1/10，因此，要选择一个比他们的最大所需的脉冲频率大 10 到 20 倍的 PFMCLK/加法频率。

该 PFMCLK/加法频率同样设定了脉冲频率的下限---八百万分之一的加法频率（无抖动）的绝对限制。约 10 MHz 的默认频率可以提供大约 1Hz 到 100 万 Hz 的一个有用的范围，并且适用于多种应用，尤其是微步进驱动器。对于全或半步驱动器，PFMCLK/加法频率可能将被设置相当低---到大约 1.2 MHz 或 600 kHz 的设置。

I7m03 控制伺服 IC m 的所有轴接口通道的 PFMCLK 频率；I6803 对手轮端口上的两个补充通道做一样的工作。在 MACRO 站，MI903、MI907 和 MI997 控制这个频率。时钟控制电路的输入是一个 39.3216 MHz 的信号；这可以通过 1、2、4、8、16、32、64 或 128 倍的分频来建立 PFMCLK 信号。因此，可能的 PFMCLK 频率和设定它们的 I-变量值是：

分频倍数	分频器 N（1/2^N）	PFMCLK 频率	I-变量值*
1	0	39.3216MHz	2240
2	1	19.6608 MHz	2249
4	2	9.8304 MHz	2258
8	3	4.9152 MHz	2267
16	4	2.4576 MHz	2276
32	5	1.2288 MHz	2285
64	6	611.44 kHz	2294
128	7	305.72 kHz	2303
* SCLK 频率= PFMCLK 频率；ADCCLK 和 DACCLK 频率在默认情况下			

分频器 N 被这些 I 变量用于确定频率。

这些变量也可独立控制编码器的采样时钟 SCLK 的频率，以及串行 D/A 和 A/D 转换器的时钟，DACCLK 和 ADCCLK，它们以同样的方式从相同的 39.3216MHz 的信号分频。如果脉冲串被送入编码计数器，SCLK 频率应该是与 PFMCLK 频率一样。上表显示了用于 PFMCLCK 的每个可能的频率的 I-变量的值，假设 SCLK 频率被设定等于 PFMCLK 频率，DACCLK 和 ADCCLK 频率保持在其默认设置。

PFM 脉冲宽度：I7m04，I6804，MI904，MI908

I7m04 控制 Turbo PMAC PMAC2 式伺服 IC m 上的轴接口通道的脉冲宽度；I6804 也对手轮端口上的补充通道这样做。MI904、MI908 和 MI994 为一个 MACRO 站上的通道设置此内容。脉冲宽度用 PFMCLK 周期指定，范围为 1～255 个周期。

脉冲之间的最小间隔等于脉冲宽度，因此最小脉冲周期是在此设定的脉冲宽度的两倍。这就设定了 PFM 输出的最大频率。如果算法要求一个更高的频率，Turbo PMAC 不会产生要求的频率，并且脉冲将被跳过。

输出模式控制：I7m n6，MI916

I7mn6 控制伺服 IC m 通道 n 的 A、B&C 命令寄存器中产生什么类型的信号；它必须设置为 2 或 3 使用 C 寄存器中的 PFM 信号。节点特定变量 MI916 对于 MACRO 站的通道控制此内容。

输出反向控制：I7mn7，MI917

I7mn7 控制脉冲信号是否被反转。值为 0 或 1 意味着 C 脉冲是高-真，2 或 3 的值意味着它是低-真。节点特定变量 MI917 为 MACRO 站通道控制此项内容。

PFM 方向反转控制：I7mn8，MI918

I7mn8 单独控制 PFM 的方向信号的极性（它不影响脉冲信号）。值为 0 表示正方向为低；值 1 表示负方向为低。节点特定变量 MI918 为 MACRO 站通道控制此项内容。

编码器解码控制：I7mn0，MI910

I7mn0 控制位置反馈信号的源和它是如何解码。节点特定变量 MI910 对 MACRO 站通道控制此项。值为 0 到 7 将设置一个连接到 PMAC 的外部信号，并且此范围

内不同的值确定该信号是如何解码的。如果有一个真正的反馈传感器，应选用这些值其中之一；3 或 7 通常用于正交编码器信号的 4-倍解码。

值为 8 选择内部产生的 PFM 信号，并自动选择信号的脉冲和方向解码。请注意，不需要外部电缆反馈 PFM 信号。如果脉冲串被外部修改到 Turbo PMAC 以满足特定步进驱动器的限制，I7mn0 必须被设置为 0 或 4 来接受这个外部脉冲和方向信号。如果没有方向信号的净反转，I7mn0 将被设置为 0。

对于一个外部反馈信号，I7mn0 的正确设置应使编码器计数器在你所希望的方向向上计数。它也必须与输出的方向感匹配；一个正的指令值（例如，**O10** 命令）必须使计数器向上计数；一个负指令值（如 **O-10**）必须使得计数器向下计数。反转输出的方向感，通过 I7mn8，或者通过改变接线。

设定电机的基本操作的参数

几个电机的 I-变量必须正确设置才能正常使用 PFM 信号。其中大部分是地址寄存器。通常电机＃1 使用轴接口 1 的电路，依此类推，但这不是绝对必要的。

激活和模式：Ixx00，Ixx01

Ixx00 必须为 1 以激活电机的软件，并且 Ixx01 必须为 0 来告诉 Turbo PMAC 不要做换相。（在此模式下，驱动器进行换相。）

命令输出地址：Ixx02

Ixx02 告诉 Turbo PMAC 将电机 xx 的输出指令值写在哪里。为使用 PFM，输出必须被写入到适当编号的轴接口电路的 C-命令寄存器。（默认是到 A 命令寄存器中）。PMAC2 式伺服芯片的 C 命令寄存器的地址是：

PFM 脉冲输出地址（Y-寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx02	**$078004**	**$07800C**	**$078014**	**$07801C**	**$078104**	**$07810C**	**$078114**	**$07811C**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx02	**$078204**	**$07820C**	**$078214**	**$07821C**	**$078304**	**$07830C**	**$078314**	**$07831C**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx02	**$079204**	**$07920C**	**$079214**	**$07921C**	**$079304**	**$07930C**	**$079314**	**$07931C**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx02	**$07A204**	**$07A20C**	**$07A214**	**$07A21C**	**$07A304**	**$07A30C**	**$07A314**	**$07A31C**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx02	**$07B204**	**$07B20C**	**$07B214**	**$07B21C**	**$07B304**	**$07B30C**	**$07B314**	**$07B31C**
IC#-通道#	**M0-1**	**M0-2**						
Ixx02	**$078414**	**$07841C**						

伺服芯片 0 和 1 是在 Turbo PMAC2 本身或在 Acc-2E 3U 格式的堆栈板上。
伺服芯片 2-9 在 Acc-24x2 或 ACC-51E 板上。
奇数伺服 IC 的通道 1-4 是双-IC 板上的通道 5-8。
MACRO ICO（MO）在手轮端口上提供两个脉冲通道。

当通过一个 MACRO 站使用这种模式，输出必须写入适当的 MACRO 节点的寄存器 2，从那里它会被复制到 MACRO 站的匹配通道的 C 命令寄存器。每个 MACRO 节点的寄存器 2 的地址是：

MACRO IC PFM 脉冲输出地址（Y-寄存器）

IC#-节点#	0-0	0-1	0-4	0-5	0-8	0-9	0-12	0-13
Ixx02	**$078422**	**$078426**	**$07842A**	**$07842E**	**$078432**	**$078436**	**$07843A**	**$07843E**
IC#-节点#	1-0	1-1	1-4	1-5	1-8	1-9	1-12	1-13
Ixx02	**$079422**	**$079426**	**$07942A**	**$07942E**	**$079432**	**$079436**	**$07943A**	**$07943E**
IC#-节点#	2-0	2-1	2-4	2-5	2-8	2-9	2-12	2-13
Ixx02	**$07A422**	**$07A426**	**$07A42A**	**$07A42E**	**$07A432**	**$07A436**	**$07A43A**	**$07A43E**
IC#-节点#	3-0	3-1	3-4	3-5	3-8	3-9	3-12	3-13
Ixx02	**$07B422**	**$07B426**	**$07B42A**	**$07B42E**	**$07B432**	**$07B436**	**$07B43A**	**$07B43E**

编码器转换表：用变量 I8000-I8191（MACRO 站上是 MI120-MI151）实现的编码器转换表，对真实或模拟的反馈寄存器进行初始化处理。Turbo PMAC 默认的转换表处理编码器通道，用基于定时器的 1/T 的子-计数插补，在其前十个入口项中。如果使用真正的增量式编码器反馈，这些都是理想的设置。

然而，如果输出脉冲序列被用于模拟的反馈，最好是处理计数而不需任何子计数插补。这将防止 PMAC 试图在脉冲之间定位并创造抖动。在 PMAC 执行程序配置编码器表窗口中，更改从 1/ T 插补的增量到无插补的增量的入口项。如果直接写入到表中的存储位置，改变格式数字从$0 到$C。Turbo PMAC2 系统的编码器寄存器的这种类型的转换的入口项是：

编**码器转换表项，无插补**

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
I8xxx	**$C78000**	**$C78008**	**$C78010**	**$C78018**	**$C78100**	**$C78108**	**$C78110**	**$C78118**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
I8xxx	**$C78200**	**$C78208**	**$C78210**	**$C78218**	**$C78300**	**$C78308**	**$C78310**	**$C78318**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
I8xxx	**$C79200**	**$C79208**	**$C79210**	**$C79218**	**$C79300**	**$C79308**	**$C79310**	**$C79318**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
I8xxx	**$C7A200**	**$C7A208**	**$C7A210**	**$C7A218**	**$C7A300**	**$C7A308**	**$C7A310**	**$C7A318**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
I8xxx	**$C7B200**	**$C7B208**	**$C7B210**	**$C7B218**	**$C7B300**	**$C7B308**	**$C7B310**	**$C7B318**

IC#-通道#	M0-1	M0-2
Ixx02	**$C78410**	**$C78418**

伺服芯片 0 和 1 是在 Turbo PMAC2 本身或在 Acc-2E 3U 格式的堆栈板上。
伺服芯片 2-9 在 Acc-24x2 或 ACC-51E 板上。
奇数伺服 IC 的通道 1-4 是双-IC 板上的通道 5-8。
MACRO IC0（M0）提供两个脉冲通道在“手轮端口”上。

反馈地址：Ixx03，Ixx04

Ixx03 告诉 Turbo PMAC 去哪里找得到它的位置-环反馈。不论使用真实还是模拟的反馈，这个位置应该是一个处理的编码器计数器的，在 Turbo PMAC 的编码器转换表中。在一个典型的设置，Turbo PMAC 的电机 xx 将采用转换表的“xth”入口，地址$003501 至$0035C0。这些都为 Ixx03 的默认值。

Ixx04 告诉 Turbo PMAC 去哪里找得到它的速度-环反馈。对于模拟反馈，这

应该被设置为与每个电机的 Ixx03 相同的地址。

输出（频率）极限：Ixx69

对于一个给定的 PFMCLK 频率，Ixx69 控制最大的脉冲频率。在 Ixx69 的最大值 32,767，最大脉冲频率将是 PFMCLK 的一半。但是，由于最短的脉冲周期是两个时钟周期打开，两个时钟周期关闭，所以最大脉冲频率可以是 PFMCLK 频率的 25%。Ixx69 的方程是：

$$Ixx69 = \frac{MaxFreq(kHz, MHz)}{PFMCLK(kHz, MHz)} * 65{,}536$$

Ixx69 的最大值是 65,536 或 16,384 的 25%。如果 Ixx69 设定为比该值高时，且写入到 PFM 寄存器中的值超过 16,384 时，脉冲产生电路将下降脉冲，频率将下降。

例如，PFMCLK 在 9.83MHz 的默认值，期望的最大频率是 500KHz，Ixx69=（0.5MHz/9.83MHz）*65,536=3,333

设置电机伺服增益的参数

如果使用真实的反馈传感器，应调整电动机就像调整一个正常的伺服电机那样。如果将脉冲序列反馈到编码器计数器来创建一个完全的电子回路，那么回路的反应是可以预测的，并且调整增益可以通过如下公式进行设置。在任一情况下，控制回路似乎对于 Turbo PMAC 充当一个速度模式伺服驱动器。

要创建一个闭-环的位置响应，固有振动频率为 25Hz，阻尼比为 1（几乎适用于所有系统），使用下面所计算的增益设置。

比例增益：Ixx30

比例增益项 Ixx30 的设置是根据方程：

$$Ixx30 = \frac{660{,}000}{Ixx08 * PFMCLK(MHz)}$$

例如，PFMCLK 为 9.83 MHz 的默认值，Ixx08 为默认值 96，Ixx30=660,000/（96*9.83）=700。

微分增益：Ixx31

微分增益项 Ixx31 被设置为零，因为回路就像一个速度-回路伺服驱动器，而且没有必要让 Turbo PMAC 添加阻尼。

速度前馈增益：Ixx32

速度前馈增益项 Ixx32 的设置是根据方程：

$$Ixx32 = 6660 * ServoFreq(kHz)$$

其中 ServoFreq 是伺服中断频率，由 I7m00、I7m01 和 I7m02（或在 Turbo PMAC2 UltraLite 上，是 I6800、I6801、I6802）确定。例如，ServoFreq 缺省值为 2.26 kHz（I7m00=6527，I7m01= 0，I7m02= 3），Ixx32=6660*2.26=15,050。

如果你设置 Ixx30 不同于其上面的公式决定的值，Ixx32 应该从它的上式决定的值进行改变，反比于 Ixx30 的变化。举例来说，如果 Ixx30 是上述方法计算的值一半，Ixx32 应该是上述方法计算值的两倍。

积分增益和模式：Ixx33，Ixx34

积分增益项 Ixx33 通常被设置为零，因为对于数字式电子回路没有偏移或干扰。有些人会用积分增益来强迫产生一个零稳态误差，即使在其他增益没有很好的设置时。

如果 Ixx33 被设置为零，积分模式 Ixx34 是无关紧要的。如果使用 Ixx33，只有当指令速度为零时 将 Ixx34 设定为 1 会接通积分增益；设置 Ixx34 设为 0

在所有的时间都将它接通。

加速度前馈增益：Ixx35

加速度前馈增益项 Ixx35 通常被设置为零，因为电子回路没有惯性要克服。不过，有些用户希望使用 Ixx35 补偿的短时延迟，是在脉冲产生时由加法过程创造的，当改变速度时这将导致小的跟随误差。对于给定的伺服更新时间，可以找到一个最佳的 Ixx35 值，几乎消除所有加速度的误差。

陷波滤波器参数：Ixx36 – Ixx39

陷波滤波器参数 Ixx36 到 Ixx39 在开环情况下设置为零，因为在电子环路本身没有共振，即使是机械系统有共振。

对设置进行测试

准备测试

在开环系统，你可以测试设置的许多初始操作而不必附加任何东西到 Turbo PMAC，因为回路完全在 Turbo PMAC 内运行。要做到这一点，你必须将 Ixx24 的位 17 设置为 1，来禁用你的位置极限。如果 Ixx24 没有其他位被设置为 1，这意味着将 Ixx24 设置至$20000。

最初的测试应该是，当命令被发送给电机时，简单地监测电机的位置和速度。PMAC 执行程序中的位置-报告窗口提供了一个非常方便的方式来监控这一点。为你要测试的电机，选择显示位置、速度和跟随误差的窗口选项。指定报告的位置和速度的单位（对于速度，每秒的计数等于赫兹）。

执行开环测试

首先，使用“0”（开环输出）命令来验证频率发生器的工作。这个命令简单地将一个正比于指令幅值的值放到输出寄存器中。这应该从脉冲发生器产生一个恒定频率的输出，在报告窗口显示一个不断变化的位置和稳定的非零（对于非-零命令）速度。现在不要担心精确值，只需看看是否可以发生一个变化。一个 **010** 命令应该建立一个正的速度和正方向上计数的位置；一个 **0-10** 命令应该创建一个负的速度和在负方向计数的位置。

对开环测试进行故障诊断

如果没有收到这样的结果，看下面：

1 、使用执行程序中的“为什么我没有动？”屏幕，看看是否有一些原因 Turbo PMAC 固件不允许该命令。例如，超程限位输入可能需要保持低电平，或 Ixx24 禁用了限制功能。

2 、如果有一个示波器或频率计数器，看看在输出引脚是否接收到脉冲串。这将有助于隔离问题。如果一个非-零的 **O** 命令下没有收到脉冲串，而且 **00** 命令下也没有脉冲序列，那么产生了脉冲但没有正确反馈。如果没有脉冲串，那么是没有正确生成脉冲。

3 、分配一个 M 变量到由 Ixx02 指向的输出命令寄存器，仔细检查以确保该地址是可以驱动一个 PFM 电路的输出命令 C 寄存器其中的一个。例如，定义 **M102 -> Y:$078004,8,16,S** 来看输出命令寄存器 1C 。现在当 O 命令发出时，监测执行程序中的查看窗口或终端窗口中的 M 变量。该值应该改变。报告值应等于 Ixx69 *（o-命令幅度）/100 。

注意

命令寄存器确实是一个 24 位寄存器。通过只分配 M 变量到顶部(高)16 位，报告的值是以 Ixx69 极限为单位。

如果报告的M变量的变化没有收到，重新检查 Ixx02 和“为什么我没有动？”窗口。如果这里收到了变化，接下来检查人机接口通道 n 的变量 I7mn6，其用于确保脉冲和方向信号被允许输出。

4、要检查反馈路径，先来看看正在使用的人机接口通道 n 的 I7mn0。它应该被设置为 8 将内部脉冲和方向信号带到编码器计数 n，或对于外部脉冲和方向，设为 0 或 4。接下来，在执行程序的配置菜单下，看看编码器转换表的设置。它应该包含一个入口指定你正在使用的编码器通道（上面列出的）的地址的正交转换，可能用 1/ T 插补，但最好没有。检查实际十六进制地址值（例如，用于伺服 IC 0 编码器 1 的$78000，$78008 用于伺服 IC0 的编码器 2）。注意在表中，结果寄存器的地址（例如$003501 用于第一个入口入口）。

5、现在检查 Ixx03 和 Ixx04。它们应该包含来自刚刚上面提到的转换表中的结果寄存器的地址。这告诉电动机使用转换表中处理的值作为其反馈。

一旦验证到了存在一个脉冲串和其反馈进入到编码器计数器以及电机位置寄存器，检查是否已经达到正确的频率范围。发出一个 0100 命令。检查频率，通过查看在窗口中所报告的速度，和/或通过检查示波器或频率计数器的输出脉冲序列。检查频率，在几个其他正和负的命令值（例如，050，0-50，0-100），以使频率正比于该命令，并覆盖所需要的范围。

如果没有得到适当的频率范围，仔细检查用来设定 PFMCLK 频率的 I7m03 的设置。同时检查 Ixx69 的值，其决定该 PFMCLK 频率下的最大频率（0100 频率）。

执行闭环测试

接下来， 用一个 J/命令关闭环路。报告的位置应保持稳定，而且报告的速度应该是零。设置 yotheur 微动 I-变量 Ixx19 至 Ixx22 来得到所需要的速度和加速度。发出一个 J+命令；向上计数，在 Ixx22 指定的速率（注意单位）。发出一个 J-命令，在这个同样的速度向下计数。

既然 PMAC 认为的闭环动作要被执行，伺服环增益的参数是非常重要的。监控性能的最简单方法是使用 PMAC 执行程序的位置窗口，配置为显示当前电机的位置、速度和跟随误差。更详细的分析可以通过数据采集绘图进行。

闭环测试故障诊断

当故障排除这些微动运动时，一定要注意 PMAC 认为电机在做什么基于脉冲反馈同电机实际上做的相比，这是非常重要的。例如，如果电机已经停止，但你通过位置窗口看到，PMAC 保持计数位置，电机可能已经失速。这里可能包括过量的速度指令，过大的负荷和共振问题。但是，如果 PMAC 也报告一个停止，在 PMAC 模拟环路已经发生一些故障，可能导致过度跟随误差关机。这里的主要可能性是 Ixx30 比例增益、Ixx32 速度前馈或 Ixx69 输出限制的值设置过低。

第七章 设立基于 TURBO PMAC 的换相和/或电流环

本节提供了详细说明，可以逐步的手动设置 Turbo PMAC 内的电机换相和/或数字电流环路闭合。很少有用户会手动执行这些步骤；大部分将采用 PC 上的 Turbo 安装程序的自动程序，即使是对于第一单元的设置。本节中的说明是针对这些用户，他们想要充分了解 Turbo PMAC 的算法以及对一个特定的应用他们是如何设置的。

7-1 开始换相设置

如果 Turbo PMAC 要执行一个电机的换相，就必须做更多而不是简单的对电机闭合位置/速度环伺服系统。几个参数必须正确设置来配置换相。

设立换相的第一个步骤是一样的，不管 Turbo PMAC 是执行电流环闭合（直接 PWM 输出模式）还是不执行（正弦波输出模式）。这些步骤在本节中描述。接下来的步骤是不同的，是基于使用哪种模式，这些在接下来的两节中描述，其中只有一个是用于一个特定的电机。最后，换相设置的最后一个步骤对于操作的两种模式又是相同的，这些在后面的部分中描述。

换相启用：Ixx01

如果 Turbo PMAC 对电机 xx 进行换相通过使用直接定址的（不通过 MACRO）伺服 IC 编码器寄存器，那么 Ixx01 必须设置为 1 启动换相并使用一个 X-寄存器用于换相反馈（由 Ixx83 寻址）。如果 Turbo PMAC 使用 MACRO 站的伺服 IC 为电机 xx 执行换向，那么 Ixx03 必须设置为 3 以启动换向并使用一个 Y-寄存器用于换相的反馈。（如果 Turbo PMAC 正在执行数字电流环路闭合，它也必须为电机执行换相。）

> 注意
>
> 数字电流环的有刷电机的直接 PWM 控制采用 Turbo PMAC 的换相算法，即使电机不需要电子换相；这种情况下，Ixx01 必须设置为 1 或 3。

换相周期大小：Ixx70＆Ixx71

Ixx70 和 Ixx71 定义换相周期(电子周期)的大小。周期等于 Ixx71 除以 Ixx70，用编码器计数（解码后）表示。Ixx70 和 Ixx71 必须都是整数，但是 Ixx71/Ixx70 不必是整数。对于一个旋转电机，通常 Ixx71 设置为每转一机械圈的计数的数量，而 Ixx70 被设置为电机的极对数(极数的一半)，它等于每转一周的换相周期数。

当跨整个 MACRO 环的换相是与泰道 MACRO 产品使用的 1 型协议，报告在寄存器 0 中的标准位置反馈是以 1/32 计数为单位的。如果这被用作换相位置反馈，由 Ixx83 指定，Ixx71/Ixx70 的单位也必须是一个计数的 1/32。例如，如果一个

4 极（2 个-电子-周期）的电机有一个 2000-线（8000-计数）的编码器，那么不是将设置 Ixx70 为 2 和 Ixx71 设为 8000，而是 Ixx70 应该设置为 2 和 Ixx71 应设置为 256,000。

当进行有刷电机的直接 PWM 控制，换向算法必须被迷惑去创造直流输出而不是其通常的交流输出。这最好通过将 Ixx70 和 Ixx71 设置为 0 来完成。

换相反馈地址：Ixx83

Ixx83 指定用于换相位置反馈的寄存器的地址。Ixx83 的默认值使用编码器寄存器用于通常被分配到电机的机械接口通道。对 PWM 应用程序，Ixx83 很少需要改变默认值。当使用板载的编码器计数器时，这些是 X-寄存器，所以 Ixx01 必须被设置为 1 而不是 3，来使用这些。通常，所使用的值是：

PMAC 式伺服 IC 换相位置反馈地址（X-寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx83	**$078001**	**$078005**	**$078009**	**$07800D**	**$078101**	**$078105**	**$078109**	**$07810D**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx83	**$078201**	**$078205**	**$078209**	**$07820D**	**$078301**	**$078305**	**$078309**	**$07830D**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx83	**$079201**	**$079205**	**$079209**	**$07920D**	**$079301**	**$079305**	**$079309**	**$07930D**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx83	**$07A201**	**$07A205**	**$07A209**	**$07A20D**	**$07A301**	**$07A305**	**$07A309**	**$07A30D**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx83	**$07B201**	**$07B205**	**$07B209**	**$07B20D**	**$07B301**	**$07B305**	**$07B309**	**$07B30D**

伺服芯片 0 和 1 是在 Turbo PMAC 本身。
伺服芯片 2-9 在 Acc-24P/V 或 ACC-51P 板上。
奇数伺服 IC 的通道 1-4 是板上的通道 5-8。

PMAC2 式伺服 IC 换相位置反馈地址（X-寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx83	**$078001**	**$078009**	**$078011**	**$078019**	**$078101**	**$078109**	**$078111**	**$078119**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx83	**$078201**	**$078209**	**$078211**	**$078219**	**$078301**	**$078309**	**$078311**	**$078319**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx83	**$079201**	**$079209**	**$079211**	**$079219**	**$079301**	**$079309**	**$079311**	**$079319**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx83	**$07A201**	**$07A209**	**$07A211**	**$07A219**	**$07A301**	**$07A309**	**$07A311**	**$07A319**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx83	**$07B201**	**$07B209**	**$07B211**	**$07B219**	**$07B301**	**$07B309**	**$07B311**	**$07B319**

伺服芯片 0 和 1 是在 Turbo PMAC2 本身或在 Acc-2E 3U 格式的堆栈板上。
伺服芯片 2-9 在 Acc-24x2 或 ACC-51E 板上。
奇数伺服 IC 的通道 1-4 是双-伺服-IC 板上的通道 5-8。

当通过在 MACRO 环上执行换相时，Ixx83 指向所用的节点的位置反馈寄存器（寄存器 0）。这些是 Y 寄存器，所以 Ixx01 必须被设置为 3 而不是 1，来使用这些。在 MACRO 1 型协议中使用的、泰道 MACRO 产品预期的值是：

MACRO 节点换相位置反馈地址（Y-寄存器）

IC#-节点#	0-0	0-1	0-4	0-5	0-8	0-9	0-12	0-13
Ixx83	**$078420**	**$078424**	**$078428**	**$07842C**	**$078430**	**$078434**	**$078438**	**$07843C**
IC#-节点#	1-0	1-1	1-4	1-5	1-8	1-9	1-12	1-13
Ixx83	**$079420**	**$079424**	**$079428**	**$07942C**	**$079430**	**$079434**	**$079438**	**$07943C**
IC#-节点#	2-0	2-1	2-4	2-5	2-8	2-9	2-12	2-13
Ixx83	**$07A420**	**$07A424**	**$07A428**	**$07A42C**	**$07A430**	**$07A434**	**$07A438**	**$07A43C**
IC#-节点#	3-0	3-1	3-4	3-5	3-8	3-9	3-12	3-13
Ixx83	**$07B420**	**$07B424**	**$07B428**	**$07B42C**	**$07B430**	**$07B434**	**$07B438**	**$07B43C**

如果使用的是较旧的 0 型 MACRO 协议，在上表所示的值上加 3（例如$078420 变成$078423）。

Turbo PMAC 的电流环或不是

Turbo PMAC 可以对电机进行换相通过为电机相位闭合或不闭合电流环。如果电流环在 Turbo PMAC 中是闭合的，Turbo PMAC 的输出是相位电压指令，通常表示为脉冲宽度调制（PWM）的数字输出。这种技术被称为直接 PWM 控制。如果电流环在 Turbo PMAC 中不闭合，Turbo PMAC 的输出是相位电流指令，通常表示为+/-10V 的模拟电压信号。这种技术被称为正弦波输出控制。

Ixx82 控制使用哪种操作模式。如果 Ixx82 大于零，Turbo PMAC 将对电机 xx 关闭电流环，而且 Ixx82 指定用于电机的电流反馈的地址。这种模式的设置是覆盖在下一节---《设置为直接 PWM 控制》。

如果 Ixx82 设置为 0，Turbo PMAC 不会对电机 xx 关闭电流环。这种模式的设置是覆盖在下面的部分---《设置为正弦波输出控制》。

7-2 设置为直接 PWM 控制

本节介绍如何设置 Turbo PMAC 为放大器的直接 PWM 控制。在这种模式下，Turbo PMAC 执行电机的所有的控制任务，包括换相和数字的电流环闭合。放大器仅执行电源转换任务。在这种模式下，Turbo PMAC 为电机的每一个相位输出 PWM 电压指令。如果对于任何电机不使用 Turbo PMAC 执行此任务，跳过这一节。只需确保对所有已激活的电机，Ixx82 设置为 0，这样 Turbo PMAC 不会为他们尝试闭合电流环。

只能使用 PMAC2 式伺服芯片进行直接 PWM 控制，无论是从 CPU 直接指挥，或者通过 MACRO 环。当基本控制器是一个 Turbo PMAC 时通过附件板做这个控制，这是可能的，但很少见。

介绍

在这种模式下，电流控制环的闭合是通过使用数字运算操作寄存器中的数值，而不是通过使用运算放大器模拟处理操作电压电平。数字技术带来许多优点：没有必要调整或个性化模块；没有温度漂移或时间漂移；计算机分析和自动调整是可能的；增益值容易储存用于备份和复制到其他系统；自适应技术是可能的。

当在 Turbo PMAC 上执行数字电流环路闭合时，几个硬件和软件功能必须正确设置才能正确利用数字电流环和直接 PWM 输出。以下部分详细介绍了如何手动执行此设置。然而，通常这些步骤是自动的，通过使用“Turbo 设置”程序运行在一个 PC 上并且与 Turbo PMAC 通信的。

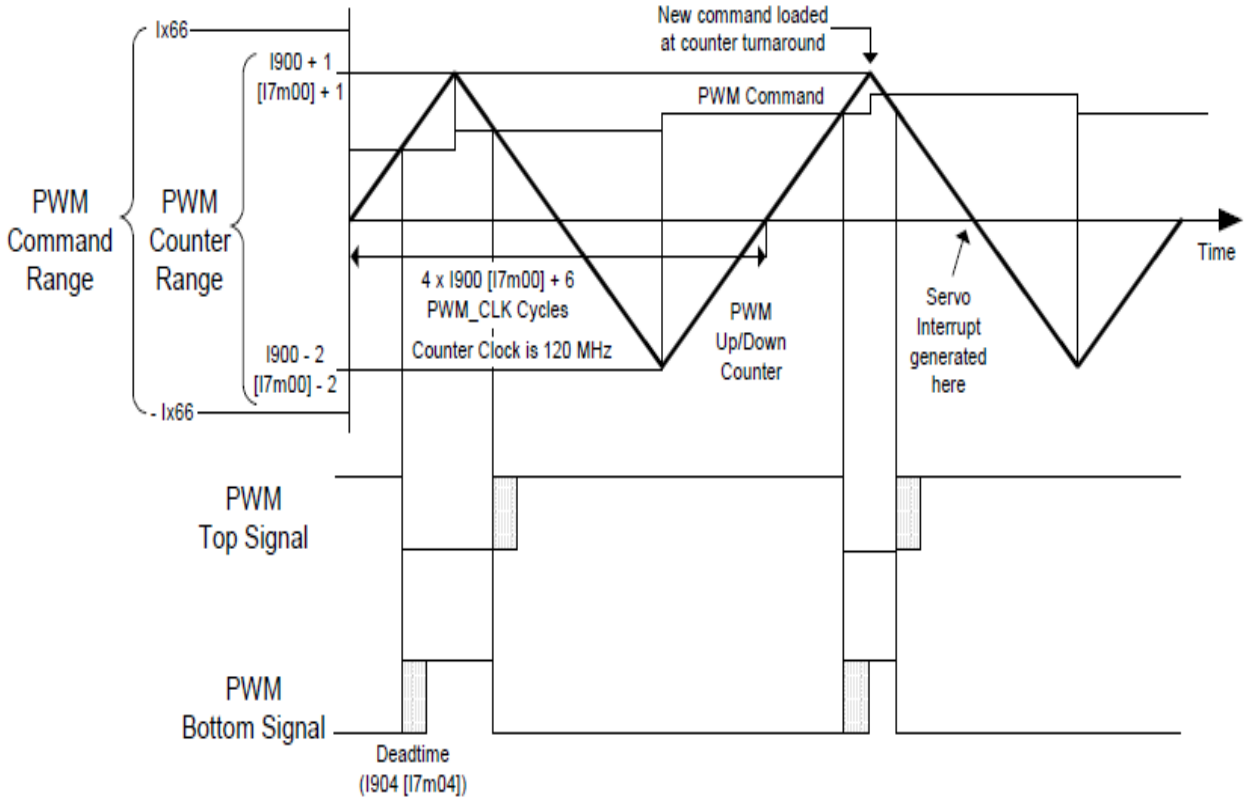
> **注意**
>
> 在本节中给出的说明是用于第一次设置与其他未知的界面。对于一个给定的到驱动器、电机和反馈装置的接口，设置的许多地方将从一个列表中采用，而且不必进行测试，或者如本节描述进行彻底的测试。对每个特定的接口或驱动器，在使用手册中应该有一个配置的专用设置的列表。相同的设置的后续版本应该是更容易的。

数字电流环的操作原理

传统上，电动机相位电流环已经用模拟方式闭合，运算放大器电路从指令的和实际的相位电流信号电平之间的差值，产生相位电压指令。这些模拟量相位电压指令转换成 PWM 格式，通过模拟比较到一个锯齿波形。

Turbo PMAC 允许电机电流环的数字闭合，数学地创建相位电压指令从表示指令电流值和实际电流值的数值寄存器中。这些数字相位电压命令转换为 PWM 格式，通过和一个产生数字锯齿波形的上/下计数器进行数字比较。模拟电流测量值必须用 ADC 转换成数字形式，在环可以闭合之前。

PMAC2 数字式 PWM 生成（每个相位）



通过以这种方式直接指令功率晶体管的开-闭状态，Turbo PMAC 最大限度地减少了伺服回路中的计算和传输延迟。这允许使用更高的增益，从而允许更大的刚性、加速度和干扰抑制。此外，数字技术允许使用电流环数据的数学变换，把测量的交流量转换成直流量用于环的闭合。在下一节中解释的这种技术，通过减少高频问题，从而显著提高高速性能。

参考系

数字电流环路的一个非常重要的优势是它在场系下闭合电流环的能力。要理解这一点的优势，需要一些基本的理论背景。

在一个电机中，有三个重要的参考系。第一个是定子系”，它被固定在电动机的非运动部分，称为定子。在无刷电动机中，电动机的电枢绕组在定子上，因此它们被固定在定子系中。

第二个系是转子系，它参考电动机的运动部件的力学，称为转子。这个系，当然，相对于所述的定子旋转。对于线性无刷电机，这其实是一个转化，但因为它是循环的，我们可以把它看作一个旋转。

第三个系是场系，参考转子的磁场方向。在一个同步电动机中，例如永磁无刷电动机，场被固定在转子上，因此，场系与转子系是相同的。在异步电动机诸如感应电动机中，场相对于转子滑动，所以场系和转子系是分开的。

在场系下工作

电机运行的物理学在场系下最好理解。垂直于转子磁场的定子的电流矢量（即，定子的电流产生的磁场垂直于转子的磁场）产生转矩。定子电流的这一部

分被称为正交电流。位置/速度回路伺服算法的输出是指令的正交电流的幅值。出于对 Turbo PMAC 诊断的目的，一个“0”命令可以用来设置一个固定的正交电流命令。

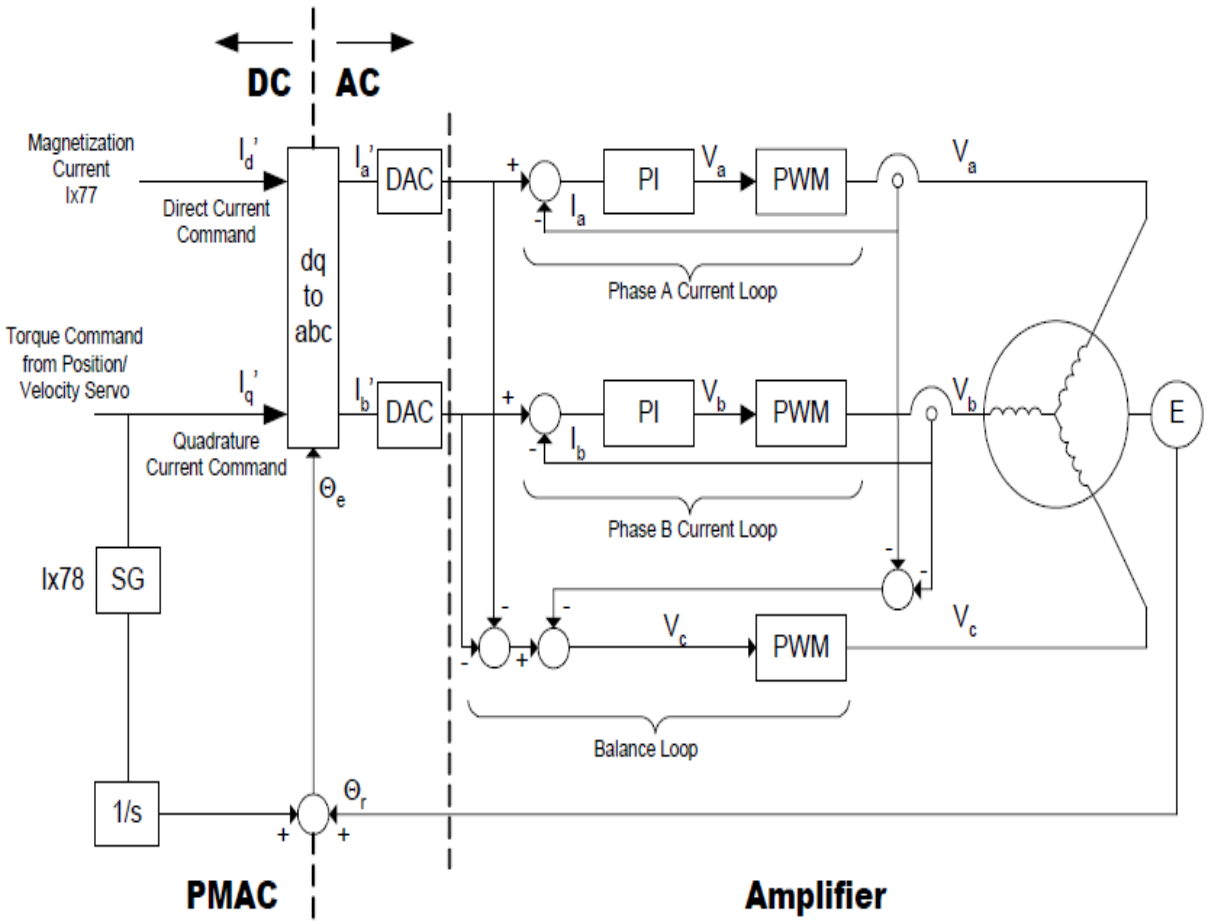
平行于转子磁场的定子中的电流矢量，感应出转子中的电流改变转子的磁场强度（当定子和转子场彼此相对旋转）。定子电流的这一部分被称为直流电流。对于一个感应电动机，这是必需的以建立一个转子磁场。对于一个永磁无刷电动机，转子的磁体始终产生一个场，所以不需要直流，虽然它可以被用来改变磁场强度。在 Turbo PMAC，电机 xx 的参数 Ixx77 决定了直流的大小。

定子系的模拟回路

在一个放大器和一个模拟电流回路中，定子绕组中环路的闭合必须是在定子系中闭合，因为电流的测量是在定子系中，而且模拟电路没有实际的方法转变这些。在这样的一个系统中，电流命令必须从场系中（在那里它们被计算）转换到定子系中，并转换为电压电平，代表各个定子相位电流指令。这些都相对比于其它代表实际定子相位电流测量的电压电平。

在电动机旋转，和/或场滑动时，这些电流值，指令的值和实际的值，都是交流量。总的回路增益以及系统的性能，在高频（高速）降低了。反电动势相位电压，它作为一个对电流环的干扰，也是一个交流量。电流环积分增益或滞后滤波器，它们旨在克服干扰，在高频率仍远远落后。

使用模拟电流环的 PMAC/PMAC2 换相



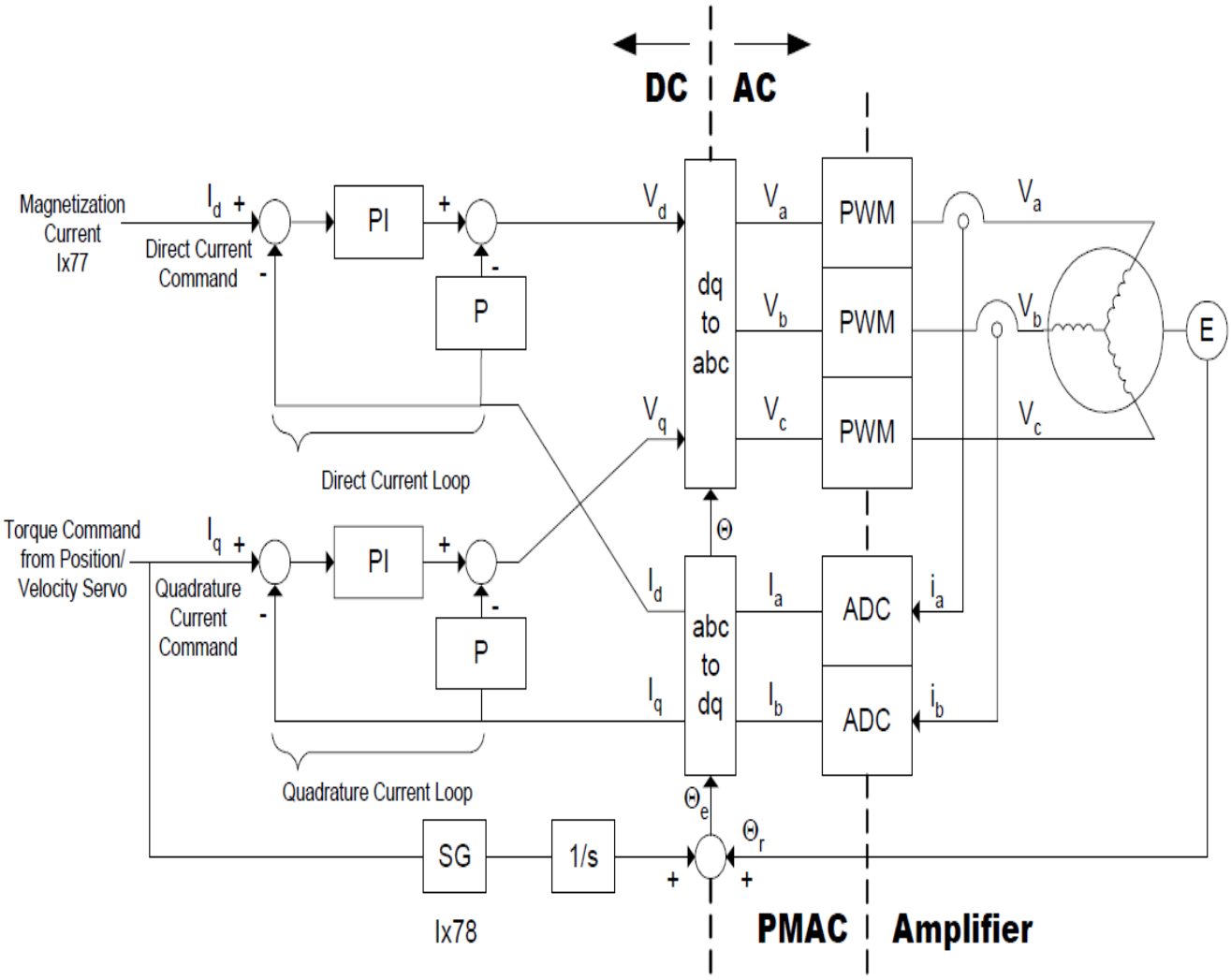
场系中的数字环

在一个用数字电流回路的系统中，可以闭合场系的电流环（但是，不是所有的系统都能这样）。不同于在环路闭合之前电流指令被从场系转换到定子系，实际电流测量值是从定子系转换到场系。在场系，直流环路闭合，而且正交电流环路闭合。这将产生一个直流电压命令和正交电压指令，这些被转换回到定子系成为相-位电压命令，其被实现为 PWM 值。

直流值和正交电流值是 DC 量，即使是电动机在旋转而场在滑动时。因此，伺服回路的高频限制是无关紧要的。这可以显著改善高速性能从相同的电机和电力电子中。

Turbo PMAC 有一个 PI（比例-积分）数字电流环。只有一组的增益，这同时用于直流回路和正交电流回路。最好在直流环路上进行调整，因为这将不产生扭矩，并且因此不产生运动。PMAC 执行程序中的电流环自动调节采用直流回路来调整。这是有效的即使是对于永磁无刷电动机，在那里的实际应用中不使用直流。电流环的性能与负荷无关，因此在调整过程中电机不需要连接到负载。对于位置/速度环调节，确实需要连接负载。

使用数字电流环的 PMAC/PMAC2 换相



硬件设置

Turbo PMAC 和直接 PWM 数字放大器之间的连接是通过一个 Mini-D 36-针连接器，有一个泰道定义的标准的引脚-输出。该连接器包含了 PWM 指令信号、ADC 的反馈信号、放大器使能和故障握手信号。通常情况下，这个连接器可以位于板级 Turbo PMAC2 的 Acc-8F 分线板上、UMAC 的 Acc-24E2 PWM 轴接口板上、配有 PWM 选项的 QMAC 的前面板上。

来自一个编码器或可能是一个旋转变压器的位置反馈，通常是直接回到了 Turbo PMAC；在这种操作模式下没有必要在驱动器中有位置反馈。

对于一个三相电机，无论是 delta 泰道-绕线或（更通常的）Y-绕线，电机的每个三相引线都连接到放大器的相位输出之一。交换三根导线的任意两个会转换输出的方向感；这种适当的匹配到反馈会在一个设置测试进行评估。关于放大器输出，旋转电机的三个引线就会改变换相周期的零点关于任何绝对传感器的零点。

对于一个由 3 相电源模块放大器驱动的直流有刷电机，两个电机引线必须连接在第一和第三放大器输出之间，对应于 Turbo PMAC 的 A 和 C 输出。

Turbo PMAC 的参数设置

大部分的 Turbo PMAC 接口硬件是可以通过 I-变量进行软件配置的。本节提供每个这类应用中重要的 I-变量的基本信息。软件参考手册中对各个 I-变量都有详细描述。

设置全局性和多通道硬件信号的参数

PWM 频率控制：I7m00，MI900，MI906

如果您是直接驱动轴（不通过 MACRO 环），设置 Turbo PMAC 的 I-变量 I7m00 去定义 PMAC2 式伺服 IC M 的 4 个通道的 PWM 频率，根据公式：

$$\mathrm{I7m00} = \mathrm{int}\left(\frac{117{,}964.8\mathrm{kHz}}{4 * \text{PWM 频率}(\mathrm{kHz})} - 1\right)$$

如果轴是从一个 MACRO 站上驱动的，站的 MI900 按照相同的公式，控制站上的前四个通道上的 PWM 频率； MI906 对站的第二组四个通道也这么做。

在指定范围内，对 Turbo PMAC 控制的驱动器设定频率。过高的频率可能会由于开关损耗导致过多的驱动器热量；过低的频率可能会由于大电流脉动纹波，导致缺乏响应性、多余的噪音、可能的物理振动以及过多的电机发热。

对任何一组通道的 PWM 频率必须和相位时钟频率有明确定义的关系。如果该通道是由产生相位和伺服时钟的同一个伺服 IC 驱动的，这种关系是自动设置的。如果另一个伺服 IC 通道上的 PWM 频率是一样的，这种关系也将自动成立。

其他通道的 PWM 频率不必和那些产生相位时钟的伺服 IC 的通道的频率相同，但它们必须与相位时钟有同步关系。对于其他通道的正常直接 PWM 工作模式，下面的关系必须满足：

$$2 * \frac{\text{PWM 频率}}{\text{相位频率}} = \{\text{整数}\}$$

如果一个伺服 IC 用于生成相位与伺服时钟，那么该 IC 的 I7m00 也设置 MaxPhase(最大相位)时钟的频率为该 IC 通道的 PWM 频率的两倍。(在 Turbo PMAC2 UltraLite 板上，MACRO IC0 的 I6800 决定最大相位频率)。最大相位时钟是 Turbo

PMAC 的相位更新任务（其中包括换相和数字电流环路闭合）可以操作的最高频率。请注意，该 I7m00 的任何改变会自动改变相位和伺服时钟频率。

硬件时钟频率控制：I7m03，MI903，MI907

I7m03 决定了伺服 IC m 上机械接口通道的 4 个硬件时钟信号的频率。也许这些可以保持在默认值。四个硬件时钟信号是 SCLK（编码器的采样时钟）、PFMCLK（脉冲频率调制器的时钟）、DAC_CLK（数模转换器时钟）、ADCCLK（模数转换器的时钟）。

如果轴是从一个 MACRO 站上驱动的，站上的 MI903 按照相同的方程，控制站上的前四个通道的硬件时钟频率； MI907 对站上的第二组的四个通道也这么做。

只有 ADCCLK 信号被直接用于数字电流环路，以控制电流环 ADC 的串行数据流的频率。ADC 时钟频率至少必须比 PWM 频率高 96 倍，但它必须在串行 ADC 的能力之内。关于设置这些变量的详细信息，请参阅 I7m03 的说明。

编码器 SCLK 频率至少应比最大计数（边沿）率大 20%，对任何轴上的编码器是可能的。可使用比这个最小值更高的 SCLK 的频率，但这些都使得数字延迟抗噪声滤波的效果较差。

PWM 死区时间控制：I7m04，MI904，MI908

对于伺服 IC m 上的机械接口通道，I7m04 确定顶部和底部信号之间的 PWM 死区时间。I7m04 的范围为 0 对 255，且死区时间为 0.135 微秒乘以 I-变量值。死区时间的设置不应小于驱动器的建议的最小值，否则可能产生过量的驱动器发热。过大的死区时间值可能会造成反应迟钝的性能。默认值为 15，其产生 2.0 微秒的死区时间，已经足够大到可以保护大多数驱动器，也足够小到不会产生反应迟钝的表现除非是 PWM 的频率非常高。一些 480V AC 电源供电的高功率驱动器将需要大约 3 微秒的死区时间，I7m04 的设置约为 23。

如果轴是从一个 MACRO 站上驱动的，站的 MI904 按照相同的方程，控制站上前四个通道的硬件时钟频率； MI908 对站上第二组的四个通道也这么做。

ADC 选通字：I7m06，MI940，MI941

对于所有接口到 DSPGATE1 PMAC2 式伺服芯片的 A/D 转换器，I7m06 定义了 24 位选通字。在每个相位周期，这个字被移出，MSB（最高有效位）在前，来启动 A/D 转换器的转换。$FFFFFE 的默认值适用于大多数的直接 PWM 放大器的 ADC。

DSPGATE1 IC 的 D 版本，于 2002 年推出，采用该字的 0 位（LSB-最低有效位）作为模式控制位用于格式化 ADC 的串行数据。如果该位被设置为 1，IC 可以接受数据流上的多达四位的标头数据，并将它翻转到 ADC 寄存器（不预期数值数据）的 LSB（最低有效位）。在泰道公司的 Geo-直接-PWM 驱动器中的 ADC，2002 年底推出，有这个标头数据，所以需要此位被置位。

当使用 Turbo PMAC 与 Geo-直接-PWM 放大器时，伺服 IC 的 I7m06 的 ADC 选通字必须设置为$1FFFFF，设置该模式-控制位为 1，并预期只有 1 个标头位。

如果轴是从一个 MACRO 站驱动的，站的 MI940 以同样的方式对站上的前四个通道控制 ADC 选通字；MI941 对站上的第二组的四个通道也这么做。

设定每个通道硬件信号的参数

对于伺服 IC m 上用于 PWM 输出的每一个机械接口通道 n（n=1～4），几个通道专用的 I-变量必须正确设置。在 Turbo PMAC 系统本身，这些变量被命名为 I7mnp，其中 m 指定伺服 IC 号，n 指定通道号，p 指定参数号。在一个 MACRO 站，这些变量被命名为“节点-特定的”MI-变量 MI91p（从 Turbo PMAC 寻址，为 **MS{节点}，MI91p**），其中 p 是参数号。

输出模式控制：I7mn6，MI916

I7mn6（在一个 MACRO 站上是节点特定变量 MI916）必须被设置为 0 来指定，对于一个 3 相电机，所有的 3 个输出 A、B 和 C 都是在 PWM 格式。

设置电动机运行的参数

对每个电机 xx，几个 I-变量必须设置来启用和配置那个电机的数字电流环。当然，对于任何激活的电机，Ixx00 必须被设置为 1，不论数字电流环路是否用于该电机。

命令输出地址：Ixx02

Ixx02 通过指定地址来指示 Turbo PMAC 将它的对电机 xx 的输出命令放在何处。对于配有板载 PMAC2 式伺服芯片的 Turbo PMAC，Ixx02 的默认值使用 PWM 寄存器 A、B 和 C，对于通常分配给电机的机械接口通道。（PMAC 式伺服芯片不能被用于直接 PWM 控制）。对于 PWM 应用程序，Ixx02 很少需要改变默认值。指定的实际地址是 PWM 寄存器 A 的地址；然后 Turbo PMAC 也会自动写入到 B 和 C 寄存器中。通常，所使用的值是：

PWM 本地命令输出地址（Y-寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx02	**$078002**	**$07800A**	**$078012**	**$07801A**	**$078102**	**$07810A**	**$078112**	**$07811A**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx02	**$078202**	**$07820A**	**$078212**	**$07821A**	**$078302**	**$07830A**	**$078312**	**$07831A**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx02	**$079202**	**$07920A**	**$079212**	**$07921A**	**$079302**	**$07930A**	**$079312**	**$07931A**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx02	**$07A202**	**$07A20A**	**$07A212**	**$07A21A**	**$07A302**	**$07A30A**	**$07A312**	**$07A31A**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx02	**$07B202**	**$07B20A**	**$07B212**	**$07B21A**	**$07B302**	**$07B30A**	**$07B312**	**$07B31A**

伺服芯片 0 和 1 是在 Turbo PMAC2 本身或在 Acc-2E 3U 格式的堆栈板上。
伺服芯片 2-9 在 Acc-24x2 或 ACC-51E 板上。
奇数伺服 IC 的通道 1-4 是双-伺服-IC 板上的通道 5-8。

当通过在 MACRO 环进行直接 PWM 控制，Ixx02 指向所用的 MACRO IC 和节点的一组三个 MACRO 输出寄存器（0、1、2）中的第一个（寄存器 0）。在 MACRO 1 型协议中使用的、由泰道公司的 MACRO 产品预期的值是：

PWM MACRO 指令输出地址（Y-寄存器）

IC#-节点#	0-0	0-1	0-4	0-5	0-8	0-9	0-12	0-13
Ixx02	**$078420**	**$078424**	**$078428**	**$07842C**	**$078430**	**$078434**	**$078438**	**$07843C**
IC#-节点#	1-0	1-1	1-4	1-5	1-8	1-9	1-12	1-13
Ixx02	**$079420**	**$079424**	**$079428**	**$07942C**	**$079430**	**$079434**	**$079438**	**$07943C**
IC#-节点#	2-0	2-1	2-4	2-5	2-8	2-9	2-12	2-13
Ixx02	**$07A420**	**$07A424**	**$07A428**	**$07A42C**	**$07A430**	**$07A434**	**$07A438**	**$07A43C**
IC#-节点#	3-0	3-1	3-4	3-5	3-8	3-9	3-12	3-13
Ixx02	**$07B420**	**$07B424**	**$07B428**	**$07B42C**	**$07B430**	**$07B434**	**$07B438**	**$07B43C**

如果使用的是较旧的 0 型 MACRO 协议，在上表所示的值上加 1（例如，$078420 变成$078421）。

电流反馈地址：Ixx82

Ixx82 指示 Turbo PMAC 到哪里寻找它的用于电机 xx 的电流反馈值。它也会作为变量告诉 Turbo PMAC 本身是否进行电流环路闭合。如果 Ixx82=0（默认），Turbo PMAC 将不对电机 xx 执行电流回路；任何电流回路必须在放大器中执行。如果 Ixx82>0，Turbo PMAC 将查看由 Ixx82 指定地址的 Y-寄存器，并且对于多相位电机，下一个更低的定址寄存器，去获取其电流回路的电流反馈信息。

通常情况下，指定的寄存器是 Turbo PMAC 的伺服 IC 中的串行 ADC 移位寄存器，或者是在 MACRO IC 中已经带来环上的电流信息的匹配寄存器。指定的实际地址是 ADC 的寄存器 B 的地址；Turbo PMAC 然后也会自动从 A 寄存器中读取。通常情况下，直接命令轴时（不通过 MACRO 环），使用的值是：

板-载 ADC 电流反馈地址（Y-寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx82	**$078006**	**$07800E**	**$078016**	**$07801E**	**$078106**	**$07810E**	**$078116**	**$07811E**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx82	**$078206**	**$07820E**	**$078216**	**$07821E**	**$078306**	**$07830E**	**$078316**	**$07831E**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx82	**$079206**	**$07920E**	**$079216**	**$07921E**	**$079306**	**$07930E**	**$079316**	**$07931E**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx82	**$07A206**	**$07A20E**	**$07A216**	**$07A21E**	**$07A306**	**$07A30E**	**$07A316**	**$07A31E**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx82	**$07B206**	**$07B20E**	**$07B216**	**$07B21E**	**$07B306**	**$07B30E**	**$07B316**	**$07B31E**

伺服芯片 0 和 1 是在 Turbo PMAC2 本身或在 Acc-2E 3U 格式的堆栈板上。
伺服芯片 2-9 在 Acc-24x2 或 ACC-51E 板上。
奇数伺服 IC 的通道 1-4 是双-伺服-IC 板上的通道 5-8。

当通过在 MACRO 上进行直接 PWM 控制时，Ixx82 指向所使用的 MACRO IC 和节点的一组两个 MACRO 输入寄存器中的第二个。通常，所使用的值是相同的，不管该节点采用 MACRO 0 型还是 1 型协议：

MACRO 节点的 ADC 电流反馈地址（Y-寄存器）

IC#-节点#	0-0	0-1	0-4	0-5	0-8	0-9	0-12	0-13
Ixx82	**$078422**	**$078426**	**$07842A**	**$07842E**	**$078432**	**$078436**	**$07843A**	**$07843E**
IC#-节点#	1-0	1-1	1-4	1-5	1-8	1-9	1-12	1-13
Ixx82	**$079422**	**$079426**	**$07942A**	**$07942E**	**$079432**	**$079436**	**$07943A**	**$07943E**
IC#-节点#	2-0	2-1	2-4	2-5	2-8	2-9	2-12	2-13
Ixx82	**$07A422**	**$07A426**	**$07A42A**	**$07A42E**	**$07A432**	**$07A436**	**$07A43A**	**$07A43E**
IC#-节点#	3-0	3-1	3-4	3-5	3-8	3-9	3-12	3-13
Ixx82	**$07B422**	**$07B426**	**$07B42A**	**$07B42E**	**$07B432**	**$07B436**	**$07B43A**	**$07B43E**

如果使用的是较旧的 0 型 MACRO 协议，在上表中所示的值上加 1（例如，$078422 变成$078423）。

电流反馈屏蔽字：Ixx84

Ixx84 指定一个屏蔽字告诉 Turbo PMAC 由 Ixx82 指定的寄存器的哪些位要用在电流环路算法中。这允许使用各种分辨率的 ADC。它还允许使用 18 位 ADC 移位寄存器的剩下部分用于其它的信息，如故障代码。Ixx84 是一个 24 位的值，

与反馈字结合在逐位的 AND 运算中。默认值$FFF000 指定使用反馈字的顶部高 12 位。大多数直接-PWM 放大器采用 12 位 ADC，所以对于这些放大器，$FFF000 是适当的值。

PWM 比例系数：Ixx66

Ixx66，PWM 的比例系数，缩放输出指令值，以使他们有效地使用 PWM 电路。电流环路计算的结果是一个在-1.0 和 1.0 之间的分数值。在被写入一个 PWM 命令寄存器之前，该值乘以 Ixx66，在寄存器中，它以数字方式比较于一个 PWM 向上/向下计数器在 I7m00+1 和-I7m00-2 之间移动。为更好利用 PWM 电路的动态范围，Ixx66 应该被设定为比 I7m00 稍大。通常使用一个大 10%的值，允许全-开状态在最大指令值用超过约 1/6 的换相周期。

Ixx66 充当一个电机的电压限制。如果电机的放大器过大了，超过了电机的最大允许的电压，Ixx66 应相称地设置小于 I7m00，来限制电机的最大可能电压。由于 Ixx66 是一个增益，电流回路必须在任何变更后进行调整或重新调整。

伺服环输出限制：Ixx69

Ixx69 是位置/速度伺服回路输出的限制，它是转矩（正交）的电流指令输入到数字电流回路中。因此，它作为电机的一个瞬时电流限制。开环的 0-命令被表示为 Ixx69 的百分比。

在大多数其他模式，这个参数的 Ixx69 的满量程是 32,767（2^{15}-1）。但是，在 3-相直接 PWM 模式，32,767 的值对应于 A 相和 B 相的 A/D 转换器的满量程读数。从相位电流转换到直流和正交电流中涉及的数学，有效地将相位值乘以 cos（30°）或 0.866。这意味着，对应于全量程 ADC 读数的 Ixx69 的值是 32,767*0.866=28,377。因此，在直接 PWM 模式下，Ixx69 不能被设置为一个大于 28,377 的值。

放大器说明书应指定电流电平，提供从 ADC 的满量程反馈。然后采取驱动器或电机的瞬时电流限制，以较少者为准，并且根据下列关系设置 Ixx69：

$$Ixx69 = \min\left(28{,}377,\ \frac{\text{瞬时电流极限}}{\text{全量程电流}} * 28{,}377\right)$$

如果驱动器输出模拟电流读数和 ADC 是在接口板上，满量程电流值必须进行计算，从在驱动器电流检测的伏特-每-安培的增益和全量程电压进入到接口板。

如果使用 Ixx77 磁化电流的非零值，用于感应电动机的控制或对永磁无刷电动机的磁场减弱，那么在上面方程的 Ixx69 应当由$\sqrt{Ixx69^2 + Ixx77^2}$取代。

在早期的测试中，可以将 Ixx69 设置到人为的低值，以防止意外的过电流命令进入电机。

连续电流限制：Ixx57

Ixx57 指定连续电流限制的大小，用于电机/驱动系统的积分-电流算法用于热保护。如果 Ixx57 大于 0，Turbo PMAC 使用 I^2T 保护，在积分之前对电流值求平方。这是适当的，当大多数温度敏感元件是电阻性质（如电机绕组）的时候。如果 Ixx57 小于 0，Turbo PMAC 使用|I|T 保护，只是积分之前把电流取绝对值。这是适当的，当大多数温度敏感元件两端有一个基本固定的电压时，如一个全-开的双极型晶体管。

通常，是驱动器的连续电流额定值用于一个电机的 Ixx57。即使它的连续电流额定值比电动机的略高时，通常它的热时间常数很短，因此设置积分的电流参

数首先保护它。Ixx57 的计算方法类似于 Ixx69：

$$Ixx57 = \frac{连续电流限制}{满量程电流} * 28{,}377$$

积分电流限制：Ixx58

Ixx58 设定时间-积分的电流在连续电流值所允许的极限。如果时间-积分电流超过这个阈值，Turbo PMAC 将停止该轴，就像它对一个放大器故障那样。通常情况下，这个参数的设置是通过注意在瞬时电流限制的驱动器允许的时间规格。如果使用 I^2T 保护，该规范用于下面的公式：

$$Ixx58 = \frac{Ixx69^2 + Ixx77^2 - Ixx57^2}{32{,}768^2} * 伺服更新速率(Hz) * 允许时间(sec)$$

若使用|I|T 保护的情况下，规范用于下面的公式：

$$Ixx58 = \frac{\sqrt{Ixx69^2 + Ixx77^2} - \sqrt{Ixx57^2}}{32{,}768} * 伺服更新速率(Hz) * 允许时间(sec)$$

关于 I^2T 保护更详细的解释，请参阅《确保你的应用程序安全》的部分。

换相相位角：Ixx72

Ixx72 控制一个多相电机的各相位之间的角度关系。当 Turbo PMAC 以数字方式闭合电机 xx 的电流回路时，该变量的正确设置是依赖于电流测量的极性的。

如果放大器的相位电流传感器和 ADC 的设置使得一个相位的正 PWM 电压指令产生一个负电流测量值，Ixx72 必须被设置为一个小于 1024 的值：对于一个三相电机是 683，或对于一个直流有刷电机是 512。如果这些的设置使得一个正 PWM 电压指令产生一个正的电流测量值，Ixx72 必须被设置为一个大于 1024 的值：1365 是用于一个 3 相电机，或 1536 是用于一个直流有刷电机。下面描述的测试将展示如何确定正确的极性。

Turbo PMAC 中的直接-PWM 算法被优化用于三相电动机，并会导致显著的转矩脉动，当与 2-相或 4-相电机一起使用时。对于这些电机，泰道公司已经创造了用户编写的相位算法，如果有兴趣获取这些，请与工厂联系。

注意

对于您的系统，正确地设置 Ixx72 的值很重要，否则电流环将具有不稳定的正反馈并希望饱和。如果过流关闭功能无法正常工作，这可能会损坏电机、驱动器，或两者兼而有之。如果在您的驱动器中不能确定电流测量极性，参考《测试 PWM 和电流反馈操作》的部分。

对于通过数字电流回路进行的换相，Ixx72 的正确设置与编码器计数器的极性无关。这不同于使用模拟电流回路的换向（正弦波控制），其中 Ixx72 的极性（小于或大于 1024）必须与编码器计数的极性匹配。对于数字电流环，编码器计数器的极性必须设置为适当的伺服操作。对于模拟电流回路，一旦 Ixx72 极性已匹配为换相，要保证伺服回路的极性匹配。

对于有刷电机的直接 PWM 控制的特别说明

为了使用直流有刷电机的直接 PWM 算法，必须进行一些特殊的设置。其基本思路是诱使换相算法以为换向角总是停留在 0 度左右，所以进入 A 相位的电流总是正交（产生转矩）电流。本节概述了在变量的设置方面必须要做的，其中有些设置也已经在别处讨论过了。

这些说明假设：

- 有刷电机的转子磁场来自永久磁铁或由单独的装置激发的绕组磁场；场不被这个通道的相位之一控制。
- 有刷电动机的电枢的两根引线分别连接到放大器（半桥），它是由 Turbo PMAC 的 A 和 C 相的 PWM 指令驱动的。该放大器可能有一个未使用的 B 相的半桥，但是这并不需要。

对于有一个绝对相位参考的永磁无刷伺服电机，设置是一样：

- Ixx01=1（直接在 Turbo PMAC 上换相）或 Ixx01=3（通过 MACRO 环换相）。
- Ixx02 应该包含所使用的输出通道的 PWM 寄存器 A 或 MACRO 节点寄存器 0（这些是默认）的地址，只是关于无刷电机。
- Ixx29 和 Ixx79 相位偏移参数应该分别设置为最大程度地减小相对 A 和 B-相电流反馈电路的测量偏移。
- Ixx61、Ixx62 和 Ixx76 电流环增益的设置只是对于无刷电机。
- Ixx73=0，Ixx74=0：这些默认设置确保 Turbo PMAC 不会尝试为电机做一个定相搜索的移动。失败的搜索可能阻止 Turbo PMAC 启用此电机。
- Ixx77=0，命令零的（场）直流。
- Ixx78=0，用于换相计算中的零差率。
- Ixx82 应该包含所用反馈通道的 ADC 寄存器 B 的地址（只是对于无刷电机），当 ADC 寄存器 A 用于转子（电枢）电流反馈时。寄存器 B 本身应始终包含一个零值或接近零值。
- Ixx81>0：这里的任何非零的设置使 Turbo PMAC 对于电机做一个“定相读取”的操作而不是一个搜索的动作。这是一个假读，因为不管读取的是什么都被迫为零度，通过设置 Ixx70 和 Ixx71，但 Turbo PMAC 要求来完成某种形式的相位参考。 （Ixx81=1 是好的。）
- Ixx84 的设置只是对于无刷电机，指定电流 ADC 反馈使用哪些位。通常，这是$FFF000 指定高 12 位。

对于有刷电机直接 PWM 控制的特殊设置：

- Ixx70=0：这会导致换相周期的所有值与 0 相乘，作废换相矢量的旋转。
- Ixx72= 512（90°e）如果电压和电流数值的极性是相反的，或者等于 1536（270° e）如果它们是相同的。如果放大器使用 683（120° e）用于一个三相电机，这里使用 512；如果它使用 1365（240° e）用于一个三相电机，这里使用 1536。
- Ixx96= 1：这会引起 Turbo PMAC 为（不存在的）直流环路定期清除积分器，可能由于噪声或数值误差慢慢充电，并最终干扰真正的正交电流回路。

并不重要的设置：

- Ixx71（换相周期大小）并不重要，因为 Ixx70 设置为 0 打败了换向周期。
- Ixx75（上电相位参考的偏移量）并不重要，因为换相周期已经作废了。其保持为默认值 0 是好的。
- Ixx83（持续的换相位置反馈地址）不重要，因为换相已经作废。保持为默认值是好的。
- Ixx91（上电相位位置格式）并不重要，因为但凡是读取的上电相位位置都被减少到零。

测试 PWM 和电流反馈操作

注意

在许多电机和驱动系统中，潜在的故障电压和电流电平是存在的。不要试图直接在这些高电压和电流电平上工作，除非关于所有必要的安全程序，你得到充分的训练。Turbo PMAC 和接口板上的低电平信号可以更安全地访问

介绍

在建立一个直接 PWM 接口的大部分时间，并不执行这些部分中列出的所有的步骤（或者 Turbo 安装程序会自动做这些），但第一次设置这种类型的接口，或者如果有问题，这些步骤将有很大的帮助。

出于安全原因，进行所有的这些测试，电机应该从任何负载上断开。作为这些测试的结果，所有的设置都是独立于负载特性的，所以当负载连接时仍然有效。

在为数字电流环和直接 PWM 接口测试任何 Turbo PMAC 的软件功能之前，重要的是要知道的硬件接口是否工作正常。我们将使用 PMAC 的 M 变量直接访问输入和输出寄存器。此处显示的示例使用电机 1 的建议的 M 变量的定义；对于其他电机，在手册的例子部分显示有同等的建议定义。

目的

这组测试的目的是为了确认 PMAC 的、驱动器的、电动机的硬件电路的基本操作，并检查合适的相互关系。具体做法是：

- 确认编码器输入和解码的操作
- 确认 PWM 输出的操作
- 确认 ADC 的输入的操作
- 确认 PWM 输出和 ADC 输入之间的相关性
- 确定正确的电流回路极性
- 确认换相周期的大小
- 确定正确的换相极性

准备

首先定义编码器计数器的 M 变量、3 个 PWM 输出寄存器、放大器使能输出位以及两个 ADC 的输入寄存器。使用电机 1 的建议定义，利用伺服 IC0、通道 1，我们有：

```
M101->X：$078001,0,24,S;   通道 1 编码器位置寄存器
M102->Y：$078002,8,16,S;   通道 1 PWM 相位 A 指令值
M104->Y：$078003,8,16,S;   通道 1 PWM 相位 B 指令值
M107->Y：$078004,8,16,S;   通道 1 PWM 相位 C 命令值
M105->Y：$078005,8,16,S;   通道 1 相位 A ADC 输入值
M106->Y：$078006,8,16,S;   通道 1 相位 B ADC 输入值
M114->X：$078005,14;       通道 1 放大器启用命令位
```

请注意，即使一般的采用 12-位 ADC，ADC 值也被声明为 16-位变量；这使该变量的缩放与 Ixx69、Ixx57、Ixx29 和 Ixx79 是同样的单位。

在执行程序的监视窗口中监控这些值时，它非常有用，所以添加变量名到监视窗口，使程序反复查询 Turbo PMAC 它将显示的值。然后可以通过终端窗口中发出的在线命令，练习硬件。

对于这些测试， Turbo PMAC 要准备：

- I100 设置为 0 以使电机无效。
- I101 设置为 0 来禁用换相（这允许手动使用这些寄存器。）
- 确保 I7000、I7004、I7016 和 I7017 都正确设置以提供所需的 PWM 信号。
- 如果放大器使能位是 1，用命令 **M114=0** 将它设置为零。
- 所有其它电机的 Ixx00 和 Ixx01 设置为零。

位置反馈和极性测试

如果在监视窗口中观察到的 PWM 指令值不为零，将它们设置为零，用命令

M102= 0　M104= 0　M107=0

现在电机应能手动自由移动了。当转动电机时，监控监视窗口中的 M101 值。查找以下：

- 电动机被移动时，它应该改变。
- 它应该一个方向上向上计数，在另一个方向上向下计数。
- 它应该提供，在一圈或一个直线距离增量内预期的计数的数量。
- 由于电机被反复退回到参考位置，每次它应该报告（约）相同的位置值。

如果这些事情不发生，检查编码器/旋转变压器的操作，其到 Turbo PMAC 的连接，以及 Turbo PMAC 的解码变量 I7mn0。仔细检查传感器被供电。用示波器看编码器波形。

如果运动的正方向是已知的，在这里检查。如果方向是不正确的，颠倒它，通过改变 I7mn0，通常是从 7 变到 3，或从 3 变到 7。如果还不知道需要哪个方向感，稍后更改它，但那时要做另一个变更以保持正确的换相极性匹配，通常是通过交换驱动器的两个电机相位导线。

注意

由于 I100 已被设置为 0，并且 I103 可能还没有被正确设置，位置的任何变化都不会反映在电机位置窗口中。

PWM 输出&ADC 输入连接

确保在以这种方式应用任何 PWM 命令到驱动器和电机之前，所产生的电流电平是同时在驱动器和电机的连续电流额定值以内。

首先启用放大器，然后应用一个非常小的正指令值到相位 A 和一个非常小的负指令值到相位 B，用在线命令：

M114=1;　启用放大器

M102= I7000/50　M104=-I7000/50　M107= 0; A 为正，B 为负，C 为零

这提供了一个在满电压的 2%的命令进入电机，它应该是完全同时在驱动器和电机的连续电流额定值之内。它始终是一个好主意，使这些命令的总和等于零，以免放一个净直流电压在电机上；把所有三个命令放在一行上引起该变化几乎瞬间发生。

将电源应用到驱动器并启用放大器（**M114=1**），电流读数应在 ADC 寄存器接收到，由他们的 M-变量 M105 和 M106 在 Watch 窗口显示。

由于 M 变量已经被定义，+/-32,768 是满电流量程，这应该约对应瞬时电流极限。确保读取的值不超过连续电流限制，通常大约是瞬时极限的 1/3。如果该值是远低于连续电流限制，增加电压指令到最大值的 5%至 10%。例如：

M102= I7000/10　M104=-I7000/10　M107=0;　最大值的 10%

PWM/ ADC 相位匹配

Turbo PMAC 的相位 A 的 PWM 输出指令值应该引起一个大致比例的响应一个符号或另一个在 Turbo PMAC 的相位 A 的 ADC 输入中（无论相位在电机和驱动器中是什么命名）。这同样适用于 B 相。

如果任一相都没有响应，请重新检查整个设置，包括：

- 驱动器是否正确连接到 Turbo PMAC，无论是直接或通过一个接口板？
- 电机是否正确连接至驱动器？
- 驱动器是否正常供电，包括功率级和输入级？
- 接口板是否正常供电？
- 放大器是否启用（Turbo PMAC 上为 M114=1，驱动器上指示灯为 ON）？
- 放大器是否在故障条件下？如果是这样，为什么？

如果只在一个相位收到 ADC 响应，相位输出和输入可能没有正确匹配。例如，B 相的 ADC 可从由 C 相位 PWM 输出命令的相位中读取电流。确认这个，是通过尝试命令的其他组合，如下面的六个步骤的测试，并检查哪一个 ADC 响应哪一个相位命令。如果没有一个正确的匹配，改变 Turbo PMAC 和变频器之间的接线。更改驱动器和电机之间的配线不会有所帮助。

同步电动机步进动作

对于一个同步电机，该命令应引起电动机锁定进一个位置，至少是虚弱地，像一个步进电机。这个动作可能会在感应电动机上被暂时接收，是由于在转子中创建的临时涡流。然而，感应电机不会在新的位置无限期地保持一个保持转矩。

电流回路极性检查

观察 M105 和 M106 的 ADC 寄存器值的符号。这两个值应该是大致相同的大小，并且必须是彼此相反的符号。（同样，请记住这些读数可能看似嘈杂。观察噪声下的基值）。如果 M105 是正、M106 为负时，PWM 指令的符号匹配 ADC 的反馈值的符号。在这种情况下，Turbo PMAC 相位角参数 I172 必须设定为一个大于 1024 的值（1365 用于一个三相电机）。

如果 M105 为负而 M106 为正时，PWM 指令的符号与 ADC 的反馈值的符号是相反的。在这种情况下，I172 必须被设置为一个小于 1024 的值（683 用于一个三相电机）。

注意

确保 I172 的值正确设置，试图关闭 Turbo PMAC 的数字电流环路。否则会出现正反馈产生不稳定的电流回路，这可能会损坏放大器和/或电机。

如果 M105 和 M106 有相同的符号，两相的电流检测电路的极性没有正确匹配。在这种情况下，在驱动器上或 Turbo PMAC 和驱动器之间的一些线被接错，给予两个相位电流读数相反的极性。其中的一相必须是固定的。

不要试图闭合 Turbo PMAC 的数字电流环路，直到两相的电流检测电路的极性已正确匹配。这将涉及一个硬件的变化，在电流检测电路接线、ADC 电路或它们之间的连接上。作为一个对错误的额外保护，确保 Ixx57 和 Ixx58 被正确设置为 I^2T 保护，如果因不当的反馈极性而出现饱和，它会很快关闭轴。

故障排除

如果没有得到电流的预期读数，用一个搭扣式霍尔效应电流传感器探测电机电缆的电机相位电流。如果当命令电压时没有看到电流，当电动机断开时检查相-相连续性和正确的电阻。

电压六步测试

对电机/驱动器连接的一个完整的测试，尝试 3 相电机的所有六个符号组合，或 4 相电机的八个组合。最好在一个命令行中命令所有相位值来获得同步变化。

对于一个同步电机，该测试将电机步进通过一个换向周期。它可以被用来确认换相周期的计数的大小、电动机的极数和换相的方向感。

寻找什么

执行此测试时，检查以下内容：

- 获取 A 相的 M102 和 M105 之间、B 相的 M104 和 M106 之间的一致关系
- 对于同步电机（也可能是异步电机），M101 位置寄存器应该改变以大致相等的增量在每一步之间。
- 对于同步电机（也可能是异步电机），在最初的第 1 步和返回到第 1 步之间的 M101 的总变化应约等于 Ixx71/Ixx70。
- 对于同步电机（也可能是异步电机），最初的第 1 步和返回到第 1 步之间的转子位置的物理变化（如果需要，标记转子）应该等于 1 个极对：
  - 在一个 2 极电机，它应该是一个完整的机械转。
  - 在一个 4 极电机，它应该是半个机械转。
  - 在一个 6 极电机，它应该是三分之一的机械转。

- 在一个 8 极电机，它应该是四分之一的机械转。
- 在一个 100 极电机，它应该是 7.2° 的机械转。

执行测试

对于电动机 1 使用伺服 IC0，通道 1 与一个 3 相电机，用 10%的电压命令可接受的，命令可以是：

M102= 0 ,M104= I7000/10, M107=-I7000/10；第 1 步：A0，B +，C−：0º 电子。
M102= I7000/10,M104= 0 ,M107=-I7000/10；步骤 2：A+，B0，C−：60º 电子。
M102= I7000/10 ,M104=-I7000/10 ,M107= 0；步骤 3：A+，B−，C0：120º 电子。
M102= 0 ,M104=-I7000/10 ,M107= I7000/10；第 4 步:A0, B−，C+：+180º 电子。
M102=-I7000/10 ,M104= 0 ,M107= I7000/10；步骤 5:A−，B0，C+：-120º 电子。
M102=-I7000/10 ,M104= I7000/10, M107= 0；步骤 6：A−，B+，C0：-60º 电子。
M102= 0,M104= I7000/10, M107=-I7000/10；第 1 步：A0，B +，C−：0º 电子。

如果 Ixx72 小于 1024(即 683)，此测试移动电机通过换相周期在正方向上；如果 Ixx72 大于 1024（即 1365），它移动电机通过周期在负方向。在一个同步电动机中，我们可以使用位置读数来检查换相极性匹配；对于感应电机这也是可能的。

采取的操作

如果转子位置，由 M101 反映，在测试过程中在错误的方向改变，我们可能有一个换相极性不匹配。有了这样的不匹配，当被指令时电机将锁定(不是失控)。对于这种不匹配，有两种可能的修复：

1、通过改变 I7mn0 来反向反馈方向。然而，这会改变轴的方向感，可能是不容许的。

2、交换放大器和电机之间的两个相导线。通常情况下，这是在放大器上的螺丝端子上完成的。交换任意两相将以相同的方式改变极性。然而，传感器的零位和 Turbo PMAC 的换相周期的零位置之间的关系是依赖于哪两相被交换。

对于异步感应电机，如果上述测试没有引起正确的运动，尝试每个方向极性，看看哪些工作。这描述在下面。

一旦换相极性匹配已经建立，伺服极性匹配将自动建立。稍后检查这个，通过看正的 0-命令引起电机位置在正方向计数。

请记住，如果 I7mn0 随后更改来得到所需的物理方向感，交换电机相位导线来重新建立换相极性匹配。

例子

本次测试的示例运行的结果的表格是：

步骤	M102(A)	M104(B)	M107(C)	周期位置	物理位置	M104(计数)	M105(A)	M106(B)
1	+500	0	−500	0ºe	4:00	8820	< **0**	≈ **0**
2	+500	−500	0	+60ºe	3:00	9184	< **0**	> **0**
3	0	−500	+500	+120ºe	2:00	9501	≈ **0**	> **0**
4	−500	0	+500	±180ºe	1:00	9845	> **0**	≈ **0**
5	−500	+500	0	-120ºe	12:00	10218	> **0**	< **0**
6	0	+500	−500	-60ºe	11:00	10532	≈ **0**	< **0**
1	+500	0	−500	0ºe	10:00	10869	< **0**	≈ **0**

从这个测试中，我们可以得出结论：

- PWM 的操作是从根本上是工作的（我们得到了大致 6 个相等步骤）。
- 我们有一个 4 极电机，因为我们移动了 1/2 转。
- 电流 ADC 输入是工作的：我们得到了相称的响应。
- 电流的符号与电压符号是相反的，所以 Ixx72 应该是 683。
- 我们在一个周期移动了 2049 个计数，因此每转 4096 个计数应是正确的。
- 编码器计数增加，所以换向极性是正确的。

清理

当本节中的测试完成后，将零值写到命令寄存器，禁用放大器，通过该命令：

M102=0 ,M104=0 ,M107=0 ,M114=0

由于零命令进入到驱动器的所有相位，驱动器启用或禁用，ADC 寄存器应该读数几乎为零。Turbo PMAC 可以补偿非零值，通过偏移参数 Ixx79（A 相偏移）和 Ixx29（B 相偏移）。当有零的 PWM 指令时，这些偏移参数需要保持与相位的 ADC 值的符号相反的值。Ixx29 和 Ixx79 大小假设为 16 位的值；因为我们使用的是 16 位 M-变量来查看 ADC 寄存器，无论 ADC 的真实分辨率，我们只可以读取 M-变量值在零的命令，使用相反的值用于 Ixx79 和 Ixx29。

调试

随着零命令在三个输出寄存器，你可以用一个示波器观察六个 PWM 输出信号中的任何一个。所有 6 个都应具有 50％的占空比（减去 I7m04 设置的死区时间）在由 I7m00 设置的频率上。所有三个顶部-PWM 信号应与彼此同相。所有三个底部-PWM 信号应与彼此同相，并且与顶部信号有半个周期的相位差。观察顶部和底部的一对，观察顶部和底部之间的死区。

如果正命令到 A 相，负命令进入 B 相，以及 C 相为零命令，这些波形应该改变。在示波器上，观察 PWM A TOP1 的导通-时间增加，PWM A BOT1 的导通-时间减少，同时保持了死区时间。同样，PWM B TOP1 导通-时间应适当减少，并且 PWM B BOT1 的导通时间应该增加，保持它们之间的死区时间。

如果表示电流测量的模拟电压是可用的，这些可以被探测用于诊断目的。许多直接 PWM 驱动器提供模拟电流测量输出，而且 A / D 转换是在一个接口板完成的。对这些驱动器探测是很容易。如果 A / D 转换在驱动器内部完成，访问测试点来探测这些电压电平。咨询驱动器手册获取这些信号的位置和缩放。

建立基本电流回路操作

一旦已经建立了 Turbo PMAC PWM 输出电路、Turbo PMAC ADC 输入电路、他们之间所有驱动器和电机电路的正常运行，关闭电流回路。

Turbo 安装程序同时具有自动-整定和互动整定程序用于数字电流环路。大多数人使用这些程序之一或两者兼而有之来调整电流回路。这些程序还可以确认电流环读数的正确极性。然而，如本节所述，手动发出指令。

目的

这些接下来的测试的目的是设置电流环路增益，为了快速而稳定的电流响应。这是通过给电动机一个电流指令并观察电流响应来完成的。

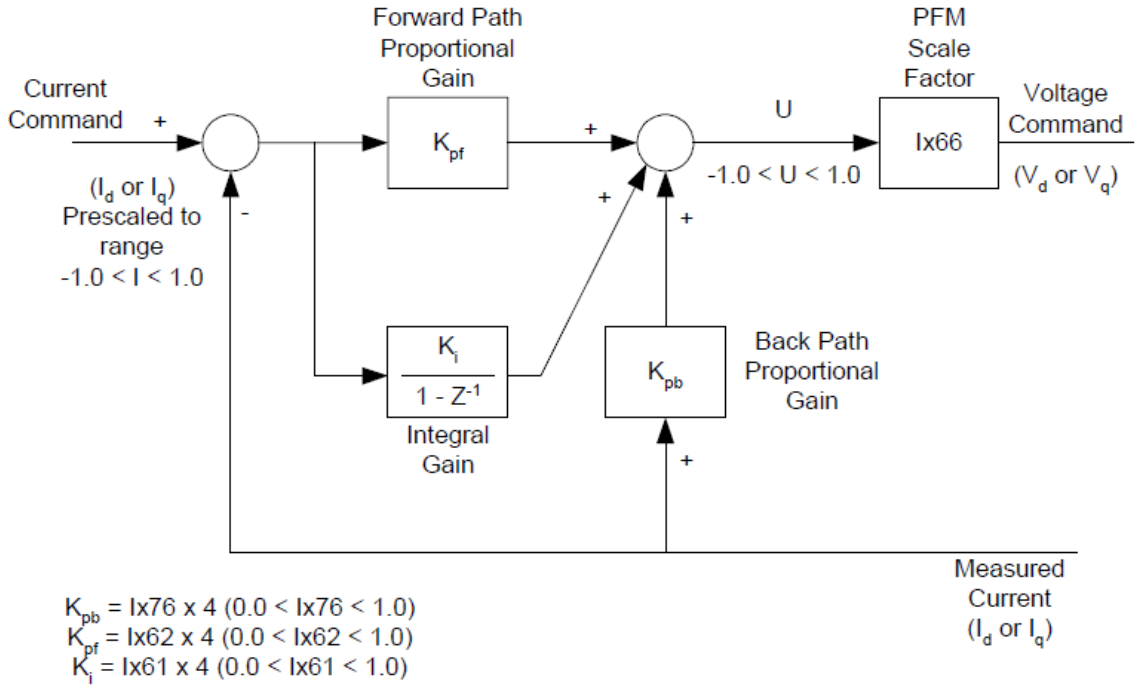
测试电流回路的关键是利用 Turbo PMAC 的 0 命令，它会关闭电流回路，同时打开位置和速度环。0-指令值的大小是产生转矩的正交电流指令的大小，表示为 Ixx69 的百分比；Ixx77 控制直流电流指令的幅度。

当测试电流回路时，最安全的是不产生任何运动；这可以只命令直流电流，通过设置 Ixx77 完成，然后发出一个 00 命令。

数字电流环增益

Ixx61、Ixx62 和 Ixx76 是 PI（比例积分）电流环的增益。它们用于多相电机的两相位。Ixx61 是积分增益项。有两个比例增益项：Ixx62 是前向-路径比例增益，Ixx76 是后向-路径比例增益。Ixx62 乘以电流误差（指令值减去实际值），结果被添加到输出命令中。Ixx76 乘以实际电流值，并从输出指令中减去。所有这三个增益项有一个 0.0～2.0 的范围，有 23-位的分辨率用于正值。

PMAC2 数字电流环路



在一个给定的电动机上，通常只使用 Ixx62 或 Ixx76 二者之一；其他增益设置为 0.0。使用 Ixx61，前向路径增益，更普遍，因为它提供更大的响应速度和带宽。如果使用 Ixx76 代替 Ixx62，只有 Ixx61 积分增益项直接连接电流命令到输出端，其积分效果滤波命令，减少了响应性。

然而，如果位置/速度伺服回路的命令是嘈杂的，如可以是低分辨率位置传感器的情况下，该滤波效果是理想的，Ixx76 可以提供比 Ixx62 更好的性能。

电流回路增益的解析计算

有了电机和放大器参数的一些基本知识，就可以直接计算电流环路增益。强烈建议，这些计算的增益可以对照通过 Turbo 安装程序的自动整定或交互整定确定的值。

所需的电机参数是：

- R_{pn} 电机相位-中性点的电阻（欧姆）= $R_{PP}/\sqrt{3}$（电机相-相电阻/$\sqrt{3}$）
- L_{pn} 电机相线-中性线电感（亨利）=$L_{PP}/\sqrt{3}$（电机相-相电感/$\sqrt{3}$）

所需要的放大器的参数是：

- I_{sat} 从相电流 A/D 转换器读取的最大的（饱和）电流（安培）。
  这是一个 DC 值，而不是一个 RMS（均方根、有效值）的 AC 值。这个值可以被导出，从电流传感器增益 Kc（伏特/安培）和 A/D 转换器可以读取的最大电压（伏特）中，V_{CMAX}:I_{sat}= V_{CMAX}/Kc。
- V_{DC} 放大器的直流总线电压。

  这可从 AC 的 RMS（均方根）电压 V_{AC} 导出：V_{DC} = V_{AC}*$\sqrt{2}$。

最后，需要的 Turbo PMAC 的参数是：

- T_P 相位更新周期（秒）
  这可以由相位更新频率 f_P 导出，KHZ：T_P= 1/（1000 * f_P）

接下来，需要电流回路的以下性能规格：

- ω_n 闭合电流回路的预期的固有频率，单位为弧度/秒
  这可从预期的固有频率 f_n 导出，单位为 Hz：ω_n（弧度/秒）=2 π f_n（Hz）。如果阻尼比（见下文）的范围是从 0.7 到 1.0，大多数情况下都是这个范围，电流回路的所需带宽基本上等于固有频率。通常使用 200Hz 至 400Hz 的值。
- ζ 期望的阻尼比（无量纲）。这里 0.7 的值产生约 5%的阶跃响应超调；这里值为 1.0 不产生超调量。

现在我们就可以根据公式计算比例电流环增益 K_{CP} 和积分电流环增益 K_{CI}：

$$K_{cp} = I_{sat}\frac{(2\xi\omega_n L_{pn}) - R_{pn}}{V_{DC}}$$

$$K_{ci} = I_{sat}\frac{T_P\omega_n^2 L_{pn}}{V_{DC}}$$

最后，计算 I-变量来表示这些增益，我们使用公式：

$$Ixx62 + Ixx76 = \frac{K_{cp} * PWM\ max\ cnt}{4 * Ixx66}$$

$$Ixx61 = \frac{K_{ci} * PWM\ max\ cnt}{8 * Ixx66}$$

在这里，PWM maxcnt（最大计数）是 I7m00 用于 PMAC Turbo 直接驱动的一个通;，对于一个 MACRO 站的通道，它是 MI900、MI906 或 MI992。

比例增益项表示为两个 I 变量的总和。Ixx62 是“前向路径”比例增益项，直接对应指令值中的变化；Ixx76 是“后向路径”比例增益项，只直接对应实际的电流值。当位置/速度环使用高分辨率的位置反馈时，电流命令的量化噪声低，使用 Ixx62 更好。当使用低分辨率的位置反馈，使用 Ixx76 更好。响应性和平滑度之间的权衡可以通过改变分配给这两个变量的比例增益项的数量来获得。

例如：

电动机具有 3.0 欧姆的相-相电阻，并且相-相电感为 39 毫亨。放大器相位-电流传感器对于 17.5 安培的电流提供他们最大 5V 的输出，ADC 对于一个 5V 的输入提供其全量程值。放大器工作需要一个 120Vrms 的 AC 电源电压。Turbo PMAC

运行在 9.03 kHz 的缺省相位更新频率。需要一个固有频率 200 赫兹、0.7 的阻尼比的电流回路。PWM 的 max-count（最大-计数）变量是在 6528 的默认值，用于电机的 Ixx66 的推荐值是 7181（大 10%）。

$$L_{pn} = L_{pp}/\sqrt{3} = 0.039/1.732 = 0.0225H$$

$$R_{pn} = R_{pp}/\sqrt{3} = 3.0/1.732 = 1.732\Omega$$

$$I_{sat} = V_{cmax}/K_c = 5.0/(5.0/17.5) = 17.5A$$

$$V_{DC} = V_{AC} * \sqrt{2} = 120 * 1.414 = 170V$$

$$T_P = 1/(1000 * f_P) = 1/(1000 * 9.03) = 0.000110s$$
$$\omega_n(rad/s) = 2\pi\omega_n(Hz) = 2 * \pi * 200 = 1256rad/s$$

$$K_{cp} = \frac{I_{sat}[(2\xi\omega_n L_{pn}) - R_{pn}]}{V_{DC}} = \frac{17.5[(2 * 0.7 * 1256 * 0.0225) - 1.732]}{170} = 3.89$$

$$K_{ci} = I_{sat} T_P \omega_n{}^2 L_{pc}/V_{DC} = 17.5 * 0.000110 * 1256^2 * 0.00225/170 = 0.401$$

$$Ixx62 + Ixx76 = (K_{cp} * PWMmaxcnt)/(4 * Ixx66) = 3.89/(4 * 1.1) = 0.884$$

$$Ixx61 = (K_{ci} * PWMmaxcnt)/(8 * Ixx66) = 0.401/(8 * 1.1) = 0.0456$$

准备实验调整

通常情况下，试验的电流回路调节是通过 Turbo 安装程序或 PMAC 执行程序进行的；二者都自动创建适当的设置进行测试。然而，要手动做测试，请按照本节中的说明。

为这个测试要准备 Turbo PMAC：

- 对于所有其它电机设置 Ixx00 为 0，不激活它们。
- 对于所有其它电机设置 Ixx01 为 0 来关闭换相。这可以确保 Turbo PMAC 有足够的计算时间足够快速地收集数据。
- 设置 I7m02 为 0 用于除以 1，使伺服更新率等于相位更新率。这允许 Turbo PMAC 在每个相位更新收集数据。）

设置待测试的电机：

- 对被测电机设置 Ixx00 为 1 来激活它。
- 设置 Ixx01 为 1 为被测电机开启相位/电流计算。早期的测试中这些被保留为零。
- 给电机一个 **k** 命令来关闭输出
- 设置 Ixx71 为 1，削弱换相算法，防止在测试过程中移动。
- 确保其他设置的 I-变量被设置为上面的指示。I7m02 和 Ixx71 只应该不同于他们在最终应用程序中的设置。
- 开始时 Ixx62=0.1、Ixx61=0.0、Ixx76=0.0 作为电流环增益。
- 设置 Ixx77 为 3000，提供约 10%的直流命令。

在数据采集的《详细绘图》部分，指定数据收集在一个伺服周期的间隔中。对于被测电机，在每个伺服周期，选择收集指令的和实际的直流寄存器。这些寄存器的地址，在下表中寻找。

指令的直流寄存器

电机#	1	2	3	4	5	6	7	8
地址	**Y:$00B8**	**Y:$0138**	**Y:$01B8**	**Y:$0238**	**Y:$02B8**	**Y:$0338**	**Y:$03B8**	**Y:$0438**
电机#	9	10	11	12	13	14	15	16
地址	**Y:$04B8**	**Y:$0538**	**Y:$05B8**	**Y:$0638**	**Y:$06B8**	**Y:$0738**	**Y:$07B8**	**Y:$0838**
电机#	17	18	19	20	21	22	23	24
地址	**Y:$08B8**	**Y:$0938**	**Y:$09B8**	**Y:$0A38**	**Y:$0AB8**	**Y:$0B38**	**Y:$0BB8**	**Y:$0C38**
电机#	25	26	27	28	29	30	31	32
地址	**Y:$0CB8**	**Y:$0D38**	**Y:$0DB8**	**Y:$0E38**	**Y:$0EB8**	**Y:$0F38**	**Y:$0FB8**	**Y:$1038**

实际的直接电流寄存器

电机#	1	2	3	4	5	6	7	8
地址	**Y:$00B9**	**Y:$0139**	**Y:$01B9**	**Y:$0239**	**Y:$02B9**	**Y:$0339**	**Y:$03B9**	**Y:$0439**
电机#	9	10	11	12	13	14	15	16
地址	**Y:$04B9**	**Y:$0539**	**Y:$05B9**	**Y:$0639**	**Y:$06B9**	**Y:$0739**	**Y:$07B9**	**Y:$0839**
电机#	17	18	19	20	21	22	23	24
地址	**Y:$08B9**	**Y:$0939**	**Y:$09B9**	**Y:$0A39**	**Y:$0AB9**	**Y:$0B39**	**Y:$0BB9**	**Y:$0C39**
电机#	25	26	27	28	29	30	31	32
地址	**Y:$0CB9**	**Y:$0D39**	**Y:$0DB9**	**Y:$0E39**	**Y:$0EB9**	**Y:$0F39**	**Y:$0FB9**	**Y:$1039**

执行电流环测试

一旦数据采集已经建立，下面的命令可以给出：

```
DEFINE GATHER          ；储备存储器用于数据采集缓冲区
＃1                    ；确保正确的电机寻址
GAT O0                 ；开始数据采集，并给出电流指令
ENDG K                 ；停止收集并结束电流指令
```

然后将数据上传和作图。其目的是获得一个对指令值的尽可能快的响应，而没有显著超调或任何不稳定。典型的电流环比例增益的范围是 0.6～0.9。典型的电流环积分增益是 0.01 左右。

清理

当这个测试完成后，恢复执行下列操作：

- 设置 I171 回到其适当值
- 设置 I7002 回到其适当值

现在应该有正常操作的电流回路。现在，使换相算法正常工作。请参阅《完成设置 Turbo PMAC 换相》的部分。

7-3 设置 Turbo PMAC 为正弦波输出控制

本节介绍，如果 Turbo PMAC 为一个电机执行换相，如何设置换相方案，但不是数字电流环路。在这种模式下，Turbo PMAC 输出两个相电流指令到放大器，通常为模拟电压，通过数字-模拟转换器（DAC）。在稳定状态下，这些电压是时间的正弦函数，所以这种模式通常被称为正弦波输出。

硬件设置

Turbo PMAC 在正弦波换相模拟输出模式中操作电机，电机需要两个模拟输出。当通过一个 PMAC2 式伺服 IC 使用时，这需要一个给定通道的 A 和 B 子通道上的 DAC。这些双-数模转换器可以在 Acc-8E 或 ACC-8A（带选项 2）板，或在 UMAC Acc-24E2A 轴接口/分线板上找到。

当通过 PMAC 式伺服 IC 使用时，这需要两个连续的通道上的单 DAC。较高（偶数）编号的 DAC 通道是 A 的 DAC；较低（奇数）编号的 DAC 通道是 B 的 DAC。

DAC 输出信号

A 和 B 的 DAC 输出应连接到正弦波输入放大器的相位指令输入。如果放大器的输入是单端的，仅使用 DAC+输出，并使互补的 DAC-输出浮动，不要将他们接地。如果放大器的输入端是互补的，同时使用 DAC +和 DAC-输出。在这两种情况下，输出连接器的 AGND 参考电压连接到放大器输入的参考电压上。

放大器启用和故障接口

在 PMAC2 型的接口和分线板上，包括 Acc-8A、8E 和 24E2A，放大器使能输出是干接点继电器。常开、常闭、下沉式或吸入式配置从 12V 到 24V 是可以选择的。常开触点（启用时闭合）被推荐为更多的故障安全操作。

在 PMAC 式板上，放大器使能输出是光隔离固态驱动器带 24V，100mA 的能力的。在大多数板上，下沉式或吸入式驱动器可以通过选择插座式驱动 IC 来选择。出厂默认的是下沉式驱动器。

放大器故障输入是光电隔离的 12V 至 24V 的输入。在较新的表面贴装集成电路设计中，隔离器是交流的光电耦合器，因此下沉式汇或吸入式源驱动器都可以使用。在旧的通孔集成电路的设计，下沉式驱动器是必需的。

编码器反馈

通常情况下，正弦波控制的完成是通过或者数字正交编码器直接连接到控制器（对于这些信号分线板只是一个直通），或模拟正弦-余弦编码器处理通过一个插值器附件。当使用一个插补的模拟编码器用于伺服反馈，通常未插补的数字编码器计数器用于换相的反馈，它不需要高分辨率的伺服。

霍尔效应换相标志

PMAC2 式伺服 IC 具有辅助标志对于每一个标记为 T、U、V 和 W 的通道。U、V 和 W 标志通常用于霍尔效应换相信号（或它们的光学等效）提供上电相位位置信息。

PMAC 式伺服器 IC 需要使用第二通道的标志（通常与第二个 DAC 是同一通道）作为辅助标志用于此目的。

Turbo PMAC 的参数设置

PMAC2 式伺服 IC 多通道设置

PMAC2 式 DSPGATE1 伺服 IC 在支持不同的硬件接口方面具有很大的灵活性。这意味着，某些寄存器必须通过 I-变量设置来支持所需的操作模式。如果使用 PMAC 式伺服 IC，这不是必需的。

硬件时钟频率控制：I7m03，MI903，MI907

I7m03（**MS{anynode}，MI903 或 MS{anynode}，MI907** 在一个 MACRO 站）决定伺服的 IC m 上的用于机械接口通道的 4 个硬件时钟信号的频率。这些也许可以保留为默认值。四个硬件时钟信号是 SCLK（编码器的采样时钟）、PFM_CLK（脉冲频率调制器的时钟）、DAC_CLK（数字-模拟转换器的时钟）和 ADC_CLK（模拟-数字转换器的时钟）。

只有 DAC_CLK 信号直接使用于正弦波输出，以控制到 DAC 的串行数据流的频率。DAC 时钟的默认频率 4.9152 MHz 是适用于所有泰道公司硬件上的 DAC。关于设置这些变量的详细信息，请参阅 I7m03 的说明。

编码器的 SCLK 频率应比任何轴的编码器可能的最大计数（边沿）率至少大 20%。可能会使用比这个最小值更高的 SCLK 的频率，但这些都使得数字延迟抗噪声滤波效果较差。

DAC 选通控制：I7m05，MI905，MI909

Turbo PMAC 会为每组四个机械接口通道产生一个共同的 DAC 选通字。这一点的完成是通过在每个相位周期移出一个 24 位字，每个 DAC 时钟周期 1 位，首先是最高有效位。I7m05（**MS{anynode}，MI905 或 MS{anynode}，MI909** 在 MACRO 站）包含这个字用于伺服 IC m 的通道。$7FFFC0 的默认值是适用于泰道公司的接口和分线板的 PMAC2 式伺服 IC 的 18 位 DAC。$7FFF00 值适用于 16 位 DAC。

单个-通道硬件设置

对用于正弦波模拟输出的伺服 IC m 的每一个机械接口通道 n（n=1～4），几个 I-变量必须正确设置。

编码器解码控制：I7mn0，MI910

I7mn0（**MS{node(节点)}，MI910** 在 MACRO 站）必须设置来正确解码换相编码器。通常情况下，使用一个 3 或 7 的值来提供一个正交编码器的 4-倍解码（编码器每线 4 个计数）。3 和 7 的不同之处在于编码器的方向感；设置此变量以便电机在期望的方向计数。

Ixx72 换相相位角参数的极性必须匹配 I7mn0 的极性对于一个特定接线；如果它是错误的，它会锁入一个位置而不产生连续转矩。确定该极性匹配的一个测试，会在下面给出。请记住，如果 I7mn0 在一个工作的电动机上变了，也改变 Ixx72。

输出模式控制：I7mn%6，MI916

I7mn6（**MS{node（节点）}，MI916** 在一个 MACRO 站）必须设置为 1 或 3 来指定通道 n 的输出 A 和 B 是在 DAC 模式下，不是 PWM。设置为 1 使输出 C（不用于伺服或换相任务，在此模式下）在 PWM 模式下，设置为 3 使输出 C 在 PFM 模式。

输出反向控制：I7mn7，MI917

I7mn7（**MS{node（节点）}，MI917** 在 MACRO 站）控制到通道 n 的 DAC 的串行数据流是否被反转。0 的默认值（非-反向）是适用于使用推荐的 Acc-8E 模拟接口板。反相串行数据流的位具有反向 DAC 电压的效果。在一个换相算法中，这相当于一个 180° 的相移，这将产生失控，如果系统在被反转之前正常工作的话。

设置电动机运行的参数

有几个 I-变量必须为每个电机 XX 设置来启用和配置电动机的正弦波输出。当然，Ixx00 必须被设置为 1 对于任何激活电机，无论电动机的输出模式，并且 Ixx01 的位 0 必须被设置为 1 以启动换相，如前面所涵盖。

命令输出地址：Ixx02

当使用 PMAC 式伺服集成电路用于正弦波控制，Ixx02 必须指定一个偶数编

号的 DAC 寄存器的地址。在这种模式下，这将导致 Turbo PMAC 同时使用这个 DAC 和下一个较低编号的 DAC（它是一个地址更高的）。两个通道必须用来换相一个马达。例如，如果指定了伺服集成电路 0 的 DAC 4 的地址，Turbo PMAC 将使用 DAC 3 和 4。下表列出了这个模式下可能的 Ixx02 值：

正弦波模式命令输出地址-PMAC 式伺服集成电路（Y 型寄存器）

IC#-通道#	0 – 1&2	0 – 3&4	1 – 1&2	1 – 3&4
Ixx02	**$078002**	**$07800A**	**$078102**	**$07810A**
IC#-通道#	2– 1&2	2 – 3&4	3– 1&2	3– 3&4
Ixx02	**$078202**	**$07820A**	**$078302**	**$07830A**
IC#-通道#	4– 1&2	4– 3&4	5– 1&2	5 – 3&4
Ixx02	**$079202**	**$07920A**	**$079302**	**$07930A**
IC#-通道#	6 – 1&2	6 – 3&4	7 – 1&2	7– 3&4
Ixx02	**$07A202**	**$07A20A**	**$07A302**	**$07A30A**
IC#-通道#	8 – 1&2	8– 3&4	9 – 1&2	9 – 3&4
Ixx02	**$07B202**	**$07B20A**	**$07B302**	**$07B309**

伺服器 IC 0 和 1 是在 Turbo PMAC 本身。
伺服器 IC2-9 是在 Acc-24P/V 或 Acc-51P 板。
奇数伺服 IC 的通道 1-4 是板上的通道 5-8。

当使用 PMAC2 式伺服集成电路用于正弦波控制，Ixx02 必须为通道指定 A 的 DAC 寄存器的地址。在这种模式下，这将导致 Turbo PMAC 同时使用通道的 A 和 B 的 DAC。下表列出了这个模式下可能的 Ixx02 值：

正弦波模式命令输出地址-PMAC2 式伺服集成电路（Y-寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx02	**$078002**	**$07800A**	**$078012**	**$07801A**	**$078102**	**$07810A**	**$078112**	**$07811A**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx02	**$078202**	**$07820A**	**$078212**	**$07821A**	**$078302**	**$07830A**	**$078312**	**$07831A**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx02	**$079202**	**$07920A**	**$079212**	**$07921A**	**$079302**	**$07930A**	**$079312**	**$07931A**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx02	**$07A202**	**$07A20A**	**$07A212**	**$07A21A**	**$07A302**	**$07A30A**	**$07A312**	**$07A31A**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx02	**$07B202**	**$07B20A**	**$07B212**	**$07B21A**	**$07B302**	**$07B30A**	**$07B312**	**$07B31A**

伺服器 IC 0 和 1 是在 Turbo PMAC2 本身或者 Acc-2E 3U-格式堆栈板上。
伺服器 IC2-9 是在 Acc-24x2 或 Acc-51E 板。
奇数伺服 IC 的通道 1-4 是双-伺服-IC 板上的通道 5-8。

当在 MACRO 环上执行正弦波形输出控制，Ixx02 必须指定 MACRO 节点寄存器的地址对于 A 的 DAC，导致使用 A 和 B 的 DAC。对于泰道的 MACRO 站所使用的 MACRO 的 1 型协议，这是节点的寄存器 0 的地址。下表列出了这个模式下可能的 Ixx02 值：

正弦波模式命令输出地址 – MACRO 集成电路（Y-寄存器）

IC#-节点#	0-0	0-1	0-4	0-5	0-8	0-9	0-12	0-13
Ixx02	**$078420**	**$078424**	**$078428**	**$07842C**	**$078430**	**$078434**	**$078438**	**$07843C**
IC#-节点#	1-0	1-1	1-4	1-5	1-8	1-9	1-12	1-13
Ixx02	**$079420**	**$079424**	**$079428**	**$07942C**	**$079430**	**$079434**	**$079438**	**$07943C**
IC#-节点#	2-0	2-1	2-4	2-5	2-8	2-9	2-12	2-13
Ixx02	**$07A420**	**$07A424**	**$07A428**	**$07A42C**	**$07A430**	**$07A434**	**$07A438**	**$07A43C**
IC#-节点#	3-0	3-1	3-4	3-5	3-8	3-9	3-12	3-13
Ixx02	**$07B420**	**$07B424**	**$07B428**	**$07B42C**	**$07B430**	**$07B434**	**$07B438**	**$07B43C**

如果使用的是较旧的MACRO类型0协议，在上表所示的值上加1（例如$078420成为$078421）。

换相相位角：Ixx72

Ixx72 设置从 A 相到 B 相的角度作为换相周期的一个分数。Turbo PMAC 拆分换相周期（360°e）为 2048 个部分。对于一个 3 相电机，从 A 到 B 的角度是一个周期的 1/3（Ixx72=683）或周期的 2/3（Ixx72=1365）。对于一个 2 相或 4 相电机，从 A 到 B 的角度是 1/4 周期（Ixx72=512）或 3/4 周期（Ixx72=1536）。

Ixx72 的正确选择取决于换相反馈编码器的方向由其接线和编码器解码变量 I7mn0 确定，也取决于电动机的各相的接线。这种选择一般是通过一个下文说明的测试进行实验确定。改变 Ixx72，也就是说，在 683 和 1365 之间等同于交换电机的两相导线。

建立基本输出操作

一个快速测试可以建立换向输出、驱动器和电机以及反馈的基本操作。本次测试使用的输出-偏移变量 Ixx29 和 Ixx79 迫使电流直接进入特定的相位并驱动电机就像一个步进电机。经测试可以用来验证-

- 输出电压从 ACC-8E 板获得
- 电流在电动机的各相位流动
- 电流使电动机锁入一个位置（它只做这些对于一个同步电机），并设置 Ixx72 换相角参数的正确的极性（它只做这对于一个同步电动机；对于一个异步感应电动机，Ixx72 的极性可能要通过试验和误差（反复试验）来确定）。

执行测试

这个测试，它可以很容易地从执行程序的终端窗口中键入一些简单的命令来完成，最好用实例说明，使用电机 1。出于安全的原因首先应当在一个空载的裸电机上进行：

```
M101-> X:$078001,24,S      ；编码器 1 相位位置寄存器
#1O0                       ；命令零输出
I129=2000                  ；第一相位的 2000 位的正偏移
M101                       ；请求位置（电机平息后）
```

```
382                    ；PMAC 的位置响应
I179=2000              ；第二相位的 2000 位的正偏移
M101                   ；请求位置（电机平息后）
215                    ；PMAC 的响应位置
```

如果伺服环反馈已经用 Ixx03 建立，位置查询 P 命令或在执行程序中的位置窗口可被用于代替 Mx01 编码器位置寄存器。

验证基本操作

将 Ixx29 设置为一个非零值应该强制一个电压在 A 相的 DAC，它可以用电压表或示波器读取。它也应强制电流在电机的匹配相位，这可以用一个电流探头进行测量。将 Ixx79 设置为一个非零值应对于 B 相的 DAC 也这样做。

当给出一个 Ixx29 偏移时，一个同步电机应锁定一个位置并保持。一个感应电动机可以短暂锁定较短的时间，由于转子的短期涡流。

当 Ixx79 偏移量增加时，同步电动机应该锁定到一个新的位置，较以前的位置远离了一部分周期。感应电机也可以做到这一点，但可能不是那么给力。

评估极性匹配

通过查看两步之间运动的方向来确定 Ixx72 的正确设置。如果该位置在负方向上改变，设置 Ixx72 小于 1024---对于 3 相电机设为 683，对于 2 或 4 相电机设为 512。如果该位置在正方向上改变时，设置 Ixx72 大于 1024---对于 3 相电机设为 1365，对于 2 或 4 相电机设为 1536。

对于这个例子中的电机，我们得出结论，如果它是一个四相电机我们想要 512 的值，如果它是一个 3 相电机就是 683 的值。如果由于系统的原因，编码器方向随后改变了，那么改变 I172 来匹配。

7-4 完成设定 Turbo PMAC 换相（直接 PWM 波或正弦波），同步电动机

至此，对于直接 PWM 控制，数字电流回路的正常运行应已建立，或基本操作应已建立用于模拟正弦波控制。换相 I-变量 Ixx70 和 Ixx71（换相周期的大小）、Ixx72（换相角度）和 Ixx83（换相反馈地址）应该已经正确设置。

接下来本节所述的步骤，对于两种类型的控制是共同的，有一共享换相算法。

确认换相极性匹配

为了 Turbo PMAC 换相算法的正常工作，输出相位的极性必须与反馈极性相匹配。如果有一个不匹配，算法将锁定电动机在一个零扭矩的点。

测试换相极性匹配

用一个同步电机，我们尝试运用一个直流指令和正交电流命令。因为我们还没有建立一个相位参考，我们不能肯定一个正交电流命令确实产生正交电流。但是，如果换相极性是正确的，这些命令中至少有一个应该引起电动机的平稳运动。

首先，我们应用一个直流命令：

```
Ixx77=3000   O0                ；～10%直流命令
```

如果这不会产生平稳运行，我们应用一个正交电流命令：

```
Ixx77=0   O10                  ；10%正交电流命令
```

为了完成测试，我们发出一个 k 命令，并确保 Ixx77 已经返回为 0。

如果这些命令中的一个产生平稳运行，换相极性是正确的，我们可以继续下一个阶段建立一个相位参考。但是，如果这些命令都不会产生稳定的运动，我们可能有换相的极性不匹配。要纠正不匹配，请参考《更正极性不匹配》，如下。

更正极性不匹配

要纠正一个换相极性不匹配，有两种可能的选择：

1、通过改变 I7mn0，来反向反馈的方向感。然而，这改变了轴的方向感，这可能是不容许的。

2、将输出方向感反向。对于模拟正弦波输出，这可以通过改变 Ixx72 进行，例如从 1365 到 683，或者通过交换放大器和电机之间的两个相位导线。对于直接 PWM，这必须通过交换相位导线完成。通常这是在放大器上的螺丝端子上完成的。交换任意两相，将以相同的方式改变它的极性。然而，传感器的零位和 Turbo PMAC 的换相周期的零位置之间的关系是依赖于交换哪两相。

通过上述方法之一改变极性匹配后，重复测试以确保该问题已得到解决。

建立一个相位参考

目的

当换相一个同步的多相电机如永磁无刷电动机时，换相算法必须知道转子的绝对位置。使用一个绝对传感器例如一个旋转变压器，相位参考必须完成只有一次，在系统的组装。使用一个增量传感器如增量式光电编码器，相位参考必须在每次系统上电时都得完成。如果增量传感器上电信号丢失，即使控制器电源被保留，相位参考必须在使能信号之前再次完成。

霍尔效应换相传感器，或同等的光学编码器，是绝对的，但是分辨率非常低（$\pm 30^{o}$e）。在一个高性能的应用，它们适合于创建一个粗略的、临时的相位基准，允许移动直到建立一个更准确的参考。

增量式编码器的索引脉冲是绝对的而且精度高，但在一般情况下，在该脉冲到达之前必须有移动。这至少需要一个粗略的定相，无论是从低分辨率绝对传感器如霍尔效应，或从一个上电定相搜索中。

警告：

对机器的安全性重要的一点是使用可靠的相位参考点的方法，无论使用绝对或增量传感器。如果相位参考是不正确的超过了相位周期的 1/4，当伺服回路关闭时会发生失控。在安装一个负载之前，用一个裸电动机仔细测试参考的相位，确保该方法是可靠的。在附加负荷之前，请确保 Turbo PMAC 故障跟随误差极限参数 Ixx11 和放大器过流故障是被激活的并正常工作。另外，还要确保所需的机械保护措施要到位。

准备

这些测试需要换向和电流回路都正常工作。仔细检查设置的变量是正确的对于这些行动，特别是早期测试中的那些可能已被更改的变量。对于电机 1，请确保：

- I100= 1 来激活电机
- I101=1，使能换相
- I170 和 I171 都被设置为正确的值

在这些测试中，我们想要访问电机相位位置寄存器，在那里 Turbo PMAC 跟踪它是在相位周期的哪个部分。相位位置寄存器是 48 位长，同时使用 X 和 Y 存储器。该寄存器的 Y-内存部分只有分数信息，所以我们将只使用 X-内存部分。它的单位是（计数* Ixx70）。寄存器是：

相位位置角度寄存器

电机#	1	2	3	4	5	6	7	8
地址	X:$00B4	X:$0134	X:$01B4	X:$0234	X:$02B4	X:$0334	X:$03B4	X:$0434
电机#	9	10	11	12	13	14	15	16
地址	X:$04B4	X:$0534	X:$05B4	X:$0634	X:$06B4	X:$0734	X:$07B4	X:$0834
电机#	17	18	19	20	21	22	23	24
地址	X:$08B4	X:$0934	X:$09B4	X:$0A34	X:$0AB4	X:$0B34	X:$0BB4	X:$0C34
电机#	25	26	27	28	29	30	31	32
地址	X:$0CB4	X:$0D34	X:$0DB4	X:$0E34	X:$0EB4	X:$0F34	X:$0FB4	X:$1034

此寄存器通常变化从-Ixx71/2 到+Ixx71/2，尽管如果监测它，有时它会跳跃 Ixx71 个单位和暂时超出这个范围。这是正常现象。访问这个寄存器对于建立一个相位参考在许多方面都有用。为电机 1 的相位位置寄存器定义建议的 M 变量：

```
M171->X:$00B4,0,24,S          ；电机 1 相位位置（计数*Ixx70）
```

将该 M-变量添加到监视窗口。

电流指令六步测试

我们这里使用的基本技术，无论是用绝对传感器的一次性相位基准或使用增量传感器的上电相位参考，都是电流命令六步试验的全部或其中一部分。这类似于上文描述的电压指令的六步测试，除了电流回路是激活的。我们使用 ADC 输入偏移寄存器来偏置相位电流反馈，和因此相位指令的输出，来在换相周期中驱动电机作为一个步进电机，到一个特定位置，通常为 0° 的位置。然后，我们可以写一个 0 到相位位置寄存器。

Ixx29 是 A 相偏移；Ixx79 是 B 相偏移。第三相是不被直接命令的；Turbo PMAC 将自动命令它作为数字电流回路的一部分，以平衡第一和第二相位。对于电机 1，电流六步测试的命令接下来的顺序和预期结果，是：

```
#1O0                    ；零幅值的开环命令
I179=3000, I129= 0      ；步骤 1：（A）0°elec，  （B）180°elec。
I179=3000 ,I129=-3000   ；步骤 2：（A）-60°elec，（B）120°elec。
I179=0 ,I129=-3000      ；步骤 3：（A）-120°elec，（B）60°elec。
I179=-3000 ,I129= 0     ；步骤 4：（A）180°elec，（B）0°elec。
I179=-3000, I129=3000   ；步骤 5：（A）120°elec，（B）-60°elec。
I179=0 ,I129=3000       ；步骤 6：（A）60°elec，  （B）-120°elec。
I179=3000 ,I129= 0      ；步骤 1：（A）0°elec，  （B）180°elec。
```

情况（A）是正确的结果对于所有直接 PWM 设置（Ixx82>0），不管 Ixx72 的设置。这是正确的结果对于正弦波输出设置（Ixx82=0）与 Ixx72<1024。情况（B）是正确的结果对于正弦波输出设置（Ixx82=0）与 Ixx72>1024。

这些命令将迫使大约最大电流的 1/10 来驱动电机到相位周期的已知位置。请记住，当你完成了这个测试，清除偏移：

I179=0 I129=0

方向-平衡 精细-定相测试

步进电机的定相测试会建立一个相位参考通常在 1 或 2 度之内。这对于许多用途是足够的，但是对于电机的相位参考的完全优化，有必要进行另一次测试，如下所述。本试验发现了最佳相位参考，通过确保关键性能指标在两个方向是相同的。通常出现在精细定相的性能改善是较好的平滑度，不是增加的扭矩。

如下面所解释，电流环路积分器寄存器仅用于直接 PWM 系统。测试仍然可以在正弦波输出系统上运行，但是在两个方向上进行比较的测量是电机速度。这可以在 PMAC 执行程序的位置窗口简单地读取。这种测量，也可在直接 PWM 系统中进行，对相位差不太敏感，如下文说明的测量，但是仍然可以引起改进提高。

这个测试对于一个给定的电动机只需要进行一次。它的目的是在电机的相位角和电机的绝对传感器（例如旋转变压器或增量编码器索引脉冲）之间建立一种关系。大多数在出厂前安装反馈装置的电机制造商都没有指定（电机相位角和传感器角度之间的）安装重复性公差要紧于 1 或 2 度，因此本次测试的结果并不需要从一台电机应用到另一个给定设计的电机中。

注意：

通常，该测试不适合用于线性电机，因为它产生相对不受控制的运动。它应该只在空载的旋转电机上来完成。在直线电机，精细定相测试可以通过调整相位寄存器完成，这样当通过一个 **O0** 命令给予 Ixx77 一个较大的值(例如 16,000）时，没有运动产生。测试应先从较小的值开始，并且用一个 **K** 命令迅速停止移动。

准备

在 PMAC 执行程序的数据采集部分的详细绘图菜单中，进行设置来采集直流积分器输出和正交积分器输出寄存器。采集周期应设置为约 10 个伺服周期。每个电机的寄存器的地址是：

直接积分器输出寄存器

电机#	1	2	3	4	5	6	7	8
地址	Y:$00BC	Y:$013C	Y:$01BC	Y:$023C	Y:$02BC	Y:$033C	Y:$03BC	Y:$043C
电机#	9	10	11	12	13	14	15	16
地址	Y:$04BC	Y:$053C	Y:$05BC	Y:$063C	Y:$06BC	Y:$073C	Y:$07BC	Y:$083C
电机#	17	18	19	20	21	22	23	24
地址	Y:$08BC	Y:$093C	Y:$09BC	Y:$0A3C	Y:$0ABC	Y:$0B3C	Y:$0BBC	Y:$0C3C
电机#	25	26	27	28	29	30	31	32
地址	Y:$0CBC	Y:$0D3C	Y:$0DBC	Y:$0E3C	Y:$0EBC	Y:$0F3C	Y:$0FBC	Y:$103C

正交积分器输出寄存器

电机#	1	2	3	4	5	6	7	8
地址	X:$00BC	X:$013C	X:$01BC	X:$023C	X:$02BC	X:$033C	X:$03BC	X:$043C
电机#	9	10	11	12	13	14	15	16
地址	X:$04BC	X:$053C	X:$05BC	X:$063C	X:$06BC	X:$073C	X:$07BC	X:$083C
电机#	17	18	19	20	21	22	23	24
地址	X:$08BC	X:$093C	X:$09BC	X:$0A3C	X:$0ABC	X:$0B3C	X:$0BBC	X:$0C3C
电机#	25	26	27	28	29	30	31	32
地址	X:$0CBC	X:$0D3C	X:$0DBC	X:$0E3C	X:$0EBC	X:$0F3C	X:$0FBC	X:$103C

执行测试

在执行此测试之前，首先用步进电机定相方法接近。然后，在 PMAC 执行的数据采集部分的终端窗口中，您可以使用下面的命令集（在命令之间等待几秒）：

```
DEFINE  GATHER          ；保留存储器,用于采集的数据
GAT  O10                ；正命令
O-10                    ；负命令
ENDG  K                 ；停止采集并停止电机
```

上传所采集的数据，并绘制直流和正交电压 VS 时间。目标是对于两个方向上的移动，平均直流电压读数是相同。正交电压在移动反转时会改变符号。要调整系统，等到测试后电机停止（M171 为常数），并对 M171 相位寄存器做一个小调整，用下面的命令：

```
M171= M171+5
```

然后根据需要重复测试，直到直流电压读数在两个方向尽可能的接近。这项测试对一个计数的相位误差敏感，所以最后的改变也许应该是±1 个计数。

使用测试结果用于绝对传感器

这个测试是有用的，仅仅在超精确的相位位置匹配到一个绝对位置传感器或一个增量传感器的索引脉冲时。在一个绝对传感器，分配一个 M 变量到传感器寄存器，并添加到监视窗口。例如：

M175-> TWR: 0,0；第一个 Acc-8D Opt7 R/D 的第一个旋变的绝对值.位置.

确保电机完全静止。现在，将读取的传感器位置值乘以 I170，并从 M171 读的相位位置中减去这个。（如果马达被手动移动从而 M171= 0，乘积可以被反向）。使用下面的语句，输入此值到 I175：

```
I175= M171-(M175* I170)
```

最后，设置 I181 来读取后续的 Turbo PMAC 复位时的绝对传感器并使用 **SAVE** 命令存储这些值。在这个电机上执行另一个相位参考。

使用测试结果用于增量索引脉冲

对于增量编码器索引脉冲，我们将使用位置捕捉功能来记录该索引的位置。如果有一个高-真索引脉冲，设置变量 I7mn2（**MS{node(节点)}，MI912** 对于一个 MACRO 站）为 1，或如果有一个低-真索引脉冲，设为 9。（要了解是哪个，定义 Mxx19 到建议的定义并把它在监视窗口中。一般来说如果是 0 会有一个高-真脉冲）。对一个 PMAC2 式伺服 IC，为确保有效的索引脉冲仅是 1 个计数宽，设置 I7mn4（**MS{节点}，MI912，** 对于一个 MACRO 站）为 1， I7mn5（**MS{节点}，MI912，** 对于一个 MACRO 站）设为编码器适当的值。

现在，分配一个 M-变量到编码器标志捕捉寄存器：

```
M103-> X: $078003,0,24,S          ；编码器 1 标志捕捉寄存器
```

将这个添加到监视窗口。电机处于静止状态，注意 M171 中相位位置值和 M101 中的编码器位置寄存器。记下这些值。现在，在回零的方向手动转动/推动电机，直到 M103 变化。新的值是编码器寄存器在索引脉冲捕捉到的值

从这个新的 M103 值中减去你的初始 M101 的值。将差额与 I170 相乘并将它添加到初始 M171 值中。其结果值我们将写入相位位置寄存器，当我们停在索引处以改进我们的初步粗略定相时。从数学上说：

$$\text{索引相位位置} = \text{I170} * (\text{索引 M103} - \text{初始 M101}) + \text{初始 M171}$$

或者，在一个数学上比较容易但是物理上更难的技术中，把 M119 放在监视窗口（或索引信号在示波器上），然后转动电机轴直到它停止在索引脉冲。读取 M171 相位位置寄存器的值。该值我们要写入相位位置寄存器，当我们停在索引处以改进我们的初步粗略定相时。

使用霍尔效应传感器用于相位参考

霍尔效应传感器或换相编码器中它们的光学等价，对于一个 3 相电机，可用于在上电时粗略定相而不需要一个定相位搜索移动。此初始定相提供合理的转矩，但它需要进行校正用于顶级操作。通常当到达索引脉冲时，校正完成，以上文描述的相同的技术用于校正，在一个上电定相搜索移动后。

霍尔效应传感器通常映射出 6 个区域，每个 60º 电子。就 Turbo PMAC 的换相周期而言，边界应该在 180°、-120°、-60°、0°、60° 和 120°。一个典型的电机制造商将传感器对准在该边界的几度范围内，因为这些是正确的边界点如果所有的换相是从换相传感器中完成的。如果安装霍尔效应传感器，将边界小心地对准在这些点。最简单的方法是用一个电流偏移将电机强制到零度点（如上所示），并调整传感器同时观察其输出来得到尽可能接近这点的一个边界。

准备

定义 M 变量到霍尔效应或其等价输入。一个 PMAC2 式伺服 IC 的通道 1 上的建议的定义是：

```
M124-> X: $078000,20          ;通道 1 W 标志
M125-> X: $078000,21          ;通道 1 V 标志
M126-> X: $078000,22          ;通道 1 U 标志
M127-> X: $078000,23          ;通道 1 T 标志（不是通常霍尔）
M128-> X: $078000,20,4        ;通道 1 TUVW 作为一个 4 位值
```

做出这些定义并将这些变量添加到监视窗口。（删除其他不再监视的变量。）在电机停止时，用手慢慢移动电机来验证这些预期会改变的输入确实会改变。

执行测试

要映射霍尔效应传感器，我们将使用电流环六步测试，或者它的一个变型，强制电机到换相周期的已知位置，并观察霍尔效应信号的状态。上文显示的电流回路测试应强制电机正好在霍尔效应区的边界。如果您使用这些命令，用手一点点移动电机在每个点来观察过渡。

或者（或此外）强制电机到每个霍尔效应区的预期中点。要做到这一点，命令序列是：

```
#1O0                          ;零幅值开环命令
I179=3000  I129=-1500         ;步骤 1：（A）-30° 电子 （B）150° 电子。
```

```
I179= 1500    I129=-3000          ;步骤 2：（A）-90° 电子 （B）90° 电子。
I179=-1500    I129=-1500          ;步骤 3：（A）-150° 电子（B）30° 电子。
I179=-3000    I129=1500           ;步骤 4：（A）150° 电子（B）-30° 电子。
I179=-1500    I129=3000           ;步骤 5：（一）90° 电子（B）-90° 电子。
I179= 1500    I129=1500           ;步骤 6：（A）30° 电子 （B）-150° 电子。
I179=3000     I129=-1500          ;步骤 1：（A）-30° 电子（B）150° 电子。
```

情况（A）是正确的结果对于所有直接 PWM 设置，无论 Ixx72 的设置。这是正确的结果对于正弦波输出的设置而 Ixx72<1024。情况（B）是正确的结果对于正弦波输出的设置而 Ixx72>1024。

请记住，当测试完成后，清除偏移：

```
I179=0      I129=0
```

明智的做法是创建一个表列出对于每个位置，M124 到 M128 的数值。一个示例表格是：

步骤	M179	M129	周期位置	物理位置	M101 (计数)	M126 (U)	M125 (V)	M124 (W)	M128 (TUVW)
1	+3000	-1500	-30°	3:30	-9001	0	1	0	2
2	+1500	-3000	-90°	2:30	-9343	1	1	0	6
3	-1500	-1500	-150°	1:30	-9673	1	0	0	4
4	-3000	+1500	+150°	12:30	-10030	1	0	1	5
5	-1500	+3000	+90°	11:30	-10375	0	0	1	1
6	+1500	+1500	+30°	10:30	-10709	0	1	1	3
1	+3000	-1500	-30°	9:30	-11050	0	1	0	2

注意：

如果 T 标志输入为 1，Mx28 的值将比表中所示值大 8。

使用测试结果

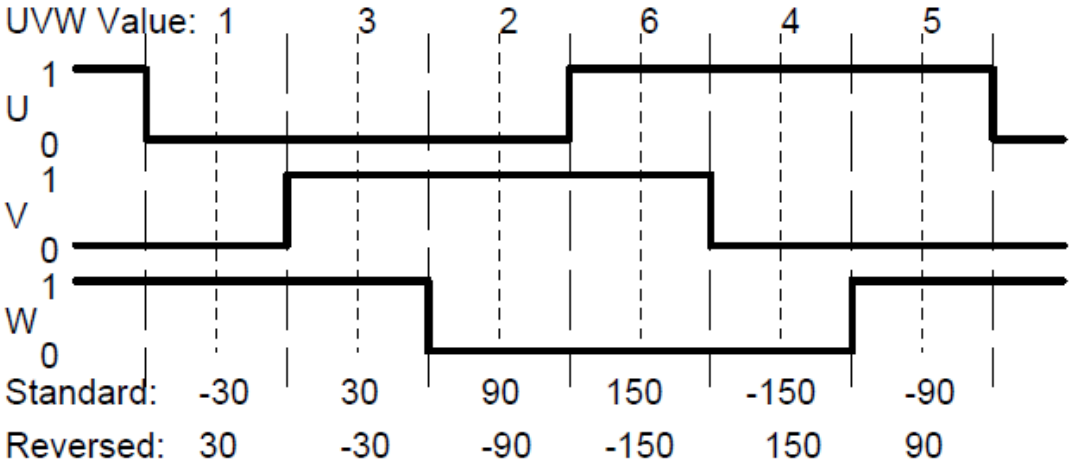
要使用霍尔效应传感器执行一个上电定相，可以使用 Ixx81 上电相位位置参数的新模式，或写一个简单的 PLC 程序在上电或复位时执行一次。

将 Ixx81 的 23 位设置为 1，来指定一个霍尔效应上电相位参考。在这种情况下，Ixx81 的地址部分指定一个 Turbo PMAC X-地址，通常是用于电机的标志寄存器的，与在 Ixx25 中相同的地址。

Turbo PMAC 期望找到霍尔效应的输入在指定的寄存器的 20、21 和 22 位。在一个标志寄存器，这些位分别匹配 CHWn、CHVn 和 CHUn 输入。霍尔效应输入传统地被标记为 U、V 和 W。

每个霍尔效应信号必须有 50%的占空比（180° e）。PMAC 可以使用由 120° e 分隔的霍尔效应换相传感器。霍尔效应传感器的方向感或零基准没有行业标准，因此必须通过 Ixx81 软件设置来处理。

如下图所示， 22 位控制霍尔效应传感器的方向感，其中 22 位的位值为 0 是标准的，位值为 1 是反向的：



该图显示了零偏移的霍尔效应波形，这样的定义，当 U-信号为低电平时（定义为霍尔效应周期的零点）V 型信号转换代表 PMAC 的换相周期的零点。

如果霍尔效应传感器不具有这种定向，Ixx81 的 16 位至 21 位可以被用来指定 PMAC 的零点和霍尔效应零点之间的偏移量。这些位可以取的值为 0～63，单位是换相周期的 1/64（5.625° e）。

该偏移量可以使用上面所示的映射测试来计算。在我们的例子中，霍尔效应零（HEZ）点被认为是在 30° e 和 90° e 之间，所以我们将其称为 60° e。偏移值可以被计算为

$$偏移 = \frac{HEZ\%360°}{360°} * 64$$

这里计算的偏移量应四舍五入到最接近的整数。

在我们的例子中，这涉及到：

$$偏移 = \frac{60°\%360°}{360°} * 64 = \frac{60°}{360°} * 64 = 10.667 \approx 11 = 0B \quad hex(16\ 进制)$$

试验表明，霍尔效应传感器是在标准方向，而不是反转的，所以 22 位保留为零。23 位(在第一个十六进制数字的值为 8)设置为 1 以指定霍尔效应传感器，Ixx81 的前两个十六进制数字变成$B5。如果在地址$C000 的标志寄存器 1 被用于霍尔效应的输入，Ixx81 将被设置为$B5C000。

软件参考手册中 Ixx81 的的描述给出了偏移所使用的通常值，用于所有的情况---霍尔效应周期的零点是在 0°、60°、120°、180°、-120°，或者-60°点---制造商通常校准传感器在其中。

注意：

在 Turbo 的 Ixx81 仅用于地址（即与 Ixx25 相同）。在 Turbo 的 Ixx91 用于 16-21，22 和 23 位。

注意，Ixx75 不以该方法用于相位位置的偏移。它可以被用于存储基于精细定相的最终校正。

整个过程总结

整个相位基准由以下步骤组成：

1、使用 Ixx81 所指定的霍尔效应传感器做一个粗略的相位参考，要么自动执行在上电或复位如果 Ixx80=1，或在**$**命令如果 Ixx80=0。

2、对电机做一个回零搜索移动，使用索引脉冲作为归零触发的一部分。

3、等待电机设置“进入-位置”（跟随误差小于 Ixx27），在零位置使用电机就-位状态位---建议的 M 变量 Mxx40---[**WHILE（M140=0）...**]

4、迫使电机相位位置寄存器为这一点的预定值，通过类似 **Mxx71=Ixx75** 的命令。

基于 PLC 的霍尔效应参考

另外，上电 PLC 程序可以用来执行霍尔效应定相。这是有用的，如果需要附加误差捕获，或者如果使用不同格式的传感器。

根据我们的示例表的结果，程序是：

```
;****************** Set-up and Definitions *********************
CLOSE                              ; Make sure all buffers are closed
M148->X:$0000C0,8,1                ; Motor 2 phasing error fault bit
M171->X:$0000B4,0,24,S             ; Motor 2 phase position register
;****************** Program to do phasing search **************
OPEN PLC 1 CLEAR
M148=1                             ; Tentatively set phasing error bit
IF (M128&7=2)                      ; Hall Effect State 1 (0 to -60 deg)?
   M171=I171/-12                   ; Set phase angle to -30 deg
   P170=1                          ; Phasing OK flag
ENDIF
IF (M128&7=6)                      ; Hall Effect State 2 (-60 to -120 deg)?
   M171=I171*-3/12                 ; Set phase angle to -90 deg
   P170=1                          ; Phasing OK flag
ENDIF
IF (M128&7=4)                      ; Hall Effect State 3 (-120 to -180 deg)?
   M171=I171*-5/12                 ; Set phase angle to -150 deg
   P170=1                          ; Phasing OK flag
ENDIF
IF (M128&7=5)                      ; Hall Effect State 4 (180 to 120 deg)?
   M171=I171*5/12                  ; Set phase angle to 150 deg
   P170=1                          ; Phasing OK flag
ENDIF
IF (M128&7=1)                      ; Hall Effect State 5 (120 to 60 deg)?
   M171=I171*3/12                  ; Set phase angle to 90 deg
   P170=1                          ; Phasing OK flag
ENDIF
IF (M128&7=3)                      ; Hall Effect State 6 (60 to 0 deg)?
   M171=I171/12                    ; Set phase angle to 30 deg
   P170=1                          ; Phasing OK flag
ENDIF
```

```
IF (M128&7=0 OR M128&7=7)          ; Invalid states
   P170=0                          ; Phasing not OK
ENDIF
IF (P170=1)                        ; Phasing OK?
   M148=0                          ; Clear phasing error bit
   CMD"#1J/"                       ; Enable motor
ELSE
   CMD"#1K"                        ; Not OK; disable motor
ENDIF
DISABLE PLC 1                      ; So program will not repeat
CLOSE
```

关于这个程序的注释：

- &7（和 7[0111]进行逐位 AND 运算）的操作的原因是删除 T-标志输入的效果，它是 4 位 M 变量的最高有效位。
- 这个定相估计有±30° e 的潜在误差。

这个试探性的定相将一直服务到我们移动电机到编码器索引脉冲，届时我们将强制 M171 设为我们通过精细定相发现的值。

上电定相搜索

如果一个非绝对的传感器用于换相，在它每一次上电时，Turbo PMAC 必须执行一个搜索移动为正确的定相参考（对于一个绝对的传感器，这仅需要在系统开发中做一次）。有几种方法可以做到定相搜索。Turbo PMAC 具有两种由固件自动执行的方法；其他的方法或这些方法的增强功能可通过 PLC 的程序执行。

一个上电的定相搜索允许对永磁无刷电动机的换相，而不需要更昂贵的并且可能不准确的绝对传感器。然而，在一些应用中一个定相搜索可能并不可靠，在这些情况下绝对传感器是必需的。

对上电定相搜索的估计应该在真正的零点位置±1-2° e 范围内，所以很多人会在整个应用程序中使用这里建立的相位。也可以在索引脉冲停止时调整估计，通过使用上述的精细定相测试的结果。

警告：

一个不可靠的定相搜索方法可能导致一个失控状态。仔细测试定相搜索方法，以确保它在所有可能的条件下正常工作。确保 Ixx11 故障跟随误差限制是激活的，并尽可能地紧密，使电机可以快速地在严重的定相搜索错误的情况下被停止。

当正确定相时，正 O-命令应该在正方向引起运动，负的 O-命令应该引起在负方向上的运动。如果接收到相反的结果，当伺服回路被关闭时会有一个危险的失控状态。

双-猜的定相搜索

Turbo PMAC 的第一个自动定相搜索方法被称为双-猜定相搜索，因为它对相位位置做出两个任意猜测，简单地使用每个猜测施加一个扭矩命令，观察电机对

每个命令的响应。基于所述两个响应的幅度和方向，Turbo PMAC 计算正确的相位参考点。然后，它根据这个参考开始换相，并关闭伺服回路以保持位置。

双-猜定相搜索快速且只需要很少的运动。外部载荷如重力和摩擦力低，它运作良好。但是，如果有显著的外部负载，它可能被证明是不可靠的定相搜索方法（不可靠的定相搜索方法是危险的），如果是这样的话，应该使用另一种方法，如下面所述的步进电机的方法。

双猜测方法的选定是通过将 Ixx80 设为 0 或 1。在 Ixx80 为 0 时，相位搜索在上电/复位周期中不会自动执行；必须使用一个$命令来执行定相搜索。在 Ixx80 为 1 时，定相搜索会自动在上电复位周期内执行，当然，也可以使用$命令在随后执行。

必须指定两个参数来告诉 Turbo PMAC 如何执行这定相搜索。Ixx73 指定在每一个猜测中转矩指令的幅度，以 16 位的 DAC 位的单位。典型值是 2000 到 6000；4000（约满量程的 1/8）是一个通常的起点。Ixx74 设置每个转矩指令的持续时间及其响应的评价，以伺服周期的单位。典型值是 3 到 10；5（缺省伺服更新大约 2 毫秒）是一个通常的起点。

步进电机定相搜索

同步电动机的定相搜索的另一自动方法是步进电机的方法。此方法强制电流通过电机的特定相位，如同一个步进电机控制器，并等待它稳定下来。有了正确的操作，这将会在换相周期的一个已知位置。这种方法相当于上述电流回路六步测试的两步。

步进电机定相搜索需要比双-猜的方法更多的运动和更多的时间，但它对于大的外部负载下精确找到相位更可靠。

步进电机的方法是通过设置 Ixx80 为 2 或 3。在 Ixx80 为 2 时，定相搜索不会在上电/复位周期自动执行；必须用一个$命令来执行定相搜索。在 Ixx80 为 3 时，定相位搜索会自动在上电复位周期执行（不推荐这样做），也可以随后用$命令执行。

在该方法中，Ixx73 控制通过各相的电流的大小，全量程用 32767 表示。通常情况下使用 3000 附近的值，约满量程的 1/10，尽管实际值将取决于负载。

Ixx74 控制搜索中所用的两步中每个的稳定时间。在这种模式下，Ixx74 的单位是伺服周期*256，在缺省伺服周期时间，约是 1/10 秒。通常使用 1-2 秒的稳定时间。

在步进电机的定相搜索，Turbo PMAC 首先强迫电流使电机放在相位周期的 +/-60° 的点，然后等待稳定时间。然后它迫使电流使得电机在相位周期的 0° 点，并再次等待稳定时间。它会检查两步之间出现了至少 1/16 周期的（22.5° ）运动。如果出现了，它迫使相位位置寄存器为 0，清除定相-搜索-误差电机状态位，并关闭伺服回路。如果它检测到比这少的运动，它会设置定相-搜索-误差位，并禁用（或停止）伺服回路。

如果步进电机定相搜索是在上电/复位周期之外完成的，在检测到放大器故障或超程限制条件时，定相搜索算法将会失败。Turbo PMAC 将设置定相搜索误差位并禁用伺服回路。如果是在上电/复位周期内完成，Turbo PMAC 不能自动检测这些错误，但搜索可能会因缺少运动而失败。

自定义定相搜索方法

编写一个自定义的相位搜索算法可能是必要的或可取的。通常，这些是作为 Turbo PMAC PLC 程序执行的，但他们往往可以使用在线命令进行试运行和调试。

在线命令是特别有用的，如果定相位搜索只在开发中进行以为一个绝对传感器建立参考的话。

大多数自定义算法是对步进电机相位搜索方法的变型。他们使用相电流偏移值 Ixx29 和 Ixx79，用一个 O0 命令来强制电流进入特定的相位，因此电动机会锁定在其相位周期的一定的物理位置。下表显示了对于 3 相电机，Ixx29 和 Ixx79 的不同组合产生的相位周期的位置。通常非零值的大小是 2000 到 3000：

Ixx29	=0	<0	<0	=0	>0	>0
Ixx79	>0	>0	=0	<0	<0	=0
(A)相位位置	0°e	-60°e	-120°e	±180°e	120°e	60°e
(B)相位位置	±180°e	120°e	60°e	0°e	-60°e	-120°e

情况（A）显示的产生的位置，对于所有直接 PWM 系统和 Ixx72 <1024 的正弦波输出系统。

情况（B）显示正弦波输出系统在 Ixx72>1024 时，产生的相位位置。

例如，下面的一组在线的命令，键入到执行程序的终端窗口，可以用来强制电机到其相位周期的零位置，设置相位位置寄存器为零，使能电机。

```
#1O0                    ; 使能电机，开环零幅值
I129=0                  ; A 相无偏移
I179=3000               ; B 相上的正偏移强制为 0 度
M171=0                  ; 写零到相位寄存器
I179=0                  ; B 相无偏移
J/                      ; 关闭伺服回路
```

键入命令之间的时间将提供足够的延迟用于设置到位置。

下面的 PLC 程序是一个很好的起点，对于步进电机定相搜索方法的变型。扩展这个程序可能是同时定相 2 个龙门电机或步进出一个位置极限。这个例子使用 Ixx73 和 Ixx74，如同他们将在自动步进电机定相搜索方法中使用。

```
;****************** Set-up and Definitions *********************
CLOSE                           ; Make sure all buffers are closed
M248->X:$000140,8,1             ; Motor 2 phasing error fault bit
M271->X:$000134,0,24,S          ; Motor 2 phase position register
;****************** Program to do phasing search ***************
OPEN PLC 1 CLEAR
CMD"#2O0"                       ; Force zero-magnitude open-loop
M248=1                          ; Tentatively set phasing error bit
P229=I229                       ; Save real Phase A bias
P279=I279                       ; Save real Phase B bias
I229=-I273                      ; Force negative bias into A
I279=I273                       ; Force positive bias into A
I5111=I274*256                  ; Starting value for countdown timer
WHILE (I5111>0)                 ; Wait for prescribed time
ENDWHILE
P271=M271                       ; Store phase position at this point
I229=P229                       ; Restore real bias to A for 0 deg
I5111=I274*256                  ; Starting value for countdown timer
WHILE (I5111>0)                 ; Wait for prescribed time
```

```
ENDWHILE
P271=P271-M271                  ; Get difference between two positions
IF (I282>0 OR I272<1024)        ; Direct PWM or check analog phase
    M271=0                          ; Set phase position to zero
ELSE                            ; Analog system with Ixx72>1024
    M271=I271/2                 ; Set phase position to 180 deg
I279=P279                       ; Restore real bias to B
IF (ABS(P271)>I271/12)           ; Greater than 1/12 cycle?
  M248=0                        ; Clear phasing error bit
  CMD"#2J/"                     ; Close servo loop
ELSE                            ; Not enough movement
  CMD"#2K"                      ; Bad phasing, kill
  SEND"PHASING FAILED"
DISABLE PLC 1                    ; Keep from executing again
CLOSE
```

警告：

确保这种类型的一个算法可以可靠地执行。如果位置传感器或驱动器断电或故障，不要尝试这个算法。Turbo PMAC 不允许该 PLC 要求开环启用状态，如果它是进入超程限制。通过 Ixx13、Ixx14 和 Ixx24 禁用自动的超程限位功能。在全定相完成之前，需要特殊的逻辑步进出一个极限。请记住，不正确的相位参考可能会导致失控的情况。确保 Turbo PMAC 故障的跟随误差限制 Ixx11 和放大器过流故障保护是激活的且是工作的。

用索引脉冲进行最终相位校正

为了做出最终的相位校正，Turbo PMAC 电机移动到索引脉冲，迫使精细定相测试中得到的值进入到相位位置寄存器。在此之前，该位置/速度伺服回路必须合理地整定；对此环的调整与任何其他 PMAC 电机的都是一样的。

通常情况下，移动到索引脉冲会是回零搜索移动，其中回零位置的触发包括索引脉冲的前沿；通常第一个索引脉冲在回零标志脉冲内。I 变量 I7mn2 和 I7mn3 控制回零触发。通常情况下，初步的定相将允许一个合理的运动一路到起始零位置。另外，也可以进行初步回零搜索到第一个索引脉冲，校正相位，然后执行实际的回零搜索。

一旦抵达换相周期中一个已知的位置，使用 **SETPHASE** 命令强制一个预定值进入相位位置寄存器。当执行 **SETPHASE** 命令，电动机参数 Ixx75 的值立即复制到相位位置寄存器。该 **SETPHASE** 命令可以用来作为一个在线的命令，在一个运动程序中，或在一个 PLC 程序中。

一个示例的运动程序段，执行回零搜索和相位校正：

```
HOME1                          ;命令回零搜索移动
WHILE (M140=0) WAIT            ;循环直到就位
SETPHASE                       ;强制值进入相位位置寄存器
```

7-5 完成设定 Turbo PMAC 换相（直接 PWM 波或正弦波），异步（感应）电机

一个交流感应电机的 Turbo PMAC 换相需要设置两个 I 变量可以保留为 0 值对于永磁无刷电机。一个变量是 Ixx77 磁化电流参数（对于永磁电机，通常是保留为 0 值但能改变他们）。另一个变量是 Ixx78 转差率-增益参数（对于永磁电机，该参数必须保留在 0 值）。

典型地，Turbo 安装程序可用于自动设置 Ixx77 和 Ixx78。该程序刺激感应电动机来推断它的参数，并对于它得到的结果适当设置这些项。本节介绍设置这些参数的分析和实验方法。这些设置独立于任何机械负载，因此任何测试可以并且应该用一个空载电机来完成。

计算 Ixx78 转差率常量

从铭牌数据计算

对于一个感应电动机的转差率常量参数 Ixx78 可以简单地从电动机和 Turbo PMAC 的基本参数中进行计算。以下是必要的：

- 电机的额定转速，通常以每分钟的转数（rpm）给定。
- 这个额定速度下给定的电气线路频率，通常是赫兹（Hz），或者周期/秒。
- 电机的极数。
- Turbo PMAC 的相位更新周期，一般是以微秒（μs）给出。

（转换为一致的单位后）可以从线路频率中减去额定速度，以获得转差率频率。这可以乘以相位更新周期（转换为一致的单位后），以获得 Ixx78 转差率。其计算公式为：

$$Ixx78 = (\omega_e - \omega_m) * T_P * \frac{I_{mag_std}}{32{,}768}$$

其中：

频率 ωe 是给定的电气频率，单位为弧度/秒。要从频率 Hz 计算时，乘以 2π（6.283）。

ωm 是额定机械极点频率，单位为弧度/秒。要从电机额定转速 rpm 和极数计算时，将速度 rpm 除以 60，与 2π（6.283）相乘，再乘以极点数并除以 2。

T_P 是以秒为单位的 Turbo PMAC 的相位更新时间。要从微秒转换时，除以一百万。对于一个 Turbo PMAC2，相位更新时间可以计算为：

$$T_p = \frac{[2 * I(I19 - 7) + 3] * [I(I19 - 6) + 1]}{11796480}$$

其中 I19 是 Turbo PMAC2 参数，包含了作为系统的相位和伺服时钟的源的伺服 IC 或 MACRO IC 时钟方向 I-变量的编号数目。通常对于一个 Turbo PMAC2

UltraLite I19，它被设置为 6807 用来指定 MACRO IC0（因此 I6800 和 I6801 设置相位更新）；通常对于一个板级的 Turbo PMAC2，它被设置为 7007 来指定伺服 IC0（所以 I7000 和 I7001 设置相位更新）；对于一个 UMAC Turbo，通常它被设置为 7207 来指定伺服 IC2（所以 I7200 和 I7201 设置相位更新）。

I_{mag_std} 是磁化电流参数 Ixx77 的值，将产生相同的额定转速/转矩点与直接操作交流线。对于第一次计算，在这里使用 3500 的值。通常，它是足够接近的。对于这种类型的操作，如果 Ixx77 的值被设置为下一节中说明的那样，回来并调整计算。

例如：

一个 4 极感应电动机在 60Hz 的电气频率具有 1740 rpm 的额定速度。它从一个 UMAC Turbo 上通过默认时钟源和频率控制。电气频率为：

$$\omega_e = 60\left(\frac{\text{周期}}{s}\right) * 2\pi\left(\frac{rad}{\text{周期}}\right) = 377.0\left(\frac{rad}{s}\right)$$

机械极点频率为：

$$\omega_m\left(\frac{rad}{s}\right) = 1740\left(\frac{\text{转}}{min}\right) * \frac{1}{60}\left(\frac{min}{s}\right) * 2\pi\left(\frac{rad}{\text{周期}}\right) * 4\left(\frac{\text{极点}}{\text{转}}\right) * \frac{1}{2}\left(\frac{\text{周期}}{\text{极点}}\right)$$

$$= 364.4\frac{rad}{s}$$

在一个 UMAC Turbo 上，默认的时钟源是伺服 IC2（I19=7207）。I7200 的默认值是 6527，而 I7201 的默认值是 0。相位更新时间可以计算如下：

$$T_p = \frac{(2 * 6527 + 3) * (0 + 1)}{117964800} = 0.000111s$$

Ixx78 现在可以计算如下：

$$Ixx78 = (377.0 - 364.4) * 0.000111 * \frac{3500}{32768} = 0.000149$$

从转子时间常数计算

偶尔，感应电动机的鼠笼式转子的 L/ R 电气时间常数可从制造商处获得（这区别于定子绕组的 L/ R 电气时间常数，并且比其大得多）。该 Ixx78 转差率可以很容易地从这个值来计算，公式如下：

$$Ixx78 = \frac{T_p}{T_r}$$

其中，Tp 是 Turbo PMAC 的相位更新时间，Tr 为转子的电气时间常数。请记住，两个时间使用相同的单位。

例子

如果以 8 kHz 的相位更新频率运行，并且转子时间常数是 0.75 秒，计算：

$$Ixx78 = \frac{T_p}{T_r} = \frac{0.000125}{0.75} = 0.000167$$

实验地优化转差率常数

对于一个给定的磁化电流，最优转差率常数将最大化电机的加速度能力。对 Ixx78 的最优值的任一方向的变动将使性能退化。简单的测试采用数据采集同时使用一个低值 O-命令（如 **O10**）来加速发动机，允许简单优化或验证优化 Ixx78 值。如果没有选择磁化电流 Ixx77 的最佳值，使用 3000 的值进行这些测试。

设置 Ixx77 磁化电流

一旦 Ixx78 转差率常数的值很好，找到磁化电流 Ixx77 参数的最佳值。Ixx77 为换相中的直流分量设置指令值，该分量与转子的测量/估计的磁场方向同相。Ixx77 对于电机决定了转子的磁场强度以及转矩常数 Kt 和反电动势常数 Ke。如果 Ixx77 的值不够大，尚未磁饱和转子，转矩和反电动势常数将正比于 Ixx77。

Ixx77 的值越高（饱和之前），则伺服环指令的每单位的正交电流产生的扭矩越多，但每单位的电机速度产生的反电动势发生器电压越高，所以一个给定的电源电压所实现的最大速度越低。Ixx77 的值越低，每单位的正交指令电流的产生的转矩越少，但每单位速度产生的反电动势电压越低，因此可以达到的速度越高。

在大多数应用中，Ixx77 将被置为一个单一的值并保持不变对于应用程序。然而，可以动态地改变 Ixx77 作为速度的函数，在高速时降低它以保持反电动势在电源电压下，扩展电机的转速范围。一般来说，这种技术被称为磁场削弱“，并可以在 PLC 程序中来实现。

如果 Turbo 安装程序不用于设置 Ixx77 磁化电流参数的值，最好是实验性地这样做。设置一个好的 Ixx78 值，只需在每个 Ixx77 的设置上发出一个低值 0-命令（如 **O10**），并观察空载电机达到的终点速度。这可以通过观看执行程序的位置窗口中的实时的速度读出来完成。如果使用数据采集功能，还要注意到该速度的加速率。

这个速度被称为该设置下的电机的基本速度。通常情况下，当直接从 50 赫兹或 60 赫兹运行时，一个 3200～3500 的 Ixx77 值将实现一个约等于电机的额定转速的基本速度。

如果 Ixx77 的测试值足够的低，以至于他们均没有磁饱和转子，试验的基本速度将约成反比于 Ixx77 的值（以及，到该速度的加速度将大致正比于 Ixx77）。如果开始增加 Ixx77 进入到引起转子磁饱和的范围内，增加 Ixx77 不会造成基本速度进一步降低，也不会引起到该速度的加速率进一步增加。

很多用户都希望 Ixx77 的值尽可能高，而同时又不会造成转子饱和。这些用户会希望找到那些导致饱和的 Ixx77 的值，然后减小 Ixx77 使得其值刚好不引起转子饱和。而 Turbo 安装程序会自动查找此设置。

7-6 电动机的直接微步进（开环换相）

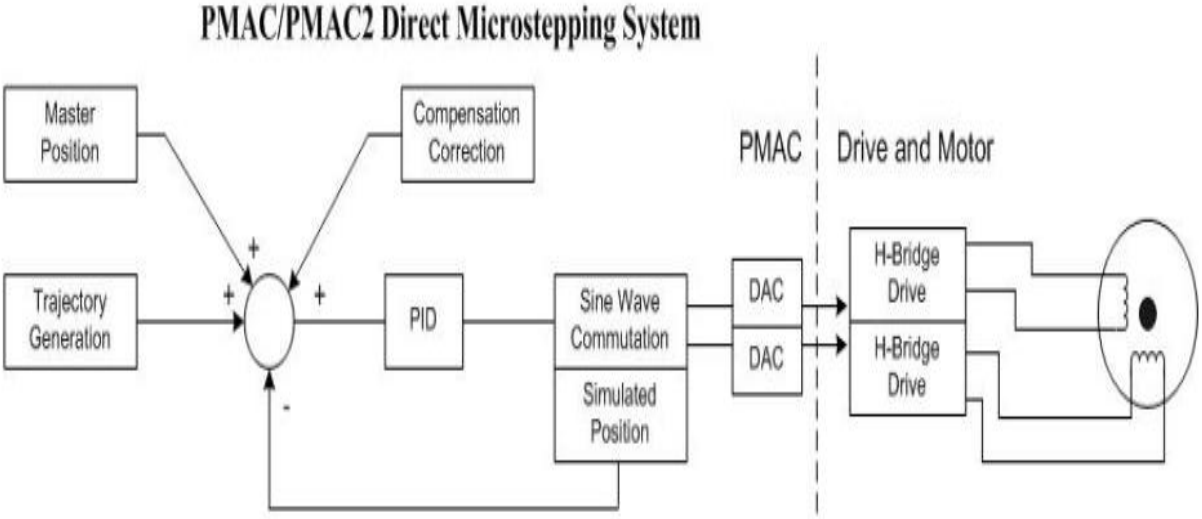
Turbo PMAC 可以执行标准步进电机的开环微步进（直接微步进），为换相和伺服算法清理内部产生的伪反馈。

这种技术与利用 Turbo PMAC 和一个脉冲和方向输出来命令一个外部的微步进驱动器不同，该技术根本不使用 Turbo PMAC 的换相算法。

当微步进时，Turbo PMAC 提供了两个模拟输出作为对电机相位的电流指令。通常对于一个微步电动机，这两个相位是电独立的，并且彼此有 90° 的相位差。在这种情况下，两个输出是简单地双向电流命令用于 H-桥放大器驱动各相。这些放大器可以是简单的转矩模式（电流模式）的直流有刷电机放大器。

Turbo PMAC 的微步进算法在每个电子周期提供 2048 个微步，是 512 微步/步。在一个典型的 200 步/转电机，这相当于每转 102,400 个微步。缺省相位更

新频率为 9 kHz 时，Turbo PMAC 可以回转超过 4,600,000 微步/秒（每秒 9000 的全步）。用少量的电动机和/或 Turbo PMAC 的快速版本，可以使用更高的相位更新频率。



设置 I-变量

设置一个电机的微步进仅仅是根据下表设定电机的 I-变量的问题。由于没有反馈，也没有必要调整。

换相启用：Ixx01

设置 Ixx01 为 3 启动 Turbo PMAC 换相，读取一个 Y-寄存器用于换相位置反馈。 （请参阅下面的 Ixx83 部分。）

命令输出地址：Ixx02

将 Ixx02 设置为所用的输出 DAC 对的较低地址(例如$078002 用于 Turbo PMAC 的 DAC1 和 DAC2 或在 Turbo PMAC2 上的 DAC1A 和 B，$078420 用于 MACRO IC0 节点 0 寄存器 0 和 1 用于远程 MACRO 站的 DAC)。该设置是一样的对于使用这些寄存器进行正常的闭环换相。下表显示了对 PMAC 式伺服 IC、PMAC2 式伺服 IC、MACRO IC 上的 DAC 对的设置。

正弦波模式命令输出地址 – PMAC 式伺服集成电路（Y-寄存器）

IC#-通道#	0 – 1&2	0 – 3&4	1 – 1&2	1 – 3&4
Ixx02	**$078002**	**$07800A**	**$078102**	**$07810A**
IC#-通道#	2– 1&2	2 – 3&4	3– 1&2	3– 3&4
Ixx02	**$078202**	**$07820A**	**$078302**	**$07830A**
IC#-通道#	4– 1&2	4– 3&4	5– 1&2	5 – 3&4
Ixx02	**$079202**	**$07920A**	**$079302**	**$07930A**
IC#-通道#	6 – 1&2	6 – 3&4	7 – 1&2	7– 3&4
Ixx02	**$07A202**	**$07A20A**	**$07A302**	**$07A30A**
IC#-通道#	8 – 1&2	8– 3&4	9 – 1&2	9 – 3&4
Ixx02	**$07B202**	**$07B20A**	**$07B302**	**$07B309**

伺服器 IC 0 和 1 是在 Turbo PMAC 本身。

伺服器 IC2-9 是在 Acc-24P/V 或 Acc-51P 板。
奇数伺服 IC 的通道 1-4 是板上的通道 5-8。

正弦波模式命令输出地址 – PMAC2 式伺服集成电路（Y-寄存器）

IC#-通道#	0-1	0-2	0-3	0-4	1-1	1-2	1-3	1-4
Ixx02	**$078002**	**$07800A**	**$078012**	**$07801A**	**$078102**	**$07810A**	**$078112**	**$07811A**
IC#-通道#	2-1	2-2	2-3	2-4	3-1	3-2	3-3	3-4
Ixx02	**$078202**	**$07820A**	**$078212**	**$07821A**	**$078302**	**$07830A**	**$078312**	**$07831A**
IC#-通道#	4-1	4-2	4-3	4-4	5-1	5-2	5-3	5-4
Ixx02	**$079202**	**$07920A**	**$079212**	**$07921A**	**$079302**	**$07930A**	**$079312**	**$07931A**
IC#-通道#	6-1	6-2	6-3	6-4	7-1	7-2	7-3	7-4
Ixx02	**$07A202**	**$07A20A**	**$07A212**	**$07A21A**	**$07A302**	**$07A30A**	**$07A312**	**$07A31A**
IC#-通道#	8-1	8-2	8-3	8-4	9-1	9-2	9-3	9-4
Ixx02	**$07B202**	**$07B20A**	**$07B212**	**$07B21A**	**$07B302**	**$07B30A**	**$07B312**	**$07B31A**

伺服器 IC 0 和 1 是在 Turbo PMAC2 本身或者 Acc-2E 3U-格式堆栈板上。
伺服器 IC2-9 是在 Acc-24x2 或 Acc-51E 板。
奇数伺服 IC 的通道 1-4 是双-伺服-IC 板上的通道 5-8。

正弦波模式命令输出地址 – MACRO 集成电路（Y-寄存器），1 类协议

IC#-节点#	0-0	0-1	0-4	0-5	0-8	0-9	0-12	0-13
Ixx02	**$078420**	**$078424**	**$078428**	**$07842C**	**$078430**	**$078434**	**$078438**	**$07843C**
IC#-节点#	1-0	1-1	1-4	1-5	1-8	1-9	1-12	1-13
Ixx02	**$079420**	**$079424**	**$079428**	**$07942C**	**$079430**	**$079434**	**$079438**	**$07943C**
IC#-节点#	2-0	2-1	2-4	2-5	2-8	2-9	2-12	2-13
Ixx02	**$07A420**	**$07A424**	**$07A428**	**$07A42C**	**$07A430**	**$07A434**	**$07A438**	**$07A43C**
IC#-节点#	3-0	3-1	3-4	3-5	3-8	3-9	3-12	3-13
Ixx02	**$07B420**	**$07B424**	**$07B428**	**$07B42C**	**$07B430**	**$07B434**	**$07B438**	**$07B43C**

直接微步进启用：Ixx96

将命令输出模式变量 Ixx96 设为 1，告诉 Turbo PMAC 它对该电机使用直接微步进。

编码器转换表入口：I8xxx

对于每一个直接微步进电动机，在编码器转换表（ECT）中设定一个两-行输入，以处理电动机相位寄存器的数据。第一设置行（I-变量）有一个“6”作为第一个十六进制数字，告诉表格读取一个 48 位的 Y/X 寄存器，其次是相位位置寄存器地址在最后五个十六进制数字。下表显示了每个电机所需的第一行设置：

电机	第一设置行	电机	第一设置行	电机	第一设置行	电机	第一设置行
1	$6000B4	9	$6004B4	17	$6008B4	25	$600CB4
2	$600134	10	$600534	18	$600934	26	$600D34
3	$6001B4	11	$6005B4	19	$6009B4	27	$600DB4
4	$600234	12	$600634	20	$600A34	28	$600E34
5	$6002B4	13	$6006B4	21	$600AB4	29	$600EB4
6	$600334	14	$600734	22	$600B34	30	$600F34
7	$6003B4	15	$6007B4	23	$600BB4	31	$600FB4
8	$600434	16	$600834	24	$600C34	32	$601034

每个入口的第二设置行应该被设置为$00B018。前三个十六进制数字指定使用 48 位寄存器的 11 位。最后三个十六进制数字指定这些位以 48 位 Y/X 寄存器的 24 位开始，它是 X-寄存器的 0 位。换句话说，使用 X-寄存器的低 11 位。

例如，要使用 ECT（编码器转换表）的前两行处理电机 1 的相位位置寄存器，I8000 将被设置为$6000B4，I8001 将被设置为$00B018。

位置反馈地址：Ixx03 和 Ixx04

将 Ixx03 和 Ixx04 都设定为一个寄存器的地址，在编码器转换表中已经处理相位位置寄存器的数据。请记得将它们设置为入口的第二行的地址。例如，对于电机 1，要使用上一节例子的 ECT 的两-行第一入口的结果，I103 和 I104 将被设置为$3502，第二行的地址。在固件版本 1.937 中启动，这可以通过分配数值到转换表 I-变量（例如 **I103=@ I8001，I104=@ I8001**）的地址来完成。因为相位位置寄存器，每个换相周期 2048 个微步，是用于位置反馈，因此一个计数定义为一个微步---一个周期的 1/2048，或者一个 2 相电机的全步的 1/512。

位置缩放因子：Ixx08＆Ixx09

将 Ixx08 和 Ixx09 比例因子都设置为 32。

伺服回路增益：Ixx30 - Ixx35

- 比例增益项 Ixx30 设置为 8192。
- 微分增益项 Ixx31 设置为 0。
- 速度前馈项 Ixx32 设置为 2048。
- 积分增益项 Ixx33 设置为 0。
- 加速度前馈项 Ixx35 设置为 2048。
- 该命令的输出限制设置为 32,767。

换相周期大小：Ixx70＆Ixx71

Ixx70 设置为 1，Ixx71 设为 2048，每个电气周期提供 2048 个计数（微步）。（512 微步/步）

换相相位角：Ixx72

对于通常的 2 相微步电动机，Ixx72 换相相位角参数设置为 512 或 1536。这两个值之间的变化会改变正旋转的方向感。如果你想尝试微步进一个 3 相电机，使用 683 或 1365。对一个给定的相位数，两种可能的设置之间的改变会反转为电机“向上计数”的方向。

电流强度：Ixx77

设置 Ixx77“磁化电流”参数来控制相位所用的电流数量。这个保持 16-位 DAC 位的最大数(或相当于命令一个 DAC 输出(电流命令到放大器))。例如，16,384

的值在每个相位上提供了+/-5V 的正弦输出。这个值可以在任何时间改变，从而改变使用的电流量。

转差率增益：Ixx78

设置 Ixx78“转差率增益”参数等于 2.0/N，其中 N 为每个伺服周期的相位周期数，设置是通过 Turbo PMAC 上的 E3-E6、Turbo PMAC2 的 I7002、典型 UMAC Turbo 上的 I7202、或 Turbo PMAC2 UltraLite 上的 I6802。这些跳线或变量的默认设置提供的 N 为 4，所以默认设置下 Ixx78 将被设置为 0.5。

换相位置地址：Ixx83

设置 Ixx83 换相位置地址参数为电机前一个相位位置寄存器的地址。对于每台电机的地址被显示在下面的表中。

前一个相位位置寄存器（Y-地址）

Motor	Ixx83	Motor	Ixx83	Motor	Ixx83	Motor	Ixx83
1	$0000B2	9	$0004B2	17	$0008B2	25	$000CB2
2	$000132	10	$000532	18	$000932	26	$000D32
3	$0001B2	11	$0005B2	19	$0009B2	27	$000DB2
4	$000232	12	$000632	20	$000A32	28	$000E32
5	$0002B2	13	$0006B2	21	$000AB2	29	$000EB2
6	$000332	14	$000732	22	$000B32	30	$000F32
7	$0003B2	15	$0007B2	23	$000BB2	31	$000FB2
8	$000432	16	$000832	24	$000C32	32	$001032

7-7 下一步做什么

一旦已采取本节中的相应步骤，电机的换相和电流回路应正确工作。该电动机应该是通过 0-命令在两个方向转动，正的 0-命令应该引起电动机位置在正方向计数，负的 0-指令应使电动机位置在负方向计数。

一旦完成这一点，下一步是设置和调整位置/速度伺服回路，无论是标准的 PID 回路，扩展的伺服算法，或一个用户编写的伺服算法。完成这个的方法相同于 Turbo PMAC 电机不带数字电流环和/或 Turbo PMAC 换相的。出于调整的目的，一个带 PMAC 换相和/或电流回路的系统对于位置/速度环看起来像一个“转矩模式”驱动器。

请记住，使用 **SAVE** 命令将这里已设置的 I-变量值存储到非易失性闪存中。

7-8 用户编写的相位算法

Turbo PMAC 支持用户编写的相位算法的安装和自动执行。如果标准换相/电流环的算法不适合获得所需的性能，可以使用这些；或者，它们可以被用于非伺服用途，保证算法以相位更新率执行。这对于 I/O 的快速更新是非常有价值的。

用户编写的相位算法必须使用摩托罗拉 DSP56300 系列汇编语言编写，并使用摩托罗拉的交叉-汇编器汇编，他们的网站上免费提供。

一个 Turbo PMAC 上只能安装一个用户相位算法。该算法可以被 Turbo PMAC

的任何电机执行。如果不同电机应执行不同的算法，这必须通过在一个用户编写的相位算法中进行分支来实现。

高效的用户编写的伺服算法用 Turbo PMAC 上使用的 DSP56300 系列处理器的汇编语言编写。这需要使用一个摩托罗拉的交叉-汇编器，可免费从他们的网站获得。此外，它还需要一个泰道公司的链接程序，称为 **CODET.EXE**，在 **Microsoft Windows** 操作系统下运行，可从泰道公司的网站免费获得。

编写算法

该算法是用摩托罗拉 DSP56300 汇编语言编写的，使用任何标准的文本编辑器。编写的代码受以下限制。

程序存储空间

该程序必须从内存位置 **P:$040800** 开始，所以代码的第一行必须是：

```
ORG    P:$40800
```

对于一个 DSP56303 处理器（80 MHz 的 CPU Opt 5CX），生成的代码必须结束在内存位置 **P:$040BFF**，为程序提供 1K 字的缓冲区。对于一个 DSP56309 处理器（100 MHz 的 CPU Opt 5DX）或 DSP56311 处理器（160 MHz 的 CPU Opt 5EX），生成的代码必须结束在内存位置 **P:$044BFF**，为程序提供一个 17-K 字的缓冲区。在所有情况下，如果使用一个用户编写的相位程序，用户编写的伺服程序被限制为一个 2-K 字的缓冲区（**P:$040000- P:$0407FF**）。

准入条件

在进入用户编写的相位，程序可以为执行电机预期下列数据在内部 DSP 寄存器中：

- R0 寄存器包含了执行电机的第一个状态字的地址（如：对于电机 1，**$0000B0**）。
- N4 寄存器包含了伺服电机寄存器的模块长度（目前是**$80**），这对从电机到电机的增量有用。这一定不能改变。
- R4 寄存器为下一个更高编号的电机包含了第一个状态字的基地址。这必须不能改变。如果从这个值中减去寄存器 N4 的内容，你会得到执行电机的定相数据的基地址。
- N0 寄存器包含的值为 2。

电机的伺服算法的转矩(正交电流)指令可在电机的正交命令寄存器中找到。这个寄存器是在一个地址，比 R0 寄存器中的地址大 15（$F）。

退出条件

关于从用户编写的相位退出，算法必须已经将其输出写入到执行该操作的寄存器中---即设置变量 Ixx02 在工厂提供的相位算法中执行的功能。从一个用户编写的相位程序退出后，Turbo PMAC 固件将不会执行此功能。一个用户编写的相位算法可以使用 Ixx02 寄存器告诉它将其输出值放于哪个地址。

用户编写相位的最后一行必须是 **RTS**（从子程序返回）。

可用的寄存器

下面的数据寄存器可能被用户编写的相位使用：

- 内部 DSP 寄存器 **R0、N0、R1、N1、R5 和 N5** 都可以使用，并且在完成时不需要被恢复。
- 内部 DSP 寄存器 **M0、M1、M4、M5、R4 和 N4** 都可以使用，但在完成

时，必须恢复到原来的值。

- 电机换相寄存器 X:$000xB2/32 到 X:$000xBF/3F，不是 I-变量寄存器，可以用来保存从周期到周期的值。只要用户编写的相位被激活，它们不被任何 Turbo PMAC 固件使用。
- 全局寄存器 X/Y:$0010F0 - $0010FF 也可使用。它们不被任何 Turbo PMAC 固件任务使用，除了在上电或复位时被设置为 0。
- **DEFINE UBUF** 命令创建的用户缓冲区的寄存器可以使用。它们不被任何的 Turbo PMAC 固件任务使用。
- 也可能使用其它寄存器，但对于 Turbo PMAC 固件的某些任务，可能会重写覆盖这些。例如，对于用户编写的伺服，可以使用寄存器用于某些 P 或 Q-变量，但将值分配给这些变量之一将覆盖寄存器。也可以对 Turbo PMAC 的标准伺服算法使用 I-变量作为增益用于用户编写的伺服。
- Turbo PMAC 内置的相位程序使用的 2048-入口正弦和余弦表位于地址 $003800 - $003FFF。正弦表是在 Y-内存中，余弦表是在 X-内存中。Turbo PMAC 固件在上电或复位时自动设置这些表，之后它不写入这些寄存器。

编程限制

不能使用任何级别的 DSP 的栈，所以不允许 **DO** 或 **JSR** 指令。

内部 DSP 地址寄存器 R2、R3、R6 和 R7 可能不被使用；修饰符寄存器 M2、M3、M6 和 M7；偏移寄存器 N2、N3、N6 和 N7。

汇编算法

汇编语言算法必须使用摩托罗拉的计算平台的交叉汇编程序汇编成 DSP56300 机器代码。按照摩托罗拉的指示来做到这一点。

链接算法

使用泰道公司的小程序“**CODET.EXE**”，其可在泰道网站上得到，将来自摩托罗拉汇编器的文件转换成一个可以直接下载到 Turbo PMAC 的格式。这个文件应该被归档在您的计算机或网络上。

下载算法

使用任何版本的 PMAC 执行程序来下载这个生成的文件到 Turbo PMAC 的程序存储器。请记住，它被下载到非易失性 RAM 内存中。要使得 Turbo PMAC 保留这个算法，在控制器复位或移除电源之前，发出一个 **SAVE** 命令。

执行算法

将电机 xx 变量 Ixx59 的 1 位设置为 1（Ixx59=2 或 3），来对于该电机选择用户编写的算法相位。如果 Ixx01 的 0 位也设置为 1，对这个电机执行用户编写相位算法，在每一个相位周期（每[I7+1]个相位中断，I7 的默认值为 0 时是每个相位中断），无论电机是开环、闭环，启用或禁用。电机的 Ixx00 甚至不用设置为 1。

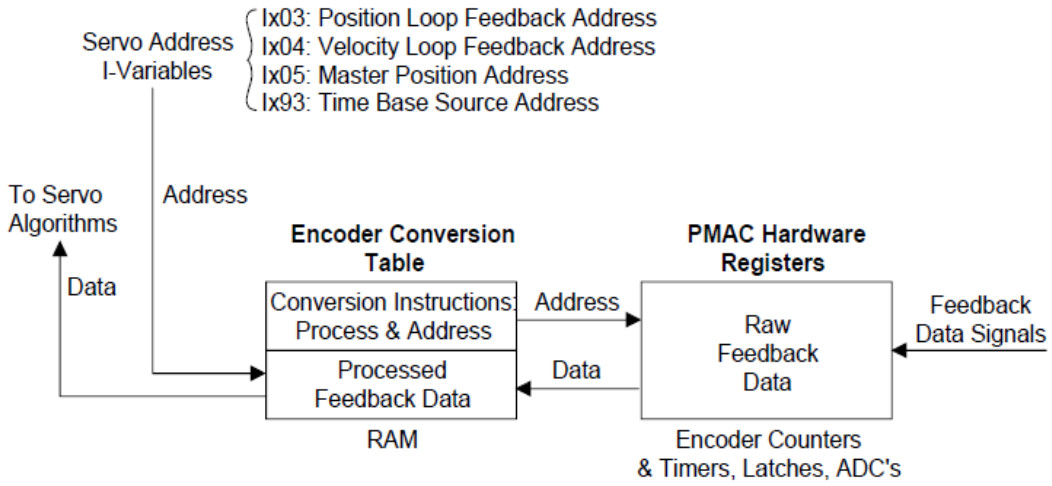
第八章 设置编码器转换表

Turbo PMAC 使用一个两步的过程为伺服算法处理其反馈和主位置信息，以提供最大功能和灵活性。（请注意，换相算法一般采用未经转换表处理的原始反馈寄存器）。对于大多数带正交编码器反馈的 Turbo PMAC 用户，这个过程可以是近乎透明的，没有必要担心细节。然而，一些用户需要了解这个转换过程中的一些细节，进行必要的变化去使用其他类型的反馈，以优化他们的系统，或执行特殊的功能。这部分是为那些用户提供的。

位置处理的第一阶段使用硬件寄存器，如带相关的定时器的编码器计数器，A /D 转换寄存器，或用于并行输入的附件卡。这些持续工作而没有直接的软件干预（虽然他们可以通过软件进行配置）而数据在伺服中断时通常被锁定。这个阶段包括在特定设备的硬件参考手册中和本手册的《设置反馈》的部分。除了这一点，该过程是由软件控制的。

大多数控制器直接使用硬件寄存器中用于反馈的原始数据或主位置数据。然而，Turbo PMAC 有一个中间步骤，使用一个软件结构被称为编码器转换表去预先处理在锁存寄存器中的信息。此表告诉 PMAC 要处理什么寄存器，以及如何处理它们，它也保存中间处理的数据。

PMAC 编码器转换表原理



此预处理步骤是必需的，或者至少是有利的，原因如下：

- 有几个片段的原始数据需要被组合成一个生成的位置号码，正如计数器和定时器用于“1/T”编码器插补。
- 有一些检查用于特殊情况（如翻转或突然变化指示性噪声，这需要做以确保可靠的数据）。
- 原始数据的数学处理，如加法、减法、微分或积分，需要去做以获得结果值。
- 原始数据只需要转移和移位，供伺服算法使用。

虽然该表的主要目的是产生处理的（转换的）值用于反馈和主位置数据供伺服算法使用，但是其它用途也是可能的。该表保证执行在每个伺服周期的开始，以及它用于处理数据的许多工具，也允许其他的用途。

8-1 转换表执行

该转换表在每个伺服周期的开头自动执行，就在伺服中断之后立即执行。在任何伺服回路执行该循环之前，该表的全部激活部分都参与执行。表中的每个入口在每一个伺服中断都被执行，即使是结果不经常使用（如当一个电机自身的伺服周期用 Ixx60 拓展）或者完全不用。

该表在每个周期按顺序执行，从顶部到底部。如果一个入口的结果被用来作为另一个入口的源，通常理想的是使得第二个入口进一步下降在表中，这样，不会导入一个伺服周期的延迟。

8-2 转换表结构

该编码器转换表有两列，一个位于处理器的 X-存储空间，一个位于 Y 存储空间。X 列保存转换后的数据，而 Y 列保持源寄存器的地址，以及每个这些源寄存器中数据的转换方法。基本上，你设置该表是通过写入到 Y-列中，PMAC 在每个伺服周期使用 Y 列的数据填补 X 列。

该表是一系列入口，其中每一个产生一个最终结果值。每个入口可以是 1、2 或 3 行长，每行中都有设置和结果数据。入口的最后的结果是在最后一个数据行上。整个表是 192 行长，尽管通常一个给定的应用程序中不是使用所有行。

Turbo PMAC 编码器转换表

X-内存			Y-内存	
（结果）			（设置）	
1、单行入口				
位（Bits）	位		位	位
5-23	0-4		20-23, 19	0-18
结果：整数	分数		方法，模式位	源地址
2、多行入口				
位	位		位	位
5-23	0-4		20-23, 19	0-18
（中间结果）			方法，模式位	源地址
…			（转换因子）	
结果：整数	分数		…	

设置结构

每个设置行是通过一个 I-变量表示的。表中的第一个设置行是通过 I8000 表示的，第二个是通过 I8001 表示，依此类推；表中的最后一个设置行是通过 I8191 表示的。由于各个入口都是 1、2 或 3 行长，每个入口使用 1、2 或 3 个设

置 I-变量，连续编号。

这些 I-变量是 24 位无符号的变量，通常表示为 6 位数字的十六进制值；如果 I9 设置为 2 或 3 的默认值，它们被报告为十六进制值。关于这些 I-变量可以取的值的详细描述，在软件参考手册中关于 I-变量 I8000-I8191 的说明。

执行程序的设置菜单

一般而言，编码器转换表 I-变量的设置，可以通过 PMAC 执行程序中的配置菜单交互完成。以下部分介绍了如何可以直接和“手动”设置。

入口的第一个设置 I-变量

一个入口的第一个设置 I 变量有三个部分：转换方法、一个模式切换位、源数据的地址。转换方法，它决定要做什么处理，包含在第一个十六进制位（20-23 位）。模式切换位，它指定处理的变体，被包含在 19 位。源地址，指定哪个寄存器或寄存器组包含原始数据，被包含在 0-18 位。

下表显示了一个入口的第一个 I 变量的结构：

入口的第一个设置行（I-变量）结构

位	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
数字			1				2				3				4				5				6	
内容	方法				*	源地址																		

*模式-切换位

例如，一个来自 PMAC 式（模式-切换位 0）伺服 IC 0 通道 1（源地址$078000）的增量式编码器的并行扩展（方法数字$8），在转换表中的第一个设置行是：

I8000=$878000

同一个并行扩展（方法数字$8），但是来自 PMAC2 式（模式-切换位 1）伺服 IC 0 通道 1（源地址$078000），在转换表中的第一行设置为：

I8000=$8F8000

一个入口的任何随后的行中的结构都取决于方法，每种类型将在下面讨论。

下表总结了每种方法数字所指定的操作。

方法数字	行#	定义的处理	模式切换	第一附加行	第二附加行
$0	1	增量式编码器 1/T 扩展	无	-	-
$1	1	ACC-28 式 A/D 转换器（高 16 位，没有翻转）	0=有符号数据 1=无符号数据	-	-
$2	2	并行 Y 字数据，无过滤	0=正常移位 1=未移位	宽度/偏移字	-
$3	3	并行 Y 字数据，有过滤	0=正常移位 1=未移位	宽度/偏移字	每个周期的最大变化
$4	2	时基缩放数字微分	无	时基比例因子	-
$5	2	积分的 Acc-28 式 A/D 转换器	0=有符号数据 1=无符号数据	输入偏置	-

$6	2	并行 Y/X 字数据,无过滤	0=正常移位 1=未移位	宽度/偏移字	-
$7	3	并行 Y 字数据，有过滤	0=正常移位 1=未移位	宽度/偏移字	每个周期的最大变化
$8	1	增量式编码器并行扩展	0= PMAC IC 1= PMAC2 IC	-	-
$9	2	被触发的时基，被冻结	0= PMAC IC 1= PMAC2 IC	时基比例因子	-
$A	2	被触发的时基，正在运行	0= PMAC IC 1= PMAC2 IC	时基比例因子	-
$B	2	被触发的时基，等待处理	0= PMAC IC 1= PMAC2 IC	时基比例因子	-
$C	1	增量式编码器，无或硬件 1/T 扩展	0=无扩展 1=1/T 硬件	-	-
$D	3	并行数据的指数滤波器	无	每个周期的最大变化	滤波器增益（逆时间常数）
$E	1	入口的和或差	0=无积分 1=积分结果	-	-
$F	-	（扩展项-类型由第二行第 1 个数字确定）	-	-	-
$F/$0	3	高分辨率插补	0= PMAC IC 1= PMAC2 IC	$0 方法数字&第 1 个 A/D 转换器的地址	A/D 转换偏差项
$F/$2	2	字节-宽并行 Y 字数据，无过滤	0=正常移位 1=未移位	$2 和宽度/偏移字	-
$F/$3	3	字节宽并联 Y 字数据，有过滤	0=正常移位 1=未移位	$3 和宽度/偏移字	每个周期的最大变化

因为大多数源地址有一个 7 在第二个 16 进制数字位，将模式-切换位设置为 1 的效果是第二个 16 进制数字从 7 改变为 F。例如，一个 Acc-28 ADC 入口是 $178005 且模式切换位为 0，被更改为$1F8005 与模式切换位为 1。

在软件参考手册中 I8000 – I8191 的描述下，对于每种类型的入口的设置有一个详细说明，每一个都有常见源地址的扩展性表格。该转换表的 PMAC 执行程序的设置菜单中对每个方法的源地址有很好的选择列表。

入口随后的设置 I-变量

如果第一行中所定义的方法指定一个多行的入口，表中的下 1 个或 2 个设置行（I-变量）是该入口进一步的设定值。这些后续行的含义取决于方法；上表给出关于这些是什么的简要说明，而软件参考手册描述了这些细节。

下一个入口

在任何入口的最后一行后，无论该入口是 1、2 还是 3 行长，表中的下一行是下一个入口的第一行。例如，如果在表中的第一行是一个 3 行入口，I8000、I8001、I8002 将被用于设置那个入口，而 I8003 将是下一个入口的第一设置行。

表的结尾

如果一个设置行（I-变量）是一个入口的第一行而被设置为 0，这标志着激活表的末尾。即使表中这点的下面还有有效入口，它们将不被执行。

结果结构

在 X 存储器中的编码器转换表的列中包含了表的处理结果。在这列中对入口的最后一行（一个单行入口的唯一的一行）的 24 位数据是最终的处理结果。这个值将被用于 Turbo PMAC 中的后续操作。此列中对入口的以前行（如果有的话）的数据包含了中间结果，这些通常是仅供内部使用。

最终的结果格式

在一般情况下，24 位的最终结果被构造为 19 位的整数（5 - 23 位）与 5 位的分数（0- 4 位）。整数值表示一个计数器（计数）、一个锁定寄存器或模拟-数字转换器的最低有效位（LSB）的整数。换句话说，主要源寄存器的 LSB 被放置在结果寄存器的 5 位，一个 5-位的移位。小数部分表示更精细的分辨率的测量或估计值（在一些方法中总是为零）；小数数据总共 5 位，结果的单位是一个计数的 1/32。需要注意的是，几种方法产生的测量分辨率比软件的一个计数更精细。

然而，并非总是如此。有可能在若干个转换方法中指定一个未移位的转换。当这样做时，源数据的 LSB 被放置在结果字的 0 位，而不是通常的 5 位。在高分辨率的模拟编码器转换中，对于硬件编码器计数器的每次计数产生 10 位的分数，所以结果单位是一个正交计数的 1/1024，或者一个编码器线的 1/4096。

当表格入口的最终结果值是由 Turbo PMAC 的伺服环使用用于反馈或主位置数据时，伺服算法软件始终考虑这个值的单位是一个计数的 1/32---使得该位代表一个计数或位于 5 位的 LSB---即使转换没有将它放在这里。这可以导致在软件计数和硬件计数之间的缩放比例不同。如果使用一个未移位转换，该软件会考虑源寄存器的 LSB 是一个计数的 1/32。如果使用高分辨率的模拟转换器时，软件会考虑正交计数器的一个硬件计数是 32 个软件计数，每个模拟编码器线产生 128 个计数。

中间结果

如果在转换表中的入口的长度超过一行，在结果列中的最后一行之前的行，保存中间结果。通常，这些是仅供内部使用；如果对于用户有任何用处，这是归档在转换方法的软件参考手册中。

使用结果

大多数情况下，在转换表中所处理的数据被用于反馈或主数据用于伺服算法，通过以下四种 I-变量之一：

- Ixx03　　电机 xx 位置环反馈地址
- Ixx04　　电机 xx 速度环反馈地址
- Ixx05　　电机 xx 主位置地址
- Isx93　　坐标系 x 时基源地址

每个变量的值是寄存器的地址，将在每个伺服周期读取以获得相应的信息用于伺服算法。这些变量总是使用在指定地址的 X 寄存器（这是表格的结果列）。

有两种方法来指定这些变量的值，以及它们使用的地址。第一种是直接指定地址，通常为十六进制数（有$前缀）。第二个方法是间接指定地址，通过参照在那个地址的 I-变量，在那个 I-变量名称的前面使用“@”符号。这允许您使用结果而无需查找地址。

下面的表显示了最常使用的表中的前 16 行的 I-变量和地址。显示所有 192 行的表格会在软件参考手册中给出，对于这些变量有正式描述。

行	I 变量	地址	行	I 变量	地址
1	**I8000**	**$3501**	**9**	**I8008**	**$3509**
2	**I8001**	**$3502**	**10**	**I8009**	**$350A**
3	**I8002**	**$3503**	**11**	**I8010**	**$350B**
4	**I8003**	**$3504**	**12**	**I8011**	**$350C**
5	**I8004**	**$3505**	**13**	**I8012**	**$350D**
6	**I8005**	**$3506**	**14**	**I8013**	**$350E**
7	**I8006**	**$3507**	**15**	**I8014**	**$350F**
8	**I8007**	**$3508**	**16**	**I8015**	**$3510**

例如，要使用表格的第八行的结果用于电机 4 的位置环反馈，可使用下列任一命令：

I403=$3508

I403=@I8007

即使地址是间接指定的，它仍然会被直接报告返回。因此，在上面的例子中，无论用哪种方式指定值，如果 Turbo PMAC 被查询 I-变量的值，将返回地址--

I403

$3508

8-3 转换方法

每个转换方法都详细介绍在软件参考手册中表的 I-变量的说明中。本节给出了一个简要的概述。

数字正交编码器

最常见的转换是为数字正交编码器（或其他脉冲序列，如进给这些计数器的脉冲和方向）。这些都是单行的入口。源地址始终是伺服器 IC 的编码器通道的基址。这些地址的完整列表在软件参考手册给出，这里给出常用地址的快速列表：

板载 PMAC 式通道 1-4：$78000，$78004，$78008，$7800C

板载 PMAC2 式通道 1-4：$78000，$78008，$78010，$78018

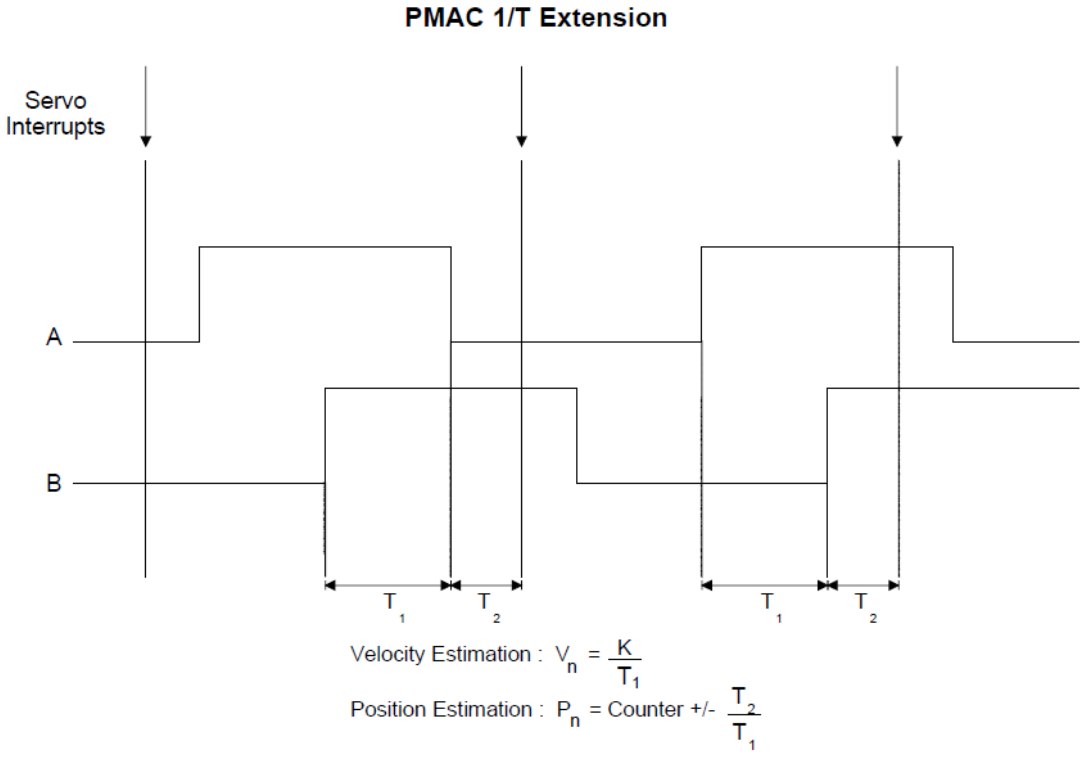
前 4 个附件 PMAC2 式通道：$78200，$78208，$78210， $78218

对该信号格式有三个转换方法：

- 软件 1/T 扩展
- 硬件 1/T 扩展
- 无扩展

软件 1/T 扩展（方法数字$0）

这是最常用的方法，并且是表中默认使用的。此方法对于整数计数值使用正交计数器，并计算两个定时器的比率以取得 5 位的分数计数值。



注意：

对于 PMAC2 式伺服器 IC 的版本 D 或更新的版本（2002 年推出），IC-通道变量 I7mn9 必须设置为默认值 0，这样两个定时器可以被读取。（仍然保留向后兼容性）。

示例：

```
I8000=$078000          ；伺服 IC0 通道 1 的软件 1/T 扩展
I8001=$078004          ；伺服 IC0 通道 2（PMAC1）的软件 1/T 扩展
```

硬件 1/T 扩展（方法数字$C，模式-切换位 1）

这种方式使伺服 IC 在其硬件上计算分数计数值。它要求 PMAC2 式伺服器 IC 的修订版“D”或更高版本（2002 引进），IC 通道变量 I7mn9 设置为 1，使能硬件的计算。

这种方法允许新的硬件位置捕捉和位置比较电路带子-计数分辨率与数字正交编码器使用。它也可以节省少量的计算时间。

示例：

```
I8000=$CF8000          ；伺服 IC 0 通道 1 的硬件 1/ T 扩展
I8001=$CF8008          ；伺服 IC0 通道 2（PMAC2）的硬件 1/T 扩展
```

无扩展（方法数字$C，模式-切换位 0）

此方法不计算小数计数值， 5 个小数位保留为 0，只使用硬件计数器的整计数值。这主要是用于模拟的反馈来自脉冲-方向输出用于开环步进控制，其中，分数计数估计可引起不受欢迎的抖动。

示例：

```
I8008=$C78200          ；伺服 IC2 通道 1 的无扩展
I8009=$C78208          ；伺服 IC2 通道 2（PMAC2 IC）的无扩展
```

模拟量正弦/余弦编码器

Turbo PMAC 支持两种转换的方法用于插补模拟正弦/余弦编码器：一种低分辨率（每线 128 或 256 个状态）和一个高分辨率（每线 4096 个状态）。源地址始终是伺服器 IC 上的编码器通道的基址。

低分辨率插补（方法数字$8）

此方法读取外部计算的五位的并行到 Turbo PMAC 的分数计数数据，，并使用正交计数器的整-计数的数据。其目的是为使用 Acc-8D Opt 8 和 ACC-8A 上的插补电路通过正常的 PMAC 伺服通道。如果该数据是通过一个 PMAC 式伺服 IC 读取，模式切换位被设置为 0；如果数据是通过一个 PMAC2 式伺服 IC 读取，它被设置为 1。这是一个单行的入口。

源地址的完整列表在软件参考手册中给出。常见的源地址是：

PMAC 板载通道 1-4：$78000，$78004，$78008，$7800C

PMAC2 板载通道 1-4：$78000，$78008，$78010，$78018

注意，所使用的编码器计数器必须被设置为 4 倍解码，通过编码器通道的解码变量 I7mn0，并且仅可以使用两个方向的其中一个。对于泰道公司的附件和标准接线，I7mn0 变量必须设置为 7。如果这是错的，小数及整计数的数据的方向感将不匹配。

示例：

```
I8004=$878100          ；伺服 IC 1 通道 1（PMAC1 IC）的低分辨率插补
I8005=$8F8018          ；伺服 IC0 通道 4（PMAC2 IC）的低分辨率插补
```

高分辨率插补（方法数字$F，第二行的扩展方法数字$0）

这种方法计算 10 位的小数计数数据（1024 个状态）在每个正交计数---每线 4096 个状态---通过读取 Acc-51 板上的 A / D 转换器的正弦和余弦信号，并

计算反正切值。整-计数数据来自数字正交计数器。如果数据通过一个 PMAC 式伺服 IC（Acc-51P）读取，模式-切换位设置为 0；如果数据是通过一个 PMAC2 式伺服 IC（Acc-51E，51C）读取，它被设置为 1。

源地址的完整列表在软件参考手册中给出。常见的源地址是：

ACC-51P 即伺服 IC2，通道 1-4：$78200，$78204，$78208，$7820C

ACC-51E 即伺服 IC4，通道 1- 4：$79200，$79208，$79210，$79218

第二行包含了两个 ADC 读取的第一个的地址，第 1 个十六进制数字是$0。对于 ACC-51P，ADC 地址比第一行中使用的通道基地址高 2；对于 ACC-51E 和 51C，它是高 5。

对于 ADC 的常见源地址是：

ACC-51P 为伺服 IC2，通道 1 - 4：$78202，$78206，$7820A，$7820E

ACC-51E 为伺服 IC4，通道 1 - 4：$79205，$7920D，$79215，$7921D

第三行是一个 24 位的偏移项，在反正切值计算完成之前，从包含 ADC 值（即使 ADC 只占据了前高 14 位）的两个 24 位字中减去。

使用的编码器计数器必须被设置为 4 倍解码，通过将编码器通道的变量 I7mn0 设置为 3 或 7。如果这个变量被改变，例如改变计数器的方向传感，新的值必须被保存，并且 Turbo PMAC 复位，以使得这种转换的正常工作（为了小数数据的方向感匹配整-计数数据的方向感）。

示例：

```
I8000=$ F78200        ；伺服 IC2 通道 1（PMAC1 IC）的高分辨率插补
I8001=$078202         ；从伺服 IC2 通道 1 读取 ADC
I8002=$004000         ；12 位 ADC 的 4 个 LSB 的偏差项
I8000=$ FF8300        ；伺服 IC3 通道 1（PMAC2 IC）的高分辨率插补
I8001=$078305         ；从伺服 IC3 通道 1 读取 ADC
I8002=$000000         ；零偏差项
```

ACC-28 模拟 - 数字转换器

该转换表可以处理来自 Acc-28 板 16 位 A/D 转换器的数据或其等效物的。源地址是接口到附件的 IC 上的 A/D 转换寄存器的地址。

标准 A/D 转换（方法数字$1）

这种方法取 A/ D 转换器中的值（无积分）然后将它移位以将 LSB 位留在结果的 5 位---TurboPMAC 软件认为的一个计数。如果 A/D 转换器提供了一个有符号值（Acc-28A），模式-切换位应设置为 0；如果 A/D 转换器提供了一个无符号值（Acc-28B，28E），模式-切换位应该被设置为 1。这是一个单行的入口，

常见的源地址是：

ACC-28A/ B 进入 PMAC 通道 1-4:$78006，$78007，$7800E，$7800F

ACC-28B 进入 PMAC2 通道 1A、1B、2A、2B:$78005，$78006，$7800D，7800E$

ACC-28E 为第一个 I/O 板，通道 1-4:$78C00，$78C01，$78C02，$78C03

示例：

```
I8004=$178006         ；伺服 IC0 ADC 1（PMAC1 IC）的转换，有符号
I8005=$1F8105         ；伺服 IC1 ADC 1A（PMAC2 IC）转换，无符号
I8006=$1F8C00         ；第一个 Acc-28E 通道 1 的转换，无符号
```

积分的 A/D 转换（方法数字$5）

这种方法拾取 A/D 转换器中的值，对它移位，所以 LSB 是在 5 位，并将其添加到前一个周期的结果，产生一个数值积分。如果 A/D 转换器提供了一个有符号值（Acc-28A），模式-切换位应设置为 0；如果 A/D 转换器提供了一个无符号值（Acc-28B，28E），模式-切换位应该被设置为 1。这是一个双行的入口。第二行保存一个偏移项，表现为一个 24 位的数字，在数值积分前，从源 A/D 数据（其 LSB 是在 8 位）中减去。

示例：

```
I8004=$578006          ；伺服 IC0 ADC1（PMAC1 IC）的积分，有符号
I8005=$000300          ；偏差项（16 位 ADC 的 3 个 LSB（最低有效位））
```

并行反馈

转换表可以处理从寄存器或寄存器组读取的并行反馈数据。虽然这基本上是一个复制操作，但是它可以有以下几个重要的功能：

- 它会将数据复制到 X 寄存器，其中伺服环的 I-变量可以访问它（大多数 I/O 寄存器是在 Y-内存中）。
- 它可以对数据进行移位，所以源寄存器的 LSB 位变成结果的 5 位，PMAC 固件将其视为一个计数。
- 它可以从源字中屏蔽掉不需要的数据位，并妥善处理翻转和软件扩展。
- 它可以执行一个过滤功能，减轻噪声在显著数据位中的影响。

并行反馈的数据可以从一个 48 位的范围内的多达 24 位的一个连续的位范围中读取，或者从在连续的寄存器中的 2 或 3 字节宽的片段，如从许多 UMAC 的 I/O 板。转换表中的入口的第一行设置的五个方法位数字来处理这种情况：

- $2：48 个可能位是来自在指定的地址的 Y 寄存器和下一个更高地址的 Y 寄存器的。转换表不执行过滤。
- $3：48 个可能位是来自在指定的地址的 Y 寄存器和下一个更高的地址的 Y 寄存器。转换表可以执行源数据的滤波功能。
- $6：48 个可能位是来自在指定的地址的 Y 寄存器和在同一个地址的 X 寄存器。转换表不执行过滤。
- $7：48 个可能位是来自在指定的地址 Y 寄存器和在同一个地址的 X 寄存器。该转换表可以执行源数据的滤波功能。
- $F：该数据位于由 Y 寄存器指定的地址和相邻的更高的地址的字节宽的片段中。在这种情况下，入口的第二设置行的第一个十六进制数字必须是$2（无过滤）或者$3（有过滤）。

在所有这些情况下，入口的第一设置行中的 19-位模式-切换位被设置为 0 以将源数据的 LSB 位放在结果的 5 位中，在其中 Turbo PMAC 固件将其视为一个计数。如果模式-切换位被设置为 1，源的 LSB 被放置在结果的 0 位中，其中 Turbo PMAC 固件将其视为一个计数的 1/32。这个备用设置可以用于非常高的分辨率的反馈，其中数值速度饱和可能是一个问题。

入口的第一设置行的 0 - 18 位指定源数据的地址。在方法$2 和$3，或$F/$ 2 和F/3 的情况下，有些源数据也可以来自下一个更高的地址。

入口的第二设置行指定使用源地址数据的哪一部分。这一行被分成四个部分：第 1 个十六进制数字，第 2 个和第 3 个十六进制数字，第 4 个十六进制数字，最后是第 5 和第 6 个十六进制数字。

并行数据入口第二设置行

16 进制数字	1	2&3	4	5&6
内容	辅助方法	位宽	字节	LSB 位置

在方法$2、$3、$6 和$7 的情况下，第一个数字始终为 0。在方法$F 的情况下，如果没有进行过滤，第一个数字是$2；如果进行了滤波，它是$3。

第二和第三个数字指定位宽---使用多少位。

在方法$2、$3、$6 和$70 的情况下，第四位始终为 0。在方法$F 的情况下，第四个数字指定数据来源于哪个字节：0 是低字节，1 为中间字节，2 是高字节。

最后两位数字指定源字节或字的哪一位被用作 LSB。对于字节-宽的数据，这是始终在$00 到$07 的范围。对于来自一个 48-位字段的数据，这个值可为 48 减去位宽。

如果要过滤源数据，入口中有一个第三设置行，其指定在单个被认为是真实的伺服循环中的源数据的最大变化（以所使用的源数据的最低有效位为单位）。如果源发生的变化比这更多（可能是由于高位的噪音），结果只变化这个数量。这是一个重要保障防止噪音和其他可能的异常。

示例：

这些前三个 3-行的入口处理一个 48 位的 Acc-14D/V 卡上的 3 个 16 位输入

```
I8000=$378A00          ; 过滤的并行 Y-数据，来自第一个 Acc-14D/V
I8001=$010000          ; 使用 16 个位，起始位 0
I8002=256              ; 每个伺服周期 256 个 LSB 最大变化
I8003=$378A00          ; 过滤的并行 Y-数据来自第一个 Acc-14D/V
I8004=$010010      ;来自 LSB 的位 16，使用 16 个位（高 8 位从下个地址）
I8005=256              ; 每个伺服周期 256 个 LSB 的最大变化
I8006=$378A01          ; 过滤的并行 Y-数据来自第一个 Acc-14D/V
I8007=$010008          ; 来自 LSB 的 Bit 8，使用 16 个位
I8008=256                ; 每个伺服周期 256 个 LSB 的最大变化
```

这些下面的几个例子读取未移位的 MACRO 寄存器

```
I8000=$2F8420      ;IC0 节点 0 寄存器 0 的未移位未过滤读取（地址$078420）
I8001=$018000      ; 使用 24 个位，起始位 0
I8002=$2F8424      ;IC0 节点 1 寄存器 0 的未移位未过滤读取（地址$078424）
I8003=$018000      ; 使用 24 个位，起始位 0
```

下一个例子从一个 Acc-14E 的字节-宽片段中读取数据

```
I8000=$F78C00        ; 从$78C00 读取
I8001=$314000        ; 过滤的并行 Y-数据 20 个位，低字节，从位 0 开始
I8002=150            ; 每个伺服周期 150 个 LSB 最大变化
```

这个例子读取定时器寄存器作为并行数据用于 MLDT 反馈

```
I8003=$378000          ; 并行滤波从伺服 IC0 通道 1 读取
I8004=$013000          ; 读取 19 个位，起始位 0
I8005=75               ; 每个伺服周期 75 个 LSB 最大有效改变
```

指数滤波器

转换表可以通过执行指数-过滤器入口对输入的数据进行过滤的功能。这个入口实现了对源数据的低通（单-极 IIR）滤波。它主要对于主位置数据是有用

的；其用于反馈数据可能会有问题，因为它引入了延迟和反馈回路中产生的稳定性问题。

这是一个 3-行的入口。第一行使用一个$D 方法数字，后面为地址。源数据必须在这个地址的 X 寄存器中，并且通常是转换表中上一个入口的结果（地址 $3501- $35C0）。

第二行包含了在源数据中的单个伺服周期中允许的最大变化。

第三行包含了滤波器的增益。从本质上讲，这是一个逆时间常数；增益项设置为 $2^{23}/(T_f+1)$，其中 T_f 是滤波时间常数，以伺服周期表示。

例如：

```
I8004=$07820C          ; 伺服 IC2 编码器 4 的 1/T 插补
I8005=$D03504          ;  I8004 结果的指数过滤
I8006=4000             ; 最大允许变化（LSB（最低有效位）/伺服周期）
I8007=524288           ;滤波器增益为 15 个伺服周期的时间常数[2^23/(15+1)]
I1205= @I8007          ; 对于电机 12 使用过滤值作为主位置
```

和与差

转换表中的和与差入口可以用来做很多有用的任务：

- 加上或减去表中的 2 个结果
- 更改表中结果的符号
- 对表的结果的值积分
- 对表中两个结果的和或差进行积分

这可以用来，例如，对旋转台的相对两侧的两个传感器的读数进行平均，取出偏心，或者利用两边传感器的差值来计算在一个龙门架的倾斜。

这个入口可以使用仅为转换表中的源数据结果。我们强烈鼓励使用表中早期的结果；使用表中的后期的结果意味着使用前一个的伺服周期的结果。

这是一个单行的入口，分为四个部分。

- 第一部分是第 1 个十六进制数字，对于这个类型的入口始终是“E”。
- 第二部分是第 2 个十六进制数字（16-19 位），其中包含四个模式-切换位：
  - 19 位，如果设置为 1，积分最终结果，将每个周期计算的和或差加入到前一个周期的最后结果中。
  - 18 位，如果设置为 1，指示 Turbo PMAC 忽略第二个源，允许这个入口去简单地求反和/或积分单个值。
  - 17 位，如果设置为 1，在将它加到结果之前，求反第二个入口，建立一个有效的减法。
  - 16 位，如果设置为 1，在将其加到结果上之前，求反第一个入口。
- 第三部分使用第 3 和第 4 个十六进制数字作为一个数字，代表相对所用的第二个源的表格开头的偏移量。这可以计算，通过从对应于该结果的 I-变量数目中减去 8000。
- 第四部分使用第 5 和第 6 个十六进制数字作为一个数字，代表相对所用的第一个源的表格开始的偏移量。这可以通过从对应于该结果的 I-变量数目减去 8000 来计算。

示例：

使用表中以下入口作为源数据：

```
I8000=$078200          ；伺服 IC2 编码器 1 的 1/T 插补
I8001=$078208          ；伺服 IC2 编码器 2 的 1/T 插补
I8002=$078210          ；伺服 IC2 编码器 3 的 1/T 插补
I8003=$078218          ；伺服 IC2 编码器 4 的 1/T 插补
I8004=$F78C00          ；字节-宽的并行读取 Acc-14E 数据
I8005=$314000          ；滤波的，20 个位，低字节，从位 0 开始
I8006=256              ；每个伺服周期 256 个 LSB 的最大变化
```

和与差的入口可以用来做几个处理：

```
I8012=$E00001          ；I8000 和 I8001 结果的总和
I8013=$E10203          ；从 I8002 的结果减去 I8003 的结果
I8014=$E40006          ；求反 I8006 的结果
I8015=$EC000D          ；积分 I8013 的结果
```

时基（微分）

转换表中的时基入口提供源数据的比例缩放的数字微分。通常，源数据是编码器的计数值，所以微分计算一个频率值。然后这个频率值可以用来作为一个坐标系的外部的时基，为了将坐标系中轴的运动从属到主编码器，生成一个电子凸轮。请参阅《同步 Turbo PMAC 到外部事件》的部分，获取使用这个数据用作时基控制的详细信息。

未触发 VS 触发的时基

一个时基入口可以是未触发或触发的。未触发时基入口更简单，但没有方法启动时基精确跟踪主编码器在特定的主位置。触发的时基入口使用 Turbo PMAC 伺服 IC 的硬件捕捉功能，精确锁定主位置，其是从属到主编码器的编程运动序列的起点。

时基入口是 2-行的入口。第一行包含了方法数字在 20-23 位（$4 用于未触发的，$9 用于触发的冻结状态，$B 是触发的等待状态，$A 是触发的运行状态），19-位模式-切换位用于触发的入口（= 0 用于 PMAC 式伺服 IC，=1 用于 PMAC2 式伺服 IC），数据的源地址被微分在 0-18 位。第二行包含时基比例因子，其乘以微分的源数据。

源寄存器---未触发的

在未触发的时基，源数据通常是表中的一个以前转换的结果---源地址$3501～$35C0---通常是一个增量编码器的 1/T 插补的结果。1/T 插补是重要的以减少由编码器的数据的数字微分产生的量化噪声，其结果有 5 位的分数，估算编码器的位置为一个计数的 1/32。

未触发的时基采用此源数据，从其中减去前一个周期的源数据，将差值乘以 2 倍的时基缩放系数，并将这个值储存为其结果。如果该结果是被用来作为坐标系的主时基，坐标系统的变量 Isx93 应包含该结果的地址。在这种情况下，时基的比例因子应被设置为 131,072 的值除以主时基的实时输入频率（计数/毫秒），并且坐标系运动程序假设主时基总是输出这个频率。

源寄存器---触发的

在触发的时基，源数据是直接来自伺服 IC 的编码器通道---源地址是 IC 上

的该通道的基地址（例如$78000 是用于伺服 IC0 通道 1，$78200 是用于伺服 IC2 通道 1）。在该方法中，时基转换计算 1/T 插补本身，计算 8 位的小数数据，和中间结果，单位是一个计数的 1/256。该中间结果被存储在入口的第一行。如果输入信号是非常高的质量，多余的小数位可进一步减少量化误差。

在运行状态（方法数字$A），触发的时基采取这个中间数据，减去前一个周期的中间数据，差额乘以 2 倍的时基缩放系数，并将这个值作为其结果。在冻结状态（方法数字$9），结果值始终为 0。在等待状态（方法位数$B），如果通道未发生触发，其结果值是 0；但是，如果通道发生了触发，触发-捕获的位置被用作前一周期的值，来计算结果，并且状态将自动更改为“正在运行”。

结果数据

如果结果是被用来作为坐标系的主要时基，坐标系统的变量 Isx93 应包含该结果的地址。在这种情况下，时基的比例因子应被设置为 16384 的值除以主时基的实时输入频率（计数/毫秒），并且坐标系运动程序假设主时基总是输出这个频率。

通常情况下，一个运动程序将立即设置方法位数为$9，在触发后在计算要执行的第一个动作的轨迹之前。运动程序在计算完这一轨迹后将自动暂停进一步的执行，因为这个动作不能开始实际的执行，是因为“冻结时基”（但要注意，在启动触发时的移动中没有计算延迟）。

接着，PLC 程序将方法数字改为$B 来等待时基。最后，当它看到捕捉触发时，该转换表将方法位数改为$A，如上面所述。

示例：

```
未触发的时基（注：TBSF---时基比例因子，RTIF---实时输入频率）
I8003=$079218            ；伺服 IC2 通道 4 编码器的 1/T 转换
I8004=$403504            ；来自 I8003 的结果的未触发时基
I8005=640                ；TBSF=131072/204.8（RTIF=204.8 计数/毫秒）
I5193= @I8005            ；使用 I8005 的结果作为 C.S.1 的时基
触发时基（注：TBSF---时基比例因子，RTIF---实时输入频率）
I8008=$AF9218            ；来自 PMAC2 伺服 IC2 通道 4 的触发时基，运行
I8009=256                ；TBSF= 16384/64（RTIF=64 计数/毫秒）
I5293=$350A              ；使用 I8009 的结果地址作为 C.S.2 的时基
```

第九章 设置伺服环

Turbo PMAC 可以为每个激活的电机自动关闭一个数字伺服回路。伺服回路的目的是命令的输出以这样一种方式，尽量使电机的实际位置匹配指令位置。它可以做多好取决于伺服环滤波器的整定---它的参数的设置---以及受控的物理系统的动态特性。

9-1 伺服更新率

伺服环的闭合（更新）频率是通过 Turbo PMAC 上的跳线决定，或 Turbo PMAC2 上的 I-变量决定的。在一个 Turbo PMAC 上，伺服中断频率的设置是通过跳线 E98，跳线 E29-E33（其分频主时钟来产生相位时钟）和跳线 E3-E6（其分频相位时钟来产生伺服时钟）。在一个 Turbo PMAC2 上，伺服中断频率是通过系统时钟源的伺服 IC m 的 I-变量 I7m00、I7m01 和 I7m02 设置的（或对于一个 Turbo PMAC2 UltraLite ，是 I6800、I6801 和 I6802）。请参考 Turbo PMAC 系统配置和自动配置的部分，获取关于这些时钟信号源以及如何设置它们的频率的详细信息。

参数 Ixx60 允许您对单个电机 xx 降低伺服环闭合率，通过指定在闭合之间跳过的伺服中断的数量。缺省值 0 在每个中断引起闭合。Ixx60 是有用的，为一个特定的电机减慢伺服更新速率，同时为其他电机保持较快的速度；它也是有用的，快速测试你是否可以通过较慢的伺服更新得到所有的电机所需的性能；此外，它可以用于减慢更新速率低于 1 kHz。然而，通常它计算上更有效地减缓所有电机的更新率，使用跳线。

改变伺服更新率的原因

在您的系统中伺服环应当以多快的速度进行更新？对于大多数应用，默认的 2.26KHz（442 微秒）更新的设置可以保留。要更改该时间有两个基本原因：

1、**增加速率的理由**：如果没有得到所要求的动态性能，加快伺服更新率（减少更新时间）。在大多数系统中，更快的更新速度意味着可以闭合更有刚性、响应性更好的回路，从而导致更小的误差和滞后。

2、**减少速率的理由**：如果比伺服回路优先级低的例程执行速度不够快，可以考虑减缓伺服更新率（增加更新时间）。它可能比必要的动态性能所需的更新得更快。如果是这样，处理器时间被浪费在不必要的额外更新上。例如，加倍伺服更新时间，从 442 微秒到 885 微秒（更新率从 2.26KHz 减半至 1.13KHz），几乎加倍了运动和 PLC 程序执行可用的时间，从而允许更快的运动模块速率和 PLC 的扫描速率。

有一些系统在较慢的伺服更新率下获得了更好的性能。一般来说，这些系统都有相对较低的编码器分辨率，通常是只在负载的一个编码器，其中微分增益不能足够调高到给予足量的阻尼，而没有由放大的量化误差造成的不稳定的高频振动。在这种情况下，减慢更新速率（增加更新时间）可以帮助得到足够的阻尼而没有过度的量化噪声。

更改伺服更新率的影响

如果改变了伺服更新时间，许多现有的伺服增益 Ixx30--Ixx39 都会表现不同。要保留等效的伺服性能，更改这些值。请参阅软件参考手册中的 I 变量描述，其中有各个增益 Ixx30-Ixx35 的详细介绍，看看这些是如何变化的。请参考下面的陷波滤波器的部分，了解如何重新计算陷波滤波器参数 Ixx36-Ixx39。

伺服更新率的变化会改变用于伺服任务的处理器时间的百分比，而它对低优先级的任务，如运动程序和 PLC 程序的计算，具有重要的意义。请参考计算功能部分，获取关于如何评估这些变化的详细信息。

如果伺服更新时间通过跳线改变，改变全局参数 I10 来匹配该变化，以使得指令轨迹是在合适的速度执行。I10 不是必须被改变，以匹配各个电机 Ixx60 的变化。

9-2 放大器的类型

Turbo PMAC 可以接口到各种放大器类型。用于特定电机或液压阀的放大器类型对于伺服回路的整定有重要的影响。每个常见类型解释如下。

伺服产生速度指令的放大器

几种类型的放大器期望收到一个 Turbo PMAC 伺服环发出的速度指令。此类的主要类型的放大器是：

- 模拟-输入速度-模式伺服放大器
- 脉冲和方向输入放大器
- 液压阀放大器

如果来自 Turbo PMAC 的伺服回路的指令值是一个速度指令，不管信号是什么类型，在 Turbo PMAC 上不需要闭合速度环。在标准的 PID 回路，这意味着微分项（D）Ixx31 可以设置为 0。

模拟-输入速度-模式放大器

模拟输入速度模式的伺服放大器关闭放大器的一个速度环，使用 Turbo PMAC 的信号作为指令速度和传感器反馈作为实际速度。至关重要的是，在尝试调整它周围的 Turbo PMAC 伺服环之前，放大器的速度环要适当整定。

在调节 Turbo PMAC 的位置环之前，一个速度模式驱动器的速度环必须很好地调整，根据它将驱动的负载。由于速度环的调整取决于负载，放大器制造商做不了最后的调整；机器制造商必须调整该环。速度阶跃响应绝对不能有任何显著的超调和振荡；如果它有的话，不可能使用 Turbo PMAC 闭合它周围的一个很好的位置环。Turbo PMAC 执行程序的调整部分有一个被称为“开环调整”的功能，可以用来给放大器速度指令阶跃，并观察绘制于屏幕上的响应。这使得调整放大器很容易，或确认它已经调整的很好。

脉冲和方向输入放大器

脉冲和方向输入放大器解释每个脉冲作为一个指令位置增量。要产生脉冲和方向指令，Turbo PMAC 伺服环会计算发送到脉冲频率调制电路的脉冲频率值。这个频率值是一个有效的速度指令。

带有这种类型接口的放大器有两种类型---步进驱动器和替换步进放大器。

步进驱动器：这种驱动器没有位置反馈。通常，对于这些电机根本没有编码器，所以 Turbo PMAC 必须使用输出脉冲序列作为模拟反馈。这需要使用 Turbo PMAC 的编码器通道，即使是没有编码器的物理连接。如果步进电机上有一个编码器，它可以以两种方式使用：

1、它可以作为定期反馈到 Turbo PMAC，正如在一个伺服电机上。在该方法中，关键的问题是分辨率和相对于由驱动器产生的步或微步的编码器的边沿的定相---可能必须用 Ixx64 和 Ixx65 创建一些死区，以防止搜索静止。

2、编码器可以只用于在移动的末尾确认位置。然而，这种技术需要使用 Turbo PMAC 的两个编码器通道：一个用于脉冲序列的模拟反馈，一个用于确认编码器。

替换步进伺服放大器

这些采用伺服电机的位置反馈，并关闭驱动器内的所有回路。不要使用驱动器的编码器信号反馈到 Turbo PMAC 的伺服环内，因为驱动器和控制器中的位置环将相互冲突。有了这些驱动器，使用来自 Turbo PMAC 的指令脉冲序列作为模拟反馈。

当使用模拟反馈时，也可以用解析法设置 Turbo PMAC 伺服增益。请参阅《设置 Turbo PMAC2 为脉冲和方向控制》部分，获取详细信息。当使用真正的编码器反馈时，调整伺服环就像对一个模拟速度模式驱动。

液压阀放大器

液压阀放大器，无论是用于伺服阀还是比例阀，都控制正比于指令输入的流体体积流量。因为流进或流出液压缸的流体正比于筒体的移动部件的速度，所以进入阀放大器的命令是一个有效的速度命令。

伺服产生转矩/力命令的放大器

有几种类型的放大器也需要 Turbo PMAC 伺服环来关闭速度环，使该伺服环的输出为一个扭矩或力命令。如果 Turbo PMAC 对这个电机不做换相，则扭矩/力命令输出到放大器；如果 Turbo PMAC 做换相，这个命令是输入到换相算法。需要控制器关闭速度环的放大器的主要类型是：

- 模拟输入扭矩模式放大器
- 正弦曲线输入放大器
- 直接 PWM 电源模块放大器

如果来自 Turbo PMAC 伺服环的指令值是一个扭矩或力命令，不管信号是什么类型，Turbo PMAC 伺服必须对电机关闭速度环。在标准的 PID 回路，这意味着它的微分项（D）Ixx31 必须被设置为非零值。需要这个微分作用来得到稳定性所需要的阻尼作用。因为电动机产生的转矩或力正比于电机的电流，所以伺服发出的转矩/力指令也可以认为是一个电流指令。

没有必要调整放大器的任何东西当负载连接到电机上，因为在这些类型的放大器中没有速度环闭合。可能需要的调整仅取决于电机的属性，因此这可能甚至可以由放大器的制造商来完成。

模拟输入扭矩模式放大器

模拟输入“转矩模式”放大器接受一个表示扭矩/力的模拟电压，以及电流、

指令。这些放大器关闭一个内部电流环，而且如果用于无刷电机，也执行电机换相。这些类型的放大器偶尔使用的另一名称是“跨导放大器”，标志着一个电压输入产生一个成比例的电流输出。

正弦曲线输入放大器

一个正弦输入放大器接受两个相位-电流命令，在稳态是时间的正弦函数。这种类型的放大器期望控制器去计算换相，利用位置/速度环伺服的转矩/力指令作为电流幅度指令进入到换相。该放大器以此方式进行电流环闭合。

直接 PWM 电源模块放大器

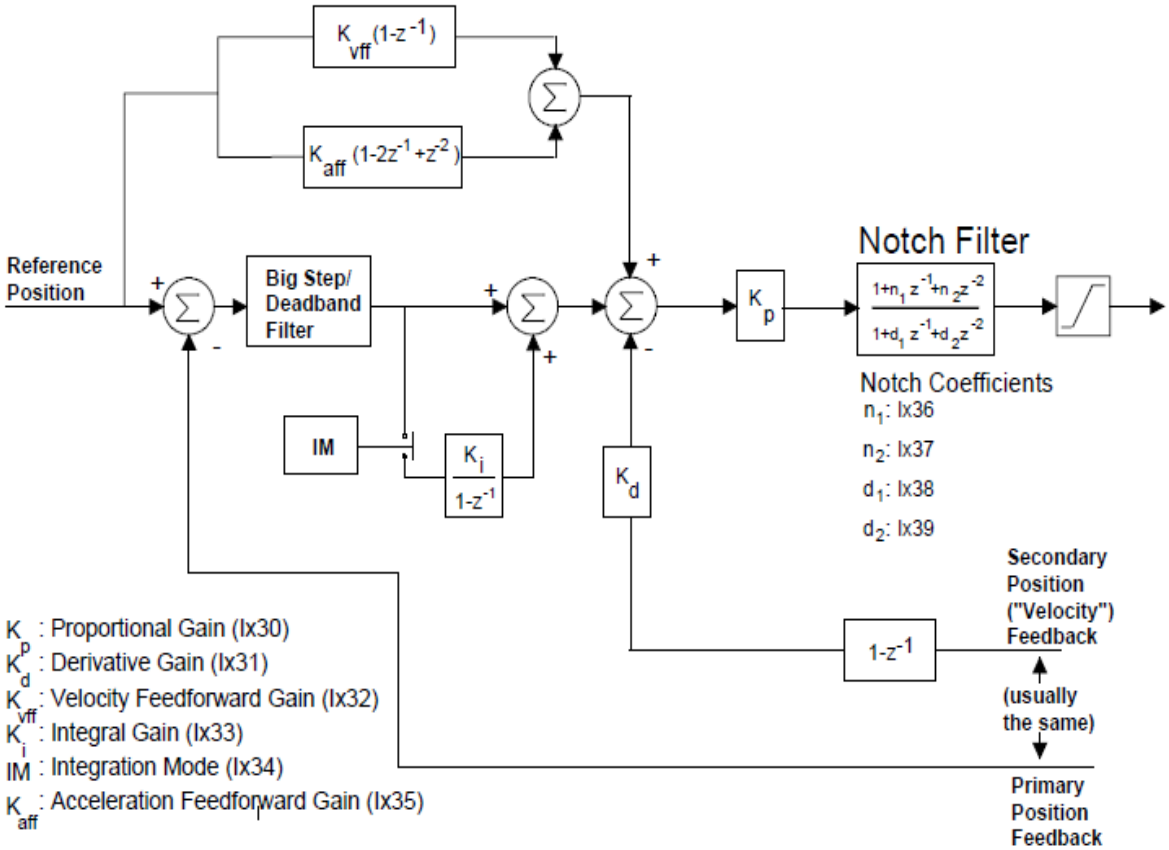
一个直接 PWM 电源块放大器接收相位电压指令编码为实际的脉冲宽度调制的开关命令用于功率晶体管。这种类型的放大器期待控制器来计算换相和电流环，使用位置/速度-环伺服的扭矩/力指令作为电流幅度命令，进入换相和电流回路中。该放大器不以此样式执行控制功能。

9-3 PID/前馈/陷波伺服滤波器

Turbo PMAC 的 PID/前馈/陷波伺服滤波器（简称为 PID 滤波器）是目前最常用的伺服滤波器。它很容易理解和调整，但功能强大到足以为绝大多数系统提供优异控制。

PID 滤波器是 TurboPMAC 上默认的选择。若要使用电机 xx 的 PID 滤波器，确保 Ixx59 的 0 位设置为 0，禁用任何的用户编写的伺服算法，并且 Iyy00/50（I3300 用于电机 1，I3350 用于电机 2，等）设置为 0，禁用扩展的伺服算法。

PMAC PID +陷波伺服滤波器



PID 反馈滤波器

PID 反馈滤波器由比例（P）、积分（I）和微分（D）项组成，每一项都对控制效果有它自己的贡献。

Ixx30 比例增益项

由 Ixx30 设置的比例增益项为位置误差提供基本纠正措施，提供一个正比于误差大小的控制效果，以尽量减少误差。单独的比例增益行为像一个弹簧，而比例增益项的大小是弹簧常数，这个增益项越高，弹簧作用越硬。

在 Turbo PMAC 中，Ixx30 是整个环路的增益项，后乘其他的增益项，而不仅仅是一个比例增益项。这使得，如果一个外部增益项（例如，编码器的分辨率或放大器增益）改变时，可以单独改变 Ixx30；它也分别将微分和积分的增益项规格化为一个时间常数和逆时间常数。

Ixx31 微分增益项

由 Ixx31 设置的微分增益项提供了一个阻尼效果，通过提供一个正比于实际速度的、与该速度相违背的控制效果。在这方面，它的作用就像一个车辆悬架的缓冲器或减震器。微分增益项越高，阻尼作用越明显。

一个稳定的位置环需要某种形式的微分作用---实际上是一个速度环。如果一个调整良好的速度环闭合在放大器中，Turbo PMAC 的 Ixx31 微分增益项可以设置为 0。然而，如果没有外部闭合的速度环，稳定的运行需要 Ixx31 为正值。

请注意，在 Turbo PMAC 中，这个增益作用在实际位置的导数，而不是位置误差的导数，如在某些其它的控制器。这允许单纯使用双电机-负载反馈，通过电机上的一个独立传感器用于微分作用（由 Ixx04 指定），从负载的传感器用于比例和积分作用（由 Ixx03 指定）。

Ixx33 积分增益

由 Ixx33 设置的积分增益项对一些效应引起的稳态误差提供校正，如摩擦，重力负载，切削负荷，模拟偏移。积分增益项控制位置误差积分项充电放电的速度；增益越高，作用越快

Ixx34 是一个单-位的变量，控制积分增益项被激活的时间。在默认值 1，积分增益只有当指令速度为零（在移动结束）才是激活的。当 Ixx34 为 0，积分增益项始终是激活的。当调整其他项时，一般最好是将 Ixx34 设置为 1 尽可能减少误差，不带积分器的；如果剩余误差小，通常是设置 Ixx34 为 0 使得积分器动态补偿剩余误差。

前馈滤波器

一个反馈滤波器是误差驱动的，因此在其采取任何操作之前，指令位置和实际位置之间必须存在误差。前馈的动作，另一方面，是只依赖于指令的轨迹，因此并不需要误差引起动作。前馈的基本思路是直接将你对所需控制效果的最佳估计应用于执行命令轨迹，而无需等待建立位置误差。然后反馈项只需要响应这个估计中的误差，这通常是相当小的。

在一个调整良好的系统中，超过 95％的控制作用来自前馈项，反馈项只是对估计中的干扰和不完善提供小的更正。Turbo PMAC 的 PID 过滤器具有速度和加速度前馈项，这里有介绍，和非线性摩擦前馈项，包含在伺服环修改器的标题

下。

Ixx32 速度前馈项

速度前馈项 Ixx32 添加一些量到控制工作中直接正比于指令速度，以克服潜在的比例于速度的位置误差。这些误差来自几个方面。第一个占主导地位的来源，来自速度反馈项，它提供了稳定性所需的阻尼，无论是在 Turbo PMAC 之中（Ixx31 项）还是在外部完成。其他次要的速度相关的误差来源包括电机中的磁损耗和实际的粘性阻尼损耗。

正确设置速度前馈将消除比例于速度的跟随误差分量。如果 Turbo PMAC 对电机闭合速度环，通常最佳的 Ixx32 将等于或稍大于 Ixx31。

Ixx35 加速度前馈项

加速度前馈项 Ixx35 添加一些正比于指令加速度的量到控制工作中，以克服潜在的与加速度成比例的位置误差。这些误差来自抵抗加速度的惯性基本倾向。如果没有加速度前馈，就会有一个比例于加速度的跟随误差分量。

正确设置加速度前馈将消除比例于加速度的跟随误差分量。该 Ixx35 加速度前馈项是该系统的惯性的估计值，直接提供一个比例于它的力或力矩，和指令加速度。

实际的 PID/前馈算法

用于计算电动机 xx 的指令输出的 PID/前馈算法的实际的方程式如下：

$$CMDout(n) = 2^{-19} * Ixx30 * [\{Ixx80 * [FE(n) + (Ixx32 * CV(n) + Ixx35 * CA(n)) / 128 + Ixx33 * IE(n) / 2^{23}]\} - Ixx31 * Ixx09 * AV(n) / 128]$$

其中：

- CMDout(n)是伺服周期 n 的 16 位输出指令（-32768 到+32767）。它被转换成一个-10V 至+10V 的输出。 DACout（n）由 Ixx69 限制。
- Ixx08 是一个内部位置缩放项用于电机 xx（通常设定为 96）
- Ixx09 是一个内部缩放项用于速度环用于电机 xx（通常设定为 96）
- FE（n）是伺服周期 n 中的跟随误差，单位是计数，它是周期的指令位置与实际位置之间的差值［CP（n）- AP（n）］
- AV（n）是伺服周期 n 的实际速度，这是在每个伺服周期的最后两个的实际位置的差值［AP（n）- AP（n-1）］
- CV（n）是伺服周期 n 的指令速度：每个伺服周期的最后两个指令位置之间的差值［CP（n）-CP（n-1）］
- CA（n）是伺服周期 n 的指令加速度，它是每个伺服周期最后两个指令速度之间的差值［CV（n）- CV（n-1）］
- IE（n）是伺服周期 n 的积分的跟随误差：

$$\sum_{j=0}^{n-1}[FE(1)]$$

（对于所有的伺服周期,积分是激活的。当 CV 不等于零时，Ixx34=1 关闭到积分器的输入，而不是积分器的输出。）

陷波滤波器

Turbo PMAC 的标准伺服环包括一个陷波滤波器。这是一个二阶的双二阶滤波器作用于伺服环的 PID 部的输出端，其主要目的之一是在伺服响应中创造一个陷波（低频率响应的），为了抵消谐振。

此过滤器有几种可能的用途：

- 反谐振（陷波）滤波器
- 低通滤波器
- 速度环积分
- 超前-滞后滤波器

每种用途将在下文中各自的部分介绍。

滤波器结构

对于那些熟悉控制理论的（没有必要使用滤波器！），Turbo PMAC 的陷波滤波器系统的形式是：

$$\frac{N(z)}{D(z)} = \frac{1 + N_1 z^{-1} + N_2 z^{-2}}{1 + D_1 z^{-1} + D_2 z^{-2}} = \frac{1 + Ixx36 z^{-1} + Ixx37 z^{-2}}{1 + Ixx38 z^{-1} + Ixx39 z^{-2}}$$

I-变量 Ixx36、Ixx37、Ixx38 和 Ixx39 各自的取值范围是-2.0 至+2.0；他们是 24 位的值，一个符号位、一个整数位和 22 位小数位。

用来创建一个陷波

在反馈控制中，陷波滤波器是一个反谐振滤波器，用于抵消物理共振。虽然关于如何设置陷波滤波器有许多不同的理念，我们建议设置一个轻微阻尼的带阻滤波器，中心频率为约 90%的谐振频率，和一个大阻尼的带通滤波器，通频带稍大于谐振频率（以减少滤波器本身的高频增益）。该带阻滤波器实施在滤波的分子上[N(Z)]，在控制项上创造零点；带通滤波器实施在滤波的分母上[D(Z)]，在控制项中创造“极点”。

自动陷波说明

Turbo PMAC 执行程序允许您非常简单地设置一个陷波滤波器，而不需要了解一个陷波滤波器的工作原理。最简单的方法就是输入你想要控制的机械共振频率。执行程序将计算带阻和带通滤波器所需的特性，计算系数，并将它们下载到 Turbo PMAC 中。

或者，您也可以单独指定带阻和带通滤波器所需的特性。滤波器的各个部分的这两个特征是固有频率 ωn 和阻尼比 ζ 。执行程序将计算系数来实现这些特性，并将它们下载到 Turbo PMAC 中。

手动陷波说明

要手动计算陷波滤波器的系数，需要考虑陷波滤波器的连续传输函数：

$$G(s) = \frac{s^2 + 2\xi_z \omega_{nz} + \omega^2_{nz}}{s^2 + 2\xi_p \omega_{np} + \omega^2_{np}}$$

先从滤波器的五个参数入手：

- ωnz：零点的固有频率，单位是弧度/秒（不是 Hz）
- ζz：零点的阻尼比
- ωnp：极点的固有振动频率，单位为弧度/秒（不是 Hz）
- ζp：极点的阻尼比
- Ts：伺服回路的采样周期，单位是秒（不是毫秒）

要从 Hz 计算为弧度/秒，乘以 2π（6.283）。要从以 kHz 为单位的采样率计算成以秒为单位的采样周期，首先将采样率乘以 1000 得到 Hz，然后取倒数。请记住，采样周期等于（Ixx601）乘以伺服中断周期。首先，计算以下的中间值：

$$\alpha_z = 1 + 2\xi_z\omega_{nz}T_s + \omega^2{}_{nz}T^2{}_s$$

$$\alpha_p = 1 + 2\xi_p\omega_{np}T_s + \omega^2{}_{np}T^2{}_s$$

然后计算出滤波器的系数：

$$Ixx36 = \frac{-(2\xi_z\omega_{nz}T_s + 2)}{\alpha_z}$$

$$Ixx37 = \frac{1}{\alpha_z}$$

$$Ixx38 = \frac{-(2\xi_p\omega_{np}T_s + 2)}{\alpha_p}$$

$$Ixx39 = \frac{1}{\alpha_p}$$

最后，修改比例增益项，以补偿该滤波器形成的直流增益变化：

$$Ixx30_{新} = Ixx30_{旧}\frac{\omega^2{}_{np}}{\omega^2{}_{nz}}\frac{\alpha_z}{\alpha_p}$$

例如，假设我们已经确定了机械耦合的共振频率为 55 Hz。为了补偿这一点，我们决定放置一个固有频率为 50 Hz 的小阻尼的带阻滤波器（阻尼比 0.2），和一个固有频率为 80 Hz 的大阻尼带通滤波器（阻尼比 0.8），以限制该滤波器的高频增益。伺服更新时间是默认的 442 微秒。

$$T_S = 442\mu s * 10^{-6}\frac{S}{\mu S} = 0.000442s$$

$$\omega_{nz} = 2\pi * 50Hz = 314.2rad/s$$

$$\omega_{np} = 2\pi * 80Hz = 502.7rad/s$$

$$\alpha_z = 1 + 2\xi_z\omega_{nz}T_s + \omega^2{}_{nz}T^2{}_s$$
$$= 1 + 2 * 0.2 * 314.2 * 0.000442 + 314.2^2 * 0.000442^2 = 1.0748$$

$$\alpha_p = 1 + 2\xi_p\omega_{np}T_s + \omega^2{}_{np}T^2{}_s$$

$$= 1 + 2 * 0.8 * 502.7 * 0.000442 + 502.7^2 * 0.000442^2 = 1.4049$$

接下来，我们计算出滤波器的系数：

$$Ixx36 = \frac{-(2\xi_z\omega_{nz}T_s + 2)}{\alpha_z} = \frac{-(2 * 0.2 * 314.2 * 0.000442 + 2)}{1.0748} = -1.912$$

$$Ixx37 = \frac{1}{\alpha_z} = \frac{1}{1.0748} = 0.930$$

$$Ixx38 = \frac{-(2\xi_p\omega_{np}T_s + 2)}{\alpha_p} = \frac{-(2 * 0.8 * 502.7 * 0.000442 + 2)}{1.4049} = -1.667$$

$$Ixx39 = \frac{1}{\alpha_p} = \frac{1}{1.4049} = 0.712$$

最后，我们计算直流增益调节，假设的例子，我们现有的比例增益项 Ixx30 是 500,000：

$$Ixx30_{新} = Ixx30_{旧}\frac{\omega^2{}_{np}}{\omega^2{}_{nz}}\frac{\alpha_z}{\alpha_p} = 500{,}000\frac{502.7^2}{314.2^2}\frac{1.0748}{1.4049} = 979{,}169$$

用来创建一个低通滤波器

另外，如果降低操作的粗糙度比高系统带宽更重要，也可以使用该过滤器组件作为一个低通滤波器。通常情况下，如果使用一个低分辨率的位置传感器，使用一个低通滤波器。

自动说明

Turbo PMAC 执行程序允许一个低通滤波器的设置起来很容易，而不需要了解它是如何工作的。输入滤波器的截止频率（高于该频率，不能通过许多信号强度），并选择要建立一阶还是二阶滤波器。执行程序将计算带阻和带通滤波器所需的特性，计算系数，并将它们下载到 Turbo PMAC 中。

手动说明

手动计算一个低通滤波器时，指定弧度/秒的截止频率，以及单位为秒的伺服更新频率。

一阶滤波器：要计算一阶低通滤波器，考虑滤波器的连续传递函数：

$$F(s) = \frac{1}{s/\omega + 1} = \frac{\omega}{s + \omega}$$

其中 ω 是截止频率，单位是弧度/秒（等于 2πf，其中 f 是截止频率，单位是 Hz）。值 ω 等于 1/τ，其中 τ 是滤波器的时间常数。

接着，将其转换为一个离散的时间传递函数，使用近似值 s =（1-Z^{-1}）/Ts，其中 Ts 是伺服更新时间，单位是秒。

$$F(z) = \frac{\omega}{\frac{1 - z^{-1}}{T_S} + \omega} = \frac{\omega T_S}{1 - z^{-1} + \omega T_S} = \frac{\omega T_S}{1 + \omega T_S} * \frac{1}{1 - \frac{1}{1 + \omega T_S}z^{-1}}$$

在 Turbo PMAC 而言，乘以增益项到现有的增益项 Ixx30：

$$Ixx30_{新} = Ixx30_{旧} * \frac{\omega T_S}{1 + \omega T_S}$$

极点项使用一阶陷波滤波器极点参数 Ixx38。如果滤波器仅被用作一个低通滤波器，其他滤波器的参数 Ixx36、Ixx37 和 Ixx39 被设置为零。

$$Ixx38 = -\frac{1}{1 + \omega T_S}$$

例如，为了在 Turbo PMAC 上实现一个一阶低通滤波器，截止频率为 50Hz，伺服更新时间为 442 微秒，我们计算：

$$\omega T_S = 2 * \pi * 50 * 0.000442 = 0.139$$

$$Ixx30_{新} = Ixx30_{旧} * \frac{0.139}{1 + 0.139} = 0.122 * Ixx30_{旧}$$

$$Ixx38 = -\frac{1}{1 + 0.139} = -0.877$$

$$Ixx36, Ixx37, Ixx39 = 0$$

二阶滤波器：要计算一个二阶低通滤波器，我们考虑广义二阶滤波器的连续传输函数：

$$F(s) = \frac{\omega^2{}_n}{s^2 + 2\xi\omega_n + \omega^2{}_n}$$

其中ωn 为滤波器的截止频率，单位是弧度/秒，ξ是阻尼比---0.707 的值在这里产生一个 Butterworth 滤波器。

首先，计算出以下的中间值：

$$\alpha = 1 + 2\xi\omega_n T_S + \omega^2{}_n T_S{}^2$$

然后计算出滤波器的系数：

$$Ixx38 = \frac{-(2\xi\omega_n T_S + 2)}{\alpha}$$

$$Ixx39 = \frac{1}{\alpha}$$

如果这个过滤器没有其他的用途，Ixx36 和 Ixx37 应设置为 0。

最后，修改你的比例增益项，以补偿该滤波器形成的直流增益变化：

$$Ixx30_{新} = Ixx30_{旧}\frac{\omega^2{}_n T_S{}^2}{\alpha}$$

例如，如果要在 Turbo PMAC 上实现一个二阶低通滤波器，具有 60Hz 的截止频率和 0.707 的阻尼比，有 250 微秒的伺服更新时间，我们计算以下内容：

$$\omega_n T_S = 2 * \pi * 60 * 0.000250 = 0.0942$$

$$\alpha = 1 + 2 * 0.707 * 0.0942 + 0.0942^2 = 1.146$$

$$Ixx38 = \frac{-(2 * 0.707 * 0.0942 + 2)}{1.146} = -1.861$$

$$Ixx39 = \frac{1}{1.146} = 0.873$$

$$Ixx30_{新} = Ixx30_{旧}\frac{0.0942^2}{1.146} = 0.00774 * Ixx30_{旧}$$

$$Ixx36, Ixx37 = 0$$

用来创建一个速度环积分

这个滤波器也可以被用来创建一个速度环内的积分器，独立于位置环的积分增益项。这个额外的积分器可以提供额外的刚度和抗扰动。但是，它可能妨碍对加速度命令的快速响应。

手动说明

考虑速度环中的 PI 滤波器，传递函数为：

$$V(z) = K_{pv} + \frac{K_{iv}}{1 - z^{-1}}$$

其中K_{pv}是速度环比例增益，K_{iv}是速度环积分增益。这可以被操作以产生：

$$V(z) = \frac{K_{pv}(1 - z^{-1}) + K_{iv}}{1 - z^{-1}} = \frac{K_{pv} + K_{iv} - K_{pv}z^{-1}}{1 - z^{-1}} = (K_{pv} + K_{iv})\frac{\left(1 - \frac{K_{pv}z^{-1}}{K_{pv} + K_{iv}}\right)}{1 - z^{-1}}$$

在 Turbo PMAC 而言，增益项$(K_{pv} + K_{iv})$乘到现有的增益项 Ixx30：

$$Ixx30_{新} = Ixx30_{旧} * (K_{pv} + K_{iv})$$

零点和极点项分别使用一阶陷波滤波器参数 Ixx36 和 Ixx38。如果滤波器仅被用作一个积分器，二阶参数 Ixx37 和 Ixx39 设置为零。

$$Ixx36 = -\frac{K_{pv}}{K_{pv} + K_{iv}}$$

$$Ixx38 = -1$$

用来创建一个超前-滞后滤波器

如果根是实根而不是虚根，该滤波器可用于简单地作为一个超前-滞后过滤器。超前-滞后滤波器在性能上与 PID 滤波器非常相似。当滤波器的设置分析地而不是实验地确定时，它是非常有用的。当需要一个基本的超前-滞后滤波器伺服时，所有的伺服增益 Ixx31 到 Ixx35 应该被设置为 0；Ixx30 仍作为广义的增益项。

PMAC 执行程序目前还没有任何屏幕去协助超前-滞后滤波器的自动规范。

手动说明

一个数字超前-滞后滤波器的广义解析形式是：

$$L(z) = K\frac{(z+a)(z+c)}{(z+b)(z+d)}$$

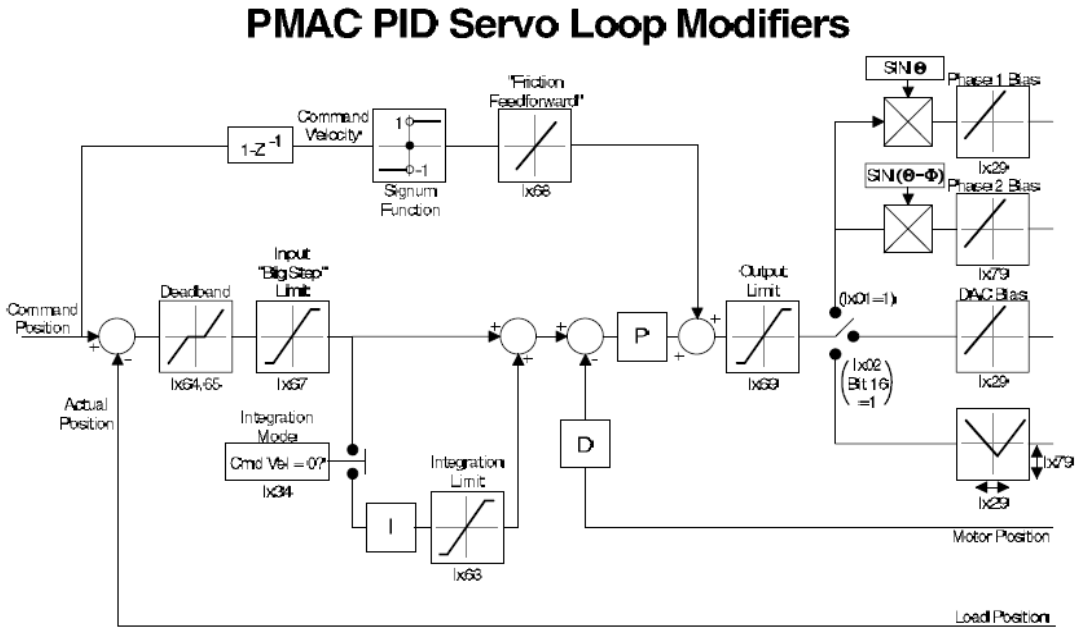
其中(z+a)/(z+b)是超前滤波器，a <b；(z+c)/(z+d)是滞后滤波器，c>d；K 是直流增益项。在 Turbo PMAC 的实时实现中，该滤波器的传递函数为：

$$L(z) = K\frac{1 + acz^{-1} + c^2z^{-2}}{1 + bdz^{-1} + d^2z^{-2}}$$

Turbo PMAC 项 Ixx30 设为 K；Ixx36 设置为 ac；Ixx37 设置为c^2；Ixx38 设定为 bd；和 Ixx39 设置为d^2。

伺服回路修改器

PID 滤波器有几个修改项---控制术语中的非线性---这是重要的以优化滤波器的性能和安全性。每一个都简述如下。

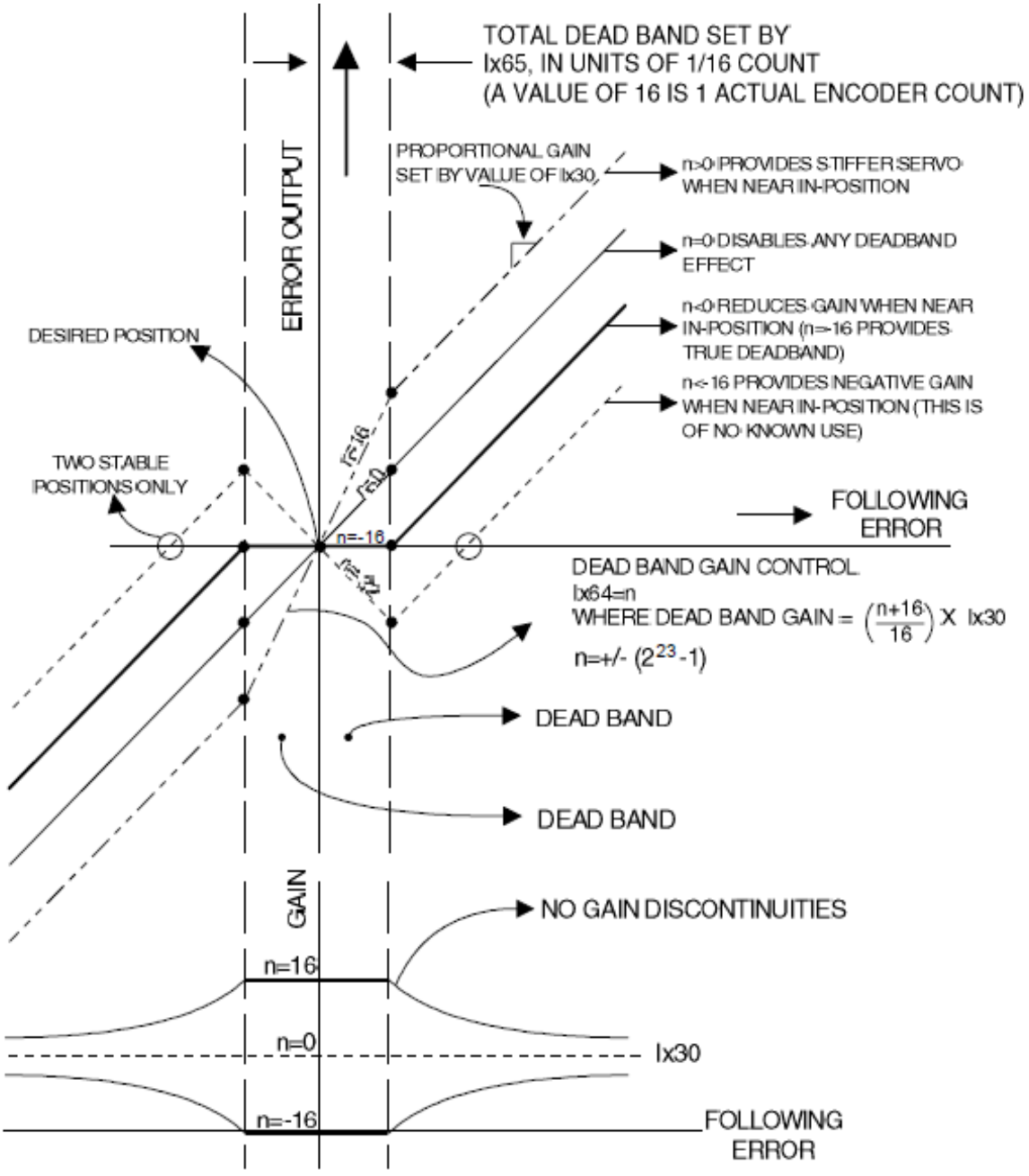


Ixx63：积分极限

Ixx63 是积分器的饱和极限，它限制了积分器的输出的大小。如果设置为负值，则如果积分器饱和，它会跳闸伺服回路，通过一个故障的跟随误差。设置 Ixx63 为 0 会清除积分器以及它的输出。(积分增益项 Ixx33 设置为 0，仅仅停止进一步输入到积分器。)

Ixx64，Ixx65：死区补偿

Ixx64 和 Ixx65 允许同时创建一个死区以及补偿物理死区(如在液压比例阀)。Ixx65 就伺服跟随误差的幅度而言指定死区补偿区域的大小。Ixx64 指定该区域内的增益相对于总比例增益 Ixx30。Ixx64 的正值增大区域内的增益，补偿物理死区；Ixx64 的负值减小区域内的增益，Ixx64=-16 产生真正的死区。



Ixx67：跟随误差极限

Ixx67 是关于输入到滤波器的 P 和 I 的跟随误差大小的饱和极限。它并不限制真实的跟随误差，或误差值相比于 Ixx11 和 Ixx12 跟随误差极限参数。Ixx67 设置为 0 将禁用 PI 控制项，同时保持微分项 D（即速度环）和前馈项在激活状态。如果只真正需要速度控制，此设置非常有用。如果 Ixx67 设置为 0，Ixx11 也应该设置为 0，以确保电机在一个故障跟随误差故障上不跳闸。

Ixx68：摩擦前馈

Ixx68 是一个非线性的摩擦前馈项。Ixx68 的幅度，乘以瞬时指令速度的符号，直接加到伺服滤波器的输出上。它是为了补偿干（库仑）摩擦。

Ixx69：输出限制

Ixx69 是关于伺服滤波器的输出的饱和极限。如果输出值是由 Ixx69 限制的，位置环积分器的输入将自动关闭，并且在此条件下，积分器的输出保持不变。

Ixx29，Ixx79：偏移项

如果 Turbo PMAC 不对电机 xx 进行换相，Ixx29 是伺服滤波器的输出的固定偏移项，单位是一个 16 位 DAC（即使使用其他一些设备）。如果 Turbo PMAC 对电机进行换相，Ixx29 和 Ixx79 作为换相算法中的相位偏移。

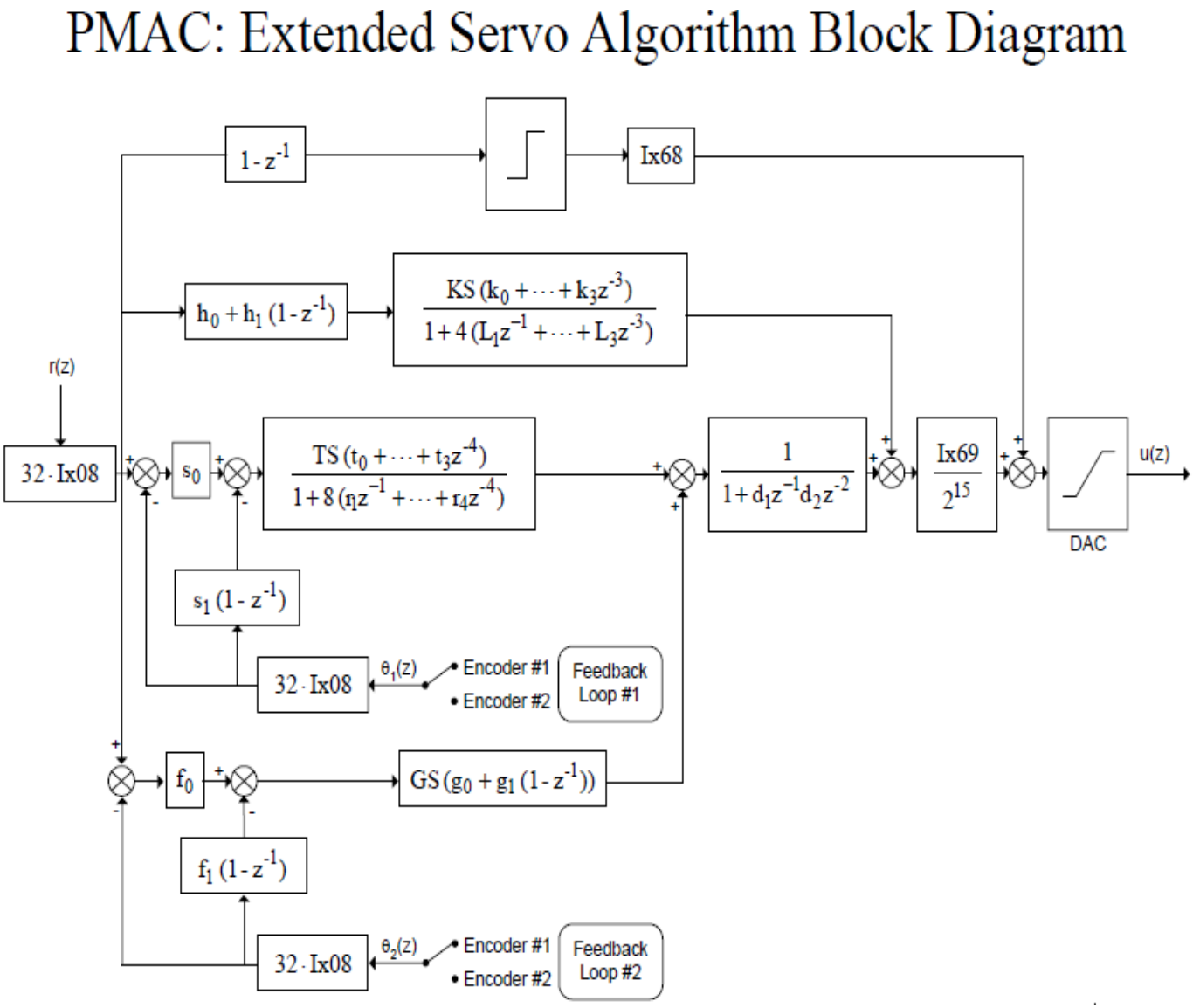
9-4 扩展的伺服算法

对于动态性能很差的系统，如多个谐振和低频共振系统，扩展伺服算法（ESA）可以用来代替 PID 滤波器。伺服滤波器的选择可在一个电机-电机基准上进行。

ESA 比 PID 虽然更强大、更灵活，但是不容易理解或交互地调整。PEWIN32PRO 执行程序对于 ESA 有自动调节软件。然而，手动调整 ESA 需要出色的控制理论知识和经验。这个讨论假设有这样的知识。

ESA 在 7 块多项式系数中有 30 个术语项。正如 PID，它使用 Ixx03 和 Ixx04 反馈地址变量支持单路和双路反馈。然而，ESA 关于双重反馈完成的任务更灵活，它不局限于独立的位置和速度环路。Ixx03 选定的反馈和指令轨迹值是预乘以比例系数 Ixx08，Ixx04 选择的反馈是预乘以比例系数 Ixx09。

正如 PID，或一个用户编写的伺服，使用 ESA 可带或不带基于 Turbo PMAC 的换相。如果电机不由 Turbo PMAC 换相，计算的控制输出被写入到 Ixx02 指定的寄存器；如果它由 Turbo PMAC 换相，控制输出是转矩（正交）命令进入换相算法。



选择电机 xx 的 ESA 是通过将第一个补充的电机 I 变量 Iyy00/50（I3300 用于电机 1，I3350 用于电机 2 等---见软件参考手册中的完整表）设置为 1，并将 Ixx59 的位 0 设置为默认值 0 以禁用“用户编写的伺服”。在这个设置下，伺服环项是补充的 I-变量 Iyy10/60 到 Iyy39/89（I3310-I3339 用于电机 1，I3360 -I3389 用于电机 2，等等）。下面的表列出了用于每个增益项的变量：

奇数电机的 I 变量	偶数电机的 I 变量	增益名	范围	奇数电机的 I 变量	偶数电机的 I 变量	增益名	范围
Iyy10	Iyy60	s0	-1.0≤变量<+1.0	Iyy25	Iyy75	TS	-2^{23}≤变量<2^{23}
Iyy11	Iyy61	s1	-1.0≤变量<+1.0	Iyy26	Iyy76	L1	-1.0≤变量<+1.0
Iyy12	Iyy62	f0	-1.0≤变量<+1.0	Iyy27	Iyy77	L2	-1.0≤变量<+1.0
Iyy13	Iyy63	f1	-1.0≤变量<+1.0	Iyy28	Iyy78	L3	-1.0≤变量<+1.0
Iyy14	Iyy64	h0	-1.0≤变量<+1.0	Iyy29	Iyy79	k0	-1.0≤变量<+1.0
Iyy15	Iyy65	h1	-1.0≤变量<+1.0	Iyy30	Iyy80	k1	-1.0≤变量<+1.0
Iyy16	Iyy66	r1	-1.0≤变量<+1.0	Iyy31	Iyy81	k2	-1.0≤变量<+1.0
Iyy17	Iyy67	r2	-1.0≤变量<+1.0	Iyy32	Iyy82	k3	-1.0≤变量<+1.0
Iyy18	Iyy68	r3	-1.0≤<+1.0	Iyy33	Iyy83	KS	-2^{23}≤变量<2^{23}
Iyy19	Iyy69	r4	-1.0≤变量<+1.0	Iyy34	Iyy84	d1	-1.0≤变量<+1.0
Iyy20	Iyy70	t0	-1.0≤变量<+1.0	Iyy35	Iyy85	d2	-1.0≤变量<+1.0
Iyy21	Iyy71	t1	-1.0≤变量<+1.0	Iyy36	Iyy86	g0	-1.0≤变量<+1.0
Iyy22	Iyy72	t2	-1.0≤变量<+1.0	Iyy37	Iyy87	g1	-1.0≤变量<+1.0
Iyy23	Iyy73	t3	-1.0≤变量<+1.0	Iyy38	Iyy88	g2	-1.0≤变量<+1.0
Iyy24	Iyy74	t4	-1.0≤变量<+1.0	Iyy39	Iyy89	GS	-2^{23}≤变量<2^{23}

ESA 由一系列模块组成，大多数是乘法项，每次取的一个输入值--可能是另一个模块的输出，并计算一个输出值--可能是另一个模块的输入。许多模块具有多项式传递函数，一个 n 次多项式表示模块的 n 个周期历史的存储。

其名称包括一个字母和一个数字的术语（项），乘以一个控制值，该值是 i 周期大，其中 i 是在名称中的数字（如 t2 乘以一个两个周期大的值）。如果该术语项在该模块的分子中，它乘以该模块的一个输入值；如果是在模块的分母，其乘以该模块的一个输出值。这些术语项的范围是+/-1.0，具有 24 位分辨率。

其名称由两个字母组成的术语项，第二个字母为 S，乘以整个模块的结果。这些术语项被视为一个整数，范围是+/-8,388,608。

PID 项 Ixx30--Ixx39，Ixx63--Ixx65 和 Ixx67 是不被使用的。Ixx68 作为“摩

擦前馈”术语项用于 ESA，就像它用于 PID。Ixx69 用于 ESA，但与 PID 的方式略有不同。在 PID，Ixx69 是一个截断限制对于控制作用输出，不影响较小的指令值；而在 ESA，它是一个输出比例因子，影响所有的输出指令值。

9-5 级联伺服环

Turbo PMAC 伺服环的开放式结构和指定寄存器用于其输入和输出的能力，为用户提供了强大的功能，如能够“级联”伺服回路。在这种技术中，一个伺服环的输出（1 个 Turbo PMAC“电机” ）被用作一个输入到另一伺服环，使得两个回路的能力实现在一个执行器中。外环不直接驱动执行器；而是动态地修改内环的设定点，为了驱动它自己的误差为零。

该技术具有许多可能的用途。最常见的是要能够在标准的位置环周围闭合一个辅助环。辅助环控制某些受位置环运动影响的量，如施加的扭矩或力，或相对一个表面的距离。环路的耦合可以开启和关闭，从而允许在控制模式之间轻松切换。

这种技术的常见用途包括：

- 卷筒纸张紧
- 限制转矩的螺纹传动
- 金属折弯
- 受控力部分插入
- 不平坦表面上的高度控制（例如，用于自动聚焦）

在这些应用中，内环是一个典型的标准位置环，驱动一个实际的执行器，通过一个标准位置反馈装置，如编码器或旋转变压器。要设立这样一个应用程序，第一步是使得该环工作在标准的定位模式下（运行在连续速度，如果适当）。

在这些应用中，外环使用一个反馈传感器，测量任何被控的量。通常，这些力或力矩传感器，如应变计或张紧的舞者手臂，或采用电容式或超声波机制的距离（间隙）传感器。

通过接合和脱离外环，用户可以切换控制，标准位置控制只使用内环--当不符合表面的阻力，而辅助功能的控制—以受控力推压一个表面。进行转换很简单，并且操作平稳。

这种技术的第二个用途是对单个电机建立一个比标准的滤波器更复杂的滤波器（例如合并一个双陷波滤波器）。通过使用第一个滤波器的输出作为到第二个的输入，可以把它们结合在一起，并获得命令和输出之间的两个滤波器的动作。虽然总的原理是一样的，但是设置的细节和使其工作的流程会有所不同。下面的章节覆盖了建立混合控制的过程。再往下的特殊章节描述了使用两个环进行设置来控制单个量的差异。

选择使用的 Turbo PMAC 电机

任何两个 Turbo PMAC 电机都可以用于内部和外环。如果将信息从外环传递到内环不需要积分（见下文），为外环使用较低编号的电机可避免增加一个伺服周期的延迟。这可能为关闭外环提供更高的性能。

然而，在许多应用中，外环的性能将是足够高的，即使是有添加的延迟，有

些用户会发现增加附加外环电机作为一个更高编号的电动机比实际的电机更容易的。例如，电机 1、2 和 3 分别是实际的 X、Y 和 Z 轴，电机 4 被添加为 W 轴以关闭作为 Z 轴的电动机 3 周围的环路。

内环常规设置

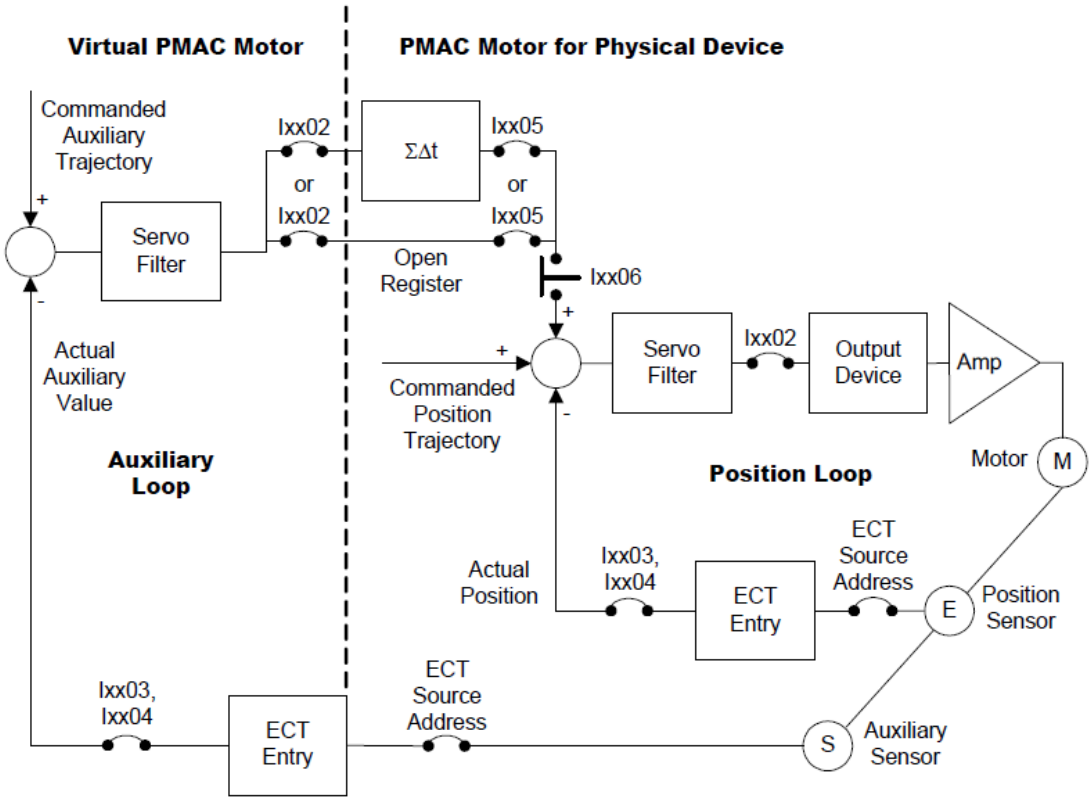
在混合控制应用中，首先设置内环为标准的定位电机，当以位置/速度模式控制该执行器时，把它调整好并根据需要运行。对于混合控制，我们将简单地添加一个外环命令到主位置输入；否则，这个内环的运行将保持相同。

外环常规设置

设置外环来使用替代传感器作为其反馈，通过其 Ixx03 和 Ixx04 反馈地址参数（通过转换表）。通常，此环路的控制量将不会是一个位置值，但由于所有的 Turbo PMAC 术语都是就位置而言，你应该提防可能出现的混乱。反馈传感器的一个最低有效位（LSB）被 Turbo PMAC 认为是一个计数。正如定位电机，轴定义语句的比例因子可以允许你编程这个电机，以工程单位（如牛顿或磅）。

你将无法调整外环，直到将其与内环路相连。下一节介绍连接环路的步骤。

用于混合控制的级联回路



连接环路

内环正常工作后，且外环的基本设置也已完成，将环路连接起来。

积分外环的命令，或不积分

有时，在用作一个内环的位置输入之前，外环的输出将被进行数值积分。如果它被积分，外环的命令本身将是一个有效的速度值；如果它没有被积分，该值将是一个位置值。在一般情况下，如果外环被接合时的稳态状况有一个非零的速度，如在卷筒纸张紧应用中，该值应积分。然而，如果稳态状况是在零速度，如在部分插入应用中，不应使用积分。

外环输出地址：Ixx02

该技术的关键是将外环的输出连接到内环的主位置。指定伺服输出地址的外环电机变量 Ixx02，可以指定任何开放的寄存器。对于该值这仅仅是一个临时保存寄存器，所以它并不需要是一个输出设备的寄存器。通常，地址范围为 $0010F0-$0010FF 的开放式 RAM 寄存器用于保存这个值。

如果该电机的 Ixx01 被设置为 2，Ixx02 指定的寄存器将是一个 X 寄存器。（请注意，外环电机将永远不会被换相，所以永远不会有必要将 Ixx01 的 0 位设置为 1）。如果在将它用于内环之前不需要积分这个值（该决定在下文讨论），将该值写入一个 X 寄存器将允许内环直接读取该值，而不经过编码器转换表的任何处理。然而，如果该值必须被积分，外环电机的 Ixx01 应该被设置为 0，因此该值被写入到一个 Y 寄存器中。

在转换表中积分

如果外环的命令输出在将它用作一个内环的输入之前被积分，将保存寄存器视为与 Acc-28 A/D 转换器的寄存器一样。积分 Acc-28 式寄存器的一个转换表入口，其首个十六进制方法数字是$5。该寄存器被视为一个有符号的量，所以 Bit-19 模式切换位被设置为 0。例如，如果保存寄存器的地址是 Y:$0010F0，入口的第一行将是$5010F0。

入口的第二行是一个偏移值，积分之前从读数中减去。由于这是使用一个计算值而不是测量值，这应该被设置为 0。

内环主地址：Ixx05

内环电机的 Ixx05 主位置地址变量包含了编码器转换表的结果的地址。这必须是一个 X 寄存器。如果外环将其输出直接写入一个 X 寄存器，内环电机的 Ixx05 可以包含这同一寄存器的地址。例如，如果电机 4 拾取一个开放式寄存器 X:$10F1 中的值，这可能是通过 **I405=$10F1** 指定的。

如果外环的命令是通过该转换表进行处理，该 Ixx05 将包含转换表中的结果的地址。请记住，你可以参照转换表 I-变量编号设置这个地址。如果转换表入口是通过 I8004 和 I8005 进行设置，您可以指定使用结果值用于电机 5，通过 **I505=@ I8005**。

内环跟随使能和模式：Ixx06

内环电机的 Ixx06 变量控制是否接合外环。当 Ixx06 的 0 位被设置为 0 时，不接合外环，而内环将独立运作。当 Ixx06 的 0 位设置为 1 时，外环被接合，其输出将修改内环的总指令位置。

内环电机的 Ixx06 也控制外环的校正与内环指令位置的交互方式。当 Ixx06 的 1 位（跟随模式控制位）设置为 0，内环的指令位置是相对于一个固定的原点，并且这些指令移动有效地抵消了任何通过主位置端口进来的校正。当 Ixx06 的位

1 设置为 1（偏移模式）时，通过主端口位置进来的校正有效地偏移了编程指令运动的原点，允许指令运动和主校正叠加在一起。这种模式的区别是真的，即使禁用跟随。

当外环被接合（Ixx06 的 0 位=1）时，通常跟随必须在偏置方式（Ixx06 的 1 位=1），使得 Ixx06 所需的值是 3 进行此操作。即使运动程序中没有明确的命令用于此时分配给内环电机的轴，对包含该电机的坐标系统的任何运动命令，都隐式地命令电机运动到其先前的指令位置。如果跟随不在偏移模式，这将除去自上次编程的移动或移动片段进来的修正。

当跟随被禁用（Ixx06 的 0 位=0）时，要命令内环电机到一个明确的物理位置，将跟随放在正常模式下（Ixx06 的 1 位=0），使 Ixx06 所需的值是 0 用于此操作。

内环主比例因子：Ixx07

内环电机的 Ixx07 变量，主比例因子，是该用途下的外环增益项。设置为 1，以使得净额外环增益（内环 Ixx07 乘以外环 Ixx30）尽可能低。如果必要反转两个环之间的耦合的方向感，它可以被设定为负的值。

调整外环

在混合控制的情况下，通常你将只需要外环的比例增益（Ixx30）或可能的积分增益（Ixx33）。大多数应用程序不需要微分增益（Ixx31），因为在大多数应用中外环只是试图保持恒定的指令值，通常前馈项（Ixx32 和 Ixx35）并不重要

如果你在内环使用它之前，积分外环的指令值，Ixx30 比例增益项可能会非常低（通常约为 10）。

可以使用执行程序的调整工具来调整外环增益作为一个标准的环。

编程外环电机

外环接合时，命令外环电机的位置将使得外环的反馈回路来计算内环命令的偏移，企图驱动外环的反馈装置到指令值。这个外环命令可以是一个电机微动指令，或是一个编程的轴命令。如果一个编程的轴命令，分配有外环电机的轴可以与内环电机在同一个坐标系中，或在一个不同的坐标系中。

最常见的是，外环电机将分配给与内环电机在同一坐标系统的一个轴，并且在相同的坐标系中进行命令。轴的命名规则和标准（如 RS/EIA-267）认为这些是次要轴，当与一个 X 轴匹配时，建议名字为 U，与 Y 匹配时名字为 V，与 Z 匹配时名字为 W。

设置示例

在这个例子中，电动机 1、2 和 3 分别是 X、Y 和 Z 轴，在笛卡尔坐标系统 1 中。每一个都使用正交反馈，分辨率为 0.1 微米，并且编程以毫米为单位。电动机 4 是用来控制表面上垂直工具的间隙高度。它采用的是一个电容间隙传感器通过一个 Acc-28 的 16 位 A/ D 转换器，ADC 的 LSB（最低有效位）测量 0.25 微米。它被分配到相同的坐标系下的 W 轴，编程单位也是毫米（间隙）。

```
I8003=$1F8E00          ;  ECT 第 4 入口:无符号的 A/D 来自 Y:$78E00
间隙虚拟电机基本设置
I400=1                 ; 激活电机 4
I401=2                 ; 输出到一个 X 寄存器
I402=$10F0             ; 输出到开放式寄存器 X:$10F0
I403=@I8003            ; 使用 ECT 第 4 入口的结果用于位置
I404=@I8003            ; 使用 ECT 第 4 入口的结果用于速度
垂直定位电机连接到虚拟电机
I305=$10F0             ; 使用＃4 输出作为主输入
I307=1                 ; 设置尽可能低的主比例因子
坐标系设定
＆1
＃1->10000X            ; 以毫米为单位的 X 轴定位
＃2->10000Y            ; 以毫米为单位的 Y 轴定位
＃3->10000Z            ; 以毫米为单位的 Z 轴定位
＃4->4000W             ; 以毫米为单位的垂直间距
```

改变控制模式

每当跟随模式控制位改变时，为内环重新排列相应的电机和轴的位置之间的关系。虽然这会在程序启动时通过 R 或 S 指令自动完成，但是如果该位是在一个运动程序的中间被改变，明确地命令重新调整，通过发出 **PMATCH** 命令。这是一个在线的命令；从一个运动程序中，它必须用 **CMD“PMATCH”**句法发布。

此命令通常必须用 DWELL 命令被前后包围起来；第一个是停止任何前瞻和混合（这可以是一个 DWELL 0），第二个是给出在后台在线命令时间执行命令队列，在计算下一个编程运动之前。

模式更改示例

下面的代码段说明了如何实现接合外环。本例延续了上文的例子，其中，Z 轴被分配到内环的电机，所以它是一个位置轴，W 轴已被分配到外环的虚拟电机。

```
Z10                    ; 内环上的纯位置移动
DWELL0                 ; 停止前瞻
I306=3                 ; 接合跟随，在偏移模式
CMD“＆1PMATCH”         ; 重新调整电机和轴位置
DWELL10                ; 给 PMATCH 命令的执行时间
W5                     ; 外环命令
```

下面的代码段说明了如何实现脱开外环。

```
W5                     ; 外环命令
DWELL0                 ; 停止前瞻
I306=0                 ; 脱开跟随，在正常模式
CMD“＆1PMATCH”         ; 重新调整电机和轴位置
DWELL10                ; 给 PMATCH 命令的执行时间
Z0                     ; 内环上的纯位置移动
```

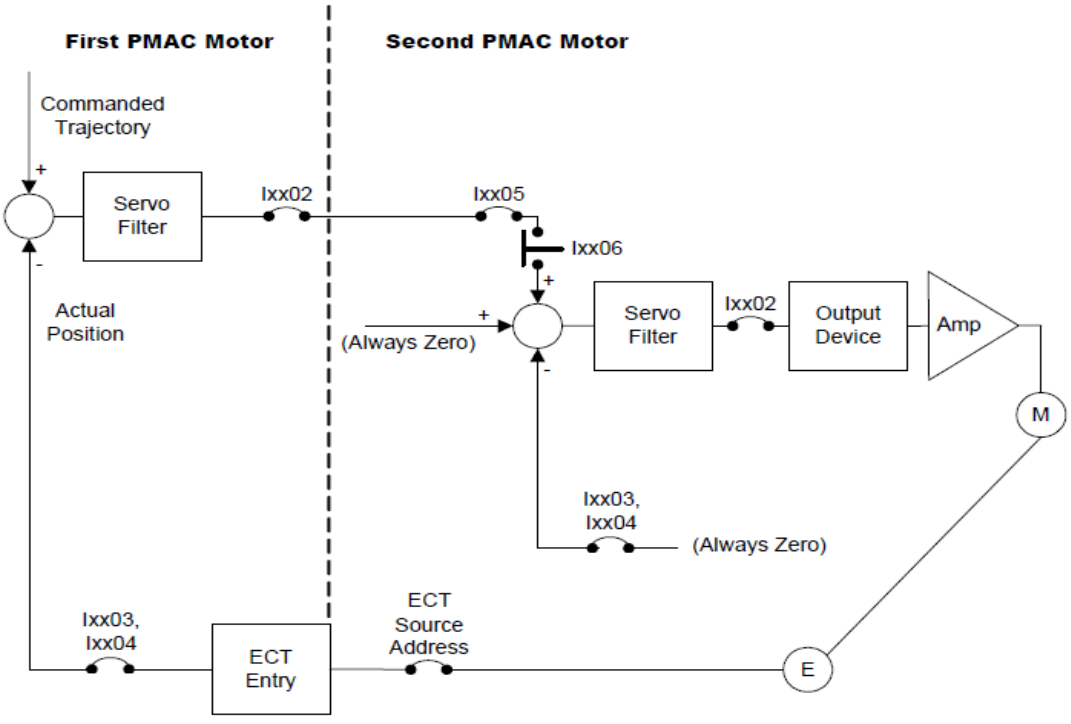
扩展单环设置的特别说明

当使用级联伺服回路用于单个控制量的扩展滤波时，设置的几个细节会有所不同。首先设置外环，使用真实的反馈装置，并直接写入可以命令放大器的寄存器中，如果有必要，进行换相。这个伺服环通过自身应尽可能调整好。

当准备好接合内环时，请执行以下操作：

1、对于外环电机，设置 Ixx01 为 2 指向一个输出到 X 寄存器。这也将禁用电机换相，如果你之前正在使用它。
2、对于外环电机，设置 Ixx02 为一个开放式寄存器的地址，通常为$0010Fx。
3、对于内环电机，设置 Ixx00 为 1 来激活它。
4、对于内环电机，设置 Ixx01 和 Ixx02 成与单独运行外环电机时相同的值。如果采用 Turbo PMAC 换相，还要为内环电机将 Ixx70-Ixx84 设置为与外环电机相同的值。
5、对于内环电机，将标志位参数 Ixx24 和 Ixx25 设置为与单独运行外环电机时相同的值。
6、对于内环电机，将反馈地址参数 Ixx03 和 Ixx04 设置为一个总是被设置为 0 的寄存器的地址。建议是转换表末尾的寄存器$35C0。
7、对于内环电机，将 Ixx05 设置成与外环电机 Ixx02 是同一个地址以拾取中间值。
8、对于内环电机，设置 Ixx06 为 1，启动使用 Ixx05 寄存器。
9、对于内环电机，将主位置比例因子 Ixx07 设置为 1。
10、对于内环电机，设置增益项 Ixx30 为 65,536，Ixx31- Ixx39 设为 0。这应该使内环透通且性能应该是与单独的外环相同。
11、现在调整内环电机的增益来获得你想要的额外滤波。

用于单量控制的级联回路



9-6 用户编写的伺服算法

Turbo PMAC 支持安装和自动执行用户编写的伺服算法。如果标准 PID 和 ESA 的滤波器不适合获得所需的伺服性能，可以使用这些。或者，它们可以用于非伺服目的，保证以伺服更新速率来执行该算法。这对于 I/O 的快速更新有价值。

有两种方法来创建这些用户编写的伺服算法。首先是所谓的开放式伺服的编译方法，用 Turbo PMAC 的高级语言，由 PEWIN32 PRO 执行程序编译。第二种是使用 DSP56300 系列的摩托罗拉交叉汇编程序的汇编方法。编译的方法更易于使用，更灵活，允许使用浮点数学。汇编方法生成的代码是更高效，更紧凑。这两种方法在下面讨论。

一个 Turbo PMAC 上只可安装一个用户编写的伺服算法。该算法可以通过 Turbo PMAC 的任何电机来执行。如果您希望不同的电机执行不同的用户编写的伺服算法，这必须通过在一个算法中进行分支来实现。

开放式伺服编译算法

Turbo PMAC 的开放式伺服功能允许以一种高级语言编写自定义算法，这将在 Turbo PMAC 的高优先级伺服中断时执行。该算法可以用于实际的伺服控制功能，也可以用于必须执行在一个非常高的优先级的其他任务，如超高频的 I/O，特殊的前处理反馈数据，或特殊的后处理伺服命令。

在被下载到 TurboPMAC 的活动内存之前，开放式伺服算法被编译成主机中的 DSP 机器代码。他们可通过使用 **SAVE** 命令保留在 Turbo PMAC 的非易失性闪存中。在执行时，它们仅更换电机的标准伺服回路的算法。所有其他任务，包括轨迹生成，运动和 PLC 程序的执行，以及安全检查，仍然是由 Turbo PMAC 内置的固件执行。

开放式伺服功能是第二种方法用于在 Turbo PMAC 中创建用户编写的伺服算法。先前，它的完成仅需要使用摩托罗拉的交叉汇编器，用 DSP56300 系列的汇编语言编写算法，并将编译代码下载到 Turbo PMAC。开放式伺服功能允许编写这些算法而不需要了解和使用汇编语言。

Turbo PMAC 仅仅可保存并执行一个开放式伺服算法。该算法可以通过多台电机运行。下面的部分解释了这个算法的工具访问电机特定的寄存器，如果不同的电机需要不同的程序，这必须由算法中明确的逻辑来处理。

编译的开放式伺服程序类似于编译的 PLC 程序，但有两个关键的区别：

- 开放式伺服算法在伺服中断上运行，保证每个周期都执行（否则 TURBO PMAC 将有看门狗）；编译的 PLC 程序要么运行在实时中断（PLCC0）有可能运动程序计算的优先占用，或在后台（PLCC1-31），没有确定的执行速率。
- 开放式伺服算法有特定的访问机制，用于伺服功能的特殊寄存器。

要求

开放式伺服需要一个 Turbo PMAC 控制器（Turbo PMAC、Turbo PMAC2、UMAC 或 QMAC）和版本 1.938 或更高版本的固件来执行算法。它需要一个 PC 运行着 PEWIN32PRO 版本 3.2 或更高版本的 PMAC 执行程序。

计算功能

开放式伺服提供了强大的计算功能，非常便于编写复杂和灵活的算法

访问 Turbo PMAC 变量

开放式伺服算法可以利用所有 Turbo PMAC 的 I、P、Q 和 M-变量，酌情读取以及写入到他们。正如在其他用户的程序，它使用浮点运算来处理这些变量的值，甚至是那些存储为定点值的（见浮点与定点数学，下文）。从开放式伺服算法中访问 Q-变量，根据坐标系统 1 的寻址方案，无论执行开放式伺服算法的电机被分配到哪个坐标系统中。

编译器分配的指针变量

为了直接和高效地访问 Turbo PMAC 寄存器，开放式伺服算法支持两种类型的指针变量，其中寄存器分配是在编译的时间，而不是在程序执行的时间。

L-变量是短（24 位）寄存器的指针，视为整数（定点）值。这些工作以与 L-变量同样的方式完成在编译的 PLC 程序。他们可以访问 X 或 Y 短寄存器，无论是作为全部 24 位寄存器（只视为有符号整数），或作为部分的寄存器 1、4、8、12、16 或 20 位宽（视为带符号或无符号的整数，除了唯一无符号的 1-位变量）。

F-变量是长（48 位）寄存器的指针。如果 F-变量定义是 L 格式（例如 **F1-> L:$10F0**），寄存器被访问作为一个 48 位浮点寄存器。如果 F-变量定义是 D 格式的变量（如 **F2-> D:$ 88**），寄存器被访问作为一个 48 位有符号整数，但从或到 Turbo PMAC 的 48 位浮点格式的转换是自动执行，因此它可以用在浮点数学。

注意：

不要混淆 L-变量和 L 型格式的 F-变量和 M-变量，L 型变量是短-字的编译器的指针，而 L 型格式的 F-变量和 M-变量是长-字变量。

Turbo PMAC 本身不能识别 L-变量或 F-变量，这些变量只对主计算机上的编译器有意义。

相比之下，当使用 Turbo PMAC 的 M 变量指针时，寄存器的分配是在执行（程序）行时，每次执行时。每次访问变量时，这个分配要求约 600 纳秒的额外计算时间（在 100 MHz 的 CPU）。然而，这允许在执行过程中改变 M 变量定义，从而启动诸如间接寻址的技术。

可以使用 L-变量用于快速整数运算，同时保留 M 变量定义的运行时间的灵活性，但是这增加了上文描述的运行时间定义访问计算惩罚。相比于对编译器直接定义寄存器的 L-变量，你可以参考一系列的 L 变量，到 Turbo PMAC 的 M 变量定义，通过 **LMOVERLAY {start},{end}**编译器指令。此指令必须领先于实际的开放式伺服程序。例如，**LMOVERLAY 10,20** 指示编译器，L10 到 L20 的定义是在运行时间分配的，通过分别使用 M10 到 M20 的定义，在执行每个语句的时候，而不是在编译时间。

使用 M 变量定义和在运行时访问这个定义，允许间接寻址技术，通过使用另一个指针变量实时修改 M 变量定义。

浮点与定点数学

开放式伺服的每个语句可以使用浮点或整数（定点）数学执行。在一个浮点语句，所用的所有变量，都通过一个 48 位浮点的中间工作格式进行处理，无论

该变量的存储格式是什么。浮点语句可以使用任何 Turbo PMAC 的 I、P、Q 或 M-变量以及编译器的长 F-变量指针。他们不能使用编译器的短定点 L-变量指针。这些语句中使用的所有常量都存储为 48 位浮点值。

在一个整数语句，使用的所有变量，通过一个 24 位有符号整数的中间工作格式进行处理，而不管该变量的存储格式是什么。整数语句只能利用编译器的短定点 L-变量指针。他们不能使用编译器的长的 F-变量指针，或 Turbo PMAC 的 I、P、Q 或 M-变量。在这些语句中使用的所有常量存储为 24 位有符号整数。所有使用的常量和运算的中间值必须符合这些整数的范围：-8,388,608～+8,388,607。在一个整数语句中不能使用数学函数（例如 SIN，COS）。

如果一行中一个常量出现在任何变量之前（例如，**IF（0 = P1）…**），该行被假定为浮点数。这意味着语句 **IF（0= L1）**是非法的，因为它混合了浮点和定点数据类型。

短-字整数运算会比对长浮点运算同样的操作快 10 倍以上，但通常不会有动态范围来处理实际的控制计算。然而，他们对相关的逻辑的高效计算有用。

注意

PMAC 内置的伺服算法（PID 和扩展伺服算法）使用长[48 位/56 位]定点数学。开放式伺服不支持此格式。

如果需要在整数和浮点数据类型之间转换，开放式伺服提供了 ITOF（整数到浮点）和 FTOI（浮点到整数）功能。该 ITOF 功能（带一个整数表达式作为其引数），可用于浮点声明语句中，如：

P20= ITOF（L10+ L11）*3.14159/P628

FTOI 功能（带有浮点表达式作为引数）可用于定点声明，如：

L10= L9+ FTOI（P5*100）-5

FTOI 函数将浮点表达式的值四舍五入到最接近的整数。

不允许在表达式中嵌套 FTOI 和 ITOF 功能。

数组。开放式伺服算法支持两种类型的数组：变量数组和寄存器数组。二者都提供了有用的功能。

变量数组：变量数组与 Turbo PMAC 的标准 PMAC I、P、Q 和 M 变量一起工作。数组索引的编号放在圆括号内，并对指定类型的变量指定变量号。决定这个编号的表达式是一个浮点表达式，因此它可以使用 Turbo PMAC 的 I、P、Q 或 M 变量，常量（这将被视为浮点值）和编译器的 F-变量，但它不能使用编译器的 L 型变量（除非该值已通过 ITOF 功能转换为浮点）。浮点表达式所得到的值被自动四舍五入到最接近的整数，以选择要使用的变量号。使用这些变量数组语句的一些实例是：

P(P1)= P10*32

P30= I(ITOF(L10)*100+30)* P29

寄存器数组：寄存器数组与编译器的短的 L 变量和长的 F-变量一起工作。在启动实际的开放伺服算法之前，这些数组必须声明到编译器。在使用中，数组的索引号被置于方括号内，并且指定相对该数组声明开头的地址偏移。决定这个数字的表达式是一个定点的表达式，所以它只能使用编译器的 L 型变量和常量，在一个 24 位有符号整数的范围之内。

在启动实际开放伺服算法之前，L 和 F-变量寄存器数组必须声明给编译器。这些定义的实例是：

L100-> X：$010000[32]

F200-> D：$030040[64]

F300-> L：$030080[128]

声明的数组的大小必须是 2 的幂，范围为 2～8192。L-变量寄存器数组始终使用完整的 24 位 X 或 Y 寄存器，将这些值视为有符号整数。

运算符

正如任何 Turbo PMAC 的用户程序，开放式伺服可以利用下面的数学和逻辑运算符：

- +（加）
- -（减）
- ×（乘）
- /（除）
- %（取模，余数）
- &（逐位与）
- |（逐位 OR）
- ^（逐位 XOR-异或）

所有这些运算符可以在浮点或整数语句中使用。整数除法将结果舍入为最接近的整数；在分数恰好为 0.5 的情况下，将舍入到下一个更正的整数（例如，-7.5 为-7，7.5 为 8）。

比较器

正如任何 Turbo PMAC 的用户程序，开放式伺服可以在条件语句中利用如下比较：

- =（等于）
- >（大于）
- <（小于）
- ～（约等于 [0.5 以内]）
- !=（不等于）
- !>（不大于，小于或等于）
- !<（不小于，大于或等于）
- !～（不约等于[不在 0.5 以内]）

～和!～的比较器只能用在浮点语句中。请注意，<、>、>=和<=比较器，可以在某些编程语言中使用，不能在开放式伺服或其他 Turbo PMAC 程序中使用。

函数

正如任何 Turbo PMAC 的用户程序，开放式伺服可以利用下面的数学函数。请注意，这些函数只能用于开放式伺服内的浮点声明：

- SIN（正弦三角函数）
- COS（余弦三角函数）
- TAN（正切三角函数）
- ASIN（反正弦三角函数
- ACOS（三角反余弦）
- ATAN（三角反正切）
- ATAN2（特殊的 2-参数，四象限的反正切）*

- ABS（绝对值）
- INT（范围内的最大整数）
- EXP（取幂）
- LN（自然对数）
- SQRT（平方根）

如果 Turbo PMAC 变量 I15 设置为默认值 0，三角函数使用度（°）；如果 I15 设置为 1，他们使用弧度。每次执行三角函数时，评估 I15 的现值。

注意

ATAN2 函数使用 Q0 作为其第二个参数-余弦参数。第一个参数-正弦参数-是在紧随 ATAN2 其后的圆括号内。开放式伺服使用的 Q0 始终是坐标系 1 的 Q0，无论执行电机被分配到哪个坐标系统中。

特殊的饱和度-检查函数

开放式伺服有一个特殊的函数，对照无符号的限制幅度来检查一个符号量，和在极限幅度下饱和该量。该函数 **FLIMIT（{value}，{limit}）**将参照于正值**{limit}**的符号量**{value}**和负值-**{limit}**进行比较。如果**{value}**大于**{limit}**，函数返回**{limit}**的值；如果**{value}**值小于**-{limit}**，函数返回**- {limit}**的值。否则，该函数只返回**{value}**的值。这两个量---**{value}**和**{limit}**---必须是浮点表达式（表达式可以是简单的变量或常量）。

例如，语句：

```
P2 = FLIMIT（P1，20000）
```

相当于：

```
IF（P1>20000）
   P2=20000
ELSE
   IF（P1<-20000）
      P2=-20000
     ELSE
     P2 = P1
     ENDIF
ENDIF
```

该 FLIMIT 函数减少了源代码和编译后的代码的大小。

特殊访问电机值和寄存器

开放式伺服算法有几个特殊的语句，支持实用的功能用于伺服环执行。

MTRNUM 函数：MTRNUM 函数返回当前执行算法的电机的编号（1-32）。编号被返回为一个定点（整数）的值。例如，当执行开放式伺服算法用于电机 2 时，语句 **L10= MTRNUM*100** 将定点变量 L10 设置为 200；或当用于电机 18 时，设置为 1800。同样，当执行开放式伺服算法用于电机 2 时，语句 **P10= ITOF（MTRNUM*100+30）**将浮点变量 P10 设置为 230；当用于电机 18 时，设置为 1830。

COPYREG 命令：该 **COPYREG** 命令将执行电机的五个关键寄存器复制到五个连续的 P 变量，在那里他们可以很容易地用于计算。用户不必知道这些寄存器的地址。在复制的过程中，Turbo PMAC 自动将数据转换成 48 位浮点格式。

此命令的语法是 **COPYREG{P-变量名}**，其中**{P-变量名}**指定复制数据的第一个变量的编号。由这个命令复制的 5 个寄存器是：

- 实际速度（1/[Ixx09*32]个计数/[Ixx60+1]伺服周期）
- 期望速度（1/[Ixx08*32]个计数/[Ixx60+1]伺服周期）
- 跟随误差（1/[Ixx08*32]个计数）
- 实际位置（1/[Ixx08*32]个计数）
- 预期位置（1/[Ixx08*32]个计数）

实际位置值是从由 Ixx03 选定的寄存器上得到的（位置环反馈地址），源值乘以比例因子 Ixx08 并扩展成一个 48 位的长字。实际的速度值是从 Ixx04 选择的位置值中导出的（速度环反馈地址），将这个周期的实际速度环位置值减去上一个环闭合时的值，并将差值与比例因子 Ixx09 相乘。注意，该比例因子不必与预期速度的比例因子相同。

对于所需的位置值，Turbo PMAC 添加了轨迹指令位置和主位置（从位置跟随，或电子齿轮功能），然后减去补偿的位置（从位置，或丝杠补偿表），产生一个净额的预期位置值。预期速度值仅仅是这个周期的预期位置值减去上一个环闭合时的值。

跟随误差值是预期位置减去实际位置。减法采用 48 位定点值；然后差值被转换为浮点格式。直接使用跟随误差值有几个优势。首先，它可以节省一些计算时间。其次，当指令位置和实际位置变得非常大时，它保留小数的位置数据更好。

如果使用命令 **COPYREG P5**，实际-速度值将被复制到 P5，期望速度复制到 P6，跟随误差复制到 P7，实际位置复制到 P8，期望位置复制到 P9。注意实际的和预期的速度寄存器之间的单位不同。（预期速度值通常不用于实际的伺服环闭合。Turbo PMAC 在数值积分过程中使用该寄存器来计算每个伺服周期的预期位置值。）

执行电机的寄存器的偏移：编译器的 L-变量和 F-变量可以通过对执行电机的特定寄存器的地址偏移量进行声明。在这种方式下，它们能自动正确地在电动机之间进行索引，从而允许多个电动机使用相同的变量和代码。这些变量可以通过到电机的 R0 寄存器的偏移量进行声明，其中 R0 寄存器是电机的命令输出寄存器（$BF 用于电机 1），或通过到电机的 R1 寄存器的偏移，R1 是电机的状态寄存器（$B0 用于电机 1）。L-变量可以这种方式声明到 24 位的 X 或 Y 寄存器； F 变量可以这种方式声明到 48 位的定点或浮点寄存器。一些例子：

L220-> X:（R1-$ 27） ；Ixx08 比例因子寄存器
L270-> Y:（R 1+0） ；电机状态寄存器
F392-> D:（R1-$24） ；电机主位置寄存器
F34-> L:（R0+11） ； Ixx16 最大指令速度

偏移量的范围必须是：-64<={偏移}<=63（-$40<={偏移}<=$3F）。

返回值：**RETURN** 命令使用其后面括号内的整数值，并将它放在一个 24 位的有符号整数寄存器中，在那里 Turbo PMAC 的标准固件将会拾取它并用它作为伺服命令。通常情况下，指令值将被计算为一个浮点值，所以必须通过 **ITOF** 函数将其转换成一个整数。 **RETURN** 命令的典型用途是：

RETURN（FTOI（P345））
L10= FTOI（P92/65536）
RETURN（L10）

RETURN 命令通常是一个开放伺服算法的最后一行。将其放在算法的较前几行也不会导致 Turbo PMAC 早点使用命令数据。如果开放式伺服程序用于一个伺服回路闭合之外的任务，没有必要使用 **RETURN** 命令。在这种情况下，当开放式伺服算法到达程序结束所需的 **CLOSE** 语句时，它会自动将一个 0 写入到保持寄存器中。

Turbo PMAC 将采用产生的结果值，并把转矩补偿寄存器（通常来自电机的 TCOMP 转矩补偿表）的内容加到它身上。如果 Turbo PMAC 不对该电机进行换相（Ixx01 的 0 位=0），它将采用这个总和值，并把它复制到由 Ixx02 指定的寄存器中。在这种情况下如果你没有使用 RETURN 命令，Turbo PMAC 仍然将已放置在保持寄存器中的、被 **RETURN** 命令使用的零值复制到由 Ixx02 指定的寄存器中。

如果 Turbo PMAC 对该电机（Ixx01 的 0 位=1）进行换相，它将使用所产生的总和值作为“正交电流”（扭矩）指令输入到换相算法中。在这种情况下，Ixx02 从换相算法指定多个输出寄存器。

该 Ixx29 和 Ixx79 偏移项是由 Turbo PMAC 自动添加的，就好像使用了内置的伺服算法。

返回值必须是在-8,388,608～+8,388,607 范围内的一个整数值。大多数与 Turbo PMAC 的自动伺服环相关的命令的输出范围被表示为 16 位的值，范围是 -32,768 到+32,767。与 **RETURN** 相关的值因此大了 256 倍。实际的命令输出设备不必有这个完整的 24 位分辨率（也可能不会）。然而，一般情况下，一个 n 位的输出设备使用 24 位返回值的高“n”位。

变量值赋值

一个开放式伺服算法中的数学运算是通过变量赋值语句执行的，就像在其他 PMAC 程序中。这些语句的语法规则与其他 PMAC 的解释和编译的程序中的是一样的。任何 I、P、Q、M、L 或 F-变量都可以分配一个值，无论是直接引用还是作为任何数组的一部分。

逻辑控制

开放式伺服算法中的逻辑分支和循环控制的执行是通过 **IF /[ELSE]/ ENDIF** 分支结构和 **WHILE/ ENDWHILE** 循环结构，就像在其他 PMAC 程序中。这些语句的语法规则与 **PMAC PLC** 程序中的是一样的；他们并不支持运动程序中可能的一些功能（比如在同一行作为一个条件的动作），但他们支持运动程序中不可能的一些功能（如多行条件）。请参考软件参考手册中的程序命令章节，获取详细信息（参见 **IF、ELSE、ENDIF、WHILE、ENDWHILE、AND、OR**）。

如果一个开放式伺服中使用 **WHILE/ENDWHILE** 循环，用户有责任确保该算法永远不要在一个循环停留太长的时间以至于其他任务受到损害。对于任何其他的同等或较低优先级的任务，Turbo PMAC 不会自动从开放式伺服的一个循环中释放。未能及时地从循环中释放可能会导致伺服误差（未能通过下一个伺服中断完成一个周期的伺服中断任务）、运行时错误（未能及时计算即将开始的运动的指令运动方程，导致运动程序中止）、或看门狗定时器错误（未能及时循环通过所需的所有任务，导致 Turbo PMAC 完全关闭）。

处理器利用率

伺服算法是由 Turbo PMAC 的处理器所执行的最重要的任务之一，但远远不是唯一的一个。虽然 Turbo PMAC 的 DSP 处理器非常有效率，但是处理器仍然有可能超负荷，特别是开放式伺服的编译的代码执行浮点算法时。这些运行时没有标准的伺服算法的效率好，因为后者用汇编语言编写并使用定点数学。

一般情况下，建议用于相位和伺服工作的处理器时间的部分不超过 50%，是为了给低优先级的任务预留足够的时间，如运动程序和 PLC 程序计算。为了找出开放式伺服算法占用多少的处理器时间，请参阅《评估 Turbo PMAC 的计算负载》的章节。这可以在本手册的《Turbo PMAC 的计算特性》中找到。

内存利用率

Turbo PMAC 的 DSP563xx 的 CPU 采用了哈佛结构，具有独立的程序或指令存储器和数据存储器。开放式伺服的实际指令被加载到 P 程序存储器中；它使用的所有数据寄存器是在 X 和 Y 数据存储器。

如果选定了一个 Opt 5Cx 80 MHz 的 CPU 配置，采用 DSP56303，只有 3K 字的程序存储器可用于开放式伺服（P：$040000 到 P：$040BFF）。这对于大的算法可能不够，因此建议用户选择 100 MHz 的 Opt 5Dx 或 160 MHz 的 5Ex CPU 配置，分别利用 DSP56309 和 DSP56311 的 CPU，用于任何复杂的算法。有了这些处理器，19K 字的程序存储器可用于开放式伺服码（P：$040000 到 P：$044BFF）。作为参考，下面的模仿基本 PID 算法的开放式伺服的例子占用 330 字的程序内存。

下载器会提示刚编译的程序占有多少字的程序存储器。这也可以通过读取内存位置 P：$040014 获得（使用 **RHP：$040014** 在线命令），其中包含 Turbo PMAC 程序存储器中开放式伺服算法结束时的位置；减去起始位置$040000（注意这些都是十六进制值）获得该程序的长度。

对于一般的数据存储器，大多数用户会利用一些 Turbo PMAC 的 8192 P-变量来存储值。用户有责任跟踪哪些 P-变量用于开放式伺服算法、哪些是用于其他用户任务。Q-变量也可用于开放式伺服算法，它们始终根据坐标系统 1 的编号访问。用户也有责任确保这些 P 或 Q-变量不用于其他任务。

许多用户会发现利用那些未使用的电机特定的寄存器有优势，因为开放式伺服正在执行用于该电机。被列为 PID 还是 ESA 算法的现有增益或中间值的寄存器都可以被开放式伺服安全地使用。在$000092-$0000AC 范围内的寄存器（用于电机 1；其它电机的是等效寄存器）除了$0000A5 之外（前一个净额预期位置）都可以用于此处。请注意，在$00009E 处的积分位置误差寄存器和$00009A 处的前一个净额预期速度寄存器在环路打开时会自动归零。不必要以与自动伺服算法相同的格式使用这些寄存器。

请注意，这些 PID 或 ESA 算法的寄存器表示一个 I 变量，其中任何一个将被写入到该 I-变量的命令所覆盖；这样一个寄存器中的值通过 **SAVE** 命令复制到闪存中；这样一个寄存器中的上次保存的值在电路板复位时从闪存中复制。一些用户可能想利用这些 I-变量寄存器作为增益用于自己的伺服算法---没有必要将它们用于相同的目的，或以相同的比例，如同内置算法那样。

对于大量的额外数据存储器，建议使用用户缓冲区设置，通过在线 **DEFINE UBUFFER** 命令。用户缓冲区占用大量的位于 X/Y 数据存储器的高端的寄存器。在 Opt 5x0 标准内存配置，数据存储器的末尾是在 X/Y:$0107FF；如果声明了 **DEFINE UBUFFER 2048**，所有的数据存储器从$010000 到$0107FF 都保留用于用户自己的目的。在 Opt 5x3 扩展内存配置，数据存储器的末尾是在 X/Y:$03FFFF；有一个

默认的 65,536 字的用户缓冲区，保留了所有的内存寄存器从 X/Y：$030000 到 X/Y:$03FFFF 用于用户使用。用户同样有责任确保用于开放式伺服数据存储的 UBUFFER 中的寄存器不用于其他用途。

编写开放式伺服程序

开放式伺服程序应该在一个纯文本编辑器中编写，如在新的 PEWIN32PRO PMAC 执行程序中的编辑器。虽然程序可以写在任何纯文本编辑器中，它必须由 PEWIN32PRO 编辑器的下载功能进行编译。在这个程序中，2001 年 10 月发布，下载程序会自动识别开放式伺服程序，编译他们，并下载所产生的机器代码。（旧版本的 PMAC 执行程序没有能力这样做。）

在包含开放式伺服、先于实际程序的文件中必须是所有的 L-变量和 F 变量指针定义、所有＃**define** 宏替换，或＃**include** 引用那些包含这些定义和替换的访问文件。记住，内置的编译器不会将这些定义和替换下载到 Turbo PMAC，它使用他们正常地做汇编。

OPEN SERVO 命令对于编译器是一个信号，在下载之前，跟进到 **CLOSE** 命令的语句被编译成 DSP 的机器代码。在解释缓冲区所使用的 **OPEN** 命令后的 **CLEAR** 命令，对于开放式伺服算法并不是必需的，因为下载新编译的代码会自动清除旧的代码，但它仍可以在这里使用。

例 1：比例控制

下面的算法显示了最简单的可用开放式伺服算法之一，利用电动机的 Ixx30 参数作为比例增益实现一个简单的比例控制规律。

```
OPEN SERVO                                          ；以下行进行编译
CLEAR                                               ；没有必要，但可以接受
COPYREG P30                                         ；复制跟随误差到 P32
P35= P32* I（ITOF（MTRNUM*100+30））/65536          ；乘以增益，比例
RETURN（FTOI（P35））                               ；变成一个整数，并输出
CLOSE
```

例 2：双二阶滤波器

下一个例子显示了实现一个双二阶滤波器，能够在多台电机运行，为每个电机存储周期之间的值。它使用＃**define** 替换宏来保持代码的可读性，以及变量的 **MTRNUM** 函数和寄存器数组来分隔开每个电机的存储值。变量数组（用户更容易访问）用于用户设置的“增益”，寄存器数组（算法访问更快）用于算法计算的存储值。

这个开放式伺服程序实现以下传递函数：

$$U(z) = K_p \frac{(z+a)(z+c)}{(z+b)(z+d)} E(z)$$

它通过下面的差分方程实现这个目标：

$$u_k = K_p[e_k + (a+c)e_{k-1} + ace_{k-2}] - (b+d)u_{k-1} - bdu_{k-2}$$

为了正确启动算法，它读取伺服周期计数器，并将它与上上次执行这个算法时的计数器进行比较。如果算法在前一周期没有执行，它为算法将“历史”值清零。它也对指令输出进行饱和度检查。该算法假定一个标准的内存选项（5x0），其数据存储器结束于 X/Y:$0107FF，定义有最少 2048 字的一个 UBUFFER。

定义和替换

```
#define     Kp          P(ITOF(MTRNUM*100+30))      ; Gain term is Pxx30
```

```
#define     A           P(ITOF(MTRNUM*100+31))      ; A zero is Pxx31
#define     B           P(ITOF(MTRNUM*100+32))      ; B zero is Pxx32
#define     C           P(ITOF(MTRNUM*100+33))      ; C pole is Pxx33
#define     D           P(ITOF(MTRNUM*100+34))      ; D pole is Pxx34
#define     E           P42                         ; Error term e(k) (not saved)
#define     Temp1       P45                         ; Temporary value
#define     Temp2       P46                         ; Temporary value
#define     Temp3       P47                         ; Temporary value
#define     U           P48                         ; Output term u(k) (not saved)
#define     LastE       F1[MTRNUM-1]                ; e(k-1) is F1[#-1]
F1->L:$010000[32]                                   ; Float reg array in UBUFFER
#define     PrevE       F2[MTRNUM-1]                ; e(k-2) is F2[#-1]
F2->L:$010020[32]                                   ; Float reg array in UBUFFER
#define     LastU       F3[MTRNUM-1]                ; u(k-1) is F3[#-1]
F3->L:$010040[32]                                   ; Float reg array in UBUFFER
#define     PrevU       F4[MTRNUM-1]                ; u(k-2) is F4[#-1]
F4->L:$010060[32]                                   ; Float reg array in UBUFFER
#define     ServoCycle L0
L0->X:$0,0,24,S                                     ; Servo cycle counter
#define     LastServoCycle          L1[MTRNUM-1]
L1->X:$010080[32]                                   ; Register array in UBUFFER
#define     ServoExtension L2
L2->Y:(R1-$21)                                      ; Ixx60 register as integer
#define     OutputLimit 8388607
```

启动实际算法

```
OPEN SERVO CLEAR
COPYREG P40                         ; Following error into P42
; If loop was not closed last cycle, zero out stored values
IF (ServoCycle-LastServoCycle!=ServoExtension+1)
    LastU=0
    PrevU=0
    LastE=0
    PrevE=0
ENDIF
LastServoCycle=ServoCycle           ; Store for next cycle
Temp1=Kp*(E+(A+C)*LastE+A*C*PrevE)  ; Compute TF numerator
Temp2=(B+D)*LastU+B*D*PrevU         ; Compute TF denominator
Temp3=Temp1-Temp2                   ; Combine for net command
U=FLIMIT(Temp3,OutputLimit)         ; Saturation check
PrevE=LastE                         ; Store values for next cycle
LastE=E
PrevU=LastU
LastU=U
RETURN(FTOI(U))                     ; Return command value as integer
```

```
CLOSE
```

例 3：PMAC 的 PID 滤波器

下面的例子模仿 Turbo PMAC 的基本 PID 回路（无陷波、死区补偿、位置误差限制或摩擦前馈项）的作用，但有浮点计算。它采用 PID 自己的 I-变量，访问为 L-变量用于速度，然后转换为浮点值。

这个程序是用户编写的伺服，复制了 PMAC 的 PID 回路。它采用了 PMAC 的用户手册中定义的下面的公式，但修改为 PMAC 通过的术语：

```
K16 = 2**-16
K128 = 1/128
K23 = 2**-23
FE = DPOS - APOS - From PMAC in Ix08 * 32 counts
AVEL, DVEL - From PMAC in Ix09 * 32 counts/servo period
IPOS = ITOF(Ix33) * FE * K23 + IPOS
DACEL = DVEL - PDVEL
PDVEL = DVEL
DACOUT = ITOF(Ix30)*K16 * ( FE + K128 *(ITOF(Ix32) * DVEL +
ITOF(Ix35) * DACEL - ITOF(Ix31) *AVEL) + IPOS)
#define   K16   0.000015259     ; Constant of 2^-16
#define   K128   0.0078125      ; Constant of 1/128
#define   K23   0.000000119     ; Constant of 2^-23
使用 L 型变量以便快速访问 PID I-变量寄存器。
#define Ix08 L1
L1->Y:(R1-$27)
#define Ix09 L2
L2->Y:(R1-$14)
#define Ix30 L3
L3->Y:(R1-$17)
#define Ix31 L4
L4->X:(R1-$1E)
#define Ix32 L5
L5->X:(R1-$21)
#define Ix33 L6
L6->X:(R1-$11)
#define Ix35 L7
L7->Y:(R1-$1D)
#define Ix63 L8
L8->Y:(R1-$11)
#define Ix69 L9          ; 注意这是 24 位的值，不是 16 位
L9->Y:(R1-2)
#define STATUS L10
L10->X:(R1+0)
#define IPOS F1          ; 积分位置误差寄存器
F1->L:(R1-$12)           ; 电机打开环路时自动清零
#define PDVEL F2         ; 前一个预期速度寄存器
```

```
F2->L:(R1-$29)          ; 电机打开环路时自动清零
#define AVEL P0         ; 浮点实际速度
#define DVEL P1         ; 浮点净额预期速度
#define FE P2           ; 浮点跟随误差
#define APOS P3         ; 浮点实际位置
#define DPOS P4         ; 浮点净额预期位置
#define DACOUT P5       ; 浮点指令输出
OPEN SERVO CLEAR
COPYREG P0              ; 复制电机 AVEL,DVEL,FE,APOS,DPOS 到 float P0..4
; FE, APOS, & DPOS in 1 / [Ix08* 32] counts
; AVEL in 1 / [Ix09*32] counts/servo period
; DVEL in 1 / [Ix08*32] counts/servo period;
; if( Ix34 = 1 && Des_Vel0 = 1 OR Ix34 = 0)
; then integrate IPOS = IPOS + FE * Ix33 && Limit to Ix63
If (STATUS&$12000 = $12000 Or STATUS&$10000 = 0)    ; Test Ix34 mode
IPOS = FLIMIT(ITOF(Ix33)*FE*K23+IPOS, ITOF(Ix63)*ITOF(Ix08)*2)
; Scale Ix63 to include Ix08
EndIf
DACOUT = FLIMIT(ITOF(Ix30)*K16 * ( FE + K128 *(ITOF(Ix32) * DVEL +
ITOF(Ix35)* (DVEL - PDVEL) - ITOF(Ix31) *AVEL) + IPOS), ITOF(Ix69))
        PDVEL =DVEL         ; 存储用于下一个周期
RETURN(FTOI(DACOUT))
CLOSE
```

下载程序

每次执行程序下载器将开放式伺服算法的编译后的代码发送到 Turbo PMAC 时，它首先彻底清除 Turbo PMAC 中现存的用户编写的伺服代码。因此，要始终重新编译整个开放式伺服，即使是最轻微的变化。在这些实例的开头使用的 **CLEAR** 命令因此是不需要的，但它可以用于与解释程序的样式一致性。

请记住，下载过程只将开放式伺服算法放在易失性活性 RAM 内存中。为了使加载到 Turbo PMAC 的算法通过复位或上电循环，首先使用 SAVE 命令将其复制到非易失性闪存中。

执行开放式伺服程序

对于给定的电机执行开放式伺服算法，电机的 Ixx00 必须等于 1，这样电机的计算被激活。Ixx59 的 0 位必须设置为 1（Ixx59= 1 或 3），这样它就会选择开放式伺服产生的用户写伺服算法而不是内置的伺服算法（PID 或 ESA）。最后，对于该电机，必须关闭伺服回路。如果电动机在开环启用状态，或者停止（开环禁用）状态，开放式伺服算法将无法执行。

开放式伺服算法遵守每个电机 xx 的 Ixx60 伺服周期延长参数。正如内置 PID 和 ESA 的伺服算法，它在每[Ixx60+1]伺服中断时执行。在 Ixx60 默认值为 0 时，它在每个伺服中断时执行。

那些采用开放式伺服算法用于实际闭合伺服回路之外的任务的用户一定要小心，不要无意中禁用该算法。该算法在上电或复位时不会立即执行，除非电机的 Ixx80 的 0 位被设置为 1。一个**<CONTROL-K>**（停止所有）命令禁用所有电机的伺服回路，包括一个运行开放式伺服算法的伪电机。Ixx24 的 21 位和 22 位在

默认设置下，任何电机的放大器故障或故障跟随误差故障将禁用所有电机的伺服回路，包括一个运行开放式伺服算法的伪电机。

开放式伺服对于所选的电机仅替换实际的伺服环闭合算法。作为伺服中断的一部分来完成的其他任务，包括编码器转换表处理、指令轨迹生成、位置跟踪、时基控制，都通过内置的固件运行，不管是否对于任何电机选择开放式伺服。此外，在伺服中断之外完成的相关任务，如核对故障跟随误差限制，是通过内置的固件执行的。

汇编的用户编写的伺服算法

用户编写的高效的伺服算法可以使用 Turbo PMAC 的处理器 DSP56300 系列的汇编语言编写。这需要使用摩托罗拉的一个交叉汇编器，可从他们的网站上免费获得。还需要从泰道公司获取一个连接程序，称为 **CODET.EXE** 并运行在 Microsoft Windows 操作系统下，可从泰道的网站免费获取。

编写算法

该算法使用任何标准的文本编辑器、使用摩托罗拉 DSP56300 汇编语言编写。代码的编写受以下限制。

程序存储器空间

该程序必须从内存位置 P:$040000 开始，所以代码的第一行必须是：

```
ORG P:$40000
```

对于一个 DSP56303 处理器（80 MHz 的 CPU Opt 5CX），生成的代码必须结束在内存位置 P:$040BFF，为程序提供一个 3-K 字的缓冲区。对于一个 DSP56309 处理器（100 MHz 的 CPU Opt 5DX）或 DSP56311 处理器（160 MHz 的 CPU Opt 5EX），生成的代码必须结束在内存位置 P:$044BFF，为程序提供 19-K 字的缓冲区。然而，在所有情况下，如果使用一个用户编写的相位程序时，该程序开始于存储器位置 P:$040800，将用户编写的伺服程序限制为 2-K 字的缓冲区。

准入条件

关于进入用户编写的伺服，该程序可以为执行电机在内部 DSP 寄存器预期以下数据：

- A10 寄存器包含一个 48 位整数，表示预期的速度，单位是每个伺服周期 1/（Ixx08*32）个计数
- B10 寄存器包含一个 48 位整数，表示预期的位置，单位是 1/(Ixx08*32）个计数
- Y 寄存器包含一个 48 位整数，表示实际位置，单位是 1/（Ixx08*32）个计数
- R0 寄存器包含执行电机的命令输出和指令偏置寄存器的地址（例如 $0000BF 用于电机 1）。这必须不能改变。
- R1 寄存器包含了执行电机的第一个状态字的地址（如：$0000B0 用于电机 1）。这不能改变。
- N1 寄存器包含电机伺服寄存器的模块长度（目前是$80），这可能对从电机到电机的增量有用。这不能改变。

退出条件

从用户编写的伺服退出，Turbo PMAC 固件期望找到控制作用值，在 A 寄存器中作为一个 24 位的有符号整数。如果 Turbo PMAC 对该电机启用换相（Ixx01

的 0 位 = 1），此值被用作“正交电流” （扭矩）输入到换相算法。如果 Turbo PMAC 对这个电机禁用换相（Ixx01 的 0 位=0），这个值被 Turbo PMAC 固件复制到由 Ixx02 指定的寄存器中。在这种情况下，不是所有的寄存器都可以使用。PMAC 式伺服 IC 的 DAC 寄存器只使用前高 16 位。PMAC2 式伺服 IC 的 DAC 寄存器只使用前高 18 位。

用户编写的伺服的最后一行必须是：

JMP <$001

可用的寄存器

用户编写的伺服可以使用下面的数据寄存器：

- 内部 DSP 寄存器 R4、N4、R5 和 N5 都可以使用，并且完成时不需要恢复。
- 内部 DSP 寄存器 M0、M4 和 M5 都可以使用，但完成时必须恢复到原来的值。
- 电机中间值寄存器 X：$000x93/13 到 X：$000x9A/1A 可用于保存周期之间的值。只要用户编写的伺服被激活，它们不被任何 Turbo PMAC 固件使用。
- 全局寄存器 X/Y:$0010F0 - $0010FF 也可使用。它们不被任何 Turbo PMAC 固件任务使用，除了上电或复位时被设置为 0。
- **DEFINE UBUF** 命令创建的用户缓冲区的寄存器可以使用。它们不被任何 Turbo PMAC 固件任务使用。
- 也可以使用其它的寄存器，但有可能 Turbo PMAC 固件的某些任务会重写这些。例如，对用户编写的伺服可以使用寄存器用于某些 P 或 Q-变量，但给这些变量之一赋值将重写寄存器。也可使用 Turbo PMAC 标准伺服算法的 I-变量作为用户编写的伺服的增益。

编程限制

不要使用任何级别的 DSP 栈，所以不允许 DO 或 JSR 指令。

不要使用内部 DSP 地址寄存器 R2、R3、R6、R7；修饰符寄存器 M2、M3、M6、M7；偏移寄存器 N2、N3、N6、N7。

汇编算法

汇编语言算法必须使用计算平台的摩托罗拉的交叉汇编器汇编成 DSP56300 机器代码。按照摩托罗拉的指示来做到这一点。

链接算法

使用泰道公司的小程序 **CODET.EXE**，可在泰道网站获得，将摩托罗拉汇编器生成的文件转换成一种可以直接下载到 Turbo PMAC 的格式。该文件应归档在计算机或网络上。

下载算法

使用任何版本的 PMAC 执行程序将这个生成的文件下载到 Turbo PMAC 的程序存储器中。请记住，它被下载到非易失性 RAM 内存中。要使 Turbo PMAC 保留这个算法，在控制器复位或移除电源之前，发出一个 **SAVE** 命令。

执行算法

将电机 xx 变量 Ixx59 的 0 位设置为 1（Ixx59=1 或 3）用来选择用户编写的伺服算法。正如 PID 或 ESA，电机的伺服回路必须是闭合的以便执行该算法。如果电机是启用开环或停止，它也不会执行。如果使用用户编写的伺服算法用于非伺服任务，要知道某些命令（如**<CTRL-K>**）或其他电动机的故障情况（如果 Ixx24 因故障指定停止其他电机，故障跟随误差或放大器故障，）可以禁用你的算法。

第十章 电机补偿表和常数

Turbo PMAC 能够对其电机的位置和力矩执行复杂的基于表格的修正。这些允许补偿系统中那些在实际应用中使用不能传感器测量的缺陷（尽管必须使用可以测量缺陷的参考传感器表征误差）。

注意

死区补偿，由电机参数 Ixx64 和 Ixx65 控制，是伺服反馈算法的一部分。它涵盖在本手册的《设置伺服回路》一章。

注意

刀具半径补偿是一个坐标系统的功能，而不是一个电机的功能。它是涵盖在本手册的《编写和执行运动程序》一章。

10-1 位置补偿表

Turbo PMAC 能够执行基于表格的位置校正，俗称丝杠补偿。这种技术，也有其他的名字，允许一个修正表输入到 Turbo PMAC 作为电机位置的一个函数。Turbo PMAC 可以存储多达 32 个这样的补偿表。

这些表最常用于补偿在位置传感器（通常在电机的背面）和要控制位置的负载之间的机械误差缺陷。在许多系统中，电机旋转时线性地移动负载的丝杠是定位误差的最大来源，因此传统上这些表被称为丝杠补偿表。然而，这些表也可以用于补偿传感器本身的误差缺陷。

源、目标和分配的电机

每个电机都有一个属于它的表；也就是说，该 **DEFINE COMP** 命令创建一个表将其分配到目前的定址的电机，而且每个电机只能有一个表分配给它。除非另有指定，表中采用该电机的位置信息（源数据）以确定在表中的位置，并且也将其校正加到该电动机上（目标数据）。然而，源电机，或源电机和目标电机二者，都可以被指定为该表所属电机以外的电机。（如果两个电机是不同的，属于一个电机的表的概念只对 Turbo PMAC 自身内务管理有用。）

标准丝杠补偿

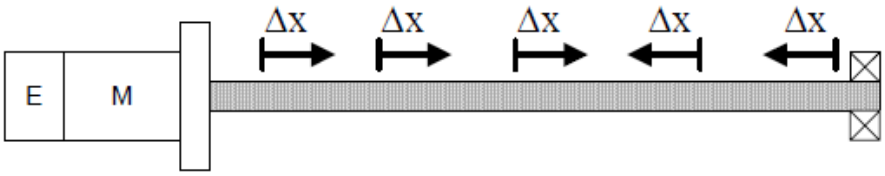
单个源电机的位置补偿表，以及目标电机与源电机同一个电机，形成了一个标准的丝杠补偿表。这是表格最常用的类型，因为在行程方向上的误差作为该行程的函数往往是最大的误差。

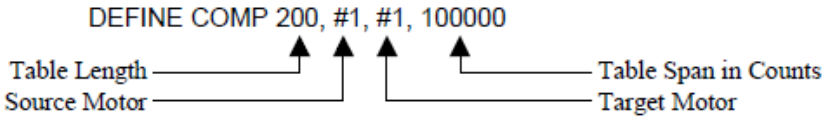
PMAC 补偿表

标准丝杠补偿

例如 Δx= f(x)

- 通过旋转编码器得到线性编码器的精度（几乎!）
- 通过线性传感器表征系统
- PMAC 的输入误差





使用交叉轴补偿

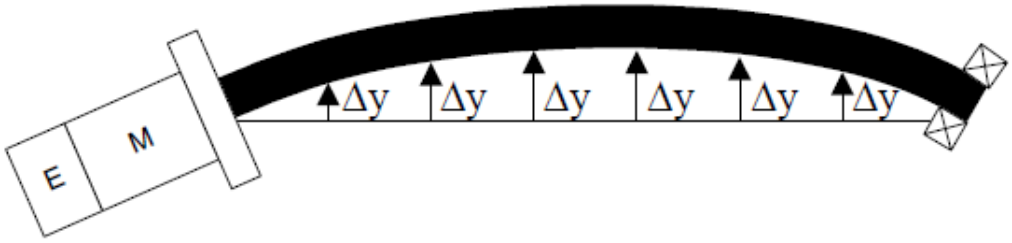
一个表能够有独立的源和目标电动机，具有多种用途。第一种是传统的补偿不完全的几何形状，如在一个弓形丝杠。举例来说，在一个 XY 工作台上，如果 X 轴丝杠是弓形，Y 轴的位置应该得到一个作为 X 轴位置的函数的校正。如果电机＃1 是 X 轴，电机＃2 是 Y 轴，保存这种校正的表将电机＃1 作为源电机，电机＃2 作为目标电机。

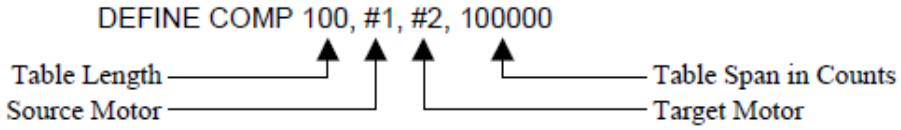
PMAC 补偿表

交叉轴补偿

例如 Δy =f(x)

- 对弓形丝杠有用
- 可用于构建电子凸轮表





交叉轴补偿的第二个用途是通常被称为电子凸轮。在这种情况下，目标电动机的整个运动是由补偿表中的入口引起的，而不仅仅是修正引起的。这种实施电子凸轮操作的方法比 Turbo PMAC 的时基跟随有两种显著优势，创建电子凸轮的另一种方法：补偿表是双向的---主电机可以转向任一方向---而且它是绝对的，所以定相只是简单将轴回零。

时基方法，其中从属电机的运动程序定义运动，保留的优势是，可以通过程

序中的数学和逻辑动态更改，在点之间能够二阶或三阶插值，而不是补偿表的一阶插值。请参阅本手册的《同步 Turbo PMAC 到外部事件》一章，获取细节。

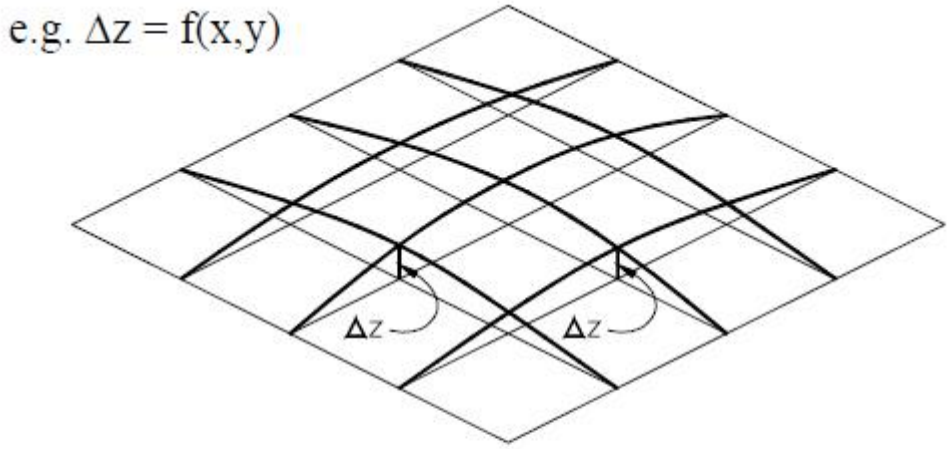
表的维度

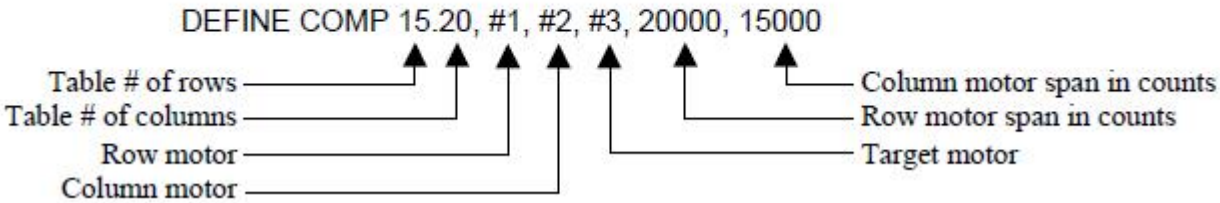
Turbo PMAC 目前支持一维（1D）和二维（2D）的位置补偿表。将来还有可能支持三维（3D）的表。1D 表有一个源电机；二维表中有两个源电机。需要注意的是，单个源电机与目标电动机不同的表（交叉轴的补偿表）仍然是一个一维表。

PMAC 补偿表

2D（平面）补偿表

例如 Δ z= f(x, y)





注意

三维补偿可在 Turbo PMAC 上通过使用运动学子程序实现，可以用来通过算法计算修正。校正方程要适合于所观察的测量结果，可能是通过多项式方程的最小二乘法拟合的。请参阅本手册《设立一个坐标系统》一章中的运动学算法部分获取详细信息。注意，无论是校正公式（逆向运动学）还是其反向（正向运动学），都必须被输入。

如果需要算法补偿而不是基于表的补偿，运动学方程也可用于一维和二维补偿。坐标系中的所有电机的修正必须在同一运动学算法中完成。通过对算法的系数参数化，该修正可动态调节（例如，作为温度的函数）。

使用期望的与实际的位置

位置补偿表可以使用源电机的预期位置或实际位置来计算其校正。在大多数应用中使用哪一个都没关系，但是如果源电机和目标电机是同一个，电机的增益非常高，并且有显著的修正，利用实际位置会影响伺服回路的性能，有效地改变回路增益作为位置的函数。在这些情况下建议使用预期位置。请参阅下面关于如何指定使用预期位置的说明。

请注意，在这两种情况下，该表是原始（未校正）电动机位置的函数。由于表的整个目的是允许命令移动到校正后的位置，如果目标电机与源电机是同一电机，在一个确定的指令数字位置值，修正一般不会与同一数值下的原始位置相同。

每个电机多个表

一个电动机可为任何位置补偿表提供源数据；它也可以是任何位置补偿表的目标电机，各个表的修正是到目标电动机的添加剂。例如，一个电动机可以有一个重复的精细补偿表（用于周期性的误差，如传感器偏心）和一个非重复的粗调表。另外，校正可以应用到一个电机，同时作为它自身位置和另一电机位置的函数。

表的范围

这个补偿直接定义为一系列源电机位置，在零计数开始（最近的零点或上电/复位位置），在正方向上移动。这个范围的大小被声明为 **DEFINE COMP** 命令的最后一个参数。这个参数的单位是源电机的计数。入口之间的间距是总范围除以入口的数量（这是 **DEFINE COMP** 指令的第一个参数）。表的第一个入口定义相对源电机零位置的 1 个间距的修正，第二个入口是两个间隔，依此类推。

翻转

超过这个范围之外，未校正的位置翻转到此范围内---本质上是一个取模（余数）运算---在补偿完成之前。对于二维表，翻转发生在两个方向。翻转允许补偿旋转轴若干转，以及简单补偿编码器偏心。当然，如果该表大到足以覆盖整个源电机的行程时，翻转功能将永远不使用。

如果电机有一个行程范围到零的负端，并且此处需要补偿，应该产生这些入口如同是他们超过了电机范围的正端。例如，如果电动机行程是+/-50000 个计数，而且是每 500 个计数产生一个表格入口（因此总数 200 个入口），该表的设置是通过一个 **DEFINE COMP200,100000** 命令。前 100 个入口将涵盖 500 至+50000 的计数范围，最后 100 个入口将覆盖-50,0000 到 0 的计数范围。（通常情况下，表被引用，因此在源电机的零点位置有一个零校正，所以表中的最后一个入口应为 0。）从本质上讲，-50,000 至 0 的范围将被映射为+50000 到+100000 的范围。

当表被下载到 Turbo PMAC 中，如果全局变量 I30（在 V1.939 固件中是新的）设置为默认值 0，在源电机 0 计数处的修正值始终为 0。在这种情况下，表中的最后一个入口必须被设置为 0，否则在校正中会有一个不连续（因此有一个位置跳跃）源电机会超过表的任一端，翻转校正。大多数情况下，在电机的零位置处的校正被定义为 0。

当该表被下载到 Turbo PMAC，如果 I30 被设置为 1，源电机 0 计数处的修正值被设定为等于表的最后一个入口，以保证表中的平滑翻转。如果该表中的最后一项是 0，则结果将是相同的，无论 I30 的设置是什么。

确定补偿值

确定输入到补偿表中的值，是通过将应用程序所使用的传感器的原始测量值与安装用于校准处理的参考传感器进行比较完成的。将轴（或多个轴）移动到您想要做一个入口的原始位置。在此位置读取参考传感器。表中的入口将是原始位置减去参考（基准）位置，这两个值都缩放到 Turbo PMAC 软件计数的 1/16，两个值都是从电机的原始（零点）位置测量的。

对于标准丝杠补偿表（一维表，一个电机同时作为源电机和目标电机），泰道公司提供了一个 PC 软件包叫做 Flycal，可以动态进行这些测量，并自动快速生成这些表。

输入表格

位置补偿表是通过在线命令输入到 Turbo PMAC 中。首先是 **DEFINE COMP** 命令，它定义了表的大小和间距，以及使用哪些电机作为其源电机和目标电机。在此之后，是一系列的数值常量，用空格和/或回车符隔开，它们依次输入到表格中。 （如果没有填充的表，发送到 PMAC 的数值常量赋值给变量 P0）。

位置补偿表必须按顺序定义，从那些分配到编号较高的电机，到那些分配到编号较小的电机。

输入一维表

如果要输入的位置补偿表的源电机和目标电动机都相当于定址的（分配的）电机，并且该表使用该电机的实际位置来计算校正，那么就没有必要在 **DEFINE COMP** 命令中指定电机。该命令看起来是这样：

#2 DEFINE COMP 25, 500000

这个命令建立一个表，使用电机 2 同时作为源电机和目标电机，使用实际位置来计算校正。发送到 Turbo PMAC 的下一批 25 个数值常量将被输入到表格中。第一个值将是在 500,000/25=20,000 计数处的修正量（这里的 20，000 表示未校正的位置）。第二个值将是在 40000 未校正的计数处的修正量，等等。第 25 个入口是 500,000 计数处的修正量；如果在输入的时候 I30 被设置为 1，这也将是 0 计数处的校正量。校正本身的单位是 1/16 个计数。

如果您希望，无论是源电机还是目标电机都与定址的电机不同，或者你想使表作为源电机预期位置的函数，你必须在 **DEFINE COMP** 命令中显式声明电机。在这种情况下，命令看起来是这样的：

＃1 DEFINE COMP25，＃2D，＃2，500000

这个命令建立一个表，同时使用电机 2 作为源电机（从它的净额预期位置工作）和目标电机（使用修正值），即使表被分配到电机 1。如上述情况下，下一批的 25 个数值常量将被输入到该表中，入口的第一个值是 20,000 计数处的校正，第二个值是 40,000 个计数处的校正，等等。

一维表示例

下面是一个一维表入口的简单的例子

```
#1
DEFINE COMP 8,4000          ; 8 个入口的表，有 4000 个计数
                            ; 电机 1；使用电机 1 作为源电机(实际位置)
                            ; 电机 1 也是源电机因为没有指定别的电机
-160                        ;4000/8 (500) 计数处的修正是-160/16 = -10 个计数
80                          ; 1000 计数处的修正是 5 个计数
120                         ; 1500 计数处的修正是 7.5 个计数
96                          ; 2000 计数处的修正是 6 个计数
20                          ; 2500 计数处的修正是 1.25 个计数
-56                         ; 3000 计数处的修正是-4.5 个计数
-12                         ; 3500 计数处的修正是-0.75 个计数
0                           ; 4000 (和 0) 计数处的修正是 0 个计数
```

在这个例子中，在 1300 计数的原始位置处的校正将是在 1000 计数和 1500 计数处的校正之间的线性插值，如下所示：

$$\text{校正值} = 5 + (7.5 - 5) * \frac{1300 - 1000}{500} = +6.5 \text{ 个计数}$$

输入 2 维表

如果位置补偿表中有两个源电机，建立一个 2D 表，或平面表，该入口稍微复杂一些。在两个维度上声明表的长度（以入口的数目）以及该表的跨距。声明两个源电机，和通常的目标电机（虽然默认的是定址的电机）。建立表的命令看起来像这样：

```
#2  DEFINE  COMP  15.20, #1D, #2D, #3, 20000, 15000
```

此命令指定分配给电机 2 的表将有 15 行和 20 列。因此，每行有 20 个入口，每列有 15 个入口。

电机 1 是第一个源电机（行电机），使用它的预期位置；每行表示电机 1 的位置的计数值的一个行跨距，每一列表示电机 1 的一个恒定原始位置。

电动机 2 是第二个源电机（列电机），使用它的期望位置；每一列表示电机 2 的计数的一个列跨距，并且每行表示电机 2 的一个恒定原始位置。

电机 3 是目标电动机；校正被应用到电机 3。

每行的跨距是 20,000 计数，所以沿行维度的方向上，入口之间的间隔是 20,000/20 =1000 计数（电机 1 的）。每列的跨度为 15,000 计数，所以沿列维度方向上，入口之间的间距是 15,000/15=1000 计数（电机 2 的）。

发送到 Turbo PMAC 的下一批（15+1）*（20+1）-1=335 个数值常量输入到这个表中。为什么入口是这个数字？因为在每行和每列的零点位置和最大位置都有入口（因此是 n+1 项），但是在两个源电机的原点没有明确的入口（因此最终值减去 1）。

在这个例子中，第 1 个入口是在电机 1（原始）位置 1000、电机 2（原始）位置 0 处的校正值- 即在（1000，0）处。第 2 个入口是在（2000，0）处。第 20 个入口是在（20,000，0）处。 第 21 个入口将在（0，1000）处，第 22 个入口在(1000,1000)处,等等。该表的第 335 个入口和最后一个入口将是在(20,000，15,000）处的校正值。如果 I30 被设置为 1，该值也是在（0，0）处的校正值。通常情况下，原点处的校正值是定义为零，作为对其他校正值的参考点。

如果运动有任何可能会超过了表格所声明的跨距，不管是否用于目的性翻转，在每一行两端的入口应该是相同的，对于每个列也是同样的。否则，在表格的边缘的校正值会出现不连续性。

二维表中的入口要注意三件事情。首先，行数和列数是用一个句号而不是用逗号分开的。其次，行数（在上述例子中的 15）是第一个输入的，其次是列数，但各行之间的间隔（计数）是由列的跨距（上面例子中的 15,000）来确定的，输入在行的跨距后。最后，为允许在 Turbo PMAC 进行高效的计算，0 行和 0 列以及 n 行和 n 列都必须输入。

二维表示例

下面的例子显示了一个简单的二维表的入口，该所示的形式可以方便地由用户读取。（Turbo PMAC 不需要输入表格时，每行都在单独一行上，但为了可读性推荐使用这个）。注意第 0 个入口有一个隐含的校正值 0，0 行和 4 行是相同的，正如 0 列和 5 列相同。

```
#3 DEFINE COMP 4.5, #1D, #2D, #3, 50000, 40000
38 45 –22 –35 0              ; Row 0, Columns 1-5
24 56 13 –34 –8 24           ; Row 1, Columns 0-5
18 43 –9 –65 32 18           ; Row 2, Columns 0-5
-6 28 22 –38 12 –6           ; Row 3, Columns 0-5
0 38 45 –22 –35 0            ; Row 4, Columns 0-5
```

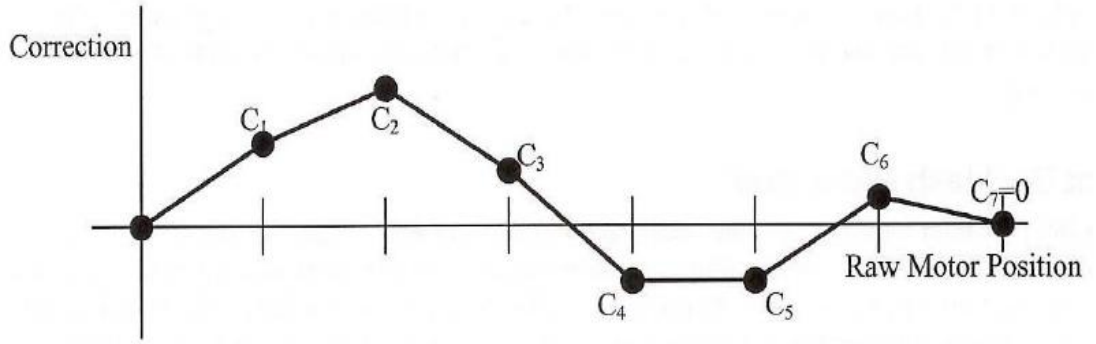
在这个例子中每一行包含电机 1 的跨距（0 – 50,000 计数）在电机 2 的一个恒定位置；每列包括电机 2 的跨距（0 – 40,000 计数）在电机 1 的恒定位置。

启用和禁用表

当 I51 被设置为 1，所有位置补偿表（以及间隙和扭矩补偿表）被启用。I51 设置为 0 时，所有这些表禁用。

修正的有效计算

位置补偿在伺服环路（每个伺服周期）内进行，以获得最大的速度和精度。Turbo PMAC 拾取源电机的位置，找到表中的匹配位置。通常，这是在一维表中的两个入口之间，或者在二维表的 4 个入口之间，所以 Turbo PMAC 这些项之间进行线性插值（加权平均）为当前伺服周期取得校正值。然后，将这个校正值加到目标电动机的预期位置上。表中的校正入口必须是整数值，单位是目标电动机的 1/16 个计数（因此，比如，一个 48 的入口表示 3 个计数）。



一个目标电动机的净额校正值在每个伺服周期都存储在该电机的一个特定的寄存器中。对于电机 1，该寄存器是在 D：$000090。对于电机 xx 的这个寄存器的建议 M 变量是 Mxx69。该寄存器的单位是 1 /（Ixx08* 32）个计数，与其它电机位置寄存器相同。如果 I51 被设置为 1，则该寄存器在每一个伺服周期都被清零，然后将该电机用作其目标电机的每个表的结果是被添加到该寄存器中。如果 I51 设置为 0，则允许直接将值写入该寄存器。不管 I51 的设置，此寄存器中的值在每个伺服周期都被添加到电机的净额预期位置值上。

很重要的一点是要明白，该表的修正值是作为未校正的电机位置的函数来计算的，无论是使用期望的或实际的位置。对于一个使用同一电机作为源电机和目标电机的表，校正后的位置是通过定义不同于未经校正的位置。由于补偿随位置而变化，在给定的校正的位置处的补偿，通常与同一数值的未校正的位置处的补偿不完全一样。二者之间的差值很小，但如果你查看净额补偿寄存器的话，会很明显。如果您要验证一个表所产生的的校正值，理解这一点是很重要的。

报告表格信息

一个位置补偿表的报头信息---通过 DEFINE COMP 命令输入---可以通过 **LIST COMP DEF** 命令查询。表的内容可以通过 **LIST COMP** 指令查询

删除表格

DELETE COMP 命令删除分配给当前定址电机的位置补偿表(无论这个电机是表的源电机还是目标电机)。位置补偿表必须依次删除，从那些分配给低编号电机的，到那些分配给高编号电机的。

10-2 间隙补偿

Turbo PMAC 可以为所有的电机执行复杂的间隙补偿。如果位置反馈利用电机上的传感器而且在耦合到负载时有一个物理间隙（如在一个典型的轮系），对于一个给定的传感器-报告的位置，负载的物理位置将根据运动的方向而不同。除非这被补偿,否则在应用程序中会产生明显的位置误差。

通过 Turbo PMAC 的间隙补偿，当逆转指令速度的方向时，一个预编程的间隙距离就加到指令位置上或从指令位置中减去。此间隙距离在电机的行程中可以是恒定的，或它可以是电动机位置的一个函数。该间隙被引入或去除的速率是可

编程的，如同是引入或删除的间隙所需的反向的幅度。间隙量可被任何位置报告隐藏。

常数间隙参数

电机 xx 的变量 Ixx86 是常数间隙距离参数。当电动机的指令移动的方向从正变化到负时，这个值被引入到激活的间隙补偿寄存器，从名义指令位置中减去。当电动机的指令移动的方向从负变化到正时，间隙补偿寄存器的值被减为零。

需要注意的是，Ixx86 的正值增加了额外距离到电机反转的行程，从而补偿真实的物理间隙。（唯一已知的实际使用负间隙参数是在当电机作为电子齿轮从属于一个轴时，有较大的物理间隙）。Ixx86 的单位是一个计数的 1/16，因此该值应为间隙补偿的计数值的 16 倍（如 Ixx86=72 指定 4.5 个计数的间隙）。

间隙参与率

变量 Ixx85 控制着电机 xx 反转时，间隙被引入或去除的速率。这允许用户优化出迅速但光滑的间隙补偿。当检测到反转时，在每个后台循环（每个 PLC 的每次扫描之间），一个等于 Ixx85 的值加到激活的间隙补偿寄存器中或从中减去，直到寄存器值等于 Ixx86 或 0。在一般情况下，应该使用能产生平稳转换的 Ixx85 的最高值。

间隙滞后

变量 Ixx87 为电机 xx 控制在净额指令位置的新方向上的计数值，其必须在 PMAC 确定发生反转和改变间隙之前看到。因此，它作为一个“滞后”项。如果使用一个主编码器驱动电机，这就特别重要，所以主编码器轻微的抖动不会造成间隙的重复引入和去除。Ixx87 的单位是 1/16 个计数，所以 64 的默认值提供了一个 4 个计数的滞后。

间隙补偿表

通过 **DEFINE BLCOMP** 命令创建的间隙补偿表，可用于创建一个随定址电机位置而变化的间隙距离。通常这是与丝杠补偿表联合使用，以产生一个双向的丝杠补偿表的效果。在这种情况下，间隙表（被加到常数间隙参数）包含了正向补偿和负向补偿之间的差值。泰道公司的 Flycal 校准程序用于 PC 可以自动生成这些表。

输入表格

间隙补偿表的输入和操作很像位置丝杠补偿表。然而，不存在交叉轴或多轴间隙补偿表。属于一个电机的表格为该电机提供了间隙校正，作为该电机的位置的函数。间隙补偿表必须按顺序来定义，从那些属于较高编号电机的到那些属于较低编号电机的。

一个电机的间隙补偿表是通过在线命令 **DEFINE BLCOM P{入口}，{计数长度}** 为定址电机声明的。{入口}定义在表中点的数量， {计数长度}以电机的计数定

义表的跨度。因此，表中的入口之间的间距为{计数长度}/{入口}。

表中的第一个入口定义了距离电机零位置一个间隔处的间隙，第二个入口是在两个间隔处的，依此类推。通常情况下，表中的最后一个入口是 0。

计算间隙

对于一个给定的电机位置，从间隙表获得的间隙距离的值将被添加到 Ixx86 “常数”间隙参数上。从表中在电机 0 位置（回零位置）的间隙距离被定义为零，因此，如果使用一个间隙表，Ixx86 应包含在回零位置的间隙数值。然后该表应该保持有相对此数的差值。对于一个给定电机位置，间隙表的贡献是通过在相邻入口之间进行线性插值来计算的。

只有当运动的最近的指令方向是负时，电动机的齿隙表才是激活的；如果电机在负方向行进到达这个位置并被指令静止时，它仍然是激活的。在操作中，该表读取当前的名义电机位置，并计算两个最接近的表格入口的加权平均值，在表格的点之间创建一阶插值。

间隙补偿直接定义用于电机位置的范围，范围开始于零计数，沿位置方向行进到 **DEFINE BLCOMP** 命令最后一个参数声明的计数长度处。入口之间的间距是这个长度除以入口的数量（入口数量是指令的第一个参数）。表中的第一个入口定义距离电机零位一个间距处的校正，第二个入口定义两个间隔处的校正，依此类推。

超过此范围外，在补偿完成之前，未校正的位置被“翻转”在此范围内。该翻转的发生与丝杠补偿表的完全一样；参考那个描述获得详细信息。

报告表格信息

间隙补偿表的报头信息---用 **DEFINE BLCOMP** 命令输入---可以用 **LIST BLCOMP DEF** 命令查询。该表的内容可以通过 **LIST BLCOMP** 命令查询

删除表格

DELETE BLCOMP 命令擦除当前定址电机的间隙表。间隙补偿表必须按顺序删除，从那些分配给编号较低电机的到那些分配到较高编号电机的。

启用和禁用间隙

恒定间隙参数 Ixx86 总是（潜在的）激活的；也就是说，任何时候只要 Ixx86 是一个非零值，就会有间隙补偿。如果 I51 被设置为 1，间隙表是有效的；如果 I51 被设置为 0，它们是无效的。

间隙表示例

试想一下分配给电动机 3 的一个轴，其校准已执行，通过负载上的一个精确线性测量装置，它在两个方向上工作，而且线性参考装置为电机编码器的设定位置读取的读数在下面（表示为电机编码器的单位）：

电机位置（计数）	0	500	1000	1500	2000	2500	3000	3500
负载位置+（计数）	0*	510	995	1492.5	1994	2497.5	3003.5	3500.5
负载位置-（计数）	5	516	998.5	1494	2000	2501	3010.5	3508.5
*参考点；定义为零点								

只有补偿表工作在正方向，所以补偿表中的入口应该是正向负载位置和电机位置二者之间的差值的负值，单位表示为 1/16 个计数：

电机位置（计数）	0	500	1000	1500	2000	2500	3000	3500
负载-电机（计数）	0*	+10	-5	-7.5	-6	-2.5	+3.5	+0.5
电机-负载（1/16 计数）	0*	-160	+80	+120	+96	+40	-56	-8
*参考点；定义的零点								

创建这些校正的位置（丝杠）补偿表定义是：

#3 DEFINE COMP 8, 4000

-160 80 120 96 40 -56 -8 0

注意，第一项是用于在 500 个计数处的校正，并且所添加的最后一个入口为 0，是在 4000 计数及 0 计数处的校正。

在电动机 0 位置处有 5 个计数的间隙，所以 Ixx86 应设置为 5*16，或 80。

接下来，间隙补偿表应包含负向负载位置和正向负载位置之间的差值，减去 Ixx86：

电机位置（cts）	0	500	1000	1500	2000	2500	3000	3500
Load（-）-Load（+）（cts）	5	6	3.5	1.5	6	3.5	7	8
Load（-）-Load（+）-Ixx86（cts）	0*	1	-1.5	-3.5	1	- 1.5	2	3
Load（-）-Load（+）-Ixx86（1/16cts）	0*	16	-24	-56	16	-24	32	48
*参考点；定义的零点								

间隙表定义来创建这些校正将是：

#3 DEFINE BLCOMP 8, 4000

16 -24 -56 16 -24 32 48 0

注意，第一项是用于在 500 个计数处的校正，并且所添加的最后一个入口为 0，是用于在 4000 计数及 0 计数处的校正。

注意

虽然同一个电机的间隙表的范围和间距通常与其丝杠补偿表是相同的，但是这不是必需的。甚至一个电机不需要有丝杠补偿表以使该电动机有一个间隙表。

10-3 转矩补偿表

Turbo PMAC 能够为伺服环的输出创建一个校正表，作为电机位置的一个函数。通常情况下，该功能将在转矩模式与伺服环使用（不论 Turbo PMAC 是否也进行电机换相），所以这个函数被称为转矩补偿表。

转矩补偿表的输入和操作很像提供了位置校正的丝杠补偿表。然而，没有交叉轴或多轴转矩补偿表。属于一个电机的表为该电机提供了一个扭矩校正，作为该电机的位置的函数。

如果电机的伺服环正在输出一个速度指令，转矩补偿表的校正实际上是速度校正。

输入表格

一个电机的转矩补偿表通过在线命令 **DEFINE TCOMP{入口}，{计数长度}**为定址电机声明的。{**入口**}定义表中的点的数量，{**计数长度**}定义了表的跨度，单位是电机的计数。因此，表中入口之间的间距为{**计数长度**}/{**入口**}。

表中的第一个入口定义距离电机零位一个间距处的校正，第二个入口是在两个间隔处的校正，依此类推。如果 I30 设置为 1 来启用自动的表格翻转（强烈推荐），表的最后一项提供了在**{计数长度}**和在零位置处的校正。否则，如果需要在翻转点处是一个连续校正，那么表中的最后一个入口必须为 0。

该校正是直接定义在电机位置的范围 **0～{计数长度}**。对于此范围之外的电机位置，在校正应用之前，该位置是“翻转”到此范围内。以这种方式，周期性干扰，如电机齿轮转矩可以得到补偿。表的末尾处的校正相当于在零位置处的校正；如果你想要该校正是非零值，当你输入表时必须将 I30 设置为 1，所以 Turbo PMAC 将在零处的校正强制为此值。

如果您在输入表时将 I30 设置为 0，表的最后一项必须设置为 0，以在表翻转时获得校正的连续性。因为在电机的零位置处所需的扭矩校正一般不为零，在这种情况下，表中的入口应该包含在该点所需的转矩校正和在零位置处所需的转矩校正二者之间的差值。产生的表格将产生一个恒定的转矩偏移，然后伺服环的积分增益项将弥补这一偏差。

在表格定义命令的后面，发送到 PMAC 的下一个**{入口}**常数放入表中作为表中的入口。表中入口的单位是有符号的 24 位值，范围是-8,388,608 至+8,388,607。这些值比那些影响输出的 I 变量的有符号的 16 位值大 256 倍，如 Ixx69，Ixx57，Ixx29 和 Ixx79，它们的范围是-32,768 到+32,767。表中的入口之间的点处的修正值与表中的相邻值中进行线性地插值。

如果下面的表被输入：

```
#1 DEFINE TCOMP 8, 2000     ；用于电机1的表格，8个入口，2000个计数
32000                       ；在2000/8=250计数处的校正是32000/256=125， 16-bit DAC bits
-12800                      ；在500计数处的校正是–12800/256=-50 ，16-bit DAC bits
21248                       ；在750计数处的校正是21248/256=83， 16-bit DAC bits
-24832                      ；在1000计数处的校正是–24832/256=-97， 16-bit DAC bits
15360                       ；在1250计数处的校正是15360/256=60 ，16-bit DAC bits
-11008                      ；在1500计数处的校正是–11008/256=-43， 16-bit DAC bits
```

33024 ;在1750计数处的校正是33024/256=129， 16-bit DAC bits
-25600 ;在 2000(& 0)计数处的校正是–25600/256=-100 ，16-bit DAC bits

然后 600 计数处的校正应用于一个 16 位 DAC 将是：

$$\text{校正} = -50 + \frac{600 - 500}{750 - 500}(83 - [-50]) = 3 \text{ bits}$$

如何计算表中入口的值

转矩补偿表最常用于修正由于“齿轮转矩”效应产生的电动机的转矩脉动。确定表中的入口的最简单的方法是命令一个移动到表中每个你想要的点，并使用积分增益来驱动误差为零，等待电机停止在该位置。然后读取伺服环的命令输出为 24 位的值。此值最好在电机的正交/转矩指令值（X：$0000BF 用于电机 1）中读取。读取为一个 24 位的值（如在执行程序的 Watch 窗口中使用 **RX:$BF**），这就是那个位置的该表的入口。

报告表格信息

一个转矩补偿表的表头信息---用 **DEFINE TCOMP** 命令输入---可以用 **LIST TCOMP DEF** 命令查询。该表的内容可以通过 **LIST TCOMP** 命令查询

启用和禁用表格

当 I51 设置为 1 时所有的转矩补偿表（以及位置和间隙补偿表）被启用。I51 设置为 0 时，所有这些表禁用。

校正的有效计算

转矩补偿在伺服环内（每个伺服周期）进行，以获得最大的速度和精度。Turbo PMAC 拾取电机的位置并找到表中的匹配位置。通常，这是在表中的两个入口之间，所以 Turbo PMAC 这些项之间线性插值（加权平均）为当前伺服周期取得校正。然后，将该校正加到电动机的位置/速度伺服环所需的输出中。环路输出和校正在这一点都是 24 位的值。

一个电动机的转矩校正在每个伺服周期都存储在电机的一个特定的寄存器中。对于电机 1，这是在 **Y：$0000BF** 处的寄存器，带校正的净额输出是存储在 **X：$0000BF**。如果 I51 设置为 0，允许将值直接写入校正寄存器。不管 I51 的设置，此寄存器中的值在每个伺服周期都被加到电机的净额预期输出值上。

删除表格

DELETE TCOMP 命令擦除目前定址电机的转矩补偿表。转矩补偿表必须按顺序删除，从那些分配给编号较低电机的，到那些分配到较高编号电机的。

第十一章 TURBO PMAC 通用 I/ O 的使用

Turbo PMAC 控制器有与伺服操作无直接关系的大量的输入/输出功能。I/O 点均是数字和模拟的、输入和输出。板级 Turbo PMAC 控制器有一些板载的通用 I/O，以及更多可以用附件板添加。通过模块化 UMAC 系统，I/O 板可根据具体应用的需要进行添加。

本节概述了 Turbo PMAC 系列的通用 I/ O 功能。更多细节可以在特定的 Turbo PMAC 硬件参考手册或有关特定附件板的手册中找到。

11-1 Turbo PMAC 的通用 I/ O（JOPTO）端口

Turbo PMAC 上的 JOPTO 端口（Turbo PMAC PC、PCI 和 VME 上的 J5）提供八个通用数字量输入和 8 个通用数字输出。每个输入和输出在相对的列上每个都有其自己相应的接地引脚。该 34-针连接器是设计用于简单接口到 OPTO-22 或等效的光隔离 I/O 模块。为此，泰道公司的 Acc-21F 是 180 厘米（6 英尺）的电缆。请注意，这些 I/O 点在 Turbo PMAC 本身上都没有被光学隔绝。

硬件特性

Turbo PMAC 出厂标配了 ULN2803A 下沉式（集电极开路）输出 IC 用于 8 路的输出。这些输出电流每个可以高达 100 毫安，但必须有一个上拉电阻变高。

不要将这些输出直接连接到电源电压，否则过多的电流消耗会导致损坏 Turbo PMAC。

注意

跳线 E1 和 E2 设置错误会损坏 IC。

JOPTO 连接器的引脚 33 提供一个高压侧电压（+5 至+24 V），并通过连接跳线 E1 的引脚 1 和 2 拉高输出。跳线 E2 还必须连接引脚 1 和 2 用于 ULN2803A 下沉输出。

注意

跳线 E1 和 E2 设置错误会损坏 IC

通过用 UDN2981A IC 替换 ULN2803A， 可以使这些输出成为源级驱动器。这种 IC（Turbo PMAC PC 上的 U3，Turbo PMAC Lite 上的 U26，Turbo PMAC VME 上的 U33）配有插座，所以可以很容易地更换。对于这个驱动器，应使用下拉电阻。对于 UDN2981A 驱动 IC，跳线 E1 必须连接引脚 2 和 3，E2 跳线必须连接针脚 2 和 3。

跳线 E7 控制的八个输入的配置。如果它连接引脚 1 和 2（默认设置）时，

输入偏置为高侧电压（+5 V 至+24 V）为 OFF 状态，他们必须被拉低为 ON 状态。这些输入最好被认为是 24V-容错的 5V 逻辑，切换点为 2-3V。如果 E7 连接引脚 2 和 3，输入被偏置到地为 OFF 状态，必须拉高为 ON 状态。在这两种情况下，高电压被 Turbo PMAC 软件解释为 0，一个低电压被解释为 1。

软件访问

通常，这些输入和输出是通过使用 M 变量进行软件访问的。在所建议的一组 M 变量的定义中，变量 M1 至 M8 都是用于分别访问输出 1 到 8，M11 到 M18 分别访问输入 1 至 8。此端口映射为 Turbo PMAC 的内存空间在 Y-地址$078802。

11-2 Turbo PMAC 多路复用 I/O（JTHW）端口

一个 Turbo PMAC 的 JTHW（J3）连接器上的多路复用端口有八个输入线和 8 个输出线。输出线可用于复用端口上的大量的输入和输出，泰道公司提供附件板和软件结构（特殊 M 变量定义 TWB，TWD，TWR 和 TWS）以利用此功能。端口上可以进行菊花链式连接多达 32 个复用 I/O 板，以任意组合。

另外，端口的八个输入和八个输出可以直接使用，通过分配 M 变量到 I/O 点他们本身。此端口映射到 Turbo PMAC 的内存空间在 Y 地址$078801。用于这种用途的建议 M 变量定义是 M40 到 M47 用于八个输出，M50 到 M57 用于八个输入。

11-3 Turbo PMAC 控制面板端口

一个 Turbo PMAC 的 JPAN 连接器上的控制面板端口（Turbo PMAC PC，PCI，VME 上的 J2），是一个 26 针的连接器，有专用的控制输入端、专用的指示输出、一个正交编码器输入和一个模拟输入。控制输入为低电平有效，在他们的自动功能中有内部上拉电阻。他们已经预定义了功能，除非在控制面板-禁用 I-变量（I2）已被设置为 1。如果是这样的话，它们可以用作通用输入，通过将 M 变量分配给其相应的内存映射位置（Y 地址$078800 的位）。

控制面板输入

JOG-/、JOG+/、PREJ/（返回到预点动位置）和 HOME/输入影响由 FDPn/线所选择的电机（见下文）。STRT/（运行）、STEP/、STOP/（中止）和 HOLD/（进给保持）输入影响由 FDPn/线选择的坐标系。

警告

当保持点动输入之一为低电平时，不要改变选择器的输入。松开点动输入也**不会**停止先前选定的电机。这可能会导致一个危险情况。

四个低-真的 BCD 码输入线 FDP0/（LSBit），FDP1/，FDP2/和 FDP3/（MSBit）形成一个低-真的 BCD 编码的四位，（同时）选择激活的电动机和坐标系统。通常这些都受控于一个 4 位的电机/坐标系选择器开关。变量 I59 组选择开关来指定哪一组的八个电机和坐标系。这些输入线选择的电机将响应电机特定的输入。它也将其位置跟随功能打开（Ixx06 的 0 位被自动设置为 1）；取消选择的电机将其位置跟随功能关闭（Ixx06 的 0 位被自动设置为 0）。

控制面板输出

JPAN 连接器上有五个专用的低-真的输出，通常用来点亮 LED。他们是 BRLD/（缓冲区-请求 LED），IPLD/（就-位 LED），EROR/（故障跟随错误 LED），F1LD/（第一次警告-跟随误差 LED）和 F2LD/（当看门狗定时器跳闸时为真）。BRLD/，ERLD/和 F2LD/是全局状态线。IPLD/和 F1LD/是坐标系特定的状态线。如果 I2=0，它们指的是面板选择的坐标系（通过 FDPn/和 I59）。如果 I1=1，它们指的是主机选择的坐标系（&n）。

如果 I2=0 但没有选择坐标系（所有 FPDn/输入是悬空的或拉高），这些线可以用作通用输出，由 Y：$078802 的 20-23 位定址。

11-4 Turbo PMAC2 通用 I/O（JIO）端口

一个 Turbo PMAC2 的 JIO 端口（在 UMAC Turbo 的 Acc-5E 上）有 32 个离散的数字 I/O 线用于通用。该线是按字节配置的用于输入或输出（在 DSPGATE2 的 I/O 芯片上，该线是单独配置为输入或输出，但缓冲器 IC 只能按字节配置），并可单独配置为反相或非反相格式。

硬件特性

当配置为一个输出时，每个线有一个 5V 的 CMOS 图腾柱驱动器。该驱动器可以吸收或提供高达 20 毫安电流。在每线中有一个 10kΩ 上拉电阻到 5V 用于输入，但驱动器 IC 可以将该线保持为高或低电平而不管这个电阻。当配置为一个输入时，缓冲区 IC 表现为一个高阻抗的输入，吸入式或是下沉式；没有明显的电流流过。线上的上拉电阻将线偏置为高电平，而没有任何有效的将线拉为低电平在较低的阻抗。

注意：

由于所有这些线在上电或复位时默认为输入，任何被用作输出的线会在上电或复位时拉至+5 V，直到软件将它们配置为输出。

建议的 M-变量

注意：

在一个 UMAC Turbo 系统中，DSPGATE2 IC 可以驱动 JIO 端口在一个除标准$078400 外的其他基址上。然而，这是不太可能的,所以下面的讨论假定 IC 是$078400 的标准基地址。

32 个 I/O 线被内存-映射到 PMAC 的地址空间在寄存器 Y：$078400 和 Y：078401 中。通常，这些 I/O 线是通过 M 变量单独访问的。建议的 M-变量的完整列表会显示在软件参考中，少数的如下所示：

```
M0->Y:$078400,0       ; I/O00 Data Line; J3 Pin 1
M1->Y:$078400,1       ; I/O01 Data Line; J3 Pin 2
…
M23->Y:$078400,23     ; I/O23 Data Line; J3 Pin 24
M24->Y:$078401,0      ; I/O24 Data Line; J3 Pin 25
M25->Y:$078401,1      ; I/O25 Data Line; J3 Pin 26
…
M31->Y:$078401,7      ; I/O31 Data Line; J3 Pin 32
```

方向控制

对于每个 I/O 位，方向控制位是在匹配的 X 寄存器的相应位上。例如，I/O03 的方向控制位是位于 X：$078400,3；I/O30 的方向控制位位于 X：$078401,6。因为缓冲器 IC 只能按字节进行切换，最好是定义 8 位的 M-变量用于方向控制。建议的定义是：

```
M32->X:$078400,0,8          ; Direction control for I/O00 to I/O07
M34->X:$078400,8,8          ; Direction control for I/O08 to I/O15
M36->X:$078400,16,8         ; Direction control for I/O16 to I/O23
M38->X:$078401,0,8          ; Direction control for I/O24 to I/O31
```

这些 M 变量只能取值 0 或 255（$FF）；0 设置字节为输入，255 设置字节为输出。

此外，用于每个字节的双向缓冲器 IC 都有一个方向控制线，可访问为一个软件控制位。这些控制线和位必须匹配 ASIC 的方向位。缓冲区方向控制位是在 PMAC 地址 Y：$070800，其中 0 到 3 位控制 JIO 端口的四个字节。位值为 0 指定输入，位值为 1 指定输出。建议的 M 变量定义如下：

```
M33->Y:$070800,0      ; Buffer direction control for I/O00 to I/O07
M35->Y:$070800,1      ; Buffer direction control for I/O08 to I/O15
M37->Y:$070800,2      ; Buffer direction control for I/O16 to I/O23
M39->Y:$070800,3      ; Buffer direction control for I/O24 to I/O31
```

在上电或复位时自动设定的默认配置中，I/O00 到 I/O31 被设置为输入（M32 到 M39=0）。这么做是为了最大的安全性考虑；不可以将线强行进入一个不需要

的高或低电平的状态。任何这些用作输出的线必须由用户程序改变为输出，通常这是在 PLC1 上完成的，作为一个复位的 PLC，上电或复位时扫描一次，然后禁用本身。

反相控制

JIO 端口的每一线是单独可控的，关于它是一个反相的 I/ O 点（0= +5 V，1=0V），或一个非反相的 I/ O 点（0= 0V;1 =+5 V）。寄存器 X：$078404 和 X：$078405 包含了反向控制位：

- X：$078404 的 0 到 23 位分别控制 I/000 到 I/023。
- X：$078405 的 0 到 7 位分别控制 I/024 到 I/031。

控制位的值为 0 设置相应的 I/ O 点为非反相。控制位的值为 1 设置相应的 I/O 点反向。在上电或复位时，PMAC 自动设置 JIO 端口的所有 I/O 点为非反相。

备用用途

JIO 端口上的每个通用 I/O 点有一个备用用途，作为一个补充的固定使用的 I/O 点在一个补充的机器接口通道上（1* 或 2*）。该点关于通用或固定使用是独立控制的，通过控制寄存器 Y：$078404 和 Y：$078405。请参阅这些内存-I/O 映射的寄存器去看每个点的备用用途。在上电或复位时，Turbo PMAC 自动设置端口所有的 I/O 点用于通用用途。

注意：

缓冲器 IC 的字节宽方向控制必须正确设置为 I/ O 点的备用用途，就像为通用 I/O 用途那样。

11-5 Turbo PMAC2 多路复用 I/ O 端口（JTHW）

JTHW 复用器端口有 16 个一般用途的离散的数字 I/O 线。大多数人会将他们用在八个输入和八个输出端口的默认配置下，并支持泰道的复用的 I/O 附件。当以这种方式使用时，不需要 I/O 点的特殊设置。然而，如果需要直接使用这些 I/O 点，下面的讨论解释了它们的使用。

该线是按字节配置为输入或输出（在 DSPGATE2 的 I/O IC 上，这些线单独配置为输入或输出，但缓冲器 IC 只能按字节配置），以及单独配置为反相或非反相格式。

注意：

Turbo PMAC2 的变量 I20 为该系统中的第一个 MACRO IC 和 DSPGATE2 IC（其控制 JTHW 复用器端口）指定基地址。这个变量必须正确设置为自动复用器端口功能以正常工作。通常，此基址是$078400，下面的讨论假定系统是这种方式设置。

复用器端口配件

泰道公司提供了多种配件可连接到 JTHW 复用器端口，访问这些板的 I/O 有自动的固件支持。这些配件提供了一个直接的扁平电缆连接到 JTHW 端口，端口在上电或复位时自动配置与任何这些板一起工作。这些附件包括：

- Acc-8D Opt.7 旋变-数字转换器板
- Acc-8D Opt.9 安川绝对值编码器接口板
- Acc-18 指轮板（过时）
- Acc-34x 32-输入 32-输出板

对于 Acc-8D Opt.7 和 Acc-8D Opt.9 板，特殊设置的 Ixx91 和 Ixx95 支持从这些设备中读取绝对位置。也有特殊的 M 变量格式来访问大多数这些板上的 I/O 数据：

- Acc-8D Opt.7：TWS-格式的 M 变量用于绝对位置数据
- Acc - 18：TWD 格式的 M 变量用于数字的二进制编码的十进制读取
- Acc-34x：TWS-格式的 M-变量用于串行访问 32 位的输入或输出字

硬件特性

当配置为一个输出时，多路复用器端口上的每个线都有一个 5V 的 CMOS 图腾柱驱动器。该驱动器可以吸收或提供高达 20 毫安电流。在每线上有一个 10kΩ 的上拉电阻到 5V 用于输入，但驱动器 IC 可以不管这个电阻而将线保持为高或低电平。当配置为输入时，缓冲器 IC 表现为一个高阻抗输入，汇或源，没有显著的电流流过。线上的上拉电阻器将线偏置为高电平，没有任何有效的将线拉为低电平在低阻抗。

建议的 M-变量

16 个 I/O 线是内存-映射到 PMAC 的地址空间在寄存器 Y：$078402。通常，这些线被用作一个单元，有专门设计的复用的 I/O 附件和适当的复用 M 变量（TWB、TWD、TWR 和 TWS 格式），在这种情况下 Turbo PMAC 自动处理这些 I/O 线的直接控制。然而，这些线也可单独使用 M-变量访问。M 变量的完整列表会显示在软件参考手册中，少数几个如下所示：

```
M40->Y:$078402,8          ; SEL0 Line; J2 Pin 4
…
M47->Y:$078402,15         ; SEL7 Line; J2 Pin 18
M48->Y:$078402,8,8,U      ; SEL0-7 Lines treated as a byte
M50->Y:$078402,0          ; DAT0 Line; J2 Pin 3
…
M57->Y:$078402,7          ; DAT7 Line; J2 Pin 17
M58->Y:$078402,0,8,U      ; DAT0-7 Lines treated as a byte
```

方向控制

在上电或复位时自动设定的默认配置中，DAT0 到 DAT7 都设置为非反相输入；SEL0 到 SEL7 通过一个零（低电压）值设置为非反相输出。如果使用任何的复用器端口配件，该配置不能被改变。

每个 I/O 位的方向控制位是在匹配 X 寄存器的相应位。例如，DAT3 的方向控制位是位于 X：$078402,3；SEL6 的方向控制位是位于 X：$078402,14。因为缓冲器 IC 只能按字节进行切换，最好是定义 8 位的 M-变量用于方向控制。建议的定义是：

```
M60-> X：$078402,0,8          ;方向控制用于 DAT0 到 DAT7
M62-> X：$078402,8,8          ;方向控制用于 SEL0 到 SEL7
```

这些 M 变量只能取值 0 或 255（$FF）；0 设置字节为输入，255 设置字节为输出。

此外，每个字节的双向缓冲器 IC 都有一个方向控制线，可以访问为一个软件控制位。这些控制线和位必须匹配 ASIC 的方向位。在 ISA 和 PCI 总线版本的 Turbo PMAC 中，缓冲方向控制位是在 Turbo PMAC 地址 Y：$070800，4 位和 5 控制 JTHW 端口的两个字节。位值为 0 指定输入，为 1 指定输出。建议的 M 变量定义如下：

```
M61-> Y：$070800,4            ;缓冲器方向控制用于 DAT0 到 DAT7
M63-> Y：$070800,5            ;缓冲器方向控制用于 SEL0 到 SEL7
```

在 Turbo PMAC 的 VME 总线的版本，缓冲方向控制位是在 Turbo PMAC 地址 Y：$070802， 0 位和 1 位控制 JTHW 端口的两个字节。位值为 0 指定输入，为 1 指定输出。建议的 M 变量定义如下：

```
M61-> Y：$070802,0            ;缓冲方向控制用于 DAT0 到 DAT7
M63-> Y：$070802,1            ;缓冲方向控制用于 SEL0 到 SEL7
```

如果需要改变其中任一 I/O 字节，它必须由用户程序来完成。通常这是在 PLC1 上完成，作为一个复位的 PLC，在上电或复位时扫描一次，然后禁用本身。

反相控制

JTHW 端口在每一行都是单独可控的，关于它是一个反相的 I/ O 点（0= +5 V；1=0V），或一个非反相的 I/O 点（0= 0V；1=+5 V）。寄存器 X：$078406 包含了反相控制位：

- X：$078406 的 0 到 7 位分别控制 DAT0 到 DAT7。
- X：$078406 的 8 至 15 位分别控制 SEL0 到 SEL7。

控制位的值为 0 设置相应的 I/ O 点为非反相。控制位的值为 1 设置相应的 I/ O 点为反相。在上电或复位时，PMAC 自动设置 JTHW 端口所有的 I/ O 点为非反相。要使用 JTHW 端口任何的复用 I/O 附件板，端口所有的 I/O 点必须保持非反相。

备用用途

JTHW 端口上的每个通用 I/O 点有一个备用用途，作为一个补充固定使用的

I/ O 点，在一个补充的机器接口通道（1*或 2*）上。这些点关于一般用途或固定使用是独立控制的，由控制寄存器 Y：$078406 控制。请参阅在内存-I/O 映射中的本寄存器，看每个点的备用用途。在上电或复位，Turbo PMAC 自动设置端口所有 I/O 点用于一般用途。

注意：

由于字节宽的方向控制缓冲器 IC，在其备用用途中不可能使用 JTHW 的所有 I/O 点。

11-6 Turbo PMAC 模拟输入（JANA）端口

只有当选择 Opt.12 用于 Turbo PMAC2 或 Turbo PMAC PCI 板卡时，才存在模拟输入（JANA）端口。Opt.12 提供了 8 路的 12 位模拟输入（ANAI00-ANAI07）。Opt.12A 提供 8 个额外的 12 位模拟输入（ANA08-ANAI15），共 16 个输入。

硬件特性

模拟输入可以作为单极输入，范围是 0V 到+5 V，或双极性输入，范围在-2.5V 至+2.5 V。每个输入有一个直列的 470Ω输入电阻和一个 0.033μF 电阻接地。这在每个输入线上提供了 16 微秒的时间常数。

Turbo PMAC 的模拟-数字转换器需要+5 V 和-12V 的电源。这些电源是不隔离于 PMAC 的数字+5 V 电路的。如果 Turbo PMAC2 插入到总线（ISA、PCI、VME），这些电源都取自总线的供电电源。在一个独立的应用程序中，这些电源必须在接线端子 TB1 上产生。

-12V 和匹配的+12 V 的电源电压在 J1 连接器上是可用的，来为提供信号的模拟电路供电。Turbo PMAC 不使用+12 V 电源；它仅仅是为了方便传递到 J1 连接器上。如果需要使用这个电源，它必须是来自总线电源，通过 Turbo PMAC 的总线连接器，或来自 TB1。

复用原理

在任何给定的时间，Turbo PMAC 的处理器只能使用一对模拟-数字转换器寄存器。数据显示到 Turbo PMAC2 地址 Y:$078800 的处理器;数据出现在 Turbo PMAC PCI 的 Y：$078808 中。选定的模拟输入 0 到 7（ANAI00-ANAI07）的数据出现在低 12 位；选定的模拟输入 8 到 15（ANAI08-ANAI15）的数据出现在高 12 位。这个数据仅在选择 Opt.12A 时存在。

通过写入到同样的字地址 Y：$078800，来选择输入和启动转换。写入低 12 位的 0～7 值选择那个数字的模拟输入通道（ANAI00-ANAI07），转换在单极模式（0V 至+5 V）。0～7 的值写入高 12 位选择大 8 个数的模拟输入通道（ANAI08-ANAI15）在单极模式。如果写入到低 12 位或高 12 位的值大于 8（8～15），相同的输入通道被选中，但转换是在双极性模式（-2.5V 到+2.5 V）。

解除复用的 I-变量

Turbo PMAC I-变量 I5060-I5096 允许自动解除复用这些复用的 A/ D 转换器（以及解除复用位于外部 Acc-36 和 Acc-59 电路板上的复用 A/D 转换器）。

I5060 控制在解除复用环中访问的 A/D 转换器对的数量，多达 16 对。变量起始于 I5061，并且可能到 I5076，指定每个 ADC 对要读取的 Turbo PMAC 地址。JANA 端口上所有八对的的地址位于$078800 中。

变量起始于 I5081，并且可能到 I5096，指定读取位于由编号低 20 的 I-变量指定的地址处的哪一对 ADC（例如 I5081 是用于 I5061），以及它是如何进行转换。I5081-I5096 是 24 位值，由 6 个十六进制数字表示。合法值的格式是$00m00n，其中 m 和 n 可以取 0 到 F 的任何十六进制值。

对于 Turbo PMAC2 上的在板的 Opt.12 和 12A 的 ADC，m 值决定了读取 Opt.12A 附带的 ANAI08 到 ANAI15 的输入中的哪一个，以及它是如何被转换，根据下面的公式：

- m = ANAI#-8 ；0 至+5 V 的单极性输入
- m = ANAI# ；-2.5V 到+2.5 V 的双极性输入

对于 Turbo PMAC2 上的板载 Opt.12 和 12A 的 ADC，n 值决定了 Opt.12A 附带的输入 ANAI00 到 ANAI07 中哪一个被读取，以及它是如何被转换的，根据下面的公式：

- n = ANAI# ；0V 至+5 V 单极性输入
- n = ANAI#+8 ；-2.5V 到+2.5V 的双极性输入

该 A/D 解除复用的结果是放在寄存器中，地址是 Y:$003400 到 Y:$00341F，使用这些寄存器的 12～23 位。源寄存器的低 12 位中找到的 A/D 转换器的值被放置在偶数地址的寄存器中，在源寄存器的高 12 位发现的 A/D 转换器的值被放置在奇数地址的寄存器中。参考 Turbo PMAC 内存映射或 I5061-I5076 的描述，获取细节。结果寄存器的建议 M-变量是 M5061-M5076。

在操作中，Turbo PMAC 在每个相位周期读取一个 ADC 对，并且将其复制到相应的内存寄存器中。因此，它会每 I5060 个相位周期读取每个 ADC 对。如果这些值被用作伺服环的反馈，该环路的关闭不应该比读取 ADC 更频繁。

11-7 UMAC 数字 I/O 板卡

UMAC 拥有广泛的数字 I/O 板卡系列。下表总结了这些板和它们的属性：

ACC	输入#	输入范围	输入驱动器	输出#	输出范围	输出驱动器	注意
9E	48	12V-24V	下沉式/吸入式，通过接线选择	0	-	-	隔离的 I/O
10E	0	-	-	48	5V-24V	原厂选择，吸入式/下沉式	隔离的 I/O
11E	24	12V-24V	下沉式/吸入式，通过接线选择		5V-24V	原厂选项吸入式/下沉式	隔离的 I/O
14E	0-48	5V	TTL/下沉式	48-0	5V	带上拉的下沉式	TTL，每个可逆
65E	24	12V-24V	仅吸入式	24	5V-24V	仅吸入式	隔离的，受保护的 I/O
66E	48	12V-24V	仅吸入式	0	-	-	隔离的，受保护的 I/O
67E	0	-	-	48	5V-24V	仅吸入式	隔离的，受保护的 I/O

寻址 UMAC I/O 板

所有这些电路板采用泰道的内存-映射 IOGATE ASIC，它支持 48 个 I/O 点，映射为 6 个连续定址的 8 位寄存器。在接下来的两个 8 位寄存器中是设置和控制位。IOGATE IC 的基地址是通过 Acc-9E、10 和 11E 板上的跳线设置的，是由 14E、65E、66E、67E 板上的 DIP 开关设置的。

Acc-14E、65E、66E 和 67E 板卡支持由 UMAC CPU 自动识别。状态变量 I4950 - I4965 表明这些电路板中的哪些存在，以及在什么地址。而 Turbo 安装程序可以自动识别并显示这些变量的结果，表明哪些板存在。它还可以识别 Acc-9E、10E 和 11E 电路板的存在，但由于这些旧的板卡缺乏自我识别功能，它不能分辨每一个都是哪种类型的板卡。

带跳线集地址的板卡

对于 Acc-9E、10E 和 11E 板，板的基地址设置的完成是通过将一个跳线放在 E-点 E1-4 的其中一个且仅仅一个上。板卡上的寄存器被发现在八个连续的地址：

{基地址}到{基地址+7}。在其中发现寄存器的 Turbo PMAC 的 24 位字的字节---低（0 - 7 位），中（8-15 位）或高（16-23 位）---是由跳线库 E6A- H 的设置决定的。用于这些板卡基地址的设置汇总于下表：

地址跳线	E6A H Pins 1&2	E6A H Pins 2&3	E6A H Pins 1&2
E1	Y:$078C00 Bits 0 - 7	Y:$078C00 Bits 8 - 15	Y:$078C00 Bits 16 -23
E2	Y:$078D00 Bits 0 - 7	Y:$078D00 Bits 8 - 15	Y:$078D00 Bits 16 -23
E3	Y:$078E00 Bits 0 - 7	Y:$078E00 Bits 8 - 15	Y:$078E00 Bits 16 - 23
E4	Y:$078F00 Bits 0 - 7	Y:$078F00 Bits 8 - 15	Y:$078F00 Bits 16 - 23

带开关-集地址的板卡

对于 Acc-14E、65E、66E 和 67E 板卡，板的基址是由 DIP 开关 SW1-1 到 SW1-4 的设置确定的。当这些板卡均采用 Turbo UMAC CPU 时，SW1-5 和 SW1-6 必须始终为 ON。这些板总是出现在 24 位字的低字节（0 - 7 位）。

开关设置	SW1-3 ON SW1-4 ON	SW1-3 OFF SW1-4 ON	SW1-3 ON SW1-4 OFF	SW1-3 OFF SW1-4 ON
SW1-1 ON SW1-2 ON	Y:$078C00	Y:$079C00	Y:$07AC00	Y:$07BC00
SW1-1 OFF SW1-2 ON	Y:$078D00	Y:$079D00	Y:$07AD00	Y:$07BD00
SW1-1 ON SW1-2 OFF	Y:$078E00	Y:$079E00	Y:$07AE00	Y:$07BE00
SW1-1 OFF SW1-2 OFF	Y:$078F00	Y:$079F00	Y:$07AF00	Y:$07BF00

注意，在某些情况下，这两种类型的电路板可以被设置为相同的地址。当然，没有任何寻址冲突非常重要。

设置 UMAC I/O 板

IOGATE IC 本身的 I/O 点是按字节选择为输入或输出。但是，只有 Acc-14E TTL 电平的 I/O 板给你一个选择，关于哪些 I/O 点是输入以及哪些是输出。在这些所有其他的板上，周围的缓冲区/驱动电路决定每个 I/O 点必须如何使用。IOGATE IC 必须在每次上电/复位时设置，以确定每个 I/O 点的方向，这通常是在一个“一次性”的 PLC 程序中进行的。每块板的手册显示了可以完成这个的示例程序代码。

在典型应用中，对于与 UMAC I/ O 板的操作很少需要对 IOGATE IC 进行设置。然而，IOGATE IC 具有特殊的功能，在不寻常的应用中非常有用。以下部分详细介绍了如何使用 IOGATE IC 的控制寄存器和设置寄存器用来提供极大的灵活性。

控制寄存器

在地址{基址+7}处的控制寄存器允许配置 IOGATE IC 用于各种应用程序。控制寄存器包括 8 个回-读/写位--- 0- 7 位。

方向控制位

控制寄存器的 0 到 5 位简单地控制 I/O 的方向用于匹配编号的数据寄存器。也就是说，位 n 控制{基址+ n}处的 I/O 的方向。控制位值为 0（默认值）允许一个写操作到数据寄存器，启动寄存器上的每一线的输出功能。启动输出功能，

并不妨碍使用任何或全部线作为输入，只要输出关闭（不导通）。控制位值为 1 不允许一个写操作到数据寄存器，禁用输出，保留寄存器用于输入。

例如，Bit 3 的值为 1 将禁用写入功能到地址{基址+3}的数据寄存器，确保线路 IO24 – IO31 始终被用作输入。

寄存器选择控制位

控制寄存器的 6 位和 7 位一起选择在地址{基址+0}到{基址+5}处的哪 4 个寄存器可以被访问。他们还选择哪两个寄存器可以在{基址+6}来选择。

下表说明这些位如何选择寄存器：

Bit7	Bit 6	组合值	字节值*	{基址+0}到{基址+5}寄存器选择	{基址+6}寄存器选择
0	0	0	$00	数据寄存器	数据寄存器
0	1	1	$40	设置寄存器 1	设置寄存器
1	0	2	$80	设置寄存器 2	
1	1	3	$C0	设置寄存器 3	不详
*0-5 位设为 0					

在一个典型的应用程序中，6 和 7 位的非零组合值只用于 IC 的初始配置。这些值被用于访问在另外地址的设置寄存器。配置完成后，6 和 7 位都写入零，所以可以访问在另外寄存器的数据寄存器。

设置寄存器

在 IC 地址{基地址+0}到{基地址+ 5}的每个地址处总共有四个寄存器可访问：三个 8 位的设置寄存器和一个 8 位的数据寄存器。设置寄存器控制数据如何写入数据寄存器以及从其中读取。

设置寄存器 1:反向控制

在从{基地址+ 0}到{基地址+ 5}的每个地址处的设置寄存器 1,其的选定是通过向{基地址+7}处控制字的 6 位写入一个 1 以及写入一个 0 到 7 位,其是反转控制寄存器用于在相同的地址处的数据寄存器。设置寄存器 1 的每一位控制在相同的地址的数据寄存器的匹配位的反转。

设置寄存器值 1 的位值为 0 指定一个反相的 I/0 点用于同一地址的数据寄存器的匹配位。也就是说，对于一个输出端，0 值产生一个低电平（导通）的输出，1 的值产生一个高电平（不导通）的输出。对于一个输入，一个线拉为低电平产生一个 1 值，而线拉高或允许悬空为高电平会产生一个 0 值。

设置寄存器 1 的位值为 1 指定一个非反相的 I/0 点用于同一地址的数据寄存器的匹配位。也就是说，对于一个输出端，0 值产生一个高电平（不导通）的输出，1 值产生一个低电平（导通）的输出。对于一个输入，一个线拉低产生一个 0 值，而线拉高或允许悬空为高电平产生一个 1 值。

设置寄存器 2：读取控制

在每个{基地址+0}到{基地址+5}地址的设置寄存器 2，它被选中是通过向在{基地址+7}处的控制字的 6 位中写入一个 0 以及写入一个 1 到 7 位，是同一地址的数据寄存器的读取控制寄存器。设置寄存器 2 的每一位控制从同一地址的数据寄存器的匹配位中读取哪些数据。

设置寄存器 2 的位操作是依赖于同一地址的设置寄存器 3 的匹配位的设置。如果设置寄存器 3 的匹配位为 0，选择非锁定的输入，设置寄存器 2 的位，控制读取引脚的值，还是读取可写寄存器中的值。设置寄存器 2 的位值为 0 选择从同一地址数据寄存器的匹配位读出引脚值；位值为 1 选择可写寄存器的值。

如果设置寄存器 3 的匹配位为 1，选择锁存输入，设置寄存器 2 的位则控制是读取直接锁存的数据，还是读取一个格雷码-二进制的转换结果值。设置寄存器 2 的位值为 0 选择从同一地址的数据寄存器的匹配位读出直接的锁存值；该位的值为 1 选择读取格雷码-二进制的转换结果值。

设置寄存器 3：锁存控制

在每个{基址+0}到{基址+5}地址的设置寄存器 3，它被选中是通过向在{基址+7}处的控制字的 6 位写入 1 以及写入 1 到 7 位中，它是锁存控制寄存器用于同一地址的数据寄存器。设置寄存器 3 的每一位控制是从同一地址的数据寄存器的匹配位上读取锁定的还是未锁存的数据。

设置寄存器 3 的位值为 0 选择从同一地址的数据寄存器的匹配位读取非锁定的数据，该位值为 1 选择读取锁存的数据。

对于这两种锁存和未锁存的设置，设置寄存器 2 的匹配位控制从数据寄存器读取什么类型的数据。

数据寄存器

在{基址+0}到{基址+5 }的每个地址处的数据寄存器，它被选中是通过向{基址+7}处的控制字的 6 位写入 0 以及写入 0 到 7 位中，为匹配那个地址的 8 个输入/输出线提供工作接口。处理器读取或写入数据寄存器，以访问输入/输出线。

如果控制字的 n 位（n = 0 至 5）的值为 1，到地址{基址+ n}处的数据寄存器的一个写操作对 I/O 线没有影响，对寄存器相关的所有 8 个线有效地禁用了输出功能。

对数据寄存器的一个读操作可以访问 4 种数据类型之一用于与寄存器相关的每个 I/O 线（单独选择），这取决于同一地址的设置寄存器 2 和 3 是如何配置的。

下表总结了设置寄存器的位，如何控制在数据寄存器的匹配位中读取哪些数据：

设置寄存器 3 的位值	设置寄存器 2 的位值	数据类型读取
0	0	引脚值读取
0	1	可写寄存器读取
1	0	锁存输入读取
1	1	转换的格雷码读取

引脚的数据读取

如果选择要读取引脚的值，那么 8 个 I/O 线的线路电压被读取为一个输入，即使它是被驱动为一个输出。

可写寄存器数据读取

如果选定读取可写的寄存器值，那么处理器写到数据寄存器中的值被读回，即使这不匹配引脚的电压状态。

锁存输入读取

如果选择要读取锁存的输入，那么地址{基址+ n}的线 En 的下降沿最后锁存的输入值被读取，即使输入值已经改变了。

转换的格雷码读取

如果选择读取转换的格雷码输入，那么地址{基址+ n}的线 En 最后锁存的，并通过格雷码转换电路处理的输入值会被读取。

第十二章 确保应用程序安全

泰道公司数据系统在 Turbo PMAC 控制器上提供了许多安全功能，并投入不少资源使 Turbo PMAC 成为一个安全的产品。但是，对于使用 Turbo PMAC 的控制系统的安全，系统设计师必须负有最终的责任，因为他使用 Turbo PMAC 中和系统的其他部分的安全功能。

12-1 跟随误差极限

Turbo PMAC 对于每个电机有三个跟随误差极限。跟随误差是在任何时间的指令位置和实际位置之间的差值。跟随误差限制是一个重要保障针对严重的系统故障，如反馈信号丢失，它可能会导致危险情况如全速失控。

故障跟随误差限制

其中的一个限制（Ixx11）是故障限制，它导致电动机的停机：伺服环路开通、输出为零、放大器禁用（即电机停机）、电机的移动和电机坐标系的程序中止。

哪个电机停止---只是违规超出极限的电机，坐标系或 Turbo PMAC 的所有这些---是由故障电机 Ixx24 的 21 位和 22 位决定的。此限制适用于其中有严重问题的情况（例如反馈或功率级丢失），所有操作应停止。

在由于故障跟随误差限制导致电机或多个电机停机后，闭环控制可以重新建立，通过 J/命令（单个电机）、A 指令（坐标系）或者<CTRL-A>命令（整个卡）。

警告：

虽然这种限制可以通过将 Ixx11 设置为零而禁用，但是在那些可潜在杀死或伤害人甚至是导致财产损失的任何应用程序中，这都是强烈不鼓励的。禁用故障限制去除了针对可能会导致失控的严重故障情况的一个重要保障，使整个系统达到全功率输出，比任何人的反应都快。

调整好电机的伺服回路对于安全以及性能方面是重要的。在正常运行期间真正的跟随误差越小， Ixx11 的限制可设置的越紧密而无需干扰行程。在这方面特别重要的是前馈项，它可以极大地减少在高速和高加速度时的误差。

警告跟随误差极限

第二个限制（Ixx12）是一个警告限制---当超过该限制时，Turbo PMAC 为电机和电机的坐标系设置状态位，并可以在控制面板连接器、机器接口设置输出线，并通过可编程中断控制器（用于 ISA 和 PCI 总线的 Turbo PMAC）。这允许采

取特殊动作，无论是通过 PLC 程序由 Turbo PMAC 本身，由主机完成（主机可以通过中断或查询卡发现），还是通过一个外部信号通知操作员完成。

如果 Ixx97 的 0 位被设置为 1，电机的警告跟随误差状态位可以作为触发条件用于任何 Turbo PMAC 的自动触发运动（回零搜索移动，点动-直到触发，编程 RAPID-模式移动-直到触发）。这使得一些任务很容易实现，如回零到一个硬停止，限制扭矩的螺纹传动等。

跟随误差的积分保护

除了每个电机的 Ixx11 变量提供正常的跟随误差保护外，如果跟随误差的时间积分值超过了预设值，Turbo PMAC 可以关闭电机。这种积分的误差功能可以对某些测量跟随误差的幅度永远不会非常大的情况起到保护作用保护---例如，一个丢失的反馈之后有一个很短的指令移动。

只有在 Ixx63 积分极限参数小于零时，Turbo PMAC 才会对跟随误差的积分进行检查。当这种情况时，Ixx11 的大小被用于检查正常的未积分的跟随误差。此外，PID 积分器的值与 Ixx63 积分限制幅值进行比较。如果积分值饱和在 +/-Ixx63（在 PID 回路的限制功能不会让它超过此值），那么 Turbo PMAC 将由于一个积分的跟随误差故障而跳闸（停止）这个电机，就像它对一个正常的跟随误差故障那样。

为了积分的跟随误差极限是有效的，Ixx33 积分增益必须大于零，并且设定尽可能的高值。此外，Ixx34 积分模式参数必须被设置为 0，从而使积分器在编程移动过程中是打开的状态。

请记住，如果该命令的输出已经饱和在 Ixx69，积分器会停止增加幅值。Ixx63 的大小必须足够小，这样它会在输出饱和之前跳闸。将导致输出在 Ixx69 饱和的 Ixx63 的幅值是：

$$|\text{Ixx63}| = \left(\frac{\text{Ixx69} * 2^{23}}{\text{Ixx08} * \text{Ixx30}}\right)$$

为使关闭功能是有效的，Ixx63 的大小必须小于该值。记住，输出的组成不止来自积分器，例如来自比例增益。在一个裸电机上，测试一下，看看这个限制是否可以可靠地停止电机。

当电动机由于积分的跟随误差故障而停机，标准的跟随误差故障电机状态位被置位。此外，一个单独的积分跟随误差故障电机状态位被置位。当重新启用电机时这两个位被清除。

12-2 位置（超程）限制

Turbo PMAC 具有硬件和软件超程限位功能。这是为了防止指令运动意外地超出位置的合法范围。

硬件超程限位开关

一个 Turbo PMAC 系统的每个伺服接口通道的轴接口电路都有正和负的硬件

超程限位开关输入。此输入电路的确切性质和连接限位开关的说明是描述在每个 Turbo PMAC 与轴接口附件的硬件参考手册中。

一般来说，这些输入是光隔离的，通过一个故障保护电路设计。限位开关必须常闭，当轴未在限制中时传导电流通过光隔离器。这种导电状态在伺服 IC 通道的标志寄存器中产生一个零状态，处理器必须读取这个零值以允许那个方向的运动。

凡是阻止电流流过光隔离器的东西，无论是实际达到极限，电缆断开，或限制电路电源的丢失，都在伺服 IC 中产生一个 1 状态。当处理器看到时，它不会允许在该方向上的运动。

电机的变量 Ixx25 必须包含了限位开关接线的通道的标志寄存器的地址。Ixx24 的 17 位必须被设置为默认值 0 以使用这些限制输入。如果该位被设置为 1，Turbo PMAC 不会监视这些限制输入。如果电机没有限位开关，该位被永久设置为 1；它可以暂时被设置为 1 用于操作如回零到一个极限。

到达限制时，Turbo PMAC 会以 Ixx15 定义的用户编程的速度减速违规的电机。如果那时电机是在一个运行着运动程序的坐标系中，该坐标系中的所有电机都以其自己的 Ixx15 速率减速停止。该效果等同于发出一个 A（中止）命令。如果坐标系统已经执行了一个路径移动，这种减速并不一定是沿着那条道路。

注意：

只要它检测到一个限制条件，Turbo PMAC 就使电机的指令轨迹以 Ixx15 的速率停止。如果当时有显著的跟随误差，实际的位置可以尝试赶上指令位置用很长的一段时间。对于足够大的跟随误差，有可能指令位置早已超过限制并进入硬停止。很重要的是设置一个合理的故障跟随误差极限，并有足够超过限位开关的空间来吸收达到那个跟随误差限制的误差。

限制输入的引脚对方向敏感：正端的限位引脚仅停止正方向的移动（那些从反方向撞到限位），负端的限制引脚仅停止负方向的移动（那些从正方向撞到限位）。这使得可以指令一个移动从你运行到的限位中离开。然而，这也必须将你的限位开关连接到正确的输入，否则他们将是无用的。

软件超程限位变量

Turbo PMAC 对每台电机也有正和负的软件限位，以补充或代替硬件限制。这些限制可以使用电机的期望以及实际的位置。电机变量 Ixx13 和 Ixx14 分别为电机 xx 定义的正和负的实际位置限制，单位是计数。这些限制是参照电机的零点（原点）的位置，并且如果相关轴的编程原点是偏移的，限制不会改变。

Turbo PMAC 不断将电机的实际位置与这些限制进行比较。超出这些限制的行为等同于触及一个硬件限制。这些参数中的一个零值将禁用该软件的限制。

如果电动机的 Ixx24 的 15 位被设置为 1，Turbo PMAC 也将电机的期望位置---为编程运动提前计算---与软件限制进行比较。这使电机完全停止在限制范围内，而不只是在限制处开始减速。这反过来又提供一个机器周长的额外的运动范围。

注意：

Turbo PMAC 不能为不确定的微动移动 J+和 J-预先计算预期的位置，所以不会开始对这些移动减速，直到目前的期望位置超出软件限位。如果需要微动动作停止在限制范围内，J+命令应该由一个确定的微动来代替到正极限；一个 J-命令应该由一个确定的微动取代到负极限。

对于一个 LINEAR 或 CIRCLE-模式移动，执行时激活特殊前瞻缓冲区，当超过一个期望位置极限时，移动将沿编程路径正好停止在限位处，减速是由电机的 Ixx17 最大加速度参数控制的。这等同于\快速停止命令。在这种情况下，在改变超限的限制参数后，有可能恢复沿路径的运动。(如果 Ixx24 的 14 位被设置为 1，在前瞻模式的这些运动在撞到限位时将不会停止；违规电机的指令位置将饱和在限制值。)

电机变量 Ixx41 定义电机的实际和期望的位置限值之间的差值（单位是计数），允许预期位置限制设定在实际位置限制内。正向的期望位置极限设定在（Ixx13- Ixx41)；负向的期望位置限制设置为（Ixx14 +Ixx41)。这是重要的以确保该前瞻内路径移动的限制减速停留在路径上，并且如果需要的话启动恢复该程序。Ixx41 应设置得足够大，以确保在超过期望位置限制后减速的过程中，实际位置不超过实际位置的限位。

在回零搜索运动中，该软件位置限制是自动禁用的，直到发现回零触发。一旦找到触发后，该软件限位被重新激活，使用新的起始位置作为参考。该软件位置限制总是参考电机的零位，不管是由一个回零搜索运动、绝对位置读取确立的，还是在上电或复位时确立的。如果分配给电机的轴的编程原点是偏移的，软件位置限制不会自动用编程原点进行偏移。

12-3 速度极限

矢量速度限制

Turbo PMAC 有一个矢量速度极限参数（Isx98)，被称为每个坐标系的最大进给速度。对于这个坐标系统运行的程序，该程序中的任何 F（进给速度）命令的值与 Isx98 进行比较。如果该值大于 Isx98，使用 Isx98 的值作为进给速度指令。Isx98 表示为与进给命令相同的用户速度的单位（轴长度单位除以 Isx90 毫秒)。

电机速度限制

Turbo PMAC 为每个电机有一个可编程的速度极限参数（Ixx16），单位是每毫秒的计数，它有多种功能。首先，如果电机的快速-选择参数 Ixx90 被设置为默认值 1，它可以作为指令速度用于 RAPID（快速）-模式移动的电动机。

第二，对于简单的线性(LINEAR)模式移动而禁用运动分段(Isx13=0)，Ixx16 作为允许的最大速度。如果电动机所需的指令速度超过了电机的限制，那么移动会变慢从而不超过速度限制。在多轴编程移动中，坐标系中的所有轴都按比例减慢从而路径不会发生变化。

此外，对于执行直线（LINEAR）和圆弧（CIRCLE）模式的移动（并启用分段（Isx13>0）以及激活特殊前瞻缓冲区），它作为各段运动的最大速度。这对于采用 Turbo PMAC 的运动学方程编程的非笛卡尔坐标系统特别有价值；可在不经意间命令非常高的电机速度接近“奇异点”。前瞻算法可以事先检测到这些问题，并沿路径减缓运动进入问题点，观察 Ixx17 电机加速度极限。

速度与这些限制进行比较时假设无进给速度倍率（100%值）；如果使用进给速度倍率（又名时基控制），速度极限随倍率而缩放。

12-4 加速度限制

Turbo PMAC 对于每个电机有两个可编程的加速度限值，一个用于手动、回零和快速（RAPID）移动模式（Ixx19），一个用于直线（LINEAR）和圆弧(CIRCLE)模式的编程移动（Ixx17）。这两个参数的单位都是每（毫秒平方）的计数。PVT 和 SPLINE(样条)模式的移动并不遵守任何的这些限制。

如果电机所需的指令加速度在速度和加速时间参数中的变化超过了电机的极限，那么加速时间延长从而使不超过加速度极限。在一个多轴的编程移动中，坐标系中的所有轴都成比例地减慢从而使路径不会发生变化。加速度与这些极限进行比较时假设无进给速度倍率（100%值）；如果使用了进给速度倍率（又名时基控制），加速度极限会随着倍率百分比的平方而缩放。

未启用特殊前瞻缓冲区，Ixx17 加速度限制只适用于线性模式同时禁用运动分段（Isx13=0）。在这种操作模式下，加速度的时间可以延长等于运动的移动时间。如果要观察加速度极限而延伸的不够长，该撞到该限位。

启用分段（Isx13>0）和激活特殊前瞻缓冲区，该 Ixx17 极限工作于线性和圆弧-模式移动。确保所有这些动作观察这个加速度限制是特殊前瞻缓冲区的最重要的功能。

12-5 指令输出限制

Turbo PMAC 有一个可编程的输出限制（在 Turbo PMAC 发送到放大器或内部换相算法的命令中）用于各个轴（Ixx69），充当一个扭矩/力限制用于电流模式、正弦波或直接 PWM 放大器，或作为一个速度限制用于速度模式放大器。如果这个极限是来改变伺服环的命令，那么 Turbo PMAC 的抗积分饱和保护被激活，以防止当出现极限情况时发生振荡。此外，还有一个反馈滤波器允许看到的关于误差大小的限制（Ixx67“位置误差”极限），它的作用是以一种可控的方式减缓一个

太突然的移动。

12-6 积分电流（I²T）保护

如果时间积分电流电平超过一定的阈值，Turbo PMAC 可以设置用来故障一个电机。这可以保护放大器及/或电机不会因过热而受损坏。它可以随时间积分电流的平方- 通常被称为 I^2T 保护，或着随时间积分电流的绝对值- 通常被称为|I|T 保护。当最为热敏感的元件是电阻性时使用 I^2T 保护（如电机绕组或 FET（场效应）晶体管），因为它们的功耗正比于电流的平方。当是最热敏感的元件有一个恒定的电压压降时使用|I|T 保护（例如，双极型晶体管），因为它们的功耗正比于电流的大小。因为使用电流的平方是比较常见的，这种保护一般被称为“I^2T”。

有些放大器有自己的内部积分电流保护，但很多其他的没有。Turbo PMAC 的积分电流保护可以在任一情况下被使用。它可以与任何由 Turbo PMAC 计算电流指令的放大器使用，不论 Turbo PMAC 是否执行换向和/或数字电流环路功能。如果 Turbo PMAC 正在为电机闭合电流环，该功能使用测得的电流值；否则它使用指令电流值。这种保护是不适合用于那些在其中 Turbo PMAC 输出一个速度指令的系统，无论是作为一个模拟电压还是一个脉冲频率。

两个 I 变量控制每个电机的 I^2T 保护功能。Ixx57 是连续电流限制幅度。它的单位与 Ixx69 瞬时输出限制的相同，都是 16 位 DAC 的位（即使使用其他一些输出设备）。两者都有 32,767 的最大幅度，这是 Turbo PMAC 的最大可能的输出的大小。如果 Ixx57 是一个正值，使用 I^2T 保护；如果 Ixx57 是一个负数，使用|I|T 保护。通常 Ixx57 是 Ixx69 的幅度的 1/4 至 1/2。

Ixx58 是积分电流限制参数。如果 Ixx58 设置为 0，此功能被禁用。如果 Ix58 大于 0，Turbo PMAC 将积分电流值与 Ixx58 进行比较。当积分的电流值超过此值，Turbo PMAC 将故障这个电机，仿佛发生一个放大器故障。违规电机被停机；如果是在一个运行运动程序的坐标系中，中止该运动程序；其他电机根据 Ixx24 的 21 位和 22 位的设定而停机。

Turbo PMAC 的I^2T功能的工作原理是根据下面的方程，：

$$\text{sum} = \text{sum} + \left[\left(\frac{I_q}{32768}\right)^2 + \left(\frac{I_d}{32768}\right)^2 - \left(\frac{Ixx57}{32768}\right)^2\right]\Delta t$$

Turbo PMAC 的|I|T 功能根据下面的公式工作：

$$\text{sum} = \text{sum} + \left[\left(\frac{|I_q|}{32768}\right) + \left(\frac{|I_d|}{32768}\right) - \left(\frac{|Ixx57|}{32768}\right)\right]\Delta t$$

其中：

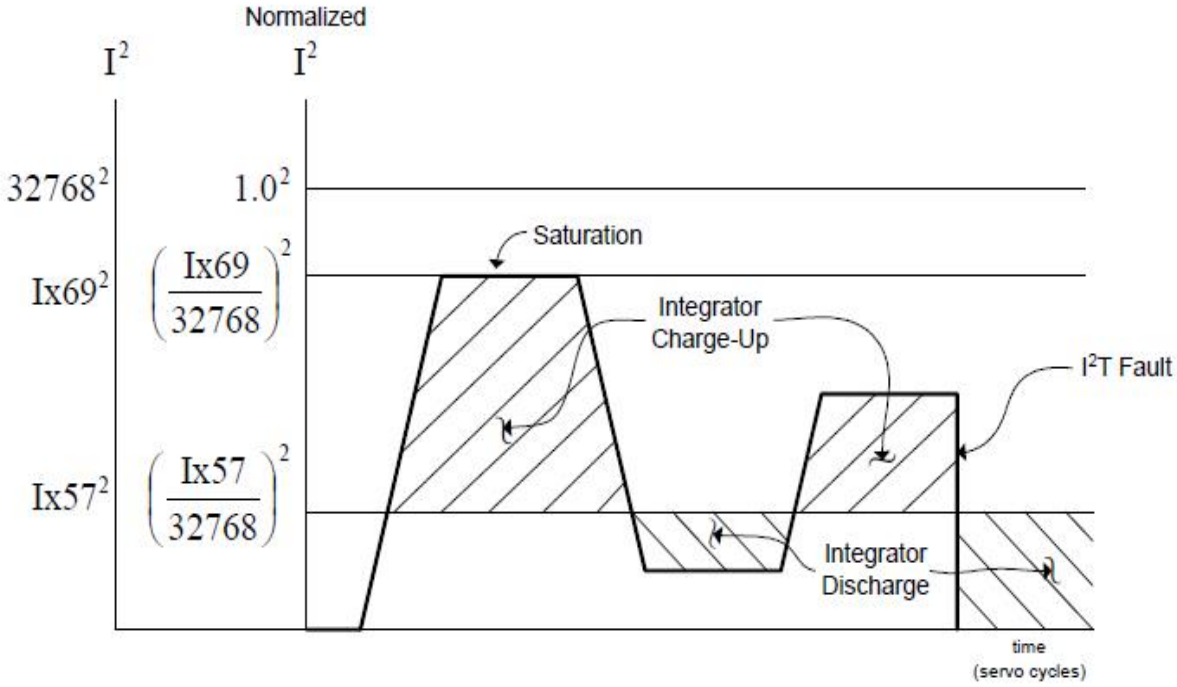
I_q （正交电流）是 PID 滤波器的产生扭矩的指令输出，单位是 16 位 DAC；

I_d （直流）是由 Ixx77 设置的磁化电流指令。通常是零，除非 Turbo PMAC 正在进行感应电机的矢量控制。

Δt 是自伺服周期中最后一个采样后的时间

如果 Sum（总和）超过 Ixx58，将发生积分电流故障。当指令电流电平低于 Ixx57，Sum（总和）会减少，但它永远不会低于零。

PMAC I^2T 保护功能



例如：有了命令输出限制 Ixx69=32767（最大）、积分电流限制 Ixx57=16384（最大值的一半）以及磁化电流 Ixx77=0，电机会有障碍，而且指令输出饱和在 32767。积分电流保护功能将在此时间内计算：

$$\text{sum} = \text{sum} + [1^2 + 0^2 - 0.5^2]\Delta t = \text{sum} + 0.75\Delta t$$

sum(总和)将以每个伺服周期 0.75 的速率增加。在 2.25 kHz 的缺省伺服周期的更新速度下，总和将以每秒 2250*0.75=1688 的速度增加。如果你想使电机在这种情况的 3 秒后跳闸，应该将 Ixx58 设置为 1688*3=5064。

当一个电机发生积分的电流故障时，Turbo PMAC 的反应就像是有一个放大器故障错误。违规电机被停机，有可能其他电机根据 Ixx24 的 21 位和 22 位的设置被停机。Turbo PMAC 将放大器故障电机状态位进行置位。对于一个积分电流故障，Turbo PMAC 还设置了一个单独的积分电流故障电机状态位。当电机被重新启用时这两个位都清零。

注意：

当 Turbo PMAC 不对I^2T保护的电机换相时，确保磁化电流参数 Ixx77 仍设置为 0。在这种设置中，Ixx77 不会影响操作，但会影响积分电流计算。

12-7 放大器启用和故障线

对每个电机使用放大器使能（AENAn）输出和放大器故障（FAULTn）输入线对于安全操作很重要。不使用使能线时，若要禁用放大器，则依赖于 Turbo PMAC 的模拟输出和放大器的模拟输入中的精确的零偏置。

不使用故障线，Turbo PMAC 不知道一个放大器何时已关闭，并不采取适当的动作。

注意：

对于放大器使能信号默认的下沉式驱动程序，使用低电平有效的使能极性（低电压--导通--被使能；高电压--不导通--被禁用）为电源的丢失提供了更好的故障安全保护。如果 Turbo PMAC 的计算部分的+5 V 电源或+15 V 模拟电源丢失，放大器将被自动禁用，因为输出晶体管将进入其非导通状态。如果你需要这样的故障安全保护，但又不能将此极性的信号直接连接到放大器，您必须使用中间电路来改变信号格式。有了备用的源化驱动器，高-真使能极性提供了更好的故障安全保护。

12-8 编码器丢失检测

大多数的 Turbo PMAC 控制器对每个编码器输入具有编码器丢失检测电路。设计用于差分线路驱动器输出的编码器，该电路用异或门（XOR）监控每个输入对。如果编码器工作正常并连接到 Turbo PMAC，该对的两个输入应该是相反的逻辑状态----一个高而另一个低---从异或门产生一个真值的输出。

如果已经为编码器配置了输入电路，从而使该对的两条线都有上拉电阻（这不是默认的---无论是一个 E-点跳线必须改变或一个 SIP 电阻包反转在其插座），然后编码器或电缆出现故障会引起两个输入为同一个逻辑状态，从异或门产生一个假值的输出，并设置一个错误状态位。注意，在这种设置中，单端编码器不能用在通道上。

下表显示了每个通道的电阻包用于有此功能的 Turbo PMAC 和配件。要启用编码器的丢失功能，电阻包的引脚 1（标记是包装上的一个圆点）应该被放置在插座的管脚 1 的相对的一端（标记是电路板上的白墨水正方形）。对于 4 通道的附件（Acc-24E2x 和 24C2x），所示的前 4 个通道用于定址为偶数伺服 IC（2、4、6 或 8）的配件；所示的第二组 4 个通道是用于定址为奇数伺服 IC（3、5、7 或 9）的配件。

编码器信号丢失电路的电阻包

设备	Ch. 1	Ch. 2	Ch. 3	Ch. 4	Ch. 5/1	Ch. 6/2	Ch. 7/3	Ch. 8/4
PMAC-PCI	RP60	RP62	RP66	RP68	RP97	RP99	RP103	RP105
PMAC2-PCI	RP43	RP48	RP44	RP49	RP104	RP109	RP105	RP110
QMAC	RP55	RP57	RP66	RP68	-	-	-	-
Acc-24P2	RP74	RP75	RP85	RP86	RP139	RP140	RP150	RP151
Acc-24E2	RP22	RP24	RP22*	RP24*	RP22	RP24	RP22*	RP24*
Acc-24E2A	RP22	RP24	RP22*	RP24*	RP22	RP24	RP22*	RP24*
Acc-24E2S	RP19	RP21	RP27	RP29	RP19	RP21	RP27	RP29
Acc-24C2A	RP33	RP34	RP63	RP64	RP33	RP34	RP63	RP64
*在附件模块的子板上								

下面的表显示了每个装置的每个通道的编码器丢失状态位的地址。该地址始终用于 Y 寄存器。在某些地址中所示的 x 表示十六进制数字 8、9、A 或 B，与卡本身的基地址相同。该位值为 1 是有效编码器信号；0 来表示编码器信号丢失。

设备	Ch. 1	Ch. 2	Ch. 3	Ch. 4	Ch. 5/1	Ch. 6/2	Ch. 7/3	Ch. 8/4
PMAC-PCI	$70801,1	$70801,2	$70801,3	$70801,4	$70801,5	$70801,6	$70801,6	$70801,7
PMAC2-PCI	$78403,8	$78403,9	$78403,10	$78403,11	$78403,8	$78403,9	$78403,10	$78403,11
QMAC	$78403,8	$78403,9	$78403,10	$78403,11	-	-	-	-
Acc-24P2	$7xF00,0	$7xF00,1	$7xF00,2	$7xF00,3	$7xF00,4	$7xF00,5	$7xF00,6	$7xF00,7
Acc-24E2	$7xF08,5	$7xF09,5	$7xF0A,5	$7xF0B,5	$7xF0C,5	$7xF0D,5	$7xF0E,5	$7xF0F,5
Acc-24E2A	$7xF08,5	$7xF09,5	$7xF0A,5	$7xF0B,5	$7xF0C,5	$7xF0D,5	$7xF0E,5	$7xF0F,5
Acc-24E2S	$7xF08,5	$7xF09,5	$7xF0A,5	$7xF0B,5	$7xF0C,5	$7xF0D,5	$7xF0E,5	$7xF0F,5
Acc-24C2A	$7xF08,5	$7xF09,5	$7xF0A,5	$7xF0B,5	$7xF0C,5	$7xF0D,5	$7xF0E,5	$7xF0F,5

截至撰写本文时，检测到编码器信号丢失时不采取任何的自动动作。用户要想要对检测到编码器信号丢失采取行动，应该写一个 PLC 程序来寻找编码器丢失位的变化，并采取适当的行动。一般来说，唯一适当的反应是停止（开环、零输出、禁用）丢失编码器反馈的电机；其他电机也可能被停机或中止。

下面的示例显示，在一个 Turbo PMAC 上，作为电机 1 的反馈的编码器 1 信号，一旦检测到该信号丢失，如何停止所有电机。

```
#define    Mtr1OpenLoop        M138
#define    Enc1LossIn          M180
#define    Mtr1EncLossStatus   P180
#define    Lost                 0        ; Low-true fault
#define    OK                   1
Mtr1OpenLoop->Y:$0000B0,18,1             ; Standard definition
Enc1LossIn->Y:$078403,8,1                ; CTRL0 input
OPEN    PLC    18    CLEAR
; Logic to disable and set fault status
IF (Mtr1OpenLoop=0 AND Enc1LossIn=Lost)      ; Closed loop, no enc
        CMD^K
        Mtr1EncLossStatus=1
ENDIF
; Logic to clear fault status
```

```
IF (Mtr1OpenLoop=0 AND Enc1LossIn=OK    AND Mtr1EncLossStatus=0)
        Mtr1EncLossStatus=0
ENDIF
CLOSE
```

请参阅单独的硬件参考手册，获取这个功能实现的更多细节。

12-9 用户编写的安全算法

您可以在 PLC 程序中很容易编写自己的安全校验算法。这些算法最好实现在在一个编译的后台 PLC 程序，它扫描的速度与 Turbo PMAC 自身内置的安全检查一样---每个后台周期一次。上面的监控编码器信号丢失的程序是这种类型程序的很好的例子。

12-10 看门狗定时器

Turbo PMAC 有一个板载的看门狗定时器。该子系统为软件和硬件故障提供了故障安全关闭。为了阻止跳闸硬件电路，看门狗定时器需要两个基本条件得到满足。首先，它必须看到一个大于约 4.75V 的直流电压值。如果电源电压低于此值时，电路的继电器将跳闸，该卡将关闭。这可以防止由于电压不足导致寄存器的损坏。

第二个必要条件是，该定时器必须看一个频率约大于 25 赫兹的方波输入（由 Turbo PMAC 软件提供）。在前台，伺服中断程序递减计数器（只要计数器大于零）时，引起计时器的最低有效位进行触发。这个位被馈送到定时器本身。在每个后台周期结束时，CPU 将计数器值复位到由变量 I40 设定的最大值（或到 4096，如果 I40 被设置为默认值 0）。

如果卡，不管是什么原因，是由于硬件或软件问题，不能以 25 Hz 或更高的频率反复设置和清除该位，定时器将跳闸而 Turbo PMAC 系统将关闭。

看门狗定时器跳闸时的动作

当定时器由于欠电压或欠频率而跳闸，系统被锁存到复位状态，一个红色的 LED 指示看门狗故障。该处理器停止运行而且无法通信。所有伺服、MACRO 和 I/O 芯片被迫进入它们的复位状态，这迫使离散输出关闭、比例输出（DAC、PWM、PFM）为零电平。

Turbo PMAC 系统具有离散输出，指示看门狗定时器的状态。在 Turbo PMAC 板上，有一个固态集电极开路输出所谓的“FEFCO/”， 在定时器跳闸时开启。在 Turbo PMAC2 系统中有一个硬触点继电器有常开和常闭触点。在一个系统中，如果该卡故障，这些输出应该是用于降压电源到放大器和其他关键电路。

一旦看门狗定时器跳闸， Turbo PMAC 的电源必须循环关闭、打开，或 INIT/硬件复位线必须采取低电平，再高电平，以恢复正常运作。

诊断看门狗定时器跳闸

由于看门狗定时器的目的是在各种硬件和软件故障时跳闸，并且跳闸使其无法查询卡，所以可以很难确定跳闸的原因。建议通过下面的过程找出原因：

1、正常重置 Turbo PMAC（重新初始化跳线断开）。如果它不立即重新跳闸，有间歇性的软件或硬件问题。检查以下内容：
   - 有时候软件事件超载处理器（例如额外的伺服中断任务，频繁的前瞻）或可能的错误指令（查找固件或程序校验和）。查阅本手册的《评估 Turbo PMAC 的计算负载》部分。
   - 5V 电源的干扰
   - 连接松动

2、如果在步骤 1 中有一个即时的看门狗定时器跳闸，重新初始化跳线为 ON 并上电。如果现在不跳闸了，板上存在一个伺服/相位的任务加载频率问题，或一个即时软件问题。检查以下内容：
   - 相位和伺服时钟频率以及 Turbo PMAC 使用的电机数量。这些频率可能需要被减少。
   - 一个上电时立即运行的 PLC0 或 PLCC0 程序（I5 保存为 1 或 3），占用了太多的时间。
   - 用户编写的伺服或相位程序未正确返回。

3、如果在步骤 2 中存在即时的看门狗定时器跳闸，检查硬件问题：
   - 正确设置时钟频率跳线(Turbo PMAC)
   - 外部伺服和相位时钟的跳线设置，但没有提供
   - 断开所有配件并反复查看是否是他们造成的问题
   - 检查充足的 5V 电源电平（检测 Turbo PMAC 的 CPU，而不是电源）
   - 检查硬件损坏

4、如果在步骤 3 中没有发现问题，固件重载（引导）跳线为 ON，并上电。如果这里看门狗定时器未跳闸并且你可以做基本的通信(???应该引起一个 BOOTSTRAP ROM 应答)，那么你的操作固件有问题，它必须重新加载。

12-11 硬件停止指令输入

Turbo PMAC 控制器有硬件的输入，可以通过用户设置的减速来停止一个移动或程序。中止（Abort）输入会停止所选择的坐标系统中所有轴的运动，由电动机/系统选择输入来确定，立即执行，并且每个电机减速的速度由 Ix15 设定。保持（Hold）输入会执行相同的功能，不同之处在于轴减速相互协调，使得在减速过程中期望的多轴路径会不变。

这些专用的输入是在 Turbo PMAC 的控制面板连接器上（JPAN；J2）。它们作用在哪个坐标系上是通过二进制数确定的(二进制数由四个低电平有效的输入线 FPD0/（LSBit)、FPD1/、FPD2/和 FPD3/（MSBit）产生)。零值（全为高）禁用该功能；1 到 8 的值选择编号的坐标系。

12-12 主机产生的停止指令

这些功能和其他几个也可以从主机上进行，通过一个或两个字符的命令。例如，<CTRL-A>执行相同的功能，与对选定的所有坐标系执行中止（Abort）输入是一样的，而 A 会中止软件定址的坐标系。<CTRL-O>保持所有坐标系，而 H 保持软件定址的坐标系。此外<CTRL-Q>停止预定移动末端的所有的程序，而 Q 停止软件定址的坐标系的程序。<CTRL-K>立即停用所有的电机，而 K 则禁用软件寻址的电机（如果电机是位于一个运行着运动程序的坐标系中，一个 Abort（中止）命令应该在 K 命令之前发出。

这些命令的任何一个可以从一个 Turbo PMAC 程序内发出，使用 **COMMAND "{command}"** 或 **COMMAND^{letter}**语法。然而，当从该坐标系统中运行的一个运动程序内发出一个电动机停机（K）的命令时，该命令将被自动地拒绝。

下表总结了用于停止运动的不同命令及其属性：

指令	范围	立即开始？	停止在路径？	停止在编程路径？	减速速率	注意
J/	电机	是	–	否	Ixx19-21	
A	坐标系	是	否	否	Ixx15	无简单的重启
<CTRL-A>	全局	否	否	否	Ixx15	无简单的重启
Q	坐标系	否	是	是	TA, TS	完成计算的移动
<CTRL-Q>	全局	否	是	是	TA, TS	完成计算的移动
/	坐标系	是	是	是	TA, TS	当前执行移动的结束*，撤消混合
H	坐标系	是	是	否	Isx95	类似%0
<CTRL-O>	全局	是	是	否	Isx95	类似%0
****	坐标系	是	是	否	Ixx17/Isx95	如果在前瞻之外作为 H 命令
K	电机	是	否	否	---	必须首先停止程序
<CTRL-K>	全局	是	否	否	---	必须首先停止程序

12-13 程序校验和

固件校验和

Turbo PMAC 不断作为后台任务来计算其内部程序（固件）的校验和。每一次它计算完校验和，它将这个值与内存中的参考寄存器已手动输入的正确的值进行比较（通过 **CHECKSUM** 的命令获得）。Turbo PMAC 在出厂时预装了固件版本的正确参考值。

如果 Turbo PMAC 检测到在其计算的校验和与参考校验和之间不匹配，它会设置全局状态位（X:$000006 的 12 和 13 位---用???命令访问），并停止执行任何的校验和操作。这使得计算值冻结在运行的校验和寄存器 X：$001080 之中。Turbo PMAC 在固件校验和出错的情况下确实采取任何其他行动；它是由主机或 Turbo PMAC PLC 程序来决定采取什么行动。

当 Turbo PMAC 升级到新固件，是通过更换在标准 CPU 部分的 PROM 芯片或下载新固件到 Option CPU 部分的闪存 EEPROM 芯片中，参考校验和的值将自动为新的固件更新。

用户程序校验和

Turbo PMAC 不断地计算固定用户程序的缓冲区的校验和作为后台任务。每一次它计算完校验和，它将这个值与其中一个缓冲区上次被封闭时计算的、存储在 X:$001090 的校验和的值进行比较，。

如果 Turbo PMAC 检测到这两个校验和不匹配，它设置一个全局状态位（X:$000006 的 13 位---使用？？？访问），并停止执行任何程序或固件的校验和操作（通信校验和是独立的），冻结运行的校验和值在 X:$001080 中。它不会自动关闭操作。如果有一个校验和出错，由主机或 Turbo PMAC PLC 程序来决定采取什么行动。

12-14 通信数据完整性

Turbo PMAC 提供了多种技术来保证数据的有效传输，包括串行和总线的串口奇偶校验、帧错误检测、串行全双工通信以及双向通讯校验和计算。有关这些技术是如何工作的，更多详情请参阅本手册的《编写主机通讯程序》部分。

第十三章 执行电机的基本移动

一旦电动机被定义并基本上工作在合理的增益，对电动机命令一些基本的动作。手动（点动/慢跑）命令用于进行电机的简单的动作，独立于其他电机，无需编写运动程序。使用这些运动用于开发、诊断和调试，但他们也可能在实际的应用中使用。

另一种简单的电机移动是回零搜寻移动。这基本上是一个微动-直到触发类型的移动，其中 Turbo PMAC 命令电机移动直到它看到一个预先定义的触发。然后，它使得电机停止并返回到触发位置（可能有一个偏移量），并且将电动机位置设置为零。

当不知道起始位置在何处时应该执行一个回零搜寻移动。如果有一个增量位置传感器，并且不知道上电时的位置；因此，在这种类型的系统中，通常回零搜寻移动是首先进行的移动。但是，如果起始位置已知，而只是想回到那个位置，没有必要做一个回零搜寻移动，只需命令一个移动到零位（例如：J=0 或 X0）

手动和回零运动的轨迹与快速模式程序移动是一样的。然而，这些移动都是直接指定给电机，电机通过数字指定，而不是指定给（字母指定的）轴。这些运动是以未缩放的单位描述的（所有的都基于计数和毫秒）。

13-1 手动移动控制

手动运动的速度和加速度是由点动电动机的 I-变量控制的。目的地是通过手动命令控制。这些将在下面说明。

手动加速度

电机 xx 的变量 Ixx20 指定手动、回零、编程的快速模式移动的加速时间，以毫秒为单位。Ixx21 指定这些运动的 S 曲线加速度分布图的每一半的时间，也以毫秒为单位。如果 Ixx20 小于 2 倍 Ixx21，所用的加速时间将是两倍的 Ixx21。

手动/回零/快速移动的加速速率极限是由 Ixx19 设置（单位是计数/毫秒 ^{2}）。如果 Ixx20 和 Ixx21 太小以至于超过了 Ixx19，那么加速时间被延长使 Ixx19 不被超过。要指定加速度速率而不是加速时间，将加速时间参数设置的足够小，而总是使用极限加速速率参数。

注意：

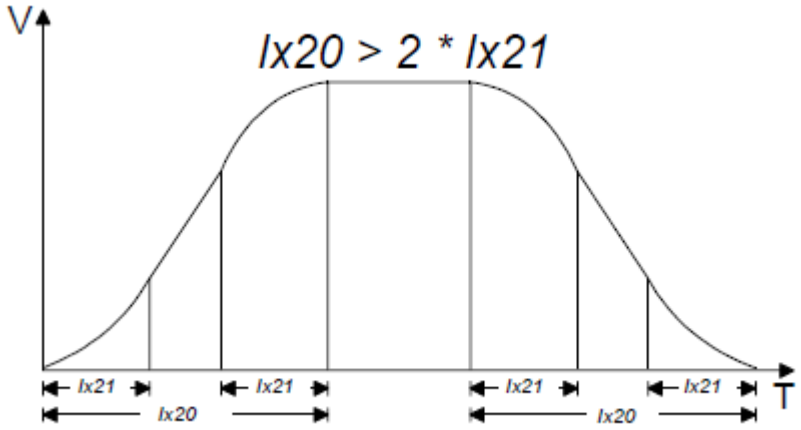
要指定加速率，不要将加速时间参数 Ixx20 和 Ixx21 都设置为零。这将导致在移动计算中出现一个除以零的错误，可能导致不稳定的运动。最小加速时间设置应是 Ixx20=1 和 Ixx21=0。

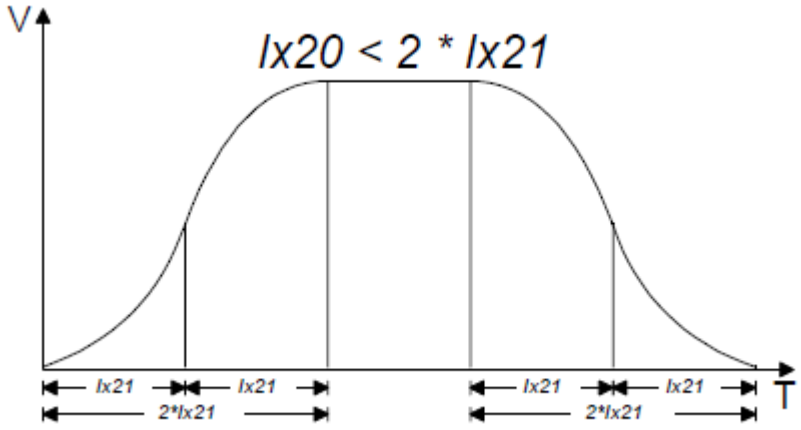
手动速度

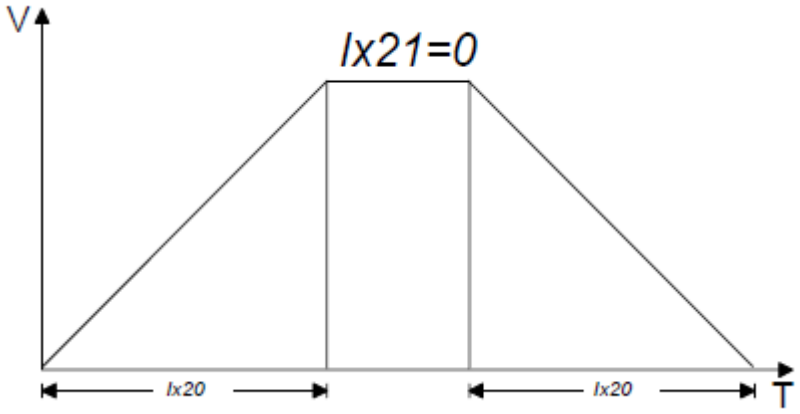
手动速度是由 Ixx22 指定，这是速度的大小，单位是每毫秒的计数。方向由手动命令本身指定。

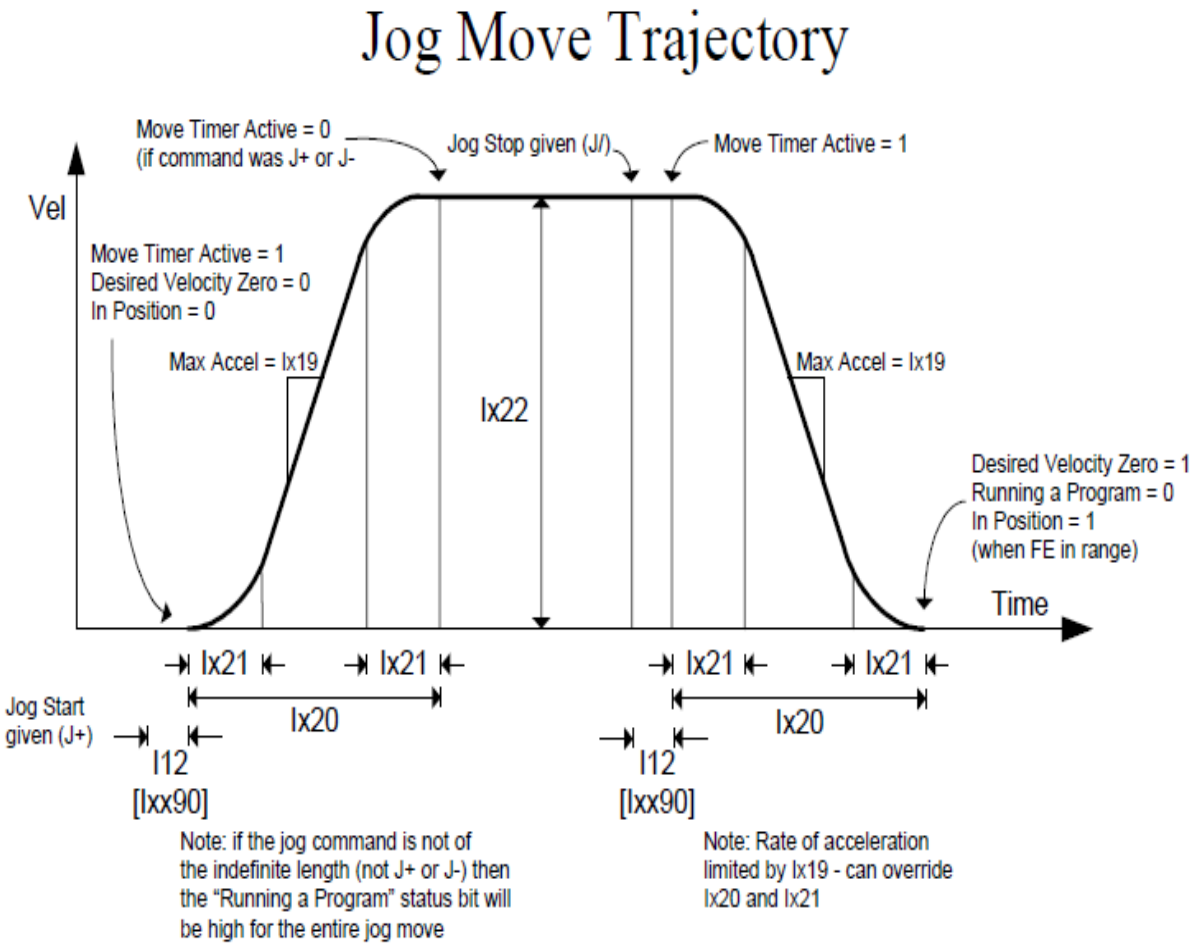
电机 x 运动变量

Ix20 加速时间（JOG，HOME）
（单位：毫秒）；整数
Ix21 S 曲线时间（JOG，HOME）
（单位：毫秒）；整数









手动命令

手动控制电机的命令是在线（即时）命令，是电机专用的，它们作用在当前的定址电机上。

注意：

如果电机是位于当前正在执行运动程序的坐标系统中，电动机的手动命令将被拒绝，即使运动程序并没有命令那个电机移动。如果 I6 被设置为 1 或 3 ，PMAC 将会报告 ERR001。

不确定的手动命令

J+命令对于定址电机是一个不确定的正向手动；J-命令是一个不确定的负向手动；J/命令结束手动，而使电机减速之后留在位置控制。J/命令有可能保留指令位置在一个分数计数处，这可能导致在相邻的整数计数值之间抖动。如果这是一个问题，J!命令可以被用来强制指令位置到最近的整数计数值。（或者，Ixx64 和 Ixx65 创建的死区的 1/2-计数能防止在分数计数值处抖动）。

手动控制到指定位置

可给出手动指令控制指定的位置，或者指定的距离。J=命令手动到上次的预定手动位置；J={**常数**}命令慢跑到命令中指定（未缩放）的位置；J= ={**常量**}命令慢跑到命令指定（未缩放）的位置，并使该位置为预手动位置；J^ {**常量**}命令慢跑到距发送命令时的实际位置的指定的距离处（J^0 可用于接收剩余的跟随误差）；J:{**常数**}命令慢跑到相对发送命令时的指令位置的指定距离处。

变量指定的手动移动

慢跑移动到一个由变量指定的距离或位置是可能的。每个电机都有一个特定的寄存器(L:$0000D7 用于电机 1,L:$000157 用于电机 2 等,建议的 M 变量 Mxx72）它保存下一个变量手动命令要移动的位置或距离。该寄存器包含一个浮点值，缩放为编码器计数。应该通过一个 L 格式（浮点）M 变量访问它。J=*命令使 PMAC 使用这个值作为目标位置。J^*命令使 PMAC 使用该值作为相对发送命令时的实际位置的一个距离。J:*命令使 PMAC 使用该值作为相对命令发送时的指令位置的一个距离。

在微动移动时发出命令

允许对一个已经慢跑的电机发出一个手动命令。在收到新的命令时，Turbo PMAC 将闯入已经计划的轨迹并创建一个平滑混合到新命令的轨迹中，由目前的加速度和速度的命令决定。现有的轨迹收到新的命令后延伸出 Ixx92 毫秒（默认 10 毫秒),在这段时间计算新的轨迹。计算新移动必须能够在 Ixx92 毫秒内完成。

手动命令使用的参数

每次给出这些命令之一时，那时的加速度和速度参数控制对命令的响应。要改变一个有效手动移动的速度或加速度参数，更改相应的参数，然后发出另一个手动命令。

13-2 触发的电机移动

Turbo PMAC 的触发的动作是两个移动，一个预触发部分和一个后触发部分。遇到触发事件时，Turbo PMAC 将闯入预触发移动中并计算一个后触发移动，在相对触发点的预先指定距离处结束。

触发的移动的类型

有三种类型的触发电机运动：
1、回零搜寻运动（在线或运动程序）
2、在线的手动--直到触发的运动
3、运动程序快速移动--直到触发
这些动作基本上都以相同的方式工作，只是在不同的背景下。触发和位置捕捉功能在所有三种类型的运动中是相同的。每个将在下面进行详细说明。

触发条件的类型

对于这些触发运动，基本上有两种类型的触发器：输入触发器和跟随误差触发器。电机变量 Ixx97 的 Bit 1 确定哪种用于电机的触发运动。

输入触发

如果 Ixx97 的 Bit 1 是 0（Ixx97=0 或 1），则电机的触发条件是一个输入触发。在此，缺省情况下，该触发是由映射到由 Ixx25 指定地址的寄存器的一个或两个输入引起的。通常情况下，Ixx25 包含一个伺服 IC 的标志寄存器的地址，或一个含有从 MACRO 环复制的标志数据的、内存中的寄存器的地址。对于超精密硬件捕获（见下文），Ixx25 必须指定与用于位置反馈环的编码器相同的硬件通道的标志。如果 Ixx24 的 18 位设置为 1，Turbo PMAC 希望捕捉触发和位置来自 MACRO 环上；如果 Ixx24 的 18 位是 0，Turbo PMAC 希望这些来自直接连到控制器 CPU 的硬件通道中。

触发信号和边缘

伺服 IC 通道的两个设置变量确定哪些信号的哪些边缘导致触发。(用于伺服 IC m 通道 n 的）I7mn2（在 MACRO 站上是节点特定变量 MI912）指定是否使用索引通道，是否使用输入标志，以及索引和/或标志的哪个边沿会引起触发。如果索引和标志都被选中，这两个信号与芯片内的布尔硬件逻辑组合以产生触发信号。

如果 I7mn2 指定使用一个标志，I7mn3（MACRO 站上是节点特定变量 MI913）选择使用通道的哪四个输入标志。对于一个 PMAC 式伺服 IC，选用的是 **HMFL**、**+LIM**、**-LIM** 和 **FAULT**；对于一个 PMAC2 式伺服 IC，选用的是 **HOME**、**MLIM**、**PLIM** 和 **USER**。

边沿与电平触发

在 PMAC 式和 PMAC2 式伺服 IC 之间有细微的差别，如果触发移动开始时输入已经在触发状态，会导致不同的故障模式（其他的没有差别）。PMAC 式伺服 IC 是边沿触发，需要一个从非触发状态到触发状态的阶跃过渡（边沿）以产生实际触发。有了这些芯片，在这种情况下就不会产生触发，而且预触发移动将继续下去，也许无限期。

然而，PMAC2 式伺服器 IC 是电平触发，只需要 IC 看到输入在触发状态，以生成实际的触发。有了这些芯片，在这种情况下触发会立即产生，而且会有很少的或根本没有运动。

双触发的优点

在回零搜索移动中，通常的做法是使用一个回零开关和索引通道的组合作为回零触发条件。编码器的索引通道，虽然是精确的和可重复的，但是在大多数应

用中不是唯一的，因为电动机能够转动超过一周。而回零开关，虽然独特，通常不是非常精确和可重复的。通过两者的逻辑组合，可以从开关获得唯一性，并从索引通道获得精确度和可重复性。在这个方案中，回零开关用于选择哪个索引通道脉冲被用作回零触发。

虽然在这种类型的应用中回零开关不需要进行非常精确地放置，但重要的是它的触发边沿安全地保持在相同的两个通道的索引脉冲之间。此外，回零开关脉冲必须足够宽，始终包含至少一个索引通道脉冲。

跟随误差触发

有时当遇到一个障碍时如硬停止，如使用一个端部挡块用于回原点回零参考时，希望发生一个触发。为了支持这种类型的功能， Turbo PMAC 允许触发发生在一个警告跟随错误条件下，而不是一个输入标志下。这有时被称为“扭矩模式”触发，因为它在一定扭矩数量上才有效地触发（除了速度模式放大器），因为输出转矩指令正比于跟随误差。它也被称为“扭矩限制”模式，因为它提供了一种简单的方法来创建（限制转矩的、并且达到转矩限制时就停止的）移动，（如限定扭矩的螺杆驱动）。

要启用此扭矩模式触发用于电机 xx，Ixx97 设置为 3，同时指定跟随误差触发和软件捕获（没有硬件信号来创建一个硬件捕获）。在这种模式下，对于移动-直到触发的触发是警告跟随误差状态位的一个真状态。变量 Ixx12 设置电机的警告跟随误差的幅值大小，单位为一个计数的 1/16。当 Turbo PMAC 检测到跟随误差的幅值已经超过了这个值时，它会读取当前的反馈位置作为触发位置，然后相对这个位置移动。

当使用转矩模式触发，一个好主意是将积分增益项 Ixx33 设置为 0，以防止当遇到一个硬停止时积分器会大量“充电”。当遇到阻塞时，将 Ixx69 输出限制设置的低一点以直接限制可能的扭矩也是可取的。

请注意，如果在移动开始时，警告的跟随误差状态位为真，触发几乎立即发生。

捕获触发位置

因为后触发移动会结束在一个指令位置，表示为相对于在触发时的位置，Turbo PMAC“捕获” 触发时的位置是必要的。从根本上说，这样做的方式有两种：硬件捕获和软件捕获。

硬件捕获

一个 Turbo PMAC 的伺服芯片有专门的寄存器，在收到预先指定的输入触发状态时，立刻锁存编码器计数器。锁存的动作完全发生在 IC 硬件，无需软件操作，所以捕获的位置精确到精确的计数，无论电机的转速是多少。这意味着，没有必要减慢移动得到一个准确的捕获。

选择硬件捕获用于电机 xx 的触发动作是通过将 Ixx97 设置为默认值 0，同时指定硬件捕捉和输入触发。如果选择硬件捕获，Ixx03 选择的位置环反馈一定要来通过一个伺服芯片的编码器计数器。它必须使用与 Ixx25 选择的标志集是相同的硬件通道。这意味着，如果你在电机上使用双反馈，Ixx25 指定的标志集应该与您的位置环反馈是相同的通道，不是你的速度环反馈。

硬件捕获与 Acc-51 插补器

通过一个 Acc- 51 高分辨率插值器采用正弦编码器反馈，对触发移动使用硬

件捕获功能，必须激活一些附加的固件功能（介绍在固件版本 V1.940 中）。因为捕获标志必须与位置反馈回路是相同的伺服 IC 和通道，Ixx25 必须被设置为插补板通道的地址。然而，插值器板不提供放大器的启用和故障标志，或超行程限位标志，所以变量 Ixx42 和 Ixx43 分别用于为这些标志指定不同的地址（大概在轴卡上）。

接着，因为 Acc-51 对伺服 IC 的硬件计数器的每个增量产生 10 位的小数计数数据，而不是通常的 5 位，Ixx24 的 11 位必须被设置为 1 用于捕获的硬件计数值正确缩放，当转换为电机位置时。

最后，从（用在 Acc-51C 和 ACC-51C 插补板卡的）PMAC2 式“DSPGATE1”伺服 IC 的修订版本 D（于 2002 年年初推出）开始，硬件捕获的子计数分辨率成为可能。请注意，这种能力对于插补器板的硬件捕获是不需要的，并且不能用于回零。要对伺服 IC 中的通道激活此功能，变量 I7m n9 必须被设置为 1。要在电机 xx 上使用这个分数数据用于移动-直到触发，Ixx24 的 12 位必须设置为 1。

软件捕获

如果因为某些原因对触发位置使用硬件捕获是不可能的，则软件捕获是可能的。对电机 xx 指定软件捕获是通过设置 Ixx97 为 1(输入触发)或 3(错误触发)。如果已选软件捕获，当软件注意到触发已经发生，Turbo PMAC 软件采用最新的伺服周期的电机实际位置作为触发位置，不论其来源。

当使用软件捕获时，实际的触发和 Turbo PMAC 的位置捕获之间有一个后台周期的潜在延迟，可能是几毫秒。这种延迟可能会导致捕捉位置不准确，在触发时电机的速度必须保持足够低，以实现足够准确的捕捉。对于回零，可使用一个两步的程序：一个快速、不准确的捕获，之后是一个缓慢、准确地捕捉。

后触发移动

检测到触发时，Turbo PMAC 将闯入前触发移动的轨迹并创建一个平滑混合到后触发的移动轨迹中，由当前加速度和速度的命令确定。后触发移动结束在捕捉位置的预定的距离处---指定这个距离的方法取决于运动类型（见下面的每个部分）。检测到触发之后，现有的轨迹延伸出 Ixx92 毫秒（默认 10 毫秒），在这段时间内计算新的轨迹。后触发移动的计算必须能够在 Ixx92 毫秒内完成。

当发现触发时，从先触发运动融合到后触发移动、以及从后触发移动到停止，将使用加速度参数 Ixx19、Ixx20 和 Ixx21。如果后触发移动足够长能够达到这个速度，那么后触发的移动速度大小与前触发运动的一样。通常情况下，后触发移动将涉及预先触发移动的反转，但这并不一定如此。

回零搜索移动

回零搜索移动的目的是，当使用增量位置反馈传感器时，确定一个绝对位置基准。移动--直到触发的结构是理想的，用于发现（确立起始位置的）传感器，并自动返回到这个位置。

回零加速度

回零搜索移动的加速度是与手动运动相同的参数控制---Ixx19（最大加速

度)、Ixx20（加速时间）和 Ixx21（S 曲线时间)。这些都描述在上述手动控制移动的部分中。

回零速度

Ixx23 指定回零搜索移动的速度和方向。如果 Ixx23 大于零，预先触发的回零搜索运动将是正方向。如果它小于零，先触发移动将在负方向。Ixx23 的大小（单位：计数/毫秒）同时控制先触发和后触发运动的速度（它们应该足够长，能够达到该速度)。

回零触发条件

回零搜索运动的触发条件，如同其他的触发运动，是由 Ixx97、Ixx24 和 Ixx25 指定的，如上文面详细描述的。如果未找到触发，前触发移动会一直继续下去，或直到由一个错误条件停止，如到达了超程限制。

后触发移动

变量 Ixx26 指定从触发捕获的位置到后触发运动结束时之间的（有符号）距离。 Ixx26 的单位是 1/16 计数。指令后触发移动的终端是新的电机零位置（电机的“‘home ”位置)。电机的报告位置参考的变化发生在后触发移动的开始。只要完成了这一点，报告的位置都参考这个新的零位置（在轴定义语句中加上或减去任意轴的偏移---如果轴定义为**#1->10000X+3000**，该起始位置将被报告为 3000 计数)。此外，在这一点上，电机的“home search in progress”状态位被清除，而“home complete”状态位被置位。

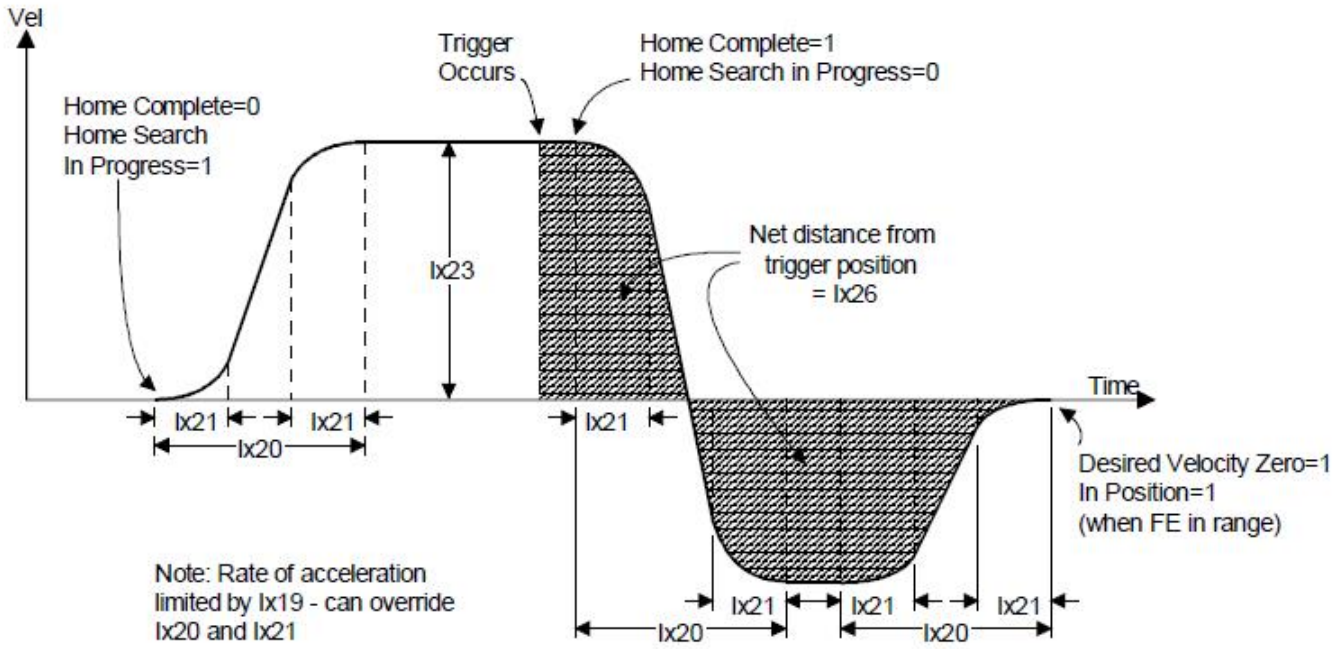
如果后触发移动因为一个错误条件而失败，电机没有必要重新回零，因为起始位置是已知的。一旦问题的根源已被清除，一个如 **J=0** 的命令可以将电机移动到起始位置。

如果使用了软件超程限制（Ixx13、Ixx14 不等于零)，在寻找触发时被自动禁用后，此时它们被重新启用。然后计算到这个新的零位置的轨迹，如果必要，包括减速和反转。需要注意的是，如果一个软件极限过于接近零，电机可能在到达限位之前无法停止和反转。在正常终止时，电动机将在位置控制之下停止，它的命令位置等于起始位置。如果有一个跟随误差，实际位置将是不同的，偏移了跟随误差的量。

未能找到触发

如果不能找到正在寻找的触发条件，一个回零搜索的先触发移动将无限期地继续下去。通常在这种情况下，可以通过行程末端的超程限位开关或故障跟随误差限位来停止它。如果你想对先触发移动的距离有一个编程限制，你应该使用一个增量的手动-直到-触发的命令或编程的移动-直到-触发，第一个值指定没有触发下的移动的距离，第二个指定从触发到后触发移动末端的距离（取代 Ixx26)。一旦后触发移动完成，可以用一个 **HOMEZ** 命令（见下文）来设置该位置为电机的零位置。

回零搜索移动轨迹



Home 命令

执行回零搜索移动既可以通过一个在线命令（可使用 **COMMAND“ ”**语法从 PLC 程序中给出）或一个运动程序语句。

在线指令

一个回零搜索移动可以通过在线的电机特定的命令 **HOME**（短格式 **HM**）启动，例如**＃1HM**。这是一个简单的命令来启动回零搜索；Turbo PMAC 不提供自动的迹象表明移动已经完成，除非设置识别就-位（IPOS）中断。

监测完成：如果从主机或从 PLC 程序监控电机，看它是否已经完成了回零移动，最好查看“home complete”（回零完成）和“desired velocity zero”（期望速度零）电机状态位，用**?** 命令或 M 变量查看。在上电或复位时，回零完成位设置为零；它在回零搜索移动的开始也被设置到零，即使上一个回零搜索移动已成功完成。在电机停下来之前，一旦在回零搜索移动中发现了触发，它被设置为 1。

注意：

在移动过程中，“home search in progress”（回零搜索进行中）位是“home complete”（回零完成）位的反：找到触发前它是 1，触发后立即是 0。因此，监测也应该寻找预期速度零的状态位变为 1，这将表明后触发运动的结束。

监控错误：一个强大的监测算法也要寻找回零搜索移动可能结束于一个错误条件的可能性。这通常是一般错误监测的一部分，在任何时候都寻找超行程限制、故障跟随误差和放大器故障。如果在回零移动时确实发生了错误，进行区分是重要的，它是在触发之前发生，还是之后发生。如果错误发生在发现触发之后，Turbo

PMAC 知道起始零位在哪里，不需要重复回零搜索。一旦错误原因已经固定，电机可以简单地移动到起始零位置，通过一个如 J=0 的命令。

缓冲的运动程序命令

指令一个回零搜索移动也可以从运动的程序内部进行，通过 **HOMEn** 命令，其中 n 为电机号。请注意，此命令指定一个电机，不像其他运动程序命令指定一个轴移动。在一个运动程序中，Turbo PMAC 的自动程序序列化例程会监控运动的结束。当移动成功完成后，程序继续执行下一个命令。

多个回零移动可以一起启动，通过命令指定一列或范围的电机编号（例如 **HOME1,3** 或 **HOME2.. 6**）。执行进一步的程序需要等待所有这些电机都完成他们的回零运动。单独的回零命令，即使是在同一行上（例如 **HOME1 HOME2**）将按顺序被执行，在第二个开始之前要完成第一个。不可能从单个运动程序上部分执行重叠的回零移动。

注意：

小心注意在线命令和缓冲指令二者在语法上的差异。在线命令仅仅是 **HOME** 或 **HM**，它作用于当前的定址电机，因此电机号必须在命令前（如 **#1HM**）。而缓冲命令，电机号是命令的一部分，紧随在字母 **HOME** 或 **HM** 其后（如 HM1）。

从 PLC 程序中回零

Turbo PMAC PLC 程序可以命令回零搜索运动，通过给出在线命令语句 **COMMAND“”**（例如 **COMMAND“#1HM”**）。这些命令只是启动回零搜寻移动；如果需要，必须写入代码来监视完成。在特定的命令字符串中必须指定电机编号，或通过 **ADDRESS#n** 语句指定；如果没有这条语句，电机寻址在 PLC 程序中不是模态的。

运动程序与 PLC 程序回零

下表总结了使用运动程序和 PLC 程序回零之间的差异。

运动程序	PLC 程序
程序执行点停留在包含 Home 命令的那一行，直到回零移动完成。	PLC 不自动监视回零移动的开始和结束。
Home 命令可以与编程的轴运动相结合。	轴运动只能通过 JOG 命令来执行。
坐标系必须准备执行一个运动程序。	坐标系不需要做好准备运行运动程序。
只能对定义在运行着程序的坐标系中的电机进行回零。	可以对未定义在目前正在运行程序的坐标系中的任何电机回零。
电机可同时回零、或逐个回零，或两者的任意组合。所有一起启动的电机在启动下一个动作之前必须完成。	电机可以按任何顺序回零。这包括在一台电机回零移动时启动另一台电机。
运动程序必须由一个在线指令、PLC 程序或其他运动程序来启动。	PLC 可以通过一个在线命令、PLC 程序、另一个运动程序启动，或者在上电或复位时自动启动。

零移动回零

想要不命令任何的移动就声明当前位置为起始的零位置，使用 **HOMEZ**（在线）或 **HOMEZn**（运动程序）命令。这些很像 **HOME** 命令，但不同之处是他们立即采取当前指令位置作为起始的零位置。**HOMEZ** 命令不使用 Ixx26 偏移。这个无移动的回零是非常有用的，对于早期开发（真正的回零程序开发出之前），以及 Turbo PMAC 的自动回零程序不支持的专门回零例程。

注意：

如果在发送 **HOMEZ** 指令时存在一个跟随误差，**HOMEZ** 命令后的报告的实际位置并不精确为零，它将等于跟随误差的负值。

回零到限位开关

可以使用一个限位开关作为回零开关。但是，对于正常完成的移动，首先禁用限位开关的限位功能；如果不这样做，则限制功能将中止回零搜寻移动。即使这样，零位置也已被设置；可以用 J=0 的命令移动电机到零位置。

注意：

在 PMAC 式伺服通道上，限位开关的极性与很多人期望的相反。- LIMn 的输入应连接到限位开关行程的正端；+LIMn 的输入应连接到限位开关行程的负端。

要禁用开关的限位功能，将电机变量 Ixx24（位值$20000）的 Bit 17 设置为 1。使用零点偏移参数 Ixx26 将零点位置超出限位开关外是一个好主意，这样可以在回零搜寻移动后立即重新启用该限制，而不在限制之中。

下面的示例显示两个快速程序完成这种类型的回零。一种使用运动程序，另一个使用 PLC 程序。同样的功能也可以通过在线命令完成。

```
;********** Motion Program Set-up Variables (to be saved) *************
CLOSE
I123=-10                    ; Home speed 10 cts/msec negative
I124=$000000                ; PMAC-style flags, normal mode
I125=$78000                 ; Use Servo IC 0 Channel 1 flags for Motor 1
I126=32000                  ; Home offset of +2000 counts
                            ; (enough to take you out of the limit)
I7102=3                     ; Capture on rising flag and rising index
I7103=2                     ; Use +LIM1 as flag (negative end switch)

;********** Motion program to execute routine **********************
OPEN PROG 101 CLEAR
```

```
I124=$20000                    ; Disable +/-LIM as limits
HOME1                          ; Home #1 into limit and offset out of it
I124=$0                        ; Re-enable +/-LIM as limits
CLOSE                          ; End of program

;********** PLC Set-up Variables (to be saved) ************************
CLOSE
I123=-10                       ; Home speed 10 cts/msec negative
I124=$000000                   ; PMAC-style flags, normal mode
I125=$78000                    ; Use Servo IC 0 Channel 1 flags for Motor 1
I126=32000                     ; Home offset of +2000 counts
                               ; (enough to take you out of the limit)
I902=3                         ; Capture on rising flag and rising index
I903=2                         ; Use +LIM1 as flag (negative end switch)
M133->X:$B0,13,1               ; Desired Velocity Zero bit
M145->Y:$C0,10,1               ; Home complete bit

;********** PLC program to execute routine **********************
OPEN PLC 10 CLEAR
I124=$20000                    ; Disable +/-LIM as limits
CMD"#1HM"                      ; Home #1 into limit and offset out of it
WHILE (M145=1)                 ; Waits for Home Search to start
ENDWHILE
WHILE (M133=0)                 ; Waits for Home motion to complete
ENDWHILE
I124=$0                        ; Re-enable +/-LIM as limits
DIS PLC10                      ; Disables PLC once Home is found
CLOSE                          ; End of PLC
```

多步回零程序

你可能需要一个回零程序而它只用一个 Turbo PMAC 的回零移动是不能执行的。在这种情况下，您将使用两个（或可能更多）回零搜寻运动，改变二者之间的运动参数。虽然这可以通过一个在线命令序列来完成，但是创建一个小的运动程序来执行序列会更容易。

向哪个方向回零？这些情况中最常见的是，在上电时我们不知道它在回零触发的哪一侧。在这种情况下，移动到一个限位开关，以确保它是在行程的一端（这可以由回零到限位处来实现，就像在上面的例子）。然后在另一个方向做一个回零移动进入实时的回零触发。实现这个的一个示例运动例程是：

```
CLOSE OPEN PROG 102   CLEAR
I223=10                        ; Home speed 10 cts/msec positive direction
I224=$20000                    ; Disable hardware limits
I225=$78208                    ; Servo IC 2 Channel 2 for flags
I226=0                         ; No home offset
```

```
I7222=2                    ; Capture on rising edge of a flag
I7223=1                    ; Use PLIM2 as flag (positive end limit)
HOME2                      ; Home into limit
I223=-10                   ; Home speed 10 cts/msec negative direction
I224=$0                    ; Re-enable hardware limits
I7222=11                   ; Capture on flag low and index channel high
I7223=0                    ; Use HOME2 (home flag) as trigger flag
HOME2                      ; Do actual homing move
CLOSE
```

执行此的一个示例 PLC 程序例程是：

```
CLOSE
M233->X:$130,13,1          ; #2 Desired-velocity-zero bit
M245->Y:$140,10,1          ; #2 Home complete bit
OPEN PLC 11 CLEAR
I223=10                    ; Home speed 10 cts/msec positive direction
I224=$20000                ; Disable hardware limits
I225=$78208                ; Servo IC 2 Channel 2 for flags
I226=0                     ; No home offset
I7222=2                    ; Capture on rising edge of a flag
I7223=1                    ; Use PLIM2 as flag (positive end limit)
CMD"#2HM"                  ; Home into limit
WHILE (M245=1)             ; Waits for Home Search to start
ENDWHILE
WHILE (M233=0)             ; Waits for Home motion to complete
ENDWHILE
I223=-10                   ; Home speed 10 cts/msec negative direction
I224=$0                    ; Re-enable hardware limits
I7222=11                   ; Capture on flag low and index channel high
I7223=0                    ; Use HOME2 (home flag) as trigger flag
CMD"#2HM"                  ; Do actual homing move
WHILE (M245=1)             ; Waits for Home Search to start
ENDWHILE
WHILE (M233=0)             ; Waits for Home motion to complete
ENDWHILE
DIS PLC11                  ; Disables PLC once Home is found
CLOSE                      ; End of PLC
```

已经回零？当在上电时不知道它是否已经进入回零触发时会发生类似的情况。在这里，最简单的解决办法是写一个程序，用于评估这种情况；如果是在触发之中，它会在执行真正的实时回零之前移动出来。

```
;*************** Motion Program Set-up variables (to be saved) *********
CLOSE
M320->X:$078210,20,1                ; Variable for Servo IC 2 Ch. 3 home input
I325=$078210                        ; Use Flags3 for Motor 3
;************** Motion program to execute routine **********************
OPEN PROG 103 CLEAR
IF (M320=1)                         ; Already in trigger?
   I323=10                             ; Home speed 10 cts/msec positive direction
   I326=1600                           ; Home offset +100 counts (to make sure clear)
   I7232=11                            ; Capture on falling flag and rising index
   I7233=0                             ; Use Home3 as flag
   HOME3                               ; "Home" out of switch
ENDIF
I323=-10                            ; Home speed 10 cts/msec negative direction
I326=0                              ; No home offset
I7232=3                             ; Capture on rising flag and rising index
I7233=0                             ; Use HMFL3 as flag
HOME3                               ; Do actual homing move
CLOSE                               ; End of program

;***************PLC Set-up variables (to be saved) ********************
CLOSE
M320->X:$078210,20,1                ; Variable for Servo IC 2 Ch. 3 home input
I325=$078210                        ; Use Flags3 for Motor 3
M333->X:$01B0,13,1                  ; Desired Velocity Zero bit
M345->Y:$01C0,10,1                  ; Home complete bit
;**************** PLC program to execute routine **********************
OPEN PLC 12 CLEAR
IF (M320=1)                         ; Already in trigger?
   I323=10                             ; Home speed 10 cts/msec positive direction
   I326=1600                           ; Home offset +100 counts (to make sure clear)
   I7232=11                            ; Capture on falling flag and rising index
   I7233=0                             ; Use Home3 as flag
   CMD"#3HM"                           ; “Home” out of switch
   WHILE (M345=1)                      ; Waits for home search to start
   ENDWHILE
   WHILE (M333=0)                      ;Waits for home motion to complete
   ENDWHILE
ENDIF
I323=-10                            ; Home speed 10 cts/msec negative direction
I326=0                              ; No home offset
I7232=3                             ; Capture on rising flag and rising index
I7233=0                             ; Use Home3 as flag
CMD"#3HM"                           ; Do actual homing move
```

```
WHILE (M345=1)                    ; Waits for home search to start
ENDWHILE
WHILE (M333=0)                    ; Waits for home motion to complete
ENDWHILE
DIS PLC12                         ; Disables PLC once home is found
CLOSE                             ; End of program
```

存储回零位置

Turbo PMAC 自动存储电机在最新的回零搜寻移动时捕捉的编码器位置。此值保存在电机编码器回零捕捉寄存器[Y:$CE（电机 1），Y:$14E（电机 2）等，建议 M 变量 Mxx73]，在电动机没有绝对的上电位置时，寄存器在上电/重置时设置为零。如果获得位置基准是通过读取一个绝对传感器（Ixx10>0），如从一个旋变上，该寄存器保存读取的绝对位置的负值。在任一种情况下，它包含了编码器计数器的零位置（通电位置）和电机零点（原点）位置之间的差值，缩放单位是计数。

该寄存器有两个主要用途。首先，它为使用编码器位置捕捉和位置比较的寄存器提供了参考。这些寄存器参考编码器的零位置，该零位置是上电位置而不是起始（电机零点）位置。该寄存器保存两个位置之间的差。从编码器位置（通常是从位置捕获）上减去这个值，以获得电机的位置，或加到电机位置上来获得编码器位置（通常用于位置比较）。

例如：要移动一个轴，直到找到触发，然后将捕捉编码器的位置转换为电机的位置，你可以使用下面的 M 变量的定义：

```
M103->X:$078003,24,S              ; Servo IC 0 Channel 1 position-capture register
M117->X:$078000,17                ; Servo IC 0 Channel 1 position-capture flag
M125->Y:$0000CE,24,S              ; Motor 1 encoder position offset register
```

现在，你可以使用一个运动程序段如下所示：

```
INC                               ; Incremental moves
TM10 TA10                         ; Move segment time 10 msec
WHILE (M117=0)                    ; While no trigger to capture position
    X0.2                          ; Command next move segment
ENDWHILE
P103=M103-M125                    ; Read captured position; subtract offset to
                                  ; get motor position at trigger
```

这个寄存器的第二个用途是来确定编码器计数器是否已丢失了计数。完成这个可以通过在一个操作后执行第二次回零搜索移动，并第二次回零搜索移动后的寄存器的内容与第一次回零搜索移动后的内容进行比较。

慢跑（手动）-直到-触发

手动-直到触发的功能允许手动移动由一个触发中断，以及被一个相对到触发位置的移动所终止。它类似于一个回零搜索运动，除了不改变电机的零位置，并且在没有触发时有一个特定的目的地。

一个电机的“慢跑-直到触发”功能是通过将一个**^{常量}**说明符添加到一个正规的“明确的”手动命令的末尾来完成指定的，其中该{**常数**}是在停止之前、

相对于触发位置、要走的距离，单位是编码器计数。不能使用 J+和 J-不定的手动命令。

一个手动到触发的手动命令类似 J =10000^100，J=*^-50 或 J：50000^0。^之前的值是没有触发时的目标位置或要走过的距离（取决于微动命令的类型）。如果该第一个值是由一个*符号表示，PMAC 在一个预定义的电机寄存器（建议 M 变量 Mxx72）中寻找位置或距离。第二个值是相对于触发位置的、要行进的距离。此值始终表示为一个距离，无论手动命令是什么类型。这两个值都表示为编码器计数。

设置电动机的触发条件类似于设置回零搜索移动，上面已经解释。

Turbo PMAC 使用命令时的手动参数 Ixx19-Ixx22 用于先触发移动，以及使用在触发时的这些参数的值用于后触发移动。

在触发时位置传感器的捕捉值会存储在一个专用寄存器中（Y:$D8 用于电机 1，Y:$158 用于电机 2，等等）用于以后访问。该单位是计数；对于增量编码器，它们是相对于上电或复位的位置。

在手动到触发运动的开始，Turbo PMAC 将电机“home-search-in-progress”（回零搜索进行中）状态位（第一个电机状态字的 Bit 10，返回在一个？命令上）设定为真（1）。在找到触发时，或在运动结束时，该位被设置为假（0）。

在一个慢跑-直到触发运动的开始，Turbo PMAC 还将电机的触发移动状态位（第二个电机状态字的 7 位，返回在一个？命令）设置为真，并保持它为真至少到运动结束。如果在移动过程中发现一个触发，该位在后触发运动的结束时被设置为假；但是，如果前触发移动结束时没有找到一个触发，该位在运动的结束时保持为真。因此，该位可用在移动的结束告诉是否成功找到触发。电机“期望-速度- 零”状态位可以用来确定运动的结束。

运动程序移动-直到-触发

移动-直到-触发的结构可以从一个运动程序内使用。在这个版本中它是快速移动模式的一个变体。执行这些运动与在线慢跑-直到-触发运动完全一样，但他们被描述的稍有不同。

指令一个程序移动-直到-触发是通过{轴}{数据}^{数据}的语法。基本的例子是 X50^-2 和 Y（P1）^（P2）。第一个值是轴的目的地，如果没有找到触发，表示为轴的工程单位。该值可以是一个位置或距离，分别取决于轴是处于绝对还是增量模式。第二个值是从触发捕获位置到后触发移动结束的距离，表示为轴的工程单位。运动程序必须在快速模式下用于触发的操作，否则只是前触发移动被执行到指定的端点。

移动的指令加速度由 Ixx19、Ixx20 和 Ixx21 指定，如同其他的触发运动。如果 Ixx90 是在默认值 1，移动的速度幅度通过最大速度参数 Ixx16 指定；或者如果 Ixx90 为 0，通过手动速度参数 Ixx22 指定。指定触发条件和捕获方法如同其他触发动作，如上所述。设置状态位如同在线的手动-直到-触发动作。

有关这些移动的更多信息，请查看在《编写和执行运动程序》的部分的快速模式运动以及在软件参考手册中{轴}{数据}^{数据}运动程序语句的描述。

13-3 开环移动

开环移动，如他们的名字所暗示的，不执行闭环位置控制。他们打开伺服环路，只把指定大小的命令放在输出上。典型地，这些被用于诊断目的，但它们也可以在实际应用中使用。

执行这些移动使用电机的专用 O{常数}在线命令，其中{常数}表示输出的幅度为 Ixx69 的百分比，Ixx69 是电动机的最大输出参数。此命令不是一个运动程序的一部分，并且当一个电机的坐标系统正在执行一个运动程序时，即使它没有移动该电机，也不能将此命令发到该电机上。

如果 Turbo PMAC 不换相电机，该命令将在电机的单输出上创建一个恒定的信号。如果 Turbo PMAC 换相电机，该命令将设置（输入到电机正弦换相算法的）信号幅度。

要执行一个变量 O-命令，定义一个 **M** 变量到滤波器结果寄存器（X:$AE 用于电机 1，等），命令一个 OO 到电机，把电机放在开环模式，然后为 M 变量分配一个变量值。这种技术甚至可以工作在 PMAC 换相的电机上。

PMAC 执行程序的调整部分采用开环移动，让您诊断和调整放大器的响应。

第十四章 TURBO PMAC 的计算功能

Turbo PMAC 拥有先进的计算功能，可以减荷一台主机的许多运算，甚至是以之前不可能的方式进行的独立运算。卡上板载的用户程序的变量和常量可以执行许多算术、逻辑和超越运算。

14-1 计算优先级

作为一台多任务的、实时计算机，Turbo PMAC 具有一个精心设计的优化方案，以确保在需要时完成重要的任务，以及所有的任务都合理地快速执行。该方案的目的是要尽可能的隐藏其复杂性，但也提供灵活性以优化控制器，用于特别需要。按优先级顺序排列的任务是：

1、单字符的 I/O
2、换相更新
3、伺服更新
4、实时中断任务
5、 VME 邮箱处理
6、后台任务

单字符的 I/O

从串行端口或主机端口上输入一个单字符，或者发出一个单字符（从 ISA、PCI、USB 或以太网），在 Turbo PMAC 上是最高的优先级。这个任务对于每个字符只需要两个指令周期，但将其作为这么高的优先级是为了保证 Turbo PMAC 在进行逐字符操作时不能失去主机的控制。对于 Turbo PMAC 的总计算时间，这个任务从来不是其主要组成部分。注意，该任务不包括处理一个完整的命令，后者在一个较低的优先级（见下文）。

换相更新

换相（相位）更新在 Turbo PMAC 是次高的优先级。每一个相位中断（相位时钟周期），MACRO 环（如果存在的话）会更新新的信息，如果 I5060>0，会读取一对多路复用的 ADC（例如：Opt.12，Acc-36），将数据复制到解除复用的 RAM 寄存器中。

每（I7+1）相位中断，Turbo PMAC 对换相有效的各个电机（Ixx01 的 0 位=1）执行换相的计算。对于每个通过 Turbo PMAC 换相的电机，对于 80 MHz 的 CPU，这个任务在每个更新周期需要 1-2 微秒。

相位时钟频率由下面的确定：

- Turbo PMAC 上的跳线 E98 和 E29--E33
- 一个（非 UltraLite）Turbo PMAC2 上的 I7m00 和 I7m01（用于时钟源伺服 IC m 由 I19 设置）
- Turbo PMAC2 UltraLite 上的 I6800 和 I6801

默认的更新频率为 9 kHz（110 微秒的周期）。在默认情况下，每个电机的换相大约需要 Turbo PMAC 的 1%-2%的计算能力。

伺服更新

伺服更新是在 Turbo PMAC 的第三高的优先级。在每个伺服中断（伺服时钟周期），Turbo PMAC 处理编码器转换表的每个入口以准备原始的反馈和主数据让伺服算法使用。然后，它会对每个激活的电机 xx（Ixx00=1）执行伺服更新计算。此更新包括插值计算下一个瞬间指令位置、使用该值的伺服回路闭合的计算、实际的位置值、以及伺服增益项来计算指令输出。（如果 Ixx60 设为大于 0，电机 xx 的伺服回路闭合算法可以跳过中断周期。）这些伺服计算对于一个 80 MHz 的 CPU，在每个更新周期需要约 4 微秒的时间。

伺服时钟频率由以下因素决定：

- Turbo PMAC 上的跳线 E98、E29--E33 和 E3--E6
- （非 UltraLite）Turbo PMAC2 上的 I7m00、I7m01 和 I7m02（用于时钟源的伺服 IC m 由 I19 设置）
- 一个 Turbo PMAC2 UltraLite 的 I6800，I6801 和 I6802

默认的更新频率是 2.25kHz（440 微秒的周期）。在默认情况下，每个电机的伺服更新需要大约 1%的 Turbo PMAC 的计算能力。查看《关闭伺服回路》获得优化更新速率的信息。

实时中断任务

实时中断（RTI）任务是 Turbo PMAC 上的第四高的优先级。在伺服更新任务之后他们立即发生，速率由参数 I8 控制（每个 I8+1 伺服更新周期）。在这个优先级上发生 2 个显著任务：运动程序移动规划和 PLC0 方案（已经解释和编译）。

运动程序移动规划

运动程序移动计划包括运行完一个运动程序，直到遇到下一次移动或 dwell 停止指令，并为移动序列的下一部分计算运动方程。每次 Turbo PMAC 开始执行一个新的移动时，它设置一个内部标志以提示要准备程序的下一个运动。这个规划发生在下一个 RTI（实时中断）。

PLC 程序 0，编译的 PLC 程序 0

PLC0 和 PLCC0（编译的 PLC0）是特殊的 PLC 程序，执行优先级比其他的 PLC 程序更高。这意味着它们是用于执行频率比其他 PLC 任务更高的少数几个任务中。只要前一个 RTI(实时中断)的任务已经完成,每个实时中断时执行 PLC0 和 PLCC0。就干扰任务的进程而言，PLC0 和 PLCC0 是 Turbo PMAC 潜在的最危险的任务。如果他们太长，他们就会占用后台任务的时间。你注意到的第一件事是，通讯和后台的 PLC 任务将变得缓慢。在最坏的情况下，看门狗定时器就会跳闸，关闭卡，因为在后台的内务处理任务没有足够的时间来更新它。

VME 邮箱处理

VME 信箱寄存器读取或写入最多 16 个字符的块是 Turbo PMAC 第五高的优先

级。这种情况发生的速率是由主机控制。这不会占用很大一部分的 Turbo PMAC 的计算能力。

后台任务

在没有被任何较高优先级任务占用的时间，Turbo PMAC 将执行后台任务。有三种基本的后台任务：命令处理，解释和编译的 PLC 程序 1－31，内务处理。Turbo PMAC 的计算负载控制这些后台任务的频率：执行的高优先级任务越多，后台任务执行的速度越慢；后台任务越多，他们执行的越慢。

解译的 PLC 程序 1--31

解译的 PLC 程序 1- 31 在后台执行。每个后台周期，Turbo PMAC 将执行一次扫描（到最后或者到一个 **ENDWHILE** 语句）一个激活的解释的后台 PLC 程序，不被任何其他后台任务中断（虽然它可以被更高优先级的任务中断）。在每一个后台 PLC 程序的每次扫描之间，Turbo PMAC 将执行一般的内务管理，并响应主机命令，如果有的话。

编译的 PLC 程序 1--31

编译的 PLC 程序（PLCC 程序）1-31 是在后台执行。每个后台周期，Turbo PMAC 将执行一次扫描（到最后或者到一个 **ENDWHILE** 语句）所有激活的编译的后台 PLC 程序，从最低编号开始到最高编号，不被任何其他后台任务中断（虽然它可以被更高优先级的任务中断）。在上电/复位时，在第一个 PLC 程序运行之后，PLCC 程序才运行。

主机命令响应

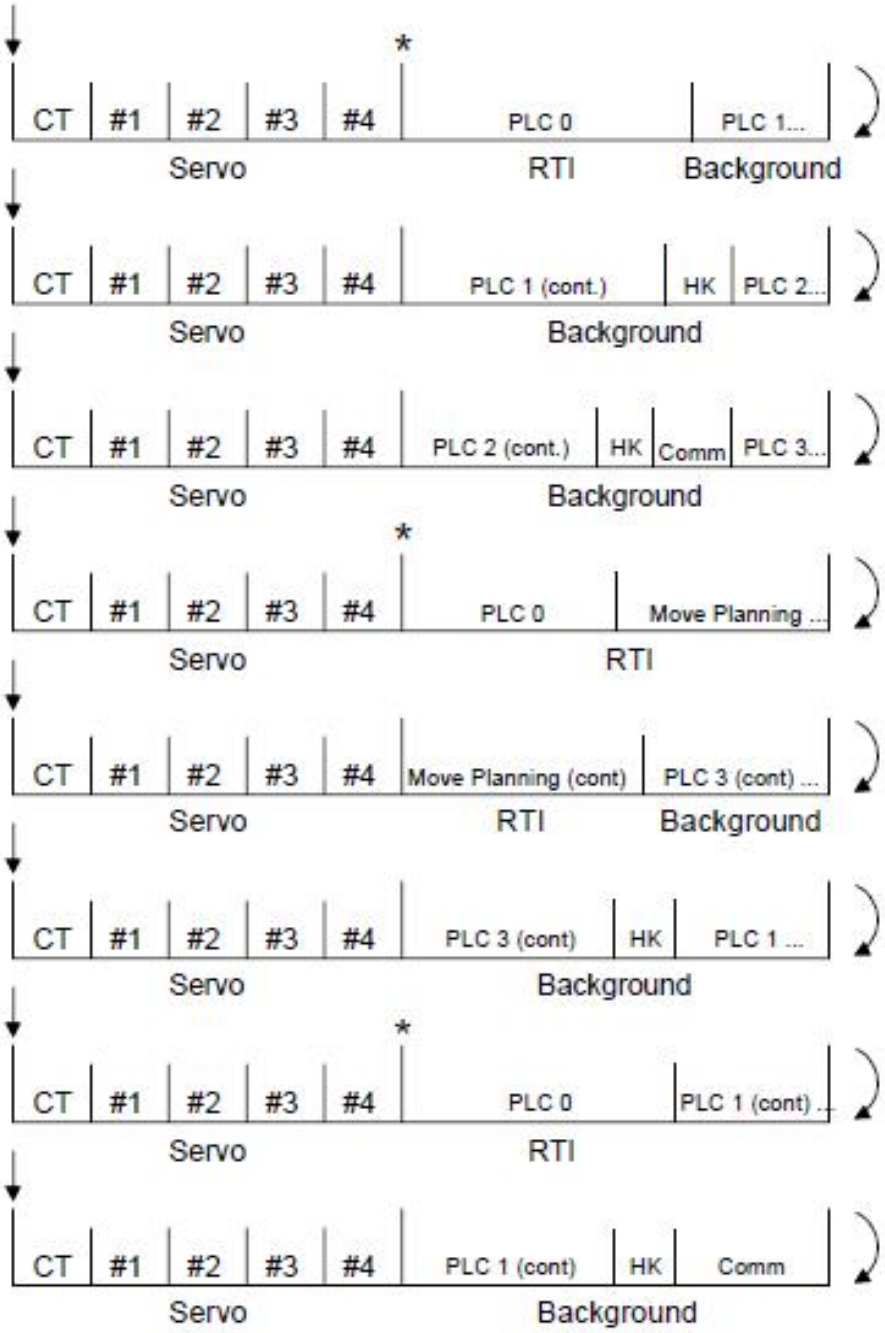
从任何端口收到一个控制字符对 Turbo PMAC 而言是一个信号，它必须响应命令。最常用的控制字符是回车（**<CR>**），它告诉 Turbo PMAC 将前面的所有字母数字字符作为一个命令行。其他控制字符有自己的含义，独立于接收的任何字母数字字符。在这里，Turbo PMAC 将对命令采取适当的行动，或者如果它是一个非法的命令时，它会向主机报告错误。

一般的内务管理

每个后台周期，Turbo PMAC 执行其内务管理职责，以保持自身正确更新。其中最重要的是安全限制检查（跟随误差、超程限位、故障、看门狗等）；一般的状态更新也在这里完成。虽然这是在一个低优先级，但是要保证一个最低频率，因为如果这个频率太低，看门狗会跳闸、关闭卡片。您可以设置这个所需的频率，通过改变变量 I40 的默认值 0。如果 I40 设置为大于 0 的值，这个值是伺服周期数，在其内必须完成每一个后台周期，否则看门狗定时器将跳闸。（如果 I40 设置为 0，这个数字是 4096 个伺服周期）。

下图显示了 Turbo PMAC 的多任务的分时线的例子，集中于伺服、实时中断和后台任务。

PMAC 多任务示例



CT - 转换表
#n - 电机 n 的伺服更新
HK - 内务管理
comm - 通信线路处理

↓ - 伺服中断

RTI - 实时中断任务
* - RTI 的开始

优先级的优化

通常情况下，Turbo PMAC 将有足够的速度和计算能力来执行所有要求的任务。有些应用程序会在一个特定的优先级放一个大的指令，为了使 Turbo PMAC 更高效地运行，应该做一些优先级的优化。

当 Turbo PMAC 开始运行时间不够时，会发生一些问题，如通信迟缓、PLC/PLCC 扫描率变慢、运行时间错误甚至跳闸看门狗定时器。上述症状的具体解决方案，在本手册的专门解决那些主题的章节中讨论。这些问题的一般的解决方案是双重的。

首先，高优先级的工作可以减慢或移动到一个较低的优先级的位置。诸如编码器转换表、PLC/PLCC0 和实时中断（RTI）的工作应进行评估。检查查看是否这些工作中的一切都是必要的，或者它的一部分可以被移动到一个较低的优先级或减缓。例如，一个 5 轴的应用程序可能不需要编码器转换表的入口 6～9。也许 PLC0 可以作为 PLCC1 完成，或 RTI（实时中断任务）可以在每第 4 个或第五个伺服周期时进行。

其次，工作可以被调整到一个优先级，使他们不太重要。大的 PLC 程序可以分割成几个较短的 PLC 程序。这通过在 PLC 的扫描中给出更多的中断增加了内务管理和通信的频率。运动程序 WHILE（条件）WAIT 语句可以如下：

```
WHILE(condition)
        DWELL20
ENDWHILE
```

这将为同等的或低优先级的其他任务提供更多的时间，如 PLC 程序和通信。

14-2 评估 Turbo PMAC 的计算负荷

Turbo PMAC 控制器提供了一些设施，允许你计算放在处理器上的运算负荷。在计算这些负载时，要使用几个关键定时器寄存器。这些寄存器被缩放，这样定时器的 1 个增量是 DSP 的两个时钟周期。因此，如果 DSP 运行在正好是 80 MHz 的时钟频率下---12.5 纳秒的时钟周期---计时器的 1 个增量将是 25 纳秒。

DSP 的时钟频率是倍增于 19.66 MHz 的晶体时钟频率，使用保存的 I52 值，根据下式：

$$\text{DSP 频率} = \frac{19.66\text{MHz}}{2} * (\text{I52} + 1)$$

在周期方面，定时器增量---2 个 DSP 周期---可以计算为：

$$\text{定时器增量(ns)} = \frac{203.4}{\text{I52} + 1}$$

相位中断任务

有两个关键定时器寄存器，用于评估相位中断的任务的计算负荷，如换相、电流环路闭合、ADC 解除多路复用。第一个是 DSP 中的硬件计时器，在地址 X:$FFFF8C。该寄存器保存了最后两个相位中断之间的定时器增量的数量，建立相

位中断的周期。这可以用来验证你的相位周期，以及与其他寄存器，验证计算的占空比。

第二个寄存器，位于 X:$000037，为最后一个中断保存从开始的相位中断任务到结束时的计时器增量的数量。当除以相位中断之间的时间时，这会得到相位中断任务的占空比。

伺服中断任务

另一个定时器寄存器可以用来评估伺服中断的任务的计算负荷，如转换表、插值、位置/速度环闭合以及数据采集。该寄存器位于 Y:$000037，保存最后一个中断的伺服中断任务从开始到结束时，计时器增量的数量。

如果这个时间加上相位任务时间小于相位中断之间的时间（X:$37 +Y:$37< X:$FFFF8C），那么这就是伺服任务占用的实际时间了。然而，如果这些时间的总和大于 1 个相位循环（X:$37+Y:$37> X:$FFFF8C），则伺服任务被相位任务中断（至少一次），中断的相位任务的时间必须被减去（见下面的例子）。

当伺服任务的净额时间除以（相位中断周期与每个伺服中断的相位中断的数量二者的）乘积时，结果是伺服中断任务的占空比。请注意，某些伺服任务，如数据采集、前台电机数据报告、甚至伺服环路闭合如果 Ixx60>0 时，不必在每个伺服周期都执行，所以占空比可以有所不同。

实时中断任务

两个定时器寄存器提供实时中断（RTI）任务（如 PLC0、PLCC0 和运动程序的计算）的负载信息。第一个寄存器，在 X:$00000B，保存最后一个中断的 RTI 任务从开始到结束时，计时器增量的数量。第二个寄存器，在 Y:$00000B，保存从上电或复位以来，一组实时中断任务从开始到结束时，计时器增量的最大数目。

如果这些时间再加上相位和伺服任务的时间，总和小于相位中断之间的时间（X:$37+ Y:$37+ X/Y:$0B<X:$FFFF8C），那么这些就是实时中断任务占用的实际时间了。但是，如果这些时间大于一个相位循环（X:$37+ Y:$37+X/Y:$0B> X:$FFFF8C），那么实时中断任务被相位任务中断（至少一次），而中断相位任务的时间必须被减去。此外，如果这些时间大于一个伺服周期，那么实时中断任务也被伺服任务中断（见下面的例子）。

将实时中断任务的最新净额时间除以（相位中断周期、每个伺服中断的相位中断次数和每个实时中断的伺服中断的数量，三者的乘积）产生实时中断任务的占空比。实时中断任务的占空比在应用程序中可以有很大的不同，所以建议计算一个运行平均值来计算一般负载。

总中断任务

计算 Turbo PMAC 中断任务的总的占空比可以通过对三种类型的中断任务的占空比进行求和。在一般情况下，建议相位和伺服任务的占空比不超过 50%，且所有前台任务的占空比不超过 75%。这些都不是严格的限制---有可能超过他们，但如果这些准则被超过，所有操作的时间应仔细评估。

示例监控程序

下面的示例代码，可以用来监测总的中断任务的占空比：

```
M70->X:$FFFF8C,0,24          ; 相位中断之间的时间
M71->X:$000037,0,24          ; 相位任务的时间
M72->Y:$000037,0,24          ; 伺服任务的时间
M73->X:$00000B,0,24          ; 实时中断任务的时间
P70=4                        ; 每个伺服中断有4个相位中断
P76=16                       ; 平均占空比的滤波器的长度

OPEN PLC 17 CLEAR
P71=M71/M70                  ; 相位任务占空比
P69=INT((M71+M72)/M70)       ; 相位中断伺服的次数
P72=(M72-P69*M71)/(M70*P70)  ; 伺服任务占空比
P68=INT((M71+M72+M73)/M70)   ; 实时中断任务被相位任务中断的次数
P67=INT((M71+M72+M73)/(M70*P70))  ; 实时中断任务被伺服任务中断的次数
P73=(M73-P68*M71-P67*(M72-P69*M71))/(M70*P70*(I8+1))
                             ; 实时中断任务占空比
P74=P71+P72+P73              ; 最新的总的前台占空比
P75=(P75*(P76-1)+P74)/P76    ; 平均总前台占空比
CLOSE
```

后台周期时间

有两个定时器寄存器，对评估执行一个后台周期所需的时间很重要。这些寄存器都参与了看门狗定时器的操作。在地址 Y:$000025 的寄存器在每个后台周期的结束时设置为 I40 的值。（如果 I40 为 0，寄存器被设置为 4095）。直到下一个后台周期完成，这个寄存器在每个伺服周期递减。数据采集功能是非常有用的以确定后台周期占多长时间。

在 X:$000025 处的寄存器包含自 Turbo PMAC 的最后一个上电或复位以来，Y:$000025 达到的最低值。如果该值接近 0，Turbo PMAC 已接近于跳闸其看门狗，而定时器和后台任务例如 PLC 程序执行、通信响应和安全检查会变得缓慢。

14-3 数字值

Turbo PMAC 可以以许多形式存储和处理数字值，以定点和浮点值。摩托罗拉 56300 DSP，用作 Turbo PMAC 的 CPU，是一个定点处理器，有内置的 24 位和 48 位的运算能力（加上一个 56 位的累加器）。然而，Turbo PMAC 的固件实现了一套完整的浮点例程。

内部格式

内部伺服、插值和换相例程的所有操作是定点运算，24 位和 48 位，以最大速度完成。用户程序，运动程序和 PLC 程序，使用 48 位浮点运算是为了最大范围和普遍性。甚至当读取和/或写入到定点寄存器时，中间格式都是浮点值。这允许用户任意混合不同类型的变量，让 Turbo PMAC 自动处理类型匹配

该规则唯一的例外是编译的 PLC 程序；在一个声明中只包含“L 变量”和整型常量，可以容纳在一个有符号的 24 位范围（+/-8M），中间格式是有符号的 24 位整数。参考在本手册的《编写和执行一个 PLC 的程序》一章的编译的 PLC 部分，获取更多细节。

一般的浮点格式为 48 位长，36 位尾数和一个 12 位的指数。这提供了范围是$\pm 2^{\pm 2047}$，或$\pm 3.233\times 10^{\pm 616}$，这对该卡的任何可预见的用途提供了足够的范围。

48 位浮点寄存器的内部格式显示在下面的表中：

X-字												
位：	23	22	21	20	19	18	17	16	15	14	13	12
部分	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数
值：	-2^{0}	$+2^{-1}$	$+2^{-2}$	$+2^{-3}$	$+2^{-4}$	$+2^{-5}$	$+2^{-6}$	$+2^{-7}$	$+2^{-8}$	$+2^{-9}$	$+2^{-10}$	$+2^{-11}$
位：	11	10	9	8	7	6	5	4	3	2	1	0
部分	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数
值：	$+2^{-12}$	$+2^{-13}$	$+2^{-14}$	$+2^{-15}$	$+2^{-16}$	$+2^{-17}$	$+2^{-18}$	$+2^{-19}$	$+2^{-20}$	$+2^{-21}$	$+2^{-22}$	$+2^{-23}$
Y-字												
位：	23	22	21	20	19	18	17	16	15	14	13	12
部分	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数	尾数
值：	$+2^{-24}$	$+2^{-25}$	$+2^{-26}$	$+2^{-27}$	$+2^{-28}$	$+2^{-29}$	$+2^{-30}$	$+2^{-31}$	$+2^{-32}$	$+2^{-33}$	$+2^{-34}$	$+2^{-35}$
位：	11	10	9	8	7	6	5	4	3	2	1	0
部分	指数	指数	指数	指数	指数	指数	指数	指数	指数	指数	指数	指数
值：	$+2^{11}$	$+2^{10}$	$+2^{9}$	$+2^{8}$	$+2^{7}$	$+2^{6}$	$+2^{5}$	$+2^{4}$	$+2^{3}$	$+2^{2}$	$+2^{1}$	$+2^{0}$

尾数： 尾数---标准格式的浮点数的尾数。尾数应该在范围 0.5<=尾数<1.0，或 -1.0<=尾数<-0.5。数字恰好为 0 时尾数= 0.0。

指数： 指数---修改格式的浮点数的指数。在这里 12-位的值（0<=指数<=4095）应该减去 2047 的值（n=指数-2047；-2048<= n<=2047），然后 2 升高到 n 次幂，将得到的值乘以尾数。数字恰好为 0 时，指数=0。

接收值

作为命令行的一部分，从主机发送的常数值将作为 ASCII 文本发送，无论是作为十进制值或十六进制值。十六进制值必须在前面加一个$字符，它们必须是无符号的，而且他们不包括小数值。十进制值可以是正的或负的，并且可以包括分数值。Turbo PMAC 数值解释器不支持指数表示法，它有限的值的范围为$\pm 2^{\pm 35}$，或$\pm 3.43\times 10^{\pm 10}$。超过此范围的值被截断为范围的最大值或最小值。

示例：

1234
3
03 (前导零可以有)
-27.656
0.001
.001 (不需要前导零)
$ff00 (解释为十六进制)

报告值

作为响应行的一部分，Turbo PMAC 报告数值到主机，以十进制 ASCII 文本的形式的（虽然如果 I9=2 或 3，地址值可以用十六进制 ASCII 码的形式进行报告---见下文）。该值的报告器仅限于在$\pm 2^{\pm 47}$，或$\pm 1.41 \times 10^{\pm 14}$的范围内传递值。超过此范围的值被截断为范围的最大值或最小值。

14-4 地址

Turbo PMAC 采用一个摩托罗拉 DSP563xx 作为其处理器。该 DSP563xx 的内存和 I/O 具有双 24 位的地址空间（其中共 19 位是由 Turbo PMAC 所使用）。（请注意，Turbo PMAC 的 I/ O 是内存映射，与您的 PC 不一样它不具有独立的 I/ O 空间）。当在 Turbo PMAC 内指定地址时，你必须说明的内存的哪一半（X 或 Y）---或其全部（L）作为一个长的 48 位的字---后跟一个可选的冒号，然后是数字地址本身。数字地址是一个常数，在$000000 - $07FFFF 的范围内。通常，地址值被指定为一个十六进制值（用$前缀）。

不要把 Turbo PMAC 本身的内存和 I/O 地址与主机的混淆起来。

合法的地址指定的例子是：

Y:$078A02 (包含机器I/O的字)
X:136 (电机1的指令位置– 也有**X:$88**)
X$078003 (捕获的编码器 1 的位置)

这种形式的地址指定是具体使用在 M 变量的定义和直接读（R）、写（W）命令中。有指定地址的 I-变量，但通常这些是预先定义到 X 或 Y 空间，因此，需要的只是数值。数据-采集-地址的 I-变量（I5001-I5048）在数值前使用一个额外的十六进制数字来指定内存的一半（见 I5001 的描述）。

14-5 变量

Turbo PMAC 有几种类型的变量。在 Turbo PMAC，变量是由一个单字母（I、P、Q 或 M）指定的，后跟一个从 0 到 8191 的数。每个字母代表一个不同的变量类型，每种类型有其自己的属性。不同类型会共享这样的特性：当他们的名字被引用在表达式中，使用变量的当前值（读取它们）；在一个方程中能够给它们赋值（写入它们）。

在 Turbo PMAC 中，用户不能指定自定义的变量名，但是在 PMAC 执行程序的编辑器中有一个替换（宏）方案，允许使用用户定义的变量名写程序，但在下载过程中要将这些名字改变为 Turbo PMAC-合法的变量名。这种替代方案强烈推荐用于管理大型应用程序。

I-变量

I-变量（初始化或设置变量）确定对于一个给定应用程序设置该卡的特性。他们是在内存中的固定位置，并有预先定义的含义。大部分是整数值，其范围对于于不同变量是不一样的。有 1024 个 I 变量，从 I0 到 I8191，它们的结构如下：

I0 – I99	Global card setup---全局卡设置变量
I100 – I199	Motor 1 setup---电机1的设置变量
I200 – I299	Motor 2 setup---电机 2 的设置变量
…	
I3200 – I3299	Motor 32 setup---电机32的设置变量
I3300 – I4799	Supplemental Motor setup---补充的电机设置变量
I4900 – I4999	Configuration status---配置状态
I5000 – I5099	Data gathering/ADC demux setup---数据采集/ADC解除多路复用的设置变量
I5100 – I5199	Coordinate System 1 setup---坐标系1的设置变量
I5200 – I5299	Coordinate System 2 setup---坐标系 2 的设置变量
…	
I6600 – I6699	Coordinate System 16 setup---坐标系16的设置变量
I6800 – I6999	MACRO IC setup---MACRO IC的设置变量
I7000 – I7999	Servo IC setup---SERVO IC的设置变量
I8000 – I8191	Encoder conversion table setup---编码器转换表设置变量

赋值

分配给 I 变量的值可以是常量或表达式。如果发送时没有打开的缓冲区，该命令是在线（立即）的；如果缓冲区是开启的，该命令是缓冲的程序命令。

示例：

```
I120=45
I120= I120+5
I（P1*100+20）= 10
```

限制范围

对于范围有限的 I-变量，尝试分配一个超出范围的值不会导致错误。该值是自动翻转到范围内，通过取模运算（截断）。例如，I3 的范围为 0 至 3（4 个可能的数值）。命令 I3=5 将（5 取模 4）=1 的值赋给变量。

非易失性存储

当一个值被分配给一个 I-变量时，一个易失性的活动存储寄存器中的值（在断电时不保留它的值）已经被除去。使用 SAVE 命令将所有这些有效值复制到非易失性闪存中。在上电或复位时，最后一个保存到闪存中的值，被自动复制回到活动内存中。

默认值

所有 I-变量的默认值包含在制造商提供的固件中。它们可以通过 I{常数} =*

命令单独使用，或者通过 I{常数}..{常数} =*的命令在一定范围内使用。在非默认设置时，当电路板通过$$$***命令重新初始化或通过重新初始化跳线进行复位时，所有默认设置从固件复制到活动内存中。最后保存的值不会丢失；他们只是未被使用。参考 I-变量描述部分获取各个变量的功能。

P-变量

P-变量是通用的用户变量。他们是 48 位的浮点变量，在 Turbo PMAC 的内存的固定位置，但没有预定义用途。有 8192 个 P-变量，从 P0 到 P8191。一个给定的 P-变量与其他的 P 变量是一样的;所有的坐标系统可以访问所有的 P-变量(对比一下 Q 变量，他们被耦合到一个给定的坐标系中，见下文)。这允许在不同的坐标系之间传递有用信息。P-变量可以在程序中任意使用用于任何所需目的：位置、距离、速度、时间、方式、角度、中间计算等。

使用的内存类型

P-变量可以位于主内存或补充性电池支持的参数存储器(如果选择了 Opt.16)中。如果 I46 设置为 0（默认）或 2，P-变量都位于主存储器，可以快速访问（1 个等待状态），但若不用 SAVE 命令将值复制到闪存，其值将不会保留。上电或复位时，最后保存的值从闪存拷贝到 RAM 中的活动变量寄存器中。

如果 I46 设置为 1 或 3，P 变量均位于选项 16 的电池供电的 RAM 中，它的访问速度慢（9 个等待状态），但在断电时其值由电池自动保留。

特殊用途的 P-变量

一般来说，Turbo PMAC 固件没有自动使用 P-变量。然而，它可以特殊使用变量 P0 – P32 和 P101- P132，如下所述。

如果一个只包括一常量值的命令发送到 Turbo PMAC，Turbo PMAC 将该值分配给变量 P0（除非已经定义了特殊的表格缓冲区，如补偿表或激励表，但尚未填满---在这种情况下，常数值将被输入到表中。例如，如果命令 342<CR>发送到 Turbo PMAC，它会将它解释为 P0=342<CR>。）

此功能是为了便于简单操作终端接口。然而，这确实意味着，P0 用于其他用途不是一个好主意，因为它很容易不小心改变这个变量的值。

如果您的应用程序使用运动子程序在刀尖（轴）位置和接头（电机）位置之间进行转换，变量 P1- P32 和 P101- P132 用于这些子程序的电机位置（Pn 为电机 n 位置；如果 PVT 移动被转换，P10n 是电机 n 的速度)。如果您正在使用运动子程序，要确保不使用子程序中的 P-变量用于任何其他目的。

Q-变量

Q-变量，如 P-变量一样，是通用的用户变量---48 位的浮点变量，在内存中的固定位置，没有预先定义的用途。然而，一个给定的 Q 变量(以及其包含的值)的寄存器的意义是依赖于哪个坐标系在利用它。这允许多个坐标系统使用相同的程序（例如，包含直线 X（Q1+25）Y（Q2），但他们自己的 Q 变量有不同的值（在这种情况下，意味着不同的目标点)。

分配 Q-变量

总共有 8192 个 Q-变量。如果只使用一个坐标系(Coord.Sys1(即坐标系 1)

---指定为&1），使用所有这些变量：Q0 到 Q8191。坐标系 2（&2）的 Q 变量会重叠这些：&2 的 Q0 与&1 的 Q4096 相同，&2 的 Q4095 与&1 的 Q8191 相同。（该 Q 缓冲区实际上是旋转的，所以&2 的 Q4096 与&1 的 Q0 相同，&2 的 Q8191 是&1 的 Q4095）。因此，两个坐标系都有 4096 个独特的 Q-变量：Q0 到 Q4095。

注意：

对于覆盖另一个坐标系 Q 值的变量，没有提供保护。用户有责任保持 Q-编号在适当的范围内。

如果使用 3 或 4 个坐标系，对于每个坐标系有 2048 个独特的 Q 变量（Q0 – Q2047）。如果使用 5 至 8 个坐标系，对于每个坐标系有 1024 个独特的 Q 变量（Q0 – Q1023）。如果使用 9 至 16 个坐标系，对于每个坐标系有 512 个独特的 Q 变量（Q0 – Q511）。

下表显示了当 Q 变量被存储在主内存时，每个坐标系统存储 Q0 的地址。这表明他们可能是如何重叠的。

坐标系#	Q0 地址	坐标系#	Q0 地址	坐标系#	Q0 地址	坐标系#	Q0 地址
1	$8000	5	$8400	9	$8200	13	$9200
2	$9000	6	$8C00	10	$8600	14	$9600
3	$8800	7	$9400	11	$8A00	15	$9A00
4	$9800	8	$9C00	12	$8E00	16	$9E00

寻址一个 Q 变量集

命令的类型决定使用哪一组 Q 变量。当从主机的一个在线（即时）命令访问 Q 变量时，使用当前的主机寻址（通过&n 命令）的坐标系的 Q 变量。

当从一个运动程序语句（包括运动子程序）中访问一个 Q 变量，属于运行程序的坐标系统中的 Q 变量被使用。如果一个不同的坐标系统运行相同的运动程序，它会使用不同的 Q-变量

当从 PLC 程序语句中访问一个 Q-变量，由那个 PLC 程序与 ADDRESS 命令寻址的坐标系统的 Q-变量被使用。每个 PLC 程序可以寻址一个特定的坐标系统，并独立于其他 PLC 程序和主机寻址。如果在 PLC 程序中没有使用 ADDRESS 命令，该程序使用在坐标系 1 的 Q 变量。

使用的内存类型

Q-变量可以位于主内存或辅助的电池支持参数存储器中（如果选定了选项 16）。如果 I46 设置为 0（默认值）或 1，Q-变量都位于主内存，它可以快速访问（1 个等待状态），但如果没有 SAVE 命令将值复制到闪存，其值将不会保留。上电或复位时，最后保存的值从闪存拷贝到 RAM 中的活动变量寄存器中。

如果 I46 设置为 2 或 3 时，Q-变量均位于选项 16 的电池供电的 RAM 中，它的访问速度慢（9 个等待状态），但当电源被移走时，其值由电池自动保留。

特殊用途的 Q-变量

几个 Q-变量具有需要注意的特殊用途。该 ATAN2（两个参数的反正切）函数自动使用 Q0 作为第二个参数（余弦参数）。READ 命令分别将其读取的值前面加

上字母 A 到 Z 放在 Q101 到 Q126 中，将表示已经读取哪个变量的屏蔽字放入 Q100 中。一个运动程序中的 S（主轴）语句将它后面的值放入 Q127。

如果应用程序使用运动子程序在刀尖（轴）位置和接头（电机）位置之间进行转换，坐标系的变量 Q1- Q10 和 Q11- Q19 用于这些子程序的轴数据（Q1 - Q9 是用于轴位置； Q10 告诉 PVT 移动是否正在被转换；如果 PVT 运动已经被转换，Q11 - Q19 是用于轴速度）。如果使用运动子程序，要确保不使用子程序的 Q-变量用于任何其他目的。

M-变量

提供 M 变量是为了允许用户方便访问 Turbo PMAC 的内存和 I/O 空间。通常，一个定义只需要进行一次，用一个在线的命令。必须用 SAVE 命令保存该定义通过断电或者复位。定义一个 M 变量是通过将其分配到一个位置并定义该位置的值的大小和格式。一个 M 变量可以是一位、一个半字节（4 位）、一个字节（8 位）、1-1/2 个字节（12 位）、一个双字节（16 位）、2-1/2 个字节（20 位）、一个 24 位字、一个 48 位定点双字、一个 48 位浮点双字、或双端口 RAM 和 JTHW 复用器端口的特殊格式。

一共有 8192 个 M 变量（M0 至 M8191），与其它的变量类型一样，M 变量的数目可以一个常量或一个表达式指定：M576 或 M（P1+20）。

M-变量定义

完成 M 变量的定义是使用定义箭头---由减号和大于号组成(->)。一般来说，只需要进行一次定义，并在一个在线的命令中，因为它是存储在电池供电的 RAM 或保存到闪存中。如此定义的 M 变量可以被重复使用。

一个 M 变量可以采取以下类型之一，由定义的地址前缀指定：

X:	1到24位的定点，在X-内存中
Y:	1到24位的定点，在Y-内存中
D:	48 位的定点，跨越 X-和 Y-内存
L:	48位的浮点，跨越X-和Y-内存
DP:	32位定点 (X和Y的低16位) (用在双端口的RAM)
F:	32位浮点 (X和Y的低16位) (用在双端口的RAM)
TWD:	从拨轮开关端口复用的BCD解码
TWB:	从拨轮开关端口复用的二进制译码
TWS:	复用串行的I/O解码从指轮端口
TWR:	复用的串行旋转变压器解码从拨轮端口
*:	没有地址定义；使用定义字的一部分作为通用变量

如果定义了一个 X 或 Y 型的 M 变量，则还必须定义使用的起始位、位数、格式（解码方法）。

典型的 M 变量定义语句是：

```
M1->Y:$078C02,8,1
M102->Y:$78003,8,16,S
M103->X:$078003,0,24,S
M161->D:$8B
M5141->L:$2041
M50->DP:$060401
```

M51->F:$0607FF
M100->TWD:4,0.8.3,U

请参阅手册的在线命令参考部分，关于各类型的 M 变量定义的说明。这可以在本手册的《与 Turbo PMAC 通讯》部分找到。许多建议的 M 变量的定义也在软件参考手册中给出。

这是一个好主意，准备一个单独的文件包括所有的 M 变量定义，并在这个文件的顶端放有命令 **M0..8191 ->***。这将删除所有的现有定义，并有助于防止“杂散”的 M 变量定义造成的神秘问题。

M 变量定义存储为 48 位代码，在 Turbo PMAC 内存的地址$004000（用于 M0）到$005FFF（用于 M8191）。Y 寄存器包含了定义所指向的寄存器的地址；X 寄存器包含一个代码，该代码确定寄存器的哪一部分被使用以及它是如何解释的。

如果另一个 M-变量指向 Y 寄存器，它可以用来改变对象寄存器。这种技术的主要用途是建立寄存器阵列，用来工作通过内存的表。

限制范围

许多 M 变量的取值范围比 Turbo PMAC 的全计算范围更小。如果一个超过 M 变量范围的值被赋值给那个 M 变量，Turbo PMAC 将值自动翻转在该范围内，并且不报告任何错误。

例如，对于只有 1 位的 M 变量，写入到变量的任何奇数最终为 1，任何偶数最终为 0。如果试图把一个非整数值放在一个整数 M 变量中，Turbo PMAC 自动舍入到最接近的整数。

使用 M-变量

一旦定义完成，一个 M 变量可用在程序中，就像任何其他变量---通过表达式。当计算表达式时，Turbo PMAC 读取定义的内存位置，基于定义的大小和格式计算一个值，并在表达式中使用它。

在表达式中使用 M-变量应注意。如果一个 M 变量可以通过一个伺服程序改变（如瞬时指令位置），该程序比后台表达式求值有更高的优先级，那么不保证该值不会在评估的中间发生改变。举例来说，如果在表达式（M16-M17）*（M16+M17）中的 M 变量是瞬时伺服变量，也不能保证 M16 或 M17 在表达式的两个地方将具有相同的值或 M16 和 M17 的值将来自同一个伺服周期。第一个问题可以通过设置 P1 = M16 和 P2 = M17 来克服，但对于第二个问题没有通用的解决办法。

用于间接寻址

作为指针，M 变量可以用于一种被称为间接寻址的技术来访问一系列的寄存器，而无需为每个寄存器定义一个单独的变量。这种技术使用两个 M 变量。第一个被分配到一个寄存器所关心的地址区域中，与所需类型的格式，第二个被分配给包含第一个定义的地址的寄存器。

M 变量的地址定义是在 Turbo PMAC 内存的固定位置，起始于$004000（用于 M0），结束在$005FFF（用于 M8191）。在每个这些地址处的 X 寄存器保存确定 M 变量格式的代码；Y 寄存器保存所指向的寄存器的地址。通过改变该 Y 寄存器的内容，你可以改变该 M 变量指向的寄存器的地址。

这种技术最好通过实例说明。假设一个 2048 字的 UBUFFER 已经使用标准内存在 TurboPMAC 创建完毕。该 UBUFFER 将占据地址$010000 至$0107FF。在这个缓冲区，我们要创建一个 2048-入口的浮点正弦表。我们将通过两个 M 变量定义开始：

```
M64->L:$010000          ；浮点 M-变量定义到 UBUFFER 的开始
M65->Y:$004040,0,12     ；M64 定义地址，低 12 位
```

现在，通过改变 M65 的值，我们改变了 M64 指向的地址。请注意，通过将 M65 只指定到 M64 定义地址的低 12 位（最后 3 个十六进制数字），我们可以在这种情况下，只需把值赋给 M65，表示相对寄存器组开头的偏移量。要创建正弦表，使用下面的代码：

```
M65=0                           ；M64 指向 L:$010000
WHILE（M65<2048）
M64=SIN（360*M65/2048）         ；写正弦值
M65=M65+1                       ；索引 M64 指到下一个寄存器
ENDHWILE
```

14-6 运算符

Turbo PMAC 运算符和任何计算机语言的作用是一样的：他们组合数值来产生新的值。运算符的详细描述在软件参考手册中给出；这里给出概述。

算术运算符

Turbo PMAC 采用四个标准的算术运算符：+、- 、*和/。该标准的代数优先规则是：乘法和除法在加减法之前执行，相同优先级的运算从左到右执行，括号内的操作先执行。

取模运算符

Turbo PMAC 也有‘%’取模运算符，运算符前面的值除以运算符后面的值时，其产生结果余数。值可以是整数或浮点数。该运算符对于处理翻转的计数器和计时器特别有用。

当取模操作是由一个正 x 值完成的，其结果的范围可以从 0 到 x（不包括 x 本身）。当取模操作是由一个负的 x 值完成的，其结果的范围可以从-x 到 x（不包括 x 本身）。当一个寄存器可以在任一方向翻转时这个负的模操作是有用的。

逻辑运算符

Turbo PMAC 有三种逻辑运算符做逐位操作：&（逐位与），|（逐位或）和^（逐位异或）。如果使用浮点数，则操作适用于小数以及整数位。&、*和/三者具有相同的优先级；|和^具有与+和-相同的优先级。使用括号可以覆盖这些默认的优先级别。

注意：

这些逐位的逻辑运算符是不同于在复合条件下使用的简单的布尔运算符 AND 和 OR 的（参阅）。

14-7 函数

正如在任何编程语言中，Turbo PMAC 的数学函数对常量或表达式进行数学运算，以产生新的值。一般格式为：**{函数名}（{表达式}）**

可用的函数有：

SIN: 正弦三角函数
COS: 余弦三角函数
TAN: 正切三角函数
ASIN: 三角反正弦（arcsin）
ACOS: 三角反余弦（arccos）
ATAN: 三角反正切（arctan）
ATAN2: 特殊两个参数，四象限三角反正切
LN: 自然对数（e 为底的对数）
EXP: 指数运算（e^x）
SQRT: 正平方根
ABS: 绝对值
INT: 截断为整数（负向无穷大）

全局 I 变量 I15 控制三角函数是使用度（I15=0）还是弧度（I15=1）。每个函数的详细说明，包括定义域和值域，在软件参考手册中给出。

14-8 表达式

Turbo PMAC 表达式是一个数学构造，包括常量、变量和函数，通过运算符连接。表达式可以用于赋值给一个变量，以确定一个运动程序的参数，或作为条件的一部分。常量可以是一个表达式，所以如果语法是**{表达式}**，常数也可以用作更复杂的表达式。非常量表达式没有多余的括号，与指定**{数据}**不同（见下文）。

表达式的实例是：

```
512
P1
P1-Q18
1000* COS（Q25*3.14159/180）
I100* ABS（M347）/ATAN（P（Q3+1）/6.28）+5
```

14-9 {数据}语法

对于 Turbo PMAC 的目的，如果指令语法需要**{数据}**，使用一个未用括号括起来的常数，或者是一个被括号括起来的表达式。（由于常量可以是一个表达式，所以把一个常量放在一个括号内是合法的，但是这需要更多的存储空间和更多的计算时间。）

举例来说，如果列出的命令语法是 T**{数据}**，使用 T100、T（P1+250*P2）或 T（100）（这是合法的，但是浪费）是合法的。

14-10 变量值赋值语句

这种类型的语句计算并分配赋值给一个变量。当一个赋值语句被发送到 Turbo PMAC，如果一个程序的缓冲区是打开的，该语句被添加到缓冲区。如果不是，它会立即执行。标准的赋值语法是：

{变量名}={表达式}

其中**{变量名}**指定使用哪个变量，而**{表达式}**表示要分配给该变量的值。

I-变量默认赋值

一个声明的语法：

{I-变量}=*

将为指定的 I-变量分配该变量的制造商的默认值（而不是用户的最后一次保存的值）。

同步的 M-变量赋值

在一个运动程序中，当 Turbo PMAC 正在混合移动或样条运动时，它必须在实际的运动点之前在程序中进行计算。这是必要的，以便能够将移动完全混合一起，并且还能够做合理的速度和加速度的限制。在混合时的计算可能发生在实际运动一个、两个或三个移动的前方，这取决于运动的模式。

为什么需要

当赋值给变量是计算的一部分，当看着实际执行移动时，该变量会在程序中他们的位置之前得到他们的新值。通常，对于 P 和 Q 变量，这不是一个问题，因为它们存在只是为了有助于进一步运动的计算。然而，对于 M 变量，尤其是输出，这可能是一个问题，因为通过一个正常的变量值赋值语句，该行动将比预期更早发生，看着语句在程序中的位置。

例如，在程序段

```
X10          ；移动 X 轴至 10
M1=1         ；打开输出 1
X20          ；移动 X 轴至 20
```

你可能预期，在 X 轴达到位置 10 时，输出 1 将被打开。但是，由于 Turbo PMAC 是提前计算，在移动到 X10 的开始，它会已经计算出程序到下一步的运动，包括两者之间的所有程序语句，包括打开输出的 M1=1。因此，在使用这种技术时，打开输出比预期的要早。

他们如何工作

同步 M 变量赋值语句被实现为解决这个问题的方案。当在程序中遇到这些语句时，它不会被立即执行；而是将动作放在堆栈上，在程序的下一步行动实际开始时，它才执行。这使得输出动作正确同步于运动动作。

在修改后的程序段

```
X10          ；移动 X 轴至 10
M1==1        ；同步打开输出 1
X20          ；移动 X 轴至 20
```

在移动到 X10 的开始，遇到声明 **M1= =1**（双等号表示同步赋值），但实际上并没有执行该操作，直到融入下一步（X20）开始时才执行。

注意：

对于同步赋值，执行实际的赋值是在开始混合到新的移动时，这通常是提前于设定的点。在直线和圆弧模式的移动中，这种混合发生在距离指定的中间点 V*TA/2 远的地方，其中 V 是轴的指令速度，而 TA 是加速（混合）的时间。

另外，还要注意赋值是与指令位置同步的，不一定是实际的位置。伺服回路有责任使指令位置和实际位置二者密切匹配。

在 Turbo PMAC 正在执行分段移动（Isx13>0）的应用中，同步的 M 变量赋值是在在程序运动开始加速之后的第一个 Isx13 样条段开始时才执行的。

Turbo PMAC 会检查，查看是否将同步赋值取出，并在每一个实时中断（每 I8+1 个伺服周期）时执行它们。I8 越小，伺服周期时间越小，同步输出的定时控制越严格。

注意：

当程序结束或暂时停止时，在程序中的最后一步或 DWELL 后的同步的 M-变量赋值并不执行。使用一个 DWELL 作为程序的最后一个语句来执行这些语句。

如果在执行他们的时间之前因为程序结束或停止而导致同步赋值留在队列中，它们可以通过在线的 **MFLUSH** 命令从队列中删除。

语法。 有四种形式的同步的 M-变量赋值语句：

M{常数}=={表达式} ；直线等号赋值
M{常数}&={表达式} ；与-等号赋值
M{常数}|={表达式} ；或-等号赋值
M{常数}^={表达式} ；异或等号赋值

在所有这些形式，当该行在程序中遇到时，执行移动之前，先评估计算语句右侧的表达式。该表达式的值，变量数，以及运算符被放置在一个栈中，在适当的时间执行。

执行

当开始实际执行适当的移动时，这些数据项都从堆栈中取出，并且执行实际的操作。在==语法的情况下，此时该值被简单地赋值给变量。在其他形式的情况下（&=，|=和^=），读取该变量，在变量值和表达式的值之间执行逐位布尔运算（分别是 AND、OR、XOR），并将结果写回到变量。

特殊的布尔功能

这些布尔赋值运算符与==语有细微的差别。考虑两个语句作用于一个 8 位的

M-变量，它试图使所有的奇数位为 1，而保持偶数位为：

M50 == M50＆$AA

M50＆=$AA

当 M50 被读取用于操作时，在两个语句之间的区别是明显的。在第一种情况下，在程序中当该语句第一次被评估求值时，读取 M50 变量。在第二种情况下，当操作被取出堆栈时，在变量被写入之前，读取变量。在该第二种技术中，一些其它的任务改变 M 变量的值是不可能的。

其他变量的同步赋值

只有 M 变量可以在这些同步赋值中使用，但 M 变量可以赋值给任何其他变量（I、P 或 Q）的寄存器，所以这些同步赋值也可以有效地使用于其它变量类型。请参考详细的内存映射获得其他变量的地址。

局限性

这些同步赋值有几个限制，用户必须知道：

有效形式：首先，这些语句可能不会用于任何的指轮-复用器端口的 M 变量形式（TWB、TWD、TWR 或 TWS）。布尔赋值（＆=、|=、^=）不能用于任何两倍宽度的 M 变量形式（D、L 或 F）。

队列限制：挂起的同步输出必须存储在有限大小的队列中。如果没有启用（见下文）的特殊前瞻缓冲区，对于所有坐标系统队列固定为 256 个字，存储每个赋值需要两个字。全局变量 I68，它控制激活坐标系统的数目，决定这个队列有多少被赋值给每个坐标系。

下表显示了对于 I68 的不同的值，多少字的队列赋值给每个激活的坐标系，对于使用和不使用刀具半径补偿的每个移动又能进行多少赋值。（刀具半径补偿要求运动程序提前一个额外的移动，因此减少了每个移动的数量）。请注意，每个赋值需要两个字，每个移动是一个额外字。

I68 值	激活的最高编号的坐标系统	每个坐标系同步 M 变量堆栈	每个移动的最大同步 M 变量赋值，无刀具补偿	每个移动的最大同步 M 变量赋值，有刀具补偿
0	坐标系 1	256 字	63	42
1	坐标系 2	128 字	31	20
2-3	坐标系 3-4	64 字	15	10
4-7	坐标系 5-8	32 字	7	4
8-15	坐标系 9-16	16 字	3	2

如果您使用的是特殊的多块前瞻功能，待处理的同步赋值的存储需求是显著的。为此，特殊前瞻缓冲区允许你明确的预留空间用于挂起的同步输出以及挂起的移动段。如果给出下面的命令定义一个前瞻缓冲区：

＆1 DEFINE LOOKAHEAD 500,100

那么坐标系 1 的一个前瞻缓冲将被定义足够大，可以为坐标系的每个电机存储 500 个移动段（每一个 Isx13 的编程时间），加上 100 个同步的 M-变量赋值。

14-11 比较符

比较符计算两个值（常量或表达式）之间的关系。它是用来确定一个运动或 PLC 程序的条件的真值。Turbo PMAC 的有效的比较符是：

= （等于）
!= （不等于）
> （大于）
!> （不大于；小于或等于）
< （小于）
!< （不小于；大于或等于）
～ （约等于---1 以内）
!～（不是约等于 - 至少差 1）

这些详细描述在软件参考手册中的数学功能中。

注意：

<=和>=是无效的 Turbo PMAC 比较符。他们分别用比较符>和!<代替。

14-12 条件

一个条件可以被用来控制运动或 PLC 程序的程序流程。它被评估为真或假。它可用于在 IF 分支语句或 WHILE 循环语句。Turbo PMAC 支持简单和复合条件。

注意：

一个命令行的条件--IF 或 WHILE--必须用括号括起来。

简单条件

一个简单的条件由三部分组成：

{**表达式**}{**比较符**}{**表达式**}

如果由比较符定义的两个表达式之间的关系是有效的，那么条件为真；否则，条件为假。在命令中的简单条件有：

WHILE（1<2） （**总是为真**）
IF（P1>5000）
WHILE（SIN（P2-P1）!> P300/1000）

请注意，括号是必需的，将条件括起来。

注意：

不像在一些编程语言中，一个 Turbo PMAC 条件可能不只是一个值，评估为零或非零（如 IF（P1）是无效的）。必须明确地用两个表达式和一个比较符表示一个条件。

复合条件

复合条件是一系列由逻辑运算符 AND 和 OR 连接起来的简单条件。复合的条件是对简单条件的值由布尔代数运算进行评估的。在 Turbo PMAC，AND 的执行优先级高于 OR（也就是，OR 对由 AND 连接的简单条件块进行运算）。发现一个假的简单条件后，Turbo PMAC 将停止评估复合的条件。复合条件在命令行中的例子有：

```
IF (P1>-20 AND P1<20)
WHILE (P80= 0 OR I120>300 AND I120<400)
IF (Q16!<Q17 AND Q16!> Q18 OR M136<256 AND M137<256)
```

注意：

一个单行的复合条件包含的简单条件不能用括号分隔。例如，IF ((P1>-20) AND (P1 <20)) 是一个非法条件，将被拒绝为非法语法。

单行条件操作

在 Turbo PMAC 运动程序（而不是在 PLC 程序），对真条件执行的操作可以与条件本身放在同一行上。在这种情况下，不需要 ENDIF 或 ENDWHILE 标记条件操作的结束；该行的末尾被自动标记为条件操作的结束。这种形式的例子有：

```
IF (P1<0) P1=0
WHILE (M11=0) DWELL 10
```

在 Turbo PMAC 旋转程序缓冲区中只允许单行的条件语句。多行的条件是不允许的，因为它不能保证要跳跃到的行位于旋转缓冲区中。

多行条件

在 Turbo PMAC PLC 程序（而不是在运动程序）中，允许复合条件在多个程序行。条件的第一行必须以 IF 或 WHILE 开头；条件的下面几行必须以 AND 或 OR 开头。一个程序行中，简单和复合条件始终在不同行的条件进行组合之前进行评估。在多行的条件之间，AND 的优先级高于 OR。一个单行的条件被发现为假后，

Turbo PMAC 将停止评估一个多行的 AND 条件。

一个例子是：

```
IF（M11=1 OR M12=1）
AND（M13= 1 OR M14=1）
...
```

14-13 定时器

每个激活的坐标系（他们的编号从 1 到 I68+1）有两个定时器变量：Isx11 和 Isx12。这两个 24 位的寄存器是通用定时器供用户程序使用。每个伺服周期，Turbo PMAC 递减他们一次。通常写入一个等于等待时间的值，单位是伺服周期。然后程序等待寄存器变得小于 0。该寄存器将继续向下计数，直到他们达到-2^{23}（-8,388,608）。他们不会翻转回正值。

由于这些定时器以伺服周期为单位，而大多数用户会选择以毫秒为单位来工作，所以必须完成一个转换。从毫秒转换到伺服周期，乘以2^{23}（8,388,608）并除以 I10 的值。

一个坐标系的定时器变量可以被 Turbo PMAC 上的任何任务使用，包括运行在其它坐标系统的运动程序。

例如：

在一个 PLC 程序，要打开一个输出，保持固定数目的毫秒：

```
M1=1                          ；打开机器输出 1
I5111=125*8388608/I10         ；设置定时器为 125 毫秒，以伺服周期
WHILE（I5111>0）              ；等待计数器计数下降到零
ENDWHILE
M1=0                          ；关闭机器输出 1
```

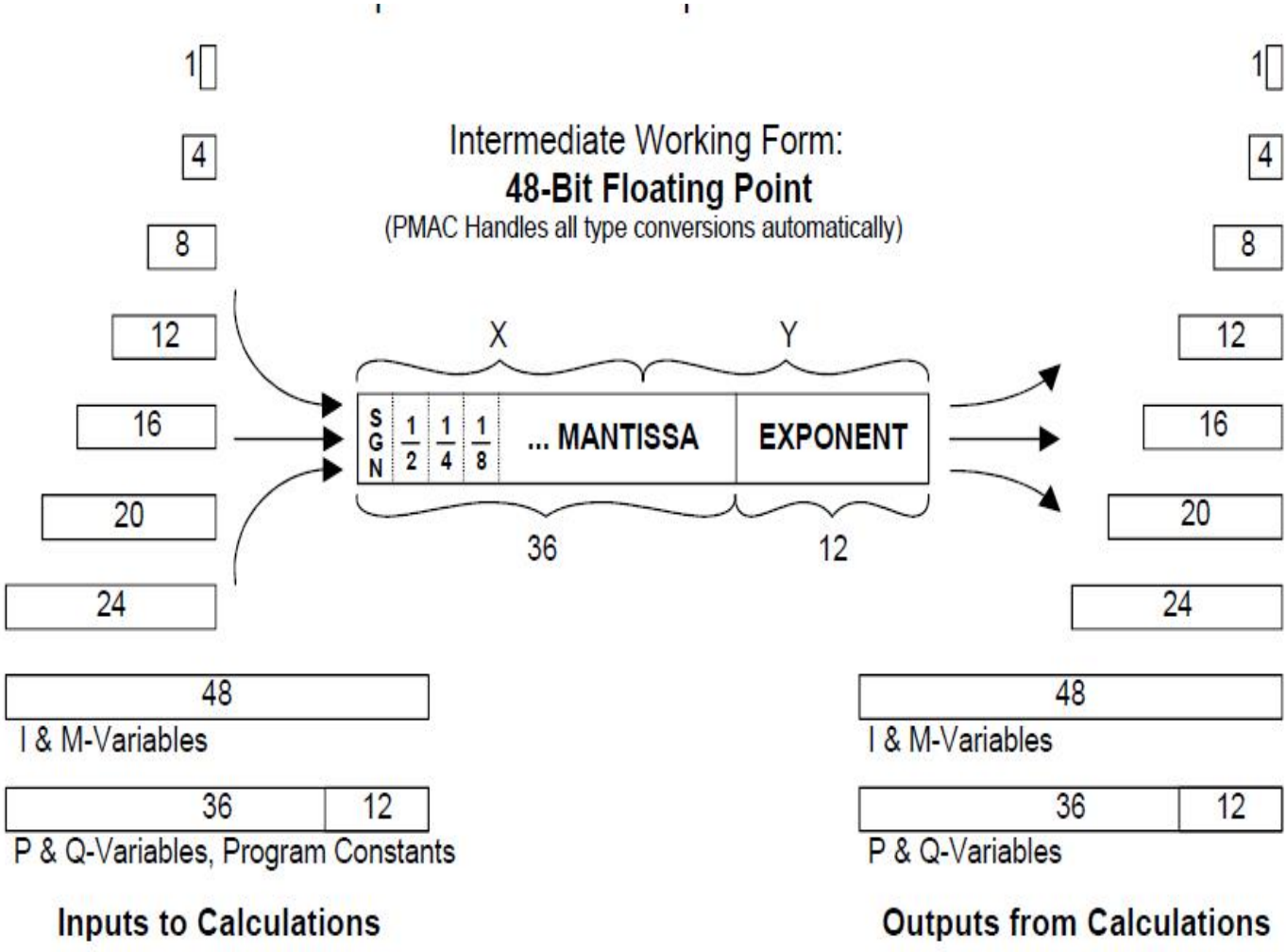
14-14 计算的注意事项

当 Turbo PMAC 正在进行计算一个 PLC 程序、运动程序或在线时，它采用 48 位的浮点格式用于计算的中间形式。这使得 Turbo PMAC 可以自动在不同的数值格式之间进行转换，并使其能够对 P 和 Q-变量进行按位运算，即使它们是浮点值。

将一个数字转换成 48 位格式的过程是非常快的，而且在大多数的 Turbo PMAC 的应用中并不明显。然而，对于计算要求苛刻的应用，跳过转换步骤可以帮助提高 Turbo PMAC 的速度和效率。在这种应用中，使用 P、Q 和 L-（长）格式的 M 变量跳过转换步骤（它们已经是 48 位的浮点格式），并比其他变量类型计算得更快。

PMAC 程序的数学计算

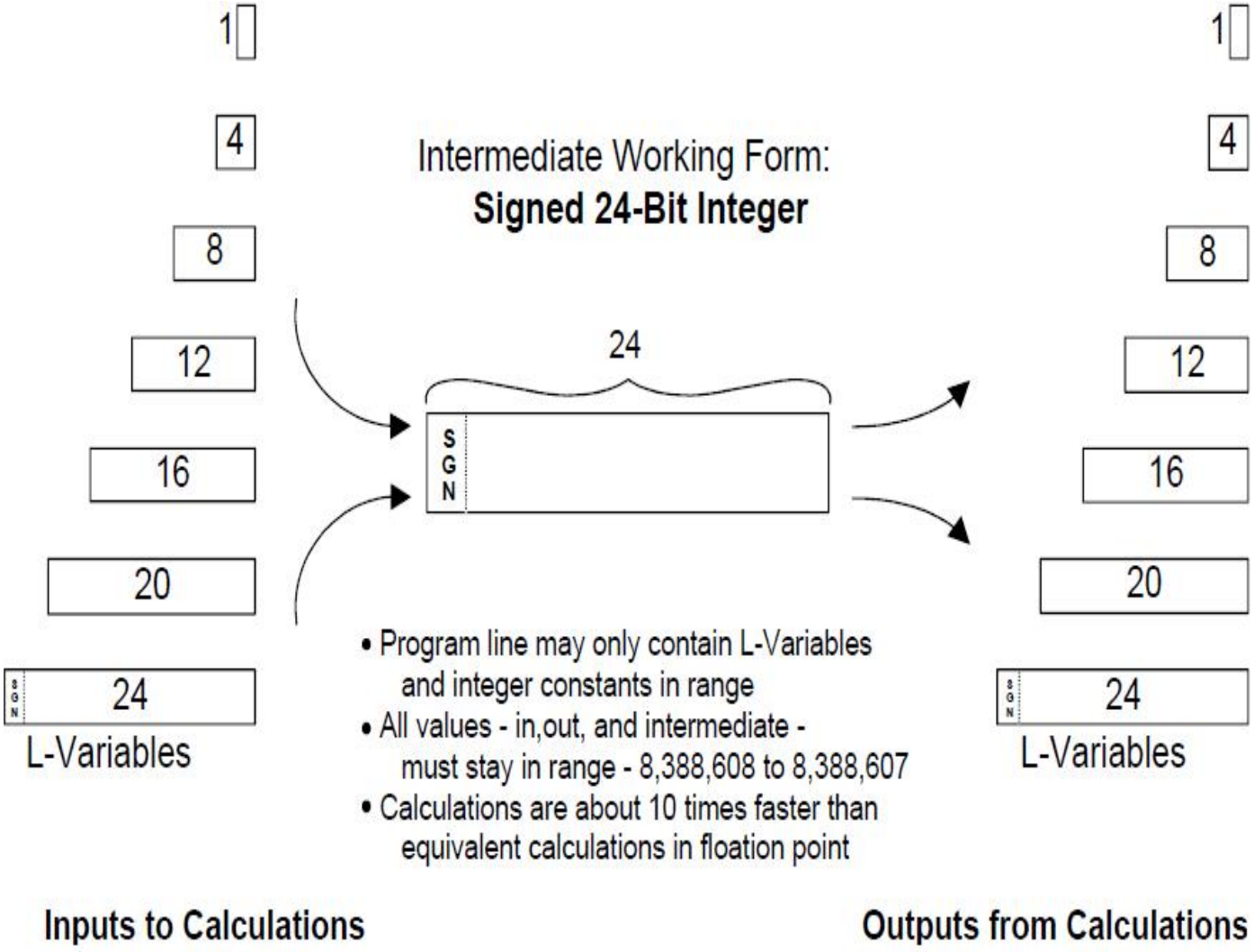
对于所有的运动和 PLC 程序 除了定点的编译的 PLC 行



然而，当 Turbo PMAC 在一个编译 PLC 程序（PLCC）中用 L-变量进行计算时，它使用 24 位的定点格式用于计算的中间形式。这使得 Turbo PMAC 可以以极快的速度执行计算。L 型变量的计算比使用中间浮点数据格式的计算快 10 倍左右。

PMAC 程序数学计算

定点的编译的 PLC， 只是程序行



第十五章 设置一个坐标系

一旦你已经设置好电机，使他们得到了良好整定，并进行控制的慢跑和回零搜索动作，你会想对电机设置一个或多个坐标系统，使您可以运行运动程序。

Turbo PMAC 有几种方法可以协调多个运动，无论它们是否都在 Turbo PMAC 的直接控制下。取决于用户的情况和需要，可以实施下面的协调策略。

15-1 什么是坐标系？

Turbo PMAC 中的坐标系统是一组的一个或多个用于同步运动的目的的电机。一个坐标系统（即使只有一台电机）可以运行一个运动程序；而电机本身不能。Turbo PMAC 最多可以有 16 个坐标系，地址为＆1 到＆16，以一种灵活的方式（例如，1 或 2 个电机的 16 个坐标系统，9 个电机 1 个坐标系统，8 个电机的 4 个坐标系统等）。

一般来说，如果你想某些电机以协调的方式移动，把它们放在同一坐标系中。如果你希望他们彼此独立移动，把它们放在不同的坐标系。不同的坐标系统可以在不同的时间（包括重叠的时间）执行独立的程序，或者甚至可以在不同的（或重叠）时间运行相同的程序。

必须首先建立一个坐标系，通过为该坐标系中的轴分配电机。对于电机（执行器）和轴（刀具坐标）之间的简单关系，这是通过在线的命令称为“轴定义语句”（见下文）完成的。对于更复杂的关系，它是通过编写描述关系的特殊的运动子程序来完成（覆盖在下面的部分）。

一个坐标系必须至少有一个电机分配给系统中的轴，否则它不能运行运动程序，甚至是该程序的非运动部分。当编写一个坐标系的一个程序时，如果需要多台电机同时运动，他们的行动命令简单地放在同一行上，这样将运动进行协调。

15-2 什么是轴？

一个轴是一个坐标系中的一个元素。它可以被看作是一个工具的坐标，或相对于刀具的机械坐标。一个轴线通常是类似于一个电动机，但不是一回事。一个轴是由字母指定。一个坐标系中可以有多达九个独立的轴，从 X、Y、Z、A、B、C、U、V 和 W 中选择（也可以一个轴分配多个电机）。通常情况下，定义一个轴是通过将其指定给电机，用一个比例因子和偏移量（X、和 Z 可以被定义为三个电机的线性组合， U、V、W 也可以的）。以这种方式定义的轴线，与其相关联的变量是缩放的浮点值。

单-电机的轴

在绝大多数的情况下，电机和轴之间有一个一一对应。也就是说，在坐标系中单个电机被分配到单个的轴。即使在这种情况下，然而，匹配的电机和轴也不完全同步。该轴的刻度为工程单位，并且只处理指令位置。除 PMATCH 函数，计算只能从轴位置指令到电机命令位置，而不是其他方式。

多电机轴

在一个坐标系统中，一个轴可以分配多个电动机。这常见于龙门系统中，其中在交叉件两端的电机总是做同样的动作。通过分配多个电机到同一轴线上，程序中单个编程的轴移动可以给多个电机相同的指令移动。这通常在两个电机上完成，但 Turbo PMAC 以这种方式可使用多达八个电机。记住，电动机仍然有独立的伺服环路，并且实际的电机位置不一定是完全一样的。

以这种方式协调并行龙门电机一般是优于使用主/从技术（可以通过 Turbo PMAC 上的位置跟随功能做到，描述在本手册的《同步 Turbo PMAC 到外部事件》的一节中）。在主/从技术中，在编码器上测得的主机的实际轨迹，以及所有的干扰和量化误差，都变成为从机的命令运动轨迹，而其实际轨迹会有更大的误差。从属电机的命令轨迹的粗糙度使得很难或甚至不可能使用前馈，这将引入一个滞后。事实上，如果主机有一个干扰，从机将会看到它，并试图与之相匹配，但如果从机得到一个干扰，主机将无法看到它。

具有紧凑机械联接装置的龙门架电机，必须小心其启动和回零。在一般情况下，电机通电时，没有理想地彼此对准。通常的程序是，对带从电机的一台电机进行回零移动搜索，其次做一个足够远的偏移，使得第二个电机（即从电机）知道它如何到达其回零触发位置。接着第二个电机作为主机，并告知它做一个回零搜索移动，而第一个电机成为它的从属电机。这将使第一个电机稍微偏离其回零位置，现在可以告诉它移动到那里，只需一个 J =0 命令。然后关闭从属电机，这样我们就可以通过关节轴的命令相同地控制电机。

虚拟轴

一个坐标系中的轴可以不与电机连接（即虚拟轴），在这种情况下，该轴的编程的移动没有引起运动，但事实上，对该轴进行移动编程会影响其他轴和电机的移动。例如，如果一个轴需要正弦轮廓，做到这一点最简单的方法是设置第二个虚拟轴，让它循环地进行插值移动。

15-3 轴定义

坐标系是通过轴定义语句建立的。定义轴是通过匹配一个电动机（它有编号）到一个或多个轴（这是由字母指定）。

匹配电机到轴

最简单的轴定义语句大概是＃1 -> X。这只是将电机＃1 分配给目前定址坐标系的 X 轴。当该坐标系中执行 X 轴移动时，电机＃1 会进行移动。

缩放和偏移

轴定义语句还定义了轴的用户单位的缩放比例。例如，＃1->10000X 也匹配

电机＃1 到 X 轴，但此语句设置 10,000 个编码器计数为一个 X 轴用户单位（例如英寸或厘米）。此缩放功能几乎普遍使用。一旦在声明定义了该比例，你可以以工程单位编程轴而不需要再次处理缩放。

该语句＃1->10000X+20000 也设定轴的零点与电机零点（起始位置）的距离为 20,000 计数（2 个用户单元）。这种偏移很少使用。此外，轴的定义语句可以匹配电动机到笛卡尔坐标轴的线性组合（见下文），允许一个坐标系的旋转，或正交校正。

轴类型

一个轴可以有几个属性，具体如下。请注意，对于大多数轴功能，使用什么类型的轴，或者赋予它什么字母并不重要。然而，对于某些功能，仅可以使用特定的轴名称。

笛卡尔轴

笛卡尔轴是一组两个或三个轴中的其中一根，以便沿一个轴的移动是两个或三个电机的运动的线性组合。X、Y 和 Z 形成一组笛卡尔坐标轴；U、V 和 W 组成另一组。此外，有若干命令（NORMAL，圆弧移动），可以通过分别使用矢量 I、J 和 K 而引用 X、Y 和 Z 轴，。

为了使笛卡尔轴是多台电机的线性组合，用轴定义语句的扩展形式来完成。举例来说，可通过以下轴定义语句，从电机得到一个 30 度的轴旋转：

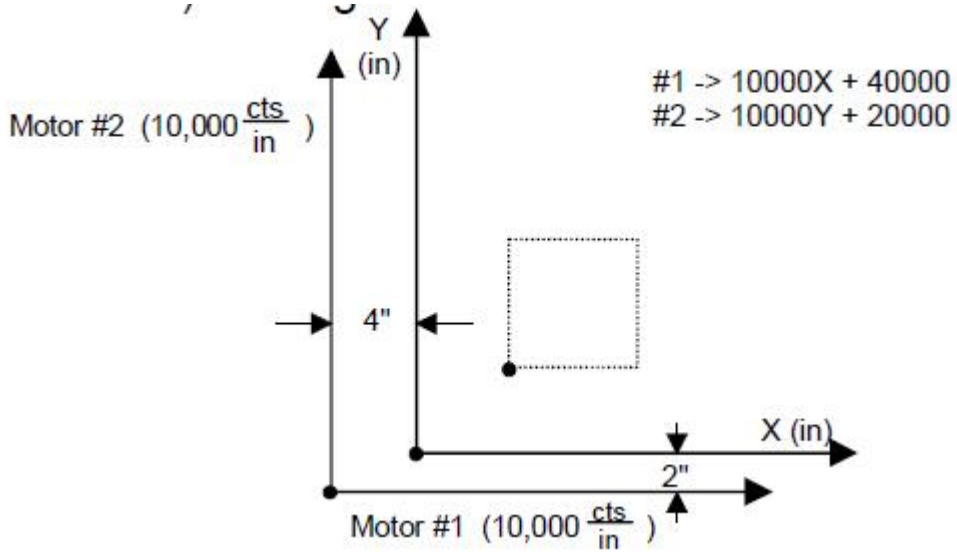
```
＃1 ->8660.25X-5000Y
＃2 ->5000X+8660.25Y
```

在这种情况下，一个 Y 轴（或 X 轴）移动的请求将同时引起两台电机＃1 和＃2 移动。

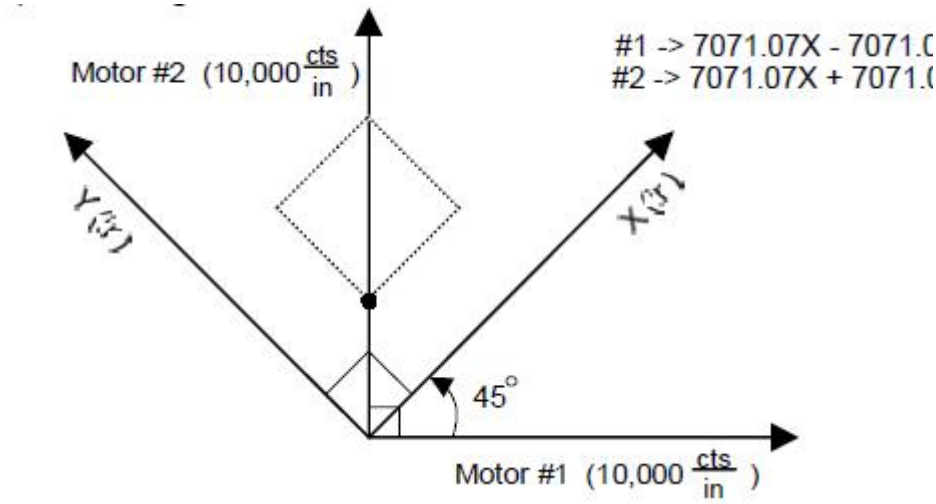
只有 X、Y 和 Z 笛卡尔坐标轴可用于 PMAC 的圆弧插补程序、刀具半径补偿程序以及矩阵轴转换程序。如果你想要在其它轴上进行圆弧插补，可以通过在子程序中混合的短移动和三角函数来完成。

PMAC 坐标定义

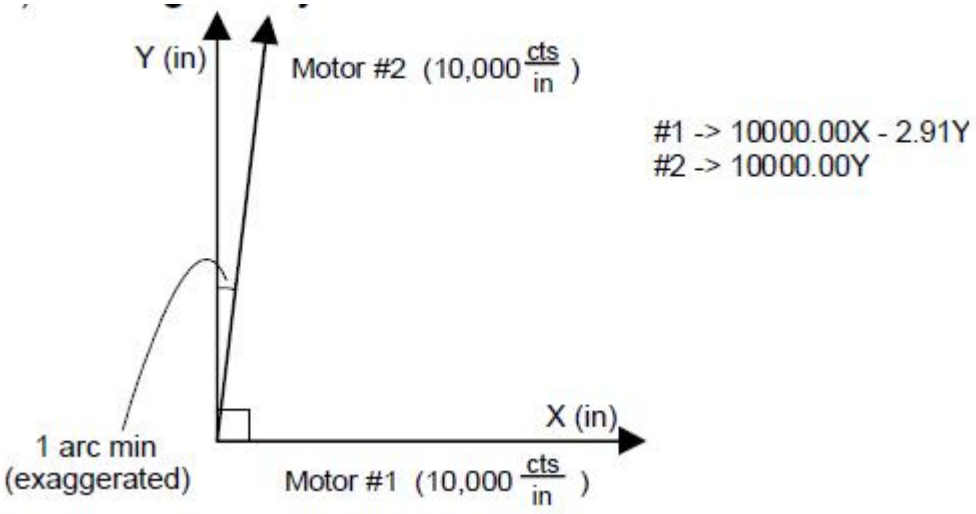
1）缩放和平移



2）缩放和旋转



3）正交校正



旋转轴

一个旋转轴允许翻转，但不能被分配到组合的电机。旋转轴必须命名为 A，B 或 C。翻转在技术上是一个电机的功能，通过电机 xx 的 Ixx27 指定，但是只能当电机被分配到一个旋转轴时才能操作。当在程序中指定一个绝对的轴移动时，翻转允许电机采取旋转范围内的最短路径或指定的方向到目的地。

向量进给轴

一个矢量进给轴是坐标系统的轴，可以对指定的进给移动进行计算。一个指定的进给移动时间是进给轴的矢量距离除以进给率本身（矢量距离是由勾股定理

计算为轴距离的平方和的平方根)。如果要在同一个语句中命令移动其他的轴，它们将在同样的计算时间内进行线性插值运算。

默认的进给轴是笛卡尔坐标轴 X、Y 和 Z。此设置可用 FRAX（进给轴）命令来改变。任何参与 CIRCLE 模式移动圆弧中的一个轴自动成为该移动的一个矢量进给轴，即使没有进行这样的明确声明。

从轴位置转换到电机位置

从技术上讲，从轴（刀尖）的位置到电机（关节或执行器）位置的转换被称为逆运动学的转换。在每一个编程移动，或者在分段移动的每一个移动段，Turbo PMAC 自动从编程的轴位置（以用户工程单位）转换为电机位置（：计数)。

在用轴定义语句定义的轴，Turbo PMAC 简单地将轴的值插入到定义语句的方程，并计算所得到的电动机的位置。(该轴定义语句因此是一种逆运动学方程)。在使用运动子程序定义的轴，Turbo PMAC 执行用户编写的逆运动学子程序来计算这些（见手册内的《坐标系统运动学计算》)。

从电机位置转换到轴位置

从技术上讲，从电机（关节或执行器）位置转换到轴（刀尖）的位置被称为正向运动学的转换。只有一种类型的计算，在其中 Turbo PMAC 自动执行这些正向运动学计算。这是在 PMATCH（位置匹配）函数中，该函数将从指令电机位置转换为指令轴位置。这只用于少数几种情况。

首先，当一个运动程序以 R（运行）或 S（步骤）命令开始时，Turbo PMAC 内部自动执行一个 PMATCH 命令来为第一个移动计算起始轴位置。在一个运动程序内，它通常假定前一个移动的端点是后续移动的起点，所以在每一个移动时并不做这些计算。然而，当启动一个程序时，它做这些计算，因为有一个电机可能已被独立移动的机会（如慢跑运动、开环动作、停在一个错误条件)；换句话说，轴不知道电机运动到哪里了，电机位置和轴位置可能匹配不正确。

其次，如果电机位置和轴位置之间的关系是在运动程序内改变（例如，改变位置跟随偏移模式，直接写入位置偏置或坐标轴缩放寄存器)，在下一个编程移动之前，发出一个 PMATCH 命令（使用 CMD“PMATCH”结构)。否则，下一个移动将无法正常执行。

对于一个用简单定义语句定义的轴，PMATCH 函数反转轴定义语句的方程，使用电机指令位置，并求解轴的指令位置。如果一个轴分配多个电机（例如＃1->10000X，＃2->10000X)，PMATCH 计算使用编号较低的电动机的指令位置。对用户编写的正向运动学子程序的轴（见下文)，该子程序自动执行 PMATCH 函数。

该PMATCH函数假设已在坐标系中完成每个电机的位置参考---无论是回零搜索移动或读取绝对位置传感器。每个电机都有一个“home complete status bit”（回零完成状态位)，如果上述任一项已经完成，该位会设定为真，并在应用层面进行检查。

有些用户希望读取瞬时电机位置寄存器（特别是电机实际位置）---Turbo PMAC 不会自动计算瞬时轴位置---并将其转换为坐标轴位置用于进行位置报告

的目的。这些正向运动学计算必须在应用层面完成，无论是在 Turbo PMAC（通常在 PLC 程序）或主机上。

15-4 坐标系统的运动学计算

Turbo PMAC 提供了结构，可以使复杂的运动学计算变得易于实施和执行。当在刀尖坐标和机构执行器（接头关节）的匹配位置之间有一个非线性数学关系时，需要进行运动学计算，特别是在非笛卡尔几何形状中。它们在机器人应用最常见，但也可用于其它非机器人的执行器类型中。例如，在有一或两个旋转轴的 4 轴或 5 轴的机床中，希望编程刀具尖端的路径，并让该控制器计算必要的电动机位置。

这种性能可以允许机器运动在刀尖的自然坐标中进行编程- 通常是笛卡尔坐标，无论机器的基本几何形状。运动例程被积分器嵌入在控制器中，而且操作对编程人员和机器操作员是不可见的。这些例程对机器可以是不变的，但通过参数化和/或逻辑，他们可以进行正常的变化，如刀具长度和不同的末端执行器。

在 Turbo PMAC 的术语，刀尖坐标系用于字母指定的轴，并具有用户指定的工程单位。关节坐标用于数字指定的电机，并且具有“计数”的原始单位。

注意：

PMAC 的标准轴定义语句处理关节电机和刀尖轴之间的线性数学关系。本节中涉及到更难的非线性关系的情况。

正向运动学计算使用关节位置作为输入，并将其转换为刀尖坐标。在刀尖坐标的编程移动序列的开始，需要进行这些计算，为第一个编程移动建立起始坐标。也可使用相同类型的计算用于报告刀尖坐标中执行器的实际位置--从关节传感器的位置进行转换得来。（Turbo PMAC 正向运动程序缓冲区不支持此位置报告功能，但可以用在 PLC 程序中使用与此目的功能相同的计算。）

逆运动学计算使用刀尖位置作为输入，并将他们转换为关节坐标。刀尖坐标中每个编程移动的终点需要进行这些计算，并且如果到终点的路径很重要，它们也必须在移动过程的周期性间隔中进行。

注意：

正式的机器人分析会在关节位置以及关节的执行器位置之间进行区别。通常，虽然这两个位置是相同的，但是存在一些情况例如当两个电动机差动地驱动一个关节，二者的差别就很大了。如果系统的关节位置和执行器位置之间有差别，运动学计算必须包括此差别，使执行器位置和刀尖位置之间一直这样，以关节位置作为一个中间步骤。本文档仅指关节位置，尽管在某些应用中这在技术上是指执行器的位置。

创建运动学程序缓冲区

Turbo PMAC 实现执行运动学计算，是通过特殊的正向运动学和逆运动学程序的缓冲区。每个坐标系统都可以有这些程序缓冲区，并且其中的算法可以自动地在规定的时间执行，称为运动程序的子程序。

创建正向运动学程序

在线的 OPEN FORWARD 命令将打开正向运动缓冲区用于输入定址的坐标系统。在线 CLEAR 命令清除该缓冲区的任何现有内容。随后，发送到 Turbo PMAC 的对 PLC 程序合法的任何程序命令（除了 ADDRESS，CMDx 和 SENDx）将被输入到打开的缓冲区。在线 CLOSE 命令停止进入缓冲区。

在执行任何正向运动程序之前，Turbo PMAC 会自动将坐标系每个电机 xx 的当前指令电机位置放在全局变量 Pxx 中。这些都是浮点值，单位是计数。然后程序可以使用这些变量作为“输入”到计算。

在执行完正向运动程序之后，Turbo PMAC 将采取 Q1-Q9 的值，以用户的工程单位，并将他们复制到坐标系的 9 个轴目标位置寄存器中。在那里，他们被用作其后的第一个编程移动的起始位置。下面的表显示了每个变量影响其位置的轴，以及每个寄存器的建议 M 变量编号（所列用于调试目的）。

轴位置 Q 变量	轴字母	目标寄存器建议 M 变量	轴位置 Q 变量	轴字母	目标寄存器建议 M 变量	轴位置 Q 变量	轴字母	目标寄存器建议 M 变量
Q1	A	Msx41	Q4	U	Msx44	Q7	X	Msx47
Q2	B	Msx42	Q5	V	Msx45	Q8	Y	Msx48
Q3	C	Msx43	Q6	W	Msx46	Q9	Z	Msx49

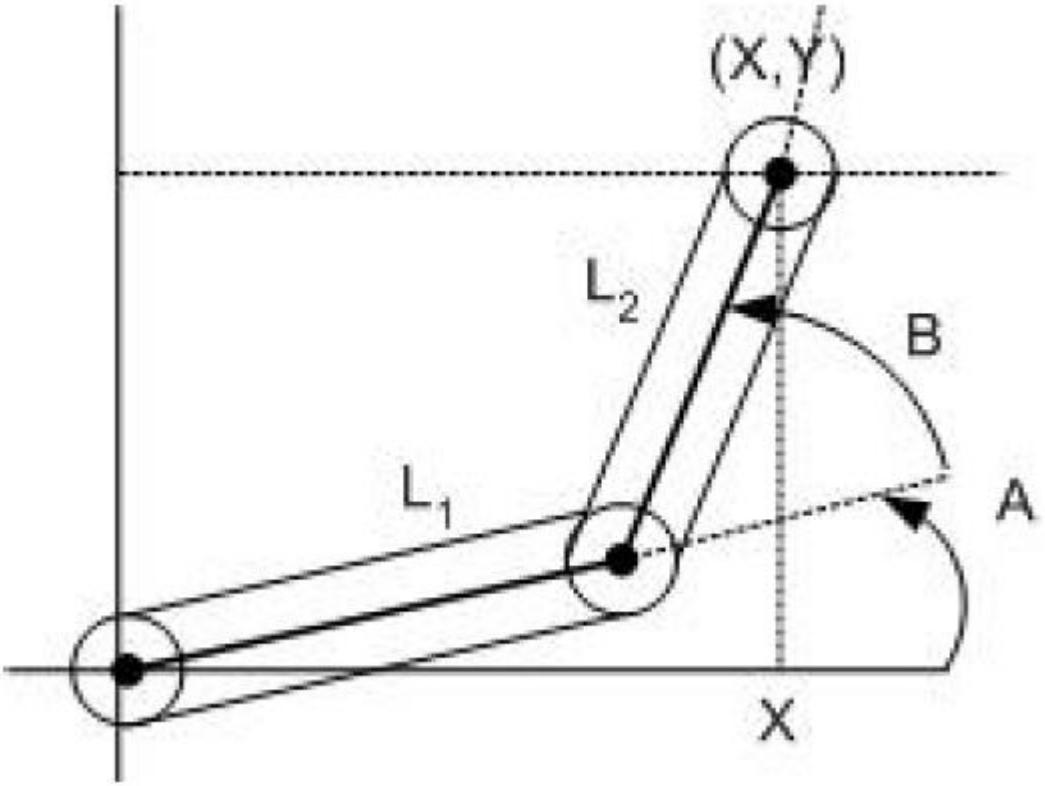
正向运动程序的基本目的是采用在电机的 P1- P32 中发现的关节位置值，计算匹配的刀尖坐标值，并将他们放置在 Q1-Q9 的变量中。

检查你的正向运动学程序是一个好主意，以确保已为每个电机正确建立位置基准，无论是通过回零搜索移动还是绝对位置读取。这可以通过评估每台电动机的回零完成状态位来完成;如果尚未完成建立参考，可以设置运行时间错误位(见示例)。

保留的变量

如果一个系统中使用运动学计算，全局变量 P1- P32 和坐标系变量 Q1- Q10 不能用于任何其他目的，因为 Turbo PMAC 在执行运动学例程时将自动地写入这些变量。(Q10 是用于区分涉及速度计算和不涉及速度计算的逆运动学计算，如下所述)。如果使用了涉及 PVT 模式移动的反向运动学计算，附加地，全局变量 P101-P132 和坐标系变量 Q11-Q19 不能用于任何其他目的，因为 Turbo PMAC 将在执行逆运动学例程的速度部分时自动地写入这些变量。

例子



以一个 2 轴的肩-肘关节机器人为例，上臂长度为 400mm（L1），下臂的长度为 300mm（L2）。肩关节（A）和肘关节（B）为每度 1000 计数的分辨率。当两个关节都在他们的零度位置时，两个链路都沿 X 轴延伸。正向运动学方程为：

$$X = L_1\cos(A) + L_2\cos(A + B)$$

$$Y = L_1\sin(A) + L_2\sin(A + B)$$

对坐标系统 1 的 Turbo PMAC 正向运动的程序，将电机 1 的肩关节角度和电机 2 的肘关节角度转换为 X 和 Y 刀尖坐标（以毫米为单位），要实施这些方程，可使用下面的设置和程序：

```
; 设置程序
I15=0                          ; 三角函数的计算,单位为度
&1                             ; 寻址坐标系 1
M145-> Y：$0000C0,10,1         ; 电机 1 回零完成位
M245-> Y：$000140,10,1         ; 电机 2 回零完成位
M5182-> Y：$00203F,22,1，      ; 坐标系 1 运行时间错误位
Q91=400                        ; L1
Q92=300                        ; L2
Q93=1000                       ; A 和 B 的每度的计数

; 重复执行正向运动程序缓冲区
&1 OPEN FORWARD                ; 正向运动学 CS 1
CLEAR                          ; 清除现有内容
```

```
IF (M145=1 AND M245=1)          ; 正确的位置引用?
   Q7= Q91* COS (P1/Q93) + Q92* COS ((P1 + P2) / Q93)    ; X 位置
   Q8= Q91* SIN (P1/Q93) + Q92* SIN ((P1 + P2) / Q93)    ; Y 位置
ELSE                            ; 无效；停止操作
M5182= 1                        ; 设置运行时间错误位
ENDIF
CLOSE
```

正向运动学程序必须计算坐标系中所有坐标轴的轴位置，不管逆运动学程序是否计算所有的电机位置（见下文）。举例来说，如果这条手臂在刀尖坐标系有一个垂直轴，而坐标系 1 的正常的轴定义语句：**#3 ->100Z**（每毫米 100 个计数--电机和轴之间是一个线性关系），上述程序仍需要一个命令行，如 **Q9= P3/100**，执行该电机/轴的正向运动学计算。

注意：

如果正向运动的算法是不正确的，不产生逆运动学算法的一个真正的数学反转，那么执行完正向运动学算法后，在执行的第一个移动的开始会有一个突然的和潜在的危险跳跃。请确保在开发初期，Ixx11 故障跟随错误限制设置得尽可能紧密，以确保任何大的错误都会导致一个跳闸，并不会导致剧烈的运动。

迭代求解

有些系统，特别是平行连杆机构，如斯图尔特平台（六足），对正向运动学方程没有合理的封闭形式解，并且需要反复的数值求解。通常情况下，这些情况是由正向运动程序的一个循环 **WHILE... ENDWHILE** 构造处理的。不允许无限期的循环---如果该解没有收敛在预期的周期，程序应停止（见逆运动学方程，下面，如何停止程序的例子）。

在这种情况下，最好是将 I11 程序计算延迟变量保持在其默认值 0，这样可以根据需要进行计算。如果 I11 大于 0，并且正向运动学计算加上第一个移动计算不能在 I11 毫秒内完成，Turbo PMAC 将以一个运行时间错误停止该程序。在任何情况下，如果正向运动学计算占用的时间大于 25 毫秒，可能会跳闸看门狗定时器。

位置报告的正向运动学

正向运动学计算的另一个用途是位置报告的功能，在任何时间读取实际关节位置，并将其转换为刀尖位置用于报告。在 Turbo PMAC 的正向运动程序缓冲区不支持此功能。（将程序同时用于初始位置计算和位置报告有可能导致潜在的重叠使用和寄存器冲突。）

如果应用程序要求 Turbo PMAC 来进行正向运动学计算用于位置报告以及建立初始刀尖位置，应该使用一个 PLC 程序进行位置报告的计算。可使用下面的 PLC 程序用于“肩-肘关节”机器人的位置报告功能：

```
;实际位置寄存器的 M 变量定义
M162->D:$8B              ;电机 1 的实际位置
M262->D:$10B             ;电机 2 的实际位置

;用于位置报告的正向运动 PLC 程序的缓冲区
OPEN PLC10                          ;坐标系 1 的正向运动学
CLEAR                               ;清除现有内容
P51= M162/（I108*32* Q93）    ;实际 A 位置（度）
P52= M262/（I208*32* Q93）    ;实际 B 位置（度）
Q27= Q91* COS（P51）+ Q92* COS（P51+ P52）;实际的 X 位置
Q28= Q91* SIN（P51）+ Q92* SIN（P51+ P52）;实际 Y 位置
CLOSE
```

建立反向运动学程序

在线的 **OPEN INVERSE** 命令打开反向运动学缓冲区用于输入定址坐标系。在线 **CLEAR** 命令清除该缓冲区的任何现有内容。随后，任何发送到 Turbo PMAC 的 PLC 程序的合法数学或逻辑程序命令(这不包括 **ADDRESS**、**DISPLAY**、**CMDx** 或 **SENDx**）将被输入到打开的缓冲区。在线 **CLOSE** 命令停止进入缓冲区。

在执行任何逆运动学程序之前，Turbo PMAC 将每个轴的当前轴目标位置放在 Q1 – Q9 的变量中。这些都是浮点值，以工程单位。然后程序可以使用这些变量作为“输入”到计算。下面的表显示了每个轴的变量：

轴位置 Q 变量	轴字母	轴位置 Q 变量	轴字母	轴位置 Q 变量	轴字母
Q1	A	Q4	U	Q7	X
Q2	B	Q5	V	Q8	Y
Q3	C	Q6	W	Q9	Z

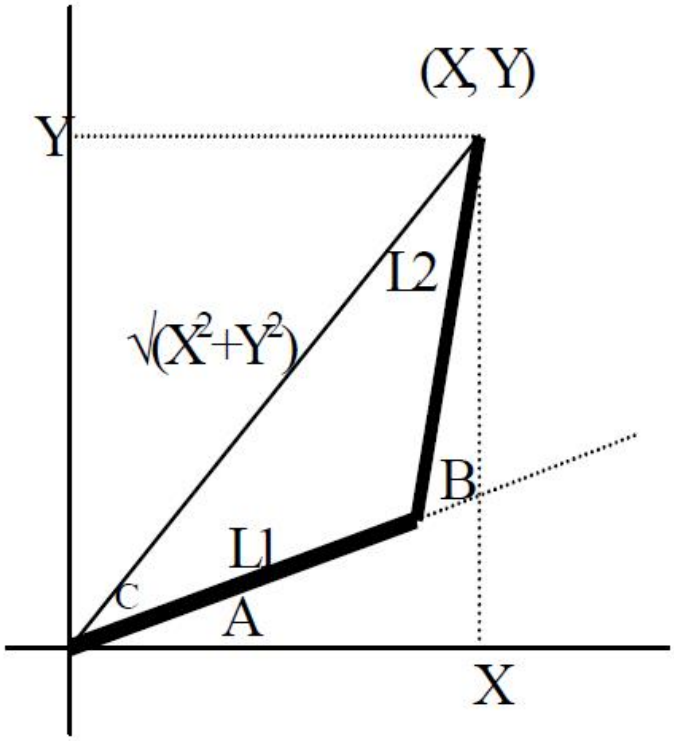
在执行完任何反向运动学程序后，Turbo PMAC 将读取这些变量 Pxx(P1 – P32）的值（对应于坐标系中的电机 xx)，通过使用 **#xx->I** 的轴定义语句。这些都是浮点值，而且 Turbo PMAC 能够他们以“计数”为原始单位。Turbo PMAC 将这些值自动复制到目标位置寄存器（建议 M 变量 Mxx63)，在那里它们被用于这些电机的精插补。

在坐标系中也有其它电机未定义为逆运动学的轴；这些电机直接从轴定义语句获取其位置值，并且不受逆运动学程序影响。

那么，逆运动学程序的基本目的，是采用坐标系统所用轴的 Q1- Q9 中的刀尖位置值，计算匹配的关节坐标值，并将他们写入 P1 – P32 中的变量。

例子

继续我们的两轴肩-肘关节机器人的例子，而且为了简单起见，限制 B 值为正（右手臂的情况)，我们可以写逆运动学方程如下：



$$B = +\cos^{-1}\left(\frac{X^2 + Y^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

$$A + C = a\tan2(Y,X)$$

$$C = +\cos^{-1}\left(\frac{X^2 + Y^2 - L_1^2 - L_2^2}{2L_1\sqrt{X^2 + Y^2}}\right)$$

$$A = (A + C) - C$$

在坐标系统 1 的 Turbo PMAC 逆运动学程序中，这些等式，将以毫米为单位的 X 和 Y 刀尖坐标转换为电机 1 的肩关节角和电机 2 的肘部角度，要实现这些，可以使用下面的程序。系统常数 Q91、Q92 和 Q93 与上述正向运动学程序中的是相同的。

```
;设置程序
&1
#1->I                          ;电机 1 分配给坐标系 1 的逆运动学轴
#2->I                          ;电机 2 分配给坐标系 1 的逆运动学轴
M5182-> Y：$00203F，22,1        ；CS1 运行时间错误位

;预计算的附加的系统常数
Q94= Q91*Q91+Q92*Q92          ; L1^2+ L^2
Q95= 2*Q91*Q92                ; 2*L1*L2
Q96= Q91*Q91-Q92*Q92          ; L1^2 - L2^2
```

```
;逆运动学算法被重复执行
&1 OPEN INVERSE                ; 逆运动学的 CS 1
CLEAR                          ;清除现有内容
Q20= Q7*Q7+ Q8*Q8              ; X^2+ Y^2
Q21=（Q20-Q94）/ Q95           ; COS（B）
IF（ABS（Q21）<0.9998）        ;有效的解 w/1 度 边际?
   Q22= ACOS（Q21）            ; B（度）
   Q0= Q7                      ; X 为 COS 的 ATAN2 参数
   Q23= ATAN2（Q8）            ; A+ C= ATAN2（Y，X）
   Q24= ACOS（(Q20+ Q96）/（2* Q91* SQRT（Q20））)   ; C（度）
   Q25= Q23-Q24                ; A（度）
   P1= Q25* Q93                ;电机 1=1000A
   P2= Q22* Q93                ;电机 2=1000B
ELSE                           ;无效；停止操作
   M5182= 1                    ;设置运行时间错误位
ENDIF
CLOSE
```

示例的注释：

- 通过选择正的反余弦解，我们会自动选择右臂的情况。在一个更一般的解，我们必须根据某些准则，选择是使用正值或负值。
- 通过将更多的运算组合到一个赋值语句中可以提高计算效率。在这里拆分计算是为了解释清楚。
- 这个例子没有使用执行程序允许的替换宏来为变量替换有意义的名称。强烈建议在复杂的应用中使用这些替换宏。
- 在没有逆运动学的解时，此示例会停止程序。它通过设置坐标系统的“运行时间错误”状态位，这会导致 Turbo PMAC 停止执行运动程序，并发出中止（A）命令。可使用其他策略解决这一问题。

如果这个机器人在刀尖有一个垂直轴，电机和轴之间的关系可以用一个普通的线性轴定义语句（例如＃3->100Z，每毫米 100 个计数）定义，不使用特殊的逆运动学程序来计算电机的位置。或者，电机可以被定义为一个逆运动学轴线（＃3 -> I），而电机位置可在逆运动学程序中进行计算（例如 Q3 = Q49* 100，设定电机 3 距离 Z 轴的位置，每单位是 100 个计数）。

旋转轴翻转

如果系统中的旋转的逆运动学轴可以“翻转”，逆运动学程序必须明确地处理翻转计算。A、B 和 C 轴的自动翻转功能与 Ixx27 并不适用于逆运动学轴。妥善处理翻转的关键是采取新值和旧值之间的差值，并确保这种差值是在±180°范围内。这可以在 Turbo PMAC 上通过’%’模（余数）运算符来完成。这个差值随后被添加到旧值上。在数学上，该方程是：

$$\Delta\theta = \left(\theta_{\text{新临时值}} - \theta_{\text{旧}}\right)\%(-180)$$

$$\theta_{\text{新值}} = \theta_{\text{旧}} + \Delta\theta$$

在 Turbo PMAC 上，当取模操作通过一个负操作数‘-n’（如-180）完成时，

其结果总是在±n 范围内。

例如，如果在上述例子中 A 轴可以翻转，行 Q25= Q23-Q24 可以被替换为：

```
Q25= P1/Q93+（Q23-Q24-P1/Q93）%-180                    ;处理翻转情况
```

值（P1/Q93）是$\theta_{旧}$，来自逆运动学的前一个周期或来自最初的正向运动学；

值（Q23-Q24）是$\theta_{新临时值}$，单位都是度。

速度计算标志

在每一个移动模式中除了 PVT 模式，Turbo PMAC 将坐标系的变量 Q10 自动设置为 0，作为逆运动学程序的一个标志而不计算速度值。如果想要同时使用 PVT 模式和其他模式，须在逆运动学程序中明确地评估计算 Q10（见下文）。

迭代解

有些机器人的几何形状没有封闭形式的逆运动学解，并且需要迭代数值解。通常，这些情况是由逆运动学程序的一个循环 WHILE... ENDWHILE 构造处理的。在逆运动学程序中多次执行 WHILE 循环也不禁用混合，就如同在他们的主运动程序中（由于双跳回规则），但是过分的迭代可以导致无法在规定的时间内完成计算。这将导致一个运行时间错误，自动中止程序。

PVT 模式的反向运动学程序

在逆运动学程序中，Turbo PMAC 还可以支持将速度从刀尖空间转换到关节空间，以使用 PVT 模式的运动学计算。对于 PVT 模式移动，完成位置计算与任何其他移动模式是一样的。还必须完成计算一组额外的速度转换。

当激活运动学（Isx50=1）并执行 PVT 模式移动时，在执行每个反向运动学程序之前，Turbo PMAC 自动将 PVT 语句的指令轴速度值写入坐标系统的变量 Q11 - Q19 中。这些都是有符号的浮点值，以工程速度的单位，通过工程长度/角度单位和坐标系的 Isx90 时间单位（例如毫米/分钟或度/秒）定义。下表显示了用于每个轴的变量：

轴速度 Q 变量	轴字母	轴速度 Q 变量	轴字母	轴速度 Q 变量	轴字母
Q11	A	Q14	U	Q17	X
Q12	B	Q15	V	Q18	Y
Q13	C	Q16	W	Q19	Z

Turbo PMAC 也将 Q10 设置为 1，在这种模式下作为一个逆运动学程序的标志，它应该使用这些轴（刀尖）的速度值来计算电机（关节）速度值。

在这种模式下，在执行完反向运动学程序后，Turbo PMAC 将读取（定义为反向运动学轴线（＃XX-> I）的每个电机 xx 的）变量 P1xx（P101 - P132）的值。这些都是浮点值，Turbo PMAC 期望他们的单位是每 Isx90 毫秒的计数。Turbo PMAC 将使用它们作为电机（关节）的速度值连同 Pxx 的位置值来为电机创建一个 PVT 移动。

对于 PVT 移动，逆运动学程序不仅要采用 Q1 -Q9 的轴（刀尖）位置值，并将它们转换成 P1 - P32 的电机（关节）位置值；而且还必须采用 Q11- Q19 的轴线（刀尖）速度值，并把它们转换成 P101- P132 中的电机（关节）速度值。技术上，速度转换由机构的“逆雅可比矩阵”解组成。

例子

继续上述实例的“肩-肘关节”机器人，关节速度作为刀尖速度的函数：

$$\dot{A} = \frac{L_2 \cos(A+B)\dot{X} + L_2 \sin(A+B)\dot{Y}}{L_1 L_2 \sin B}$$

$$\dot{B} = \frac{[-L_1 \cos A - L_2 \cos(A+B)]\dot{X} + [-L_1 \sin A - L_2 \sin(A+B)]\dot{Y}}{L_1 L_2 \sin B} = -$$

角度 A 和 B 已计算在逆运动学程序的位置部分。注意，如果角度 B 接近 0 度或 180 度，该速度变成无穷大。由于在我们的例子中，我们限制 B 为正值，我们将任何小于 1° 或大于 179° 的 B 值（sinB<0.0175）的解看成一个错误。

```
&1
OPEN INVERSE
CLEAR
{Position calculations from above}
IF (Q10=1)                                  ; PVT 模式?
    Q26= SIN (Q25)                          ;sin (B)
    IF (Q26>0.0175)                         ;不接近奇异点?
      Q27= Q91* Q92* Q26                    ; L1* L2* sinB
      Q28= COS (Q25+ Q22)                   ; cos (A + B)
      Q29= SIN (Q25+ Q22)                   ; sin (A + B)
      Q30= (Q92* Q28* Q17+ Q92* Q29* Q18) / Q27    ; dA/dt
      Q31= (-Q7* Q17-Q8* Q18) / Q27                ; dB/dt
      P101= Q30* Q93                        ;#1 速度，计数/（Isx90 毫秒）
      P102= Q31* Q93                        ;#2 速度，计数/（Isx90 毫秒）
    ELSE                                    ;奇异点附近
      M5182= 1                              ;设置运行时间错误位
    ENDIF
ENDIF
CLOSE
```

请注意，在这种情况下，检查 B 是否接近 0° 或 180° 是冗余的，因为我们已经在逆运动学算法的位置部分完成了该检查。此检查显示在这里是为了说明该方法的原理。在这个例子中，如果太靠近一个奇点会创建一个运行时间错误；也可以使用其他的策略。

带运动学的坐标系统转换

通过运动学算法建立的坐标系统仍然可以使用在线{axis}=和轴编程起点的缓冲的 PSET 转换，就如同一个带标准轴定义语句的坐标系。当执行这些命令时，

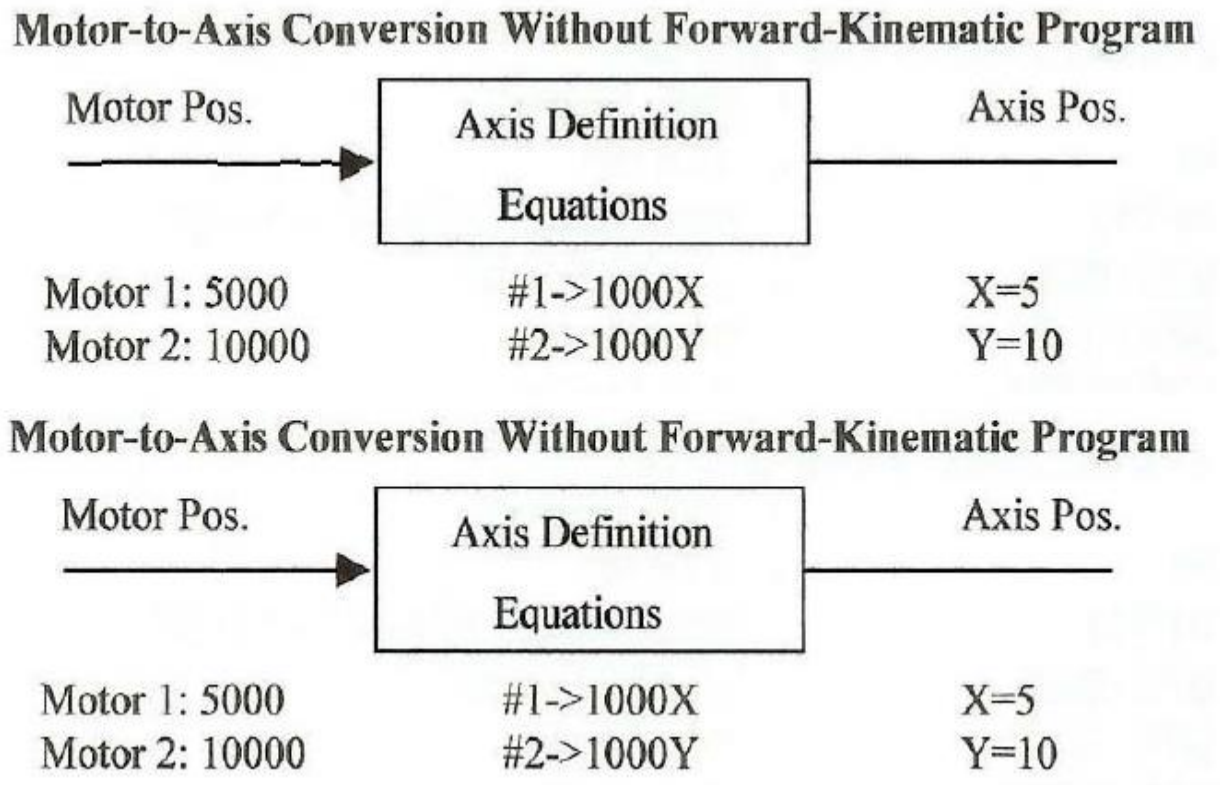
Turbo PMAC 执行反向运动学算法，为坐标系中每个受影响的电机计算一个新的位置偏置寄存器（建议的 M 变量 Mxx64）。这对于用户是不可见的；效果是偏移坐标轴的编程原点。X、Y 和 Z 轴的轴转换也可使用运动算法。

执行运动学程序

一个坐标系统中一旦创建了正向运动学和逆运动学程序缓冲区，当将坐标系的 I-变量 Isx50 设置为 1 进而启用运动学计算时，Turbo PMAC 将在适当的时间自动执行它们。在适当的时候访问运动学程序并不需要修改运动程序。

如果 Isx50 是 1，每次发送给坐标系一个 R（运行）或 S（步进）命令，将自动执行正向运动程序。这样做是为了确保对于初始移动的计算，起始刀尖（轴）位置是正确的，即使是自从上次编程运动以来已经完成了关节（电机）移动，如点动。如果 Isx50 是 1，每次给坐标系一个 **PMATCH** 命令时，也自动执行正向运动学程序。

（有了 Isx50=0 和正常的轴定义语句，Turbo PMAC 执行同样的功能，通过数学地反转轴定义语句的公式，来从目前的指令电机位置推导出起始轴位置。轴定义语句是技术上的反向运动学方程，所以他们数学的逆形成了正向运动学方程。由于标准的轴定义语句只限于数学上的线性方程组，通常它们的逆可以自动推导得出。）

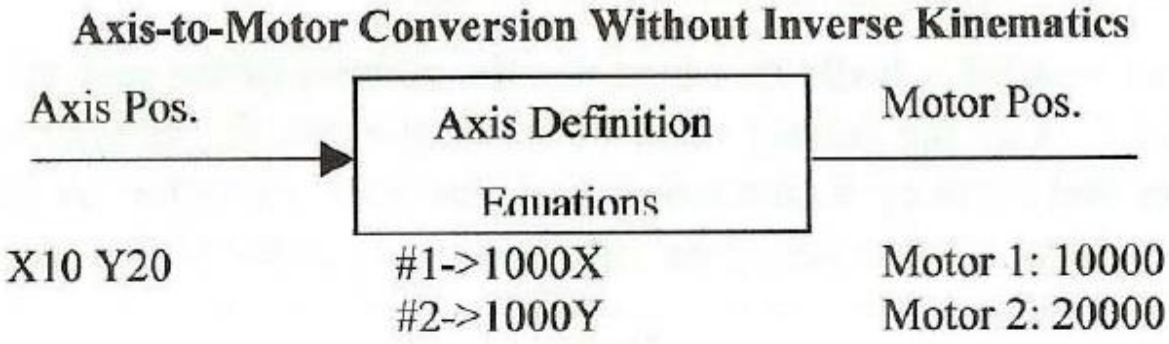


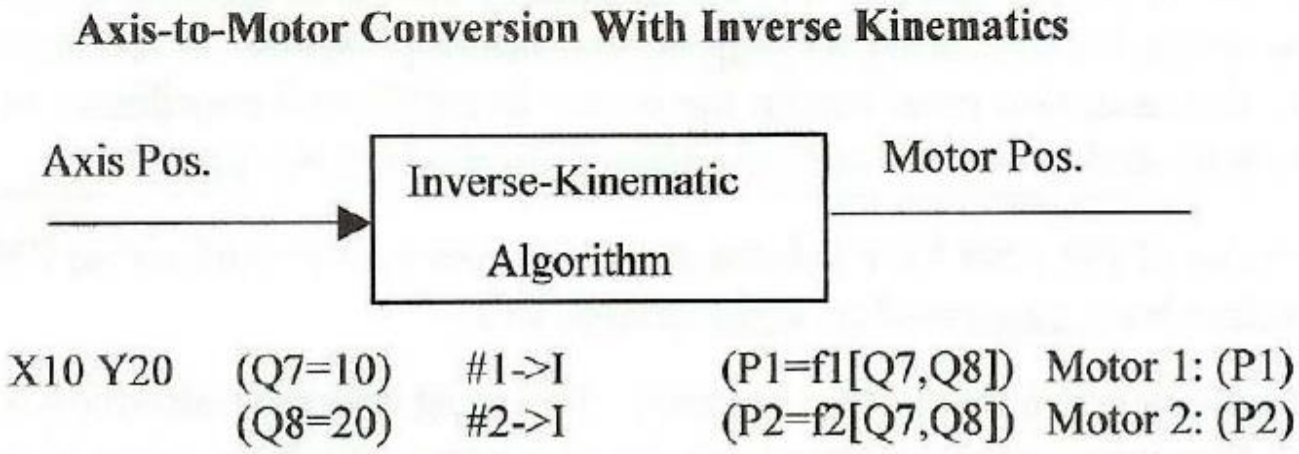
Motor-to-Axis Conversion Without Forward-Kinematic Program
Motor Pos.
Axis Definition
Equations
Axis Pos.
Motor 1: 5000
Motor 2: 10000
#1->1000X
#2->1000Y
X=5
Y=10

在运动程序的执行过程中，每次 Turbo PMAC 计算新的坐标轴位置时将自动执行反向运动学程序。对于不分段的移动，这发生在每个移动程序块的终点，如

那些 RAPID 模式下的。对于分段的移动(LINEAR 和 CIRCLE 模式移动，Isx13>0)，它发生在每个中间段的终点---每 Isx13 毫秒。

（对于正常的轴定义语句，Turbo PMAC 执行同样的功能，通过使用轴定义语句的方程从轴位置推导出电机位置。）





当逆运动学程序仅在编程的终点执行时，如在 RAPID 模式下，所有的插值发生在关节空间。在这种情况下，如果编程的终点相距很远，点之间的刀尖路径是是不好定义的，并且通常也不会是一条直线。

当反向运动学程序是在每个中间段的边界执行时，粗插补（分段）是在刀尖空间完成的，因此该路径很好定义。将该段的坐标转换为关节位置后，段的边界之间的精细插补工作是作为三次样条在关节空间完成的，但如果各个段很接近（每个通常为 5 至 20 毫秒）理想的刀尖路径的任何偏差可忽略不计。

如果坐标系的特殊前瞻缓冲区是激活的（LINEAR 或 CIRCLE 模式移动，为坐标系统定义前瞻缓冲区，Isx13>0，Isx20>0），为关节（电机）计算的内部样条段自动进入前瞻缓冲区。他们不断地核对每个电机的位置、速度以及加速度极限。这使得 Turbo PMAC 自动检查和纠正发生在奇异点附近的运动异常，所以你不必做这些。

15-5 坐标系统时基

每个坐标系都有它自己的时基，可以帮助控制在该坐标系的插值移动的速度。Turbo PMAC 的插补程序在每个伺服周期给一个“历经时间”寄存器以增量。当伺服周期真正的时间被硬件设置用于整个系统（通过 Turbo PMAC1 的跳线 E98、E29-E33 和 E3-E6 上，通过一个无 MACRO 的 Turbo PMAC2 的硬件控制变量 I7m00、I7m01 和 I7m02，或有 MACRO 的 Turbo PMAC2I 的 6800、I6801 和 I6802）并且对一个给定的应用不会改变时，在每个伺服周期 “历经时间”寄存器的增量，只是内存存储器中的一个数字。它不必与周期的真实物理时间相匹配。

2^{23}（8,388,608）个时基寄存器单位等于 1 毫秒。时基寄存器的缺省值是等于 I10 的值。I10 的出厂默认值 3,713,707 代表 442 微秒的默认物理伺服周期时间。

如果时间基址寄存器的值通过 I10 加以改变，插值移动的移动速度将不同于编程的速度。许多人把这种能力称为进给倍率修调。注意，物理时间不发生变化，所以伺服环路动态特性保持不变。

每个坐标系都有一个变量 Isx93，它包含了坐标系用作时基的寄存器的地址。在 Isx93 的默认值下，在坐标系从主计算机的%命令设置的一个寄存器中获得时基信息。一个%100 命令在该寄存器中放置一个等于 I10 的值；一个%50 命令将一个等于 I10/2 的值放在这个寄存器中。

注意：

一个未分配到任何坐标系统中的电机使用坐标系 1 的时基值执行其点动和回零运动。若要使用其他时基值，分配电机到不同的坐标系，即使没有必要写一个运动程序来运行这个电机。

不管时基的信息源是什么，一个%查询命令会使得 PMAC 以 I10 的百分比的形式返回目前时基的值。

时基信息可以来自其他的来源。命令源时基的最常见的替代源是外部的频率源时基，其中时基值正比于主编码器的频率。这提供了一个强大的位置同步从属机制，通常被称为电子凸轮。

请参阅使用外部时基的说明，在本手册的《同步 Turbo PMAC 到外部事件》的部分。

第十六章 编写和执行运动程序

运动程序是 Turbo PMAC 的主要机制，用于通过相关的数学、逻辑和 I/O 运算描述所需的运动。它们提供了一个简单但功能强大和灵活的手段用于描述运动和与该运动同步的操作。

Turbo PMAC 一次可以容纳多达 224 个运动程序。任何坐标系统可以在任何时间运行任何这些程序，即使另一个坐标系已经在执行同一程序。Turbo PMAC 可以同时运行的运动程序同板卡上定义的坐标系统（最多 16 个）一样多。一个运动程序可以调用任何其他的运动程序作为子程序，带或不带参数。

对 Turbo PMAC 的运动程序语言最好的描述，也许是在（如 BASIC 或 Pascal 的）高层次计算机语言与 G-Code（RS-274）机床语言之间的一个交叉桥梁。事实上，只要它已经被正确设置，它可以直接接受直线“G 代码”程序。它有一个计算机语言的计算和逻辑结构，以及非常类似于机床语言的移动规范结构。在程序中的数值可以被指定为常量或表达式。

16-1 执行运动程序序列

Turbo PMAC 运动程序的一个强大功能是，他们可以为计算自动定序，同时与编程运动保持同步。不像许多运动编程语言，这里没有必要在你的程序显式结构中，等待一个编程运动的结束。相反，Turbo PMAC 的操作系统自动监视程序移动的执行进度，并在编程移动结束时触发挂起的计算（运动、I/O 和/或逻辑）。这大大简化了运动程序序列的编写。

该方案的关键含义是，运动程序的计算仅发生在编程移动的边界处。如果您想要计算发生在其他时间，应在 Turbo PMAC 的 PLC 程序中执行这些计算。详细信息参阅本手册的《编写和执行 PLC 程序》一节。

16-2 流程控制

在一个运动程序中，Turbo PMAC 有 WHILE 循环和 IF... ELSE 分支控制程序流程。这些结构可以无限嵌套。此外，还有 GOTO 语句，带常量或变量参数（变量 GOTO 可以执行与一个 Case 语句相同的功能）。 GOSUB 语句（常量或变量目的地）允许可以在程序内执行子程序。CALL 语句容许输入其他程序作为子程序。子程序入口不必在开头---语句 CALL20.15000 使得程序 20 的入口在行 N15000 处。 GOSUBs 和 CALLs 只能嵌套 15 层深。

16-3 G 代码

为了处理机床样式的 G 代码（G 代码提供了直接访问由 CAD / CAM 程序创建的零件程序），Turbo PMAC 将一个 Gnn 语句视为 CALL 1000.nn000。行上的后面

的值（例如 X1000）可以被视为传递的参数，如同对一个固定循环，或无参数执行的子程序，返回，并执行该行的其余部分（如同对模态 G 代码一样）。机床设计者编写了**程序 1000** 来实现他想要的 G 代码，允许定制和增强功能。泰道公司提供一个样本文件实现所有标准的 G-代码。 M、S、T 和 D 代码也类似地实现。

16-4 模态指令

Turbo PMAC 运动程序的语句许多都是模态的性质。这些包括移动模式，它用于指定一个移动命令将生成什么类型的轨迹；这一类别包括 LINEAR、RAPID、CIRCLE、PVT 和 SPLINE。移动可以通过 INC 和 ABS 的命令指定为增量方式(距离）或绝对式（位置）---由轴单独选择。移动时间（TA、TS 和 TM）和/或速度（F），在模态指令得以实现。模态指令可先于他们影响的移动命令，也可以与这些移动命令的第一个在同一行上。

16-5 移动命令

移动命令本身包括一个单字母的轴说明符，和后跟的一个或两个值（常量或表达式）。在同一行中指定的所有轴，在执行该行时，将以协调的方式同时移动；连续的行将按顺序执行（之间带或不带停止，由模式决定）。取决于实际的模式，指定的值可以意味着目的、距离和/或速度（见轨迹特征的部分）。

16-6 运动程序轨迹

Turbo PMAC 的最突出的特点是它的轨迹生成算法的强大功能和灵活性。这些算法允许执行很多各种高难度动作，并允许你自己在易用性和控制程度之间进行权衡。重要的是要记住，这些轨迹只是一系列指令位置。它是由每个轴的伺服回路来试图使实际位置匹配指令位置。在本节中讨论的所有的时间、速度、距离和文件都是指令的那些，除非另有说明。

16-7 线性混合运动

最容易进行的移动类别是线性混合运动类别。在这种类型的移动，一个轴线以指定速度移向目标位置，以受控制的方式从该速度上加速和减速。如果连续指定一个以上的移动而且中间没有停顿，第一个运动将以同类型加速减速控制方式融入第二个运动中。

线性混合移动模式是运动程序的默认的模式。如果在其他移动模式中，该程序可以通过 LINEAR 语句而放置在这个模式下。该程序可以跳出 LINEAR 模式，通过其他移动模式语句（如 CIRCLE1、CIRCLE2、RAPID、PVT、SPLINE）。在每个程序中声明 LINEAR 模式，而不依赖于默认，这是良好的编程习惯。LINEAR 语句是相当于 RS-274 的 G 代码 G01。

位置或距离说明

一个线性模式移动的目标点是被指定在移动命令本身（例如 X10Y20）。每个轴的指令目的地可以被指定为一个相对于编程原点的位置（如果轴是在 ABS 绝对模式）或相对最后指令位置的距离（如果轴是在 INC 增量模式）。程序规定其中之一；Turbo PMAC 自动计算出其他剩下的。

进给率或移动时间说明

您可以用一个 F 命令指定移动的目标速度（进给速度），或使用 TM 命令指定移动的时间。如果指定了 F，对移动时间进行计算；如果指定了 TM，计算进给率。对于一个给定的移动距离，这两个值之间是倒数的关系。运动时间和进给率值是模态的；它们会影响所有后续的直线（和圆弧）模式移动，直到在程序中指定另一个 F 或 TM 值。

TM 时间的单位是毫秒；F 速度的单位是进给轴的用户长度（或角度）单位除以坐标系统变量 Isx90 以毫秒定义的时间单位。如果 Isx90 是在 1000 的默认值时，F 的单位是每秒的长度单位；如果 Isx90 设置为 60,000，F 单位是每分钟的长度单位。

如果在上电或复位后没有指定 F 或 TM 值，Isx89 的值作为一个进给率值用于移动。在程序中指定的任何 F 值与最大进给速度参数 Isx98 进行比较；如果大于这个参数，使用 Isx98 来代替。

注意：

进给速度是一个幅值，因此，应始终为正数。一个负的进给速度会导致与坐标系定义中正的定义相反的运动。

矢量进给轴

如果由进给率（而不是时间）指定一个多轴的运动，使用 FRAX 命令（在线或缓冲的）可以有进一步的灵活性来指定哪个轴控制矢量进给率，速度分摊在这些轴之间，使他们的矢量组合（平方和的根）是指定的速度。Turbo PMAC 计算移动时间是使用进给率轴的矢量距离除以编程进给率。因此，对于运动的每个不同的角度，不必单独计算各轴的速度。如果一个非进给率轴要求一个同时移动，该移动要在计算进给率轴的相同的时间完成。坐标系的默认的进给率轴是 X、Y 和 Z 轴。

如果在同一行中有其他的命令轴（非进给率轴），Turbo PMAC 将矢量进给率轴的计算的移动时间，与用非进给率轴的最大距离除以坐标系的备用进给率参数 Isx86 所产生的移动时间，对二者进行比较。无论这些移动的时间哪个最长，用于所有的轴。

矢量进给速度的计算示例（Isx86=40）

INC FRAX(X,Y) X3 Y4 F10	Vect Dist = SQRT($3^2 + 4^2$) = 5 Move Time = 5/10 = 0.5 Vx = 3/0.5 = 6 Vy = 4/0.5 = 8
INC FRAX(X,Y) X3 Y4 Z12 F10	Vect Dist = SQRT($3^2 + 4^2$) = 5 Vect Move Time = 5/10 = 0.5 Non-Vect Dist = 12 Non-Vect Move Time = 12/40 = 0.3 Vx = 3/0.5 = 6 Vy = 4/0.5 = 8 Vz = 12/0.5 = 24
INC FRAX(X,Y,Z) X3 Y4 Z12 F10	Vect Dist = SQRT($3^2+4^2+12^2$)=13 Move Time = 13/10 = 1.3 Vx = 3/1.3 = 2.31 Vy = 4/1.3 = 3.08 Vz = 12/1.3 = 9.23
INC FRAX(X,Y,Z) C10 F10	Vect Dist = 0, Non-Vect Dist = 10 Move Time = 10/40 = 0.25 Vc = 40

电机速度极限

Turbo PMAC 为每个电机 xx 提供了一个速度极限参数 Ixx16，可用于自动限制在直线模式移动中的指令速度，即使是运动程序请求一个更高的速率。此限制功能如何工作的细节取决于操作模式。

这个速度极限是激活的，无论是如果分段模式没激活（Isx13=0）在这种情况下不允许圆弧插补和刀具半径补偿，或者是如果分段模式被激活（Isx13>0）和特殊前瞻缓冲区被激活（Isx20>0，定义的前瞻缓冲区）。

如果速度极限是激活的，Turbo PMAC 将运动程序要求的电机速度大小与每个电机的 Ixx16 限制进行比较。如果任何电机的编程要求超过了限制，运动时间延长，使电机不会超过它的极限；这将自动按比例减慢坐标系中的其它电机，使电机之间的关系（空间路径）得以维持不变。

加速度参数

加速到一个速度，或者从该速度减速，其加速度可以是恒定的，提供了梯形的速度曲线；它也可以线性变化，得到 S 形曲线速度轮廓；或者它可以是这两者的组合。指定全加速的时间（TA---默认参数是坐标系的 I-变量 Isx87）和每半个 S 曲线的时间（TS---默认参数 Isx88）。如果指定的 TA 时间比指定的 TS 时间的两倍要小，使用的 TA 时间将是 TS 时间的两倍（想要纯 S 曲线加速度的用户只需设置 TA 为 0）。Turbo PMAC 对于 TA 和 TS 只能使用整数值。如果指定了非整数

值，在使用它用于轨迹计算之前，PMAC 将它四舍五入到最接近的整数。

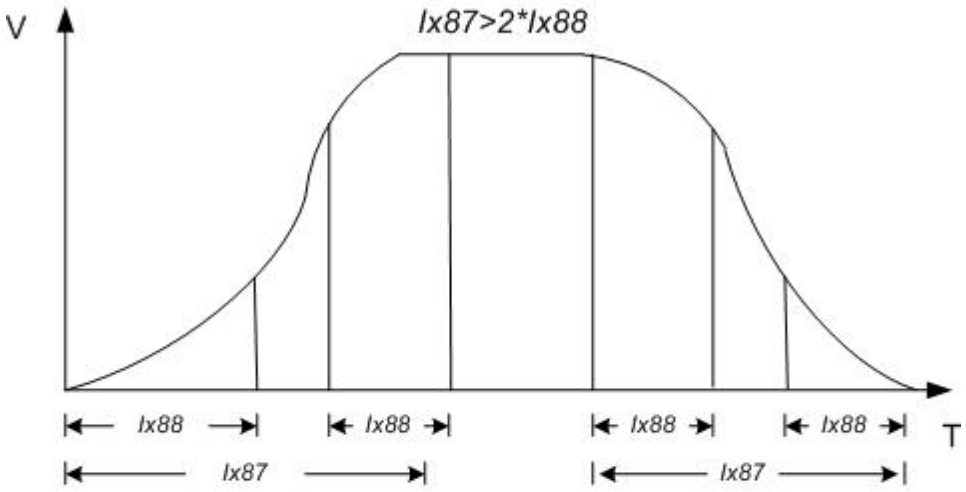
Coordinate System Variables

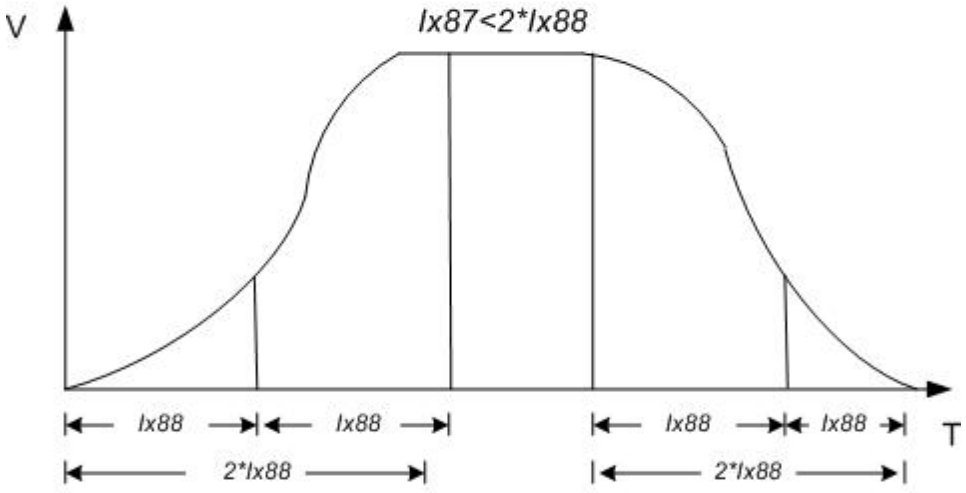
Ix87 Default Acceleration Time (Program)

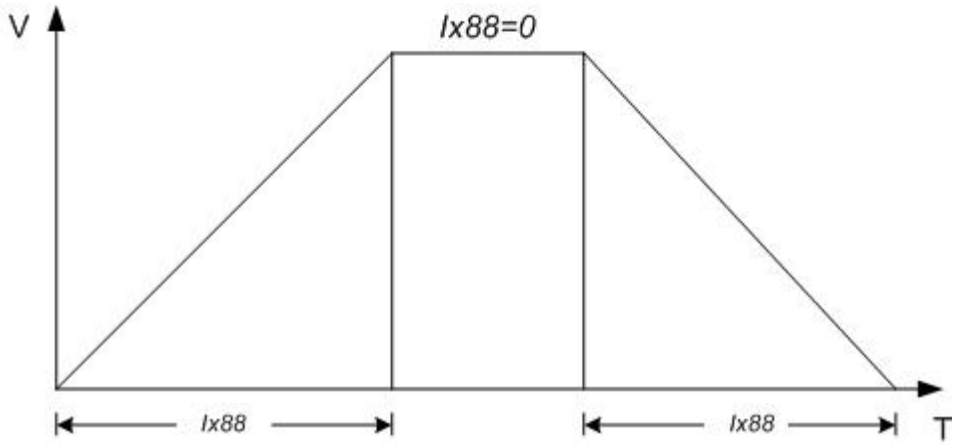
(Units: msec); integer
Overridden by TA in program

Ix88 Default S-Curve Time (Program)

(Units: msec); integer
Overridden by TS in program

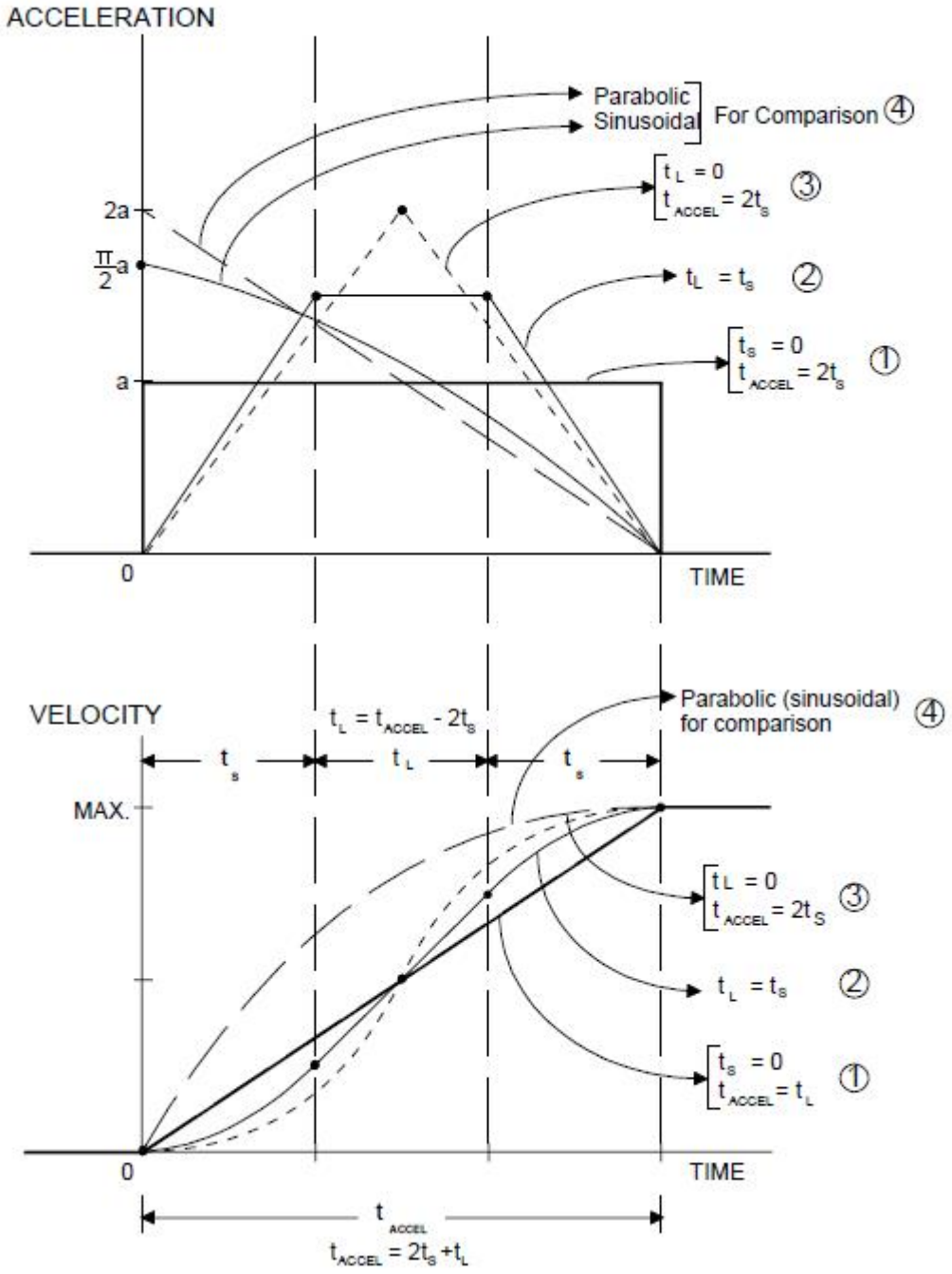






自动的“S”曲线加速

指定t_{acc}，t_s和加速度限制



加速度限制

Turbo PMAC 为每个电机 xx 提供了一个加速度极限参数 Ixx17，可用于自动限制线性模式移动下的指令加速度速率，即使是运动程序请求一个更高的速率。

此限制功能如何工作的细节依赖于操作模式。

- 分段模式不激活（Isx13=0），特殊的前瞻缓冲区不激活（Isx20=0）。这是最简单的操作模式。在这种模式下是不允许圆弧插补的。在该模式下 Ixx17 加速度限制是激活的，但如果加速将不得不延长超过一个完整编程移动以观察极限，那么该限制将被触犯。
- 分段模式激活（Isx13>0），特殊的前瞻缓冲区不激活（Isx20=0）。此模式允许圆弧插补和刀具半径补偿，并将圆弧模式和线性模式移动混合在一起，但在这一模式下 Ixx17 加速限制是不激活的。
- 分段模式激活（Isx13>0），特殊的前瞻缓冲区激活（Isx20>0，定义前瞻缓冲区）。这是最复杂的操作模式。此模式允许圆弧插补和刀具半径补偿，将圆弧模式和线性模式的动作混合在一起，对线性和圆弧模式移动激活 Ixx17 加速限制，如有必要并能延长加速度超过多个编程运动。

本节只涵盖了模式 1 的加速度限制的算法。特殊的前瞻缓冲区的加速度限制功能（模式 3）在手册的专门章节中论述这一功能，见下文。

如果从运动中计算的加速度超过了移动中涉及的任何电机的最大编程加速度（由 Ixx17 设置，计数/毫秒 ^{2}），运动中所有电机的加速度将按比例减少进而使任何电机都不超过此限制。通过空间的路径不改变，任何电机的速度分布的形状也不改变。如果你想直接指定加速度速率，将 TA 和 TS 设置到非常小，以便始终超过限制，在这种情况下，加速度由电动机的 I 变量 Ixx17 控制。

注意：

除非该坐标系是在分段模式下（Isx13>0），否则不要将 TA 和 TS（Isx87 和 Isx88）时间都设置为零，即使你打算依靠加速度的作用。因为这将导致一个除数为零的错误，可能产生不稳定的性能。

何时太有效

当将多个线性移动混合一起时，即使是对于减速到停止然后再融入下一步的运动的中间加速度，Ixx17 限制也是强制起作用的。当 Turbo PMAC 计算混合序列的每个移动时，它都必须假设这可能是序列中的最后一个运，而且它会确保该减速到停止在编程位置处时的加速度要服从此限制。这可能会导致减速时间超过了设定的移动时间(直接通过 TM 指定,或间接地通过 F 进给速度下的距离指定)，这将导致执行移动的速度比设定的速度要慢。当运动都分解成非常小的片段然后混合在一起时，这是特别的限制。该 Ixx17 限制必须设置得比最高速度除以最小段的时间得到的值要大，是为了不限制速度。这对于有效的加速度控制可能过高。

何时不够有效

特殊前瞻缓冲区未启动时，Turbo PMAC 要对比实际执行的移动提前两个移动进行计算来执行它的加速度限制，并重新计算这两个运动来将加速度保持在 Ixx17 限制之下。然而，存在一些多于两个移动的情况，有些远多于两个，就不得不重新计算这些运动从而将加速度保持在限制之下。在这些情况下，Turbo PMAC 将尽其可能地限制加速度，但是因为之前的运动已经被执行过了，他们不

可能未完成，因此加速度限制将被超过。

如果你希望稳健的加速度限制（其中的加速度限制可能需要修正一些运动的速度），那么应该使用特殊前瞻缓冲区。更多详细信息，参考本手册的 Turbo PMAC 前瞻功能的部分。

最小运动时间

如果一个指定进给率的运动段的距离太短以至于不能够到达它的目标速度，它将它的全部时间用来加速（产生一个三角形曲线而不是梯形曲线）。这样一个运动的最小时间因此是指定的加速时间（TA 或 2*TS 之中较大的那个）。对于一个单独的运动，要记住加到额外的加速时间上来减速到停止。

在一个指定时间的运动段，如果 TM 小于加速时间，该段将在加速时间内完成而不是 TM 时间。

换句话说，对于一个单独的混合运动或者混合运动段，加速时间最小时间。在某种程度上这是一种保护，防止运动时间变得太短以至于 Turbo PMAC 不能实时计算出他们。如果你工作的运动段非常短，并且你的运动序列比你想的走的要慢，可能是这个加速时间限制导致这个问题。

最大运动时间

对于一个编程的运动，最大时间是 2^{23}-1（8,388,607）毫秒，大约 2 小时 20 分钟。这是一个 TM 指令下 Turbo PMAC 接受的最大值。对于一个指定进给的运动，它也是 Turbo PMAC 计算的最大值（运动的矢量距离除以进给率）。如果运动的矢量距离除以进给率产生的时间大于 8,388,607 毫秒，Turbo PMAC 将 8,388,607 毫秒用作运动时间，而且速度将比编程的速度要快。

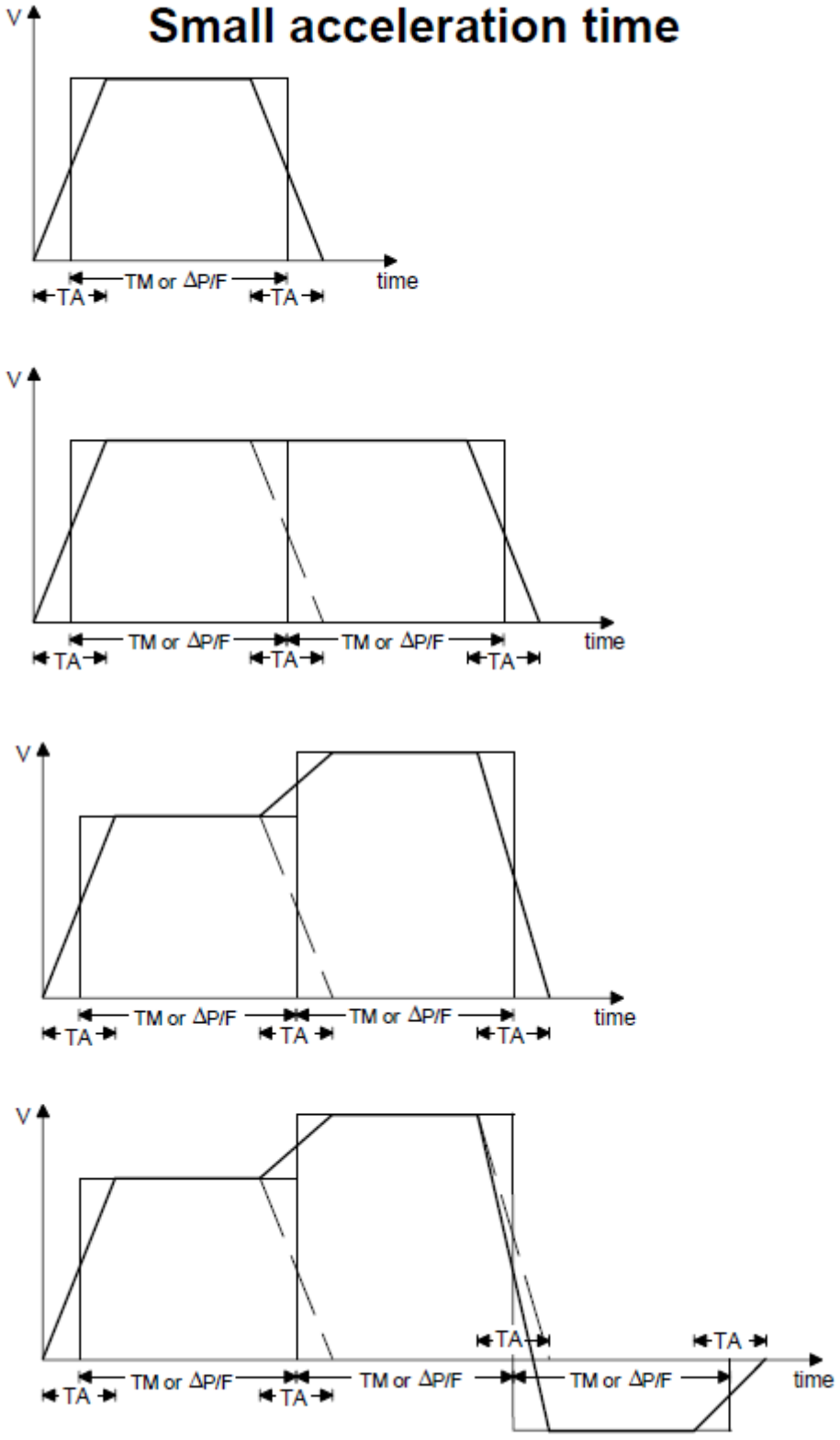
混合功能

如果连续指定多个运动而不插入任何 dwell 指令，那么每个电机将根据那时的加速度和 S-曲线值，从第一个运动的速度到第二个运动的速度进行平滑地混合。这个速度上的变化（它可以是一个零变化）开始于第一个运动为了停在指定终点位置而开始减速的那点，而不是在第一个运动的终点本身。（但是，如果 Isx92 设置为 1，在那个坐标系下“混合”的运动始终在下一个运动开始之前就已经结束了。）

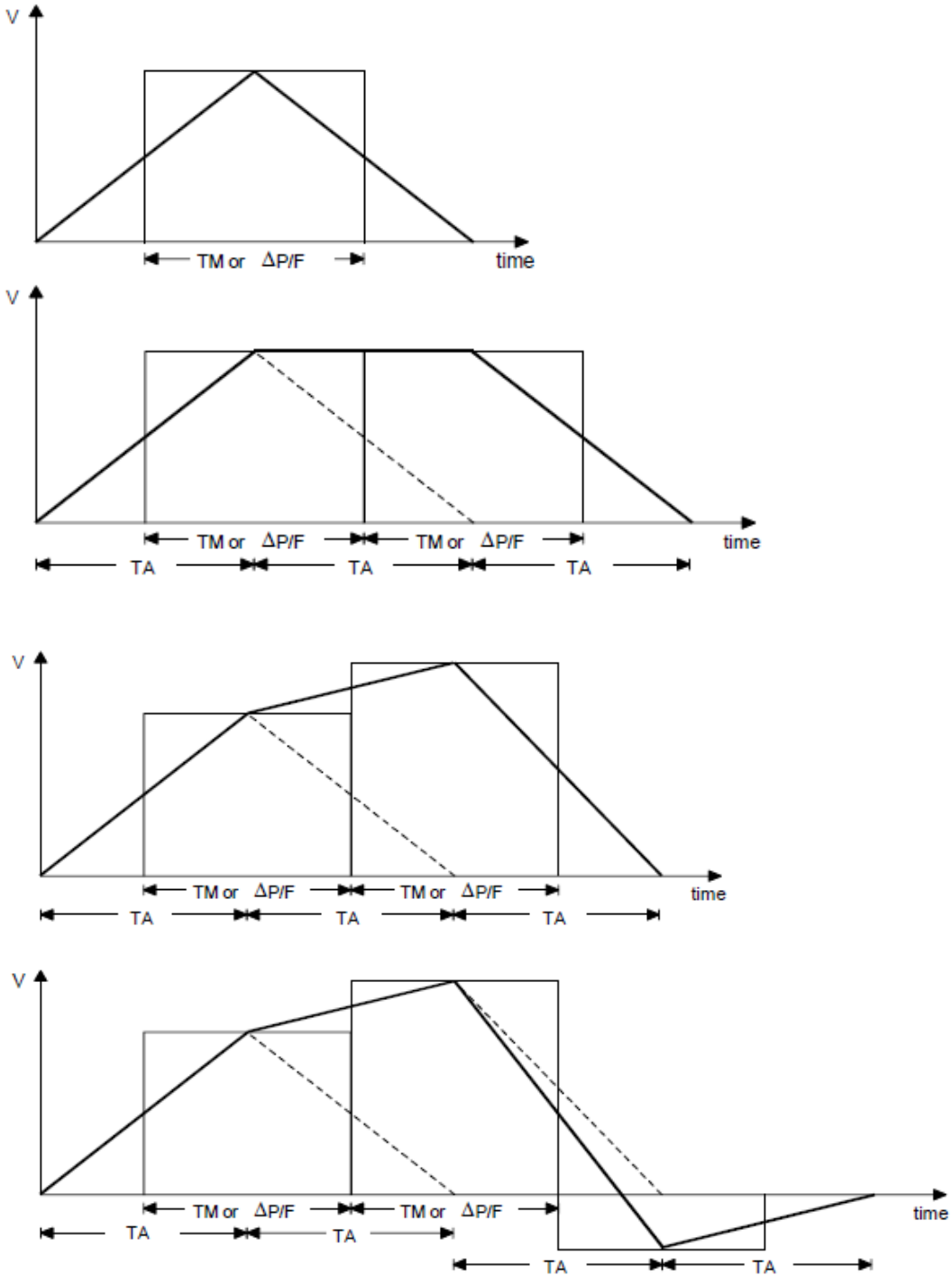
加速度参数 TA 和 TS 可以在每个运动之间改变。为了序列最后的减速到停止能使用一个不同于上一个混合加速时间的 TA 或 TS，在序列最后的运动指令的后面，但在 DWELL 或可其他以停止连续指令的功能的前面，声明新的 TA 或 TS。

Linear Mode Trajectories

Small acceleration time

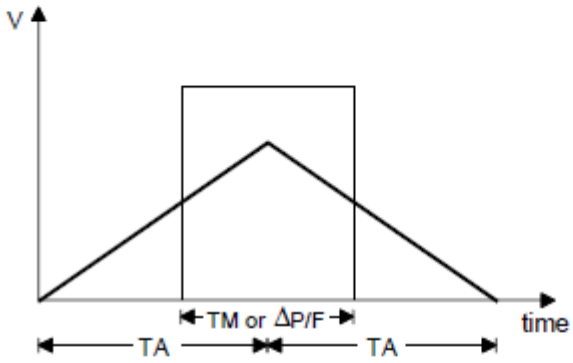


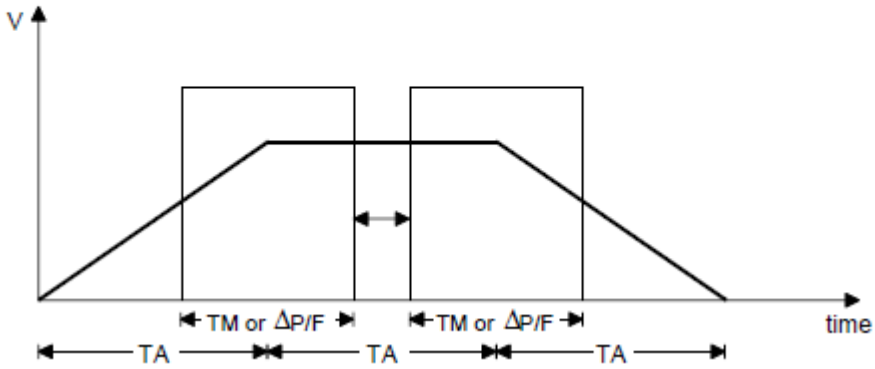
Linear Mode Trajectories
Acceleration time matches move time

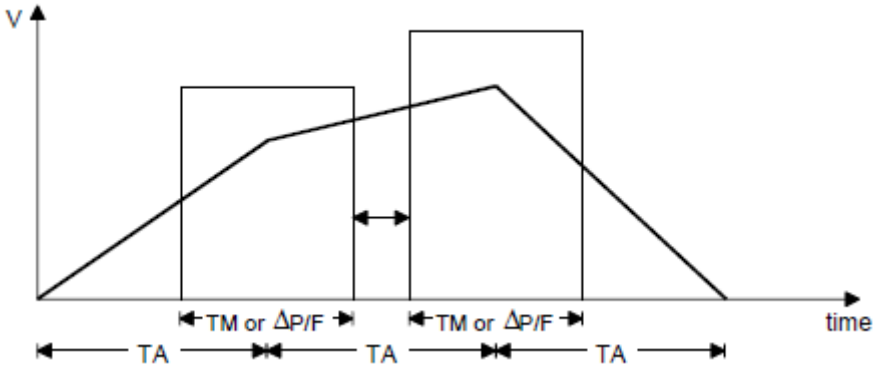


Linear Mode Trajectories

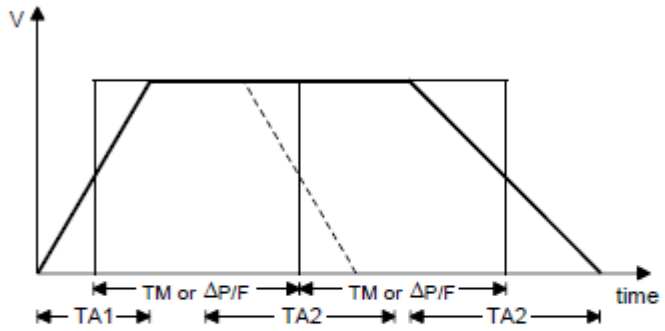
Large (velocity limiting) acceleration time

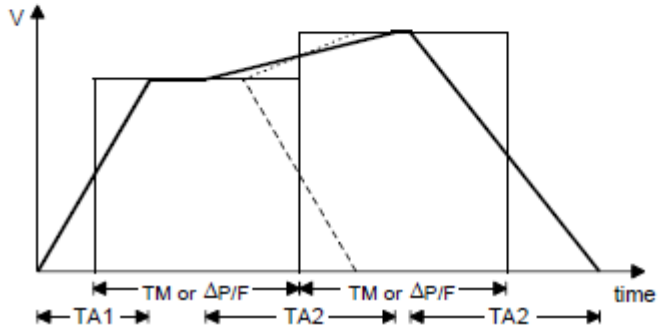


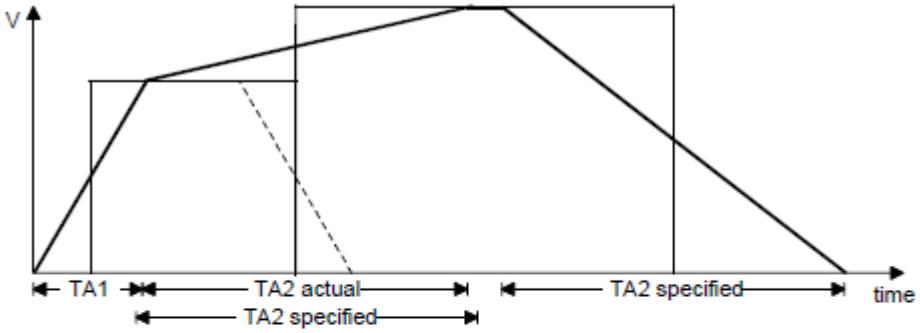


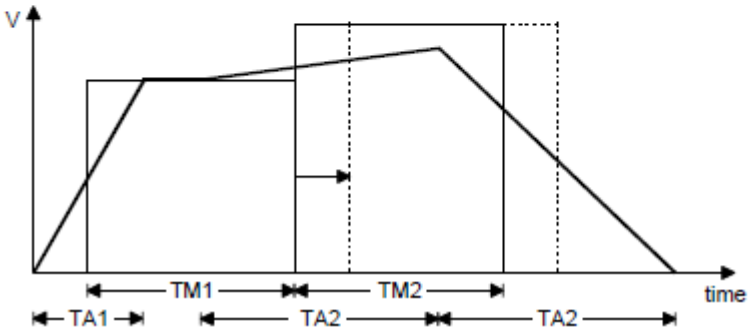


Linear Mode Trajectories
Changing acceleration times









16-8 圆弧混合运动

注意：

为了 Turbo PMAC 进行圆弧运动，坐标系参数 Isx13 必须大于零。详情参见下文。

Turbo PMAC 允许在坐标系的 X、Y、Z 轴上进行圆弧插补。如同线性混合运动，TA 和 TS 控制从停止到运动、从运动到停止、运动之间的加速度。圆弧混合运动可以是指定进给率（F）或者指定时间（TM），就像线性运动那样。可以在线性和圆弧运动之间来回进行改变而无需停止。

指定插补平面

准备圆弧运动的要做的第一件事是指定包含着圆弧的平面的方向。这是通过使用 **NORMAL** 指令指定该平面的法向矢量完成的。该命令的参数是矢量的合成幅值： I（X 轴方向）、J（Y 轴方向）、K（Z 轴方向）。一个典型的指令是 **NORMAL I0.866 J0.5 K0.0**。这里指定的法向矢量的长度并不重要；只是合成幅值之间的比率（其决定方向）重要。

标准平面

要指定 XY 平面上的圆弧，命令 **NORMAL K-1**（等价于机床代码 G17）。类似地，对于 ZX 平面的圆弧，命令 **NORMAL J-1**（等价于 G18）；对于 YZ 平面，命令 **NORMAL I-1**（等价于 G19）。

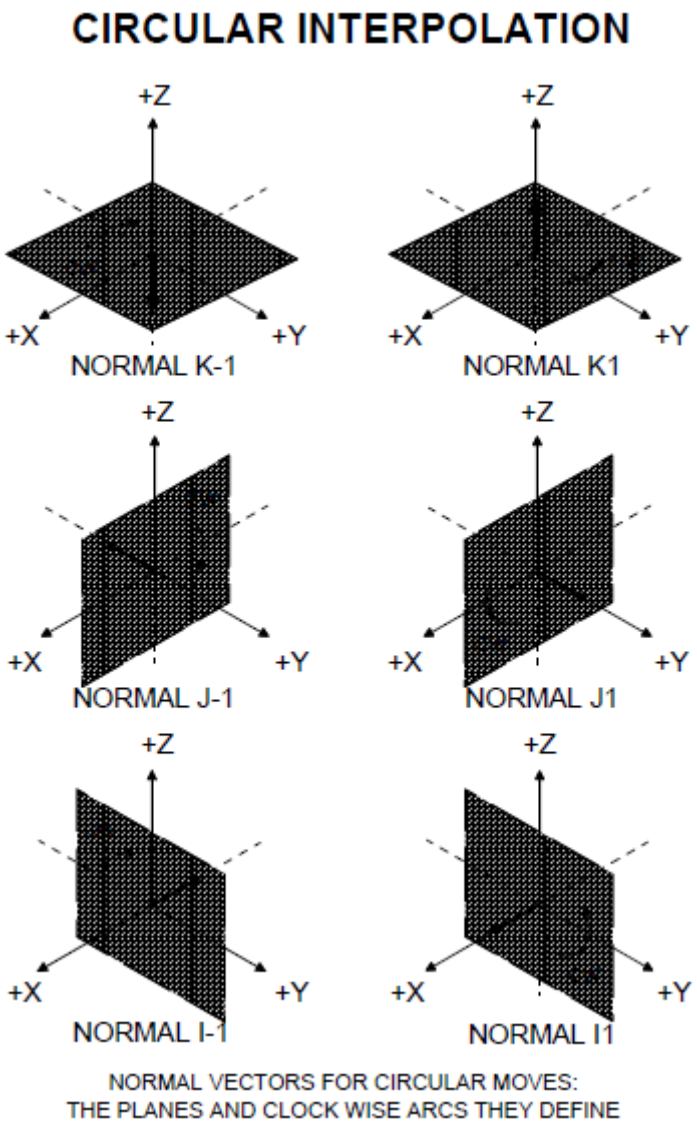
顺时针方向判断

法向矢量的方向判断是用右手准则。获得标准顺时针是通过使用沿着他们的轴指向负方向的法向矢量。

圆弧模式

要将程序放在圆弧模式下，使用程序指令 **CIRCLE1** 用于顺时针圆弧（等价于 G02）或者 **CIRCLE2** 用于逆时针圆弧（等价于 G03）。任何其他的运动模式指令--- **LINEAR**, **RAPID**, **PVT**, 或者 **SPLINEn**---将程序带出圆弧运动模式。

LINEAR 将你的程序恢复为线性混合运动模式。一旦在圆弧模式下，通过使用一个指定运动终点、和到圆弧中心的矢量或到中心的距离（半径）的运动指令来指定一个圆弧模式。终点可被指定为一个相对起点的位置或者距离，而这取决于轴是在绝对模式（ABS）还是增量（INC）模式（可分别指定）。



圆心矢量

如果使用矢量方法定位圆弧中心，矢量是通过它的 I、J、K 成分指定的（I 指定的成分平行于 X 轴，J 的平行于 Y 轴，K 的平行于 Z 轴）。这个矢量可被指定为一个相对起点的距离（即增量式）或者相对 XYZ 原点的距离（即绝对式）。这是通过在一个 ABS 或者 INC 语句中指定‘R’进行选择的(例如，**ABS(R)**或 **INC(R)**)。这将影响到 I、J、K 指定符。(无参数的 ABS 和 INC 影响所有的轴，但矢量不变)。默认的是增量式的矢量指定方法。

注意：

标准的机床使用的是增量式矢量指定方法，即使当运动终点的指定是绝对的。

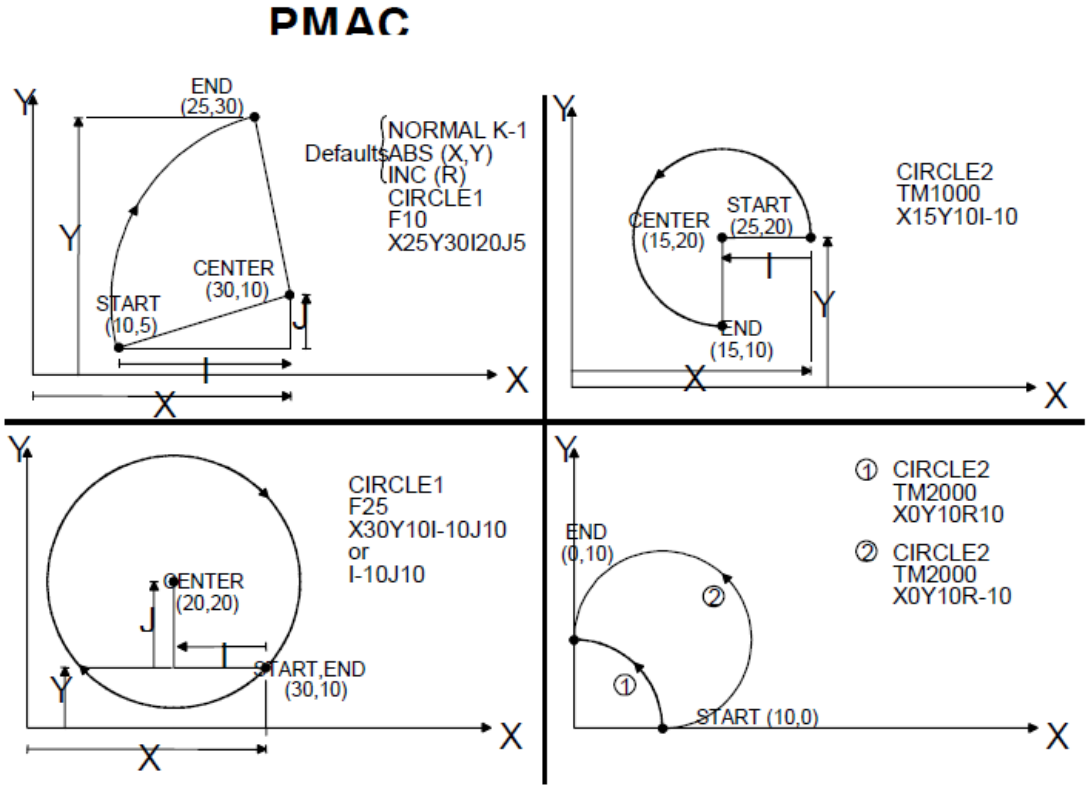
一个带矢量指定的典型圆弧运动指令是：

X1000 Y2000 I500 J-500

例子：

在 XY 平面中，从 X0 Y0 点开始，做一个顺时针的 1/4 圆弧到 X20 Y20，然后是一个线性的运动到 X40 Y20，然后是一个 3/4 圆弧顺时针到 X20 Y0。在默认的绝对运动和增量式矢量指定模式下，程序是：

```
NORMAL K-1          ; XY平面
F10
CIRCLE1             ; Clockwise circle
X20 Y20 I20 J0      ; Arc move; I=20-0=20; J=0-0=0
LINEAR
X40 Y20
CIRCLE1
X20 Y0 I0 J-20      ; Arc move; I=40-40=0; J=0-20=-20
```



半径尺寸说明

如果使用半径方法来定位圆弧的中心，那么半径是移动指令中字母 R 后的那个数字。此值始终代表相对运动起点的距离。当用半径指定时，也有必要指定运动到移动终点的圆弧是优弧（> =180 度）还是劣弧（<=180 度）。Turbo PMAC 的惯例：如果 R 值是正的，PMAC 采取劣弧路径，如果 R 是负的，采取优弧路径。R

的值不是模态的---该值必须在每个运动命令行上指定。使用半径说明，在单个移动命令就完成一个完整的圆是不可能；圆弧必须分成至少两部分。

用半径指定一个典型的圆形运动的命令是：

X1000 Y2000 R750

例如：

要做如在上述例子中同样的动作，除了半径中心说明，程序将是：

```
NORMAL K-1          ; XY plane
F10
CIRCLE1
X20 Y20 R20         ; Arc move < 180 deg
X40 Y20             ; Automatically linear
X20 Y0 R-20         ; Arc move > 180 deg
```

注意：

如果通过 AROT 或 IROT 语句使用轴变换矩阵用于缩放的目的，那么不要使用 R 半径说明。半径值不会随着轴而一起缩放。

非圆心说明

如果在一个给定的移动命令行，既没有一个矢量说明也没有半径说明，那么即使程序是在圆弧移动模式下，移动也要在起点和终点之间进行线性插补。但是，如果这样做，刀具半径补偿将不能正常工作。如果刀具补偿功能打开了，LINEAR 移动方式必须明确地声明。

矢量进给轴

在圆弧插补中使用的任何轴，都将自动成为该圆弧运动下的进给速度轴，即使他们不在 FRAX 命令中指定。其他轴可以是或可以不是进给轴。在同一个运动命令中，被指令移动的任何非进给轴都将进行线性插补，从而使得运动在相同的时间完成。这允许简单的螺旋插补。请参阅本手册中的《进给轴》的章节。

圆弧半径误差

如果终点到中心的距离与起点到中心的距离不相同，那么半径的变化在运动过程中被平滑改变。从技术上说，Turbo PMAC 会产生一个相对于时间具有恒定矢量进给率和在半径的恒定变化（恒定的 dR/d θ ）的指数螺旋曲线。

当使用 IJK 中心矢量，这个螺旋形曲线可以建立在起点、“中心”和终点之间的任何角度上。当使用 R 半径大小的说明，只有当起点和终点之间的距离大于两倍的半径时，才能创建一个螺旋线。在此情况下，起点、中心、终点都将是共线的（像一个半圈）。对于以这样方式指定的运动，这种技术只是用来允许执行一些拐角误差公差的半圆。

每个坐标系都有一个 I 变量(Isx96)，确定了完成这个补偿的距离差的极限。超过极限，就会产生运行时错误，程序将停止。这个限制允许在拐角误差和重大错误之间进行区别。不管此限制如何，如果从起点到中心的距离或从终点到中心的距离是零，将会产生一个错误，程序将停止。如果指定的向量不在插补的平面上，那么使用该向量到插补平面上的投影。

移动分段模式

Turbo PMAC 是通过一个快速且连续的三次样条法技术（叫“分段模式”）来计算圆形轨迹的。样条曲线段是由坐标系变量 Isx13 指定的一段时间（以毫秒为单位）。通常使用 5～10 毫秒的值，具体取决于由卡控制的轴数。当 Isx13 大于零，所有的混合运动 ---直线和圆弧的---都通过这种持续的三次样条法来计算。确切的移动的计算是每 Isx13 毫秒执行一次，而这些中间点之间的精插补的完成是以伺服更新率、采用高效的三次 B 样条插补（相同于 Turbo PMAC 的 SPLINE1 模式）。

如果 Isx13 是零，线性动作不使用此样条技术进行计算，而且不允许圆弧运动（如果请求一个圆弧的运动，它会作为一个线性移动来完成）。在 Isx13= 0 模式和 Isx13> 0 模式之间的线性模式移动的实际性能的差异几乎是察觉不到的，除非运动的特征尺寸与 Isx13 时间是在相同的范围内。

分段模式（Isx13>0）也需要使用刀具半径补偿（即使圆形运动都没有明确地编程）、运动学计算、特殊前瞻缓冲区。

16-9 快速模式的移动

快速模式下的移动提供了最小-时间的点到点运动，受制于预先定义电机的约束。这些运动基本上是移动中分配给指定轴的每个电机的慢跑运动。每个电机的加速度由 Ixx20 和 Ixx21 加速时间来控制的；但是，如果电动机的移动速度和这些变量确定的加速度的速率超过了 Ixx19 加速度极限，时间将被延长，使速率不会超过 Ixx19。许多用户会设置 Ixx20 和 Ixx21 到非常小的值（例如 1 和 0；不要都设置为 0），以使 Ixx19 速率控制所有这些加速度曲线。

移动的速度受控于最大速度参数 Ixx16（默认）或微动速度参数 Ixx22，具体取决于电机变量 Ixx90 的设置。一个短移动可能达不到编程的速度，只是加速和减速完成移动。

一个运动程序使用 RAPID 声明将自己置于此模式下。它被带出这种模式是通过另一个移动模式命令（如 LINEAR，CIRCLEn，PVT，SPLINEn）。RAPID 等效于 RS-274 G 代码 G00。

运动时间

在多轴快速模式移动，只有经过计算将以其指定的 Ixx16 或 Ixx22 速度用最长时间的电机，将实际被命令以那个速度移动。其它电机的指令速度相应减小，使他们有相同的距离速度比，使得所有电机得到相同的移动时间（在添加加速和减速之前）。这使得在一个直角坐标系的移动路径近似是线性的。但是，如果加速时间对于所有电机是不相同的，如到达 Ixx19 加速限制将发生的那样，那么指令移动路径不会是完美的线性。

最大运动时间限制

快速模式运动的最大移动时间为 8,388,607 毫秒，大约 2 小时 20 分钟。如果任何电机的距离和速度要求的移动时间大于该时间，那么 Turbo PMAC 将使用这个最大时间，而且速度会比编程的要快。

最小运动时间限制

快速模式运动的最小移动时间是坐标系统中任何电机的最长加速时间（Ixx20 或 2* Ixx21，以较高者为准），包括没有在程序行明确指令的电机。用户想命令很短的 RAPID 运动，并希望他们很快结束，对所有马达应将 Ixx20 和 Ixx21 设置的很小，并使用 Ixx19 加速限制控制加速度曲线。

运动路径

因为所有电机的移动时间（加速/减速之前的）都被控制为相同的时间，所以直角坐标系的移动路径将至少是近似线性的。然而，若要笛卡尔系统中的路径是真正的线性，所有电机的加速时间（非速率）必须是相同的。要获得此完全线性的路径，对于坐标系中的所有电机，Ixx20 和 Ixx21 必须是相同的，而且所有电机的 Ixx19 必须设置足够高，使得其不用于移动。

如果使用运动子程序在直角坐标系中编程一个非笛卡尔坐标系，那么 RAPID 运动模式在系统中一般不会有线性路径。快速模式运动的运动学计算仅在快速模式运动的编程终点进行，而不是在沿路径的许多点上进行。

无混合

在编程的终点，一个快速模式移动不会混合到另一个运动中；在启动下一个运动之前，所有电机将被指令至少一个瞬间的停止。然而，不像在其他模式下，快速运动模式可在沿轨迹的任意点分解，更改为不同的目标。这可以用下面所述的移动--直到触发和改变--目的地的功能来实现。

运动程序的移动--直到--触发

移动-直到-触发的功能允许一个编程的快速模式运动由一个触发器中断，并由相对于触发位置的移动来终止（停止在触发时的位置）。它类似于一个回零搜

索移动，除了电机的零位置不改变，并且在没有找到触发时有一个特定的目的地。

在 Turbo PMAC 上，移动-直到-触发是快速移动模式的一个变型。速度和加速度是由与常规快速移动相同的变量管控制的。对于一个轴的以及定义到该轴的任何电机的移动-直到-触发的功能，是通过在该轴的移动命令中加一个**^{数据}**说明符指定的，其中**{数据}**是在停止之前要运动到触发器的距离。

这使得运动-直到-触发的轴命令为**{轴}{数据}^{数据}**，类似于 **X50^-5**。第一个值是目标位置或没有触发时要行进的距离（具体取决于轴是处于绝对还是增量模式）。第二个值是相对于触发位置要行进的距离。该值始终表示为一个相对距离，无论轴是处于绝对或增量模式。这两个值都表示为轴的用户单位。

其它轴可以在同一行指定，而无需一个插入符（**^**）和第二个值。这些轴会执行一个正常的同步快速移动。然而，如果对于该坐标系的 X、Y 和 Z 轴，一个轴矩阵变换是有效的（TSELECT 被用来选择一个矩阵），如果在 X、Y 和 Z 轴的任一轴上有一个移动-直到-触发，在 XYZ 三联动的其它轴也将执行一个运动-直到-触发。如果轴线没有指定后触发移动，后那么触发距离被假定为零。如果轴根本没有指定的运动，那么假设一个零距离的先触发移动。

每个电机的触发条件的设置为与回零搜索运动是一样的：

- Ixx97 的 1 位指定是输入标志被用来创建触发，还是警告跟随误差极限状态位是触发（跟随误差的幅度超过 Ixx12）：0=标志，1=错误状态。跟随误差的触发通常被称为“扭矩限制触发。”
- 如果是输入标志来创建触发，Ixx25 指定标志寄存器。
- 如果是输入的标志创建触发，此通道的编码器/标志 I-变量 I7mn2 和 I7mn3 指定其中哪个信号的哪个边沿将导致触发。
- Ixx97 的 0 位指定是硬件捕获计数器的值被用作触发位置---适用于增量编码器信号，真实或模拟的--，还是软件读取位置用作触发位置---适用于其他类型的反馈（0=硬件，1=软件）。如果警告跟随误差状态被用于触发，必须使用该软件读取位置。

请注意，每个电机具有独立的触发功能并相对于触发进行移动，即使是电机被分配到相同的轴。如果多台电动机需要一个共同的触发信号，对于所有这些电机该相同的触发信号必须连接到标志输入。

Turbo PMAC 将按照手动慢跑/回零加速度参数 Ixx19、Ixx20 和 Ixx21，平滑融合每个电机从预触发移动到后触发移动。

在运动程序中 Turbo PMAC 进一步的运动计算之前，所有电机必须停下来，无论是停在初始指定的位置，还是在后触发的位置。这意味着没有后触发移动混合到任何后续的运动中。

触发时的传感器位置的捕捉值会存储在一个专用寄存器，用于以后访问。该单位是计数;对于增量编码器，它们是相对于上电或复位位置。

在一个编程的移动--直到--触发运动的开始时，Turbo PMAC 将电机的“home-search-in-progress”（回零-搜索-进行）状态位（一个**?** 命令返回的第一个电机状态字的第 10 位）设定为真（1）。当找到触发时，或在运动结束时，该位被设置为假（0）。

在一个编程移动直到触发运动的开始，Turbo PMAC 也将电机触发移动状态位（一个**?** 命令返回的第二个电机状态字的第 7 位）设置为真（1），并保持它为真至少直到结束移动。如果在移动过程中发现一个触发，在后触发运动的结束时，该位被设置为假；但是，如果预触发移动结束而没有找到一个触发，那么在运动

的结束时，该位保留为真。因此，该位可用于在移动结束时，告诉我们是否发现了触发。

改变目的地的移动

Turbo PMAC 为您提供了一种能力，可以通过发出一个在线命令，在某些运动的执行过程中，改变那些运动的目的地。这允许你开始以暂定的目标移动，然后在移动过程中更改目标，并可以顺利过渡到改变后的目的地。如果当前没有执行移动，此功能也允许命令一个简单的编程移动，而无需使用一个程序缓冲区。

这种技术只适用于快速模式移动。快速模式是唯一一个可以动态改变其目的地的运动模式，并且是唯一可以达到新目的地的运动模式。

“改变目的地”的命令

此功能的实现是通过在线的坐标系特定的命令**!{轴} {常数}[{轴} {常数}...]**或其变型**!{轴} Q {常数} [{轴} Q {常数}...]**。感叹号标识该命令为在线的改变目的地的命令。轴字母和相关的值指定新的目的地。

在第一种情况下（如**!X3.0Y2.7**），与每个轴字母关联的常数值直接指定轴的新目的地。通常，此第一种情况适用于当指令从主计算机发出时。

在第二种情况下（如**!XQ21YQ22**）时，“Q”字符后的与每个轴的字母关联的常数值为坐标系统指定 Q 变量的编号，该变量的值表示该轴的新目的地。例如，如果 Q21=3.0 和 Q22=2.7，然后**!XQ21YQ22** 相当于**!X3.0Y2.7**。通常情况下，第二种情况下适用于命令从 Turbo PMAC PLC 程序发出时。

在此命令中指定的值始终是新目的地的位置（相对于“程序零”），而不是相对以前指令位置的距离。也就是说，这个命令在绝对模式是始终有效的，无论轴是在绝对或增量模式。如果轴是在增量模式下，他们将留在增量模式用于后续的缓冲程序命令。

如果在发出此命令时，没有进行中的指令移动，Turbo PMAC 将执行一个快速模式的点到点移动到指定的坐标。这使得相当于一个“手动慢跑到位置”的指令，虽然目的地以用户单位指定，而不是计数。

如果发出命令时，有一个正在进行中的快速模式移动，Turbo PMAC 将每个电机的当前轨迹延长 Ixx92 毫秒。在那一点上，它会闯入到每个电机的轨迹，为每个电机计算出光滑的混合到快速模式的轨迹中，向着新的目的地，并执行修改后的轨迹。因为改变目的地的移动本身就是一种快速模式移动，其目的地可以用随后的“改变目的地“命令进行修改。

如果发出此命令时，一些其他模式的移动正在进行中，Turbo PMAC 将通过一个错误来拒绝命令。

使用“改变目的地”

“改变目的地”命令最常用于修改快速模式移动的目的地，从坐标系的旋转运动程序缓冲区中执行作为该缓冲区的最后一个移动。在典型的应用中，启动快速移动，用大致想法确定最终目的地，而一些传感器，如视觉系统，确定精确的位置。“改变目的地”命令与最终目标的精确坐标发送到坐标系中。

如果移动结束前没有收到“改变目的地”命令，在运动到最终位置开始之前会有一个短暂的停顿，但所有的轴最终停止在相同的位置，就好像移动结束之前接收了该命令。然而，请注意，在这种情况下，在初始运动的结束时，某些状态

位例如“预期-速度-零”，和“就-位”被置位，所以不能指望他们来表明已经达到修改的终点。

“改变目的地”命令也可以用来修改一个未在旋转缓冲区的末尾的快速模式移动，或一个在“固定”的运动程序缓冲区的运动。在这种情况下，需要注意几个事情。首先，如果轴在增量模式，程序中随后的运动通过改变目的地来修改。其次，如果在快速模式的移动结束后收到了改变目的地的命令，这可能是一个错误而被拒绝，具体取决于随后执行什么样的程序。

16-10 样条曲线模式移动

Turbo PMAC 可以执行两种类型的三次样条（位置 vs 时间的公式的三次方）来将轴上一系列的点混合在一起。其 SPLINE1 模式是一个均匀的非有理三次 B 样条，而其 SPLINE2 模式是一种非均匀的非有理 B 样条。当然，它可以为所有轴同时执行任一样条。样条曲线特别适合于奇异(非笛卡尔)几何形状，如 radial tables 和旋转轴机器人，那里是奇怪的轴轮廓形状甚至对于定期的刀尖运动。

工作原理

在 SPLINE1 模式下，长移动被分成时间相等的段，每段是 TM 或 TA 时间（取决于全局变量 I42 的设置）。运动程序中每个段都通过一个比如 X1000Y2000 正常的移动命令每个轴都给出一个目标位置。查看在此之前以及在此之后的移动命令，Turbo PMAC 为每个轴创建一个三次的“位置 VS 时间”的曲线，这样使得在段的边界没有速度或加速度的突变。在分段边界的指令位置可以是略“宽松”的，以满足速度和加速度的限制（参见下图）。

在实际的样条计算中使用的样条移动时间是一个 24 位的定点值，整数为 12 位和小数为 12 位。这提供了一个范围可达 4096 毫秒（刚刚超过 4 秒）和约 1/4 微秒的分辨率。如果 I42 设置为默认值 0，此时间被指定在一个 TM 命令中，它支持小数分辨率。如果 I42 被设置为 1，此时间被指定在 TA 命令中，其不支持小数分辨率。这种模式主要是为了与旧的非 Turbo PMAC 应用程序进行兼容性。

Turbo PMAC 通过采取指定点 Pi 和任一侧指定点进行加权平均，来沿着样条的每个点为各轴计算中间“路径点”WP_i。对于均匀样条，这是根据以下等式进行：

$$WP_i = \frac{P_{i-1} + 4P_i + P_{i+1}}{6}$$

Turbo PMAC 也沿着样条曲线为每个轴计算每个路径点的速度 Vi。在均匀的样条曲线，它的完成是通过拾取路径点任一侧分段的平均速度间的中途速度：

$$V_i = \frac{(P_{i+1} - P_i) + (P_i - P_{i-1})}{2T} = \frac{P_{i+1} + P_{i-1}}{2T}$$

对于非均匀样条曲线，完成类似的计算。

计算完在分段边界的精确的位置和速度， Turbo PMAC 计算满足这些约束条

件的独特的三次位置方程（抛物线速度分布图），并使用该方程进行插值。

增加段

在一系列样条动作开始和结束时，Turbo PMAC 自动为每个轴添加一个相同时间段的零距离段，并在该段与相邻段之间执行样条曲线。这将导致 S 曲线加速到停止或从静止加速。

位置调整量化

在均匀样条曲线中，对于分段 i 结尾处的一个轴，其样条指令位置和预样条（程序行）指令位置之间的差值，可以根据简单的等式来计算：

$$\mathrm{Diff_i} = \frac{\mathrm{Dist_{i+1}} - \mathrm{Dist_i}}{6}$$

其中 $\mathrm{Dist_i}$ 是样条曲线分段 i 的编程距离（无论是绝对量或增量模式），$\mathrm{Dist_{i+1}}$ 是分段 i+1 的编程距离。

5 点样条修正

在轮廓加工应用中，通常需要尽可能接近地通过一系列的点。在这些应用中，标准样条算法引入的误差可能太大而不能容忍。但是，在均匀的样条曲线，一个简单的预补偿可以明显降低样条误差。对于样条曲线的每个点 Pi，在发送到 Turbo PMAC 之前，用点 P’i 取代，用下列公式计算：

$$\mathrm{P_i'} = \frac{-\mathrm{P_{i-1}} + 8\mathrm{P_i} - \mathrm{P_{i+1}}}{6}$$

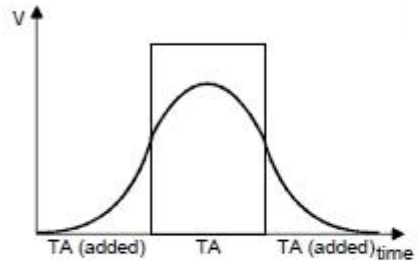
非均匀样条

Turbo PMAC 的 SPLINE2 模式类似于 SPLINE1 模式，除了不要求 TA 样条段的时间保持恒定之外。这使得 SPLINE2 模式成为非均匀的非有理的三次 B 样条，而 SPLINE1 模式是一个均匀的非有理的三次 B 样条曲线。“非有理”的规范表示在样条曲线的不同点没有独立的加权比重（比例）。

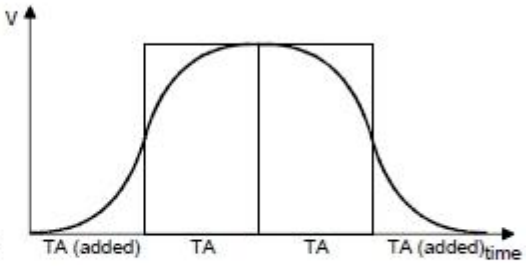
在样条曲线开头的增加段与第一个程序段具有相同的时间，在样条曲线的末尾增加的段与最后的编程段具有相同的时间。

在 SPLINE2 连续样条曲线中，任何三个连续分段的组合时间必须小于 8,388,608 毫秒，或约 2 小时 20 分钟。

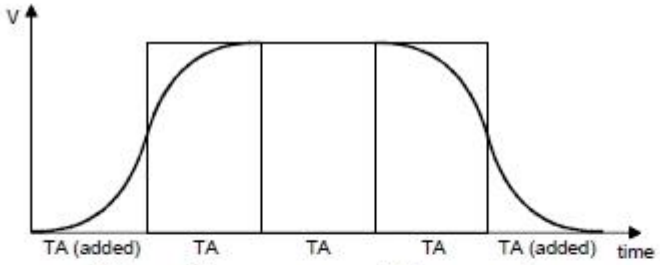
Cubic Spline Trajectories



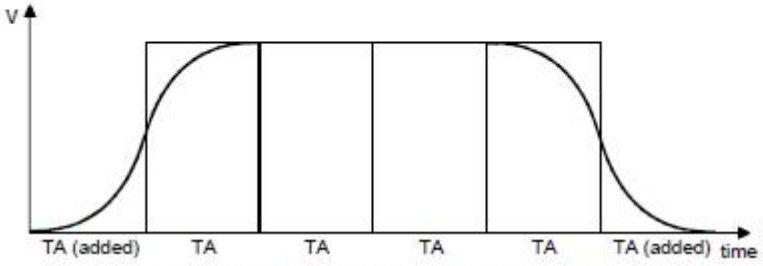
One Programmed Segment



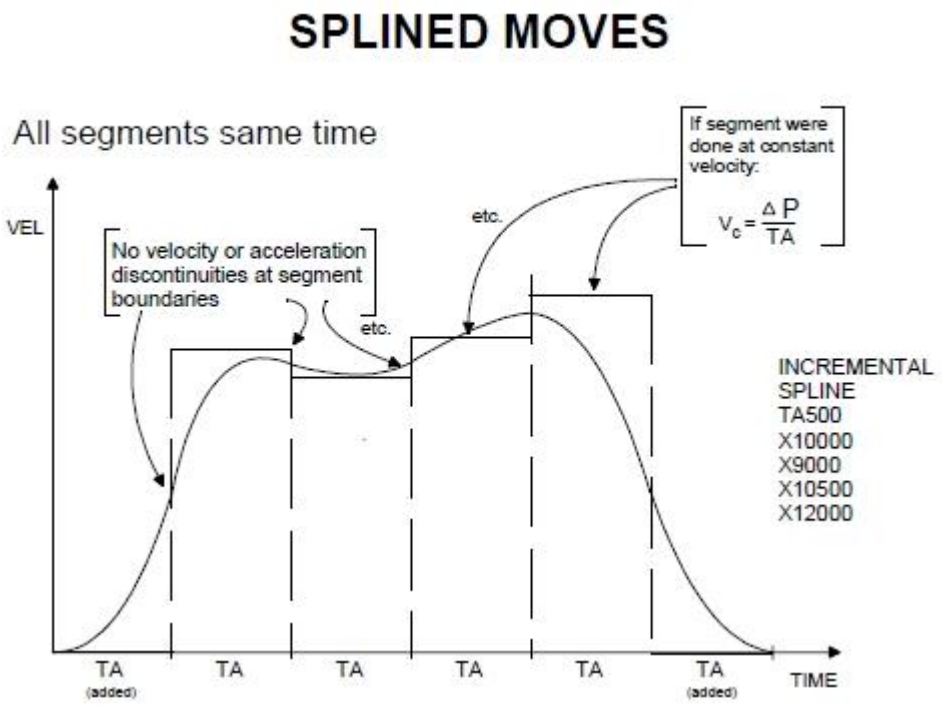
Two Programmed Segments

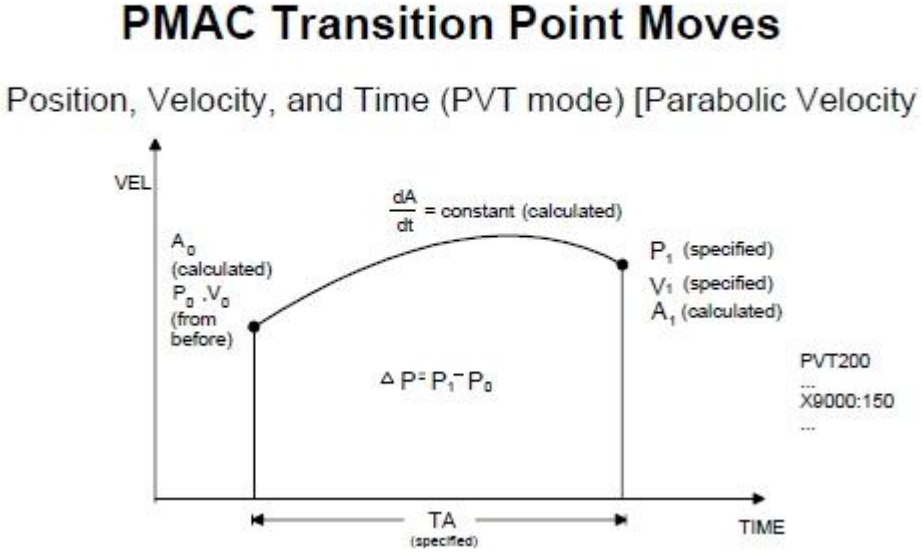


Three Programmed Segments



Four Programmed Segments





16-11 PVT 模式移动

对于想更加直接控制轨迹轮廓的用户，Turbo PMAC 提供位置-速度-时间（PVT）的移动模式。在这些运动中，你可以在移动之间转换时直接指定轴的状态（不同于混合运动）。这需要主机进行更多的计算，但允许更严格的控制轮廓形状。对于移动的每一步，您指定结束的位置或距离、结束的速度和时间段。

模式声明

将 Turbo PMAC 置于这种模式下是通过程序语句的 **PVT{数据}**，其中{数据}是浮点常量、变量或表达式，代表时间段，以毫秒为单位。如果 I42 设置为默认值 0，Turbo PMAC 将这个值转换为 24 位值，12 位的整数和 12 位的分数。这提供的范围可达 4096 毫秒（刚刚超过 4 秒）和约 1/4 微秒的分辨率。如果 I42 设置为 1，Turbo PMAC 将其转换成一个 12 位整数值，不带小数部分（四舍五入到最接近的整数）。这种模式主要是为了与旧的非 Turbo PMAC 应用程序进行兼容性。

移动时间可能在移动之间改变，无论是通过另一个 **PVT{数据}**声明，或用 **TM{数据}**语句如果 I42=0，或 **TA{数据}**语句如果 I42=1。该程序通过另一个移动模式语句（如 **LINEAR**, **RAPID**, **CIRCLE**, **SPLINE**）跳出这种模式。

运动语句

为每个轴指定一个 PVT 模式移动是通过语句形式**{轴}{数据}:{数据}**，其中**{轴}**是一个指定轴的字母；第一个**{数据}**是一个值，指定终点位置或段的距离（取决于轴是处于绝对或增量模式）；第二个**{数据}**是一个表示结束速度的值。

该位置或距离的单位是轴线的用户长度或角度单位，如在轴定义语句中设置的那样。速度的单位被定义为长度单位除以时间单位，其中长度单位与那些位置或距离的单位是相同的，时间单位由坐标系变的量 Isx90（进给速度的时间单位）所定义。给一个轴指定的速度是一个有符号的数量。

Turbo PMAC 的计算

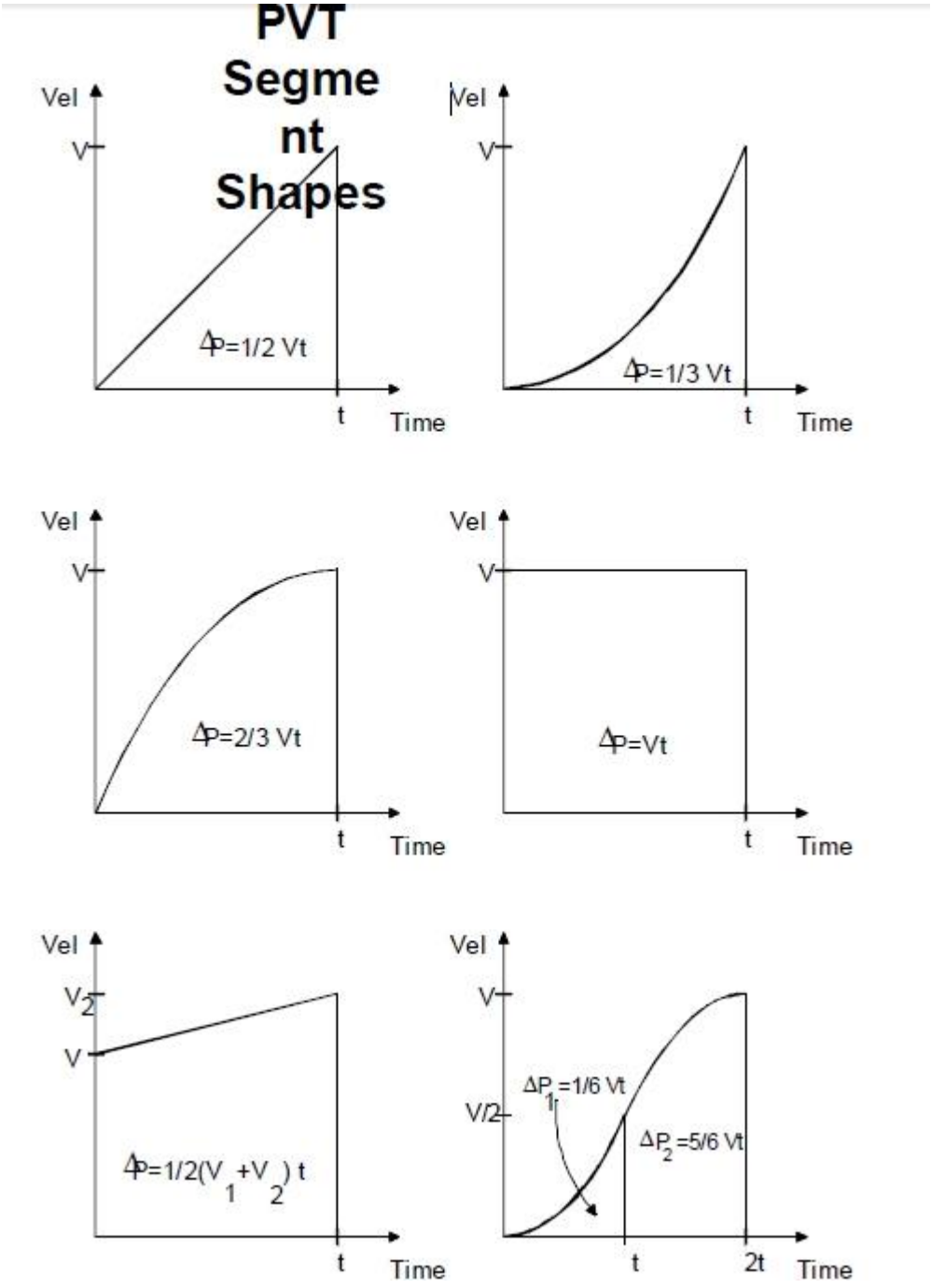
通过运动段，和起始位置和速度指定的参数（从前一个运动段的结尾），Turbo PMAC 计算出唯一的满足约束条件的三阶位置轨迹路径。这会导致运动段的线性地改变加速度，一个抛物线速度分布，和一个三次位置轮廓。

步进中的问题

既然你可以为移动（直接或间接地）指定一个非零的终点速度，所以在一个转换点移动的程序中进行步进不是一个好主意，而且在实时下载这些运动时必须非常小心。通过在一系列 PVT 移动的前后使用 **BLOCKSTART** 和 **BLOCKSTOP** 语句，这些运动的最后一个有零的结尾速度，可以使用一个 Step 命令来执行一个程序的一部分。

利用 PVT 创建任意轮廓

PVT 模式对于创建任意轨迹轮廓是最有用的。它提供了一个“搭积木”的方式来组建抛物线速度段，从而来创建任何需要的整体轮廓。下面的 PVT 段的形状图显示了常见的速度段分布曲线。PVT 模式可以创建任何其他移动模式可以创建的任何轮廓。



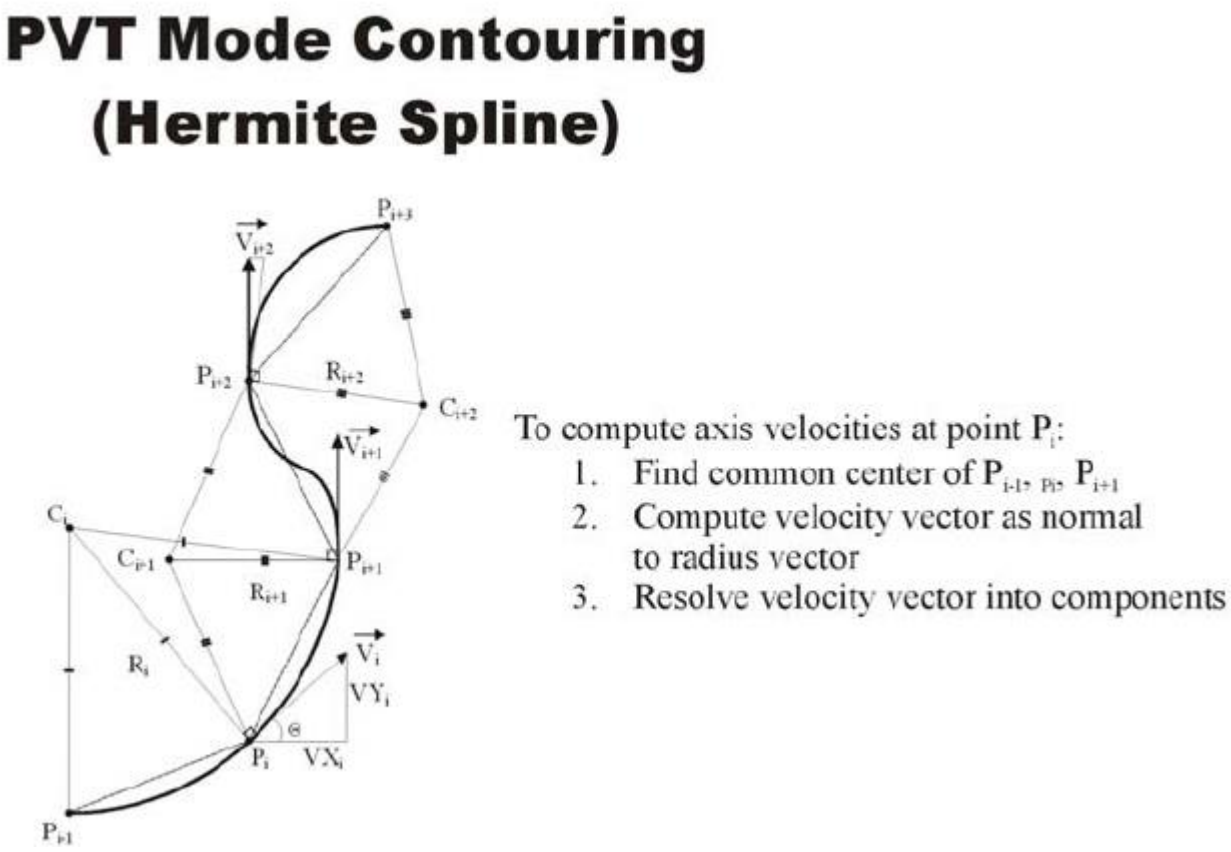
PVT 在轮廓加工中的使用

PVT 模式提供了极好的轮廓加工能力，因为它需要的插补指令路径正是通过编程点。它创建了被称为 Hermite 样条曲线的路径。线性和样条模式分别为第二和第三阶 B 样条，他们可以通过编程点的内部。

相对于 Turbo PMAC 的样条模式，PVT 产生更精确的轮廓。其最坏情况下的误差可估计为：

$$E = \frac{V^4T^4}{384R^3} = \frac{R\theta^4}{384}$$

其中，V 是速度矢量，T 是时间段，R 是曲线的曲率半径，并且 θ 是对向角。



16-12 刀具半径补偿

Turbo PMAC 在它执行的运动中提供了进行刀具（工具）半径补偿的能力。这种补偿可以在 X、Y 和 Z 轴间进行，他们应该是物理上彼此垂直。该补偿通过一个编程量为垂直于路径的运动所描述的路径自动提供偏移量，以补偿该刀具的尺寸。这使您可以编程沿着刀具的边缘进行的路径，让 Turbo PMAC 基于可以独立于程序而指定的半径大小来计算刀具中心轨迹。

Turbo PMAC 支持二维（2D）和三维（3D）刀具半径补偿。在更旧的和更普遍的 2D 补偿中，稍后在下文进行说明，首先指定补偿平面，然后是相对于路径和半径幅度的补偿方向。在较新的三维补偿中，指定表面法线向量和刀具方向矢量以及刀具的主要和次要的半径。

刀具半径补偿只适用于直线和圆弧移动模式。运动必须用 F（进给速度）来指定，而不是 TM（移动时间）。Turbo PMAC 必须是在移动分段模式下（Isx13>0）来进行补偿。（对于圆弧模式，总是要求 Isx13>0）

注意：

在 CIRCLE 模式，无任何圆心说明的移动将产生一个线性移动。这个运动被正确执行而没有有效的刀具半径补偿，但如果补偿是激活的，在这种情况下，补偿不会被正确地应用。线性移动必须执行在线性模式才能进行正确的刀具半径补偿。

定义补偿平面

要进行补偿必须指定几个参数。首先，执行补偿的平面必须使用缓冲的运动程序 **NORMAL** 命令来完成设置。任何在 XYZ 空间中的平面都可以被指定。这是通过指定一个垂直于该平面上的矢量来完成的，I、J 和 K-分量分别平行于 X、Y 和 Z 轴。

例如，在 **NORMAL K-1** 中，通过描述一个在负方向平行于 Z 轴的矢量，指定 XY 平面进行右/左补偿（**NORMAL K1** 也使用 XY 平面，但与前一个命令的右/左补偿恰恰相反）。

这个相同的命令还规定了圆弧插补的平面。**NORMAL K-1** 是默认的。而在补偿有效时，补偿平面不能被改变。

其他常见的设置是 **NORMAL J-1**，其中规定了补偿的 ZX 平面，而 **NORMAL I-1** 指定 YZ 平面。法线矢量的这三个设置分别对应于 RS-274 G 代码 G17、G18 和 G19。如果在 Turbo PMAC 程序 1000 中实施 G 代码，把它们融入到 PROG 1000：

```
N17000 NORMAL K-1 RETURN
N18000 NORMAL J-1 RETURN
N19000 NORMAL I-1 RETURN
```

定义补偿幅度（刀具半径）

补偿的幅度--刀具半径--必须使用缓冲的运动程序命令 **CCR{数据}**（刀具补偿半径）进行设置。此命令可以采用常量参数（如 **CCR0.125**），或一个括号中的表达式（例如 **CCR（P10+0.0625）**）。参数的单位是 X、Y 和 Z 轴的用户单位。在 RS-274 类型的程序，通常使用 Turbo PMAC 运动程序 1003 将这些命令纳入“刀具数据”D 代码。

刀具半径可以是负值和零值。请注意，在正和负幅值之间变化是不同于改变补偿方向的。请参阅本手册的《补偿的变化》的部分。另外，在一个非零幅值和零幅值之间进行改变是不同于接通和关断补偿的。请参见下面的相应章节。

开启补偿

这种补偿是由缓冲运动程序命令 CC1（左偏移）或 CC2（右偏移）打开的。这些分别等同于 RS-274 G 代码 G41 和 G42。如果要在 Turbo PMAC 运动程序 1000 实现 G 代码子程序，把它们融入到 PROG 1000：

```
N41000 CC1 RETURN
N42000 CC2 RETURN
```

关闭补偿

该补偿是通过缓冲的运动程序的命令 CC0 关闭的，它相当于 RS-274 G 代码 G40。如果要在 Turbo PMAC 运动程序 1000 实现 G 代码子程序，把它们融入到 PROG 1000：

```
N40000 CC0 RETURN
```

Turbo PMAC 如何引入补偿

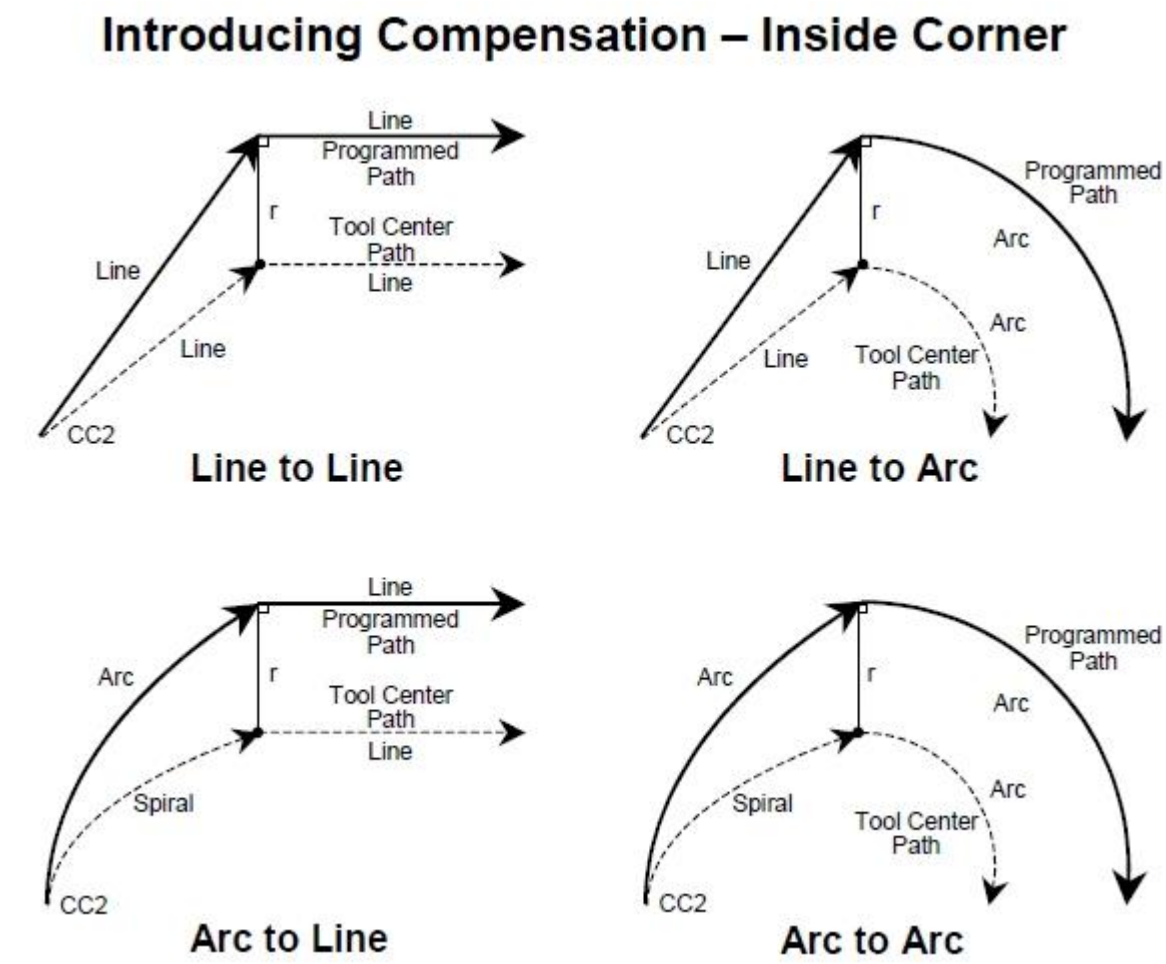
随着 CC1 或 CC2 命令打开补偿，Turbo PMAC 逐步在下一个线性或圆形模式的移动中引入补偿。这个引入运动结束在一个点，而该点距离（引入运动和第一个完全补偿移动的）交叉点有一个刀具半径那么远，同时（编程点到该补偿终点的）连线垂直于（第一个完全补偿的移动在交叉点的）路径。

注意：

很少有控制器可以使他们的引入移动是一个圆弧方式的移动。这种能力允许与切割面建立的接触非常轻柔，对于精加工切削很重要。

内角引入

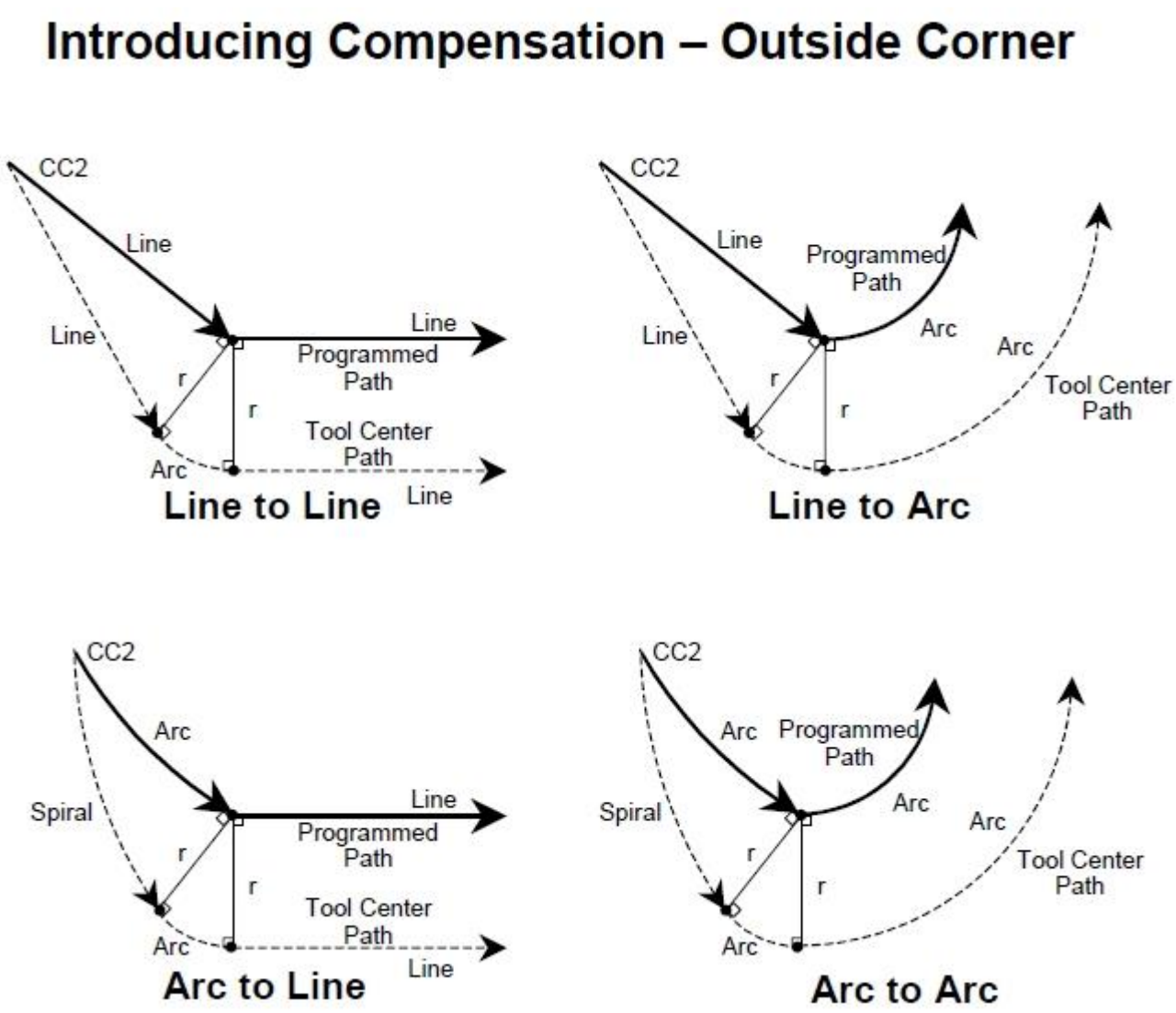
如果该（补偿）引-入运动和第一个完全补偿的运动形成一个内角，那么引入运动直接运动到这一点。当引-入移动是一种线性模式运动时，补偿后的刀具路径（切削轨迹）将在编程运动路径的一个对角线上。当引入移动是一个圆弧模式运动时，补偿后的刀具路径将是一个螺旋线。



外角引入

如果引入移动和第一个完全补偿的移动形成一个外角，引入移动首先移动到一个点，而该点距离（引入运动和第一个完全补偿移动的）交叉点是一个刀具半径那么远，同时（编程点到该补偿终点的）连线垂直于（引入移动在交叉点的）路径。当引入移动是一个线性模式移动时，这个补偿后的刀具路径将在编程的移动路径的对角线上。

当引入移动是一个圆弧模式移动，这个补偿的刀具路径将是一个螺旋。然后是添加一个（半径等于刀具半径的）圆弧移动，运动结束在一个点，而该点距离（引入运动和第一个完全补偿移动的）交叉点有一个刀具半径那么远，同时（编程点到该补偿终点的）连线垂直于（第一个完全补偿的移动在交叉点的）路径。

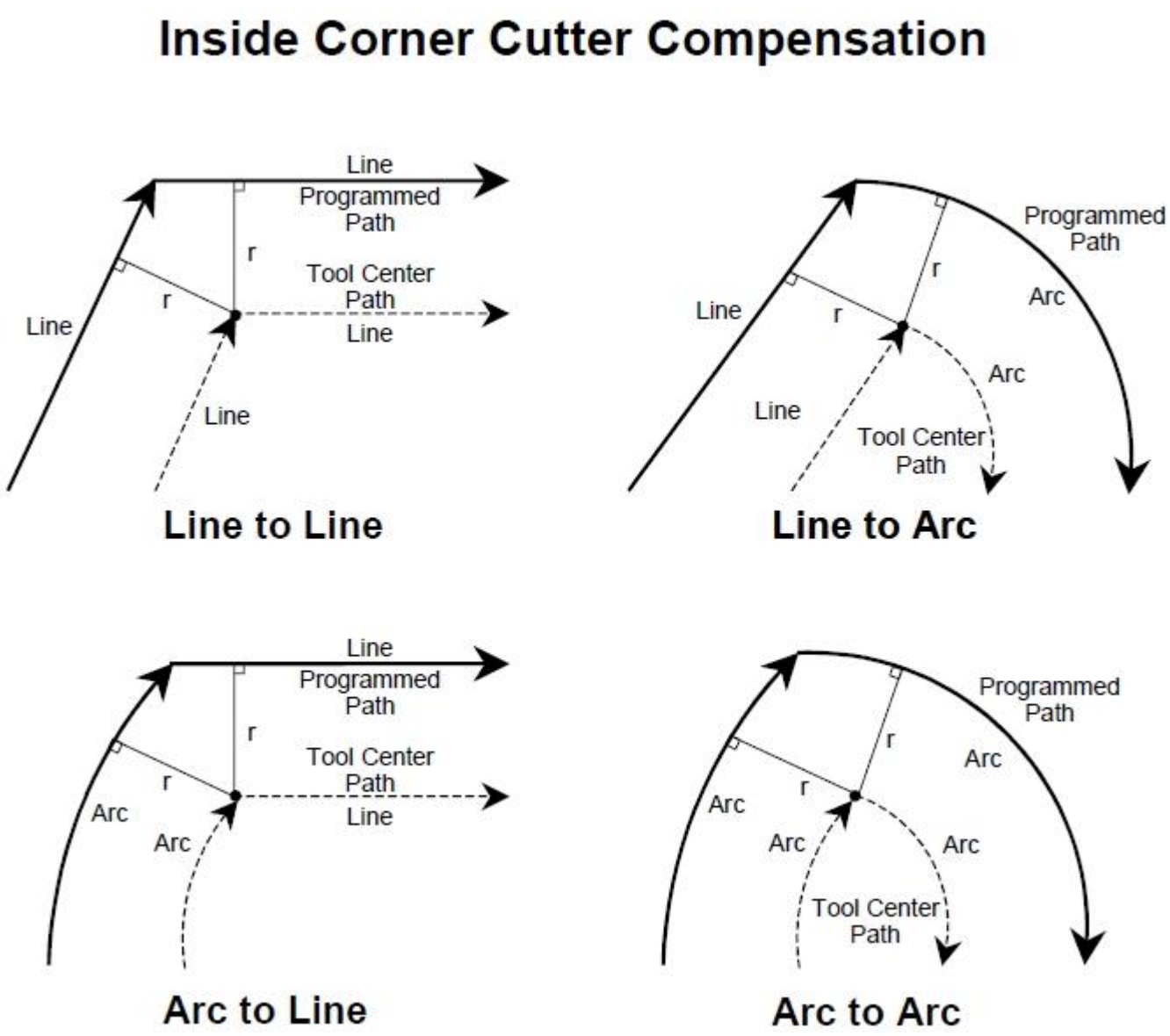


请注意，在补偿有效时，引入运动不同于补偿半径从零值变为一个非零值。无论 Isx99 是什么设置，在角处总是添加一个圆弧移动。这保证了引入移动不会切入到第一个完全补偿的移动中。

补偿的内角处理

由于有效的 TA 和 TS 时间（默认值分别通过坐标系的 I-变量 Isx87 和 Isx88 设定），内角仍然服从混合。。加速时间越大，圆角半径越大。（圆角开始和结束，相对于补偿的但非混合的角 F*TA/2 的距离）。混合部分为 S-曲线的部分越大，角会越方正。

当完全停止（如 Step，Quit，或 DWELL，在拐角处）在一个内角时，Turbo PMAC 将停止在补偿的但非混合的拐角点处。



外角处理

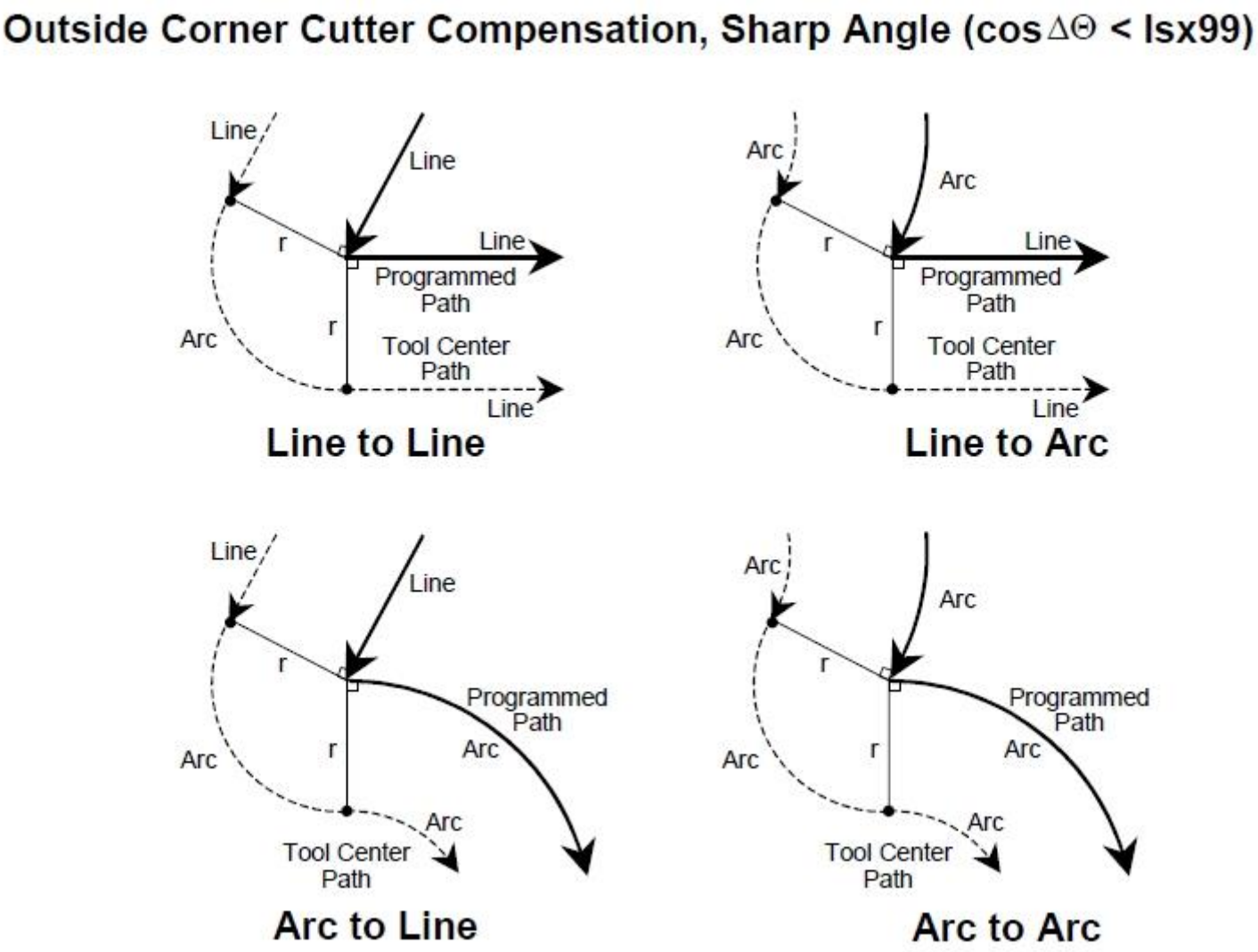
对于外角，Turbo PMAC 要么直接将传入和传出的运动融合在一起，或者增加一个弧线移动来覆盖拐角处的额外距离。它选择哪个是取决于两个移动的相对角和 I-变量 Isx99 的值。

两个运动之间的相对角度表示为，补偿平面的运动矢量的定向角的变化。如果这两个运动是完全一样的方向，定向角的变化是 0°；如果有一个直角拐角，变化为+/-90°；如果有一个完全的反向，定向角的变化是 180°。

Isx99 指定在直接混合的外角和添加弧的外角之间的边界角。它是用边界运动的定向角的变化的余弦值表示的（cos0° = 1.0，cos90° = 0.0，cos180° = -1.0）。定向角的变化等于 180 度减去拐角处的“夹角”。

尖锐的外角

如果定向角变化的余弦值小于 Isx99，这意味着拐角比指定的角度要尖锐，然后将一个圆弧移动添加在拐角处的外围。

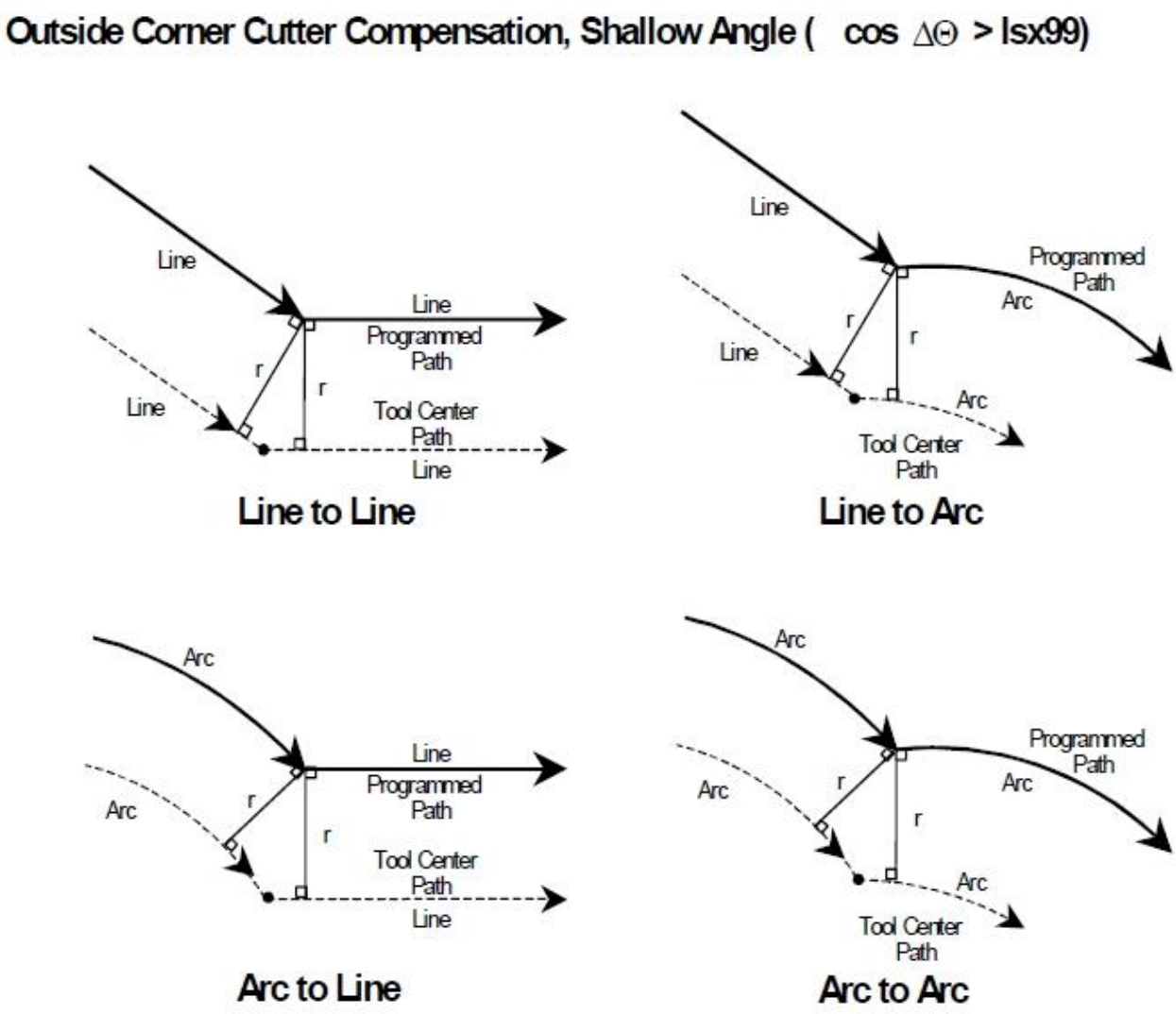


平坦的外角

然而，如果在定向角的变化的余弦大于 Isx99，表示该角比规定的角度较平坦，则运动将直接混合在一起，没有附加的弧。

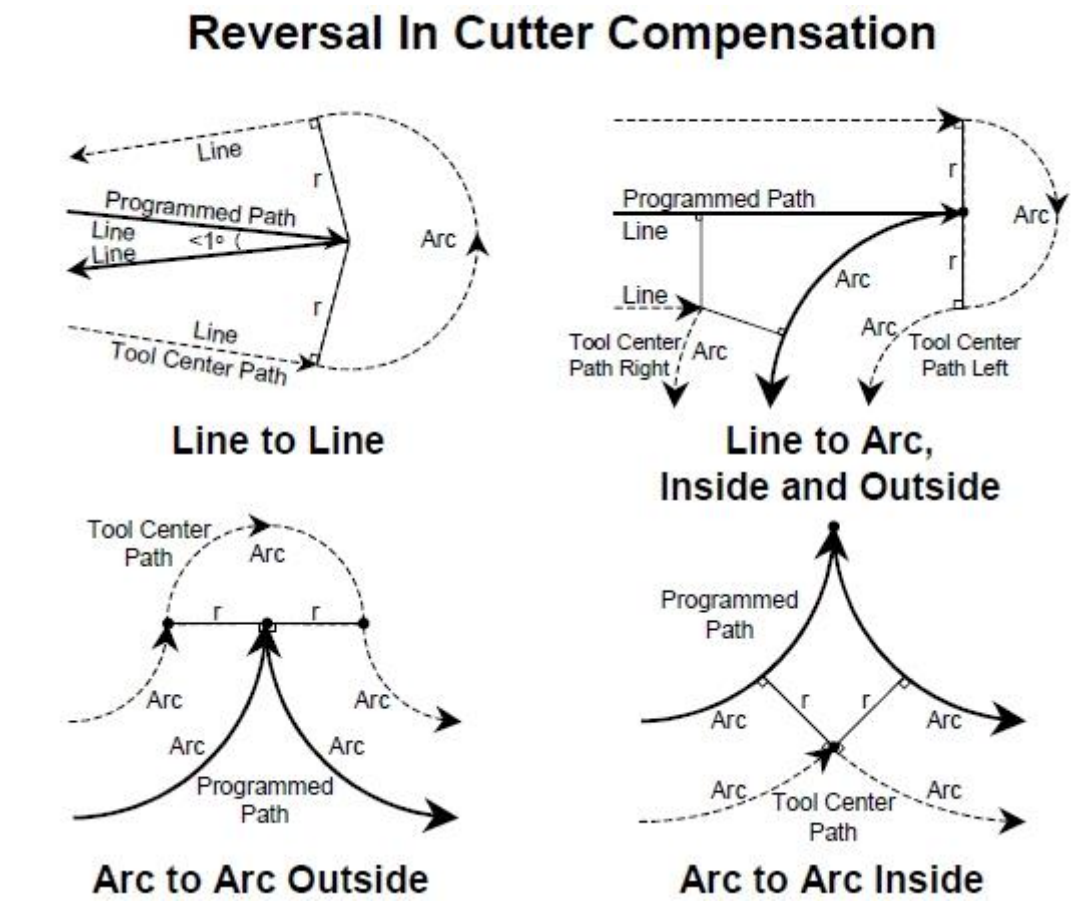
增加的圆弧防止补偿角在一个尖角外面延伸得过远。然而，作为一个附加的移动，它加速的时间是最小的时间，这可能造成在一个非常小的角度减速。虽然 Isx99 的 0.9998（cos1°）的默认值会导致任何角度变化超过 1° 时会添加一个圆弧，但是很多用户都会设置 Isx99 为 0.707（cos45°）或 0.0（cos90°），这样圆弧只会添加在尖角处。

当在一个带附加圆弧的外角处有一个完全停止时（如 Step、Quit，/，或 DWELL），Turbo PMAC 将在停止前包含该添加的圆弧运动。当在一个无添加弧的外角处有一个完全停止时，Turbo PMAC 将停止在补偿的但非混合的拐角点处。



完全反向的处理

如果在两个连续的补偿运动边界处的定向角度的变化是 180° ±1° （夹角小于 1° ），这被认为是“完全反向”并适用特殊规则。如果传入和传出的运动都是直线运动，拐角始终被认为是一个外角，以及添加一个约 180° 的圆弧移动。如果一个运动是或两个运动都是圆弧，Turbo PMAC 将检查补偿运动的内侧交点。如果找到这样一个交叉点，拐角处将被视为一个内角。否则，将被视为一个外角与一个附加的 180 度圆弧运动。

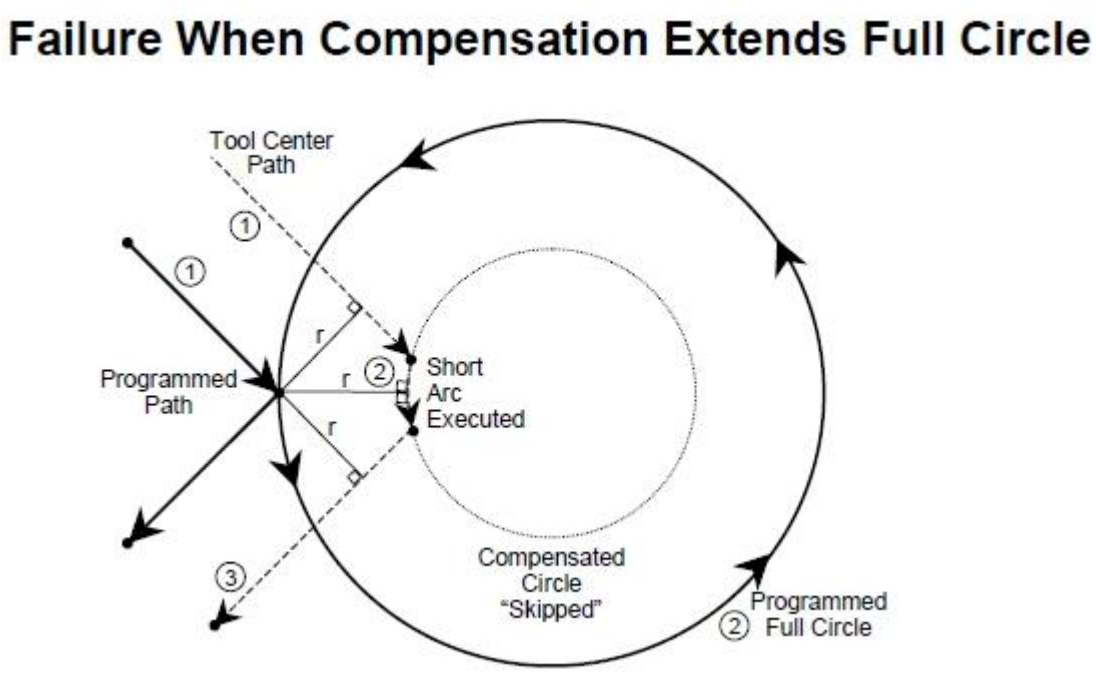


注意完整的圆

如果在刀具半径补偿中执行一个完整的圆移动，并且在一端或两端产生一个直接混合的平坦外角（不加圆弧---参见外角处理，上文），补偿的圆弧运动将延长超过 360 度，而且 Turbo PMAC 可能只是产生一个很短的弧，比预期的短 360 度（使得它看上去圆已经被“跳过”。

虽然这通常是草率的编程的结果----一个外角带一个完整的圆引起了一个过切到圆中---许多机器设计人员可能希望允许这种轻微的情况。坐标系参数 Isx97 定义了可被执行的最短弧角；最长的圆弧角为 360° 加上该角度。

Isx97 的默认值设置最小弧角为半圆的百万分之一，足以算作数字舍入，但有时对于补偿完整的圆还不够。为了处理这些情况，Isx97 应设置为一个比较大的值。



补偿移动的速度

补偿路径的刀具中心速度与通过 F 参数编程的速度是相同的。在一个圆弧移动，这意味着刀具刃口的速度（刀具与零件接触的部分）与由分数 R_{tool}/R_{arc} 编程的速度是不一样的。

补偿中的变化

当补偿是有效时，Turbo PMAC 允许同时改变补偿的半径和方向。在这些情况下，了解 Turbo PMAC 究竟是如何改变补偿路径的是非常重要的。

补偿半径的变化

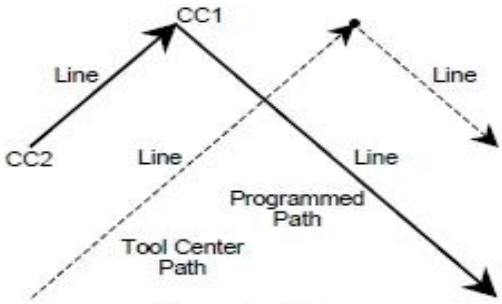
在补偿有效时，补偿半径（新 CCR 值）大小的变化是线性地引入在下一步的运动中。当这种变化是随着线性模式移动过程中引入时，补偿的刀具路径将处于编程移动路径的一个对角线上。当这种变化是通过圆弧模式移动过程中引入时，补偿的刀具路径将是一个螺旋。

补偿方向的变化

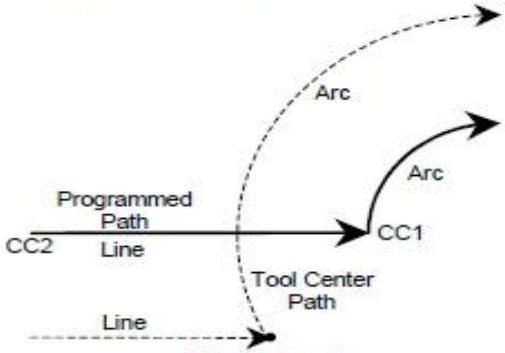
在补偿的过程中，补偿（CC1 和 CC2 之间）方向的变化通常是在两个运动之间的边界处引入的。

但是，如果在两个补偿移动路径之间没有交点，这种变化被线性引入在下一个移动中。

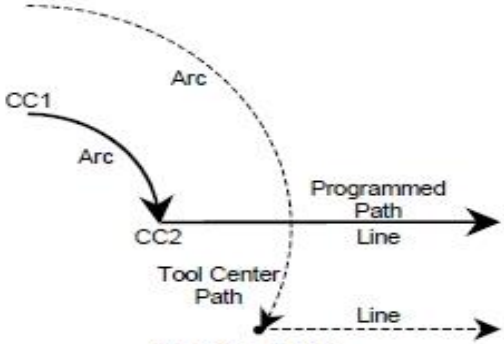
Cutter Compensation Change of Direction



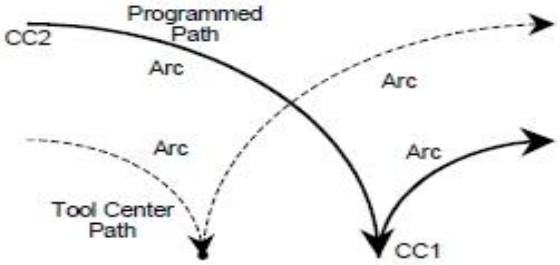
Line to Line



Line to Arc

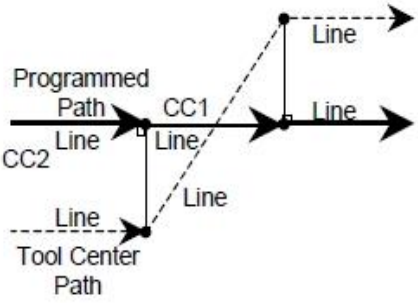


Arc to Line

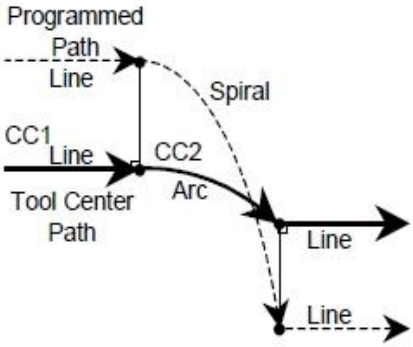


Arc to Arc

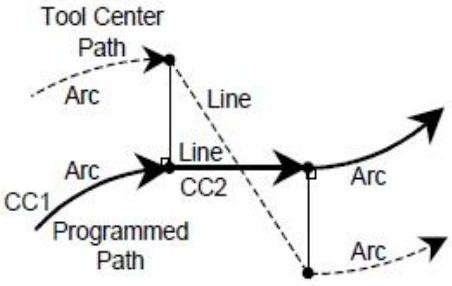
Cutter Compensation Change of Direction – No Intersection



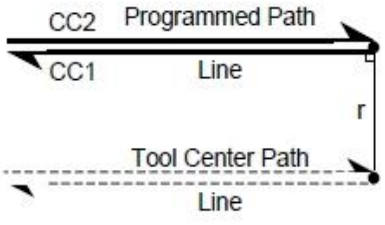
Change Through a Line



Change Through an Arc



Change Through a Line



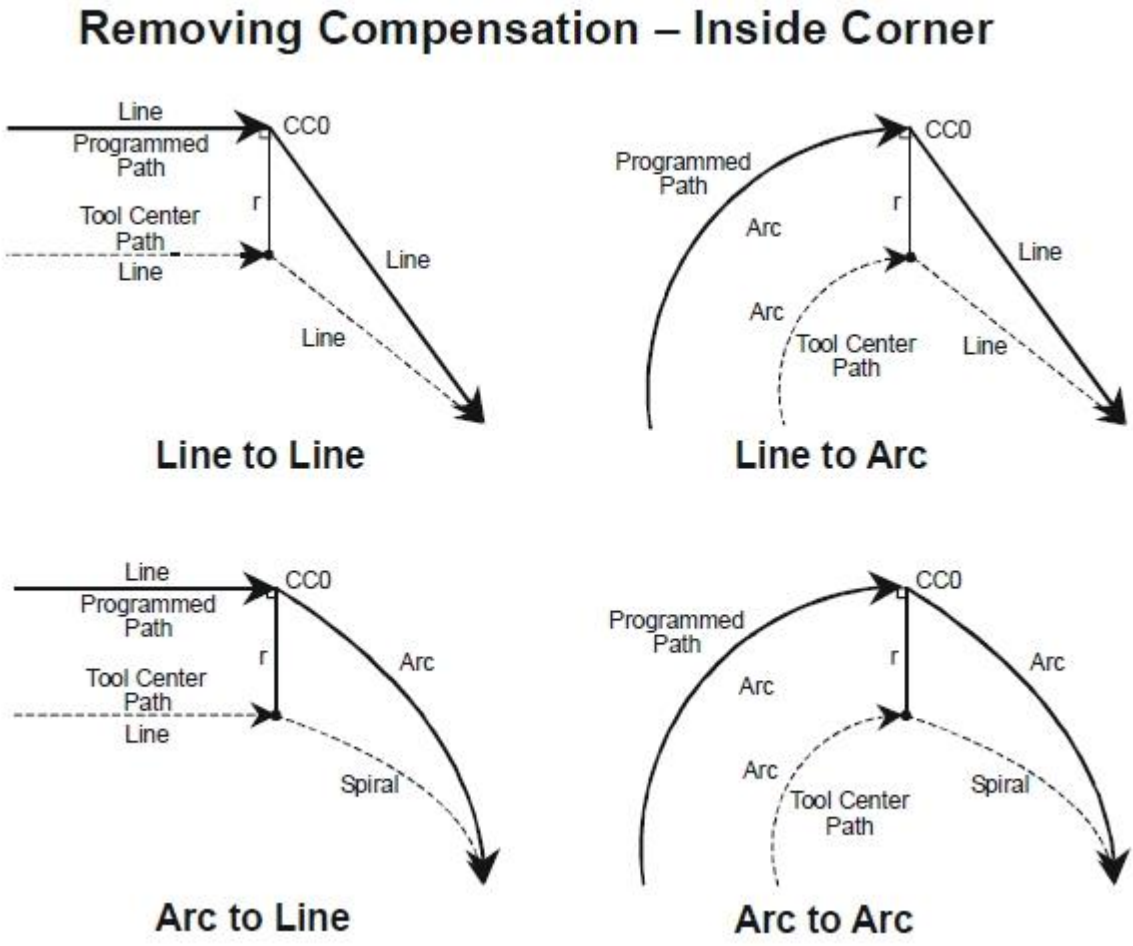
Change in Reversal

Turbo PMAC 如何移除补偿

随着 CC0 命令关闭补偿，Turbo PMAC 在下一个线性或圆弧模移动中逐渐消除补偿。这个引出移动开始于一个点，该点相对（引入移动和第一个完全补偿的移动的）交点有一个刀具半径的距离，同时（编程点与该补偿终点的）连线垂直于（第一个完全补偿移动在交点处的）路径。需要注意的是很少有控制器可以使他们的引出移动是一个圆弧方式的移动。这种能力允许非常轻柔地释放与切割面接触，对于精加工切削很重要。

内角

如果最后完全的补偿移动和引出移动形成一个内角，引出移动直接开始从这个点运动到编程终点。当引出移动是一个线性模式移动时，补偿后的刀具路径将处于编程移动路径的一个对角线上。当引出移动是一个圆弧模式移动时，补偿后的刀具路径将是一个螺旋。



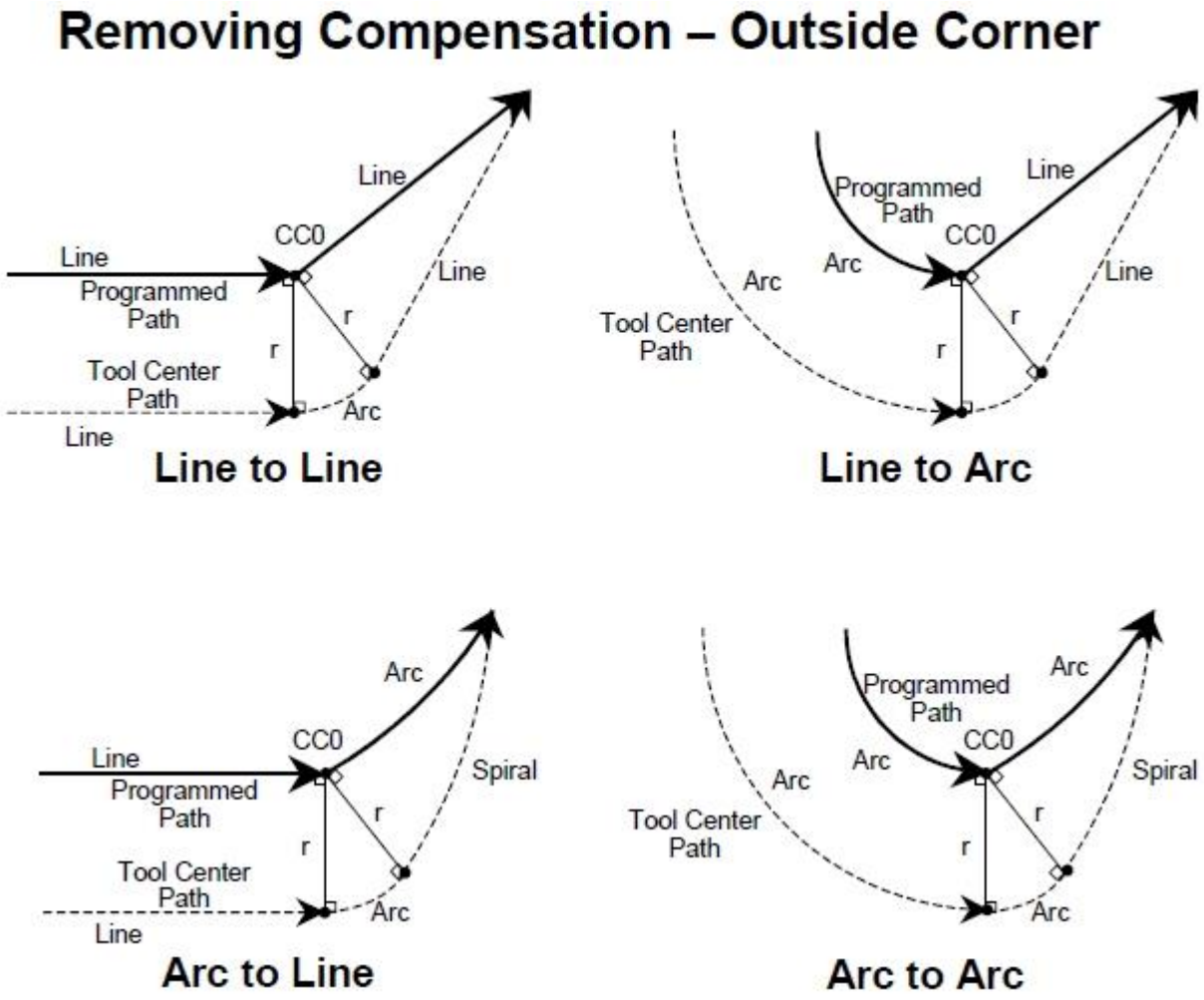
外角

如果最后一个完全补偿的移动和引出移动形成一个外角，最后的完全补偿移动结束在一个点，该点相对（最后的完全补偿移动和引出移动的交点）是一个刀具半径的距离，同时（编程点到这个补偿点的）连线是垂直于（交点处的完全补偿移动的）路径。

Turbo PMAC 然后添加一个圆弧移动，半径等于刀具半径，结束在一个点，该点的距离也是一个刀具半径，同时（编程点到这个补偿端点的）连线垂直于（引

出移动在交点处的）路径方向。

最后，Turbo PMAC 随着引出移动本身而逐渐消除补偿，在引出移动的程控终点结束。当引出移动是一个线性模式移动时，这个补偿的刀具路径将处于编程移动路径的一个对角线上。当引出移动是一个圆弧模式移动时，这个补偿的刀具路径将是一个螺旋。



请注意，这种消除补偿的行为是不同于补偿有效时改变补偿半径大小为零的。圆弧移动总是被添加在角处，无论 Isx99 是什么设置。这确保了引出移动绝不会切入到最后的完全补偿移动中。

刀具补偿中的故障

有可能 Turbo PMAC 得到的一个程序序列，其中的刀具半径补偿算法将失败，不产生期望的结果。补偿可能会失败，有三种类型的原因：

1、计算无法通过转角

2、内角比半径小

3、内弧半径小于刀具半径

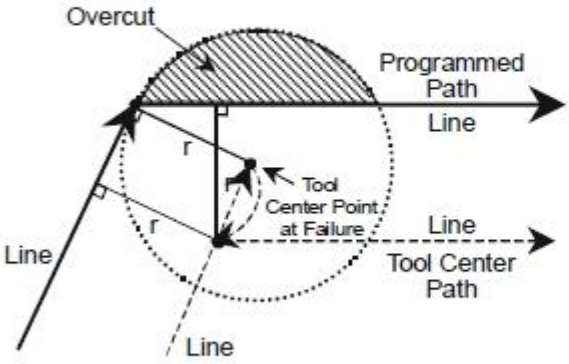
计算无法通过转角

首先，如果在计算目前的移动之前，Turbo PMAC 在程序中无法看得足够远，

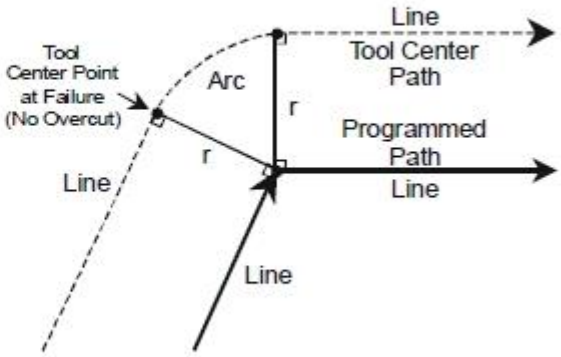
通过补偿的平面上的分量找到下一个移动，那么它将无法计算两个运动之间的交叉点。发生这种情况的几个原因：

- 在补偿平面上的下一步运动之前，有一个运动在补偿平面上没有分量（即垂直于补偿平面，如在 XY 补偿过程中仅在 Z 轴上），也没有声明 CCBUFFER 补偿块缓冲区（见下文）。
- 在补偿平面的下一步运动之前，有很多在补偿平面上没有分量的运动，数量多于 CCBUFFER 补偿块缓冲区可容纳的运动数（见下文）。
- 在补偿平面上的下一步运动之前，有 10 个以上的 DWELL。
- 程序逻辑在运动融合中引起一个暂停（例如，对于一个 WHILE 循环，循环两次）。

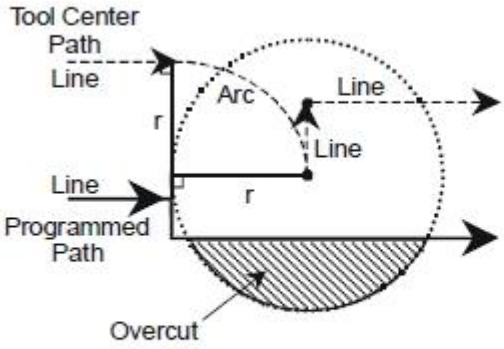
Failures in Cutter Compensation



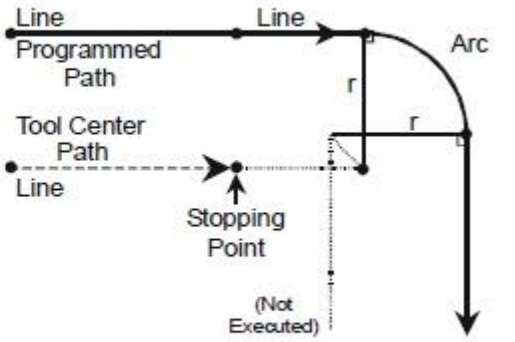
Failure to See Through Inside Corner



Failure to See Through Outside Corner



Inside Corner Smaller Than Cutter Radius



Arc Radius Smaller Than Cutter Radius

如果 Turbo PMAC 不能及时发现下一步的运动，它将结束目前的移动，仿佛交点与下一步运动形成了一个外侧角。如果，下一步的运动的确产生了一个外侧角，或继续的运动直行，补偿将是正确的。在一个外角，一个圆弧移动总是被添加在角处，无论 Isx99 是什么设置。但是，如果下一步的运动创建了一个内角，路径将过切到角中。在这种情况下，Turbo PMAC 移动到正确的交叉位置处并继续下一步的运动，使得过切本地化留在角中。

内角小于刀具半径

其次，如果补偿的路径产生一个内角，而其中的一个运动短于刀具半径，刀具半径补偿将不能正常工作。这种情况导致了补偿运动是在无补偿移动的相反方向，而且在拐角处会有过切。

内弧半径小于刀具半径

第三，如果程序请求一个圆弧移动以补偿到内角，而且编程的圆弧半径小于刀具半径，则无法计算出适当的路径。在这种情况下，Turbo PMAC 在前一个移动的结尾处通过一个运行时错误结束程序，设置寄存器 Y：$002x14 的内部运行时错误代码为 7。

刀具补偿的模块缓冲

当刀具补偿被激活时，如果应用程序需要执行垂直于补偿平面的运动，它会要求定义一个特殊的缓冲区来保存这些运动，同时 Turbo PMAC 提前扫描找出补偿平面的下一步运动，这样就可以计算出输入到该点的移动与传出移动之间的适当交点。

这个缓冲区是通过在线的坐标系特定的命令 **DEFINE CCBUF{常数}**创建的，其中，**{常数}**是一个正整数，代表缓冲区可以存储的垂直于补偿平面的运动数量。这个数字应该至少是与任意两个运动之间连续的垂直移动的最大的数量一样大。

对坐标系定义了此缓冲区，如果 Turbo PMAC 遇到一个或多个运动垂直于补偿平面而同时补偿有效，这些运动将被暂时存储在 CCBUF 中同时找到平面的下一个移动，所以交点可以正确地计算。但是，如果缓冲区没有足够的空间来存储所有找到的垂直移动，Turbo PMAC 将假设一个外角的交点；如果平面的下一个移动实际上形成了一个内角，会发生过切。

当编程运动实际存储在 CCBUF 中时，改变当前位置值的命令---**HOME**、**HOMEZ** 和 **PSET** - 是不允许的。如果 I6 被设置为 1 或 3，Turbo PMAC 将报告一个 ERR019。

该 CCBUF，它存储运动程序块用于计算正确的刀具补偿交点，不应该与 LOOKAHEAD 缓冲区混淆，后者存储这些程序块生成的小的运动“段”用于保证遵从位置、速度以及加速度限制。这两个缓冲区可在坐标系在同一时间被定义和激活。

该 CCBUF 是一个临时的缓冲区。其内容在断电或卡复位时不会保留；只有缓冲区被定义在最后 SAVE 命令时间，而且 I14 被设置为 1，才能在断电或复位时保留。

在补偿中的单步执行

在刀具补偿有效时，执行“单步”方式移动是可能的，但用户应该知道这种操作模式的几个特殊考虑。由于在执行当前移动之前，需要程序提前看得足够远找到补偿平面的下一个移动，所以执行一个 S 单步命令可能不会产生直观预期的结果。补偿移动时的单步命令使得对要执行的移动进行初步的计算，而不是实际已执行的运动。这具有以下影响：

- 在补偿的引入移动时，一个单步命令不会产生任何运动，因为下一步的

运动尚未被发现。

- 补偿平面上补偿的运动时的单步命令将导致执行前一个移动。
- 垂直于补偿平面的补偿运动时，单步命令不会产生任何运动，因为这些被保留在 CCBUFFER 中。在补偿平面的下一个移动时，一个单步命令将导致执行平面上的前一个的移动，以及所有垂直于平面的缓冲的移动。
- 引出移动上的一个单步命令将导致同时执行最后的完全补偿移动和引出移动。

不像许多控制器，刀具补偿有效时，Turbo PMAC 可以通过单步命令执行非运动程序块。然而，你应该知道，执行这些程序块可能会出现乱序，因为前一个编程移动块的运动尚未被执行。

在这种模式下同步的 M-变量赋值仍然是缓冲的、未被执行，直到实际开始执行下一个编程运动。

16-13 三维刀具半径补偿

Turbo PMAC 为它执行的运动提供了进行三维（3D）刀具（工具）半径补偿的能力。这种补偿可以在 X、Y 和 Z 轴中进行，它们应该是物理上彼此垂直的（即使分配给轴的电动机是不垂直的）。不同于更常见的二维（2D）补偿，你可以独立地指定垂直于切削面的偏移向量，和刀具方向矢量。

三维补偿算法自动使用该数据来偏移运动的所述路径，以补偿刀具的尺寸和形状。这允许您编程沿零件表面的路径，让 Turbo PMAC 计算刀具终点的中心路径。

3D 补偿只在直线和圆弧移动模式下有效，并且其目的仅是为用于线性运动。

关于术语的说明：关于旧的二维刀具半径补偿的许多文件指的只是“刀具半径补偿”，因为在当时没有 3D 补偿。3D 补偿的特定文件始终指的是“3D”补偿。

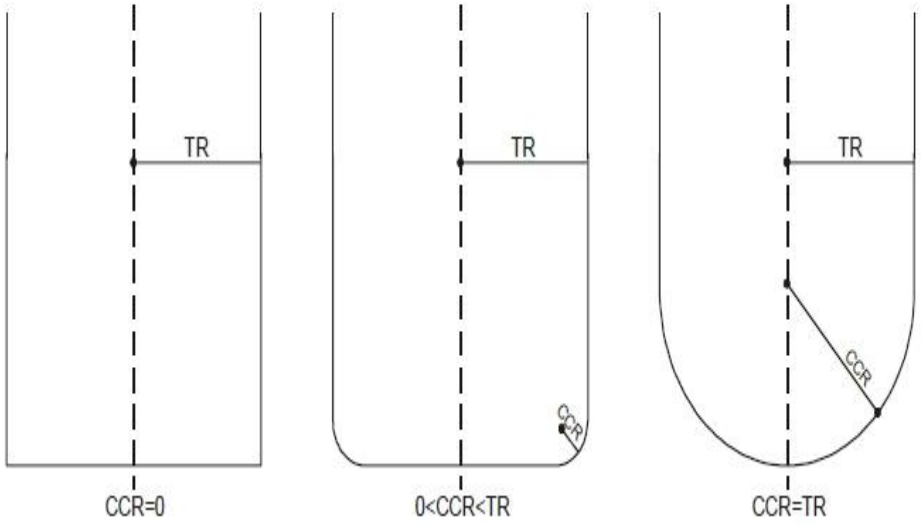
定义 3D 补偿的幅度（刀具半径）

三维补偿的幅度是由两个用户声明的半径值确定的。其中的第一个是刀具的圆角端的半径，由缓冲的运动程序命令 CCR{数据}（刀具补偿半径）设置。此命令可以采用常量参数(如 CCR2.35)，或括号中的表达式(例如 CCR(Q20-0.001))。参数的单位是 X、Y 和 Z 轴的用户单位。在操作中，补偿首先通过刀具的端部半径沿着表面法线向量对路径进行偏移（见下文）。

第二个值是刀具半径本身，该刀具轴的半径。这是由缓冲的运动程序指令 TR{数据}（刀具半径）设置的。此命令可以采用常量参数（如 TR7.50），或在括号中的表达式（例如 TR（7.50-Q99））。参数的单位是 X、Y 和 Z 轴的用户单位。在操作中，接下来偏移路径的补偿数额等于刀具半径减去刀具的端部半径，垂直于“刀具定向”向量（见下文）。

扁平头铣刀的刀端（圆角）半径为零。球头铣刀（半球形头）的刀端半径等于刀具（轴）的半径。其他刀具的刀端半径将在零和刀具半径之间。

3D Compensation: Cutting Tool Cross Sections



开启 3D 补偿

三维刀具补偿是由缓冲的运动程序命令 CC3 打开的。既然是明确地指定偏移向量，则此处没有左或右补偿。当三维补偿被打开时，该表面法线向量被自动设置为空（零幅值）向量，刀具方向的矢量也被自动设置为空向量。

直到一个表面法向量被明确声明而同时 3D 补偿有效，否则不会发生实际的补偿。刀具方向的矢量也必须声明用于任何刀具的补偿工作，除了球头铣刀以外。

关闭 3D 补偿

三维刀具补偿是缓冲运动程序的命令 CC0 关闭的，如同 2D 补偿那样。在补偿已被关闭后，补偿将在下一个线性或圆形模式的移动中被删除。

声明表面法向量

表面法向矢量的方向由一个运动程序行中声明的 **NX{数据}，NY{数据}和 NZ{数据}**来确定的。这些分量的绝对幅值无关紧要，但相对幅值确定方向。方向必须是从表面到刀具。

一般来说，所有三个组成部分应一起声明。如果只有一个或两种成分在程序行声明，剩余的组分被留在他们的旧值，这可能导致不可预知的结果。如果期望一个成分值变为零，它应明确地声明为零。

注意，表面法线向量的坐标必须表示为机器坐标。如果零件是在一个旋转工作台上，这些坐标将一般不相同于零件设计中的原始零件坐标---在发送到 Turbo PMAC 之前，矢量必须旋转成机器坐标。

表面法向量会影响运动程序同一行中的运动补偿，以及所有后续的动作，直到声明另一个表面法向量。通常的做法是，为每个运动声明一个表面法向量，只单独影响那个移动。

声明刀具定向矢量

如果切削工具的定向可以在补偿过程中改变，如在 5 轴加工中，以补偿为目的定向是通过一个工具定向矢量的方法声明的。（如果定向是恒定的，如在 3 轴加工中，定向通常是由补偿平面的法向矢量声明的，尽管也可使用该工具定向矢量。）

刀具定向矢量的方向是由运动程序行中声明的组分 TX{**数据**}、TY{**数据**}和 TZ{**数据**}来确定。这些组分的绝对幅值量无关紧要，但相对幅值用来确定方向。刀具定向矢量的方向并不重要，它可以从基部到顶端，或从顶端到基部。

一般来说，所有三个组成部分应一起声明。如果只有一个或两个组分在一个程序行中声明，其余组分则保留为他们的旧值，这可能导致不可预知的结果。如果期望一个成分值变为零，它应明确地声明为零。

注意，表面法线向量的坐标必须表示为机器坐标。如果零件是在一个旋转的工作台，这些坐标一般将不相同于零件设计中的原始零件坐标。

刀具定向矢量会影响运动程序同一行的运动补偿，以及所有后续的动作，直到声明另一个工具方向矢量。通常的做法是，为每个运动声明一个工具定向向量，只单独影响那个运动。

请注意，这里声明的刀具方向矢量不指挥运动，它只是将刀具指令的角度方向告诉补偿算法。通常，刀具角度的运动已经用 A、B 和/或 C 轴命令指令过了，往往通过一个 Turbo PMAC 的逆运动学子程序进行处理。

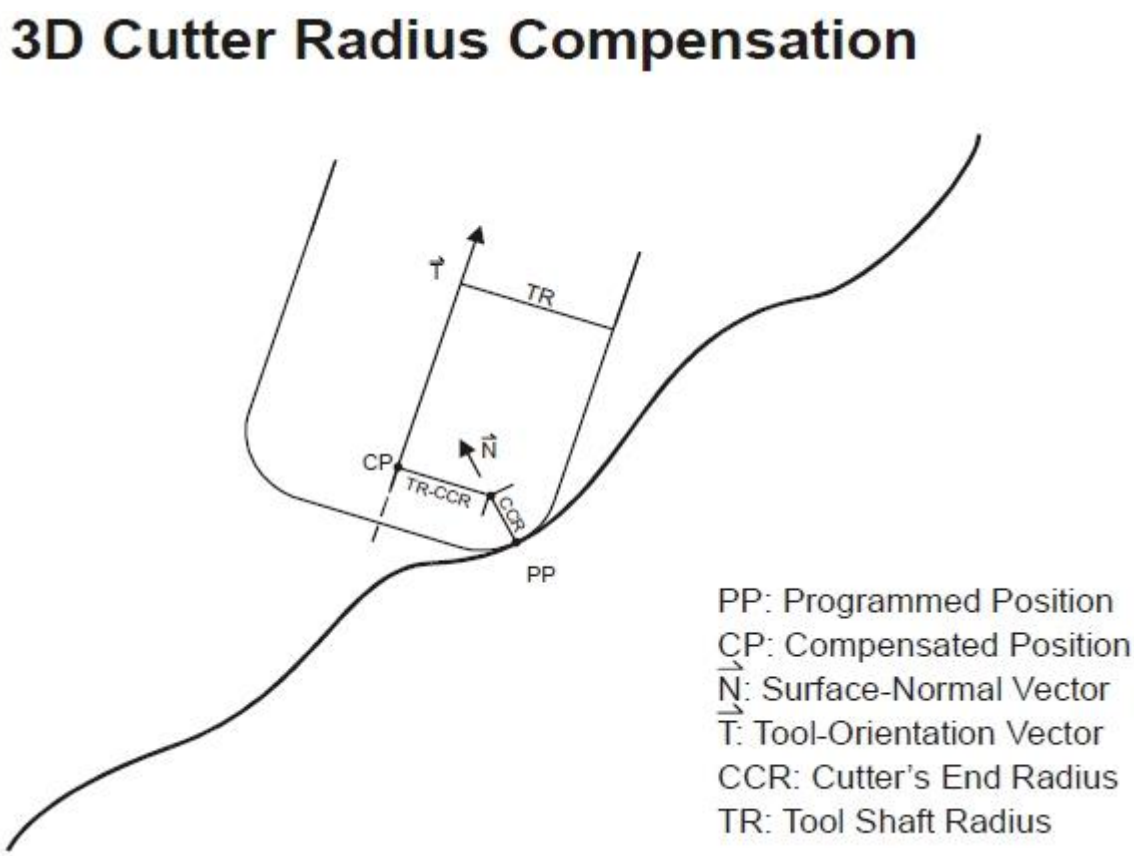
如何进行 3D 补偿

在操作中，在三维补偿有效时，Turbo PMAC 是从每个编程终点的无补偿的 X、Y 和 Z-轴位置处开始的。然后两个偏移被应用到 X、Y 和 Z 轴位置上。第一个偏移量是沿表面法线向量使用，大小等于尖端半径。使用第二个偏移量是朝向工具的中心，是在包含表面法线向量和刀具方向矢量的平面上，且垂直于刀具定向矢量，大小等于刀具半径减去尖端半径。

一旦计算了修改的终点，计算运动到终点就像它没有补偿那样。如果程序是线性模式，它将线性插值。如果程序是在 CIRCLE 模式（不建议），使用圆弧插补。

由于直接为每个移动指定结束点的偏移量，Turbo PMAC 为下一步的运动使用公式进行的计算中不存在交叉点。这意味着没有特殊前瞻或单步执行的考虑，而在二维补偿则有。

在 3D 补偿的所有运动都直接混合在一起。对于外角没有特殊的考虑，而在 2D 补偿中有。另外，对于引入和引出运动没有特殊注意的事项。引入移动是从上一个未补偿位置到第一个补偿位置的一个插值移动。引出移动是从最后补偿位置到第一个未补偿的位置的一个插值移动。



16-14 Turbo PMAC 的前瞻功能

Turbo PMAC 可以在编程轨迹进行高度复杂的超前的计算，以确保轨道不违反规定的参与运动的轴的最大量。这允许简单编写运动程序来描述指令路径。矢量进给速度成为一个制约而不是一个命令；程序的加速时间只用来定义拐角尺寸和最小移动段时间。Turbo PMAC 会自动控制沿路径的速度（但没有改变路径），以确保轴的限制不受侵犯。

前瞻计算是适用于执行任何一个编程的路径，而在该路径中在通过某些路径时需要缓慢执行进而限制了吞吐量（通过量），但是无法预见其中哪几部分需要缓慢的执行速度。而超前功能可以预见这些问题发生的地方，进而允许更快的通过大多数的路径，极大地提高通过量。

因为超前（前瞻）计算的本质是---轨迹计算是提前于实际执行的移动的，而且自动调整移动速度和时间而将运动保持在机器的限制内---所以它们并不适合于某些应用。任何需要快速反应到外部条件的应用程序不应该使用超前计算。此外，任何要求精确同步到外部运动的应用程序，如那些使用 PMAC 的外部时基功能的应用，也不应使用超前计算。

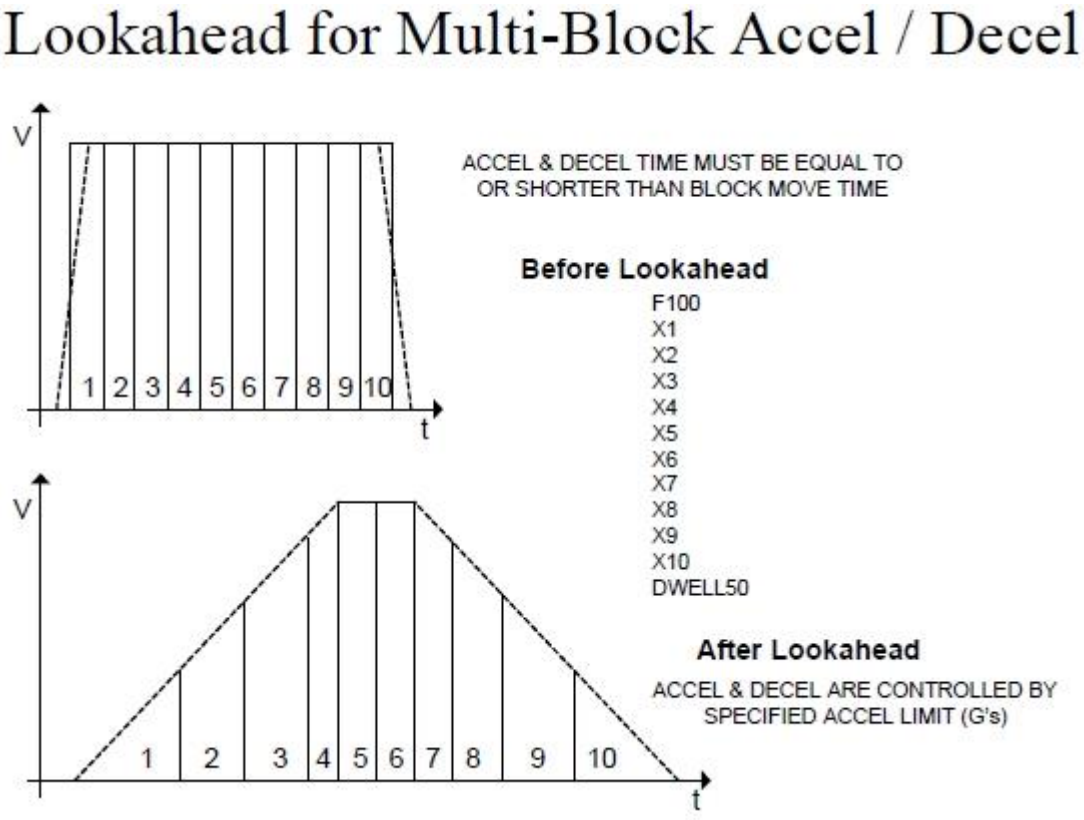
当超前计算功能启用时，Turbo PMAC 将在编程的轨迹提前进行扫描，寻找那些潜在违反它的位置、速度和加速度限制的部分。如果它看到了违反，它将随后通过预先计算的缓冲轨迹向后工作，减慢这些轨迹必要的各部分，以保持在限制范围内移动。这些计算完成在实际执行这些部分的轨迹之前。

Turbo PMAC 可以在直线和圆弧运动模式中执行这些超前计算。坐标系统必须放在分段模式（Isx13> 0），以启动超前计算，即使只使用线性方式运动。（坐标系统必须处于分段模式来执行 CIRCLE 模式移动或刀具半径补偿。）在分段模式下，Turbo PMAC 自动将运动拆分成小片段，作为一系列的光滑样条曲线来执行进而重新创造编程的运动。

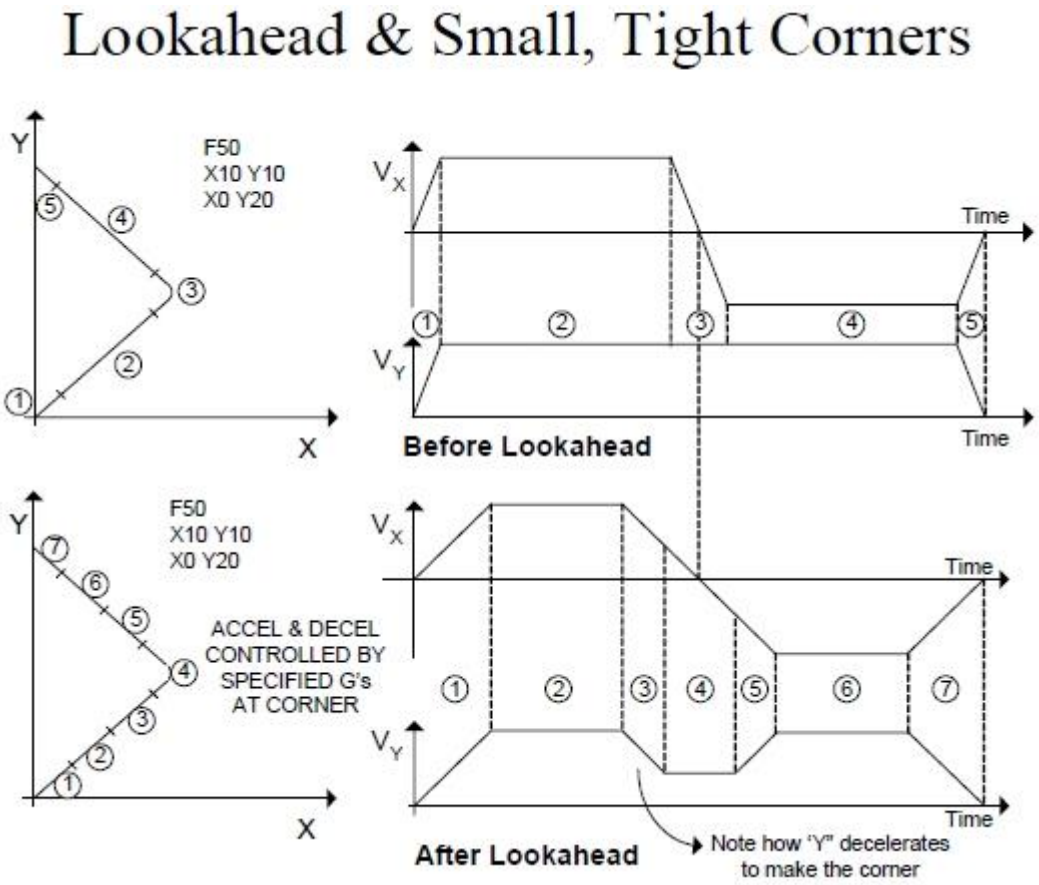
Turbo PMAC 将这些段的数据存储在该坐标系中一个专门定义的前瞻缓冲区中。当它被放入缓冲区时每个段需要花费 Isx13 毫秒的时间，但如果它或者缓冲区的一些其他段违反了速度或加速度极限，此时间可以延长。

这种技术允许 Turbo PMAC 在编程运动的中间、在编程的移动边界或在多个编程的运动中创建减速斜坡，其需要创建一个不违反约束的最快速度的移动。所有这一切都是在 Turbo PMAC 内自动地无形中完成的；零件程序员和操作员不需要了解该算法的工作原理。

下图显示了超前功能可以在多个编程运动中自动创建加速和减速曲线的原理。在这种情况下，在一个单一的编程移动中，编程的运动太短不能允许全加速到编程的速度以及从其上加速。无需对运动程序进行任何改变，超前功能将创建一个不违反加速度约束的轮廓。



接下来的图显示了超前功能如何可以自动创建一个减速到紧凑的拐角，允许缓慢拾取拐角以保持它在加速度约束条件内，然后加速到编程速度从拐角出来。这允许用户可以命令高的速度，并使得 Turbo PMAC 只在需要的地方减慢路径。请注意，在这个图的后超前预测轮廓不是时间延长，如同它真的是，这样做是为了显示轮廓上点之间的对应。



如果使用 Turbo PMAC 的逆运动学计算，在超前计算之前，从刀尖坐标转换到关节坐标，对于直线和圆弧运动模式逐段进行转换。因此， Turbo PMAC 可以在关节空间之中执行超前计算，逐个电机地执行，即使系统已经编程在刀尖坐标。

一旦已经设置好超前计算功能，超前功能的操作对于程序员和操作员是透明的。对于一个使用超前功能的运动程序，不需要改变它，尽管程序员可以选择做一些改变以利用这一功能提供的更高性能。

快速说明：设置前瞻

下面的列表快速解释了设置和使用 Turbo PMAC 上的超前功能所需要的步骤。下文中给出更详细的信息和内容。

1、通过轴定义语句，为坐标系分配所有所需的电机。
2、为坐标系的每个电机，设置 Ixx13 和 Ixx14 的正向和负向位置限制，加上 Ixx41 预期位置限制带，单位是计数。设置 Ixx24 的位 15 为 1，启动预期位置限制。
3、为坐标系的每个电机设置 Ixx16 最大速度，单位：计数/毫秒。
4、为坐标系的每个电机设置 Ixx17 最大加速度，单位是计数/ 毫秒 ^{2}。
5、以毫秒为单位，将坐标系的 Isx13 分段时间设置为最小的编程移动块的时间或 10 毫秒，以较少者为准。
6、为每个电机计算最大减速停止时间，作为 Ixx16/Ixx17。
7、通过最长减速停止时间选择电机。

8、计算需要前瞻的分段数，该减速停止时间除以（2* Isx13）。
9、将所需的分段乘以 4/3（如果需要向上舍入），并将 Isx20 超前长度参数设置为这个值。
10、如果还涉及到高块率， 将 Isx87 的默认加速时间设置为最小的模块时间，单位为毫秒；该 Isx88 默认的 S 曲线时间设为 0。
11、如果应用程序不涉及高块率，将 Isx87 默认的加速时间和 Isx88 默认的 S 曲线时间参数设置为一个值，该值可给出在编程的速度下所需的混合角的大小和形状。
12、如果你希望这些参数成为机器状态的自动的一部分，通过 SAVE 命令将这些参数存储到非易失性存储器中。
13、每次上电或复位后，为坐标系对卡发送一个 DEFINE LOOKAHEAD{#of segments}，{#of outputs}的命令，其中{#of segments}等于 Isx20 加上任何需要备份功能的片段；{#of outputs}是至少等于同步的 M-变量赋值的数量（它可能需要被缓冲超过前瞻长度）。
14、将你的运动程序加载到 Turbo PMAC。对于运动程序没有什么特别需要注意的。运动程序定义了应遵循的路径，超前（前瞻）算法可以减少沿路径的速度，但它不会改变路径。
15、执行运动程序，并让超前算法进行工作。

详细说明：设置使用前瞻

还需要几个步骤来计算和设置前瞻功能。通常情况下，计算只在机器的初始配置时进行一次。配置完成后，前瞻功能自动、无形中进行操作。

定义坐标系

超前功能检查坐标系中对所有的电机的编程动作。因此，第一个步骤是通过在轴线定义语句中将电机分配给坐标系的轴来定义坐标系的。这个动作是覆盖在本手册的《设立坐标系统》的部分。

前瞻的约束

Turbo PMAC 的前瞻算法迫使坐标系观察每个电机的 4 个约束。这些限制定义在每个电机的 I-变量中，代表最大的位置、速度和加速度。这些 I-变量必须正确设置，以使前瞻算法正常工作。

位置限制

每个电机 xx 的变量 Ixx13 和 Ixx14 分别定义被允许用于电机的最大正、负位置值（软件超程限制）。这些变量以计数所定义，并且参考电机的零点、或起始位置（通常称为机械零点）。即使为了编程，轴的原点已经被偏移了（通常称为程序零点），但是这些位置限制的物理位置不改变，他们对机床零点的参考位置不变。当轨迹正在执行这些限制时，Turbo PMAC 会检查每个电机的的实际位置，如果超出限制，程序中止，电机以 Ixx15 设定的速率进行减速。

每个电机 xx 的变量 Ixx41 定义实际位置极限和预期位置极限之间的距离，即使在前瞻中，也可以在移动计算时间检查该距离。也就是说，如果所计算的预期移动位置大于（Ixx13 - Ixx41），或小于（Ixx14+ Ixx41），这将构成一个预期位置限制冲突。仅当 Ixx24 的第 15 位被设置为 1 时，才检查期望位置限制。

在这种模式下，如果前瞻算法在编程轨迹中提前扫描，确定了坐标系中的任

何电机将超过其期望的位置限制，它会暂停程序，并强制恰好停止在该限制处。然后，它会向后工作通过缓冲的轨迹段，使电机沿路径停止在该点（在不违反任何电机的 Ixx17 加速度约束的最短时间内）。

但是，如果 Ixx24 的第 14 位也设置为 1 时，程序不会停在极限处。相反，它会继续，而违例电机（即超过了限制的电机）饱和在限制值。

当停止在前瞻内的一个预期位置限制时，该程序只是暂停，不会中止。该操作相当于发出一个\快速停止命令。有可能将路径“折回”进入该限制内，或者甚至在改变限制值后恢复向前执行。必须发出“中止”命令后，另一个程序才能启动。

但请注意，如果在减速到停止在预期位置限制的过程中，一个实际位置限制也被跳闸（超过），程序将中止，所以路径的“折回”、恢复是不可能的。出于这个原因，如果路径的折回和恢复很重要，Ixx41 应设置为一个足够大的值，以使在期望位置限位停止过程中，实际位置限制不会跳闸。

此技术允许在机器的硬停止内放置这些软件位置限制。如果没有预期位置限制，该软件位置限制无法被检测，直到实际轨迹真的超过了限制。这就要求这些限制被置于硬停止内足够远的位置，以使电机在它们超过限制后，有足够远的停止距离。（当电机到达了软件限位而无前瞻，电机的减速是 Ixx15 控制的，而不是 Ixx17，而且减速不一定沿着编程路径进行。）

速度极限

变量 Ixx16 为每个电机 XX 定义了电机允许的最大速度的大小。这些变量是以（每毫秒的计数）原始 PMAC 单位定义的，所以必须从用户单位进行一个快速的转换计算（如毫米/分钟）。

如果算法在编程轨迹进行前瞻时，确定坐标系中任何电机被要求违反它的速度极限，它会（在足够远刚好不会违反限制的那一点）减慢轨迹。然后，它会向后工作通过缓冲的轨迹段，在（不违反任何电机的 Ixx17 加速度约束的）最短时间内，创建一个沿路径受控的减速到限制的速度。

注意：

在最初的移动-块计算中，在移动数据被发送到前瞻功能之前，有几个因素可能会导致指令速度比编程速度低。首先，如果运动程序中通过 F 指令指令的向量进给速度，超过了最大进给速度参数 Isx98，那么使用 Isx98 来代替。其次，如果移动-块时间（无论是 **TM** 命令直接指定的，或通过向量距离除以矢量进给速度计算的），小于编程的加速时间（TA 或 2* TS 中二者较大的），使用编程的加速时间来代替。这将导致速度低于编程的速度。超前功能可以进一步放缓这些运动，但不能对他们进行加速。

加速度限制

变量 Ixx17 为每个电机 xx 定义了电机允许的最大加速度的大小。这些变量是以每（毫秒的平方）的计数（原始 PMAC 单位）定义的，所以必须从用户单位

进行一个快速的转换计算（如 in/sec^2，或 g's）。

如果算法，在编程的轨迹进行前瞻时，确定坐标系中任何电机将要违反其加速度极限，它会（在足够远刚好不会侵犯限制的那点）减慢轨迹。然后，它会向后工作通过缓冲轨迹段，在不违反任何电机 Ixx17 加速度约束的最短时间内创建一个沿路径的受控减速到极限的速度。

计算分段的时间

Turbo PMAC 的前瞻功能的操作是在（编程轨迹计算出的）中间运动段。每个电机的一个中间点，在编程路径的每一段都计算一次，然后用三次样条连接这些段，以伺服更新率执行一个细插补。因此，用户可设置的分段时间是前瞻功能优化的一个重要参数。

变量 Isx13 为每个坐标系 X 定义了编程轨迹中每个中间段的时间，以毫秒为单位，然后才可能通过超前功能进行扩展。Isx13 是一个整数值；如果发送一个非整数的值，Turbo PMAC 将舍入到下一个整数。如果 Isx13 被设置为 0，则坐标系不是分段模式；没有中间段被计算，并且超前功能不被启用。

设置 Isx13 分段时间，必须解决几个问题。这些包括该分段时间与最大块率的关系，它引入的小插值误差，它对 Turbo PMAC 的计算负荷的影响。这些关系的每一个被依次解决如下。

块率关系

在大多数应用中，Isx13 分段时间将被设置为小于或等于最小的块（编程运动）时间。换言之，通常由 Isx13 定义的分段速率被设定为大于或等于最大的块率。例如，如果需要的最大的块率是以每秒 500 块，那么最小的块时间是 2 毫秒，Isx13 被设置为一个不大于 2 的值。

这种关系会保持，因为当 Turbo PMAC 查找下一个分段点时，一个比分段时间小的块时间被跳过了。虽然这不会导致任何错误，但是如果控制器要跳过它们，把这些编程点放在运动程序中没有真正的意义。然而，有些人继承了旧的运动程序，点之间比实际需要的结合得更紧密，这些用户可能有理由设置其分段的时间大于他们的最小的块时。

需要注意的是编程的加速时间对最大的块率设定了一个限制。一个编程块的运动时间，甚至在前瞻之前，不得少于编程的加速时间。编程的加速时间是 TA 时间（TA= Isx87，默认）和两倍的 TS 时间（TS= Isx88，默认情况下）中的较大的那一个。在高-数据块-速率的前瞻应用中，TA 时间通常被设定为等于最小的所需块时，TS 时间通常被设定为（因为它使得角为方形）

插值误差

Turbo PMAC 用来连接中间段的点的三次样条插值技术是非常准确的，但它确实创造小的错误。这些误差可以计算如下：

$$\text{Error} = \frac{V^2T^2}{6R}$$

其中 V 是沿路径的速度向量，T 是分段时间（注意单位!），R 是该路径的曲率半径。例如，如果该速度为 100 毫米/秒（～4 in/s），分段时间是 0.01 秒（Isx13=10 毫秒），并且该速度下的最小半径为 50 毫米（～2 in），则最坏情况下的插值误差可以计算如下：

$$\text{Error} = \frac{100^2 \frac{\text{mm}^2}{\text{sec}^2} * 0.01^2 \text{sec}^2}{6 * 50\text{mm}} = 0.003\text{mm} = 3\mu\text{m}$$

如果编程的路径本身引入了路径误差，如线性插值的弦误差，这必须被添加到误差的预算内。此外，如果伺服回路的执行增加了伺服误差，这些也必须被包括在内。

计算的影响

虽然较小的 Isx13 分段时间允许更高的实际最大块率，并允许更精确的插值，但是他们增加了 Turbo PMAC 的计算要求，特别是当前瞻处于有效状态时。下表显示了在 Turbo PMAC 的基准测试的结果，显示了给定数量下执行前瞻计算的轴，他们的最小分段时间。

轴的数量	最大块率（块/秒）	最小的分段时间（ms）
2	**2000**	**1 @ 200%**
3	**1000**	**1**
4	**500**	**2**
5	**500**	**2**
6	**500**	**2**
8	**333**	**3**
12	**250**	**4**
16	**200**	**5**
注意事项： 1、这是在 80 兆赫的 Turbo PMAC 进行的测试 2、在默认的 2.25 kHz 的伺服更新率进行测试 3、没有 PMAC 电机换相或电流回路闭合进行的测试 4、较高的块率可以做到，但分段将消除功能		

需要注意的是受到这些限制，前瞻的长度只受 Turbo PMAC 的内存限制。

在一般情况下，Isx13 分段时间设定为满足（以上 3 个关切的用户要求）的最大值。然而，它很少设置大于 10 毫秒。

计算所需的前瞻长度

为了使坐标系统达到最大性能，必须前瞻每个电机所需的的时间和距离来从最大速度减速到完全停止。由于前瞻缓冲区存储运动分段，这个前瞻长度必须表示为段数。

为了计算该值，首先计算坐标系中最坏情况下每个电机停止所需的时间。这个值可以通过将电机的最大速度除以电机的最大加速度来获得。就 Turbo PMAC 参数而言：

$$\text{StopTime（msec）} = \frac{\text{Ixx16}}{\text{Ixx17}}$$

现在采取电机的最长的停止时间，并除以 2（因为分段会以最高速度进入，这需要斜坡下降（减速）到零速的一半时间）。接着，以这个值转换为段数，通过除以坐标系的分段时间：

$$\text{前瞻长度(segs)} = \frac{\text{StopTime(msec)}}{2 * \text{Isx13(msec/seg)}} = \frac{\text{Ixx16}}{2 * \text{Ixx17} * \text{Isx13}}$$

这是前瞻缓冲区中分段的数量,必须始终正确提前计算。因为在每个分段 Turbo PMAC 不完全重新计算前瞻缓冲区，它实际上必须比要求的这个段数看得更远一点。

前瞻长度参数

变量 Isx20 为坐标系告知算法在程序中提前查看多少段。这个值是段数的函数，在前瞻缓冲区必须始终正确（SegmentsNeeded，需要的段）。其计算公式为：

$$\text{Isx20} = \frac{4}{3} * \text{SegmentsNeeded}$$

设置 Isx20 为一个大于需要的值，不会增加计算量（虽然在缓冲器被填充时它确实增加了最重的计算负荷的时间）。然而，它确实需要更多的内存存储，而且它在程序响应任何外部条件时增加了延迟。

设置 Isx20 为一个比需要较小的值不会导致违反限制。但是，它可能会导致 Turbo PMAC 比 Ixx16 限制的要求更严重地限制速度，以确保不违反加速度限制。还有，锯齿状的速度分布可以被观测到。

定义前瞻缓冲区

为了使用 Turbo PMAC 坐标系中的前瞻功能，必须为坐标系定义一个前瞻缓冲区，为缓冲区保留内存。这是通过在线的坐标系统特定 DEFINE LOOKAHEAD 命令完成的。由于前瞻的缓冲区在断电或重置时是不保留的，必须在每一个上电或电路板复位时，发出此命令。

有两个与 DEFINE LOOKAHEAD 命令关联的值。第一个确定可存储在前瞻缓存器中每个电机的运动段数。最小值必须被设置为等于 Isx20

如果这个值设置得比 Isx20 大，前瞻缓冲区会存储历史数据。这个数据可以用来反向通过已经执行的轨迹。如果需要反转，缓冲区的大小应储存足够的返回段以覆盖所需的备份距离。如果对这些同步的 M-变量赋值保留的内存比必要的多，也没有惩罚，除了内存其他用途的损失。

在前瞻缓冲器中用于保留数据段的空间是依赖于，DEFINE LOOKAHEAD 命令的时间分配给该坐标系统的电机的数目。如果分配给坐标系的电机的数量改变，前瞻缓冲区的组织将是错误的，并且在坐标系改变之后，下一个运动有一个运行时错误而程序将中止。

如果坐标系统在一个使用前瞻的应用过程中必须改变，前瞻缓冲区必须首先被删除，然后变更后再次定义。下列运动程序代码显示了如何做的：

```
DWELL 10                                        ; Stop lookahead execution
CMD "&1 DELETE LOOKAHEAD"                       ; Delete buffer
CMD "&1 #4->100C"                               ; Assign new motor to C. S. 1
CMD "&1 DEFINE LOOKAHEAD 1000,100"              ; Redefine buffer
DWELL 10                                        ; Make sure commands execute
```

与 DEFINE LOOKAHEAD 命令相关联的第二个值决定可存储在前瞻缓冲区的同步的 M-变量赋值的数目（例如 M1== 1）。运动程序中同步的 M 变量赋值会延迟给 M 变量实际赋值，直至开始实际执行运动程序的下一个运动。因此，这些行动必须保留在缓冲区中等待执行。

用于这些赋值的缓冲区的大小必须与在前瞻过程中预期赋值的最大数目至少一样大。为这些同步的 M-变量赋值保留的内存比必要的内存更多，是没有惩罚的，除了内存其他用途的损失。

注意：

在前瞻下以这种方式为同步的 M-变量保留的缓冲区是截然不同于无前瞻的用于同步的 M-变量的固定大小的缓冲区。

例如，命令&1 DEFINE LOOKAHEAD500,50 为坐标系 1 创建一个前瞻缓冲区，可以为当时分配给坐标系的各个电机存储 500 个分段，再加上 50 个同步的 M-变量赋值。

通过前瞻运行一个程序

在一个坐标系中运行一个运动程序时前瞻功能被激活，假如以下条件为真：

1、坐标系统是在分段模式（Isx13> 0）。
2、坐标系被告知要前瞻（Isx20>0）。
3、自上次板上电或复位，已为坐标系定义一个前瞻缓冲区，或者如果前瞻缓冲区的结构已通过 I14=1 保存。
4、运动程序正在执行线性或圆弧模式移动。

在这些条件下前瞻功能被激活，即使当 Turbo PMAC 在每个分段正在进行逆运动学计算，以刀尖的位置转换为关节位置。这允许用户以方便的刀尖坐标编写一个运动程序，但仍然会自动遵守所有关节电机的限制。这一点尤其重要，特别是当刀尖轨迹靠近一个奇点，要求非常高的关节速度和加速度时。

其他移动模式---RAPID、SPLINE 和 PVT---可以通过定义前瞻缓冲区来执行，但是当正在执行这些动作时，前瞻功能是不激活的。

注意：

为利用前瞻的功能，绝对没有任何需要改变运动程序。

认识到前瞻功能对运动程序的几个方面的影响是重要的。每个领域涵盖在下面。

矢量进给

没有前瞻时，矢量进给值(Fxxx)是运动程序中每个程序移动块的一个命令。也就是说，每次移动都被计算，使其以编程进给速度矢量（速度）来运动。若前瞻激活，进给值仅仅是一个制约因素。移动将永远不会以较高的速度来执行，但也可在一些或全部的满足电机约束的运动中以较慢的速度执行。

如果移动是通过移动时间而不是进给率进行编程的，编程的运动时间成为一个（最低）的约束，运动绝不能在更短的时间内执行，但它可能以更多的时间来执行。

加速时间

编程的加速时间---默认情况下是 Isx87 和 Isx88，或在运动程序的 TA 和 TS---是开始前瞻前的时间。前瞻函数将控制所执行的实际加速时间，但是编程的加速时间仍然是重要的，原因有二。

首先，编程的加速时间，即 TA 或 2* TS 中较大的那个，是最小的移动-块时间。如果 PMAC 初步计算出一个更小的移动时间，通常为（矢量距离除以矢量进给速度），它会以减缓移动的方式将时间增加到等于加速时间。此检查甚至发生在前瞻（只能进一步减缓移动）之前，它是针对计算性过载的重要保障。加速时间必须设置足够低，进而不限制有效的移动。

注意：

加速时间可以被设置为 0；在这种情况下，Turbo PMAC 设定 0.5 毫秒的最小移动时间。。

其次，由于较长的移动被混合在一起，编程的加速时间和进给率控制混合的拐角尺寸。混合角在编程的角点之前开始一个 F*Ta/ 2 的距离，（其中 F 是编程的进给率，Ta 是指定的加速时间（TA）或 2 倍的指定的 S-曲线的时间（2*TS），以二者中较大的那个为准）。混合角结束时相对编程的拐角点的距离也等于这个距离。

如果前瞻算法确定该混合角落违反了一个或多个电机的加速度极限，它会在拐角处减慢路径的速度。这将使混合角的时间大于在程序中指定的时间。前瞻还将创建一个受控制的减速斜坡进入混合角，和一个控制加速斜坡来从角里运动出来。在这种方式下，拐角处的圆角大小可以很小而不违反加速度限制，也不限制远离拐角的速度。

在一般情况下，加速时间应该设置得尽可能大，而又不使得最小移动时间过大，或拐角太大。在高块-率的应用中，TA 时间通常被设定为最小的块时间，TS 时间设置为 0。在低块-率的应用中，TA 和 TS 时间通常被设置为实现所需的转角大小和形状。

轨迹滤波器

在高块-率的应用中，粗糙的运动可能会来自编程路径的“量化误差”。这可能会在切割部件时产生机械振动、可听噪声和高表面粗糙度。这些错误可能来源于编程点的有限的数值分辨率，源于如果对编程点进行扫描时的测量误差，或两者兼而有之。

这种行为可以通过插值电机轨迹的简单过滤得到补偿，通过使用每个电机 xx 的变量 Ixx40。如果 Ixx40 被设置为一个大于零的值，所希望的轨迹是通过一个简单的一阶低通数字滤波器，用于平滑的目的。（如果 Ixx40 被设置为 0.0，该滤波被禁用）。Ixx40 的值越高，滤波器的时间常数越大。

滤波器的时间常数 T_f 作为伺服更新时间 Ts 和 Ixx40 的一个函数的方程为：

$$T_f = \frac{Ixx40 * T_s}{1 - Ixx40}$$

通常，当使用过滤器时选择几毫秒的时间常数。请注意，只有期望轨迹进行

过滤，所以伺服环路的稳定性不受影响。然而，滤波会引入一个非常小的路径错误（仅对于非常大的时间常数才明显），可以根据以下公式来量化：

$$\text{Error} = \frac{V^2 T_f^2}{2R}$$

其中，V 是速度，Tf 为滤波器的时间常数，R 是该路径的曲率本地半径。例如，5000 毫米/分钟的速度（～200 英寸/分钟），2 毫秒的滤波器的时间常数，和 100mm（～4 英寸）的本地半径，路径误差将是：

$$\text{Error} = \frac{5000^2 \frac{\text{mm}^2}{\text{min}^2} * \frac{\text{min}^2}{3600\text{sec}^2} * 0.002^2\text{sec}^2}{2 * 100^2\text{mm}^2} = 1.39 \times 10^{-6}\text{mm} = 1.39\text{nm}$$

进给速度倍率修调

执行所有的前瞻计算到假定 100%的进给倍率值。如果进给倍率值，来自任何来源，从 100%开始变化时，那么速度和加速度的计算是不正确的。真正的速度值随着倍率值线性变化；真正的加速度值随倍率值的平方变化。

例如，在 200%的倍率，速度值是两倍的编程值（以及可能超过 2 倍的限制值），和加速度值是 4 倍的编程值（以及可能超过限制值的 4 倍）。

由于进给倍率可以在任何时候改变且有即时影响，所以运动在实际将被执行时前瞻功能无法预见倍率是什么。因此，不能计划倍率的任何变化，所以它假定操作在 100%。

前瞻的基本思想是从操作者的瞬时判断中除去倍率功能，并使用控制器的数学计算（它有效地充当倍率），以确保路径的正确和最佳执行。

计算能力

前瞻的计算可以把重要的实时计算负载放在 Turbo PMAC 处理器上。如果处理器不能跟上这些实时性的要求，程序执行将以一个“运行时”错误而失败，而运动将被中止。另外看门狗定时器跳闸还有一个可能性，即如果处理器不从前台的前瞻计算中释放出来用于后台任务时，也会跳闸。

重要的是，该应用程序要进行评估，以确保该前瞻计算能够在最坏情况的条件下正确的进行。当执行最大量计算的周期是在一个移动序列的开始，当 Turbo PMAC 动态填充缓冲区以提前得到指定的距离。

一个良好的最坏情况测试是：就在一个混合序列的开始时，以最大移动-块速率运行一个编程移动的运动程序。确保 Turbo PMAC 可以成功通过（高块率执行和前瞻缓冲的动态填充）的组合。要建立一个安全边际，增加倍率值到 100%以上，看看存在哪些额外的能力。强烈推荐 20%的安全边际（正确执行在 120%）。

在前瞻中停止

如果你希望在前瞻模式中停止轴的运动，他必须仔细考虑如何完成停止。认识到通过停止命令停止执行链的哪一点是非常重要的。不同的停止命令有不同的效果和不同的用途。

快速停止

该\快速停止命令会使 Turbo PMAC 立即计算并执行前瞻缓冲区内最快的停止，同时又不超过坐标系中电机的 Ixx17 加速限制。运动沿编程路径被控制停止，这不一定是停止在一个编程点（也可能不会）。此命令是等价于前瞻内的一个进给保持“feedhold”（即使二者的内部机制是完全不同的），且该命令应该是操作员在前瞻时按下 Hold 按钮时发出的命令。前瞻之外，此命令会导致实际进给暂停，仿佛已经发出 H 命令。

该\命令是以交互方式停止前瞻内操作并有恢复操作意图的最好命令。设置为在减速中发生的任何同步的 M-变量赋值将执行。

如果需要的话，电机可以慢跑远离这个停止点。此外，运动可通过<命令沿路径反向（见反向，下文）。

普通编程的运动可以通过>恢复前进、R 运行或 S 单步命令进行恢复，规定所有电机是在他们使用/命令而最初停止的相同位置处。如果有任何电机已经慢跑远离了这一点，他们必须先通过 J=命令返回。在从停止到加速的过程中，要观察加速度限制。>恢复命令将坐标系放在连续运行模式，或单步模式，取其在快速停止之前的那个模式。

模块-终点停止

/块终点命令将运动停止在（当前加到前瞻缓冲区的移动）的终点处，即使下一个移动已经被计算出来。该运动终点之前的运动段仍然加到前瞻缓冲区中，而且前瞻缓冲区的所有运动段和同步的 M-变量赋值是已经完成的。

运动受控停止在(加到前瞻缓冲区的最新)运动块的终点，而又不违反约束。但是，相对/命令给出的时间和坐标轴停止的时间，可以有显著的延迟---超过（Isx20*Isx13）毫秒，如果前瞻缓冲区已满的话。

如果需要的话，电机可以慢跑远离这个停止点。运动可以用 R 或 S 命令恢复，假设所有电机在他们使用/命令停止的相同位置处。如果有任何电机已经慢跑远离这个停止点，他们必须首先通过 J=命令返回。

退出/步进

Q 退出命令只是告诉运动程序不要计算任何进一步的运动程序块。（该 S 单步命令会做同样的事情，如果在该程序运行时给出）。运动程序最新运行块的终点之前的运动段仍然加入到前瞻缓冲区，而且前瞻缓冲区中所有运动段和同步的 M-变量赋值是已经完成的。

运动会受控停止在最新的移动块的结尾而又不违反约束。但是，相对发出 Q 或 S 指令时和坐标轴停止时，可以有显著的延迟---超过（Isx20* Isx13）毫秒，如果前瞻缓冲区已满---。

如果需要的话，电机可以慢跑远离这个停止点。运动可以通过 R 或 S 命令恢复。电机不必处于在他们最初通过 Q 或 S 指令而停止的相同位置。然而，如果希望将它们返回到该位置，应该使用 J =命令。

进给保持

H 进给保持命令将进给倍率值设置为零，立即启动，并以坐标系统变量 Isx95 控制的速率进行减速。运动沿编程路径受控制停止，这不一定是停止在一个编程点（也可能不会）。在减速下降到停止的过程中加速度限制不一定观察到。（设置为发生在减速中的任何）同步 M-变量分配，此时将执行。

如果需要的话，电机可以慢跑远离这个停止点。编程的运动可以通过 R 或 S 命令恢复，假设所有电机被指令在他们最初通过 H 命令停止的相同位置处。如果有任何电机已经慢跑远离这个停止点，他们必须先通过 J=命令返回。在从这里

的静止开始加速的过程中，不一定观察得到加速度限制。

中止

A 终止命令会立刻闯入执行的轨迹，并使得在坐标系中的所有电机受控制地停机，每个电机以 Ixx15 设置的减速率进行减速停止。该停止不一定要在一个编程点（也可能不会），而且它甚至不一定沿编程路径（也可能不会）。

前瞻缓冲区的运动段和同步的 M-变量赋值已经被丢弃，它们将无法恢复。尽管程序可以通过 R 或 S 命令恢复，但是执行时将错过前瞻缓冲区所有丢弃的段。恢复运行将需要特殊的恢复算法，所以不建议中止命令，除了错误条件下的迅速停止外。

停止所有

该<CTRL-K>停止-所有的命令会立刻闯入执行的轨迹，并通过打开伺服环路、迫使命令输出为零、禁用放大器而禁用 Turbo PMAC 的所有电机。在没有制动或再生电路的情况下，电机将自由停止。

运动段和同步的 M-变量赋值已经在前瞻缓冲区丢弃，它们将无法恢复。尽管在用 R 或 S 命令重新启用伺服回路后，程序可以恢复，但是执行时将错过前瞻缓冲区丢弃的所有段。恢复运行将需特殊的恢复算法要，所以不建议“kill-all”命令，除非是紧急情况。

注意，当电机处于一个正在执行运动程序的坐标系中时，电动机-专用的 K 停止命令是不允许使用的。该程序必须先停止，通常是通过一个 A 中止命令。

在前瞻中反转

如果前瞻缓冲区的大小大于实际前瞻所需要的，它将包含历史数据---可用于沿编程路径反向。这种功能使系统有“回扫/折回”功能，允许它轻松地沿着已经执行路径往回走。在反转过程中使用的关键命令是<备份命令，这将导致坐标系在相反的方向开始执行前瞻缓冲区的段。

备份命令

如果<命令发出时，坐标系统处于前瞻缓冲区正常的前向执行，Turbo PMAC 将在内部生成一个\“快速停止”命令来停止前进的执行，然后开始反向执行。

<命令也可在一个\快速停止命令停止执行程序后发出。<命令不能在（通过一个/块终点、Q 退出、S 单步、A 中止或<CTRL-K>停止-所有的命令、或一个自动错误终止而）停止后给出。

反向执行

在相反方向的执行将观察位置、速度和加速度的限制，就像在前向那样。请注意，如果 Isx20 被设置为 1，在正向或反向方向上都观察不到限制。在这种模式下，前瞻缓冲区被简单地用于缓冲点来启动反转，而没有实际前瞻计算的计算开销。这种模式适用于（EDM）放电加工应用（它需要快速逆转，但不需小心的加速度限制）。

如果没有被其他的命令停止，反向执行将继续，直到到达前瞻缓冲区的开始（最早存储的点）。它会在此时自动停止（以一个加速度限制内的可控减速）。在前瞻缓冲区最早存储的点永远不能来自当前连续混合运动序列的第一点之前。这意味着你不能反转进入、超过或通过以下：

- 一个 DWELL 点
- 一个快速、样条曲线或 PVT 模式移动

- 一个回零-搜索移动
- 一个（用/、Q 或 S 指令停止程序的）点。
- 一个（其中出于任何其他原因停止混合的）点（如 Isx92=1，两次回跳）

请记住，运动程序的一个 DELAY 延迟命令不会禁用混合，因此反向执行可以通过一个 DELAY 延迟点。如果在程序的执行过程中需要停止在一个点，但需要反向通过该点，那么应该使用延迟 **DELAY** 而不是 **DWELL**。

停止反向执行

反向执行可以通过使用\快速停止命令而停止在此点之前。反向执行可通过另一个<备份命令得以恢复；前向执行可以通过一个>恢复、R 运行或 S 单步命令而重新启动。>恢复命令将坐标系放在连续运行模式或单步模式，取其在备份之前的模式。

在执行一个反向时或在一个路径被反向的部分上执行前向时，不执行同步的 M-变量赋值。

在路径被反向的部分执行前向将无缝地融入到路径之前未执行的部分。此时，前瞻缓冲器的标准执行将恢复，新的点添加到前瞻缓冲区的末尾，缓冲的同步 M-变量赋值的执行再次启动。

从 Turbo PMAC 内快速反向

如果你想从 PMAC 内部反向的速度非常快，如同电火花加工应用中的快速返回，最好是绕过命令解释器（它作用在后台）。这可以通过直接写入 PMAC 程序的一个前瞻控制 I-变量中来完成。

每个坐标系统的变量 Isx21 包含了前瞻执行状态的控制位。通过直接从 PLC 程序设置这个 I-变量的值，可避免命令解释器的开销和延迟，并获得稍快的反应。使用三个值：

- 设置 Isx21 为 4 相当于发出\快速停止命令
- 设置 Isx21 为 7 相当于发出<备份命令
- 设置 Isx21 为 6 等效于通过>恢复前向命令来恢复前向运动。

如果要在其他时间监视 Isx21，你会看到在命令被处理后，权值等于 4 的位被清除。因此，您将看到以下值：

- Isx21=0，当用快速停止命令停止时
- Isx21=3，当在前瞻反向运行时
- Isx21=2，当在前瞻前向运行时

16-15 轴转换矩阵

Turbo PMAC 提供的能力可以在一个坐标系的 X、Y 和 Z 轴上进行矩阵转换运算。这些操作具有与轴线定义语句的矩阵形式相同的数学功能，但这些可以在程序中进行动态更改；轴线定义语句对于一个特定的应用程序是固定的。

该矩阵变换允许对 X、Y 和 Z 轴进行平移、旋转、缩放、镜像和斜移。他们对于英制/公制转换、原点移动、进行重复的镜像、带角度偏移的重复以及更多，是非常有用的。

该矩阵运算执行的基本数学操作如下：

$$\begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix} = \begin{bmatrix} R11 & R12 & R13 \\ R21 & R22 & R23 \\ R31 & R32 & R33 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + \begin{bmatrix} D1 \\ D2 \\ D3 \end{bmatrix}$$

基本的 X、Y 和 Z 坐标是那些由轴定义语句定义的坐标。这些语句可能会或可能不会包括到轴和电机之间的矩阵关系中。如果在定义语句中有一个矩阵关系，这些矩阵操作符将在该关系的“上面”进行操作。

建立矩阵

必须做的第一件事是为转换矩阵定义一个缓冲区空间。这是通过在线指令 DEFINE TBUF{常数}完成的，其中，{常数}表示要定义的矩阵编号。在该命令中，每个矩阵自动设置为单位矩阵。这只需要做一次，因为矩阵的空间和值将被保存在电池备份的 RAM 中，直到给出一个 DELETE TBUF 或$$$***命令。

使用矩阵

在运动程序内，命令 TSEL{常数}（转换选择）挑选一个已定义的矩阵作为坐标系的有效转换矩阵。该矩阵对于程序中下一个计算的移动仍然有效。

一旦选定，该矩阵可以通过几个程序命令进行处理。该处理用于把新的值放在所选择的矩阵。在任何涉及 X、Y 或 Z 轴的移动的计算过程中，该矩阵被使用，不管在当时包含什么值。

初始化矩阵

该 TINIT（转换初始化）命令使选定的矩阵为单位矩阵，从而转化的位置就等于未转换的位置。

绝对位移

ADIS{常数}(绝对位移)命令设置矩阵的位移部分，是通过使三个位移值(D1、D2 和 D3)等于三个用{常数}指定的 Q-变量。例如，ADIS 25 将使 X 位移等于 Q25，Y-位移等于 Q26，和 Z 位移等于 Q27。

增量式位移

该 IDIS{常数}（增量位移）命令通过添加 3 个 Q-变量值到现有位移，进而改变所选矩阵的位移部分。

绝对旋转/缩放

该 AROT{常数}（绝对旋转）命令通过使 9 个旋转/缩放值等于以由{常数}指定的 Q 变量开始的 9 个 Q-变量，来设置所选矩阵的旋转/缩放部分。例如，AROT71 将使矩阵的 R1 等于 Q71，R12 等于 Q72，依此类推，到 R33 等于 Q79。

增量式旋转/缩放

该 IROT{常数}（增量旋转）命令通过将所选矩阵乘以（由以{常量}指定的 Q 变量开始的九个 Q-变量组成的）矩阵，进而改变所选矩阵的旋转/缩放部分。该影响是增加旋转角度、乘以缩放因子。例如 IROT 100，将由 Q100 到 Q108 的值组成的矩阵乘以现有的矩阵。

详细信息

使用任何这些命令之后，对使用的 Q-变量的任何变更都不改变所选择的矩阵。另一个使用 Q-变量的命令必须被执行以改变所选择的矩阵。

注意：

当使用轴矩阵变换用于缩放，不要使用 R 半径指定圆弧插补，因为半径不随轴缩放。使用 IJK 中心向量规范来代替。

计算的影响

如果矩阵变换计算已经通过 TSELECTn 命令激活，涉及 X、Y 和 Z 轴的程序移动计算所花费的计算时间小，但有明显的额外增加量（对于 80 MHz 的 CPU 约 1/4 毫秒）。这对于运动程序可以略微降低最大移动块执行率。要禁用矩阵变换的计算，使用 TSELECT0，它取消选择所有矩阵，并停止矩阵计算。

示例

这些概念可能最好用一些简单的例子来说明。在实际使用中，更复杂的东西可以用矩阵完成的，特别是包含数学和逻辑的。

缩放范例

如果轴定义语句缩放你的轴，单位是毫米，但程序至少应该是暂时的，以英寸为单位缩放，设置矩阵如下：

```
TSEL 1                                  ; Select Matrix 1
Q11=25.4    Q12=0       Q13=0           ; Variables for first row
Q14=0       Q15=25.4    Q16=0           ; Variables for second row
Q17=0       Q18=0       Q19=25.4        ; Variables for third row
AROT 11                                 ; Use Q11-Q19 for matrix
```

需要注意的是纯缩放只使用矩阵的主对角线。进行缩放是相对于该坐标系的原点的。当然，在每个命令行不需要分配 3 个 Q-变量值，但是这可能有利于程序的可读性。

旋转的例子

在 XY 平面上，将坐标系绕原点旋转 15 度。设置矩阵如下

```
TSEL 2                                          ; Select Matrix 2
Q40=COS(15)     Q41=SIN(15)     Q42=0           ; Variables for first row
Q43=-SIN(15)    Q44=COS(15)     Q45=0           ; Variables for second row
Q46=0           Q47=0           Q48=1           ; Variables for third row
AROT 40                                         ; Assign these values to the rotation portion
```

这个变换在 XY 平面相对于固定的 XY 轴对点逆时针旋转 15 度，这是在一个右手坐标系从+Z 轴查看时（ixj= k）。或者说，从右手坐标系中的+Z 轴观察时，在 XY 平面相对于固定点，它顺时针旋转 XY 轴 15 度。

位移示例

为了分别对 Y 轴和 Z 轴偏移 5 个单位和 2.5 个单位，而使 X 轴不变。设置矩阵，如下所示：

```
TSEL 3          ; Select Matrix 3
Q191=0          ; First variable
Q192=5          ; Second variable
Q193=2.5        ; Third variable
ADIS 191        ; Assign these values to the displacement portion
```

第二个旋转的例子

为在 XY 平面中旋转坐标系 15 度，如在第一旋转的例子，但是绕一个任意点旋转，而不是原点（P1，P2）。在这种情况下，旋转矩阵与围绕原点旋转是一样的，但是需要一个位移矢量。在一般情况下，关于一个点（X0，Y0）旋转 θ 角，所需的位移矢量为：

$$\begin{bmatrix} x_0(1-\cos\theta) - y_0\sin\theta \\ x_0\sin\theta + y_0(1-\cos\theta) \\ 0 \end{bmatrix}$$

为了在 Turbo PMAC 的代码实现这个，假设 Q40 到 Q48 是如在上面的旋转例子中的那样，添加：

```
Q50=P1*(1-COS(15))-P2*SIN(15)
Q51=P1*SIN(15)+P2*(1-COS(15))
Q52=0
AROT40                  ; Create 15 degree rotation
ADIS50                  ; Create proper displacement
```

当前位置变换

当一个坐标系进行转换时，很重要的要认识到，即将到来的移动的起始位置被转化，而这对没有在这个移动中明确地指令的轴有影响效果。在绝对模式中，任何没有明确命令的轴都隐含地接收一个命令到其现有的位置，在这种情况下，是变换后的位置。例如，如果该绝对移动 X1Y0 后跟一个 45° 的旋转，这个位置被变换到 X0.707Y0.707。如果这后跟一个绝对 X2 移动指令，这相当于一个 X2Y0.707 命令，而不是一个 X2Y0 命令。

16-16 输入一个运动程序

该运动的程序语句是一次输入一个程序缓冲区到 PMAC。对于每一个程序缓冲区，第一步是用 OPEN PROG n 命令打开缓冲区用于输入（其中 n 是缓冲区号 - 范围为 1 到 32,767）。其次，如果有任何东西目前不应该保持在缓冲区，它应该被 CLEAR 命令清空。现有的行不能被编辑，也不能在现有行之间插入新行，只有新的行可以追加到末尾（当然，首先是清除整个缓冲区）。

通常在程序开发中，编辑将在基于主机的文本编辑器来完成，如 PMAC 执行程序编辑器上，而老版本的 PMAC 程序缓冲区会在每次新版本下载到卡时被清零。最后的程序语句被下载后，一个 CLOSE 命令应该被发送到卡上来关闭程序缓冲区。

在 OPEN PROG n 命令之前发出一些命令是一个好主意，以确保缓冲区空间已经为程序语句准备好了。首先，确保当前没有正在执行的运动程序（除了旋转程序）。A（用于定址的坐标系）或<CTRL-A（适用于所有坐标系）中止命令可以用来确保执行已停止。此外，请确保没有其他的缓冲区是开启的；使用 CLOSE 命令用于这一点。接下来，请确保所有打开的缓冲区空间尚未开始数据收集，使用

DELETE GATHER 命令用于这一点。

所有这些（在线）的命令可以通过实际的运动程序语句包含在编辑文件中，即使它们不是实际的程序的一部分。如果 PMAC 被要求执行 LIST PROG n，他们不会被报告为程序的一部分。(该 PMAC 执行程序编辑器，作为它的上传功能的一部分，将这些命令附加到返回的程序中。)

在文本编辑器中工作时，建议使用的格式是：

```
A
CLOSE
DELETE GATHER
OPEN PROG n
CLEAR
{program statements}
CLOSE
```

该程序被下载和缓冲区关闭之后，来执行这个程序的坐标系统必须用 B 命令“指向”程序。例如，B6 将定址的坐标系统的程序计数器指向运动程序 6 的开始。这可以通过 PC（程序计数器）的查询命令得到证实，如果它指向程序 6 的顶部，该命令应该返回 P6:0。如果它返回一个<BELL>字符，它不指向任何有效的程序。一旦坐标系计数器指向程序的顶部，可以通过 R 命令开始执行程序。B 和 R 命令可以组合成一个命令行中，如 B6R。

学习一个运动程序

使用在线 LEARN 命令可以使得 Turbo PMAC 学习一个运动程序的行。在这次操作中，轴移动到所需的位置，而且命令发送给 Turbo PMAC。然后，Turbo PMAC 添加一个命令行到打开的运动程序缓冲区来表示这一位置。这个过程可以重复学习一系列点。

该电机在移动时可以是开环或闭环。在 LEARN 命令的时候，Turbo PMAC 读取电机指令位置（在开环，指令位置总是等于实际位置），并通过有效地执行一个 PMATCH 命令而将其转换成轴位置，与轴定义公式相反。

如果 LEARN 命令指定了学习哪个轴（如 LEARN（A，B，C)），那么只有那些轴命令将被添加到程序中。如果 LEARN 命令不指定任何轴，所有的 9 轴名称的命令将被添加到运动程序。

LEARN 功能只能添加轴移动指令到程序中。运动程序的任何其他部分，包括数学、逻辑、移动模式、移动时间，必须直接发送到打开的运动程序缓冲区。

运动程序结构

Turbo PMAC 运动程序通常是运动说明语句、计算语句和逻辑语句的组合。运动说明语句被用来生成轴的指令轨迹，根据本节前面所述的每个轨迹模式的说明中的规则。计算语句可以被用来确定运动说明的参数，并且逻辑语句可以被用来确定哪些动作语句被执行，以及何时执行。

基本移动说明

最简单的运动程序仅包含运动语句。例如：

```
F5000
X10000
DWELL1000
X0
```

（请记住，在输入这个程序时，必须在程序中加上上面所解释的缓冲控制命令。）

F（进给速度）语句指定一个速度，**X** 语句命令 X 轴的实际移动，**DWELL** 语句命令指定的暂停时间。这个程序只是简单地规定一个基本的移动和返回。

默认

一个如此简单的程序依赖于相当多的默认设置和模式。这个程序使用下面的默认：线性移动模式、ABS（绝对值）轴说明，Isx87 和 Isx88 分别指定 **TA** 和 **TS** 加速时间。

控制参数

在程序中的值意味着什么也可以依赖于外部参数。X 的位置是以用户单位，如同在轴线定义语句中对 X 轴的定义。**F** 速度是用户位置单位除以进给速度时间的单位，如同坐标系变量 Isx90 设定的那样。

在多轴同步移动

要在一个坐标系中同时执行几个轴的坐标运动，干脆把他们的运动说明放在同一行。例如，如果上面的程序改变为：

```
F5000
X10000 Y10000 Z10000
DWELL1000
X0 Y0 Z0
```

X、Y 和 Z 轴将命令同时移动到 10,000，留在那里一秒钟，然后命令同时移动到 0。

如果坐标系的轴不是在一个给定的移动行指令的，那么假设该轴指令一个零距离移动（请注意，它在技术上进行移动，所以它不能到位）。

循序移动

如果程序是在直线、圆弧、PVT 或样条曲线模式，并且在一个程序不止一个运动命令行，而且之间没有 **DWELL** 或 **DELAY**（之间也可以是其他的语句），那么该运动将融合在一起，而不停止。混合的确切形式将取决于现行的移动模式（见轨迹特性）。但是，如果坐标系的 Isx92（移动混合禁用），被设置，此混合功能被禁用。

添加逻辑

可以添加一点逻辑使语言更加强大。假设我们要重复上述序列 100 次。不是将上述声明重复 100 倍，而是我们可以创建一个循环：

```
F5000
P1=0
WHILE (P1<100)
    X10000
    DWELL1000
    X0
    DWELL1000
    P1=P1+1
```

```
ENDWHILE
```

请注意，F5000 语句是不在循环内的。通过把它放在循环之前，我们可以节省 PMAC 而不必每次通过循环时都解释并执行该语句。由于它是一个模态声明，其效果保持有效。这不是必须的，但如果循环时间很短，也可以有所作为。

行标签

可以把行标签放在你的运动程序中标记该程序的特定部分。行标签的语法是 **N{常数}**或 **O{常数}**，其中，**{常数}**为 1～262143 的整数。

请注意，这些都是行标签，不是行号（即使它们是由数字指定的）。一行不需要标签，标签不必按数字顺序。这些行标签仅用于指定在 **GOTO**、**GOSUB** 和 **CALL** 命令中的跳跃（所有在下面讨论）。

GOTO 命令

Turbo PMAC 在其运动程序的语法提供了一个 **GOTO{数据}**命令，这将导致跳转到同一运动程序的行标签 **N {数据}**的位置（不返回）。一般情况下，强烈反对使用 GOTO 命令，因为有建立很难破译的程序的倾向（降低程序的可读性）。

然而，当一个 GOTO 命令中的**{数据}**是一个变量或表达式(例如 **GOTO(P20)**)，它可以用来构建一个相当于结构化的 CASE 语句的东西，创造了多个选择分支点。请参阅程序命令规范中的 GOTO 描述的一个例子。

添加变量和计算

通过使用变量和数学计算可以使得运动程序更灵活。上面的例子程序在每次执行时将命令同样的动作和相同的时间。如果必须改变该程序的任何参数，整个程序必须重新输入（或使用不同的程序）。但是，如果使用变量代替常量，只须改变变量值来改变程序的动作：

```
F(P2)
P1=0
WHILE (P1<P3)
        X(P4)
        DWELL(P5)
        X0
        DWELL(P5)
        P1=P1+1
ENDWHILE
```

变量 P2、P3、P4 和 P5 可以由主机通过在线命令设置（例如，**P2= 2000**），作为输入和/或计算的结果由 PLC 程序设置，或者甚至由这个或另一个运动程序来设置。

通过使用常量、变量、函数和运算符，一个 PMAC 运动程序中可以建立一般的数学表达式（见本手册的《计算功能》的部分）。可在单独的程序语句来实现计算，分配计算出的值给变量，然后在另一个语句中使用该变量。另外，直接使用在运动声明的表达式，在这种情况下执行该语句后将不会保留表达式的值。

子例程和子程序

可以在 PMAC 运动程序中创建子例程和子程序，来通过可重复使用的子程序创建结构良好的模块化程序。在运动程序的 **GOSUBx** 命令导致跳到同一运动程序的行标签 **Nx**。在遇到一个 RETURN 命令时，程序执行将跳回到紧跟 GOSUB 后面的命令。这将创建一个子例程。

一个运动程序的 **CALLx** 命令引起跳转到 **PROG x**，遇到一个 RETURN 命令时马

上跳回到 CALL 后面的下一个命令。如果 x 是一个整数，将跳转到 PROG x 的开始位置；如果 x 有小数部分，将跳转到行标签 N（y* 100,000），其中，y 是 x 的分数部分。这种结构允许创建特殊子程序，无论是作为一个独立的子例程，或作为子例程的集合，可以从其他运动程序中调用。

将参数传递给子例程

可通过使用 READ 语句使这些子程序调用功能更为强大。子程序的 READ 语句可以返回到调用行并取得被用作子程序参数的值（与其他字母相关联的）。A 后的值将被放置在执行程序的坐标系的变量 Q101 中，B 后的值将被放置在 Q102，等等（Z 值放在 Q126）。

这种结构对于建立机床类型的程序特别有用，其中语法必须由零件程序中的字母、数字组合单独组成。由于 Turbo PMAC 将 G、M、T 和 D 代码作为特殊子程序调用对待（见下文），所以 READ 语句可以用来让子例程访问代码后面的零件程序行的值。

例子

例如，命令 CALL500 X10 Y20 导致跳转到 PROG 500 的顶部。如果在 PROG 500 的顶部，有一个命令 READ（X，Y），其中 X 的值将被分配到 Q124（X 为第 24 个字母），Y 的值将被分配到 Q125。现在该子例程能通过 Q124 和 Q125 的的值（在这种情况下，分别是 10 和 20）工作，根据需要进行处理。

已经传递的引数

READ 语句还提供了查看已经实际传递的引数（参数）的能力（在 READ 语句中列出的字母是那些可以传递的）。坐标系的 Q100 的位被用于注意引数是否已顺利传递；如果一个 A 引数已经传递，0 位等于 1；如果 B 引数已经传递，1 位等于 1，等等，如果 Z 引数已经传递，25 位设置为 1。在最新的子例程或子程序调用中没有传递的任何参数的对应位设置为 0。

如果子例程的逻辑需要知道某个参数是否已经传递，应该用 Q100 和查询位的值之间进行逐位 AND 运算（&）。0 位的值是 1， 1 位的值为 2，2 位的值是 4，等等（对于字母表中的第 N 个字母，位值是 2^{N-1}）。例如，要查看 D-引数是否已经传递，条件是：

```
IF (Q100 & 8 > 0) ...
```

D 是第四个字母，所以该位的值是 2^3= 8。要查看引数 S 是否已经传递，S 是第 19 个字母，因此该位的值是 2^{18}=262,144，条件是：

```
IF (Q100 & 262144 > 0) ...
```

在本手册的程序命令说明部分，READ 语句说明介绍了各个字母引数的 Q 变量、位号和位值。

PRELUDE 子例程调用

Turbo PMAC 允许在运动程序或部分的运动程序中，在每个运动命令或其他字母数字命令之前创建一个自动子例程调用。如果子例程以一个 READ 命令开始，那么移动指令或字母数字命令本身会为了子程序调用而转换引数。此功能非常有用于执行机床类型程序的封闭循环，或用于为了一个执行反向运动或类似计算的子程序而将普通的移动指令转换为引数。

这种能力是通过运动程序 PRELUDE 命令来完成的。要打开此功能，在运动程序中声明 PRELUDE1，接着输入你希望要在每个后续运动指令或字母数字命令之前执行的子程序调用。这个子程序调用可以通过 CALL 命令，或 G、M、T 或 D 码来声明，其中任何一个都是一个特殊的子程序调用。在一个 PRELUDE1 声明中，

子程序调用中用于指定哪些子程序、哪些行的值必须是一个常量，它不能是一个变量或表达式。

一旦 PMAC 已经在程序中遇到了一个 PRELUDE1 命令，那么在每次遇到运动程序的一个运动指令或其他字母-数字命令时，它会执行指定的子程序调用（包括 G、M、T 和 D 代码，但不包括 N 和 O 行标签）。移动命令或字母-数字命令必须在一个程序行的开头，或立即跟随在程序行开头的 N 或 O 行标签的后面。

一旦 PMAC 已跳到了由 PRELUDE1 指定的子程序，它会对待子程序的任何运动命令或数字字母命令如同它通常的那样，这些将不会自动导致另一个子程序调用。因此，自动 PRELUDE 子程序调用不能彼此嵌套在对方内，但一个单独的 **PRELUDE** 子程序调用可以嵌套在明确的子例程和子程序调用内，并且明确的显式子例程和子程序调用可以嵌套在一个自动 PRELUDE 子程序调用中。

一个新 PRELUDE1 命令取代现有的 PRELUDE1 命令。一个 PRELUDE0 命令（无必要的引数）关闭 PRELUDE 的功能。

运行一个运动程序

一旦你的运动程序已经输入而且程序缓冲区关闭，您可以执行运动程序了。由于 Turbo PMAC 一次可以存储多个程序，你必须做的第一件事就是告诉 Turbo PMAC 的坐标系统您希望运行哪个程序（记住，在 Turbo PMAC 中是一个坐标系统执行一个运动程序；不同的坐标系可能在同一时间执行其他运动程序）。

指向程序

这是通过 **B{常量}** 命令完成的，其中的 **{常量}** 表示运动程序缓冲区的编号。在任何运动程序缓冲区已经打开之后，使用 B 命令来改变运动程序。如果多次运行相同的运动程序而不加修改，那么没有必要使用该命令的；当 PMAC 执行完成一个运动程序后，坐标系的程序计数器被自动设置为指向该程序的开始，准备再次运行它。

使用 **PC**（程序计数器）命令来查看当时坐标系指向哪一个程序。你将接收到一个 **P5:0** 的响应，这表明该坐标系是指向运动程序 5 的顶部（地址偏移量为 0）。

运行程序

一旦指向要运行的运动程序，就可以发出命令启动执行程序。如果需要连续执行程序，可以使用 R 命令（**<CTRL-R>** 同时用于所有坐标系）。在 Turbo PMAC 控制器，将 JPAN 连接器上的 START /线变低，该线与同一连接器上的 FPDn/线路选择的坐标系相连接。程序将一直执行，除非被一个命令或错误条件所停止。

步进程序

只执行一步移动或程序的一小部分，使用 S 命令（<CTRL-S>用于所有坐标系统同时），或对 JPAN 连接器上的 STEP/线路采取低电平，该线是与同一连接器的 FPDn/线选取的坐标系相连接的。程序将执行到第一个运动的 **DWELL** 或 **DELAY** **处**，或者如果它第一次遇到 BLOCKSTART 命令时，它会执行到 BLOCKSTOP 命令。

PMAC 检查

当发出一个运行或步进命令，Turbo PMAC 检查坐标系，以确保其处于正常的工作秩序。如果它发现坐标系中有任何的设置不正确，就会拒绝执行命令，发送一个<BELL>命令回主机。如果 I6 设置为 1 或 3，它会报告一个错误号码，以及告知该命令被拒绝的原因。

Turbo PMAC 将拒绝一个运行或步进命令的任何原因如下：

- 坐标系统中的一个电机同时跳闸两个超程限制（ERR010）
- 坐标系统中的一个电机正在执行一个移动（ERR011）
- 坐标系统中的一个电机不是在闭环控制（ERR012）
- 坐标系中的一个电机没有激活{Ix00=0}（ERR013）
- 没有电机分配给坐标系（ERR014）
- 一个固定的（非旋转）运动程序缓冲区被打开（ERR015）
- 未指向任何运动程序（ERR016）
- 在一个/或\ 停止命令后，坐标系中的电机不在停止点上（ERR017）

16-17 实现一个机床类型的程序

Turbo PMAC 允许执行机床类型的 RS-274（G 代码）程序，通过将 G、M、T 和 D 代码作为子程序调用。这允许机床制造商定制机器代码，但它要求制造商真正实现将执行所需操作的子例程。很多代码是相当标准的，泰道公司已经提供了这些例子。本节超越了简单的标准来讨论代码执行中涉及的敏感问题。

G、M、T 和 D-代码

当 Turbo PMAC 在运动程序遇到带一个值的字母 G 时，它把命令作为是一个 CALL（调用）运动程序 10n0，其中 n 是 G 后的值的百位上的数字。无百位的值（数学术语为取模 100）控制的是跳转到程序 10n0 的哪个行标签。这个值乘以 1000 来指定行标签的数值。当遇到一个 return 语句，它会跳回到调用程序。

例如，**G17** 将导致直接跳转至 PROG 1000 的 N17000；G117 将导致程序直接跳转到 PROG1010 的 N17000；G973.1 会导致跳至 PROG1090 的 N73100。

M 代码也是一样的，除了他们使用的是 PROG10N1；T 码使用 PROG10n2，D-码使用 PROG10n3。

大多数时候，这些代码的数字范围在 0 到 99 内，所以执行它们只需要 PROGs1000，1001、1002 和 1003。对于那些想延伸代码数字超过 100 的用户，执行它们将要求 PROGs1010、1011 等。

制造商的任务是对运动程序 10n0 到 10n3 编写程序，来以他想要的方式实现代码。一旦做到这一点，执行的方法对零件程序员和机器操作员是不可见的。

标准 G 代码

现在，我们将看到实施一些比较常见的 G 代码中涉及的问题：

G00 - 点到点定位

通常，此代码是通过使用 **RAPID** 命令而在 PMAC 中实施的。许多用户将只有 **RAPID RETURN** 作为他们实现这段代码。(由于这是对 PROG1000 的 N0 的一个 CALL（调用），而 N0 标签是自动暗示在任何运动程序的开始，你不应该显式地添加一个 **N0**，该例程必须在 PROG1000 的最顶端。)

使用一个外部进给倍率修调信号的用户经常想要在快速模式下禁用倍率。这在 PMAC 内是通过设置时基源地址变量恢复为默认值完成的，并远离外部源（例

如：I5193= $2000)。另外，如果机器有一个单独的快速倍率修调设置，这个变量可以设置为另一个外部来源。实现 G00 的文件部分看起来是这样的：

```
CLOSE
OPEN PROG 1000
CLEAR                ; To erase old version when sending new
RAPID                ; First actual line of program
I5193=$2000
RETURN
```

G01 – 直线插补方式

通常情况下，这段代码在 PMAC 中是通过使用 **LINEAR** 指令实施的。最简单的实现是 **N01000 LINEAR RETURN**。如果需要进给倍率修调，而且在快速模式下被禁用，那么子例程应设置时基源地址变量到包含外部信息的寄存器（如 I193=$350A)。

G02 – 2D（平面）顺时针圆弧模式

通常情况下，这段代码在 PMAC 中是通过使用 **CIRCLE1** 命令实施的。最简单的实现是 **N02000 CIRCLE1 RETURN**。如果需要进给倍率修调，而且它本应在快速模式下被禁用，子例程应设置时基源地址变量到包含外部信息的寄存器（如 I193=$350A)。

G03 – 2D（平面）逆时针圆弧模式

通常情况下，这段代码在 PMAC 中是通过使用 **CIRCLE2** 命令实施的。最简单的实现是 **N02000 CIRCLE2 RETURN**。如果需要进给倍率修调，且它本应该在快速模式下被禁用，子例程应设置时基源地址变量到包含外部信息的寄存器（如 I193=$350A)。

G04 – Dwell 指令

此代码需要使用 READ 命令。不同类型的 G 代码在一个 P 或 X 后有停留时间。PMAC 可以用一个 READ（P）或 READ（X）分别进行处理;; P 值将放在 Q116，X 值将被放置在 Q124。另外，时间的单位必须加以考虑。 PMAC 停留单位是毫秒。如果 G04 单位是秒，传递的值必须乘以 1000。一个典型的实现将是 **N04000 READ（P）DWELL（Q116*1000）RET**。

G09 – 精确停止

在 G 代码的一些类型，此代码会导致在两个运动之间的停止，这样运动之间将没有拐角圆角混合。在 PMAC，这可以简单地通过执行一个短的停留时间来实现。一个典型的实现将是 **N09000 DWELL10 RET**。

G17、G18、G19 – 选择平面

这些代码选择在哪个平面进行圆弧插补和刀具半径补偿。G17 选择 XY 平面，G18 选择 ZX 平面和 G19 选择 YZ 平面。在 PMAC，这是通过 NORMAL 命令进行的，NORMAL 命令指定垂直于这个平面的矢量(并且不局限于这些选择)。标准的 Turbo PMAC 执行这些代码将是：

```
N17000 NORMAL K-1
RETURN
N18000 NORMAL J-1
RETURN
N19000 NORMAL I-1
RETURN
```

在这里重要的是，RETURN 命令是在一个单独的行；否则，当 PMAC 返回到调用子程序的那一行时，NORMAL 命令会尝试从该行“捡”更多的引数。

您可能还希望在这些例程设置一些变量来表明已被指定什么平面，如果你想使用这些信息用于其它程序的话 7（如 G68 旋转）。Turbo PMAC 的圆弧插补半径补偿程序并不需要这样的一个变量。

G40、G41、G42 – 刀具半径补偿

刀具半径补偿可以通过 CC0、CC1、CC2 PMAC 命令很容易地被打开和断开，这些命令分别对应于 G40、G41 和 G42。实现这个的子程序将是：

```
N40000 CC0 RETURN       ; Turn off cutter compensation
N41000 CC1 RETURN       ; Turn on cutter compensation left
N42000 CC2 RETURN       ; Turn on cutter compensation right
```

G90 – 绝对移动模式

通常情况下，这段代码在 PMAC 中是通过使用 ABS 指令实施的。无轴列表的 ABS 命令将坐标系中所有轴放在绝对运动模式。典型的实现将是 **G90000 ABS RETURN**。如果一种 G 代码格式有 G90 使得圆弧运动中心向量也是绝对的（这是非标准），那么 ABS（R）的命令应该被添加到这个程序中。

G91 – 增量移动模式

通常情况下，这段代码在 PMAC 中是通过使用 INC 指令实施的。无轴列表的 INC 命令将坐标系中的所有轴放在增量移动模式。典型的实现将是 **G91000 INC RETURN**。如果 G 代码格式有 G90 和 G91 也影响圆弧移动中心向量的模式(非标准)，这个程序中应该添加一个 **INC（R）**命令。

G92 – 位置设置（预载）命令

如果此代码只是用来设置轴位置，实施非常简单：**N92000 PSET RETURN**。通过同一行的 return 语句，程序会跳回调用行，并使用那里的值（例如 **X10 Y20**）作为 PSET 命令的引数。但是，如果代码也被用于其他的东西，如设置主轴最高转速，子程序将需要变得更长并在程序内进行设定。

例如，如果 G92 被用于在 X、Y 和 Z 轴预加载位置，设置最大主轴转速（S 参数），并定义从刀尖到主轴中心的距离（R 参数），子程序可以是：

```
N92000 READ(X,Y,Z,S,R)
IF (Q100 & 8388608 > 0) PSET X(Q124)      ; X axis preload
IF (Q100 & 16777216 > 0) PSET Y(Q125)     ; Y axis preload
IF (Q100 & 33554432 > 0) PSET Z(Q126)     ; Z axis preload
IF (Q100 & 262144 > 0) P92=Q119           ; Store S value
IF (Q100 & 131072 > 0) P98=M165-Q118      ; Store R value
RETURN
```

在每一行中条件的目的是看看有是否引数实际上已发送到子例程调用的子例程中，如果没有，将不会使用该参数执行任何功能（见传递参数一节，上文）。在用到 S 引数的情况下，该值被简单地存储用于由其它例程以后使用，这样，一个指令的主轴速度不会超过此处所指定的限制。在 R 引数的情况下，该例程将计算当前指令 X 轴位置（M165）和声明的径向位置（R 引数：Q118）之间的差值，以获得一个偏移值（P98）。该偏移值可以由主轴程序使用去计算一个实时的径向位置。

G94 - 英寸（毫米）/分钟的模式

此代码设置了程序，使 F 值（进给速度）被解释为每分钟的长度单位（英寸或毫米）。在 PMAC，F-值被解释为是指一个速度（单位时间长度），其中长度单位是由轴定义语句设置的，该时间单元是由坐标系统变量 Isx90 设置的。由于 Isx90 的单位是毫秒，这个程序应该设置 Isx90 为 60,000。此外，因为通常 G94 用于取消 G95（通过使用主轴编码器作为外部时基源，解释 F-值为英寸（mm）/转），该例程应该将坐标系统返回到内部时基。一个典型的例程是：

```
N94000 I5190=60000          ; Feedrate is per minute
I5193=$2000                 ; Use internal time base
RETURN
```

G95 -英寸（毫米）/转的模式

此代码设置了程序，使 F 值（进给速度）被解释为主轴每转的长度单位（英寸或毫米）。在 Turbo PMAC 中，这需要坐标系统的时基通过主轴编码器进行控制。进给速度仍解释为单位时间的长度，但有了外部时基，时间将被解释为输入频率的比例，因此，也和主轴旋转速度成比例，从而进给速度是每转的有效长度。

因此，子例程执行 G95 必须使程序从主轴编码器获取其时基，并得到正确的比例常数。（其实一些常数或全部常数可提前设置）。这种外部时基功能是通过被称为编码器转换表的 PMAC 软件功能来执行的，转换表详细记录在本手册的《设置编码器转换表》一节。关于设置外部时基的详细说明是在本手册的《同步 Turbo PMAC 到外部事件》的部分。

简言之，时间和频率之间的比例因子必须在定义了一个实时输入频率（RTIF）的转换表中进行设置。然后可以编写运动程序，仿佛它总是得到这个频率。在我们的例子中，我们将采取接近或大于我们的最大值的实时主轴转速。

例如，我们使用 6000rpm（100 转/秒）作为我们的实时主轴转速。在实时，主轴转一转需要 10 毫秒，所以我们想进给速度的单位是每（10 毫秒）的长度，我们可通过设置 Isx90（进给速度时间单位）为 10 来实现。如果主轴的每一转我们有 4096 个计数（解码后），我们的 RTIF 是 4096×100=409,600 计数/秒=409.6 计数/毫秒。转换表的时基比例因子（TBSF）的等式为：

$$TBSF = 131{,}072/RTIF(cts/msec) = 131{,}072/409.6 = 320$$

该值必须为一个整数用于真正的同步，而没有任何舍入误差。如果主轴编码器的分辨率是一个 2 的幂，通常它是容易的。如果不是，以 rps 的实时主轴转速应该是 2 的幂，并且 Isx90 不会是一个整数（这是好的）。

这个缩放因子将被写入在转换表中的相应寄存器中。一般情况下，每次执行 G95 时，不用必需执行这个；而是，它可以作为系统设置的一部分。

G95 的典型的子例程将包括为坐标系设置的 Isx93 和 Isx90：

```
N95000 I5190=10             ; PMAC F is length/ 10 msec
I5193=$350A                 ; Time base source is external
RET
```

G96 - 恒定表面速度模式使能

此代码设置了程序，使主轴放在恒定表面速度（CSS）的模式下。在这种模式下，主轴的角速度实时变化，使得其经过刀尖的表面速度始终保持恒定。本质上，这意味着主轴的角速度是与主轴中心到刀尖的径向距离成反比的。这个距离通常是 X 轴的位置---这意味着 X 轴的零点位置是在主轴中心。有些 G 代码方言允许零件程序通过 G92 R（参阅 G92）创建一个 X 轴偏移，它定义了在当前 X 轴

指令位置的径向距离。

建议这里的 CSS 模式的方法是在主轴处于一个独立于其他轴的 PMAC 坐标系统中。这就允许一个主轴程序被执行和反应的速率不同于主零件程序的，但最终是要由零件程序通过变量和标志控制的。这种类型的主轴程序将在下面详细说明。

一个 G96 代码将以英尺/分钟或者米/分钟的单位来携带主轴表面速度 S 码。该值应该被放置在一个变量中用于主轴程序去读取。也应设置一个标志来表明主轴在哪个模式。请注意，主轴模式和速度都可以独立设置于主轴的开/关状态和方向（参见 M03、M04、M05）。

使用这种方法的一个典型的 G96 例程将是：

```
N96000 READ(S)     ; Read spindle surface speed into Q119
P96=Q119           ; Store spindle speed
M96=1              ; Flag to mark CSS mode
RETURN
```

G97 – 恒定表面速度禁用

该代码取消主轴的恒定表面速度模式，将主轴放在恒定角速度模式。在这种模式下，主轴转速是独立于刀具的径向位置。由于主轴轴线在独立坐标系中，执行此代码的子例程会简单地设置一个变量和一个标志用于该程序查看。通常情况下，G97 代码将以 RPM 的单位来携带主轴转速 S 码。如果是这样，例程会读取它，并把它放到一个变量中。如果没有执行，该例程允许主轴程序保持其在 G96 下从表面速度和径向距离计算的最后的 RPM。

使用这种方法的一个典型的 G97 例程将是：

```
N97000 READ(S)                          ; Read spindle RPM into Q119
IF (M100 & 262144 > 0) P97=Q119          ; Store for spindle program
M96=0                                   ; Cancel CSS mode
```

主轴程序

在 Turbo PMAC 可以以许多不同的方式来完成控制主轴轴线，这取决于主轴需要做什么。当然，主轴运行的最简单的类型，是主轴只是简单地被要求以恒定速度在一个方向或另一个方向进行长时间的移动。在这种情况下，就没有必要写一个主轴运动程序；要么 Turbo PMAC 只是输出一个与速度成正比的电压（所以主轴是开环，直到 Turbo PMAC 改变），或主轴电机被手动控制慢跑（在 Turbo PMAC 闭环控制下）。

主轴手动控制

手动控制的主轴电机不需要在任何坐标系中（它*必须*不能与其他轴在同一坐标系，否则当一个零件程序正在运行时，它不能被手动控制），但把它放不同的坐标系中是一个好主意，因为不在任何坐标系中的电机是利用坐标系 1 的时基控制（进给倍率修调）。

主轴转速的值被缩放并放入慢跑速度的 I-变量（Ix22）中，主轴的开/关功能将命令慢跑启动和停止（见 M03、M04 和 M05）。

开环主轴

如果使用开环主轴，通过使用一个 M 变量直接写入一个不使用的 DAC 输出寄存器中来实现。例如，该定义 **M425-> Y:$ C00A,8,16,S** 将变量 M425 与 DAC4 输出寄存器相互匹配。赋给该 M 变量的任何值将引起在 DAC4 输出线上有对应的电压。

在这种方法中，一个主轴-启动的命令（见 M03，M04）可以是 **M425= P10** 或 **M425=-P10**，其中 P10 已经由一个 S 码预先设定。主轴关闭命令（见 M05）是 M425=0。

在定位和主轴之间切换

有一些情况，在其中该主轴电机有时用作常规轴，进行定位移动而不是稳定的速度运行，有时作为常规主轴。在这种情况下，主轴电机将作为主坐标系中的一个轴，因此它可以做协调一致的运动。当需要主轴实际操作时，可以通过设置电机的比例增益为零并写入输出偏移寄存器（Ixx29）来创造一个伪开环模式。在该方法中 Ixx29 将被视为就像上面的 M425 那样。当然，这种模式下的操作需要一个速度回路（转速表）放大器。更多的细节参见示例 OPENCLOS.PMC。

恒定表面速度的主轴

如果你想主轴能够进行恒表面速度（CSS）的模式，你必须写一个运动程序，因为速度作为另一轴位置的函数必须是变化的。建议的方法---显示在例子 SPINDLE.PMC 中---是将运动分解为小的时间段，每个段的指令距离取决于那时系统的条件，包括指令速度、模式和刀具径向位置。

如果主轴是在 CSS 模式下以开环的方式进行控制，最好是有一个 PLC 程序作为刀具径向位置的函数修改输出指令（Mxx25 或 Ixx29）。PLC 程序的结构将是很像闭环运动程序示例 SPINDLE.PMC 的结构，除了不需要实际的移动指令；一旦进行数学处理，该值将被分配给相应的变量。

标准的 M 代码

以下各部分详细介绍实施标准的 M 代码所涉及的问题。认识到 M 变量和一个程序的 M 代码之间的区别是很重要的。要被解释为一个 M 变量，它必须在一个方程式或表达式中使用。例如，M01= 1 是指 M 变量编号 1（通常它设置机器输出 1），而 M01 本身是 M-码编号 1。

M 代码被视为运动程序 1001 中相应的行标签的子程序调用。

M00 – 程控停止

要执行此代码的例程只需要包含 STOP 命令。此代码是寻找 PROG1001 的行标签 N0，任何程序的开头总是隐含 N0，所以这一定是在 PROG1001 的最顶端。实施这个的文件的部分是：

```
CLOSE
OPEN PROG 1001          ; Buffer control command
CLEAR                   ; To erase old when sending new
STOP                    ; First line of actual program
RETURN                  ; Will jump back when restarted
```

M01 – 可选停止

通常情况下，如果操作面板上的“可选停止”开关被置位，那么这段代码是用来执行停止功能。假设这个开关被连接到 PMAC 的机器输入 1，而且变量 M11 已被分配到这个输入（这是默认的），然后执行此代码的例程是：

```
N01000 IF (M11=1) STOP
RETURN
```

M02 – 程序结束

由于 PMAC 自动识别一个程序结束，而且程序指针重置回到程序的顶部，该例程这段代码将是空的（只有 RETURN 语句）。然而，在许多系统中，大量的变量

和模式被置到默认值。结束程序的一个典型的例程是：

```
N02000 M55=0            ; Turn off spindle
M7=0                    ; Turn off coolant
M2=0                    ; Turn off conveyor
DWELL 0                 ; Execute pending synchronous M-vars
LINEAR                  ; Make sure not in circular mode
RETURN
```

M03 – 主轴正转（顺时针）
M04 – 主轴反转（逆时针）
M05 – 主轴停止

如果主轴简单在做恒定速度运动时，这些例程可以简单地发出手动慢跑命令。例如：

```
N03000 CMD "#4J+"
RET
N04000 CMD "#4J-"
RET
N05000 CMD "#4J/"
RET
```

当然这假设，Turbo PMAC 上的电机＃4 是主轴电机，而向上计数方向为顺时针方向。主轴转速已经在其他程序通过设置 I422(电机＃4 慢跑速度)而被确定。

如果 Turbo PMAC 是通过一个开环电压控制主轴，这些例程会通过写入一个 DAC 寄存器而在未使用的模拟输出上输出一个电压。例如：

```
N03000 M402=P97*P9
RETURN
N04000 M402=-P97*P9
RETURN
N05000 M402=0
RETURN
```

此示例假定 M402 被分配到 DAC4 寄存器（Y：$07800A,8,16,S），P97 是所需的主轴转速（RPM），P9 是关联转速和 DAC 位的比例因子（3,276.7 DAC 位/伏）。有关更多详情，参见主轴程序部分。

如果需要更华丽的任务，如恒定表面速度，将要求主轴有一个独立运动程序，如在上面的例子中展示的那样。如果这些 M 代码与这个例子联系起来，它们是：

```
N03000 M55=1                ; Flag for clockwise spindle
CMD "&2B1010R"              ; Start the spindle program
RET
N04000 M55=-1               ; Flag for counterclockwise spindle
CMD "&2B1010R"              ; Start the spindle program
RET
N05000 M55=0                ; Flag for spindle off
RET                         ;
```

M07 – 低能级（喷雾）冷却液打开
M08 – 高能级（大量）冷却液打开

M09 – 冷却液关闭

这些 M 代码的实际实施将是非常依赖于机器，但它通常是非常简单的。例如，如果冷却液的开/关控制被连接到 Turbo PMAC 的机械输出 7，冷却液高/低控制被连接到 Turbo PMAC 的机械输出 8，例程可仅仅是：

```
N07000 M7=1              ; Set Mach. Out. 7: Coolant On
M8=0                     ; Clear Mach. Out. 8: Low Level
RETURN
N08000 M7=1              ; Set Mach. Out. 7: Coolant On
M8=1                     ; Set Mach. Out. 8: High Level
RETURN
N09000 M7=0              ; Clear Mach. Out. 7: Coolant Off
RETURN
```

如果需要为发生的变化提供一段时间， DWELL 的声明可添加在输出的设置之前和/或之后。

M12 – 排屑传送带打开

M13 – 排屑传送带关闭

这些代码的执行，虽然依赖于机器，但是通常会简单。例如，如果传送带的开/关线被连接到机器输出 2，这些例程可以简单地是：

```
N12000 M2=1              ; Set Mach. Out. 2: Conveyor On
RETURN
N13000 M2=0              ; Clear Mach. Out. 2: Conveyor Off
RETURN
```

M30 – 带循环的程序结束

参见 M02 的描述。大多数系统中 M30 将相当于 M02，但将返回到程序的开始。

默认条件

通常情况下，一台机器运行 G 代码类型的程序需要很多的默认值和模式，除了 PMAC 在其上电或复位周期自动设置的那些。要设置这些默认值，最好是用程序 PLC 1，它是在自动上电或复位周期后第一个被执行（有效延长了在这个周期完成的内容）。这个程序中的最后一行应该是 DISABLE PLC1，防止重复执行程序。用于这样一个程序的简单的文件是：

```
CLOSE
OPEN PLC 1
CLEAR
M55=0                    ; Spindle Off
P92=3000                 ; Maximum spindle RPM
P95=1000                 ; Max spindle accel. in RPM/sec
M70=0                    ; English measurements
DISABLE PLC 1            ; So this is only executed once
CLOSE
```

16-18 旋转运动程序缓冲区

旋转运动程序缓冲器允许在执行程序过程中下载程序行，并覆盖已执行的程序行。这就能够连续执行比 PMAC 的内存空间大的程序，以及实时下载程序行。

定义一个旋转缓冲区

每个坐标系统都可以有一个旋转程序缓冲区。要为一个坐标系创建一个旋转缓冲区，寻址该坐标系统（&N）并发送 **DEFINE ROT{常量}**命令，其中**{常量}**是内存中缓冲区的大小。一个程序中的每个值（例如 X1250）占据存储器的一个字。缓冲区的大小应留出足够的空间用于存储你要加载的执行点之前的。由于大多数应用程序使用旋转缓冲区不会限制 PMAC 的内存需求，为缓冲区的大小留出较大的安全边际（即加大缓冲区）是一个好主意。

例如，在一个四轴的应用程序中为了能够在执行点之前加载 100 个程序行，其中位置使用常数值（例如 **X1000 Y12000 Z1400 A1600**），缓冲区应该至少有 400 字的内存，所以为旋转缓冲区分配 500 或 600 字将是一个好主意（如 **DEFINE ROT 600**）。

用于定义所需的缓冲区状态

为了 PMAC 能够保留空间给旋转缓冲区，那么就不可能有数据采集的缓冲区、而且在发出 **DEFINE ROT** 命令时也没有用于较高编号坐标系的旋转程序缓冲区。因此，首先必须删除任何数据采集缓冲区，并且是从高位编号到低编号来定义旋转缓冲区。例如：

```
DELETE GATHER
&3 DEFINE ROT 200
&2 DEFINE ROT 1000
&1 DEFINE ROT 20
```

准备运行

要准备在一个坐标系统中运行一个旋转程序，当寻址这一坐标系统时，使用命令 B0（到程序零--旋转程序的开头）。这必须完成在没有缓冲区打开时，否则会被解释为一个 B-轴命令。一旦这样准备好，以 R 命令开始执行程序。该命令给出时缓冲区可以是打开的或闭合的。如果 R 命令给到一个空的旋转缓冲区，那么缓冲区将只需等待发给它一个命令，然后收到后立即执行该命令。

打开用于输入

OPEN ROT 命令将打开所有已定义的旋转程序缓冲区。接着这个程序行被发送到主机寻址的坐标系（&N）的缓冲区中。大多数旋转程序缓冲区的使用者将只有一个坐标系，所以这不会是他们关注的，但是可以动态切换坐标系并可以一次使用多个旋转缓冲区。

重要的是要认识到，在 **OPEN ROT** 命令后，PMAC 将尽可能多的命令作为缓冲

的命令，哪怕是要立即执行他们（一些命令作为一个在线的命令是一样的，但是作为缓冲的命令就不一样了）。例如，当缓冲区被关闭时，一个 I100 的命令是请求 I-变量 100 的一个值，但运动程序缓冲区打开时，它是命令执行一个 100 单位半径的完整的圆！（I 值是径向向量的 X 轴分量；由于没有给出轴的位置，它们都被认为是与起始点相同）。

保持领先于执行行

处理旋转程序缓冲区的关键是知道它领先多少行；也就是说，在 PMAC 正在执行的程序行之前，需要加载多少程序行。通常情况下它会提前加载到一定数量的领先行，然后等待，直到程序执行到数量较少的领行先以内时再继续加载。一个实时应用程序的工作可能只是领先于执行行一行；一个定期下载巨大文件的应用程序会领先 1000 行，然后当程序已经赶上 500 行以内再次启动加载。

PR 命令

有几种方法分辨领先多少行（有多少行未执行）的程度。首先是 PR（剩余程序行）命令，它返回领先的行数。这提供了一个非常简单的轮询机制，但是可能不适合于实时性强的应用。

BREQ 中断

对于实时性强的应用程序，有硬件线路来处理旋转缓冲区的握手，也有变量来控制线的转换过渡点。当定址坐标系的旋转缓冲区想要有更多的程序行时，该 BREQ（缓冲区请求）线为高电平，反之它为低。这条线被连接到 PMAC-PC 的可编程中断控制器，所以它可以被用来给主机 PC 产生一个中断。（请参阅本手册的使用 PMAC-PC 来中断主机 PC）。作为补充，JPAN 连接器上提供了 BREQ/。此外，对于每个坐标系有一个缓冲区满（BREQ/）状态位。

I17 停止中断

变量 I17 控制主机可以加载多少领先行，并仍然可以使得 BREQ 为真。如果一个程序行被发送到一个旋转缓冲区，BREQ 为低电平，至少是暂时的低电平。如果执行行前面的行仍比 I17 指定的行数少，BREQ 是再次变高，它可以产生一个中断。如果是领先的行数多于 I17 行或更多，BREQ 保留为低。当通过 OPEN ROT 来输入旋转程序缓冲区或定址坐标系统被改变时，BREQ 被拉低，然后如果缓冲区中执行点前面的行数小于 I17 行，BREQ 设置为高。

I16 重新启动中断

变量 I16 控制着当旋转缓冲区的执行程序追到最后的加载行时，BREQ 在哪被重新置位。如果在执行完一行后，在旋转缓冲区的领先行不到 I16 个，BREQ 被设置为高。这可用于通知主机，需要发送更多的程序行。

通过使用这两个变量和 BREQ 线中断，可以创建一个极其快速和高效的系统用于从 PC 机实时地下载程序。

如果缓冲区用完了

如果该程序计算追上了旋转缓冲区的读入加载点，不会有错误；程序操作将挂起，直到更多的行被输入到旋转缓冲区。技术上，程序仍在运行；真正停止程序必须给予一个 Q 或 A 命令。

如果 PMAC 是在分段模式（I13>0），并且正在执行旋转缓冲区中的最后一行，只要在减速停止开始前进入了新行，PMAC 将融入新的移动而无需停止。

关闭和删除缓冲区

CLOSE 命令将关闭旋转缓冲区，就像对其他类型的缓冲区做的那样。关闭旋转缓冲区不影响缓冲程序的执行，它只是防止新的缓冲指令被输入到缓冲区，直到缓冲区被重新打开。

DELETE ROT 为定址的坐标系删除了旋转缓冲区，并取消分配已预留给它的内存。

16-19 PMAC 如何执行一个运动程序

知道 Turbo PMAC 是以如何的工作方式通过一个运动程序是重要的。一个运动程序与一个典型的高级计算机程序的根本不同之处在于它有需要花费时间的语句（移动、DWELL 和 DELAY）；在计算时间和执行时间之间有重要的区别。

基本上，一个 Turbo PMAC 程序存是将数据传递到轨迹发生器例程中，在每个伺服周期为电机计算出一系列的指令位置。运动程序的工作必须领先于实际的指令移动，以保持数据持续送入轨迹发生器。如果程序无法保持领先地位，而且下一个移动的时间出来了而轨迹发生器的数据未正确到位，Turbo PMAC 将中止程序，并把坐标系中所有电机都停下来。

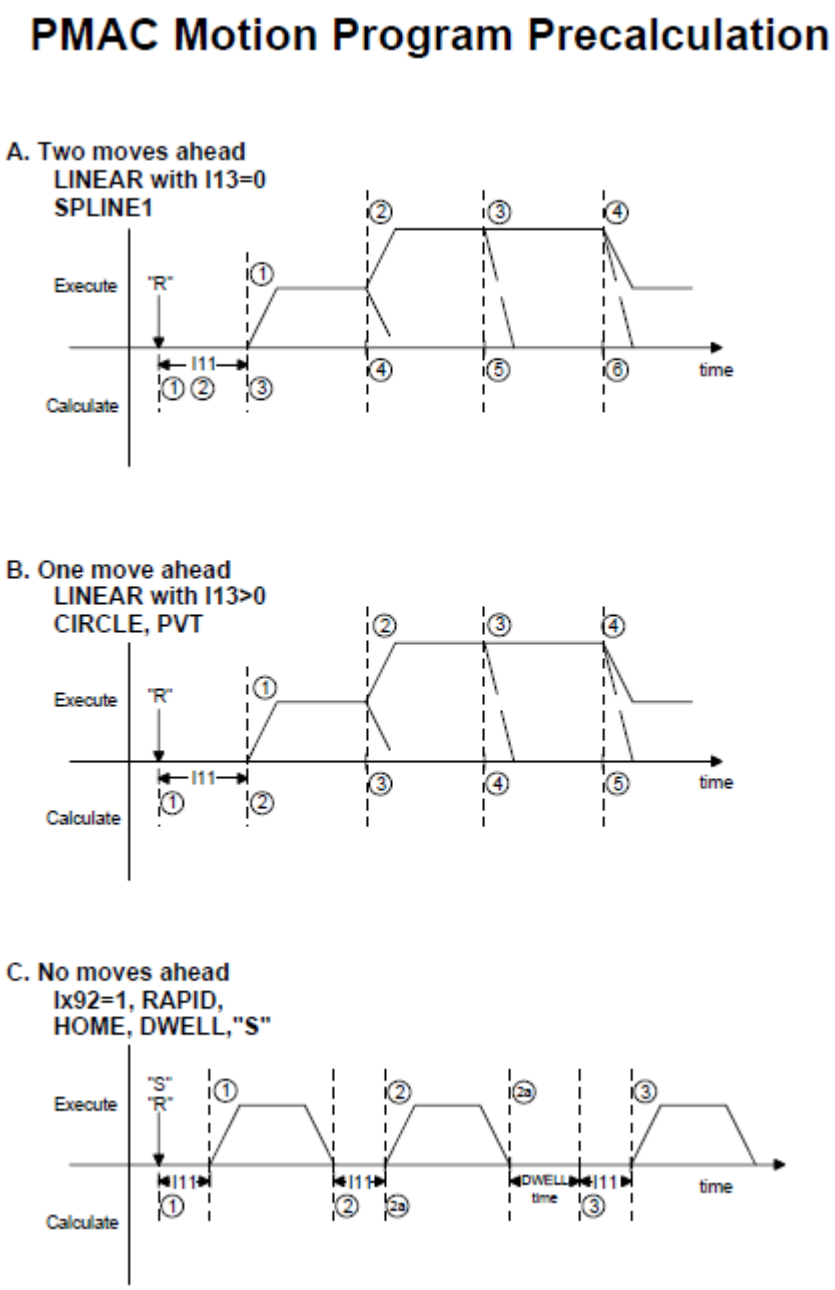
提前计算

PMAC 处理程序行是提前一个或两个运动（包括 DWELL 和 DELAY）。提前计算一个运动是必要的，以便能够将运动混合在一起；如果要执行适当的加速度和速度限制，或是要计算一个三点样条（样条模式），提前计算两个运动是必要的，。对于线性混合运动以及 Isx13（移动分段时间）为零（禁用），Turbo PMAC 提前计算两个运动，因为速度和加速度限制在这里启用。启用了特殊的前瞻缓冲区，Turbo PMAC 在连续序列中看起来有足够的领先移动，保持领先 Isx20 个段。在所有其他情况下，Turbo PMAC 是提前计算一个运动。

开始计算

一旦命令启动了程序，Turbo PMAC 会一次计算程序语句，并包括第一或第二个移动语句，这取决于移动模式和 Isx13 的设置。这可以包括多个模态语句、计算语句和逻辑控制语句。

已编程的运动实际上不会开始执行，直到已经过去 I11 毫秒后才开始，即使计算较早已完成。这使得在卡之间正确的同步，所以一个卡不会在另一个卡之前开始启动。如果 I11 设置为零，一旦计算已经完成，第一个移动将立刻启动。



计算随后的移动

只要一个运动通过轨迹产生例程而开始实际执行，一个标志就设置给 Turbo PMAC 来计算所有后续程序语句直至并包括下一步的运动声明，将因此计算的数据放入轨迹发生器的一个队列中。程序计算然后被挂起，直到轨迹发生器开始下一步的运动，而且 Turbo PMAC 执行其他任务（PLC 程序，通信等）。

时间计算不足

如果到一个运动开始执行的时候，Turbo PMAC 尚不能完成该运动的轨迹计算的话，Turbo PMAC 将中止程序，并在其状态字中显示一个运行时错误。这通常发生在移动时间设置得非常短（几毫秒），并且（/或）在移动指令之间有一个

非常大的计算量时。这个移动块计算速度的限制是非常依赖于应用程序，但每秒一般都是几百块。

如果关注这个移动计算限制，那么在开发过程中要使得运动时间持续变短，直到 Turbo PMAC 失败。那么对于最终的应用程序，请确保最小的移动时间大于这个值，通常至少为 25%的安全裕度。

如果没有提前计算

在一个运动程序中会有几个条件停止混合并停止提前计算。在这些情况下，Turbo PMAC 在它开始计算下一个或两个运动之前，会等待，直到那个操作完成。在上述任何中断之间，Turbo PMAC 将使用 I11 计算时间来延迟下一步运动的开始。

DWELL 命令

一个运动程序的 **DWELL** 指令停止了运动的混合，因此在一个 **DWELL** 期间，Turbo PMAC 将不进行计算。Turbo PMAC 不启动后续运动的计算直到 DWELL 时间完成后。相反，一个 **DELAY** 指令实质上是一个指定时间的零距离运动命令；在一个 **DELAY** 期间，Turbo PMAC 会进行计算。

HOME、快速移动

如果从程序内指令一个回零搜索移动（HOMEn）或一个快速模式移动，那么该运动不会与任何其他移动进行混合。Turbo PMAC 不启动后续运动的计算直到所有电机已经完成他们指令的这些类型的动作后。

PSET 命令

如果一个运动程序内使用一个**PSET** 命令以重新定义轴位置，那么 Turbo PMAC 将不会将 PSET 前的运动与 PEST 后的运动进行混合。Turbo PMAC 不会启动后续移动的计算直到前一个指令运动已经结束以及 PSET 命令已被执行后。

两次跳回规则

如果在试图计算出下一步的运动过程中，Turbo PMAC 在程序的逻辑中侦测到两个向后跳跃，那么 PMAC 不会尝试将上次计算出的运动与即将到来的运动进行混合。这些回跳是可以通过 **ENDWHILE** 语句或 GOTO 语句引起的；而 GOSUB、CALL 和 RETURN 跳转在这里是不计算在内的。这一规则的目的是为了防止 Turbo PMAC 如果要对短运动多循环几次而由于计算时间不够而中止程序，。

混合停止

PMAC 让前一个移动停下来，并在下次实时中断时重新开始计算程序（见 I8 描述），一直持续到它找到下一个移动语句，或有两个以上的跳回（该情况下重复该过程）。这允许无限期等待循环，但是不会造成 Turbo PMAC 因为计算时间的不足而中止运动程序。

嵌套循环

该双跳回的规则可能会导致程序员当他们在嵌套的 while 循环内计算移动的时候，无意中停止混合。考虑下面的例子，试图在内环产生一个连续的混合正弦运动，采用外环来索引正弦曲线的大小：

```
SPLINE1 TM20
P1=0
WHILE (P1<10)
        P2=0
```

```
WHILE (P2<360)
        X(P1*SIN(P2))
        P2=P2+1
    ENDWHILE
    P1=P1+1
ENDWHILE
```

前 360 个片段通过 Turbo PMAC 执行内部循环时而将被动态混合（样条）在一起。但是，当 PMAC 递增 P2 到 360 时，它到达第一个 **ENDWHILE** 并跳回到内 WHILE 条件，该条件现在是假的，所以程序跳到下面一句，递增 P1，到达第二个 **ENDWHILE**，并跳回外 WHILE 条件下，所有这些都不会遇到一个移动命令。

在这一点，Turbo PMAC 调用“双跳回”规则，并让上次编程的移动停下来。它这样做是为了防止它可能会陷入一个无限真实的循环集，而没有运动，这可意味着它来不及准备下一步的运动方程。它恢复计算是在当这一运动已完成，并将启动内部循环的下一个序列运动。

为了不断将所有这些运动融合在一起，将内循环的最后一步拉出内环外。这样一来，在两个运动命令之间两个 **ENDWHILE** 语句将不会遇到：

```
SPLINE1 TM20
P1=0
WHILE (P1<10)
    P2=0
    WHILE (P2<359)          ; Note that loop stops earlier
        X(P1*SIN(P2))
        P2=P2+1
    ENDWHILE
    X(P1*SIN(P2))                   ; Last move from inner loop
    P1=P1+1
ENDWHILE
```

循环等待：

有几种方法用于保持程序的执行同时等待发生一定的条件。通常，这是通过一个 WHILE 循环完成的，但在循环内完成对响应速度和计算负荷有影响。

最快的执行是 **WHILE（{条件}）WAIT** 循环。只要遇到 **WAIT** 命令，运动程序计算暂停，直到下一次实时中断，届时他们将重新评估条件。该运动程序有效地变得像一个单行的 PLC 程序。如果下一个 RTI 已经发生，PMAC 会立即重新进入中断服务程序，并重新评估条件。如果此问题反复出现，后台程序将缺少时间，减缓 PLC 和通信，或在最坏的情况下，跳闸看门狗定时器。通常这种情况只发生在多个坐标系是同时在 **WHILE ... WAIT** 循环中时。

具有类似速度的是一个空的 **WHILE... ENDWHILE** 循环，或者里面至少有一个没有运动的命令。每一个 RTI，这将执行两次，由双跳回规则停止。下次 RTI 时计算会恢复，或者如果中断已经发生，他们立即恢复，使得后台计算缺少时间而有可能产生同样的后果。

使用 **WHILE（{条件}）DWELL** 单行循环有助于更好地控制循环率，为后台程序提供时间。每次 DWELL 后，条件只计算一次。

提前计算的影响

在连续运动序列中，运动程序需要提前计算意味着非运动的动作---尤其是输出的设置---被程序执行，在一个或两个运动之前。对于只在程序中使用的变量，这是没有问题的，因为一切都在程序之内相继发生。

有可能将这些非运动动作在程序中向后移动一个或两个动作来使得动作发生在它们被需要时。然而，就正确的操作顺序而言，这使得读取程序非常困难。

同步的 M-变量赋值

同步 M 变量赋值语句被设计来解决这个问题。这种类型的语句使用双等号（==），而不是一个单等号。这对 PMAC 是一个标志，拖延了语句的实际执行，直到紧随其后的下一个运动开始，因此实际的动作与实际运动一致。

同步 M 变量的赋值语句的详细讨论在本使用手册的《计算功能特征》的部分，**M{常量}=={表达式}**的语法说明部分是在程序命令说明中。

第十七章 同步 TURBO PMAC 到外部事件

Turbo PMAC 有几个强大的功能帮助 Turbo PMAC 控制下的运动同步到外部事件。这些措施包括 -

- 位置跟随，俗称电子齿轮；时基控制，俗称电子凸轮
- 位置捕获，可用于定位应用
- 位置比较，其可用于精确扫描和测量应用。

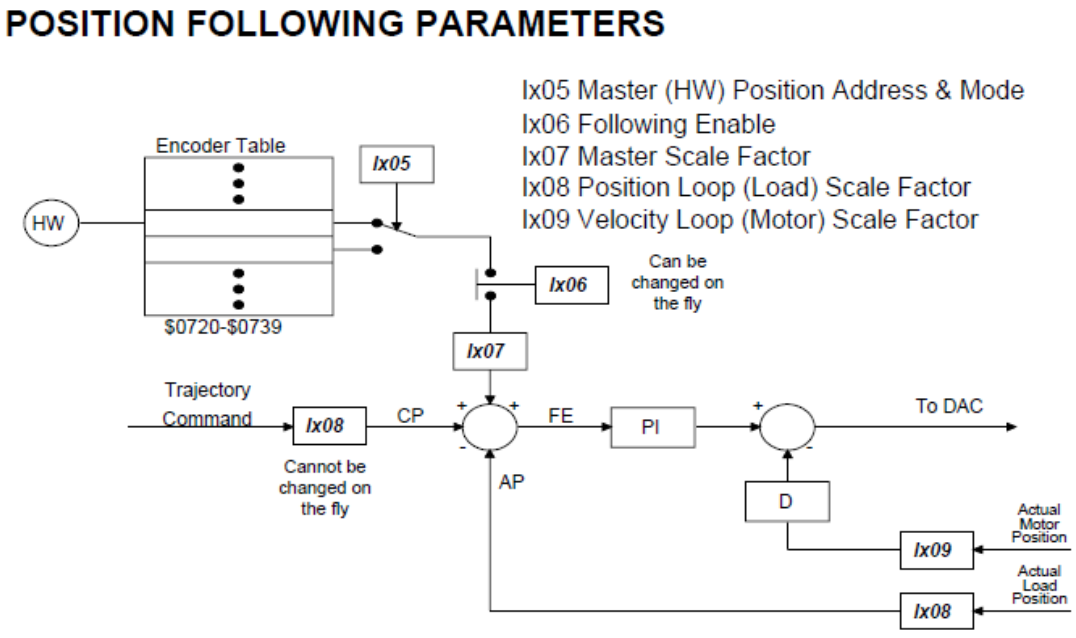
其中的每个部分涵盖在下文中。

17-1 位置跟随（电子齿轮）

Turbo PMAC 有几种方法可以协调其控制下的轴与不在其控制下的轴。最简单的方法是基本位置跟随。这是一个电机对电机的功能，而不是一个如时基跟随的坐标系功能（见下文）。源自主轴的一个编码器信号（其不在 Turbo PMAC 控制下）被送入了 Turbo PMAC 的编码器输入之一。通常，该主信号是源自一个开环驱动器或一个手轮旋钮。

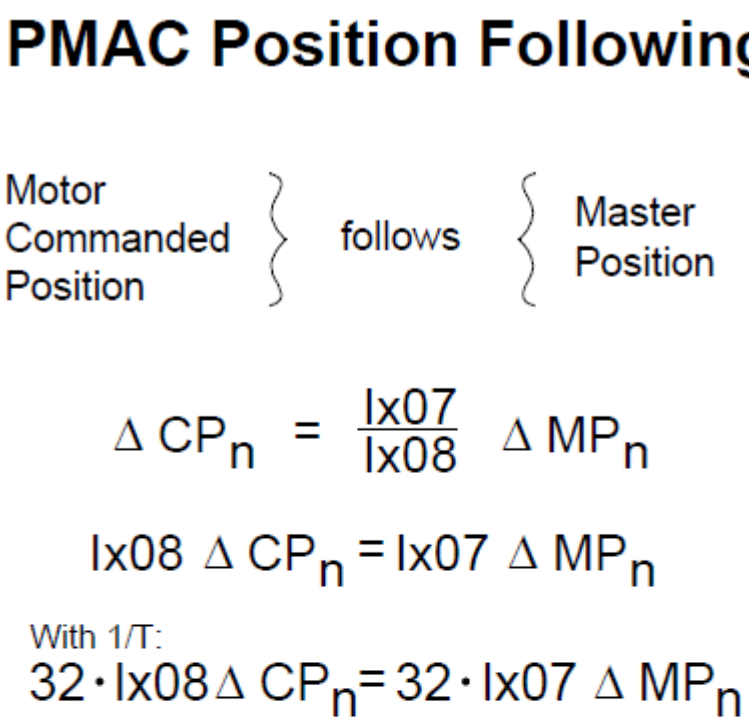
位置跟随 I-变量

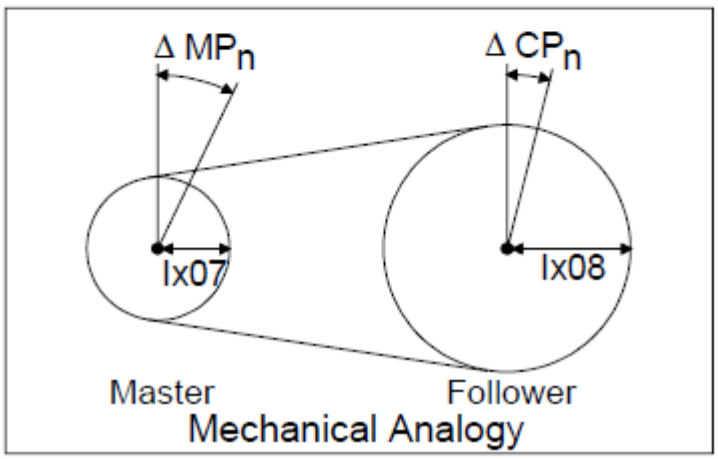
Turbo PMAC 的一个或多个电机被告知该编码器寄存器是主寄存器(用 Ixx05)。Ixx05 是一个地址 I 变量，也就是说，它的值包含了用于保存主位置信息的寄存器地址。通常情况下，该寄存器是编码器转换表中的一些处理过的位置数据。在主编码器的采样中，这可有助于完成 1/ T 或类似于减少量化噪声（这是在默认设置中自动完成的）。



每个电机的跟随比率是由 Ixx07 和 Ixx08 设置的，他们可以被看做机械传动

系统中主从齿轮的齿数，或束带轮的半径。跟随功能被接通和断开是通过 Ixx06 的 0 位控制的。当它开启时，来自主轴的输入流就像一个轨迹发生器，产生了控制跟随轴到达的一系列指令位置。





动态改变比率

要在应用程序的中间改变跟随比率，可以单独改变 Ixx07。由于 Ixx08 涉及了伺服反馈计算的缩放，因此在应用程序的中间不应被改变。

可以在即时变化的分辨率和系统的伺服性能之间进行权衡。Ixx08 比例因子越高，分辨率变化的越精细。然而，Ixx08 越高，比例增益 Ixx30 越低，防止内部的饱和发生，而且最大速度越低防止那些寄存器的内部饱和。在一般情况下，

Ixx08 应保持低于 1000。

在编程移动中叠加跟随

此外，该跟随功能可被叠加在计算轨迹上。例如，在对一个运动的金属薄条进行成型加工时，可以在不考虑薄条运动的条件下编写成型程序，来自薄条的主信号用于补偿运动。

Ixx06 的位 1 确定跟随是发生在偏置方式（其中跟随电机的报道位置并不反映由于跟随产生的变化），还是正常模式下--其中它反映变化。用户叠加跟随和计算移动时通常会希望使用偏置模式，其中位 1 为 1（所以 Ixx06=3 用于激活跟随在偏置方式下）。在正常模式下，计算出的轨迹将消除在每个编程移动或移动段由跟随引入的偏移。

在正常模式下，一个编程移动中计算电动机的指令位置是编程位置减去主位置。报告的位置（响应一个 P 或<Ctrl-P>命令）就是实际位置。在偏置方式，编程模式的电机指令位置就是编程位置。报告的位置是实际的位置减去主位置。

使用级联循环

偏置模式跟随对于级联伺服回路的实施非常有用。外环的输出被用作内位置环的主位置（不被直接发送到执行器）。在这种操作模式下，典型地，内环是一个标准的位置环；外环对力或刀具高度或一些其他过程变量闭合一个环路。使用偏移模式用于此目的允许正常轨迹命令到内部环路，并有可能从外部循环同时更正。细节请参阅级联伺服环路和设置伺服回路的部分。

变更跟随模式

跟随模式用于确定如何执行与电机和轴位置相关的计算，所以改变模式将改变如何完成计算后续的运动，在电机和轴位置之间产生一个不匹配（如果不纠正，在下一步的运动中会导致电机位置有一个跳跃）。无论跟随是否被启用，这都是有可能发生的。为了纠正这种不匹配，在处理下一个运动程序的移动指令之前，必须执行一个 PMATCH 位置匹配命令。

任何时候发出一个 R（运行）或 S（步进）命令，Turbo PMAC 自动执行一个 PMATCH 命令；但是，如果在一个运动程序的中间改变了模式，这个在线命令必须明确地发出。在该命令之前应发出一个 DWELL 指令（来停止程序前瞻）；而在该命令之后要发出另一个 DWELL 命令（给出命令时间执行在线队列）。

程序顺序看起来是这样的：

```
DWELL0                  ; Stop lookahead—停止前瞻
I106=3                  ; Change following mode—改变跟随模式
CMD"&1PMATCH"           ; Re-match positions--重新匹配位置
DWELL10                 ; Delay for PMATCH execution—延迟用于匹配
```

17-2 外部时基控制（电子凸轮）

协调外部轴的一个更复杂的方法是时基控制，在其中用输入信号的频率控制执行运动和程序的速度。时基控制在整个坐标系上进行工作。用户需要指定哪些编码器寄存器接收输入频率，以及输入频率和程序的执行速率之间的关系。这不

仅使得移动速度正比于输入频率（频率始终接近为零），而且还保持所有的位置同步。这允许诸如多道螺纹加工的操作。

什么是时基控制？

Turbo PMAC 的运动语言表达位置轨迹为时间的函数。无论移动是被时间直接指定，或者是被速度指定，轨迹最终被定义为一个位置对时间的函数。

这对于大量的应用是很好的。然而，在许多应用中，我们希望将 Turbo PMAC 轴从属于一个并不在 Turbo PMAC 控制下的外部轴（或有时是不同的坐标系中一个 Turbo PMAC 控制下独立的轴）。在这些应用中，我们要定义 Turbo PMAC 的轨迹作为主位置的函数，而不是时间的函数。

实时输入频率

Turbo PMAC 这样做的方法保留了将位置表达为时间的函数，但是使时间正比于主轴通过的距离。完成这个是通过从主轴的位置传感器上定义一个实时输入频率（RTIF），单位是计数/毫秒。例如，定义一个 32 计数/毫秒的 RTIF。然后，在时基模式下，当一个程序是指一毫秒时，它真正指的是主编码器的 32 个计数，无论那物理距离是多少。如果在从属程序中进行编程一个两秒钟的运动，那么它要真正地完成将花费主编码器的 64,000 计数。

选择 RTIF 时的约束

如果 Turbo PMAC 在它的时基计算中具有无限的分辨率和无限的动态范围，那么实时输入频率的选择将是完全任意的，任何频率可以被选择作为 RTIF，并可以使用该 RTIF 编写运动程序。然而，TurboPMAC 是在整数运算中进行它的时基计算，这就限制了分辨率，而且使用的是 24 位寄存器，从而限制了动态范围。

这些限制导致在 RTIF 的选择时有三种限制：

1、 从 RTIF 导出的时基比例因子（TBSF）必须是一个整数。该 Turbo PMAC 需要用于其计算的值不是计数/毫秒的频率，而是频率的倒数，毫秒/计数。为了让这个数字在整数值的范围，规则是由 217（131,072）乘以频率的倒数。

如果选择一个 100 计数/毫秒的值用于 RTIF，那么 TBSF 将是 131072/100 =131.072，这不是一个整数。Turbo PMAC 可以接受的只有 131 的整数部分，并会发生漂移。

选择一个是 2 的幂的实时输入频率，计数/毫秒（如 32，64，128），总是会产生一个整数 TBSF。此外，RTIF 值等于 2 的幂除以一个整数通常也是可以的。例如，204.8 计数/毫秒（= 2048/10，$=2^{11}/10$）将产生一个 TBSF 为 $2^{17}/2^{11}*10=640$。

2、 时基计算将饱和在一个输入频率（IF），其中 IF/RTIF 等于伺服更新频率，单位是 kHz。在 2.25 kHz 的缺省伺服更新频率和 32 计数/毫秒的 RTIF，可以接受的不饱和的最大输入频率是 32*2.25= 72 计数/毫秒。如果该系统可以工作到 100 计数/毫秒，选择 RTIF=32 计数/毫秒是不能接受的，但选择 RTIF=64 计数/毫秒将是可接受的（100/64 =1.5625<2.25）。

选择 RTIF 大于最大输入频率始终是可以接受的。

时基控制如何工作

时基控制的工作原理是对（每个伺服周期所发生的）指令位置更新方程关于自上次伺服周期以来所经过的时间量“说谎”。(变量 I10 包含了时间的实际数量。)伺服周期之间的实际时间不会改变，伺服回路的动态性能也没有变化。只是指令轨迹的速率随该外部频率改变，并且由于坐标系所有的轨迹一起变化，所以通过空间的路径没有改变。

使用外部时基信号的说明

使用外部时基信号需要几个步骤来设置。然而，一旦设置完成后，时基控制对用户和程序是透明的- 它是自动的。详细的典型的设置步骤如下。

步骤 1：信号解码

输入到 Turbo PMAC 的信号是一个增量编码器输入（信道 A 和 B)。该信号必须是正交信号（如一个编码器的输出）或脉冲和方向信号（脉冲进入 A，方向进入 B)。对于所使用的编码器输入，变量 I7mn0 控制解码方法，并且定义了什么是一个“计数”。例如，对于一个正交信号，I7mn0= 3 或 7 定义每个编码器周期是 4 个计数，而 I7mn0= 2 或 6 定义每个编码器周期只有 2 个计数。在 3 或 7 和 2 或 6 之间的区别是解码器用信号的哪个方向传感进行向上计数。

注意：

请确保它是在主信号进行的方向向上计数---向下计数将意味着一个负的时基，而这是 Turbo PMAC 无法处理的。

步骤 2：插值

一旦解码和计数后，在每个伺服周期，来自该信号的值被输入编码器转换表一次，就像一个位置反馈信号那样。这里强烈推荐采用 1 / T 的转换方法，因为这个方法给出信号的一个很好子计数插值（使用与计数器相关联的定时器)，它显著提高了时基信息的平滑度。确保该转换表被设置为以这种方式处理输入信号的计数器。编码器转换表的出厂默认配置是当前编码器计数器的 1/ T 转换。更多细节参见编码器转换表的描述。

步骤 3：时基计算

编码器转换表中的一个入口从上一步中拾取插补的位置信息，减去前一个伺服周期的插补位置信息，并将该差值乘以一个比例因子，以产生用于伺服周期的时基值。(这个时基值是该位置更新计算中的一个乘法因子，所以更新量是成正比于从上个伺服周期时基信号接收的计数数目。)

在此步骤中的 2 个设置项是信息源（插补位置寄存器）和比例因子。这两者都是编码器转换表中的入口（I-变量)。关于如何输入这些，详细信息请参阅表的描述。

该时基转换的方程是：

$$\%value = (100.0 * SCALE_FACTOR * INPUT_FREQ)/2^N$$

其中的%值（也称为进给倍率修调值）控制位置更新率---当它等于 100.0

时，程序和运动以“真实时间”进行工作（即，以程序中指定的时间和速度）。

SCALE_FACTOR 是整数值，必须确定大小来适当设置时基跟随。 INPUT_FREQ 是计数率（由信号和 I7mn0 确定），单位是计数/毫秒。对于正常（未触发）时基，N 等于 17（2^{17} 是 131,072）；对于触发的时基，N 为 14（2^{14} 为 16,384）---细节见下文的触发的时基。

要设定比例因子，决定一个实时输入计数频率---输入计数的速率，程序和运动应以规定的速率执行。由于值将是%100.0，解出比例因子为：

$$\text{SCALE_FACTOR} = 2^{N}/(\text{REAL_TIME_INPUT_FREQ})$$

由于比例因子必须是整数，并且我们分频成 2 的幂，使实时输入频率是 2 的幂，单位是计数/毫秒。例如，如果有一个系统，其中典型的全速输入计数频率是 60,000 计数/秒，定义实时输入频率为 64 计数/毫秒。那么这将使得未触发时基的比例因子为 131,072/64=2,048。

到目前为止，我们所拥有的是寄存器中一个正比于主频率的值。现在我们必须使用这个值，以控制我们的运动程序。

第 4 步：使用时基的计算

时基值只在一个坐标系起作用。每个坐标系都有一个 I 变量，告诉它在哪里寻找它的时基信息。对于坐标系 x 这个变量是 Isx93。Isx93 的默认值是软件控制下的寄存器的地址，不是在一个外部频率控制下。对于您希望成为外部时基控制下的一个坐标系，你必须把上文确定的缩放时基值的地址放在编码器转换表中。

例如，如果表的开始是 8 个 1/T 入口（使用 I8000 – I8007），随后的时基入口使用 I8008 和 I8009，所得到的时基值将是在 X 寄存器中以及 I8009（地址 X:$350A）。要使用此值作为坐标系 1 的时基，发出命令 **I5193=$350A**（直接指定地址），或 **I5193=@ I8009**（通过 I-变量的位置指定地址）。这两个命令产生相同的结果。

一旦该 I-变量已经被设置，对于编程和非编程的运动，分配给该坐标系统的所有电机将是在外部频率的控制下。

I 变量 Isx94 控制坐标系 x 的时基值变化的最大速率。当从主机发出时基值时（用一个**%n** 命令），这个值应该是相当低的，使得时基转换比较平稳。然而，为了保持同步至一个时基源的外部信号，此值应设置尽可能高（最大值为 8,388,607），这样时基的变化与信号的变化一样快。设定值低可以提高时基跟随的平滑，代价是在跟随中有一些‘滑动’。如果在外部时基中使用 Isx94 限制，将不能保证与主轴的位置同步。

第 5 步：编写程序

当编写外部时基控制下的程序时，编写它，好像输入信号总是在实时频率那样。在运行时，程序执行的速度正比于输入频率。对于移动时间和所指定的进给率，有全浮点分辨率。

记住，DWELL 命令总是在实时执行，无论输入频率是什么。要在正比于输入频率的程序中进行停顿，使用 DELAY 命令，而不是 DWELL。

时基示例

如果有一个卷筒以每秒 50 英寸的标称速度移动，在卷筒上有一个正交编码

器，分辨率是每英寸给出 500 线。有一个横切轴在 Turbo PMAC 控制下。当卷筒正在以额定速度运动时，在 0.75 秒进行一个切削运动， 2.50 秒之后准备开始又一个运动。卷筒编码器连接到伺服 IC2 编码器 4 的输入线上。

步骤 1：信号解码

因为卷筒编码器是伺服 IC2 编码器 4，所以 I7240 控制解码。为了获得最大的分辨率，设置 I7240 为 3 或 7，得到 x4 解码。先试试 3。在手册中的建议 M 变量的列表中寻找，注意这个编码器的编码器位置 M 变量，对于 UMAC 是 M401，对于其他类型的 Turbo PMAC 是 M1201。定义 M401 或 M1201，并反复查询它的值（可能使用执行程序监视窗口）同时在它进行的方向上调节卷筒编码器。如果随着编码器调节，该值会增加，那么 I7240 的设置正确。如果它减少，I7240 改变为 7。（如果它不改变，请检查连接。）

步骤 2：插值

接下来，看看编码器转换表的当前设置。确保编码器是通过 1/ T 转换进行处理。做到这一点的最简单的方法就是通过 PMAC 执行程序的配置菜单。如果这是不可用，直接看 I8000+变量。发出如 **I8000..8009** 的命令，并返回这样：

```
$078200
$078208
$078210
$078218
$078300
$078308
$078310
$078318
$0
$0
```

参照该软件参考手册中 I8000 变量的详细描述，请注意，此表采用 1/ T 扩展对伺服 IC 2 和 3 的所有 4 个编码器信道进行处理，所以主编码器被处理。

步骤 3：时基计算

现在在表中的一个入口设置来将插补位置转换到时基格式。看上面报告的值，无论是在这里显示的原始形式，或者在执行程序的配置窗口更明确地显示，注意没有时基入口，因此必须创建一个。

源编码器是处理在表的第四行，通过 I8003。查找这个入口的地址，请注意是在$3504。如果使用的是执行程序的配置菜单，添加一个入口到表的末尾，选择时基的方法，然后输入地址---无论是从选择列表中选择或输入地址（$3504）。直接与 Turbo PMAC 通讯，发送命令 **I8008=$403504**（初始 4 指定一个时基入口）。

现在计算缩放因子。看标称速度为 50 英寸/秒的，分辨率 500 为线/英寸和 X4 解码，并计算：

$$50\ 英寸/秒 * 500\ 周期/英寸 * 4\ 计数/周期 = 100{,}000\ 计数/秒$$

$$= 100\ 计数/毫秒$$

由于如果这个数字是 2 的幂，数学工作更为简单，声明实时“计数速率为 128 计数/毫秒。然后计算比例因子为 131,072/128=1024。如果使用的是执行程序的配置菜单，输入该值到正确的地方。如果直接通讯到 Turbo PMAC，发送 I8009=1024 命令。

第 4 步：使用时基计算

由于工作在坐标系 1，将此入口的第二行的地址赋值给 I5193（无论是通过 I5193=@ I8009 还是 I5193=$350A），以指向这个时基值。设置 I5194 为最大值 8,388,607，以便快速变化时也保持同步。

第 5 步：编写程序

在编写程序时，要工作在实时输入频率，它不同于开始的额定速度---在这种情况下，正好快 28%。因此，任何编程的速度将提高 28%；任何编程的时间会少 28%。取 750 毫秒（0.75 秒）的名义切割时间并乘以 100/128,正好得到 585.9375 毫秒。2500 毫秒返回也同样缩放为 1953.125 毫秒。（如果这些数字并不能在程序中完全给出来，直接放在程序中进行计算；Turbo PMAC 是以 48 位浮点精度进行计算。）会有一个主程序循环，例如：

```
WHILE (M11=1)          ; Cut as long as input is true
TM 585.9375            ; Cut move time
X10000                 ; Actual cut move
DELAY 500              ; Hold; part of 1953.125 msec return
TM 953.125             ; Return time; part of 1953.125 msec
X0                     ; Actual return move
DELAY 500              ; Hold; part of 1953.125 msec return
ENDWHILE
```

触发的时基

讨论的时基技术至今保持从属坐标系统与主坐标系保持完美地同步，但他们没有提供一种方式与主坐标系上的一个特定点保持同步。因此，从属周期与与主周期产生相位差。必须使用一些特殊的技术，通常涉及从一个定位标志得到位置信息，用于使得周期彼此同相。

许多时基应用程序不需要主从坐标系的周期是彼此同相（例如，切割空白纸的长度大于打印的页面），而其他由于拉伸、滑脱或不均匀的间距必须不断地重新定位。这些类型的应用程序可以使用标准的时基功能。

然而，有些应用确实需要与主坐标系保持同相位，而其中的定位程序要做到这一点是困难的或不可能的，这些应用程序可以使用转换表的触发时基功能。通过在触发之前冻结时基，然后参考那个触发捕获的位置来启动时基，这种技术允许与触发捕获的主坐标位置保持完美同步。

在转换表中触发时基的入口是类似于标准的时基入口。它是一个两行入口，在第一行中指定主编码器数据的处理过程和源地址，并且第二行指定时基的比例因子。在触发时基的入口和标准时基入口之间有两个重要的区别。首先，指定处理过程的该值是不同的，而且它在触发的过程中被改变了（$90、$A0 和 B0$，标准时基为$40）。第二，源地址是实际的主编码器计数寄存器的，而不是该转换表中处理过的编码器数据。比例因子与标准时基的是相同的。对于这个入口的规则

在转换表的说明中进行了详细的讨论。

关于触发时基的说明

使用触发时基功能包括正确设置 I-变量值、M 变量的定义和转换表入口（这些是可以提前做到），编写运动程序，并编写 PLC 程序。所有这些是依次覆盖在下面。

第 1 步：信号解码设置

主信号的信号解码与标准时基是一样：正交或脉冲和方向信号必须被解码，以使计数器向上计数。这是通过 I7mn0 设置的。

第 2 步：插值和时基设置

编码器转换表中的触发时基的转换同时处理插补值的 1/T 计数插值和时基计算。在该方法中，1/T 插补产生三个附加的分数计数位（总共 8 个），从而对于给定的计数使所得到的数值比“正常的”1/T 插值大 8 倍。在初始设置，触发时基项目是在转换表中创建的，通常在运行状态（不是冻结或等待触发）。时基的比例因子也是此处输入的，它的计算方式与标准时基是相同的，不同之处在于它对于相同的实时输入频率只有 1/8 大。

第 3 步：编写运动程序

在编写使用触发时基的运动程序时，所有轴必须停止在他们等待触发的那点。如果这不是在运动的开始，这一部分通过一个 **DWELL** 指令应立即放在前面。

在开始计算在触发时启动的运动时，时基应被冻结，以防止移动启动。这个最好是通过使用已分配给转换表中触发时基入口的进程位的 M 变量完成，。如果以前的运动是通过另一个时基源完成的，那么坐标系统的时基地址---Ix93---应改为触发时基入口。

这些运动程序中的命令的后面紧接着是触发器触发的第一个运动的计算和命令。时基被冻结时，Turbo PMAC 将执行所有的计算，但不启动这些运动的实际执行。变量 I11（计算延迟）应设置为 0，那么一旦时基启动，Turbo PMAC 将准备启动运动。

第 4 步：准备触发

计算运动的运动程序本身无法准备触发，而在计算完成之前触发不可能发生。如果发生这种情况，该程序将落后于所需的同步。因此，为了运行可靠，触发应由一个直到所有的移动计算完成才能执行的任务准备，通常是一个 PLC 程序。在 PLC 程序中，准备触发只是需要的一个简单的条件分支，它只是看时基是否被冻结，如果是，PLC 程序会准备触发。由于 PLC 程序不能中断运动程序，这就保证发生在运动程序已经完成计算移动后。

第 5 步：开始触发

一旦触发被准备完毕，Turbo PMAC 等待主编码器的位置捕捉触发发生。变量 I7mn2 和 I7mn33 确定哪些信号的哪些边缘引起触发。当 Turbo PMAC 看到触发已发生时，它启动时基，使用捕获的主位置作为时基的起点。

触发的时基示例

电机＃1 是坐标系 1 的 A 轴。它是一个旋转轴，有一个每转 2500 线的编码器在电动机上，电动机到负载是 3 比 1 的减速比。它应从属于 Turbo PMAC 的伺服 IC2 编码器 4。主编码器每转 4096 线，并且通常旋转在约 600rpm。在接收到一个执行的命令后，X 轴必须等待主编码器的索引脉冲和落后它 45 度。在接下来的主编码器的 36 度，就必须加速，然后在主编码器的 144 度内匀速运行，最后在主编码器的 36 度内减速停止。此运动必须在 A 轴的一整圈内完成。

我们将使用触发时基，由主编码器的索引脉冲触发。选择 600rpm 作为我们的主轴的“实时”速度，我们计算我们的实时输入频率（RTIF）--计数/毫秒：

$$600\text{rev/min} * \left(\frac{\text{min}}{60\text{s}}\right) * \left(\frac{4096\text{lines}}{\text{rev}}\right) * \left(\frac{4\text{cts}}{\text{line}}\right) * \left(\frac{\text{s}}{1000\text{ms}}\right) = 163.84 \text{ 计数/毫秒}$$

时基的比例因子（SF）是：

$$\text{SF} = \frac{131{,}072}{\text{RTIF}} = \frac{131{,}072}{163.84} = 800(\text{十进制})$$

在 600rpm（10 rps）的实时速度，主轴的一转需要 100 毫秒，所以主轴的 45 度需要 12.5 毫秒，以此类推。

设置和定义

```
I7240=3                 ; x4 decode of IC 2 Enc 4, set to count up in direction of motion
I7242=1                 ; ENC 4 capture trigger on rising edge of index pulse
I8008=$A78218           ; Add triggered time base entry to end of default conversion table
                        ; process $A is triggered time-base, running (post-trigger); $78218
                        ; points to IC 2 Enc 4 registers
I8009=800               ; Scale factor is 800 decimal Result is at address $350A
M199->Y:$3509,20,4      ; Method digit of conversion table entry =$9 frozen, $B armed,
                        ;$A running
&1                      ; Address Coordinate System 1
#1->83.33333333A        ; Motor 1 is A-axis in C.S. 1; 3 x 2500 x 4 cts/rev / (360 deg rev)
```

运动程序

运动程序冻结了时基，并计算第一个运动。但这一运动的实际执行不会发生，直到时基被触发。

```
CLOSE
OPEN PROG 12 CLEAR
I5193=$350A             ; Time base source address is triggered time-base conversion in table
                        ; (2nd line)
DWELL0                  ; Stop lookahead in program
M199=$9                 ; Freeze time-base
LINEAR                  ; Linear move mode
INC                     ; Incremental move specification
TA10                    ; 36 degrees of master is 10 msec
TS0                     ; No S-curve
DELAY12.5               ; 45 degrees of master is 12.5 msec
TM50                    ; 36+144 deg of master is 50 msec
A360                    ; One full revolution of slave axis
CLOSE
```

PLC 程序

PLC 程序只是看是否时基已被冻结，如果是的话，它准备时基。在准备状态下，触发时基转换表入口在每个伺服周期都查找触发。

```
CLOSE
OPEN PLC 10 CLEAR
IF (M199=$9)                    ; Has time-base been frozen?
   M199=$B                      ; Then arm for trigger
ENDIF
CLOSE
```

17-3 Turbo PMAC 与其他的 Turbo PMAC 同步

当多个 Turbo PMAC 一起使用时，卡之间是通过将伺服时钟信号从第一卡传递到其他卡来维护同步的。通过精心的编写程序，该功能允许完全协调不同卡的轴。

Turbo PMAC 提供了能力可以将多个卡放在单个应用程序中。要使得卡以协调的方式妥善工作，有几个因素必须考虑：

- 主机通信寻址
- 时钟定时
- 运动程序计时

主机通信寻址是覆盖在本手册的与 Turbo PMAC 通讯和编写一个主机通信程序的部分。定时（同步）的问题涵盖如下。

时钟定时

Turbo PMAC 卡使用一个晶体时钟振荡器（主时钟）作为基本时间测量装置。每个 Turbo PMAC 都有自己的晶体振荡器。虽然这些晶体的制造是一个非常严格的公差（50 ppm 的精度标准，选项 8 的是 10ppm），但是他们在卡与卡之间是不完全一样的。实际控制运动定时的相位和伺服时钟信号是从晶体时钟频率产生的，所以具有相同的误差。如果他们使用的是其各自的主时钟，这可能会导致在长运动序列中卡之间彼此失去同步性。通常，这只有在一个连续的移动序列持续时间超过 10 分钟时才是明显的。例如，在最坏的情况下，两个卡之间是 100ppm 的差异，在经过 10 分钟的连续运动后，会相差 60 毫秒。

在 MACRO 环上同步时钟信号

如果要在一个共同的 MACRO 环使得多个 Turbo PMAC2 保持同步，那么同步会自动在环上实现。在这种情况下，每个 Turbo PMAC2 使用 MACRO IC0 生成其自己的时钟信号（这样 I6800、I6801、I6802 控制频率），但通过使用“同步数据包”在 MACRO 环上传递而将这些信号强制到完全同步。I6800、I6801 和 I6802 应设

置为获取所有控制器相同的标称频率，所以任何的同步修正很小，并且不影响性能。

MACRO 环上的一个 Turbo PMAC2 必须设置为环的主控制器。这将迫使环上的其他设备的时钟信号为完全同步。此控制器的 I6840 必须设置为$xx30。环上其他的 Turbo PMAC2 都是主控，但不是环的主控。这些控制器的 I6840 必须设置为$xx90。

一般情况下，从环的主控发送的节点 15 数据包是同步数据包。该控制器上的 I6841 的 15 位必须被设置为 1，以使能发送该数据包。在其它控制器，I6841 的 16 至 19 位（第二个十六进制数字）都必须设置为 1（使第二十六进制数字为$F 或 15）来指定这个作为同步数据包， 而且 I6840 的 15 位必须设置为 1（例如：I6840=$8090）以使能控制器从另一个主控上接受这个数据包。

共享串行端口的时钟信号

如果要同步的 Turbo PMAC 未连接在一个共同的 MACRO 环上，解决这个问题的方法是使得所有的卡物理上共享一个公共的相位和伺服时钟信号。对于 Turbo PMAC，这是通过主串行端口的备用线（仅适用于 RS-422 连接器）完成的。多个 Turbo PMAC 的串行端口之间的一个直跨扁平电缆将这些卡的时钟信号直接连接起来。将卡之间的串行通信信号连接同一条电缆上是可以接受的，但不是必需的。将时钟信号和/或串行通信信号连接到同一电缆上的主计算机是可以接受的，但不是必要的。

每个 Turbo PMAC 系统的跳线或一组跳线控制该 Turbo PMAC 是会从自己的晶振频率生成自己的相位和伺服时钟信号并输出它们在主串行端口上，还是会预计通过其串行端口从外部来源收到一个相位和伺服时钟信号。需要注意的是一个 Turbo PMAC 设置为接收外部时钟信号，如果它没有接收到这些信号，将立即失败，而且看门狗定时器跳闸。

在一个板级的 Turbo PMAC 上，基-板的跳线 E40-E43 都必须为 ON 用于产生自己的时钟信号。如果其中任何一个被删除，它会通过串口期待它的外部时钟信号。在一个板级的 Turbo PMAC2 上，基-板跳线 E1 必须为 OFF 才能产生自己的时钟信号；如果 E1 为 ON 时，它会通过串口期待它的时钟信号。

在一个 UMAC Turbo CPU 板，跳线 E1A 必须连接引脚 1 和 2，E1B 必须连接针脚 2 和 3，用于 UMAC 产生自己的时钟信号。如果 E1A 连接引脚 2 和 3，E1B 连接引脚 1 和 2，它会希望通过串口接收时钟信号并将它们传递到 UBUS 背板上。

那些共享时钟信号的 Turbo PMAC 中只有一个可以设置以产生其自身的相位和伺服时钟。所有其他的必须设置为输入这些时钟信号。当然，Turbo PMAC 上只有产生时钟信号的的时钟频率控制跳线或 I-变量才是真正重要。然而，设置其他的 Turbo PMAC 为相同的标称频率是一个好主意。如果在系统中任何 Turbo PMAC 上生成 PWM 信号，PWM 频率被限制为 N *PhaseFreq/ 2，其中 N 是一个正整数。

该 I10 伺服周期参数在所有的这些 Turbo PMAC 上必须设置为相同的值来维持精确的同步。

与外部时基同步

如果在一个应用程序中，几个卡的轴被连接到一个外部频率时基上，并需要同步，那么所有卡的编码器计数器必须输入同一个频率信号。如果当内部时基时，也不需要具有完全的同步，就没有必要将 Turbo PMAC 的时钟信号绑在一起，因为外部频率将提供共同的时钟。

运动程序定时

不管卡是否共享有一个共同的时钟信号，多卡之间移动的同步只是受每张卡的运动程序中指定的时间控制。如果一张卡被告知做一个 3-秒长的移动，另一个做 4 秒长的移动，而且他们在同一时间开始，很明显，他们将无法一起完成。因此，几个卡为了同时运行的运动程序必须得小心编写，以便花费相同的时间量进行移动。

初始计算延迟

在收到一个 Run 运行或 Step 步进的命令后，Turbo PMAC 需要一些初始计算时间---通常为几毫秒，才可以开始第一个运动。如果几个 Turbo PMAC 被告知要同时启动一个程序，卡将一般不会花费相同的时间量来计算他们的第一个运动。如果每个卡在完成计算后立刻开始了它的第一个运动，在卡之间会损失掉同步性。Turbo PMAC 参数 I11 （运动程序计算延迟）的存在是为了防止这个问题。它决定在收到 Run 或 Step 命令和开始第一个运动之间的毫秒数。为了同步的需要，在所有卡上 I11 应该被设置为相同的值；默认值为 10（= 10 毫秒的延迟）可以在几乎所有的应用中使用。（如果 I11 被设置为 0 ，计算完成后第一个移动立即开始。典型地，在单卡的应用程序中这是 OK 的，而在多卡的应用程序就不是了。）

移动的指定时间

一般来说，这些程序中的移动应该由移动时间（TM、TA 和 TS）来指定，而不是由进给速度(F)。一个指定进给速度的移动的时间被计算为所有进给轴(FRAX)的矢量距离除以进给速度。很难确保这些不同卡的运动花费相同的时间量。

DWELL 是一个非同步运动，并且在编写多卡应用程序时不应使用。使用 DELAY 命令用来维护程序的同步。

无漂移的条件

如果运动程序是精心编写的，使用指定时间的运动，而且卡共享共同的时钟信号，那么它们可以无限期地运行而且卡之间无漂移。即使有一个同时的命令，当他们开始他们的运动程序时，卡之间仍然有一个初始的不超过几毫秒的偏移，但如果正确的编写运动程序并共享时钟信号，该偏移量将不会增加。以下部分介绍了如何最大限度地减少（通常消除）这个偏移。

最小化初始偏移

如果 PLC 程序未被启用，Turbo PMAC 卡被告诉同时启动程序时，它通常会在同一个伺服周期执行程序。不能在相同的伺服周期启动的程序将在下一个的实时中断时启动。这将是((I8+1)*伺服周期长）μS 以后。如果启用了 PLC 程序，卡之间的起始偏移量可以等于运行和编译最长的 PLC 所需的时间量。消除初始偏移执行的一个很好的方法如下：

- 初始化所有的 Turbo PMAC 卡的所有程序计数器。对于每张卡上的同名的同一个程序的简单情况，输入**@@ B1<CR>**。更复杂的情况可能是**@0B1@1&3B2<CR>**。
- 使用<CTRL-D>禁用所有的 PLC 程序。这将为一个命令给予最快的响应。
- I8 设置为 0，这迫使在每个伺服周期有一个实时中断。如果没有运行一个 PLC0，将此永久性地保留为零。
- 在简单的情况下，如果**@@**已发出，使用 R<CR>开始你的程序，或者使用<CTRL-R>开始程序，这是全局运行命令。
- 要运行一个 PLC0，设置 I8 恢复到原来的值，通常是 2。保留 I8 为 0 可

能会导致 PLC0 占用台任务的处理器时间，造成通讯中断，甚至一个看门狗定时器故障。

- 启用所需的 PLC 程序。在每个运动程序的第一行启用其各自 Turbo PMAC 卡的 PLC 程序。

示例：

```
OPEN PROG 1 CLEAR
ENA PLC 1..31
TM 1000
```

17-4 硬件位置捕捉功能

硬件位置捕获功能将一个外部事件发生时的当前编码器位置锁存进入一个特殊的寄存器。它是在硬件中完全执行，而不需要软件干预（虽然它在软件中被设置，并且稍后被服务）。这意味着，捕获唯一的延迟是硬件门延迟（在任何机械系统可以忽略不计），因此这提供了一个令人难以置信的准确捕捉功能。准确度不是由伺服更新速率限制的，因为该位置可以在伺服周期中的任何时间被捕获。

Turbo PMAC 具有高级的移动--直到--触发结构，可以利用硬件位置捕获功能，去自动结束一个指令运动停止在相对捕获的反馈触发位置一个精确的距离处。触发时基役使功能使用主位置数据的硬件捕获功能来为跟随提供一个精确的起点参考点。也可以创建你自己的低级别的算法使用硬件捕捉功能用于其他目的。

硬件捕获需求

一个 Turbo PMAC 伺服 IC 的硬件位置捕捉功能，在遇到一个标志和/或索引通道输入预先定义的变化时，锁存编码器计数器的值。请注意为了使得位置数据有这个硬件捕获能力，它必须通过一个伺服芯片的编码器计数进行处理（虽然传感器不必是一个编码器---许多旋变-数字转换器产生模拟的正交，它们的计数可以这种方式被捕获）。并行格式和模拟反馈不支持此硬件即时捕捉（他们最快的是到最近的伺服周期捕捉）。

PMAC 类型和 PMAC2 式伺服芯片都支持硬件位置捕获功能。PMAC2 式伺服集成电路的最新修订版（“D”版本和更高版本）可以选择性地支持捕捉估计的子数位置。

一个给定信道的编码器的位置计数器只能通过使用该信道的输入信号被捕获。为同时捕获多个位置，将触发信号接线到每一个要捕获其位置的通道的输入。

设置触发条件

对于自动和手动两种用途，变量 I7mn2 和 I7mn3 确定捕捉的触发条件是什么。对于 PMAC 式和 PMAC2 式伺服集成电路，也是这样。I7mn2 指定是该编码器的通道索引被用来触发，还是使用信道的一个输入标志来触发，或两者都有；以及指定这些信号的哪些边缘会引起触发。I7mn3 指定通道的四个主要输入标志的哪一个是用于触发（如果 I7mn2 指定使用一个标志触发的话）。

通过设置软件用于硬件捕获，Turbo PMAC 得到两个世界的最好性能：软件配置的灵活性，再加上硬件捕获的即时性和准确性。也可以设置一个用于回零，另一个用于探测或定位，例如，两个都有硬件捕获的高精度，而且变化时不需要重新接线。

自动移动--直到--触发功能

Turbo PMAC 有三种类型的移动--直到--触发功能，可以使用硬件位置捕获功能来锁存触发位置：

1、回零搜索运动（在线或在一个运动程序中）

2、在线慢跑（手动控制）--直到--触发运动

3、运动程序快速模式的移动--直到--触发

基本上所有这些创建同一类型的运动。所有这些都将他们的后触发移动结束在相对触发位置的预定距离处。这些运动详细描述在本手册的《执行电机基本移动》的部分。

手动使用捕捉功能

如果自动移动--直到--触发的功能没有完成所需要的操作，可以通过直接访问捕获寄存器来创建自定义函数。

位置捕捉寄存器和标志

一个 Turbo PMAC 伺服 IC 的每个通道都有位置捕捉寄存器和一个位置捕获标志。

位置捕捉标志

每个通道在通道的状态字中都有一个位置捕获标志。在 PMAC 式伺服集成电路，这是位 17；在 PMAC2 式伺服集成电路，这是位 11。当触发条件发生而且位置被锁存到捕获寄存器时，该位由 IC 的捕获逻辑设置为 1。当（全计数）捕获位置寄存器由处理器读取时，它被设置为 0。读取该寄存器的行为会自动复位并重新准备捕获逻辑。

注意：

在 PMAC 式伺服芯片中的捕获电路是边沿触发。在捕获逻辑通过读取位置捕获寄存器而被复位后，在下一个捕捉出现之前，触发输入必须创建另一个边缘到捕获状态。与此相反，在 PMAC2 式伺服集成电路的捕获电路是电平触发的。如果当捕获逻辑复位时，触发输入是仍然在捕捉状态，将立即发生另一次捕捉。

对于 PMAC 式伺服集成电路，用于位置捕获标志的 M 变量定义列于下表中。随着通道标准分配到电机，电机的标志的建议的 M 变量 xx 是 Mxx17。

Servo IC #	Channel 1	Channel 2	Channel 3	Channel 4
0	X:$078000,17	X:$078004,17	X:$078008,17	X:$07800C,17
1	X:$078100,17	X:$078104,17	X:$078108,17	X:$07810C,17
2	X:$078200,17	X:$078204,17	X:$078208,17	X:$07820C,17
3	X:$078300,17	X:$078304,17	X:$078308,17	X:$07830C,17
4	X:$079000,17	X:$079004,17	X:$079008,17	X:$07900C,17
5	X:$079100,17	X:$079104,17	X:$079108,17	X:$07910C,17
6	X:$07A200,17	X:$07A204,17	X:$07A208,17	X:$07A20C,17
7	X:$07A300,17	X:$07A304,17	X:$07A308,17	X:$07A30C,17
8	X:$07B200,17	X:$07B204,17	X:$07B208,17	X:$07B20C,17
9	X:$07B300,17	X:$07B304,17	X:$07B308,17	X:$07B30C,17

对于 PMAC2 式伺服集成电路，用于位置捕获标志的 M 变量定义列于下表中。随着通道到电机的标准分配，电机 xx 的标志的建议的 M 变量是 Mxx17。

Servo IC #	Channel 1	Channel 2	Channel 3	Channel 4
0	X:$078000,11	X:$078008,11	X:$078010,11	X:$078018,11
1	X:$078100,11	X:$078108,11	X:$078110,11	X:$078118,11
2	X:$078200,11	X:$078208,11	X:$078210,11	X:$078218,11
3	X:$078300,11	X:$078308,11	X:$078310,11	X:$078318,11
4	X:$079000,11	X:$079008,11	X:$079010,11	X:$079018,11
5	X:$079100,11	X:$079108,11	X:$079110,11	X:$079118,11
6	X:$07A200,11	X:$07A208,11	X:$07A210,11	X:$07A218,11
7	X:$07A300,11	X:$07A308,11	X:$07A310,11	X:$07A318,11
8	X:$07B200,11	X:$07B208,11	X:$07B210,11	X:$07B218,11
9	X:$07B300,11	X:$07B308,11	X:$07B310,11	X:$07B318,11

如果最近的位置捕获使用编码器的索引（I7mn2 的 0 位=1）门控到一个单正交状态宽（I7mn4=1），那么 PMAC2 式伺服集成电路上的相同的状态字的位 10 也被设置为 1。这表明，所捕获的位置是适合用于查看自上次门控-索引捕获以来是否有正确的计数数量，一个良好的安全性检查编码器计数损失。

全计数位置-捕捉寄存器

位置捕获寄存器为编码器通道包含了上次触发发生时编码器计数器的值。这是一个 24 位寄存器，单位是计数，通常被视为一个有符号的数量。这个寄存器的建议的 M 变量是电机 xx 的 Mxx03（编码器以标准顺序分配给电机）。

这里的一个计数是一个硬件计数，就是硬件编码器计数器的一个增量。这往往但不一定，与一个软件计数相同（是电机软件考虑的一个计数）。在数字正交反馈中（其中电机得到的反馈位置是有 5 位估计或测量分数---一个计数的 1/32---硬件和软件计数将是相同的。

然而，在某些情况下，对于一个带 Acc- 51 的正弦编码器的高分辨率插值，其中是每个硬件计数被计算 10 位分数，那么硬件计数和软件计数的分辨率是不

同的。在 Acc-51 位置的情况下，一个硬件计数将相当于 32 个软件计数。

对于 PMAC 式伺服集成电路，位置捕获寄存器的 M 变量的定义列于下表中。注意，捕获寄存器是只读的；到相同地址的一个写操作是一个写入到通道的位置比较寄存器中。

Servo IC #	Channel 1	Channel 2	Channel 3	Channel 4
0	X:$078003,0,24,S	X:$078007,0,24,S	X:$07800B,0,24,S	X:$07800F,0,24,S
1	X:$078103,0,24,S	X:$078107,0,24,S	X:$07810B,0,24,S	X:$07810F,0,24,S
2	X:$078203,0,24,S	X:$078207,0,24,S	X:$07820B,0,24,S	X:$07820F,0,24,S
3	X:$078303,0,24,S	X:$078307,0,24,S	X:$07830B,0,24,S	X:$07830F,0,24,S
4	X:$079003,0,24,S	X:$079007,0,24,S	X:$07900B,0,24,S	X:$07900F,0,24,S
5	X:$079103,0,24,S	X:$079107,0,24,S	X:$07910B,0,24,S	X:$07910F,0,24,S
6	X:$07A203,0,24,S	X:$07A207,0,24,S	X:$07A20B,0,24,S	X:$07A20F,0,24,S
7	X:$07A303,0,24,S	X:$07A307,0,24,S	X:$07A30B,0,24,S	X:$07A30F,0,24,S
8	X:$07B203,0,24,S	X:$07B207,0,24,S	X:$07B20B,0,24,S	X:$07B20F,0,24,S
9	X:$07B303,0,24,S	X:$07B307,0,24,S	X:$07B30B,0,24,S	X:$07B30F,0,24,S

对于 PMAC2 式伺服集成电路，位置捕获寄存器的 M 变量的定义列于下表：

Servo IC #	Channel 1	Channel 2	Channel 3	Channel 4
0	X:$078003,0,24,S	X:$07800B,0,24,S	X:$078013,0,24,S	X:$07801B,0,24,S
1	X:$078103,0,24,S	X:$07810B,0,24,S	X:$078113,0,24,S	X:$07811B,0,24,S
2	X:$078203,0,24,S	X:$07820B,0,24,S	X:$078213,0,24,S	X:$07821B,0,24,S
3	X:$078303,0,24,S	X:$07830B,0,24,S	X:$078313,0,24,S	X:$07831B,0,24,S
4	X:$079003,0,24,S	X:$07900B,0,24,S	X:$079013,0,24,S	X:$07901B,0,24,S
5	X:$079103,0,24,S	X:$07910B,0,24,S	X:$079113,0,24,S	X:$07911B,0,24,S
6	X:$07A203,0,24,S	X:$07A20B,0,24,S	X:$07A213,0,24,S	X:$07A21B,0,24,S
7	X:$07A303,0,24,S	X:$07A30B,0,24,S	X:$07A313,0,24,S	X:$07A31B,0,24,S
8	X:$07B203,0,24,S	X:$07B20B,0,24,S	X:$07B213,0,24,S	X:$07B21B,0,24,S
9	X:$07B303,0,24,S	X:$07B30B,0,24,S	X:$07B313,0,24,S	X:$07B31B,0,24,S

该寄存器中的值是参照传感器的上电或复位位置。也就是说，计数器在上电或复位时被设置为零，并从那里开始计数。如果使用它用于反馈的电机被回零，它不会重置为零。如果传感器相对上电或复位位置的行程超过了+/－8,388,608（+/-223）计数，计数值将翻转。在这种情况下，你必须跟踪记录翻转。

需要注意的是读取位置捕获寄存器的这个动作会复位触发逻辑，自动重新准备它用于另一次触发。在开始捕获序列之前，执行一个对捕获寄存器的虚读是一个好主意，以确定触发逻辑被准备好（标准的移动--直到触发功能自动执行此操作）。如果您还读取分数计数捕获寄存器（见下文），首先读该寄存器，以确保它和全计数寄存器代表相同的触发位置。

分数计数的位置-捕捉寄存器

在 D 版本和更新版本的 PMAC 式伺服芯片（2002 年开始出货），可以捕获定时器-估计的子计数的位置，以及上述的全计数的位置。这是最常用于 Acc-51 高分辨率正弦编码器插补器，但也可以用正交编码器。

如果伺服 IC m 通道 n 的变量 I7mn9 设置为 1，此功能被启用。（请注意，如

果启用此功能，传统的“软件 1/ T”插补寄存器被禁用），在这种情况下，通道基地址的 Y 寄存器的 12 - 23 位（Y：$078000 用于伺服集成电路 0 通道 1），包含了捕获的分数计数值。23 位具有半计数的值， 22 位具有¼计数的值，依此类推。在此情况中的一个计数是一个“硬件计数”，它并不一定等同于电机的“软件计数。”

分数计数位置捕捉寄存器的 M 变量定义列于下表中。电机 xx 的建议的 M 变量，假设标准匹配通道到电机，是 Mxx83。

Servo IC #	Channel 1	Channel 2	Channel 3	Channel 4
0	Y:$078000,12,12,U	Y:$078008,12,12,U	Y:$078010,12,12,U	Y:$078018,12,12,U
1	Y:$078100,12,12,U	Y:$078108,12,12,U	Y:$078110,12,12,U	Y:$078118,12,12,U
2	Y:$078200,12,12,U	Y:$078208,12,12,U	Y:$078210,12,12,U	Y:$078218,12,12,U
3	Y:$078300,12,12,U	Y:$078308,12,12,U	Y:$078310,12,12,U	Y:$078318,12,12,U
4	Y:$079000,12,12,U	Y:$079008,12,12,U	Y:$079010,12,12,U	Y:$079018,12,12,U
5	Y:$079100,12,12,U	Y:$079108,12,12,U	Y:$079110,12,12,U	Y:$079118,12,12,U
6	Y:$07A200,12,12,U	Y:$07A208,12,12,U	Y:$07A210,12,12,U	Y:$07A218,12,12,U
7	Y:$07A300,12,12,U	Y:$07A308,12,12,U	Y:$07A310,12,12,U	Y:$07A318,12,12,U
8	Y:$07B200,12,12,U	Y:$07B208,12,12,U	Y:$07B210,12,12,U	Y:$07B218,12,12,U
9	Y:$07B300,12,12,U	Y:$07B308,12,12,U	Y:$07B310,12,12,U	Y:$07B318,12,12,U

如果比例得当，这个值可以被加到全计数捕获寄存器的值上，不管触发时的运动的方向。为将这些组合成硬件计数的单位（在许多情况下它们与软件计数的是相同的），可使用以下公式：

$$\text{编码器捕获硬件位置} = \text{分数计数}/4096 + \text{全计数}$$

如果捕获是在一个 Acc-51 插值正弦编码器上完成的，下面的公式可用于计算捕获的位置，以电机的软件计数：

$$\text{编码器捕获软件位置} = \text{分数计数}/32 + \text{全计数} * 32$$

注意，这个净额位置必须被存储在一个浮点变量中，以保存分数值，并且这些方程首先用分数计数，因为读取全计数寄存器会复位触发逻辑。在 Turbo PMAC 的代码，可能是这样的：

P103=M183/32+M103*32

转换到电机坐标和轴坐标

捕获寄存器被以编码器计数缩放，参考最新的上电或复位时的位置。通常，用户工作时希望是电机坐标--单位仍然是计数并参考回零位置，或是轴坐标--以工程单位（毫米，英寸，度），并参考用户设定的原点。

需要两个值将以软件计数（可以与硬件计数相同）为单位的编码器位置来转换为电机位置用于捕获的计算。第一个值是回零捕获位置--在回零触发时捕获的编码器位置，PMAC 存储的一个值以供将来使用。第二个是回零-偏移变量 Ixx26，其中包含触发位置和回零位置之间的差。关系是：

电机捕获位置 = 编码器捕获位置 – 回零捕获位置 – 回零偏移

回零捕捉位置是一个 24 位有符号整数，表示为计数。最好是用 24 位有符号 M 变量访问。对于电机 1，寄存器和建议的 M-变量是：

M173->Y:$0000CE,0,24,S ; #1 home capture position (cts)

回零-偏移 I 变量 Ixx26 是以 1/16 计数为单位，所以在加到电动机的位置之前应该除以 16。在 Turbo PMAC 的代码，这种转换可以这样做的：

P104=P103-M173-I126/16

从电机位置到轴位置的转换涉及一个比例因子和一个偏移量，用下面的公式：

轴捕获位置 = 电机捕获位置/比例因子 – 偏移

比例因子在轴定义语句中指定，单位是每个工程单位的计数，对于一个应用程序它应该是恒定的。比例因子可以直接在公式中指定，或者 PMAC 使用的实际寄存器可以用建议的 M-变量来访问：

Mxx91 ; Axis scale factor for X, U, A, B, or C assigned to Motor xx

Mxx92 ; Axis scale factor for Y or V assigned to Motor xx

Mxx93 ; Axis scale factor for Z or W assigned to Motor xx

偏移量是位置偏差项，建议的 M 变量是 Mx64，单元是 1/（Ixx08* 32）计数。它被设定为等于上电或复位和回零时的轴定义偏移量（通常为 0）。它可以在这之后被改变，通过在线指令{轴}=或运动程序命令 PSET。

17-5 使用 Turbo PMAC 上的位置-比较功能

Turbo PMAC 具有强大的位置比较功能内置到它的伺服集成电路中，特别是在 PMAC2 式 DSPGATE1 伺服集成电路中。另外，位置比较功能提供了基于实际位置计数器的快速、准确的数字输出---当 IC 中的硬件计数器等于比较寄存器时，输出立刻触发。通常，此输出用于触发一个测量装置或点燃一个激光。

由于比较输出的触发是一个纯粹的硬件功能（虽然安装是在软件中完成），所以没有软件延迟来影响精度。比较输出可以精确到在任何速度的精确计数。

位置比较功能是在伺服集成电路的软件配置硬件寄存器中来实现。软件配置把它的灵活性给了该功能；硬件电路把它的速度给了该功能。

所用的各个硬件决定了位置比较输出到哪里，以及驱动电路的性质。对于您的产品请查阅硬件参考手册。在某些情况下，驱动 IC 是有插座的，所以你必须选择使用一个电路。

这些输出通常标签为 EQUn，可以用来驱动外部硬件，例如触发用于扫描和测量设备。

位置-比较寄存器的缩放和偏移

位置比较寄存器被以硬件编码器计数缩放。一个硬件计数是由通道编码器解码的 I-变量 I7mn0 所定义，通常每个周期或每线是 4 个计数。比较寄存器总是参考最近的上电或复位的位置，称为编码器零点。此参考不随电机回零（改变电机零位置）而改变，或不随轴偏移（改变轴的零位置）而改变。更多的细节参见下面的从电机和坐标轴进行转换的部分。

位置比较寄存器，如硬件编码器计数器，是 24 位的值可以而且经常在正常操作过程中翻转。你必须注意追踪任何翻转效果。但是，请注意，通过将一个 24 位 M-变量分配到一个比较寄存器，写入该 M 变量的动作会自动正确截断较长的值为此字的长度。

在一个 PMAC 式伺服 IC 上设置

PMAC 式“DSPGATE”伺服 IC 的每个编码器计数器有一个位置比较功能。在 Turbo PMAC-PC 和 Turbo PMAC-PCI，可以使用几个位置比较输出通过背板总线来中断主机计算机。请注意，如果双端口的 RAM ASCII 通讯中断功能被启用（I56=1 和 I58=1）时，主板通道 1 的比较输出用于此中断，所以不能用于真实的位置比较功能。

另外，每个通道的位置比较电路都有一个 24 位比较寄存器，三个控制位，和一个状态位。比较寄存器包含了当输出触发时的位置计数值。请注意，这是一个只写的寄存器；如果相同的地址被读取，那么捕获的位置值被读取。

比较控制位

一个 PMAC 式伺服 IC 通道的控制位有：

1、比较输出-使能位：此位必须被设置为 1，以从 IC 得到比较信号输出。（内部状态位始终有效）。

2、比较标志锁存控制位：如果该位被设置为 0 时，输出是暂时的---当位置计数器等于比较寄存器的值时，它被打开但打开状态只持续这一个计数。如果该位被设置为 1 时，输出被锁存---当计数器等于比较值时它打开，并且保持接通直到该锁存位被清除。这是比较常见的使用模式。

3、比较输出反向控制位：如果该位被设置为 0，关断状态是低电压输出于伺服集成电路，而导通状态是一个高电压。（根据不同的输出驱动电路，卡的实际输出可以被这个反向。）如果该位被设置为 1，断开状态是一个高电压而导通状态是一个低电压。注意，该控制位可被它本身使用，使比较输出成为一个通用数字输出，或在某些情况下，是一个软件驱动的中断到主计算机。

单状态位是位置比较标志，它显示了比较输出逻辑目前的状态（如果输出被使能，是物理输出的当前状态）。如果比较输出逻辑状态是 on，它被设置为 1；如果输出逻辑状态是关闭的它被设置为 0。

对于自动使用的位置比较电路是没有固件功能的。您的应用软件必须直接处理比较寄存器和控制/状态位。通常，这些都是通过 M 变量访问的，而且建议的 M 变量的文件包括这些定义。对于 Turbo PMAC（伺服 IC0 通道 1）的第一个通道，

这些定义是：

```
M101->X:$078001,0,24,S          ; 24-bit position counter register
M103->X:$078003,0,24,S          ; 24-bit position compare register
M111->X:$078000,11,1            ; Compare flag latch control
M112->X:$078000,12,1            ; Compare output enable control
M113->X:$078000,13,1            ; Compare output invert control
M116->X:$078000,16,1            ; Compare logic status
```

要预加载一个比较位置时，只需将一个值写入到比较寄存器（例如 M103=1250）。

例如：我们有一个 50 点的数组，从 P1 到 P50，表示相对我们想要比较输出的起始位置的距离。可以启动这个进程的程序代码（只运行一次），是：

```
P100=1                          ; Select first point of array
P101=M101                       ; Read and store starting position
M103=P101+P(P100)               ; Write first relative position to compare
M111=1                          ; Set for latched output
M112=1                          ; Enable compare output
```

反复执行程序代码（可能在一台 PLC0 或 PLCC0），可以执行完成数组的是：

```
WHILE (P100<51)                 ; Loop until 50 points completed
  IF (M116=1)                   ; Reached last compare position?
    P100=P100+1                 ; Select next point
    M103=P101+P(P100)           ; Write next relative position to compare
    M111=0                      ; Clear output by making transparent
    M111=1                      ; Set for latched output again
  ENDIF
ENDWHILE
```

在一个 PMAC2 式伺服 IC 上设置

PMAC2 式伺服 IC 的每个编码器计数器有一个位置比较功能。此外，每个伺服集成电路的第一个编码器计数器（通道 1）可以使用 ASIC 的任何其它通道的位置比较电路，因此它可以使用多达 4 个的独立比较电路。

在 Turbo PMAC2-PC 和 Turbo PMAC2-PCI 上，通道 1 和通道 5（如果存在的话）的比较输出也可以用来通过背板总线中断主机计算机。请注意，如果双端口 RAM ASCII 通讯中断功能被启用（I56=1 和 I58=1）时，主板的通道 1 的比较输出用于此中断，所以不能用于真实的位置比较功能。

此外，还有一个用于输出的内存映射状态位，而 Turbo PMAC 软件可以使用 M-变量访问它用于自己使用。

如果伺服 IC 是在一个 Turbo PMAC 上，或是直接连接到它，那么可以通过用户定义的 M 变量访问寄存器和控制/状态位。如果伺服 IC 是在一个 MACRO 站上，要访问这些寄存器和位，是通过匹配该接口通道的 MACRO 节点的预定义的站节点特定的 MI-变量实现的。

比较寄存器

每个通道的位置比较电路是基于三个内存映射寄存器：

- 比较 A（Turbo 建议的 M 变量 Mxx08；MACRO 站 **MS{节点}，MI925**）
- 比较 B（Turbo 建议的 M 变量 Mxx09；MACRO 站 **MS{节点}，MI926**）
- 比较自动-递增(Turbo 建议的 M 变量 Mxx10;MACRO 站 **MS{节点},MI922**）

当编码器计数器的值匹配通道的比较 A 寄存器或比较 B 寄存器中的值时，比较输出被从现有的状态进行触发切换；或是从 0 到 1，或从 1 到 0。触发切换发生在在任一方向从等于到不等于的转换时。例如，如果比较寄存器包含 100，则输出的触发切换将发生在正方向上的在从 100 到 101 计数的转换时，或者在负方向的从 100 到 99 计数的转换时。

此外，当输出由比较寄存器其中一个进行触发切换时，其他的寄存器将立即递增，递增数量是自动-递增寄存器中的值。如果输出是通过在正方向上的移动进行触发切换的，自动-递增寄存器中的值被添加到其他的比较寄存器中。如果输出是通过在负方向的移动触发切换的，从其他比较寄存器中减去自动-递增寄存器中的值。

如果自动-递增寄存器是非零的，所有的比较边沿应至少相隔 2 个计数。这意味着比较 A 和比较 B 寄存器应不小于相隔 2 个计数，并且自动-递增寄存器的最低非零值应为 4。

比较控制位

在一个 PMAC2 式伺服芯片的每个通道有三个控制位。它们分别是：

1、比较通道选择位：（Turbo I-变量 I7mn1；MACRO 站 **MS{节点}，MI911**）该控制位决定通道的比较电路是工作在该通道的编码器上，还是该伺服 IC 的通道 1 的编码器上。（请注意，此位对通道 1 不执行任何操作。）该控制位已被分配了一个 I 变量---伺服 ICm 通道 n 的 I7mn1。需要注意的是，当多个比较电路已分配给一个伺服 IC 的通道 1 时，用于第一个通道的比较输出是所有分配给通道 1 的比较逻辑输出的逻辑或。

2、比较直接-写的值（初始状态）：（Turbo 建议的 M 变量 Mxx12；MACRO 站 **MS{节点}，MI929**）。该控制位允许您直接设置通道的比较输出的状态，无论是作为一个比较的序列的初始状态，或是使用输出用于一般用途，或用于一个软件驱动的中断。这里的一个 1 将迫使伺服集成电路的输出上为高电压；一个 0 将迫使低电压。（根据所使用的输出驱动器，这可能是被反向在控制器本身的输出）。用户有责任确定，当他使用直接-写入功能时，他是在他所期望的 0 区或“1”区。单独写到此位不设置输出；这种情况不会发生直到写操作被使能，使用下一个控制位。

3、比较直接-写使能：（Turbo 建议的 M 变量 Mxx11；MACRO 站 **MS{节点}，MI928**）。写一个 1 到该位强制直接-写的位值在比较输出线上。只要输出状态被置位，此位会自动设定其本身回到 0。

单状态位是比较输出状态，它反映了输出在任一时刻的状态。一个 1 在这里表示伺服集成电路输出高电压，0 表示低电压。取决于在特定的配置中使用的输出驱动器，控制卡的电压可反向。

对于自动使用的位置比较电路没有固件功能。用户的应用软件必须直接处理比较寄存器和控制/状态位。通常，这些都通过 M 变量（或 MACRO 站设置 MI-变量）进行访问，而且建议的 M 变量的文件包括这些定义。对于 Turbo PMAC2（伺

服 IC0 通道 1）的第一个通道，这些定义是：

```
M101->Y:$078001,0,24,S          ; Hardware position value (counts)
M108->Y:$078007,0,24,S          ; Position compare A value (counts)
M109->X:$078007,0,24,S          ; Position compare B value (counts)
M110->X:$078006,0,24,S          ; Auto-increment value (counts)
M111->X:$078005,11              ; Position compare write enable
M112->X:$078005,12              ; Position compare direct-write (initial-state) value
M113->X:$078000,9               ; Position compare output status
```

请参考软件参考手册列表中的建议的 M 变量，查看其他通道的等价定义。

设置为单脉冲输出

如果只是需要一个单一的比较脉冲（不使用自动增量功能），请执行以下步骤：

- 将前沿的编码器值写到比较 A 寄存器
- 将后边沿的编码器值写到比较 B 寄存器中
- 设置自动-增量寄存器为零
- 在一个两步的过程中，通过直接-写功能设置初始状态：
  - 写一个值到初始状态位
  - 写 1 到直接-写使能位（这是自清为 0）
- 启动将引起比较功能的移动

例如：对于轴静止不动在约编码器位置 100 处，而编码器位置 1000 和 1010 之间需要的位置比较为 1 的值，可以用下面的代码：

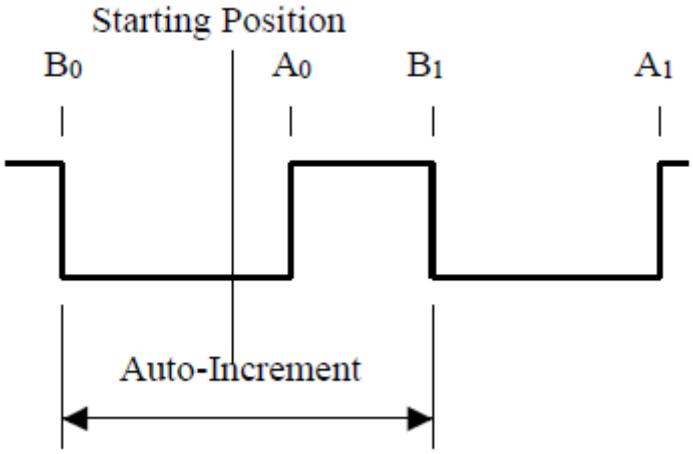
```
M108=1000               ; Set front end compare in A
M109=1010               ; Set back end compare in B
M110=0                  ; No auto-increment
M112=0                  ; Prepare initial value of 0
M111=1                  ; Enable direct write (resets immediately to 0)
{Command to start the move}
```

设置为多个脉冲输出

通过使用自动-递增功能，通过一个单独的软件安装操作可以创建多个比较脉冲。当自动-递增寄存器是一个非零值时，当另一个比较值被匹配时，寄存器的值会自动加到一个比较寄存器的值上或从后者中减去。PMAC 注意跟踪递增的方向，所以在自动增量寄存器只能使用正值，即使是编码器将在负方向计数。

设置为多个脉冲与设置为单个脉冲是一样的，不同的是一个非零值必须输入到自动递增寄存器中，并且如果移动为正向，为后边缘输入的值必须是第一个后沿的值减去自动-递增寄存器的值；或者，如果运动为负，是第一个后沿的值加自动-递增寄存器的值。

换句话说，两个比较寄存器的初始值必须“括号/包括”起始位置。当其中一个比较值被编码器计数器匹配时，另一个比较值在运动方向上递增。



例如：从上面的例子开始，需要 1000（A0）和 1010（B1）之间的比较输出进行计数，但增加一个 2000 计数的自动-递增值，起始位置约 100 计数，启动序列的程序代码可以是：

```
M110=2000                    ; Auto-increment of 2000 encoder counts
M108=1000                    ; First front edge (A0) at 1000 counts
M109=1010-M110               ; First back edge (B1) at 1010 counts
M112=0                       ; Prepare initial value of 0
M111=1                       ; Enable direct write (resets immediately to 0)
{Command to start the move}
```

分数-计数比较

从 D 版本的 DSPGATE1 PMAC2 式伺服集成电路（2002 年开始出货）开始，比较电路可以以一个分数的计数值触发输出，而不只是在最近的计数的整数位。该 IC 可以使用 1/ T 时基的技术在每个编码器的采样时钟（SCLK）周期创建一个分数计数值，并在位置比较（和捕获）电路中使用这个扩展的计数值。这种能力在 PMAC 式伺服集成电路是不可能的。

当使用一个正弦编码器通过一个插值器电路，如 Acc-51C 时，这种能力是特别有价值的。插补器大大增加了伺服反馈的分辨率，但其本身并不能增加比较功能的分辨率。但是请注意，对于扩展的比较功能不要求使用正弦编码器和插补器，它的工作原理与数字正交编码器是相同的。

这个扩展的计数模式是通过变量 I7mn9 设置为 1 而启动用于伺服 ICm 通道 n。这个变量是在 V1.937 固件中实现，但无存档直到 V1.939 软件参考手册（因为可以使用它的芯片之前没有开始发货）。随着 I7mn9 设置为出厂默认值 0，扩展的计数模式被禁用。此单-位的 I-变量是 IC 的通道的控制字的 18 位。

当该模式启用时，这两个 24 位定时器寄存器对通道的含义改变了。（这意味着你不能使用转换表中传统的软件 1/ T 计数扩展用于此通道；如果在此模式下你使用数字正交编码器，您必须使用新的“硬件 1/ T”的转换方法。）在扩展的计数模式，这两个寄存器包含四个 12 位的值。每个寄存器的低半部分包含了位置比较功能的小数计数值。对于伺服集成电路 0 通道 1 的这些值的建议的 M 变量是：

M188->Y:$078001,0,12,U ; Compare A fractional count
M189->Y:$078000,0,12,U ; Compare B fractional count

这些寄存器必须作为无符号值使用。用这种方法处理，这些寄存器取值范围为 0～4095，大小相当于 1/4096 的（整）计数值。分数计数的值越高，越接近正的整计数，而不管运动的方向。

请仔细注意这里一个计数是指什么。因为我们正在处理伺服集成电路的硬件计数器，所以这些都是硬件计数---硬件计数器的一个单位。对于几乎是普遍使用的 4-倍解码（且必须与 ACC-51 主板使用），编码器的每线（信号周期）有 4 个硬件计数。

Turbo PMAC 的高分辨率的正弦编码器插值产生每线 4096 个状态，或每个硬件计数（10 位的分数）是 1024 个状态。然而，Turbo PMAC 的伺服软件假定在它访问的位置处只有 5 位的小数计数数据。因此，一个 ACC- 51 插补器的一个软件计数为硬件计数的 1/32 的大小，所以编码器的每线是 128 个软件计数。

下表显示了，对于每个伺服 IC 和通道，启动子计数比较的 I 变量和全计数和分数计数数据的 M 变量的定义。

IC #	Chan #	Mode I-Var	Compare A Whole Count (Mxx08)	Compare A Fractional Count (Mxx88)	Compare B Whole Count (Mxx09)	Compare B Fractional Count (Mxx89)
0	1	I7019	Y:$078007,0,24,S	X:$078001,0,12,U	X:$078007,0,24,S	Y:$078000,0,12,U
0	2	I7029	Y:$07800F,0,24,S	X:$078009,0,12,U	X:$07800F,0,24,S	Y:$078008,0,12,U
0	3	I7039	Y:$078017,0,24,S	X:$078011,0,12,U	X:$078017,0,24,S	Y:$078010,0,12,U
0	4	I7049	Y:$07801F,0,24,S	X:$078019,0,12,U	X:$07801F,0,24,S	Y:$078018,0,12,U
1	1	I7119	Y:$078107,0,24,S	X:$078101,0,12,U	X:$078107,0,24,S	Y:$078100,0,12,U
1	2	I7129	Y:$07810F,0,24,S	X:$078109,0,12,U	X:$07810F,0,24,S	Y:$078108,0,12,U
1	3	I7139	Y:$078117,0,24,S	X:$078111,0,12,U	X:$078117,0,24,S	Y:$078110,0,12,U
1	4	I7149	Y:$07811F,0,24,S	X:$078119,0,12,U	X:$07811F,0,24,S	Y:$078118,0,12,U
2	1	I7219	Y:$078207,0,24,S	X:$078201,0,12,U	X:$078207,0,24,S	Y:$078200,0,12,U
2	2	I7229	Y:$07820F,0,24,S	X:$078209,0,12,U	X:$07820F,0,24,S	Y:$078208,0,12,U
2	3	I7239	Y:$078217,0,24,S	X:$078211,0,12,U	X:$078217,0,24,S	Y:$078210,0,12,U
2	4	I7249	Y:$07821F,0,24,S	X:$078219,0,12,U	X:$07821F,0,24,S	Y:$078218,0,12,U
3	1	I7319	Y:$078307,0,24,S	X:$078301,0,12,U	X:$078307,0,24,S	Y:$078300,0,12,U
3	2	I7329	Y:$07830F,0,24,S	X:$078309,0,12,U	X:$07830F,0,24,S	Y:$078308,0,12,U
3	3	I7339	Y:$078317,0,24,S	X:$078311,0,12,U	X:$078317,0,24,S	Y:$078310,0,12,U
3	4	I7349	Y:$07831F,0,24,S	X:$078319,0,12,U	X:$07831F,0,24,S	Y:$078318,0,12,U
4	1	I7419	Y:$079207,0,24,S	X:$079201,0,12,U	X:$079207,0,24,S	Y:$079200,0,12,U
4	2	I7429	Y:$07920F,0,24,S	X:$079209,0,12,U	X:$07920F,0,24,S	Y:$079208,0,12,U
4	3	I7439	Y:$079217,0,24,S	X:$079211,0,12,U	X:$079217,0,24,S	Y:$079210,0,12,U
4	4	I7449	Y:$07921F,0,24,S	X:$079219,0,12,U	X:$07921F,0,24,S	Y:$079218,0,12,U
5	1	I7519	Y:$079307,0,24,S	X:$079301,0,12,U	X:$079307,0,24,S	Y:$079300,0,12,U
5	2	I7529	Y:$07930F,0,24,S	X:$079309,0,12,U	X:$07930F,0,24,S	Y:$079308,0,12,U
5	3	I7539	Y:$079317,0,24,S	X:$079311,0,12,U	X:$079317,0,24,S	Y:$079310,0,12,U

5	4	I7549	Y:$07931F,0,24,S	X:$079319,0,12,U	X:$07931F,0,24,S	Y:$079318,0,12,U
6	1	I7619	Y:$07A207,0,24,S	X:$07A201,0,12,U	X:$07A207,0,24,S	Y:$07A200,0,12,U
6	2	I7629	Y:$07A20F,0,24,S	X:$07A209,0,12,U	X:$07A20F,0,24,S	Y:$07A208,0,12,U
6	3	I7639	Y:$07A217,0,24,S	X:$07A211,0,12,U	X:$07A217,0,24,S	Y:$07A210,0,12,U
6	4	I7649	Y:$07A21F,0,24,S	X:$07A219,0,12,U	X:$07A21F,0,24,S	Y:$07A218,0,12,U
7	1	I7719	Y:$07A307,0,24,S	X:$07A301,0,12,U	X:$07A307,0,24,S	Y:$07A300,0,12,U
7	2	I7729	Y:$07A30F,0,24,S	X:$07A309,0,12,U	X:$07A30F,0,24,S	Y:$07A308,0,12,U
7	3	I7739	Y:$07A317,0,24,S	X:$07A311,0,12,U	X:$07A317,0,24,S	Y:$07A310,0,12,U
7	4	I7749	Y:$07A31F,0,24,S	X:$07A319,0,12,U	X:$07A31F,0,24,S	Y:$07A318,0,12,U
8	1	I7819	Y:$07B207,0,24,S	X:$07B201,0,12,U	X:$07B207,0,24,S	Y:$07B200,0,12,U
8	2	I7829	Y:$07B20F,0,24,S	X:$07B209,0,12,U	X:$07B20F,0,24,S	Y:$07B208,0,12,U
8	3	I7839	Y:$07B217,0,24,S	X:$07B211,0,12,U	X:$07B217,0,24,S	Y:$07B210,0,12,U
8	4	I7849	Y:$07B21F,0,24,S	X:$07B219,0,12,U	X:$07B21F,0,24,S	Y:$07B218,0,12,U
9	1	I7919	Y:$07B307,0,24,S	X:$07B301,0,12,U	X:$07B307,0,24,S	Y:$07B300,0,12,U
9	2	I7929	Y:$07B30F,0,24,S	X:$07B309,0,12,U	X:$07B30F,0,24,S	Y:$07B308,0,12,U
9	3	I7939	Y:$07B317,0,24,S	X:$07B311,0,12,U	X:$07B317,0,24,S	Y:$07B310,0,12,U
9	4	I7949	Y:$07B31F,0,24,S	X:$07B319,0,12,U	X:$07B31F,0,24,S	Y:$07B318,0,12,U

自动-增量功能在扩展计数模式仍然可用，但自动-增量值被限制为计数的整数部分。

在此模式下运行，你需要同时写入到标准（整数）比较寄存器和分数比较寄存器。在这种模式下，写入到比较寄存器的整数值相对比较输出触发切换的计数值处有 1 个计数的偏移。这种偏移的方向是依赖于运动的方向。如果一个 n 计数的值写入到整数比较寄存器，当在正方向移动时，比较输出将在 n +1 个计数处触发切换；当在负方向移动时的，将在 n-1 个计数处触发切换。为了补偿这一点，根据运动的方向，只需加上或减去 1 个计数。例如，当在正方向上移动时，为在 743.25 计数处得到一个比较边沿，写入 742（=743-1）到整数比较寄存器中，并写入 1024（=0.25*4096）到分数比较寄存器中。

例如：P108 和 P109 是浮点变量，代表比较输出将被打开、然后关闭的第一个计数值。P110 是一个整数值，表示自动-递增的长度。运动将在正方向运行。启动序列的程序代码可以是：

```
M110=P110                          ; Auto-increment value
M108=INT(P108)-1                   ; Compare A integer component
M188=(P108-(M108+1))*4096          ; Compare A fractional component
M109=INT(P109)-1-M110              ; Compare B integer component
M189=(P109-(M109+1+M110))*4096     ; Compare B fractional component
M112=0                             ; Prepare initial value of 0
M111=1                             ; Enable direct write
```

从电机坐标和轴坐标进行转换

比较寄存器被以编码器计数缩放，参考最新上电或复位处的位置。通常情况

下，工作在电动机的坐标---单位仍然是计数并参考回零位置，或工作在轴坐标--以工程单位（毫米、英寸、度），并参照用户设定的原点。

需要两个值将电机位置转换为编码器位置用于比较计算。第一个是回零捕捉偏移，在回零触发的捕捉编码器的位置---是 PMAC 存储的一个值以供将来使用。第二个是回零-偏移变量 Ixx26，它包含触发位置和回零位置之间的差。关系是：

编码器位置 = 电机位置 + 回零捕获偏移 + 回零偏移

回零捕获偏移量是一个 24 位有符号整数，表示为计数。最好是用 24 位有符号 M 变量进行访问。电机 1 的寄存器和建议 M-变量是：

```
M173->Y:$0000CE,0,24,S          ; #1 Encoder home capture position (cts)
```

回零偏移 I 变量 Ixx26 是以 1/16 计数为单位，所以在加到电动机的位置之前应该除以 16。

例如，对于使用 PMAC2 式伺服 IC 的编码器 1 的电机 1，如果您想设置比较 1A 寄存器以在电机位置+1500 计数处进行触发，你可以使用下面的命令，在线的，在一个运动程序中，或在一个 PLC 程序中：

```
M108=1500+M173+I126/16
```

如果你也想指定分数计数值，您可以初步计算比较位置（计数）为一个浮点值。然后，你可以将其分解为整数和小数部分。修改上面的例子：

```
P108=P500+M173+I126/16                ; Compare position in counts
M108=INT(P108)                        ; Write whole-count value
M188=(P108-INT(P108))*4096            ; Write fractional-count value
```

从轴位置到电机位置的转换涉及一个比例因子和偏移量，用下面的公式：

电机位置 = 比例因子 * 轴位置 + 偏移

比例因子在轴定义语句中进行指定，单位是每工程单位的计数。在一个应用程序中它应该是恒定的。您可以在公式中直接指定比例因子，或者您也可以通过建议的 M 变量访问 PMAC 正在使用的实际寄存器：

```
Mxx91                   ; Axis scale factor for X, U, A, B, or C assigned to Motor xx
Mxx92                   ; Axis scale factor for Y or V assigned to Motor xx
Mxx93                   ; Axis scale factor for Z or W assigned to Motor xx
```

偏移量是位置偏差项，所建议的 M 变量是 MX64，单位是 1/（Ixx08* 32）计数。它被设定为等于上电或复位和回零时轴定义的偏移量（通常为 0）。它可以在这之后被改变，通过在线指令{axis}=或运动程序命令 PSET。

例如，电机 1 分配到 X 轴，如果你要设置比较 1A 寄存器在相对轴原点+2.5 工程单位处进行触发，你可以来计算电机的位置，单位是计数：

```
P1=M191*2.5+M164/(I108*32)
```

然后，您可以设置实际比较寄存器：

```
M108=P1+M173+I126/16
```

注意：

如果这个等式右边的表达式已经产生了超出比较寄存器范围的一个结果，写入到分配给该寄存器中的 M 变量的动作将适当截断值到 24 位的范围，以使比较功能将正常工作。24 位带符号寄存器范围是-8,388,608～+8,388,607。如果你试图写一个 9,388,608 值---超过翻转点一百万个计数---到这个寄存器，所产生的值将是-7,388,608，它翻转后会达到一百万计数。

17-6 同步 M-变量赋值输出

同步 M-变量赋值语句允许输出被同步地置位和清除---和运动程序启动下一个指令运动时保持同步。请注意，输出是与指令位置同步，不一定是实际的位置，这可能是由于跟随错误造成的不同。这些语句在本手册的《计算功能》部分进行详细讨论。

第十八章 编写和执行 PLC 程序

18-1PLC 程序是什么?

Turbo PMAC 有 64 个 PLC 程序可异步操作并快速重复执行---32 个编译的 PLC 程序和 32 个解译的（未编译）PLC 程序。他们被称为 PLC 程序，是因为他们执行许多与硬件可编程逻辑控制器相同的功能。PLC 程序有许多与运动程序相同的逻辑结构，但没有移动类型的语句。

大多数编译的 PLC 程序是相似于未编译的 PLC 程序，如果不相同的话。事实上，在 PLC 程序被编译之前，它被作为一个未编译的 PLC 进行测试和调试。这两种类型的 PLC 程序之间的不同点是缓冲区控制，L-变量，一些命令的语法以及编译器的使用。因为他们相似，很多关于未编译 PLC 程序的部分也适用于编译的 PLC 程序。特别是关于编译 PLC 程序的信息都包含在编译的 PLC 程序部分中。

何时使用

PLC 程序被设计用于异步于运动的计算和操作。如果你想进行的计算动作是同步于编程运动（比如在一个程序移动的边界），使用一个运动程序创建操作就可以。即使动作重复用于每个编程的运动，最好是在运动程序完成，可能在一个子程序中。

常见的用途

PLC 程序特别适用于监视模拟和数字输入、设置输出、发送信息、监控运动参数、像主机一样发出指令、改变增益，以及启动和停止运动。由于他们可以完全访问 Turbo PMAC 变量和 I/ O，以及其异步性质，他们成为运动控制程序强大的辅助手段。

64 个 PLC 程序

警告：

一个 PLC0 太大可能会导致不可预知的行为，甚至可以通过占用后台任务的执行时间来跳闸 Turbo PMAC 的看门狗定时器。

对于编译和未编译的 PLC，PLC 的程序编号均是从 0 到 31。这意味着你可以同时拥有一个已编译的 PLC n 和未编译的 PLC n 储存在 Turbo PMAC。PLC 程序 0 是一个特别快速的程序，它工作在伺服中断周期结束时，以变量 I8（每 I8+1 伺服周期）指定的频率运行。这个程序是用于几个临界任务，并且应尽量小，因为

其快速的重复执行可以占用其它任务的时间。

PLC 程序 1 至 31 在时间允许的范围内连续运行在后台，实际上是在一个无限循环中。它们是由电机相位、伺服回路闭合、移动规划和 PLC0 等优先级高的任务所中断。

18-2 输入一个 PLC 程序

PLC 程序语句的输入和向 Turbo PMAC 输入缓冲的命令行是一样的。在准备阶段，通过发出一个 CLOSE 命令以确保没有其他的缓冲区被打开是一个好主意。通过发出一个 DELETE GATHER 命令以确保内存没有被数据收集或程序轨迹缓冲区占用也是很好的做法。

打开缓冲区

对于每一个 PLC 程序，下一步是通过 OPEN PLC n 语句打开缓冲区用于输入，其中 n 是缓冲区号。其次，如果目前缓冲区有不应保存的内容，应通过 **CLEAR** 语句清空（PLC 缓冲区不能在 Turbo PMAC 本身进行编辑，它们必须被清除并重新输入）。如果缓冲区不被清除，新的语句将被添加到缓冲区的末端。

下载程序

通常在程序开发中，编辑将在基于主机的文本编辑器来完成，而在每次新版本下载到卡时，旧的缓冲区清零。完成后，通过 CLOSE 命令关闭缓冲区。打开一个 PLC 程序缓冲区自动停止该程序的运行。关闭缓冲区之后，程序仍然禁用，但可以通过 **ENABLE PLC n** 命令使程序再次有效，其中 n 是缓冲区号（0-31）。要执行一个 PLC 程序，还必须正确设置 I5。

注意：

因为 Turbo PMAC 的内存中的所有 PLC 程序是在上电启动/重置被启动的，所以在开发 PLC 程序时，最好将 I5 在 Turbo PMAC 的内存中保存为 0。这将允许复位 Turbo PMAC 而没有 PLC 在运行（如果 I5 设置正确，只有一个启用的 PLC 运行）而且 PLC 编程出现错误时，恢复也更容易。。

此技术的一般形式是：

```
CLOSE
DELETE GATHER
OPEN PLC n
CLEAR
{PLC statements}
CLOSE
ENABLE PLC n
```

关闭缓冲区

在关闭时，Turbo PMAC 进行检查以确保所有的 IF 分支和 WHILE 循环已经正常终止。如果不是，它报告一个错误，而且缓冲区是不能工作的。然后在主机上纠正 PLC 程序并重新输入它（当然，清除过程中的错误部分）。这个过程被重复用于所有被使用的 PLC 的缓冲区。

删除程序

要删除一个未编译的 PLC 程序，打开缓冲，清除内容，然后再次关闭缓冲区。这可以通过在一行的三个命令来实现，如：

```
OPEN  PLC  5  CLEAR  CLOSE
```

例子

一个 PLC 块入口的一个快速简单的例子如下所示：

```
OPEN PLC 2
CLEAR
IF (M11=1)
        I130=10000
ELSE
        I130=8000
ENDIF
CLOSE
ENABLE PLC 2
```

18-3 PLC 的程序结构

在编写 PLC 程序中最重要的是，每个 PLC 程序实际上是在一个无限循环，它会执行一遍又一遍，直到被告知停止。（这些被称为 PLC 是因为他们与硬件可编程逻辑控制器---对一系列的操作和可能的操作序列重复的扫描---很相似）

计算语句

很多由 PLC 采取的动作是通过变量的值赋值语句完成：**{变量}={表达式}**。该变量可以是 I、P、Q 或 M 型，这些动作可以影响卡的内部和外部很多事情。

也许最简单的 PLC 程序包括一行：

```
P1=P1+1
```

每次 PLC 执行时，通常是每秒几百次，P1 将增加 1。

当然，这些语句可以更加复杂。声明：

P2=M162/(I108*32*10000)*COS(M262/(I208*32*100))

能够将径向（M162）和角度（M262）位置转换到水平位置数据，同时缩放。它更新如此频繁，所以谁需要访问该信息（如主机，运算符，运动程序），可确保得到当前的数据。

条件语句

大部分动作在 PLC 程序是有条件的，取决于 Turbo PMAC 的变量如输入、输出、位置、计数器等的状态。动作应是电平触发或边沿触发；两者都可以做到，但方法是不同的。

电平-触发条件

通过一个电平触发条件控制的一个分支是更容易实现。以我们的递增变量为例子，让变量 M11 的输入控制计数，我们有：

```
IF (M11=1)
  P1=P1+1
ENDIF
```

只要输入为真，P1 将递增每秒几百次。当输入为假，P1 将停止递增。

边沿-触发条件

假设每次 M11 变为真，您想要只增加 P1 一次（触发在 M11 的上升沿有时被称为单次触发或锁存）。我们需要一个复合条件来触发动作，作为行动的一部分，我们设定其中一个条件为假，所以在下一次 PLC 扫描时该动作将不会发生。做到这一点最简单的方法是通过使用一个影子变量，它将跟随输入变量值的变化。只有当影子变量不匹配输入变量时，动作才发生。我们的代码为：

```
IF (M11=1)
  IF (P11=0)
    P1=P1+1
    P11=1
  ENDIF
ELSE
  P11=0
ENDIF
```

请注意，我们必须确保 P11 可以跟随 M11 而变化。在一个电平触发模式，我们设置 P11 为 0。我们也可以进行边沿触发，但就该程序的最终结果而言它无关紧要。它与计算时间有关，并且节省了程序行。

任何 SEND、**COMMAND** 或 DISPLAY 操作语句应该只在边沿触发的条件下进行，因为 PLC 比这些处理他们信息的操作可以循环得更快，而且如果是 PLC 连续扫描时执行这些语句，通信通道无法工作。

```
IF (M11=1)                          ; input is ON
  IF (P11=0)                        ; input was not ON last time
    COMMAND"#1J+"                   ; JOG motor
    P11=1                           ; set latch
```

```
    ENDIF
  ELSE
    P11=0                              ; reset latch
  ENDIF
```

WHILE 循环

通常，PLC 程序在单次扫描中从开始到结束执行一遍。如果程序遇到一个真的 WHILE 条件，就不是这样了。在这种情况下，程序将执行到 ENDWHILE 语句，并退出该 PLC。在执行完所有其它 PLC 后，将在 WHILE 条件语句处重新进入该 PLC，而不是在开头。只要条件为真，这个过程将重复。当 WHILE 条件为假时，PLC 程序将跳过 ENDWHILE 语句并继续执行 PLC 程序的其余部分。

如果我们想只要输入为真，就增加我们的计数器，并防止执行 PLC 程序的其余部分，我们可以进行编程：

```
  WHILE (M11=1)
    P1=P1+1
  ENDWHILE
```

这种结构使得在程序的一个部分更容易截断 PLC 运行，所以当条件为真时，同一个程序中的其他分支并不需要额外的条件就可以不执行。将它与 IF 条件进行比较（见上文）。

一些 **COMMAND** 操作语句后面应该有一个 WHILE 条件，以确保他们在继续进行 PLC 程序的其余部分之前已经生效。如果后面的第二个 **COMMAND** 语句且需要第一个 COMMAND 语句结束后才能执行时，该条件始终是 true。（请记住，COMMAND 操作语句只在后台循环的通信部分才能进行处理。）

若要一个输入停止一个坐标系的任何运动，并开始运动程序 10，使用下列 PLC 程序：

```
IF (M11=1)                          ; input is ON
  IF (P11=0)                         ; input was not ON last time
    P11=1                           ; set latch
    COMMAND"&1A"                     ; ABORT all motion
    WHILE(M5187=0)                    ; wait for motion to stop
    ENDWHILE
    COMMAND"&1B10R"                  ; start program 10
  ENDIF
ELSE
  P11=0                              ; reset latch
ENDIF
```

注意：

M5187 是坐标系定义在建议的 M 变量列表中的 In-Position bit（“就-位”位）。

精确定时

因为 PLC1～31 在 Turbo PMAC 上是最低的计算优先级，循环时间不能被精确地确定。为保持一个动作的时间是相当的精确，可以使用 WHILE 循环，但不是递增一个变量，要使用一个板上的计时器。

在 Turbo PMAC 的每个坐标系统有两个用户倒计时计时器：Isx11 和 Isx12（I5111 和 I5112 用于坐标系 1）。对于坐标系 1 到（I68+1），这些定时器是激活的。他们是 24 位带符号寄存器（范围：-8,388,608～+8,388,607），每个伺服周期，向下计数一次。通常情况下，写一个值给它，表示要等待的时间，单位为伺服周期（毫秒乘以 8,388,608/ I10），然后等待它达到零。示例代码：

```
I5111=500*8388608/I10          ; Set timer to 500 msec
WHILE (I5111>0)                ; Loop until counts to zero
ENDWHILE                       ; Exit PLC program here when true
```

如果需要更多的定时器，使用内存地址为 X：$0 的寄存器。该 24 位寄存器每伺服周期向上计数一次。为它存储一初始值，然后每次扫描时从当前值中减去初始值，差值与等待的时间量进行比较。通过将结果存到另一个 24 位寄存器，我们可以正常处理 X：$0 可能的翻转。

通过在线命令定义下列 M 变量：

```
M0->X:$0,24                ; Servo counter register
M85->X:$0010F0,24          ; Free 24-bit register
M86->X:$0010F1,24          ; Free 24-bit register
```

然后写 PLC 程序的一部分：

```
M85=M0                     ; Start of timer
M86=M0-M85                 ; Time elapsed so far
WHILE (M86<P86)            ; Less than specified time?
   M86=M0-M85              ; Time elapsed so far
ENDWHILE                   ; Exit PLC program here when true
```

18-4 编译的 PLC 程序

可以通过编译 Turbo PMAC PLC 程序得到更快的执行速度。已编译的 PLC 更快的执行速度来自两个因素：第一，消除诠释时间，二是编译的 PLC 程序有执行整数运算的能力。编译的 PLC 程序的浮点运算的运行速度比解译的 PLC 程序快两到三倍；编译形式的整数（包括布尔）运算的运行速度快 20～30 倍。

Turbo PMAC 本身不执行 PLC 程序的编译。编译是在一台 PC 上进行，所产生的机器代码然后下载到 Turbo PMAC。

Turbo PMAC 可以存储和执行多达 32 个编译的 PLC 程序以及 32 个解译（未编译）的 PLC 程序，共 64 个 PLC 程序。15K（15,360）的 24 位字的 Turbo PMAC 内存保留用于编译的的 PLC;或如果有一个用户编写的伺服时则是 14K(14,336)字。其他任务不可以使用这个内存，而编译的 PLC 也不使用任何其他的内存。

注意：

这里提到的编译代码的大小，指的是实际的编译后的代码在 Turbo PMAC 的内存占用的空间。它不是指编译器的输出文件在 PC 的硬盘驱动器上的大小。

在 Turbo PMAC 上，一个编译的 PLC 程序被标记为 PLCC N（PLC-编译的＃n）。这是为了与一个解释型的 PLC---标示的 PLC n 区分开来。相同编号的解释的和编译的 PLC 之间没有特别的关系。

执行编译的 PLC

32 个编译的 PLC 程序（PLCC0 至 PLCC31）中仅 PLCC0 运行在前台，通过实时中断（RTI）触发。 PLCC1 至 31 运行作为后台任务。

在每一个实时中断，Turbo PMAC 检查是否多个用户任务需要完成。每（I8+1）个伺服周期，实时中断发生一次。Turbo PMAC 以下列顺序检查任务：

1、运动程序运动准备：Turbo PMAC 检查看看在每一个坐标系中是否要计算程序中的下一步运动。
2、解释的 PLC 0：Turbo PMAC 检查看是否 I5 =1 或 3 以及 PLC 0 是否已启用。如果是这样，则对 PLC 0 执行一次扫描。
3、编译的 PLC 0：Turbo PMAC 检查看是否 I5 =1 或 3 以及是否 PLCC0 已启用。如果是这样，则执行 PLCC0 的一次扫描。

重要的是，PLCC0 和 PLC0 的扫描执行时间保持小于 1 个实时中断时间。否则，他们的重复执行将占用后台时间，也可能跳闸看门狗定时器。

在后台，Turbo PMAC 对一个后台解释的 PLC 程序执行扫描时，不会被任何其他后台任务中断（虽然较高优先级的任务将中断）。在每一个单独的后台解释的 PLC 程序的每次扫描之间，Turbo PMAC 将对后台所有激活的编译的 PLC 执行一次扫描。这意味着，后台编译的的 PLC 的执行的扫描率比后台解释的 PLC 的更高。例如，如果有七个激活的后台解释的 PLC，对后台解释的 PLC 扫描一次时，后台编译的 PLC 已经执行了 7 次扫描。

编写编译的 PLC 程序

您可以编写编译的 PLC，就像你写标准的解释的 PLC 那样，使用一个标准的文本编辑器比如 PMAC 执行程序中的。可以在解释的 PLC 程序中使用的所有语句也可以在编译的 PLC 程序中使用，所以一种解释型的 PLC 更改为编译的 PLC 是可能的而无需更改任何代码。

然而，编译的 PLC 程序支持额外的编程功能，提高效率和灵活性。这些功能描述如下。请记住，这些功能在解释的 PLC 是不被支持的。

编译器分配的指针变量

为了直接和高效地访问 Turbo PMAC 寄存器，编译的 PLC 程序支持两种类型的指针变量，而为该变量赋值寄存器是在编译时间进行的，而不是在程序执行时间。

L-变量：短整型指针

L-变量是短（24 位）寄存器的指针，视为整数（定点）值。这些变量在编译的 PLC 程序的工作方式与普通 L-变量是一样的。他们可以访问 X 或 Y 短寄存器，无论是全部的 24 位寄存器（只视为有符号整数），或是作为部分的寄存器 1，4，8，12，16，或 20 位宽的（视为带符号或无符号的整数，除了 1-位的变量--它只能是无符号）。

编译器的合法 L-变量名包含字母 L，后跟一个范围为 0 到 8191 的整数，共计 8192 个可能的 L 型变量（L0 到 L8191）。

由于变量参考 Turbo PMAC 内存和 I/O 空间的固定位置，L-变量将替换 M 变量，而且 L 变量定义将是与 M 变量的定义完全一样的。保留 M 变量定义也是完全可以接受的。你可能会保留 M 变量的定义用于调试目的，因为 Turbo PMAC 不会接受对一个 L 变量的值或定义进行查询的命令。通常情况下，您将使用相同的 L-变量和 M 变量的定义。

例如，JOPTO 端口上的机器输出 1 和机器输入 1 通常是由未编译的程序的如下定义引用：

```
M1->Y:$078F02,8          ; Machine Output 1
M11->Y:$078F02,0         ; Machine Input 1
```

对于编译的 PLC 程序，您可以创建等价于 M 变量的定义：

```
L1->Y:$078F02,8 ; Machine Output 1
L11->Y:$078F02,0 ; Machine Input 1
```

编译的 PLC 中的一个小例程使得机器输出 1 跟随机器输入 1，将是：

```
IF (L11=1)
    L1=1
ELSE
    L1=0
ENDIF
```

你可以通过一个 L 变量访问一个程序语句中的一个寄存器，然后通过一个整数 M 变量或 I-变量访问另一个程序声明中相同的寄存器甚至寄存器相同的部分。混合 L-变量访问和 P 或 Q-变量访问一个 P 或 Q 变量寄存器会产生荒谬的结果，因为 P 和 Q 变量访问始终把寄存器作为一个浮点数。

F-变量：长浮点指针

F-变量是长（48 位）寄存器的指针。如果 F-变量定义是一个 L 格式（例如 **F1-> L:$10F0**），寄存器被访问为一个 48 位浮点寄存器。如果 F-变量定义是一个 D 格式的变量（如 **F2-> D:$ 88**），寄存器被访问作为一个 48 位有符号整数，但会自动从 Turbo PMAC 的 48 位浮点格式进行转换或转换到其格式，因此它可以在浮点数学中使用。

注意：

使用 F-变量需要 PRO 系列的 PMAC 执行程序（PEWIN32PRO）和 1.938 或更高版本的 Turbo PMAC 固件。

Turbo PMAC 本身不能识别 L-变量或 F-变量，这些变量只对主计算机上的编译器有意义。Turbo PMAC 将拒绝执行包含 L 变量的任何未编译的命令。

注意：

不要混淆 L-变量（它是短-字的编译器指针），和 L-格式的 F-变量和 M-变量（它是长-字变量）。

比较运行时链接指针

相比之下，在用 Turbo PMAC 的 M 变量指针时，在执行一行时且每次执行它时，寄存器分配进行一次。每次变量被访问时，这项分配要求约 600 纳秒额外的计算时间（在一个 100 MHz 的 CPU）。然而，这允许在执行过程中改变 M 变量定义，从而使能诸如间接寻址的技术。

也可以使用 L-变量用于快速整数运算，同时保留了 M-变量定义的运行时的灵活性。（这确实增加了运行时定义访问计算惩罚）。不是为编译器直接定义 L-变量到寄存器，而是通过 **LMOVERLAY{start}，{end}的**编译器指令引用 L-变量的范围到 Turbo PMAC 的 M 变量的定义。此指令必须在实际的编译 PLC 程序之前。例如，**LMOVERLAY10,20** 告诉编译器的是：分配 L10 到 L20 的定义是在运行时间，通过分别使用 M20 到 M10 的定义在执行每个语句的时候进行的，而不是在编译时进行的。

使用 M 变量的定义用于一个 L 变量以及在运行时访问这个定义将允许间接寻址技术，通过使用另一个指针变量实时修改这个 M 变量定义。

浮点与定点数学

利用任何数学的编译 PLC 的每个语句可以执行浮点或整数（定点）数学。在一个浮点语句，使用的所有变量是通过一个 48 位浮点的中间工作格式进行处理的，无论该变量的存储格式是什么。浮点语句可以使用任何的 Turbo PMAC 的 I，P，Q 或 M-变量，和编译器的长的 F-变量的指针。他们不能使用编译器的短定点 L-变量指针。在这些语句中使用的所有常量存储为 48 位浮点值。

在一个整数语句中，使用的所有变量通过一个中间的工作格式（24 位有符号整数）进行处理，而不管该变量的存储格式是什么。整数语句只能利用编译器的短定点 L-变量指针。他们不能使用编译器的长的 F-变量指针，也不能使用 Turbo PMAC 的 I，P，Q 或 M-变量。在这些语句中使用的所有常量被存储为 24 位有符号整数。所有使用的常量和运算的中间值必须符合这些整数的范围：-8,388,608～+8,388,607。在一个整数语句中不可以使用数学函数（例如 SIN，COS）。

请注意，如果一行任何变量之前出现常量（例如，IF（0=P1）...）时，该行被假定为浮点数。这意味着语句 IF（0=L1）是非法的，因为它混合了浮点和定点数据类型。

短字整数运算会比执行同样的长浮点运算操作变得快 10 倍以上，但他们没有浮点运算的动态范围。

如果需要在整数和浮点数据类型之间进行的直接转换，编译的 PLC 提供 ITOF（整数到浮点型）和 FTOI（浮点到整数）的功能。该 ITOF 功能（带一个整数表达式作为其参数）可以用在一个浮点声明中，如：

```
P20=ITOF(L10+L11)*3.14159/P628
```

该 FTOI 功能（带有浮点表达式作为其参数）可以用在一个定点声明，如：

L10=L9+FTOI(P5*100)-5

该 FTOI 函数将浮点表达式的值四舍五入到最接近的整数。它不允许在表达式中嵌套 FTOI 和 ITOF 功能。

注意：

使用 FTOI 和 ITOF 需要“PRO”系列的 PMAC 执行程序（PEWIN32PRO）和 Turbo PMAC 固件版本 1.938 或更高版本。

数组

编译的 PLC 程序支持两种类型的数组：变量数组和寄存器数组。二者都提供了有用的功能。

变量数组

变量数组与 Turbo PMAC 的标准 PMAC I，P，Q 和 M 变量一起工作。数组索引的编号放在括号内，并为指定类型变量指定变量号。决定这个编号的表达式是一个浮点表达式，因此它可以使用 Turbo PMAC 的 I，P，Q 或 M 变量、常量（它将被视为浮点值）和编译器的 F-变量，但它不能使用编译器的 L-变量（除非该值已通过 ITOF 功能被转换为浮点数）。浮点表达式的所得到的值会自动舍入到最接近的整数来选择被使用的变量号。使用这些变量数组语句的一些实例是：

P(P1)=P10*32

P30=I(ITOF(L10)*100+30)*P29

寄存器数组

寄存器数组与编译器的短的 L-变量和长的 F-变量一起工作。这些数组必须在实际的开放式伺服算法开始之前被声明给编译器。在使用中，该数组索引的编号被置于方括号内，并且指定相对该数组的声明开始的地址偏移。决定这个编号的表达式是一个定点的表达，所以它只能使用编译器的 L-变量和适合 24 位有符号整数的范围之内的常量。

L 和 F-变量寄存器数组必须在实际的开放式伺服算法开始之前声明给编译器。这些定义的实例是：

L100->X:$010000[32]

F200->D:$030040[64]

F300->L:$030080[128]

该声明的数组的大小必须是 2 的幂，范围为 2～8192。L-变量寄存器数组始终使用完整的 24 位 X 或 Y 寄存器，对待这些值作为有符号整数。

注意：

使用寄存器数组需要 PRO 系列的 PMAC 执行程序（PEWIN32PRO）和 Turbo PMAC 固件版本 1.938 或更高版本。

运算符

正如任何的 Turbo PMAC 的用户程序，编译的 PLC 可利用下面的数学和逻辑运算符：+（加），－（减），*（乘），/（除），%（取模，余数），&（逐位与），

|（逐位或），和^（逐位异或）。所有这些运算符可以在浮点或整数语句中使用。整数除法将结果舍入为最接近的整数；在分数恰好为 0.5 的情况下，将舍入到下一个更正的整数（例如，-7.5 为-7，7.5 为 8）。

比较符

正如任何 Turbo PMAC 的用户程序，编译 PLC 可在条件语句利用如下的比较：=（等于），>（大于），<（小于），～（约等于[0.5 以内]），!=（不等于），!>（不大于，小于或等于），!<（不大于，大于或等于），!～（不约等于[超过 0.5]）。

～和!～比较符只能用在浮点语句中。请注意，<>，>=和<=比较符，它们可以在某些编程语言中使用，但是不能在编译的 PLC 或其他的 Turbo PMAC 程序中使用。

函数

正如任何的 Turbo PMAC 的用户程序，编译的 PLC 可利用下面的数学函数。注意，这些函数只能用于一个已编译的 PLC 内部的浮点语句：SIN（正弦函数），COS（余弦三角函数），TAN（三角函数正切值），ASIN（三角反正弦），ACOS（三角反余弦），ATAN（三角反正切），ATAN2（特殊 2 个--引数，四象限反正切值），ABS（绝对值），INT（以内的最大整数），EXP（幂），LN（自然对数），和 SQRT（平方根）。

如果 Turbo PMAC 变量 I15 设置为默认值 0，三角函数使用度（°）；如果 I15 设置为 1，他们使用弧度。三角函数每次执行时对 I15 的现值进行评估。

注意：

该 ATAN2 函数使用 Q0 作为第二个参数---余弦参数。第一个参数---正弦参数---是在括号内紧随 ATAN2 其后。所使用的 Q0 是由 ADDRESS 语句在 PLCC 中选择的坐标系的，或是坐标系 1 的 Q0（如果在那个 PLCC 没有 ADDRESS 语句被执行的话）。

速度的优化

正如上面提到的，带 L-变量的定点运算比已编译的 PLC 的浮点数学做相同操作的速度快 10 倍以上（比一个解释的 PLC 做同样的操作快 30 倍以上）。即使使用定点运算，在速度和效率上也有差异。

24 位有符号值的 L 型变量的读取和写入速度是最快的。没有偏移的无符号 1-20 位的变量是第二快的，而有符号的 1- 20 位变量和那些相对 0 位有偏移的是最慢的。该操作越慢，使用 PLC 程序的内存越多。然而，24 位 L 变量比那些小宽度的将使用更多的数据内存。因为在大多数编译 PLC 应用中，速度是比数据内存更值得关注的，通常所有不必是短的而指向一个字的特定部分的 L-变量（如用户缓冲区中所有通用的 L-变量）都是 24 位宽，即使他们不要求全范围。

对有符号的 24 位的 L-变量的一个读或写操作需要 3 个 DSP 指令周期来执行和 2 个程序存储器位置来存储。

对一个小于 24 位（1-20 位）的有符号的 L-变量的一个读操作需要 6 至 8 个 DSP 指令周期，和 5 到 7 个程序存储器位置。对于一个小于 24 位（1-20 位）的无符号 L-变量的一个读操作需要 7 至 9 个 DSP 指令周期和 6～8 个程序存储器位

置。

对一个小于 24 位（1-20 位）的有符号或无符号 L-变量的一个写操作需要 12 至 14 个指令周期和 10～12 个程序存储器位置。

在定点运算中，&，^，|，+，－ ，*运算需要 1 或 2 个 DSP 指令周期。在定点运算中，除法（/）运算名义上需要 82 个 DSP 指令周期。在定点运算中，取模（%）运算名义上需要 76 个 DSP 指令周期。

内存利用率 7

该 DSP563xx CPU 为 Turbo PMAC 采用哈佛架构，有程序或指令存储器和数据存储器的独立区域。编译的 PLC 的实际指令被加载到 P 程序存储器；它使用的所有的数据寄存器是在 X 和 Y 数据存储器。

如果选择了一个选项 5x0 标准内存配置，48K 字的程序存储器可用于编译的 PLC 的机器代码（P：$050000 到 P：$05BFFF）。如果选择了一个选项 5x3 扩展内存配置，432K 字的程序存储器可用于编译的 PLC 的机器代码（P：$050000 到 P：$0BBFFF）。执行程序的下载器会告诉你，你刚编译并下载的 PLC 程序占用了多少字的程序存储器。

对于一般的数据存储器，大多数用户会利用一些 Turbo PMAC 的 8192 个 P-变量用于浮点值。如果只需使用几个打开的寄存器，在 X/Y：$0010F0 - X / Y：$0010FF 的寄存器未被任何固件任务使用，除了在上电或复位时自动归零。

对于大量的额外的数据存储器，建议通过在线 DEFINE UBUFFER 命令使用“用户缓冲区”设置。用户缓冲区占用一些在 X / Y 数据存储器的高端的寄存器。在一个选项 5x0 标准内存配置下，数据存储器的末尾是在 X/Y：$0107FF；如果声明一个 DEFINE UBUFFER 2048，从$010000 到$0107FF 的所有数据存储器被保留用于你自己的目的。在一个选项 5x3 扩展内存配置下，数据存储器的末尾是在 X/Y：$03FFFF；有一个默认的 65,536 字的用户缓冲区，保留从 X/Y：$030000 到 X/ Y：$03FFFF 的所有内存寄存器供您使用。你有责任确保在 UBUFFER 的用于开放式伺服数据存储的寄存器不能用于其他用途。

编译 PLC

该 PMAC 执行程序的编辑器的下载功能将自动编译这些 PLC，并将由此产生的汇编机器代码传送到 Turbo PMAC。编译的过程基本上是对用户不可见的，不需要比下载解释的程序或在线命令有更多的工作。

该下载器在处理编辑器文件中遵守以下规则：

- 下载器会通过一个＃include 指令自动合并那些任何文件引用的内容。如果包含的文件（例如宏替换）的内容被引用在主文件中，必须在使用内容之前使用＃include 指令。
- 该下载器会用一个＃define 指令自动注意到任何宏替换集。必须在文件中使用宏的名称之前使用这样的一个宏替换指令，无论是直接在文件中作出，或是通过一个＃include 指令访问。
- 该下载器会自动识别 L-变量和 F-变量定义，并将它们用于后续编译的代码，但不会这些语句传输到 Turbo PMAC 中，如同对于 M 变量定义语

句那样。

- 下载器识别三种方式的注释分隔符。从一个分号（；）或双斜线（//）到行末之间所有的字符将被忽略；从“斜线-星”(/*)到星形-斜线(*/)之间的所有字符将被忽略。
- 该下载器会尝试编译 OPEN PLCC n 和 CLOSE 之间的所有语句，注释除外。在 OPEN PLCC n 后不需要 CLEAR 命令，因为下载新的编译 PLC 的这个动作会自动删除现有版本。然而，也没有必要为编译器除去 CLEAR 命令。
- 如果它在要下载的文件遇到任何编译的 PLC，下载器自动删除 Turbo PMAC 中所有的编译的 PLC。因此，有必要每次都一起编译所有的 PLCC 程序。改变单一的 PLCC 程序需要重新编译所有 PLCC 程序。
- 所有其它的命令都被编译器不变地传递到输出文件中。

下载器将编译后的 PLC 代码放在 Turbo PMAC 的激活、但挥发性 SRAM 存储器中。为在一个上电周期或复位时保留这些程序，通过一个 SAVE 命令将他们复制到 Turbo PMAC 的非易失性闪存中

运行编译的 PLC

在下载后下载器会自动激活编译的 PLC。如果 I5 设置为允许这些程序运行，它们将在下载后被立即执行。编译的 PLC 程序可以通过 ENABLE PLCC 和 DISABLE PLCC 命令单独或成组地启用和禁用。这些可以作为在线的命令，或作为运动程序、未编译的 PLC 程序、或编译的 PLC 程序内的缓冲的命令而发出。一个 PLC 程序甚至可以禁用本身。唯一的限制是你不能够从 PLC0 或 PLCC0 内使用 DISABLE PLCC 命令；在这里他们不能保证正常工作。

在上电或复位时，所有现有的 PLC 程序，编译的和未编译的，都被单独启用。如果 I5 被保存为一个值进而允许一个给定的 PLC 程序运行，它将是在上电时准备运行。PLC1 将是在上电或复位后（甚至 PLCC1 之前）执行的第一个 PLC。很多人用这个作为复位的 PLC，执行一次并禁用本身，以防止重复执行。这个 PLC 程序可以用来防止其他 PLC 程序在上电时立即执行，通过 DISABLE PLC 和 DISABLE PLCC 命令。在这种方式中，上电可作为只是一个启用 PLC 程序的选择来完成。

发送<CONTROL-D>字符是一个快速的方法，它可以禁用所有的 PLC 程序，编译的和解释的。

注意：

从来不建议使 PLC 0 或 PLCC0 在上电时运行。因此，不要保存 1 或 3 的 I5 值。相反，I5 保存为 2；然后在 PLC1 中复位 PLC，使用一个命令序列，如：。

```
DISABLE PLCC 0
DISABLE PLC 0
I5=3
```

PLC0 和/或 PLCC0 然后可以根据需要进行启用。

第十九章 写一个主机的通信程序

如果需要在实际应用中从主机电脑到 Turbo PMAC 进行通讯，必须写一个主机通信程序。在开发中使用的 PMAC 执行程序不是作为一个主机程序用于实际的应用，它的目的只是作为开发工具。

本节介绍如果编写自定义的通讯软件要采取的措施，包括低级别的驱动程序。如果使用泰道的 PCOMM32 或 PCOMM32PRO 通信库用于 Microsoft Windows 操作系统，这项工作已经由库函数完成的，没有必要处理本节所介绍的细节（尽管熟悉本节所介绍的基本问题仍然是一个好主意）。如果使用这些通信库之一，请参考该库的手册。

从根本上说，编写的主机通信例程发送 ASCII 编码字符的字符串到 PMAC 以及从 PMAC 接收字符串。创造一些低级别的例程来发送和接收单个字符和文本行；这些将被重复调用；指定不同的文本字符串来读取或写入。

注意：

通信的基本概念涵盖在与《Turbo PMAC 通讯》的部分。该学习此部分之前应对那部分进行复习。

如果存在双端口 RAM 选项，进行通信是由二进制编码的数字数据传输通过 DPRAM 的共享寄存器来完成。

19-1 Turbo PMAC 的命令/响应格式

在试图编写通信例程之前，了解 Turbo PMAC 的基本命令/响应格式是很重要的。一般情况下，发送到 Turbo PMAC 的命令以及 PMAC 的响应都是 ASCII 文本字符串，通过一个回车控制符终止（虽然在双端口 RAM，它们是被空字符终止）。也可以发送特殊的控制字符的命令（如<CTRL-A>中止所有运动），他们不需要任何文本字符或回车。

与 Turbo PMAC 进行通信的基本操作是发送一个命令，然后监视端口直到 Turbo PMAC 的响应准备好了，读取响应字符，并解释它们。

响应类型

根据给出的命令，有三种类型的响应：无-行、单行和多行。如果 Turbo PMAC 进行正确的安装而且精心构建你的通信程序，那么就可以编写简单而有效的程序来处理所有这些情况。对于高效和强大的通信程序，最重要的因素是设置 I3= 2。这会导致对所有有效文字命令的整体响应被一个<ACK>字符终止（使它成为一个独特的传输-终止的字符），和对所有无效的命令的响应是一个<BELL>字符。以下说明假定为一个 I3=2 的设置。

无-行响应

一个到 Turbo PMAC 的命令，如 J+（正的慢跑），将不要求一个文本响应，那么如果它是有效的，将只得到一个<ACK>字符响应；或者如果它是无效的，得到一个<BELL>。

单行响应

一个到 Turbo PMAC 的命令，如 P（报告定址的电机位置），会要求一个单行的文本响应，如果有效，将得到文本行响应，该响应由<CR>终止，后跟一个<ACK>。如果它是无效的，Turbo PMAC 将通过一个<BELL>字符来响应命令。

多行响应

一个到 Turbo PMAC 的命令，如 **LIST PROG 1**（报告运动程序 1 的内容），会要求一个多行文本响应，如果该命令有效，将获得多个文本行响应，每一个响应都由<CR>终止，后跟一个单<ACK>字符在结尾。如果它是无效的，Turbo PMAC 将用一个<BELL>字符响应来代替。

变型

报告错误

如果 Turbo PMAC 的变量 I6 被设置为 1 或 3，在 Turbo PMAC 对一个无效命令响应一个<BELL>字符后，它会发送一个错误代码，如 ERR004，通知主机为什么该命令被拒绝。

校验和

如果 Turbo PMAC 变量 I4 被设置为 1 或 3，Turbo PMAC 将报告对主机的命令和对主机的响应行二者进行计算而得到的校验和。命令的校验和字节被立即返回在<ACK>字符后。每个响应行的校验和字节被立即返回在终止该行的<CR >字符后。需要注意的是，Turbo PMAC 并不比较任何校验和的有效性；它是由上位机（主机）对命令和响应的校验和的有效性进行评估。

也可能在执行发送到 Turbo PMAC 的文本命令之前，评估该命令的校验和。在命令的字符已被发送后，但在导致执行它的**<CR>**字符被发送之前，主机计算机可以发送**<CTRL-N>**命令。这将导致 Turbo PMAC 计算挂起命令的校验和并将校验和字节返回到主计算机。如果主机评估的校验和为有效的，它可以发送**<CR>**导致执行该命令。如果它计算的校验和为无效，它可以发送**<Ctrl-X>**字符来清除该命令，然后重新发送该命令。如果 I4 被设置为 1 或 3，Turbo PMAC 也将报告命令的校验和并作为确认的一部分。

校验和是主要用于串行通信；他们不支持用于双端口 RAM 通信。

清除端口

发送**<Ctrl-X>**字符通过一个端口到 Turbo PMAC 将引起 Turbo PMAC 来清除该端口的命令和响应队列。在任何时间如果通讯软件不知道该端口的状态，那么可以在命令之前发送一个**<Ctrl-X>**，比如当启动该程序时，在一个通讯错误后，或与其他独立通信的程序共享端口时。如果 I63 被设置为 1，当完成清除端口后，Turbo PMAC 将响应一个<Ctrl-X>字符。清除程序也应尝试从端口寄存器本身读取一个字符，防止字符已经从响应队列移动到寄存器里。

未经请求的消息

有可能 Turbo PMAC 直接将文本字符串发送到主机计算机而无需主机首先发送一个命令，这可以从一个运动或 PLC 程序中使用 **SENDx** 或 **CMDX** 语句。（如果不使用这些语句，那么 Turbo PMAC 通信到主机仅是响应一个主机命令。）这些未经请求的信息是一个强大的功能，但如果不做好充分准备的话，有可能混淆通信例程，因为当主机期望一个命令的响应时，可能会收到此消息。

如果 Turbo PMAC 变量 I64 设置为 1，任何未经请求的消息前面会有一个<CTRL-B>字符。这就允许主机可以区分命令的响应和未经请求的消息，并采取相应的行动。如果想要得到未经请求的消息，不使用<Ctrl-X>清除端口是一个好主意，因为那可以很容易地清除一个未经请求的消息。

19-2 串口通信

当通过其主要或辅助串口通信到 Turbo PMAC 时，通常使用主计算机的一个 COM 端口。在一个标准中，这些通常是内置的 COM1 和 COM2 RS-232 端口，但它们也可以在扩展端口。大多数 COM 端口，甚至在其他类型的计算机上，都使用相同的芯片，所以他们通常在主机端有相同的寄存器。

设置接口

在每次启动系统时，主机计算机的串行端口必须通过 Turbo PMAC 的设置被初始化才能进行正常接口。这是通过一些向计算机的 I/O 空间的简单字节-写命令来完成的。

基址

首先你必须知道的是 COM 端口在计算机 I/O 空间的基址。在标准 PC 机上，COM1 端口的基地址是在 3F8 hex（十六进制）（1016 十进制），COM2 端口的为 2F8（十六进制）（760 十进制）。

波特率

您必须在主机计算机中设定波特率计数器以匹配 Turbo PMAC 的波特率（这个是由 I54 保存的值用于主串口来确定的，同时 I53 用于辅助端口）。必须给主机计算机的波特率计数器设置一个值，该值等于 115,200 除以波特率（如对于 9600 的波特率，这个值是 115,200/9600=12）。下面的程序段说明了如何可以做到这一点：

```
outportb (combase + 3, 131)                ; /* Put COM port in setup mode */
baud_count = 115200/baud                   ; /* Calculate counter value */
outportb (combase, baud_count              ; /* Write to low byte of counter */
outportb (combase + 1, baud_count/256 )    ; /* Write to high byte of counter */
outportb (combase + 3, 3)                  ; /* Put COM port back in normal mode */
                                             /* with 8 bits, 1 stop bit */
```

该命令 outportb 是一个字节-写命；combase 是基地址，baud 是波特率即每秒的位数。

在最初设置中来计算“时间限制”值是一个好主意，“时间限制”与波特率和主机电脑的速度相关。当主机轮询 Turbo PMAC 看它是否已准备好进行通信时，给计数器递增一次。如果计数器超过了“时间限制”值，主机应放弃与 Turbo PMAC 通信。根据设置的不同情况，应该要么稍后再试（如等待一些异步通信时），或者假设有一个错误条件。

发送一个字符

在查询通信时，主机必须看到串行接口寄存器的两个状态位（写-就绪位）变为 1 后，然后才能写一个字符到串行输出端口。这两个位是{基地址+5}的 5 位和{基地址+6}的 4 位。做到这一点的一个示例 C 代码段是：

```
i = 0                                                    ; /* Reset counter */
while (i++<timeout && (inportb(combase+5)&32==0)         ; /* Loop until bit true */
while (i++<timeout && (inportb(combase+6)&16==0)         ; /* Loop until bit true */
if (i < timeout) outportb(combase, outchar)              ; /* Send char. unless timed out */
```

发送一个整行只涉及重复调用这个例程，每次用不同的 outchar（输出字符）。

读取一个字符

要读取串口的字符，主机必须设置端口为读（对于读取一整行也这样），然后查询串行接口寄存器的一个状态位（读-就绪位）；当它为 1 时，字符可被读取。示例 C 代码段做到这一点：

```
i = 0                                                    ; /* Reset counter */
outportb(combase + 4, 2)                                 ; /* Set port for input */
while (i++<timeout && (inportb(combase+5)==0)            ; /* Loop until bit true */
if (i < timeout) inchar = inportb(combase)               ; /* Get char. unless timed out */
disable()                                                ; /* Disable interrupts */
outportb(combase + 4, 0)                                 ; /* Set port for output */
enable()                                                 ; /* Re-enable interrupts */
```

为了能够通过这个例程读取一个整行，只需要在该行的开头和结尾转换端口状态端口。

19-3 ISA/ PCI 主机端口通信

主机端口结构

PMAC 的主机接口（直接用于 ISA 和 PCI 总线的通信），占据了主机 I/O 空间的 16 个-地址区域的 11 个连续的地址（它不是内存映射）。在主机端，这些寄存器是通过字节-写和字节-读命令进行访问，如 outportb，inportb，outp 和 inp。

在 PMAC 方面，PMAC 固件维护直接访问这些寄存器，以响应来自主机的命令。

对于 ISA 总线，对于这些主机 I / O 空间的 11 个寄存器的位置（“基址”）进行选择，是通过 Turbo PMAC 板的跳线 E91-E92，E66-E71 的设置和 Turbo PMAC2 板的 S1 DIP 开关的设置完成的。参考《与 Turbo PMAC 通信》部分或个别的硬件参考手册，获取实际的设置信息。这些寄存器的地址范围从基址到基址+10。

对于 PCI 总线，这些寄存器的基址的位置由主机计算机的操作系统自动确定的。请参考操作系统文档，确定操作系统将这些寄存器放在哪里。如果使用泰道用于 Microsoft Windows 操作系统的 PCOMM32PRO 库，请查阅 PCOMM32PRO 的说明文件以得到在这种环境中确定主机-端口的基地址。

寄存器功能

所有这些 11 个寄存器在主机通讯都有自己的功能，虽然只有几个是常用的，而有些是根本不使用的。寄存器的功能有：

Base + 0:	中断控制寄存器
Base + 1:	命令向量寄存器
Base + 2:	中断状态寄存器
Base + 3:	中断向量寄存器
Base + 4:	（未使用）
Base + 5:	高-字节数据发送和接收
Base + 6:	中间字节数据发送和接收
Base + 7:	低字节数据发送和接收
Base + 8:	中断控制器命令字 0
Base + 9:	中断控制器命令字 1
Base + 10:	中断确认字

查询简单通信的寄存器

只通过这些地址的其中两个就可以完成基本的查询通信。{基地址+7}保存传到 PMAC 或来自 PMAC 的每个字节（字符）。读和写寄存器是分开的，所以您不必担心在另一个方向发送时会覆盖掉一个字符。

{基地址+2}保存握手状态位；尽管这就是所谓的中断状态寄存器，它可以用来与主机进行查询通信： PMAC 准备让电脑向它写一个字符时，写-就绪位（1 位）为真；PMAC 准备让 PC 读取一个字符时，读-就绪位（0 位）为真。

设置端口

对于主机端口不需要真正的设置，但最好是向高字节和中间字节寄存器写一个零值来清除它们。下面的示例 C 代码段执行此操作：

```
outportb (combase + 5, 0)          ; /* Clear high-byte register */
outportb (combase + 6, 0)          ; /* Clear mid-btye register */
```

在初始设置中计算与主机的速度相关的“时间限制”值，是个好主意。当主机查询 PMAC 看它是否已准备好进行通信时，一个计数器递增；如果计数器超过了“时间限制”值，主机应放弃跟 PMAC 通信。根据不同的设置情况，应该要么只是稍后再试（如等待一些异步通信时），或者假设有一个错误条件。

发送一个字符

要发送一个字符，主机需要等待写-就绪位变为真，则字符写入到输出端口，如此示例 C 代码片段所示：

```
i = 0                                                    ; /* Reset counter */
while (i++<timeout && !(inportb(combase+2) & 2))         ; /* Loop until bit true */
if (i < timeout) outportb(combase+7, outchar)            ; /* Write character */
```

读取一个字符

要读取一个字符，主机需要等待读取-就绪位变为真，然后读取输入端口的字符，如本示例 C 代码片段所示：

```
i=0;
while (i++<timeout && !(inportb(combase+2) & 1))         ; /* Loop until bit true */
if (i < timeout) inchar = inportb(combase+7)             ; /* Read character */
```

ISA / PCI 中断

Turbo PMAC 的 ISA 总线和 PCI 总线的版本有一个内置的可编程中断控制器（PIC），可以用来让 Turbo PMAC 通过总线中断主机。每个控制器都有其自己的一组信号可用于中断主机计算机，其中一次可使用多达 8 个。用于中断的信号的详细描述示于每个控制器的硬件参考手册中；对于目前现有的控制器的信号被总结如下。

ISA 总线的 Turbo PMAC-PC 机使用一个市售的 8259 PIC 芯片。所有其他的 Turbo PMAC 设计将（类似于但不完全相同于 8259 的）PCI 逻辑集成到集成总线逻辑 IC 中。差异如下所述。

主机中断线路

ISA 总线

对于 ISA 总线的 Turbo PMAC 板，ISA 总线上使用的中断线是通过 Turbo PMAC 的一个跳线选择的。下表列出了每个控制器的可能性。在任何给定的时间，控制器的这些跳线其中只有一个应该是 ON。

ISA-bus Interrupt	IRQ2	IRQ3	IRQ4	IRQ5	IRQ7	IRQ10	IRQ11	IRQ12	IRQ14	IRQ15
Turbo PMAC-PC	E86	E81	E82	E83	E84	E80	E79	E78	E77	E76
Turbo PMAC2-PC	x	x	x	x	x	E7	E8	E9	E10	E15
Turbo PMAC2-PC Ultralite	x	x	x	x	x	E7	E8	E9	x	E10

PCI 总线

在 PCI 总线，操作系统在 PC 上电或复位时自动地选择控制器使用的中断信号线。请查阅操作系统文档，获取关于操作系统如何确定选择哪个中断的详细信息。如果使用泰道的 PCOMM32PRO 库用于 Microsoft Windows 操作系统，请查阅 PCOMM32PRO 的文档获取处理在这种环境的中断的说明。

中断源信号

下表显示了哪种 Turbo PMAC 信号可用来产生各种 ISA 和 PCI 的 Turbo PMAC 板卡的中断。PIC 的 IRn 线的信号是在 PIC 寄存器的 Bit n。每个信号都有下面的简要说明。

PIC Input Line	**IR0**	**IR1**	**IR2**	**IR3**	**IR4**	**IR5**	**IR6**	**IR7**
Turbo PMAC-PC, Turbo PMAC-PCI	IPOS	BREQ	EROR	F1ER	HREQ	EQU1*/ EQU5/ AXEXP1 MI1**	EQU2/ EQU6/ AXEXP0/ MI2**	EQU3/ EQU7/ EQU4/ EQU8**
Turbo PMAC-PCI Lite	IPOS	BREQ	EROR	F1ER	HREQ	EQU1*/ AXEXP1 MI1**	EQU2/ AXEXP0/ MI2**	EQU3/ EQU4**
Turbo PMAC2-PC, Turbo PMAC2-PCI	IPOS	BREQ	EROR	F1ER	HREQ	EQU1*	EQU5	WDO
Turbo PMAC2-PCI Lite	IPOS	BREQ	EROR	F1ER	HREQ	EQU1*	EQU2	WDO
Turbo PMAC2-PC Ultralite, Turbo PMAC2-PCI Ultralite	IPOS	BREQ	EROR	F1ER	HREQ	CTRL0*	CTRL1	WDO
*这个信号可以由固件使用用于双口 RAM 的 ASCII 通信中断。 **其中的一个是通过 Turbo PMAC 板的上的跳线选择的								

IPOS（就-位）：如果 Turbo PMAC 确定定址坐标系中所有电机都“就-位”（即到达位置），就-位信号为真。对于一个电动机若要就-位，下面的条件必须满足：伺服回路必须关闭，预期速度必须为零，运动计时器必须关闭（它必须在某种不确定的等待，而不是在一个运动、dwell、或延时），跟随误差幅度必须小于 Ixx28。对于（I7+1）个连续的后台扫描，这些条件必须为真。

BREQ（缓冲区请求）：当一个打开的运动程序缓冲区，特别一个是旋转缓冲去，已准备好接受 PC 输入另一行时，缓冲区-请求信号为真。对于固定的运动程序缓冲区（PROG），就绪状态由 I18 控制；对于旋转程序缓冲区（ROT），就绪状态由 I16 和 I17 控制。更多的细节请参阅《编写和执行运动程序》部分中的旋转运动程序缓冲区。

EROR（致命/故障跟随误差）：当定址坐标系上的任何轴超过其由 Ixx11 设置的致命故障跟随误差限制时，致命故障跟随误差信号为真。

FlER（警告跟随误差）：当定址的坐标系中的任何一个轴超过 Ixx12 设置的警告跟随误差限制时，警告跟随错误信号变为真。

HREQ（主机请求）：当 Turbo PMAC 是准备要通过 ISA 或 PCI 总线读取或写另一个字符时，主机请求信号为真。写到 Turbo PMAC 的基地址来选择：主机请求信号是只反映写-就绪，只读-就绪，或两者兼而有之。写一个 0 的字节值到 Turbo PMAC 基址将不产生主机请求信号；写一个 1 使能该信号只用于主机读-就绪（产生一个中断）；写一个 2 使能信号只用于主机写-就绪；写一个 3 使能信号用于两者。

EQUn（通道 n 的位置比较）：该 EQUn 线包含了 Turbo PMAC 的伺服通道 n 的位置比较输出的状态。这条线可以通过编码器计数器的比较电路的自动动作进行触发切换，或通过直接写入该电路的控制位实现触发切换，这就允许使用该信号作为 Turbo PMAC 程序软件产生的一个中断。

在一个 Turbo PMAC，EQUn 线可用于软件产生的中断，通过写入到通道的 EQU 出转换-有效位中实现（通道的控制/状态字的 13 位，建议的 M 变量 Mxx13）。当编码器计数器和比较寄存器的值不匹配时，该位设置为 1 可以使得信号高电平（它可以产生一个中断）；将该位设置为 0 使得信号为低电平。

在一个 Turbo PMAC2 中，这条线可以通过直接-写功能用于软件产生的中断。这是一个两步的过程。首先，将要输出的值写入到直接-写值的位（通道的控制字的 12 位，建议的 M 变量 Mxx12）。接着，写一个 1 到直-写使能位上（通道的控制字的第 11 位，建议的 M 变量 Mxx11）。

对于建议 M 变量的定义，使用 EQU1 以触发软件生成的中断的代码是：

```
M112=1          ; Prepare to set EQU1 high
M111=1          ; Enable writing of M112 value to EQU1 (M111 is self-clearing)
```

要在下一次中断之前，清除 EQU1，代码是：

```
M112=0          ; Prepare to set EQU1 low
M111=1          ; Enable writing of M112 value to EQU1 (M111 is self-clearing)
```

如果 I56 和 I58 都设置为 1 来启动 DPRAM ASCII 与中断进行通讯，那么一个（非 UltraLite 的）Turbo PMAC 在每次它装载一个响应线到 DPRAM 的 ASCII 响应缓冲区时，将在 EQU1 线上产生一个中断。这个功能有效时，EQU1 线不能用于其他用途。

AXEXPn（轴扩展 EQU 线）：一个 Turbo PMAC-PC 板的 AXEXP0 和 AXEXP1 线是输入，他们可以从 Acc-24P 轴扩展板上输入到选定的 EQUn 线上用于中断的目的。输入到哪个 EQU 线是通过 Acc-24P 主板的跳线来确定的。AXEXPn TTL 电平线也可以通过其他外部信号驱动。

MIn（机器输入 n）：该 MI1 和 MI2 线是 Turbo PMAC 控制器上通用的 5V-24V 输入。通过所选的适当跳线，它们可以被用来中断主机。

CTRLn（控制输出 n）：该 CTRL0 和 CTRL1 信号一般都是 Turbo PMAC2 控制器未使用的线。在 Turbo PMAC2 UltraLite 的控制器上，它们可以用于软件产生的中断。通过分别将 X：$078403 的 8 和 9 位设置为 1 将他们作为输出；然后他们的检测可以改变，通过分别写到这些线的反转-控制位（X：$078407 的 8 和 9 位）来实现。

如果 I56 和 I58 都被设置为 1 以使能中断与 DPRAM ASCII 通讯，那么一个 Turbo PMAC2 UltraLite 将在每次它装入一个响应线到 DPRAM ASCII 响应缓冲区时，会在 CTRL0 线上生成一个中断。

WDO（看门狗输出）： 当 Turbo PMAC 看门狗定时器跳闸并关闭该卡时，看门狗信号为真。

集成的中断控制器

所有 Turbo PMAC2 板和所有 PCI 总线的 Turbo PMAC 板（1 和 2）的中断控制器功能被集成到总线接口逻辑中。它是 8259 之后的模型，但更简化和直接。

中断控制器结构

该 PIC 表现为在 PC 的 I/O 空间的 4 个 8 位寄存器。这些寄存器的实际地址取决于 Turbo PMAC 在 PC 的 I / O 空间的基地址（基址）的设置。对于 ISA 总线的 Turbo PMAC，这个基址是由控制器上的跳线（Turbo PMAC）或 DIP 开关（Turbo PMAC2）设定的。对于 PCI 总线的 Turbo PMAC，这个基址是由 PC 的操作系统在 PC 的初始化过程中设置的。

PIC 的这四个寄存器分别是：

- 基址+8：中断状态寄存器
- 基址+9：中断信号状态寄存器
- 基址+10：中断屏蔽控制寄存器
- 基址+11：中断边沿/电平控制寄存器

例如，对于出厂默认的 528（210 十六进制）的 ISA 总线基地址，这四个寄存器是在 536-539（218-21B 十六进制）。

每个寄存器的这八位（0 – 7）代表连接到 PIC 线 IR0-IR7 的八个中断源信号的状态或控制。

中断控制器寄存器

***中断状态寄存器*（基地址+8）：**该寄存器，当读取时，给出了这 8 个中断的状态。它会被任何中断服务程序读取以找出哪个信号中断了 PC。位值为 1 表示匹配信号的一个中断；0 表示没有中断。一个中断信号在它显示这个寄存器的一个中断之前必须是“未屏蔽”。写入该寄存器的动作将清除任何边沿-触发的中断，不管是写入什么值。

***中断信号状态寄存器*（基地址+9）：**此寄存器，在读取时，显示 PIC 的八个信号输入的状态。这在中断服务程序中是没用的，但提供了一种快速简便的方法用于查询这些信号的状态而对 Turbo PMAC 没有任何的开销。

***中断屏蔽控制寄存器*（基地址+10）：**该寄存器允许 PC 屏蔽掉用户不想中断 PC 的中断信号。在 ISA 总线的 Turbo PMAC-PC，向该寄存器的位写入 0 将使能相应的信号来中断 PC；写 1 将禁止（屏蔽）那个信号。在所有其他的 Turbo PMAC，写 1 将使能相应的信号来中断 PC；写 0 将禁止（屏蔽）那个信号。该寄存器可以在任何时间读取以找出哪些信号被屏蔽而又不会影响屏蔽。

***中断边沿/电平控制寄存器*（基地址+11）：**该寄存器允许 PC 控制一个中断信号中断 PC 是通过边沿触发还是电平触发。向这个寄存器的位写 1 设置相应的信号为电平触发的中断；写 0 设置那个信号为边沿触发的中断。默认的是边沿触发的中断，而且通常更有用。

如果一个中断是边沿触发的，为了让该信号来产生另一个中断到 PC（在 PC 通过写到基址+8 而清除中断后），那么信号必须变低，然后再变高。如果一个中断是电平触发，如果在 PC 机清除中断后，信号仍然是高电平的，它会立即再次中断 PC。

离散中断控制器

Turbo PMAC-PC 为 ISA 总线使用一个离散的 8259 中断控制器，等同于 PC 本

身的中断控制器所使用的电路。以下部分说明在 Turbo PMAC 使用此 IC 的步骤；更详细的信息可以在任何 8259 IC 的文档中找到。

初始化中断控制器

要开始，请写入到 Turbo PMAC PIC 的初始化命令字（ICW）来正常设置 PIC。虽然此 IC 是在 Turbo PMAC 中，但是它被映射到 PC 的端口空间中，作为在 Turbo PMAC 的基地址+8 和+9 的两个寄存器。

要做到这一点，请执行下列步骤：

1、写一个字节 17（十六进制）到[PMAC 基址+8]。这将设置 ICW1 为边沿触发中断。
2、写一个字节 08（十六进制）到[PMAC 基址+9]。这将设置 ICW2。
3、写一个字节 03（十六进制）到[PMAC 基址+9]。这将设置 ICW4 为 8086 模式操作。
4、写一个字节 FF（十六进制）到[PMAC 基址+9]。写到操作控制字 1 中来屏蔽所有八个中断进入 Turbo PMAC 的 PIC。

清除屏蔽中断

当您准备好接受来自 PMAC 的中断，清除你想要进入 PMAC PIC 的屏蔽中断。你写一个单字节的参数到{PMAC 基址+9]，其中 1 表示屏蔽的中断；0 表示非屏蔽的中断。例如，为了单独清除 IR4 的屏蔽，参数是 EF（十六进制）；为单独清除 IR5 的屏蔽，该参数将是 DF（十六进制）。

在这一点上，清除 PIC 正在使用的中断屏蔽。首先，禁用 PC 中断（TurboC 的命令：disable（）;）。接着，读取在 I/O 端口地址 21（十六进制）的当前屏蔽字，通过一个如这样的命令：

ch = inportb(0x21)

然后清除你将用的新的中断的屏蔽，通过在当前的屏蔽字和只启用新的中断的屏蔽字--- EF（十六进制）用于 IRQ4，F8（十六进制）用于 IRQ3---之间执行一个逐位 AND。用于这样的 C 命令是：

ch = ch & 0xef

由此产生的新的屏蔽字是被写回到 I/O 端口地址 21（十六进制），通过：

outportb (0x21, ch)

最后，重新启用 PC 中断（TurboC 的命令：enable（）;）。这样就完成了设置过程。

使用中断

为了应对在实际使用中的中断，写一个中断服务程序。究竟如何做到这一点取决于操作系统和编程语言。更多细节咨询这些项目的文档。

PCOMM32 库支持

泰道的 PCOMM32 和 PCOMM32PRO 编程库用于 Microsoft Windows 操作系统，并支持 ISA 和 PCI 中断，并通过其程序自动处理设置。详细信息请查阅这些库的文档。

19-4 VME 总线通讯

一个 VME 总线的 Turbo PMAC 通信在 VME 总线上作为从属设备，通过一组 16 个 8 -位的“邮箱寄存器”，每个都可以容纳一个字符，可以从总线和 Turbo PMAC 处理器进行访问。任选地，也有一个双端口 RAM 的集成电路带 8192 个共用的 16 位寄存器，可用于 ASCII 文本通信或二进制值的传输。

设置 VME 通讯

通过邮箱寄存器和双口 RAM 进行通讯必须设置，通过使用 10 个 I 变量：I90 - I99。如果需要使用的是这些变量的默认设置外的设置，必须使用另一个通讯端口（通常是一个串口）设置这些，然后 VME 总线通信才能成为可能。

PMAC 的执行程序有一个菜单，可以自动设置这些变量来响应所需的地址和一些其他设置，所以没有必要知道这些变量的详细信息。该软件参考手册有详细的说明来设置这些变量。

VME 邮箱寄存器通讯

注意：

通常情况下，Turbo PMAC-VME 用户购买选项 2 的双口 RAM，并使用双口 RAM 的 ASCII 通讯功能，而不是本节描述的 ASCII 邮箱通信。ASCII DPRAM 通信更容易和更快。详细信息请参阅选项 2 的 DPRAM 手册。

与 Turbo PMAC 在 VME 总线上进行通信是不同在 RS232/422 端口进行通信的。当在 VME 总线进行读取和写入 PMAC-VME 时，您必须使用 16 个邮箱寄存器。邮箱寄存器只是一组 16 个 8 位寄存器，是可从 VME 总线寻址的，起始地址是在 PMAC-VME 卡的基地址+1。也就是说，如果我们选择了一个 PMAC 的$7FA000 的基地址，第一个邮箱寄存器（邮箱寄存器＃0）是可以在位置$7FA001 进行访问的。第二个邮箱寄存器（邮箱寄存器＃1）位于$7FA003，第三个是在$7FA005，依此类推，直到第 16 个邮箱寄存器（邮箱寄存器＃15）是在$7FA01F。正如你所看到的，邮箱寄存器位于奇地址，起始位置是 Turbo PMAC 的基地址+1。我们将首先讨论，通过实例，如何通过这些邮箱寄存器发送命令到 PMAC。

通过邮箱寄存器发送命令到 Turbo PMAC-VME

当命令被发送到 Turbo PMAC，这些邮箱寄存器被写入。这是相对简单的，但是这两个规则必须遵循：

1、当发送一个命令到 PMAC 时，永远不要写入到邮箱寄存器＃1（在我们上面的例子中这将是位置$7FA003）。该邮箱寄存器有一个特殊的目的。知道了这一点，命令的第二个字符应写入邮箱寄存器＃2（第三个邮箱寄存器在位置$7FA005），依此类推。

2、最后一个写消息的第一个字符（或一个长消息的一组 15 个字符）；即首先写命令中的所有其他字符（以邮箱寄存器＃2 开始），然后写第一个字符到邮箱寄存器＃0（在位置$7FA001）。这样做的原因是，当写入到邮箱＃0 时，一接收到命令行后，PMAC 立即在所有邮箱寄存器中读取，并开始采取行动。

注意：

不要忘了以一个回车<CR>结束所有 ASCII 消息（命令）。控制字符的命令（它不需要一个<CR>），应直接写入邮箱寄存器＃0。

例如：

要发送命令来选择电机＃1 并手动控制它，来执行此操作的两个命令可以合并在同一行，并会是：＃1J+<CR>。这里有五个 ASCII 字符位置，因此我们将写入到五个邮箱寄存器。要发送这个命令，我们将须发送 5 个 VME 写命令。

我们会继续使用前面例子中的 Turbo PMAC 的相同的基地址。下表显示了在我们这样做时邮箱寄存器的内容：

Address	$7FA001	$7FA003	$7FA005	$7FA007	$7FA009	$7FA00B
Mailbox #	0	1	2	3	4	5
Character		---	**1**			

Address	$7FA001	$7FA003	$7FA005	$7FA007	$7FA009	$7FA00B
Mailbox #	0	1	2	3	4	5
Character		---	**1**	**J**		

Address	$7FA001	$7FA003	$7FA005	$7FA007	$7FA009	$7FA00B
Mailbox #	0	1	2	3	4	5
Character		---	**1**	**J**	+	

Address	$7FA001	$7FA003	$7FA005	$7FA007	$7FA009	$7FA00B
Mailbox #	0	1	2	3	4	5
Character		---	**1**	**J**	+	***<CR>***

Address	$7FA001	$7FA003	$7FA005	$7FA007	$7FA009	$7FA00B
Mailbox #	0	1	2	3	4	5
Character	#	---	**1**	**J**	+	***<CR>***

我们先写一个 ASCII1 到位置$7FA005，然后是一个 J 到$7FA007，然后是一个+到$7FA009，然后是一个回车符（ASCII 码 13）到$7FA010，最后是一个＃到$7FA001。

例如：

对于一个 15 个字符或更少的命令行，上面的例子工作得很好（包括添加的用于终止行的<CR>），但如果在命令行中包含超过 15 个字符呢？请记住，只有 15 邮箱寄存器可以被写入（不要忘了，当数据发送到 Turbo PMAC 时，邮箱＃1 将不能使用。）。

发送前 15 个字符（不发送<CR>!），其次是连续的剩余的字符，直到所有的字符都被写入。而在最后一个字符之后，发送<CR>，它告诉 Turbo PMAC 在遇到命令时行动。如果下载一个运动程序，并且程序中的语句之一恰好是下行：

IF(P1=1)DISPLAY"DELTA TAU"<CR>

这里有 27 个字符要发送，因而必须执行 27 个 VME 写命令。下面的表格再次显示信箱寄存器的内容。在写入第一组的 14 个字符后（上面的命令行中的字符 F 到 Y），邮箱寄存器看起来像这样：

Address	$7FA001	$7FA003	$7FA005	...	$7FA01D	$7FA01F
Mailbox #	0	1	2	...	14	15
Character		---	F	.	A	Y

现在，写第一个字符 I：

Address	$7FA001	$7FA003	$7FA005	...	$7FA01D	$7FA01F
Mailbox #	0	1	2	...	14	15
Character	I	---	F	.	A	Y

在这一点，Turbo PMAC 已经采取这些字符进入其命令队列中，但没有通过他们做任何事，因为没有遇到<CR>。它声明所选择的中断级别（默认为 2），并为命令提供一个接收中断向量（默认为$ A0）--这必须确认。现在，发送下面的 11 个字符（D 到 “后跟一个<CR>）：

Address	$7FA001	$7FA003	$7FA005	...	$7FA017	$7FA019
Mailbox #	0	1	2	...	11	12
Character		---	D	.	"	<CR>

最后，发送这第二字符组（也是最后一个）的第一个字符，这是一个”：

Address	$7FA001	$7FA003	$7FA005	...	$7FA017	$7FA019
Mailbox #	0	1	2	...	11	12
Character	"	---	D	.	"	<CR>

Turbo PMAC 再次声称中断为 2 级，并为命令提供一个接收中断向量。由于<CR>已被列入，Turbo PMAC 知道命令行已经完成。Turbo PMAC 现在将这一行插入到先前打开的程序缓冲区中（记住，在这个例子中，我们正在下载一个运动程序到 Turbo PMAC）。

通过邮箱寄存器从 Turbo PMAC-VME 读数据

现在已经使用邮箱寄存器将数据发送到 Turbo PMAC-VME 了，那么接下来确定如何从 Turbo PMAC 读取数据。读取数据将涉及使用由 Turbo PMAC-VME 在 VME 总线产生的中断和中断向量。在下面的例子中，Turbo PMAC 的基地址是在$7FA000 而且 I-变量 I3 被设置为 2（I3 的最佳设置，用于编写主机通信程序）。

从 Turbo PMAC 通过邮箱寄存器读取数据的关键是写值到邮箱寄存器＃1 中，以允许 PMAC 当它要输出的数据时把其数据放在邮箱寄存器中。这是可以提前做到的，从而有效地预-使能 TurboPMAC 的响应。这是我们在所有的下面的示例中使用的策略。

如果没有预先启用，那么只有当期望得到一个即时响应时，才向邮箱寄存器＃1 中写入值，这通常是在确认$A0 中断后（见例子）。如果不这样做，Turbo PMAC 将不会通过$A1 向量而中断。（不预先使能的唯一的真正优势在于可以打断一个长的 Turbo PMAC 响应以发出一个命令。）请注意，如果使用预先-启用的策略，

在上电或复位后，要预先-使能一次。阅读下面的实例之后，请参考下面的流程图。

例子

此命令行刚刚被送到 Turbo PMAC：＃1J+<CR>。此命令行不需要任何数据响应，所以 Turbo PMAC 除了一个确认应答<ACK>外不会有任何数据的响应，标志着确认收到一个有效的命令行（如果发送一个无效的命令，将发送一个<BELL>字符而不是<ACK>）。在这种情况下，Turbo PMAC 将产生一个中断，并同时发送一个中断向量$A0（如我们在 I96 中定义）。在看到这个中断和相应的中断向量后，您（VME 主机或主机）必须正确服务或确认该中断，使 Turbo PMAC 撤销其中断申请（一般情况下，当你服务任何 VME 中断时，中断向量将是可用的。）

那么 Turbo PMAC 将再次中断，这一次是通过中断向量$A1（I96 的值加 1），标志着有一个新的数据在邮箱寄存器需要被读取。现在，读取邮箱寄存器＃0（在$7FA001）拾取由 Turbo PMAC 放在那里的<ACK>字符，但是这是必要的只是验证刚才发送的命令行被 Turbo PMAC 收到作为一个有效的命令。最后，写$00 到邮箱寄存器＃1（位置$7FA003），如果需要的话让 Turbo PMAC 写入新的数据到邮箱寄存器（也可以参考下面的例子中，以更好地理解这一点）。下面的例子演示了如何读取由 Turbo PMAC 写在邮箱寄存器的数据。

例子

现在让我们假设下面的命令已被发送，要求查询电机 1 的位置：＃1P<CR>。当然，Turbo PMAC 将以包含电动机 1 的位置信息的数据来响应。比方说，目前电机 1 的位置为 19.2。我们现在想要读取邮箱寄存器来获取这些信息 Turbo PMAC 已经等着我们。我们做的第一件事是发送命令行和服务 Turbo PMAC 产生的中断（使用$A0 的中断向量）作为一个确认。

在 Turbo PMAC 已经处理了命令并把数据存入邮箱寄存器后，Turbo PMAC 通过一个中断向量$A1 第二次中断了我们。请记住，我们得到第二个中断是因为 PMAC 刚才已经把数据放置到邮箱寄存器了，现在准备好被读取。我们服务这第二个中断，并注意所附的中断向量$A1--告诉我们可以读信箱寄存器中的数据了。

其实，这些寄存器可以以任何顺序读取，但最好是以第一个邮箱寄存器开始读取这些字符直到我们：

1、遇到一个<CR>（表示行的结尾）或

2、遇到一个<ACK>（收到有效的命令行）或

3、遇到一个<BELL>（接收到无效的命令行）或

4、已读取所有 16 个邮箱寄存器（从$7FA001 至$7FA01F）。

这些邮箱寄存器可以根据需要读取很多次，因为 Turbo PMAC 将不写入新的数据到邮箱寄存器（如果 Turbo PMAC 有更多的数据要发送）直到一个$00 的值写入邮箱＃1（在这种情况下，是在$7FA003）。

在这个例子中，Turbo PMAC 将有六个字符等待读取：19.2<CR><ACK>。（我们假设 I-I3 变量被设置为 2。）数据将是在寄存器中，如下所示：

Address	$7FA001	$7FA003	$7FA005	$7FA007	$7FA009
Mailbox #	0	1	2	3	4
Character	**1**	**9**	**.**	**2**	***<CR>***

我们开始读取在$7FA001、邮箱寄存器 0 的字符。我们在邮箱寄存器 4 中看到<CR>，所以我们停止读取，并写一个$00 到邮箱寄存器 1，告诉 Turbo PMAC

可以发送更多新数据。由于 Turbo PMAC 仍必须发送最后的<ACK>，它再次中断了我们，我们将在邮箱寄存器找到：

Address	$7FA001	$7FA003	$7FA005	$7FA007	$7FA009
Mailbox #	0	1	2	3	4
Character	<ACK>	9	.	2	<CR>

现在我们开始和停止在邮箱寄存器 0，因为它包含一个<ACK>。现在，从以$7FA001 的邮箱寄存器＃0 开始，读取这些字符。回想一下，我们所说的在数据发送到 Turbo PMAC-VME 时永远不要写入邮箱寄存器＃1。这是因为只要我们写入到邮箱＃1，Turbo PMAC 将被允许写入新的数据到邮箱寄存器。（顺便说一句，我们写入到邮箱寄存器＃1 的是什么值其实也无所谓，事实上我们写入这个寄存器的动作才是最重要。但是，建议写一个$00 的值到邮箱寄存器＃1，原因在后面给出。）在写一个$00 到邮箱寄存器＃1 之后，我们可能会或可能不会再由 PMAC 中断，这取决于 Turbo PMAC 是否仍有更多的数据让我们读取。

例子

让我们再次假设下面的命令已被发送，以查询存储单元 X:$1000 到 X:$1002 中的内容：**RHX$1000,3<CR>**。比方说，这三个位置包含了值$123456、$789012 和$345678。我们想再次去读取邮箱寄存器，所以我们发送上面的命令行而且服务 PMAC 产生的中断（使用一个$A0 的中断向量）。

在 Turbo PMAC 处理完命令并把数据存入邮箱寄存器后，Turbo PMAC 通过一个中断向量$A1 第二次中断了我们。

请记住，我们得到这第二个中断，是因为 Turbo PMAC 刚才已经将数据放放置在邮箱寄存器中而现在已经准备好被读取。我们服务这第二个中断，并注意所附的中断向量$A1—它告诉我们可以读取邮箱寄存器中的数据了。在这个例子中，Turbo PMAC 将有 22 个字符被读取：123456 789012 345678<CR><ACK>，其中前 16 个是在邮箱寄存器中。（我们假设 I-I3 变量被再次设置为 2）。数据将是在寄存器中，如下所示：

Address	$7FA001	$7FA003	$7FA005	...	$7FA01D	$7FA01F
Mailbox #	0	1	2	...	14	15
Character	1	2	3	.	3	4

我们在邮箱寄存器中读取，以第一个开始，直到我们遇到一个<CR>，<ACK>，<BELL>，或者已经读完所有 16 个寄存器。在这种情况下，前 16 个字符不包含<CR>，<ACK>，或<BELL>。因此，我们在所有 16 个邮箱寄存器中读取以得到 Turbo PMAC 响应的前 16 个字符，然后写一个$00 到邮箱寄存器＃1（在这种情况下，是在$7FA003），让 Turbo PMAC 将下一个数据块放在邮箱寄存器中。Turbo PMAC 通过矢量$A1 再次中断我们，以及在邮箱寄存器剩余的字符串是：

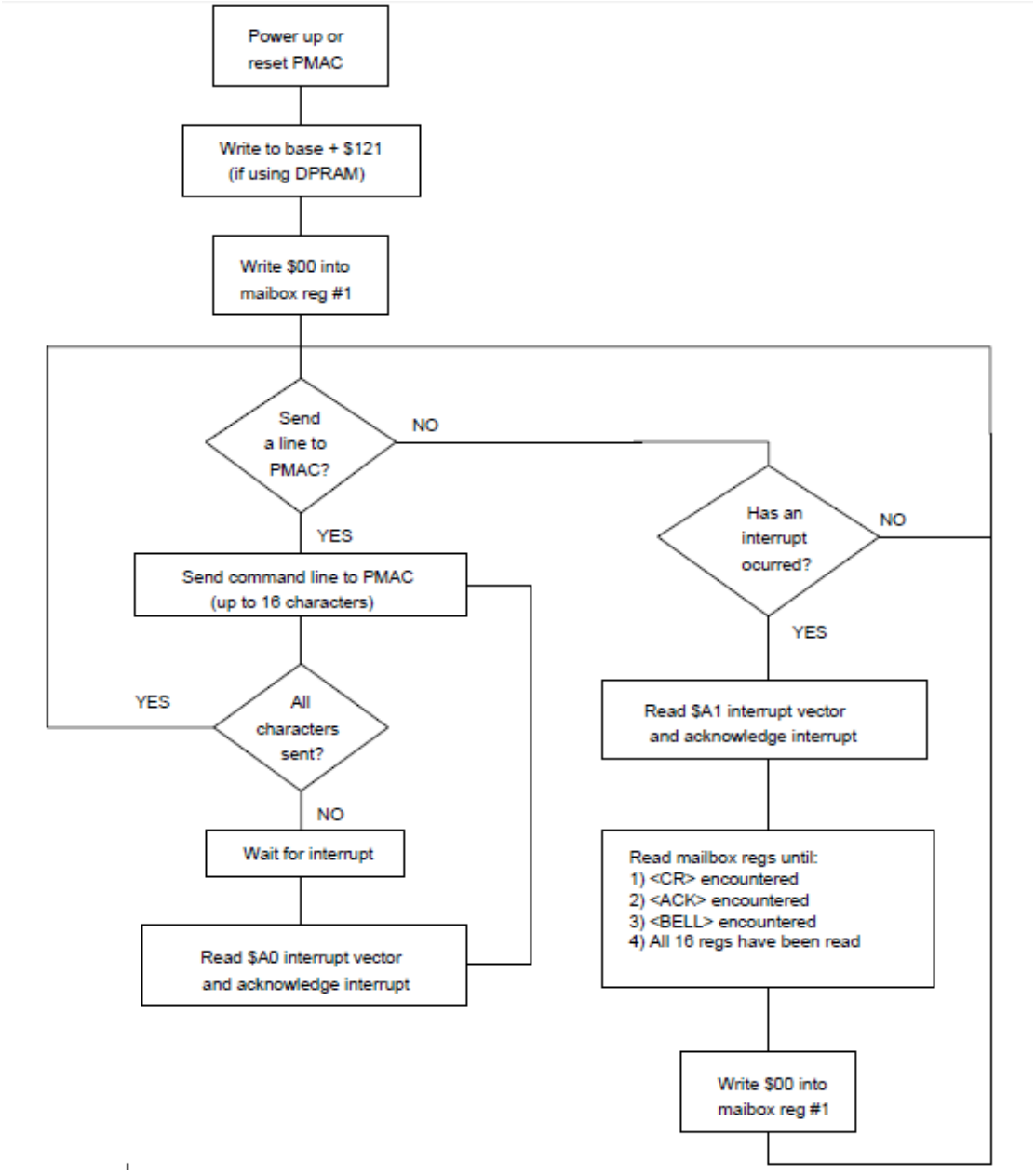
Address	$7FA001	...	$7FA009	$7FA00B	...	$7FA01D	$7FA01F
Mailbox #	0	...	4	5	...	14	15
Character	5	.	<CR>	2	.	3	4

现在，我们再次读取邮箱寄存器，寻找<CR>，<ACK>，或<BELL>。我们在信箱＃4（$7FA009）读取的第五个字符恰好包含一个<CR>，所以我们停止读取并写一个$00 到邮箱寄存器＃1。因为 Turbo PMAC 仍然须发送最后的<ACK>字符，它会再一次通过向量$A1 中断我们以及我们将在邮箱寄存器看到：

Address	$7FA001	...	$7FA009	$7FA00B	...	$7FA01D	$7FA01F
Mailbox #	0	...	4	5	...	14	15
Character	<ACK>	.	<CR>	2	.	3	4

现在，我们停止在第一个字符，<ACK>--它是作为结束传输字符，我们再次写一个$00 到邮箱寄存器 1。由于 Turbo PMAC 现在并没有任何更多的数据被读取，我们不会得到另一个中断（直到我们发出另一个命令或我们执行的 PLC 或运动程序其中一个通过 CMD 或 SEND 命令来输出数据）。

下面的图表总结了使用邮箱寄器在 VME 总线的通信过程。



Turbo PMAC-VME 的通信流程图

当我们要求 Turbo PMAC 列出一个运动程序或 PLC，命令像 LIST PROG 10 或 LIST PLC 0，Turbo PMAC 将有多行数据被读取。我们所要做的只是等待中断发

生，读取邮箱寄存器，写$00 到邮箱寄存器＃1，并等待再次被中断，重复这个过程直到 Turbo PMAC 的所有数据被发送和读取。上图是表示该过程的流程图。

19-5 双端口 RAM 通信

双端口 RAM（DPRAM）是 Turbo PMAC 的可选功能，用于与上位机的高速通信。如果每秒超过 100 个数据项需要在两个方向被转移时，建议购买它。由于这组内存有两个端口，所以 Turbo PMAC 处理器和主处理器都可直接、随机访问双口 RAM 芯片的所有的寄存器。

Turbo PMAC 系列同时支持 8K×16 和 32K×16 的双口 RAM 组。在实践中，Turbo PMAC 的设计只被提供有 8K×16 组，而 Turbo PMAC2 的设计，包括 UMAC 和 QMAC 产品，只提供有 32K×16 组。

Turbo PMAC 家族已预先设定了结构用于在主机和控制器之间的数据传输，而且还允许用户定义自己的数据结构。预先定义的结构包括：

- 控制面板功能
- 电机数据报告缓冲区
- 后台数据报告缓冲区
- ASCII 命令和响应缓冲区
- 数据采集缓冲区
- 后台变量复制缓冲区
- 二进制旋转程序下载

物理配置和连接

在 Turbo PMAC-PC 上，双端口 RAM 选项是一个独立的半槽板，通过 2 个短的带状电缆连接到 Turbo PMAC 的 CPU 板，并有其自己的 ISA 总线连接器。在其他板级的 Turbo PMAC 上，双端口 RAM 是一个上-板的选项，其中 DPRAM 芯片是直接安装在 PMAC 上。

UMAC Turbo 的 CPU 板将 DPRAM 作为一个上-板的选项，以支持可能存在的 PC/104 接口。UMAC Turbo 的 Acc-54E USB/以太网板标配了其自己的双口 RAM 芯片，通过 UBUS 背板接口到 CPU 板。通过 Acc-54E 的 USB 或以太网通信将使用此双口 RAM 芯片，它不能使用在 CPU 板的双口 RAM 芯片。

UMAC Turbo 的新的集成 CPU 和高速通信板集成了 USB 和以太网通信功能。可选的板载双口 RAM 支持这些端口。

该 UMAC-CPCI 的 CPU 板集成了 USB 和以太网通信功能。可选的板载双口 RAM 支持这些端口。以链接 UMAC-CPCI CPU 板到一个 CPCI 背板的（计划的）“桥”板都有自己的双口 RAM 芯片适用于 CPCI-端口通信。

该 QMAC 控制板上有可选的板载 DPRAM，支持 USB 和以太网通信。

19-6 主机地址设置

双端口 RAM 在 Turbo PMAC 的地址空间中有一个固定的地址空间。然而，它在主计算机的地址空间可以根据卡的设置有所不同。该卡在主机的地址的说明是

完全在软件中完成的；没有要设置的跳线或 DIP 开关。

ISA 总线设置

在 Turbo PMAC 中有 2 个设置变量用于 ISA 总线的 DPRAM 在 PC 的内存空间寻址：I93 和 I94。（请注意，标准的“主机”总线通信端口被映射到 PC 的 I/O 空间，并与双口 RAM 的内存地址没有任何关系。）由于 PC 使用字节寻址，在 PC 中必须找到或者创建一个 16K×8 插槽的内存空间用于 8K×16 的 DPRAM。对于 32K×16 的 DPRAM，或者是必须找到 64K×8 插槽的存储空间，或是找到 16K×8 插槽和使用“组”寻址。

请注意，PC/104 总线是与 ISA 总线完全软件兼容的，所以这些指令适用于设置 3U Turbo PMAC（Turbo Stack 或 UMAC Turbo）的 PC/104 接口上的 DPRAM。

通常，在一台 PC 中，使用的内存空间是在 640K（$0A0000）和 1M（$100000）之间，那里不属于标准内存。其他设备也可能会占据这个空间区域。VGA 显示器往往占用的空间是从 640K 到 704K（$0A0000 至$0B0000），BIOS 往往占据从 960K 到 1M（$0F0000 到$100000）的空间。

DPRAM 可以定位在 1M（$100000）和 16M（$FFFFFF）之间，但大多数操作系统无法容忍在其正常的 RAM 寻址中有一间隙，所以 DPRAM 必须放在常规 RAM 的末端后。由于大多数 PC 现在有超过 16M 的内存，这通常是不可行的。

因此，在大多数 PC 中，DPRAM 是定位在 704K（$0B0000）和 960K（$0F0000）之间。默认设置定位它的范围是从$0D4000 到$0D7FFF。

I93 是一个 8 位的值，为 DPRAM 指定 ISA 总线地址位 A23-A16。通常情况下，它被指定为一个 2-数字的十六进制值，这两个数字是与 6-数字的 ISA 十六进制地址的前两位相同，在默认情况下是$0D。

I94 是一个 8 位的值，它控制 DPRAM 在 ISA 总线的寻址。如果只有一个 16K×8 的块保留给 DPRAM，它也指定 ISA 总线地址位 A15 - A14。I94 通常被指定为一个 2 个-数字的十六进制数。

如果在 ISA 总线上的一个 16K×8 的内存是用于 DPRAM，那么第一个数字应设置为等于 6-数字基地址的第三个数字。它可以取值为$0、$4、$8 或$C。对于默认的基址$0D4000，它应该被设置为 4。如果使用一个 64K×8 的存储器，第一位数字应设置为 0。

第二位数字代表寻址模式。它应该被设置为 5 以使用 ISA 总线上的一个 16K×8 的地址空间。它应该被设置为 4 以使用 64K×8 的地址空间。

例如，要使 ISA 总线上从$0EC000 到$0EFFFF 的 16K×8 的内存，I93 应设置为$0E，I94 应设置为$C5。要使用 ISA 总线上从$0C0000 到$0CFFFF 的一个 64K×8 的内存，I93 应设置为$0C，I94 应设置为$04。

为了实现这些设置，并保存以供将来使用，必须使用 SAVE 命令将这些 I-变量值存储在非易失性闪存中，该卡必须复位（$$$命令）。重置卡会将 I93 和 I94 的保存值复制到 RAM 中的 I-变量寄存器中，然后分别复制到激活的位于 X:$07000A 和 X:$070009 的控制寄存器中。

如果一个较大的（32K×16）双口 RAM 使用 16K×8 的内存块，PC 一次只看到四分之一的双口 RAM。按照上面的说明，这将是第一个四分之一（在 PMAC 侧最低的地址）。为了获得在双口 RAM 的其他部分，必须使用“组选择”的过程。

I94 可以通过 1 和 3 位控制组选择，但它仅用于在上电/复位时，所以它不

适合动态的组选择。因此，最好是直接使用在 X：$07000A 的有效控制寄存器。随着建议的 M 变量的定义 M94-> X：$07000A，0,7，以及 I94 设置为上述建议以在上电/复位时选择组 0，下面的公式可以用来选择 4 组中的一（竖线“|”表示逻辑逐位 OR 运算）：

```
M94=I94 | $00      ; Bank 0 (PMAC addresses $060000 - $060FFF)
M94=I94 | $02      ; Bank 1 (PMAC addresses $061000 - $061FFF)
M94=I94 | $08      ; Bank 2 (PMAC addresses $062000 - $062FFF)
M94=I94 | $0A      ; Bank 3 (PMAC addresses $063000 - $063FFF)
```

VME 总线设置

VME 总线上的双口 RAM 的地址设置集成了通用 VME 设置，包括邮箱寄存器，使用变量 I90-I99。

I90 控制 VME 地址修改器。它应该被设置为$39 用于 24 位寻址，或者$09 用于 32 位寻址。

I91 控制地址修改器中的“不在乎”（忽略）位。通常情况下，它应该被设置为$04。

I92 控制 VME 总线地址位 A31 - A24（当使用 32 位寻址同时用于邮箱寄存器和双口 RAM 时）。通常情况下，它被指定为两个十六进制数字，并且它应该与 32 位地址的前两个十六进制数字是相同的。举例来说，如果双口 RAM 的基地址为$18C40000，I92 将被设置为$18。当设置 24 位寻址时，不使用 I92。

I93 控制 VME 总线地址位 A23-A16 用于邮箱寄存器。尽管对于邮箱寄存器和小的 DPRAM 有可能这些地址位是相同的，但是通常它们是不同的。

I94 控制 VME 总线地址位 A15-A08 用于邮箱寄存器。如果 A23-A16 位对于邮箱寄存器和双口 RAM 是相同的，必不可少的是要设置 I94 以便在邮箱寄存器的 512 个地址和双口 RAM 的 16K 寄存器之间没有冲突。

I95 控制着当 PMAC 通过总线中断主机时使用哪个中断线。$01 到$07 的值分别选择 IRQ1 到 IRQ7。如果 I56 被设置为 1，I58 被设置为 1，那么 Turbo PMAC 将在 DPRAM ASCII 通信中使用该中断线。

I96 控制当 Turbo PMAC 中断主机时所提供的中断向量。如果因 PMAC 已把一个 ASCII 响应行放在双口 RAM 中而申请中断，那么提供的中断向量等于(I96+1)。

I97 控制 VME 总线地址位 A23 -A20 用于双口 RAM。它通常指定为一个 2 位十六进制值。第一位数应始终设置为 0。如果使用 24 位寻址，第二位数应设置为等于地址的 6 个十六进制数字的第一个；如果使用 32 位寻址，设置为地址的 8 个十六进制数字的第三个。例如，如果基地址是$700000 在 24 位的寻址，I97 应设置为$07。如果基地址是$18C40000 在 32 位的寻址，I97 应设置为$0C。

I98 控制是否启用双口 RAM。启动 DPRAM 的访问时它应该被设置为$E0。

I99 控制 VME 总线的地址宽度。它应该被设置为$90 用于 24 位寻址，或为$80 用于 32 位寻址。

为了实现这些设置，并保存以供将来使用，使用 SAVE 命令将这些 I-变量值存储到非易失性闪存中，该卡必须复位（$$$命令）。重置卡会将 I90- I99 的保存值复制到 RAM 中的 I-变量寄存器，然后复制到在 X:$070006- X:$07000F 的活动控制寄存器中。

在每个上电/复位后必须采取进一步的步骤来选择 VME 地址线 A19- A14 用于

双口 RAM。选择这些地址线使用的是一个动态页-选择技术，它必须使用即使是只有一个单“页”的 DPRAM。一个页面包含一个 16K×8 的内存地址组---对于小的（8K×16）双口 RAM，此页面选择整个双口 RAM。对于大的（32K×16）双口 RAM（当可用时），这个页面选择四分之一的双口 RAM。

这些地址线的选择是通过在 VME 总线上写一个字节到(邮箱基址+$121)中。邮箱基地址是由上次上电/复位的 I92、I93 和 I94 的设置定义的。如果邮箱基地址是在默认值$7FA000，此字节必须被写入 VME 总线地址$7FA121。

该字节的 0～5 位必须分别包含双口 RAM 的页面的 A14-A19 的值。计算此值的一种方法是拾取 24 位寻址中双口 RAM 的页面基地址的第二和第三个十六进制数字，或者 32 位寻址的第四和第五个十六进制数字，并将这个值除以 4（右移 2 位）。例如，如果基址是 24 位寻址的$780000，该字节应设置为$20（$80/4 =$20）。如果基地址是在 32 位寻址的$18C40000，该字节应设置为$10。

注意：

常见的是，此字节值将是$00，和一些 Turbo PMAC-VME 板上电时这个字节已定为$00。然而，对某些主板来说不是这样，所以用户不应该指望此默认设置。为了稳健运行，此字节必须在每次上电/复位后都写值。

PCI 总线

当通过 PCI 总线使用双口 RAM 通讯时，电脑的操作系统自动在总线上建立双口 RAM 芯片的基址。请参阅操作系统的文件了解如何查找和使用由操作系统确定的基地址。

USB/以太网

当双口 RAM 使用 USB 或以太网通信时，主机实际上并没有直接访问线连接的 Turbo-PMAC 端的双口 RAM 芯片。然而，USB 和以太网的实现支持创建一个虚拟共享内存接口，使更高级别的例程可以工作仿佛是有直接访问。

内存地址映射

在主机一侧和另一侧的 Turbo PMAC 之间进行内存地址的映射是很简单的。使用这种内存是在两侧匹配地址的问题。对于 Turbo PMAC，DPRAM 是在固定地址范围$060000 至$060FFF（$063FFF，对于大的 DPRAM）的额外的内存。由于每个数字地址上 Turbo PMAC 有 2 个寄存器（X 和 Y），小的 DPRAM 对于 Turbo PMAC 是 4K×32 的内存块;大的双口 RAM 作为 16K×32 的内存块。当指定 DPRAM 的 PMAC 的十六进制地址时，一个’$’前缀的汇编语言惯例用于表示使用的是十六进制数字。

可能主机使用字节寻址。因此，小的 DPRAM 对于主机电脑是一个 16K×8 的内存块。大的 DPRAM 作为一个 64K×8 的内存块。由于在主机的双口 RAM 的地址范围对于不同的应用程序会有所不同，所以当涉及到单个寄存器时我们只能采用相对于基地址的偏移。当指定双口 RAM 的主机十六进制地址偏移量时，使用一个’0 x’前缀的 C 语言习惯表示使用十六进制数字。

因为 Turbo PMAC 使用 32 位寻址，而主机采用 8 位寻址，所以对于 PMAC 的

每 1 个数字地址，主机使用 4 个数字地址。下表显示了对于 DPRAM 的关键地址，此地址递增是如何工作的。

Turbo PMAC Address	Host Address Offset	Example Host Address
Y:$060000	0x0000	0x0D0000
X:$060000	0x0002	0x0D0002
Y:$060001	0x0004	0x0D0004
X:$060001	0x0006	0x0D0006
…	…	…
Y:$060450	0x1140	0xD1140
…	…	…
Y:$060FFF	0x3FFC	0xD3FFC
X:$060FFF	0x3FFE	0xD3FFE
…	…	…
Y:$063FFF	0xFFFC	0xDFFFC
X:$063FFF	0xFFFE	0xDFFFE

下面两个方程可以有助于计算匹配的双口 RAM 的地址：

$$PMAC_address = \$060000 + 0.25 * (Host_address - Host_base_address)$$

$$Host_address = Host_base_address + 4 * (PMAC_address - \$060000) + Offset$$

其中：

Offset(偏移)=0 用于访问 Y 存储器，或用于 X 和 Y 一起作为 32 位

Offset(偏移)=2 用于单独访问 X 存储器

DPRAM 自动功能

Turbo PMAC 提供了许多设施，用于使用 DPRAM 在主机和 Turbo PMAC 之间来回传递信息。所有这些功能在 DPRAM 都有专门的寄存器。下表显示了每个功能和它使用的地址。

主机地址偏移	DPRAM 功能	Turbo PMAC地址
0x0000	控制面板功能（待定）	$060000
0x006A	电机数据报告缓冲区	$06001A
0x0676	后台数据报告缓冲区	$06019D
0x0E9E	DPRAM ASCII 命令缓冲区	$0603A7
0x0F42	DPRAM ASCII 响应缓冲区	$0603D0
0x1046	后台变量读缓冲区控制	$060411
0x104C	后台变量写缓冲区控制	$060413
0x1050	二进制旋转程序缓冲区控制	$060414
0x113E	DPRAM 数据采集缓冲控制	$06044F
0x1140	可变大小的缓冲区&开放式使用空间	$060450
0x3FFC	小 DPRAM（8K×16）的末端	$060FFF*
0xFFFC	大 DPRAM（32K×16）的末端	$063FFF*
*Turbo PMAC 内存寄存器 Y:$3F 包含最后一个 DPRAM 地址的 Turbo PMAC 的地址，		

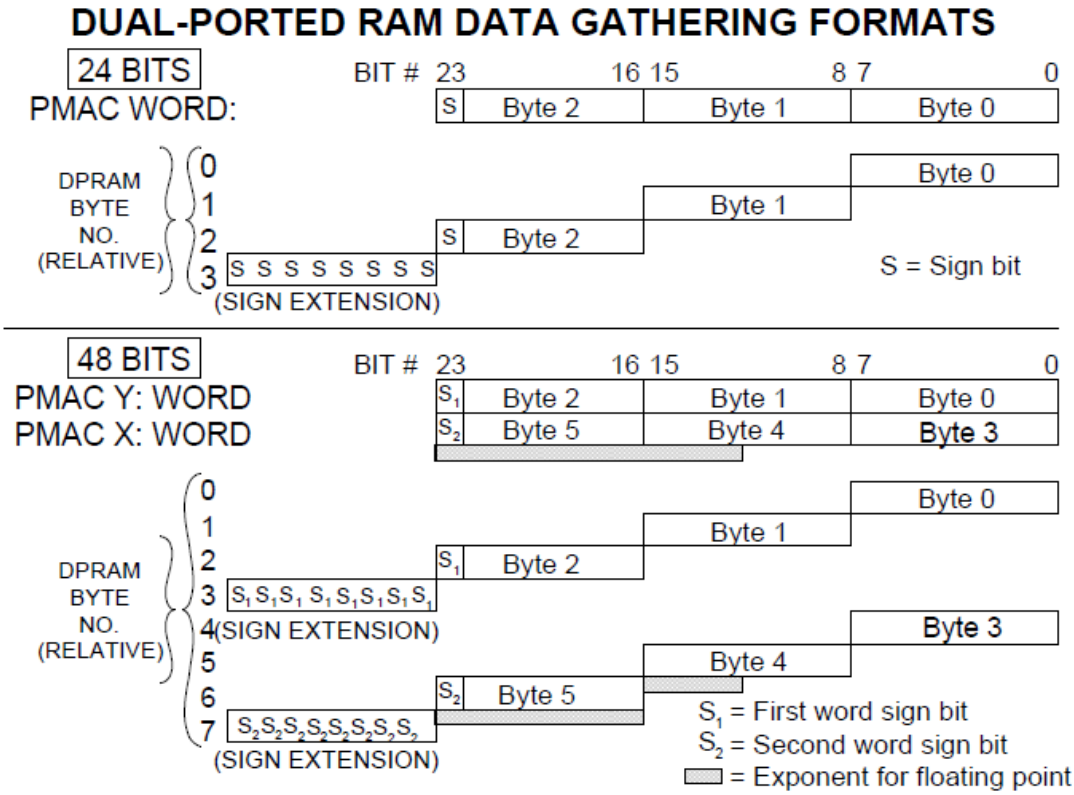
加上 1（$061000 或$064000）。

DPRAM 数据格式

数据以 32 位符号扩展的形式存储在 DPRAM 中。也就是说，每个 PMAC 的短（24 位）数据是被符号扩展并存储在 32 位的 DPRAM 中。最重要字节（最高有效字节）是全 1 或全零，匹配位 23。每个长字（48 位）被视为 2 个 24 位字，每个短字被符号-扩展为 32 位。主机必须重新组装这些字到单个值中。该数据在 DPRAM 中显示为 Intel 格式：更低有效字节和字出现在编号较小的地址。

为重新组装主机上的一个长定点字，采取更低有效的 32 位字，并屏蔽掉符号扩展（高八位）。在 C 语言中，这个操作可以用一个逐位 AND 完成：（LSW 和 16777215）。把这个结果作为一个无符号整数。其次，采取更高有效字和 16,777,216 相乘。最后，把两个中间结果加在一起。

为重新组装主机的一个长浮点字，对待更低有效字与上述定点情况是一样的。取更高有效字的低 12 位（MSW 和 4095），乘以 16,777,216，并加到屏蔽的更低有效字中。这形成了浮点值的尾数。现在取更高有效字的下一组 12 位（MSW& 16773120）。这是 2 的幂的指数，可以与尾数进行组合以形成完整的值。



DPRAM 电机数据报告缓冲区

Turbo PMAC 可以为 DPRAM 提供关键的电机数据，在那里它可以由主机电脑轻松快速地进行访问。如果启用此功能，Turbo PMAC 将关键电机寄存器复制到

DPRAM 中的固定寄存器。

前台与后台：此复制功能可以完成作为 Turbo PMAC 中的前台（中断）任务，或者作为一个后台任务。除非在以保证的高频得到数据是很重要的，否则强烈建议复制在后台完成，以免占用 Turbo PMAC 其他重要任务的时间。即使当信息被用于实时操作显示时，也推荐后台传输的方法。

启用前台复制：设置 I48 为 1 使能电机数据的前台复制。对于前台复制，I47 设置更新周期。如果 I47 是大于 0，每 I47 个伺服中断，Turbo PMAC 将电机寄存器复制到 DPRAM 中。而 I47 为 0 时，Turbo PMAC 将检查每个伺服周期，看看主机是否采用以前的数据。如果是的话，它会复制当前周期的数据用于一个“根据要求”的传输。

启用后台复制：设置 I57 为 1 使能电机数据的后台复制，并将 I48 设置为 0 自动禁用前台复制。I49 也必须设置为 1 使能后台复制坐标系统和全局数据（见下一节）。如果希望不传输任何坐标系统数据，将 0x0674（Y:$06019D）的“最大坐标系号”寄存器设置为 0（见下一节）。

对于后台复制，I50 设置更新周期。如果 I50 大于 0，在每一个后台周期，Turbo PMAC 将检查自它上次复制这个数据到 DPRAM 时是否已经过去了超过 I50 伺服周期。如果是这样的，它将复制现有数据。对于 I50 为 0 时，Turbo PMAC 将检查每一个后台周期，看看主机是否采用了以前的数据。如果是，它将复制现有数据，用于一个“根据要求”的传输。

电机说明：DPRAM 的专用的 32 位屏蔽字是用来指定哪些电动机的数据将被复制到 DPRAM，无论是用于前台或后台传输。这个字可以从 PMAC 侧或主机侧进行设置。格式如下：

PMAC Address X:$06001C; Host Address Offset 0x0072

位	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
电机	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17

PMAC Address Y:$06001C; Host Address Offset 0x0070

位	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
电机	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1

位值为’1’使能传输与该位相关的电机数据；值为’0’禁止传输。这些位可以在任何时候改变;新的值对于下一次传输是有效的。将整个 32-位的字设置为 0，将停止所有电机数据复制。

数据复制：对于启动的每个电机，下面的值被转移到 DPRAM：

- 电机跟随误差
- 电机伺服命令
- 电机伺服状态
- 电机一般状态
- 电机的位置偏差
- 电机滤波后的实际速度
- 电机主位置
- 电机净额实际位置

数据的地址：有关每个电机这些值的确切寄存器的详细信息，在软件参考手册中咨询 Turbo PMAC 内存映射的部分。

前台握手：如果使用前台传输(I48=1)，Turbo PMAC 将设置 0x006E(X:$06001B)的 15 位为 0，同时它将电动机数据复制到 DPRAM，并且它完成后就设置该位为 1。如果该位为 0，主机不应该尝试读取数据。如果 I47 设置为 0 用于根据要求的传输，主机应该在读取数据后设置该位为 0 以告知 Turbo PMAC 是时候提供下一组数据了。

如果使用前台传输，Turbo PMAC 也复制 24 位伺服周期计数器的值到 0x006C（Y：$06001B）以及 0x006E（X：$06001B）的 0-7 位，以“时间-戳记”数据。

主机在它正在读取数据时可以设置 0x006A（X：$06001A）的 15 位为 1。如果 Turbo PMAC 看到该位为 1，当它已准备好更多的数据转移到 DPRAM 时，它会跳过这个周期。主机必须确保当它读完时，此位设置为 0，以允许 Turbo PMAC 传送新数据。

后台握手：如果使用电机数据的后台传输（I57= 1 和 I49= 1），握手和使用后台坐标系统和全局数据缓冲区的是一样的。Turbo PMAC 在它正在复制坐标系和全局数据到 DPRAM 时，会将 0x067A（X：$06019E）的 15 位设置为 0。一旦它完成，就设置该位为 1。如果该位为 0，主机不应该尝试读取数据。如果 I50 设置为 0 用于“根据要求”的传输，主机应在读取数据后设置该位为 0 来指示 Turbo PMAC 是时候提供下一组数据了。

如果使用后台传输，Turbo PMAC 也复制 24 位伺服周期计数器的值到 0x0678（Y：$06019E）和 0x067A（X：$06019E）的 0-7 位中以时间-戳记该数据。

主机可在它正在读取数据以时设置 0x0676（X：$06019D）的 15 位为 1。如果 Turbo PMAC 看到该位为 1，当它已准备好更多的数据转移到 DPRAM，它会跳过这个周期。主机必须确保当它读完时，此位设置为 0，允许 Turbo PMAC 传送新数据。

DPRAM 后台数据报告缓冲区

Turbo PMAC 可以为 DPRAM 提供关键的全局性和坐标系数据作为一个后台功能，在那里它可以由主机电脑轻松快速地进行访问。如果启用此功能，Turbo PMAC 将复制关键的全局性和坐标系统寄存器到 DPRAM 的固定寄存器中。

启用复制：设置 I49 为 1 启动将全局和坐标系统数据复制到 DPRAM 作为一个后台功能。 I50 设置更新周期。如果 I50 大于 0，每一个后台周期，Turbo PMAC 将检查自它上次复制这个数据到 DPRAM 时是否已经过去了超过 I50 个伺服周期。如果是，它将复制现有数据。在 I50 为 0 时，Turbo PMAC 将检查每一个后台周期，看看主机是否采取了以前的数据。如果是这样，它将复制现有数据，用于一个“根据请求”的传输。

坐标系统说明：0x0676（Y：$06019D）的 0 -4 位指定 Turbo PMAC 中编号最高的坐标系的编号，其中的数据将被复制到 DPRAM。从坐标系 1 到这个坐标系所有的数据每次都复制。如果这个值被设置为 0，坐标系统的数据传输将被停止。

复制数据：对于每一个启用了复制的坐标系，下列数据将被转移：

- 坐标系进给速度/移动时间
- 坐标系时基值（进给倍率修调）
- 坐标系倍率源地址
- 坐标系状态字
- 坐标系轴目标位置（ABCUVWXYZ）

- 坐标系程序状态
- 旋转缓冲区的剩余坐标系程序行
- 移动段的剩余坐标系时间
- 加/减速的剩余坐标系时间
- 坐标系程序执行地址偏移

数据地址：关于每个电机这些值的确切寄存器的详情，在软件参考手册中咨询 Turbo PMAC 内存映射。

握手：Turbo PMAC 在它将坐标系统和全局数据复制到 DPRAM 时将 0x067A（X：$06019E）的 15 位设置为 0，一旦它完成了复制就设置该位为 1。如果该位为 0，主机不应该尝试读取数据。如果 I50 设置为 0 用于一个“根据要求”的传输，主机应在读取数据后设置该位为 0 来指示 Turbo PMAC 是时候提供下一组数据了。

Turbo PMAC 还将 24 位伺服周期计数器的值复制到 0x0678（Y：$06019E）和 0x067A（X：$06019E）的 0-7 位中以“时间戳记”数据。

主机可以在它正在读取数据时将 0x0676（X：$06019D）的 15 位设置为 1。如果 Turbo PMAC 看到该位为 1，当它已准备好更多的数据转移到 DPRAM 时，它会跳过这个周期。主机必须确保当它读完时，此位设置为 0，允许 Turbo PMAC 传送新数据。

DPRAM 的 ASCII 通讯

Turbo PMAC 可以通过 DPRAM 进行 ASCII 通信，也可以通过正常总线通信接口、主串行端口和辅助串行端口。它可以在多个端口接受命令，并同时提供响应。DPRAM 提供了最快路径的 ASCII 通讯。

启用：该 DPRAM 的 ASCII 通讯是通过设置 I58 到 1 而启用的。如果 I58 设置为 0，Turbo PMAC 不会检查 DPRAM 用于 ASCII 命令。

发送一个命令行：要发送一个 ASCII 命令行到 Turbo PMAC：

1、确保在 0x0E9C（Y：$0603A7）的主机-输出控制字的 0 位---主机数据就绪位--为 0，以确保 Turbo PMAC 已经读取前一个命令。对于 Turbo PMAC 上电/复位后的第一个命令，该位可能要由主机设置为 0。

2、写 ASCII 字符到 ASCII 命令缓冲区（始于 0x0EA0（Y：$0603A8））。两个 8 位的字符被打包到一个 16 位字中；第一个字符被放置到低字节。后续字符被放入连续的高地址，每个 16 位字是两个。（在字节寻址，每一个字符被写入一个地址，该地址比前一个字符的高 1。）在一个命令行最多可以发送 159 个字符。

3、终止字符串是通过<NULL>字符（字节值 0）。不要使用回车符来终止字符串，如在其他端口上。

4、设置在 0x0E9C（Y：$0603A7）的主机输出控制字的 0 位---主机数据就绪位---为 1，告诉 Turbo PMAC 已经准备好了一个命令字符串让它来读取。然后 Turbo PMAC 将在下一后台周期读取此命令，设置该位为 0，并对命令执行相应的操作。

请注意，PCOMM32 库的通信程序自动完成所有这些操作。如果写一个自定义的低级别的通信程序，这个操作基本上就是一个字符串复制操作。

发送一个控制字符命令：控制-字符的命令可以通过 DPRAM 通过一个独立于

ASCII 文本命令的专用寄存器发送。要通过 DPRAM 发送一个控制-字符命令到 Turbo PMAC：

1、确保控制字符字节--- 0x0E9E（X：$0603A7）的 0-7 位设置为 0。对于 Turbo PMAC 的上电/复位后的第一个控制-字符命令，此字节可能要由主机设置为 0。
2、写控制字符到 0x0E9E（X：$0603A7）的 0-7 位。
3、每个后台周期，Turbo PMAC 将读取该字节。如果该字节包含了一个非零值，Turbo PMAC 将对命令执行相应的操作，并设置字节为 0。

读取响应行：要通过 DPRAM 从 Turbo PMAC 读取一个 ASCII 响应行：

1、等待在 0x0F40（Y：$063D0）的主机-输入控制字变成大于 0，表示一个响应行已准备就绪。
2、解释这个寄存器中的值，以确定响应是什么类型。如果 15 位为 1，Turbo PMAC 会在命令中报告一个错误，并且除了这个字外没有响应。在这种情况下， 0-11 位将命令的错误号编码为 3 位 BCD 码。

如果 15 位是 0，那么不存在错误，并有一个响应字符串。8 和 9 位会告诉是什么引起了响应。如果它们形成的值为 0，是来自主计算机的命令引起了响应。如果它们形成一个 1 值，一个内部 CMDR 语句引起了响应。如果它们形成的值为 2 时，是一个内部 SENDR 语句引起了响应。注意 0-7 位的值。这些将决定这是否是响应的最后一行（见第 5 步，下面）。

3、读取始于 0x0F44（Y：$0603D1）的响应字符串。两个 8 位的字符被打包到一个 16 位字；第一个字符被放置到低字节。后续字符被放入连续的高地址，每个 16 位字是两个。（在字节寻址，每个字符是从比前一个字符高 1 的一个地址读取的）。一个单一的响应行最多可以发送 255 个字符。该字符串用 NULL 字符（字节值 0）终止，方便 C 样式字符串的处理。对于 Pascal 式的字符串处理，在 0x0F42（X：$0603D0）的寄存器包含了字符串中的字符数（加 1）。
4、清除 0x0F40（Y：$063D0）的主机-输入控制字为 0。Turbo PMAC 不会发送另一个响应行直到它看到这个寄存器设置为 0。
5、如果主机-输入控制字的 0-7 位中包含了一个值$0D（13--十进制，“CR”），这将不是响应中的最后一行，应重复步骤 1- 4。如果他们包含的值是$04（6 十进制，“ACK”），这是响应中的最后一行。

请注意，PCOMM32 库的通信程序自动完成所有这些操作。如果写一个自定义的低级别的通信程序，这个操作基本上就是一个字符串复制操作。

DPRAM 通讯中断

如果 I56 被设置为 1，每当它有一个响应行准备好让主机读取时，Turbo PMAC 将中断主机。该中断有潜力使主机通讯的效率更高，因为计算机并不需要查询 DPRAM 去看何时一个响应已准备就绪。

VME 中断：在任何 VME 总线的 Turbo PMAC，这种中断将出现在由 I95 指定的 VME 总线中断线路上。它将有一个中断向量等于（I96+1）。

ISA 中断：在 ISA 总线的 Turbo PMAC，该中断将出现在由板上的 E-跳线选定的 ISA 总线断线（IRQN）。通过这个中断信号线，板卡上的中断控制器 IC 可从八个不同的源（IR7-IR0）传递中断。

在一个 Turbo PMAC-PC，源 IR7 用来产生中断。跳线 E85 必须为 ON，跳线 E82 - E84 必须关闭用于此功能工作。这将 EQU4 线（编码器 4 的位置比较）连接输入到中断控制器---在这种情况下，该编码器的位置比较功能不能用于其它目的。

在一个 Turbo PMAC2-PC，源 IR5 是用来从 EQU1 线（编码器 1 的位置比较）产生中断。在这种情况下此编码器的位置比较功能不被用于其他目的。

在一个 Turbo PMAC2-PC UltraLite 中，源 IR5 是用来从 DSPGATE2 IC 的 CTRL0 线产生中断，它不具有其它功能。

Turbo PMAC 将继续申请这个中断源，直到主机清除了主机-中断控制字。因为这一点，当它从另一个源得到一个中断时，主机可以看到源仍然有效。

DPRAM 后台变量读取缓冲区

后台变量数据读取缓冲区允许你在后台周期将多达 128 个用户指定的 Turbo PMAC 寄存器复制到 DPRAM。这个功能是通过 I55 控制。该缓冲区具有两种操作模式，单用户和多用户。默认模式是单用户。当控制字 0x1044（Y:$060411）的第 8 位设置为零时，它是有效的。当控制字的 8 位设置为 1 时多用户模式被激活。

一般说明：缓冲区有三个部分。第一部分是标头：4 个 16 位字（8 个主机地址）包含握手信息并定义该表的其余部分的位置和大小。这是在 DPRAM 中的一个固定位置（见下表）。

第二部分包含被复制到 DPRAM 的 Turbo PMAC 寄存器的地址说明。对于被复制的每个 Turbo PMAC 位置，它占用两个 16 位字（四个主机地址），开始于标头中指定的位置。

第三部分，紧接在第二部分的末尾，包含了 Turbo PMAC 寄存器的复制信息。）对于每一个复制的短（X 或 Y）的 Turbo PMAC 位置，它包含了 2 个 16 位字（四个主机地址；对于每一个复制的长的 Turbo PMAC 位置，包含了 4 个 16 位字（8 个主机地址）。数据格式与双端口 RAM 的数据收集的是相同的。

寄存器映射

后台变量读取缓冲区第 1 部分
定义和基本握手

地址	说明
0x1044 (Y:$060411)	PMAC 到主机（0 位=1 为单用户模式）数据就绪。PMAC 完成更新缓冲区--主机必须清除用于更多的数据。
0x1046 (X:$060411)	伺服定时器（在数据准备时间进行更新）
0x1048 (Y:$060412)	数据缓冲区的大小（每个以 32 位长整数计算）
0x104A (X:$060412)	数据缓冲区的起始 Turbo PMAC 偏移，相对于可变-缓冲区空间的开头$060450（如：$0100 用于起始 PMAC 地址$060550--主机地址偏移 0x1540）

后台变量读取缓冲区第 2 部分
变量地址缓冲区格式（2×16 位字）

X：内存 15 位：数据就绪（多用户模式）	X：内存 4-5 位：读取的变量类型 0-3 位：地址的 16-19 位	Y：内存 读取的寄存器的 PMAC 地址的 0-15 位	双端口数据长度
1= PMAC 数据就绪 0=主机请求数据	4-5 位=0：PMAC 变量 Y：内存。	变量的 PMAC 地址	32 位
1= PMAC 数据就绪 0=主机请求数据	4-5 位=1：PMAC 变量 长	变量的 PMAC 地址	64 位
1= PMAC 数据就绪 0=主机请求数据	4-5 位=2：PMAC 变量 X：内存	变量的 PMAC 地址	32 位

启用：要启动这个缓冲区的操作：

1、将缓冲区第二部分的起始位置写到寄存器 0x104A（X：$060412）。这个位置被表示为 Turbo PMAC 地址相对于可变-缓冲区空间起始位置（$060450）的偏移，而且对于 8K×16 的 DPRAM 它必须是介于$0000 和$0BAF 之间，或对于 32K×16 的双口 RAM 是介于$0000 和$3BAF 之间。

2、从上步中指定的 DPRAM 的位置开始，写入要复制的寄存器的 Turbo PMAC 地址和寄存器类型。第一个 16-位的字包含了第一个寄存器的 Turbo PMAC 地址的低 16 位；而第二个 16 位字的 0-3 位保存这个地址的高 4 位，4- 5 位取值为 0、1 或 2，分别为第一个寄存器指定 Y、长或 X。第三和第四个字则指定第二个寄存器的要复制的地址和类型等等。

3、写一个表示缓冲区大小的数值到寄存器 0x1048（Y：$060412）。该值必须介于 1 和 128 之间。当 Turbo PMAC 看到该值大于零以及个别数据就绪位是零时，它准备开始将您指定的寄存器复制到 DPRAM。

4、要启用单用户模式，写一个零到在 0x1044（Y：$060411）的控制字。要启用多用户模式，写一个 256（置位第 8 位并清除第 0 位）到这个控制并设置每个变量的数据类型寄存器（X 存储寄存器）的 15 位=0。这将告诉 Turbo PMAC 主机已经准备接收数据以及据的模式是什么。

5、设置 I55 为 1。这将同时启用后台变量数据报告功能和后台变量数据写入功能。

单用户模式步骤：在操作过程中，Turbo PMAC 将在每个后台周期尝试将数据复制到缓冲区中---在每个 PLC 程序的每次扫描之间。如果控制字 0x1044 的 0 位设置为 1，它就会假设认为主机没有读完上一个周期的数据，所以它会跳过这个周期。如果 0 位为 0，它会复制所有指定的寄存器。

当 Turbo PMAC 完成了复制指定的寄存器时，它会复制伺服定时器寄存器（X：$000000）的低 16 位到 0x1046（X：$060411）的 DPRAM 中。然后，它置位控制字 0x1044（Y：$060411）的 0 位，让主机知道，它已经完成了一个周期。

当主机要读取这些数据时，它应该检查控制字 0x1044 的 0 位（数据就绪位）

已被置位。如果它已被置位，主机可以开始读取和处理双口 RAM 中的数据。当它完成时，应清除数据就绪位让 Turbo PMAC 知道，它可以执行另一个循环。

多用户模式程序：这种模式的操作类似于上述的单用户模式。主要的区别是，控制字不再用作为一个全局性握手位用于更新缓冲区。它仅用于启用或禁用多用户模式。在多用户模式下，控制字永远不会被 Turbo PMAC 修改。现在，握手是在一个单个变量基础上，由变量的数据类型说明符的第 15 位控制。

在每个后台周期，每一个未编译的 PLC 程序的每次扫描之间，Turbo PMAC 将尝试将数据复制到缓冲区中的每个变量。每个变量的数据类型说明符的 15 位控制是否允许 Turbo PMAC 更新该特定变量的值。如果变量的数据类型说明符的位设置为 1，Turbo PMAC 将不会更新变量。将 15 位设置为 0 的任何变量将被更新。

当 Turbo PMAC 完成了服务缓冲区，它将伺服定时器寄存器（X：$000000）的低 16 位复制到 DPRAM 的 0x1046（X：$060411）。这是不依赖于更新缓冲区的任何变量。

当主机想要读取一个寄存器时，它应该检查数据类型说明符的 15 位（数据就绪位）是否已被置位。如果它已被置位，主机可以开始读取和处理该寄存器中的数据。当它完成时，应清除数据就绪位，让 Turbo PMAC 知道，它可以更新寄存器下一个周期。

数据格式：每个 24 位（X 或 Y）寄存器被符号扩展为 32 位。对于一个 48 位（长）寄存器，每个 24 位的一半是符号扩展到 32 位，总共是 64 位。该数据扩展在最后一个地址指定寄存器后立即开始。

禁用：要禁用此功能，您可以设置大小寄存器 0x1048（Y：$060412）为 0，或者干脆保留各个数据就绪位置位。

DPRAM 后台变量数据写缓冲区

后台变量数据写入缓冲区实际上是与上述后台变量数据读取缓冲区相反的。它可以让你最多写 32 个用户指定的寄存器或寄存器的特定位而不使用通信端口（PC 总线，串行或 DPRAM 的 ASCII I/O）。这可让您设置任何的 Turbo PMAC 变量而无需使用 ASCII 命令，如 M1= 1，而且不用担心一个打开的旋转缓冲区。这个功能是通过 I55 控制。

一般说明：缓冲区有两个部分。第一部分是标题：2 个 16 位字（4 个主机地址）包含握手信息并定义该表其余部分的位置和大小。这是在 DPRAM 中的一个固定位置（Turbo PMAC 地址$060413 如示于下表）。

第二部分包含被复制到 Turbo PMAC 的 Turbo PMAC 寄存器的地址说明。对于每个被写的 Turbo PMAC 位置，它占用 6×16 位字（12 个主机地址），起始于标题中指定的位置。

寄存器：

后台变量数据写入缓冲区第 1 部分
定义和基本握手

地址	说明
0x104C (Y:$060413)	主机到 PMAC 数据传输。清除时，PMAC 被更新。主机必须设置为另一个更新。
0x07E8	数据缓冲区的起始 Turbo PMAC 偏移，相对于可变-缓冲空间的开头

(X:$060413)	$060450（如：$0100 用于起始 PMAC 地址$060550-主机地址偏移 0x1540）

后台变量写入缓冲区第 2 部分
每个数据结构（6X16 位）的格式

地址	X-寄存器内容	Y-寄存器内容
n	11-15 位：偏移（=0 - 23）-将写入值的目标寄存器的起始位号 6-10 位：宽度（=0、1、4、8、12、16 或 20 - 0 代表 24 位）-将写入值的目标寄存器的位数 3- 5 位：目标寄存器类型 =0：Y 寄存器 =1：长（X/Y-寄存器） = 2：X 寄存器 0-2 位：目标寄存器地址的高 3 位（16-18）	目标寄存器地址的 0-15 位
n+1	数据字 1 的高 16 位	数据字 1 的低 16 位
n+2	数据字 2 的高 16 位（仅用于写入长寄存器）	数据字 2 的低 16 位（仅用于写入长寄存器）

启用：要启动这个缓冲区的操作：

1、将缓冲区第二部分的起始位置写到寄存器 0x104E（X：$060413）。这个位置被表示为 Turbo PMAC 地址相对于可变缓冲空间起始位置$060450 的偏移，而且它对于 8K×16 双口 RAM 必须是介于$0000 和$0BAF 之间，或对于 32K×16 的双口 RAM 介于$0000 和$3BAF 之间。

2、从上步中指定的双口 RAM 的位置开始，写入要复制的寄存器的 Turbo PMAC 地址和寄存器类型。第一个 16 位字包含了第一个寄存器的 Turbo PMAC 地址的低 16 位。第二个 16 位字取值范围为 0 到 65535，来指定这个目标 Turbo PMAC 寄存器的类型、宽度、偏移和地址的高 3 位。第三、第四、第五和第六个字指定要写入的数据。

注意：

如果您指定地址 0，你将终止该写操作。将不执行缓冲区进一步的写操作。

3、写一个表示缓冲区大小的数值到寄存器 0x104C（Y：$060413）中。该值必须介于 1 和 32 之间。当 Turbo PMAC 看到该值大于零，它准备开始复制已指定的寄存器到 Turbo PMAC 中。当它完成后，将这个寄存器的值更改为 0。

4、设置 I55 为 1。这同时启用后台变量数据读取功能和后台变量数据写入

功能。

过程：在操作过程中，在 0x104C（Y：$060413）不为零时，Turbo PMAC 会在后台周期把缓冲区中的数据复制到 Turbo PMAC 中。如果该寄存器为 0，它就会认为主机尚未完成将数据放置在缓冲区中，也不会写到 Turbo PMAC。一旦这个寄存器被设定为一个从 1 到 32 的数，它将复制那么多（数量等于刚才设定的值）的寄存器，从标头起始地址信息的开头开始，从 DPRAM 复制到 Turbo PMAC 中。

当 Turbo PMAC 完成了复制指定的寄存器，它设置寄存器 0x104C（Y:$060413）为零，以让主机知道，它已经完成了一个循环。

当主机要更新这个缓冲区，它应该检查看看该 0x104C（Y：$060413）是否为零。当它完成时，它应该设置地址/数据结构。然后设置 0x104C（Y：$060413）为复制到 Turbo PMAC 的寄存器的数目以让 Turbo PMAC 知道，它可以执行另一个循环。

数据格式： Turbo PMAC 的 X 和 Y 寄存器将使用长 32 位数据字 1。32 位数据字 2 不用于这种情况。高 8 位是符号扩展位。

对于一个 48 位的 Turbo PMAC 整数或浮点值，应该使用该 L（长）格式。L-格式将有合计 48 位，其中低 32 位在长 32 位数据字 1 中，而高 16 位是在 32 位数据字 2 中。该数据在最后一个地址说明寄存器后立即启动。

禁用：要禁用此功能，只需使得 0x104C（Y：$060413）设置保留为零。

双口 RAM 的二进制旋转程序传输缓冲区

Turbo PMAC 的双口 RAM 的二进制旋转程序传输缓冲区允许上位机以其内部的二进制存储格式将运动程序指令发送到 Turbo PMAC 中，是为了这些命令的最快传输速度。Turbo PMAC 的 16 个坐标系的每一个都可以在双口 RAM 中有自己的二进制旋转传输程序缓冲区。

对于使用此功能的每个坐标系统也必须有一个旋转运动程序缓冲区定义在 Turbo PMAC 的内部 RAM 中。这是通过**&n DEFINE ROTARY{大小}**命令完成的。这些内部旋转运动程序缓冲区在断电或复位时是不保留的，所以他们必须在每次板上电或复位后进行定义。如果定义有多个内部旋转程序缓冲区，它们必须从最高编号的坐标系统到最低的来定义。

DPRAM 中的二进制旋转程序传输缓冲区对于内部旋转程序缓冲区是简单的透-通缓冲区。当 Turbo PMAC 在 DPRAM 缓冲区接收来自主机的二进制格式的运动程序的命令时，它会将这个数据复制到内存中的旋转缓冲区中。最终的结果是一样的，就如同一个 ASCII 程序指令已通过任意一个端口被送往 Turbo PMAC 那样，但传输的速度更快，原因如下：

1、没有各个字符的握手。

2、不用将 ASCII 命令语法分析到内部二进制存储格式。

3、在一个通信周期可以处理多个命令行。

如果 I45 设置为默认值 0，Turbo PMAC 在每个后台周期检查 DPRAM 中的二进制旋转缓冲区，并将任何新的内容传输到内部旋转程序缓冲区中。如果 I45 设置为 1，它会在每一个实时中断，将二进制缓冲区作为一个高优先级的前台任务来检查。

Delta Tau 的 PCOMM32 通信库例程提供自动支持二进制旋转程序传输缓冲区。

概述：每个坐标系统的二进制旋转传输缓冲区有两个部分。第一部分是标题，在 DPRAM 的固定地址。每个二进制旋转传输缓冲区中的标题占有 6 个 16 位字，并包含关于缓冲区第二部分的大小和状态的关键信息。

缓冲区的第二部分是在 DPRAM 中一个由用户在标题中指定的位置。它包含了实际的二进制格式的运动程序的命令。该部分的大小还是通过用户指定在标题中。

寄存器：下表显示了标题的结构。给定的地址是用于第一个坐标系。其它坐标系的标题紧跟其后。这些地址可以在软件参考手册中的内存映射中找到。

第一个坐标系旋转缓冲区的地址	说明
0x1050 (Y:$60414)	PMAC 到主机的二进制旋转型缓冲区状态字
	15 位=1：错误（停止处理命令）
	14 位=1：内部旋转缓冲区已满（忙标志）PMAC 索引停止更新。
	7 位= 0 代码 错误
	------ ----------
	1 内部旋转缓冲区大小=0
	或 DPRAM 旋转缓冲区大小=0
	这些标志是由 PMAC 置位和复位。忙标志
	被置位当 PMAC 内部旋转缓冲区已满时。然而
	，这并不意味着二进制 DPRAM 旋转缓冲区已
	满。忙标志复位是当 PMAC 内部旋转缓冲区
	未满或 DPR 二进制旋转缓冲区为空时。
0x1052 (X:$60414)	坐标系号和使能控制，0-4 位代表坐标系＃；如果 0<坐标系＃<17，缓冲区启用
0x1054 (Y:$60415)	主机二进制旋转缓冲区索引- PMAC 地址偏移，相对于 0x105A 设置的缓冲区的起始地址
0x1056 (X:$60415)	PMAC 二进制旋转缓冲区索引 -PMAC 地址偏移，相对于 0x105A 设置的缓冲区起始地址
0x1058 (Y:$60416)	二进制旋转缓冲区的大小 - 以 PMAC 地址（=主机地址/4）
0x105A (X:$60416)	起始二进制旋转缓冲区 PMAC 地址偏移量-相对于 DPRAM 可变缓冲区空间开头（$060450）

使用缓冲区：首先，必须通过**&n DEFINE ROT** 命令建立 Turbo PMAC 的内部二进制旋转程序缓冲区。

接着，必须设置 DPRAM 传输缓冲区的标题信息。传输缓冲区的起始地址和大小必须声明。缓冲区不应该重叠任何其他 DPRAM 的使用。大小参数必须是偶数，6 个 PMAC 地址的绝对最小值。大小应该声明足够大以不限制吞吐通过量。每个基本 PMAC 命令分量（如 X10）占据了传输缓冲区的 2 个 PMAC 地址。每个后台周期会发生传输，通常为几毫秒。

两个索引都应该被设置为零，以表明双方都指向缓冲区的开始。接下来，应给出**&n OPEN BIN ROT** 命令，从而使 Turbo PMAC 检查传输缓冲区的新数据。

现在，二进制格式的指令可以被加载到传输缓冲区中，而且主机索引更新。由于 Turbo PMAC 从缓冲区读取命令，所以它会更新 PMAC 索引。

二进制命令结构：通常情况下，PCOMM32 程序自动生成二进制命令格式。如果您需要此格式的知识，请与工厂联系。

DPRAM 数据采集缓冲区

Turbo PMAC 的数据采集功能，可以在 DPRAM 创建一个旋转缓冲区，使主机可以拾取正在采集的数据。这种方式中，数据采集缓冲区的大小不受 Turbo PMAC 自身的存储器容量的限制。如果 I5000 被设置为 2 或 3，DPRAM 中的数据采集缓存区被选定；若 I5000 被设置为 3，它是以一个旋转的方式使用--这是缓冲区通常被使用的方式。

DPRAM 的数据采集缓冲区总是开始于地址 0x1140（Y：$060450）。它的大小是由 DEFINE GATHER{大小}命令确定的，其中{大小}设定相对起始的 PMAC 地址的数量。这个大小值存储在 0x113C（Y：$06044F）。

变量 I5001 到 I5048 确定要采集的潜在的寄存器。I5050 和 I5051 都是 24 位屏蔽变量，用来确定采集 48 个可能源的其中哪些。I5049 决定采集周期，单位是伺服周期。

实际的采集是通过在线 GATHER 命令启动的，并通过在线 ENDGATHER 命令停止。在 Turbo PMAC 收集数据到 DPRAM 时，它领先于指针显示地址偏移下一个采集的项目将被放置哪里。该指针被存储在 0x113E（X：$06044F）。主机电脑必须留意这个指针的变化来指示更多的数据已经被拷贝到 DPRAM。

19-7 Turbo PMAC 以太网协议

通过一个以太网端口通信是完全支持用于微软 Windows98、2000、ME、XP 操作系统的，是通过 PCOMM32PRO 驱动程序库。在这些操作系统下，用户编写通信程序，应查阅 PCOMM32PRO 的手册。本节介绍相对于其它操作系统，如何编写以太网通讯驱动程序。

本部分适用于对 TCP/ IP 协议族中使用的 Berkeley 套接字有一个基本了解的应用程序的程序员。在试图阅读或理解本手册内容之前，复习基本的套接字，并理解它们，然后再继续。必须了解的基本的套接字功能是接收、发送和套接。

本手册中的例子仅用于演示目的，并传达如何与 Turbo PMAC 通信的概念。因此，该示例不包括错误检查和超时（时间限制）。然而泰道的实际生产代码包括这些，并大力鼓励应用程序开发者在他们的代码中包括错误检查和超时，以防止意外停机和反应迟钝的行为。

Turbo PMAC 的以太网通信端口是使用 TCP/IP 协议族的 UDP 或 TCP 协议在 1025 端口进行通讯。因此，程序员应该在端口 1025 的 PMACPORT 打开一个数据报套接字（UDP）或一个流式套接字（TCP）。

```
sock = socket(PF_INET,SOCK_DGRAM,0)              ; // UDP Mode
                    or
sock = socket(PF_INET,SOCK_STREAM,0)             ; // TCP Mode
// Embedded Ethernet's IP address
// The port that the embedded program is listening on.
sin.sin_port = htons(PMACPORT);
```

```
connect(*sock,(struct sockaddr*)&sin,sizeof(sin));
```

PMAC 以太网协议命令数据包说明

命令数据包。所有的命令都以下结构形式通过套接字发送：

```
/ Ethernet command structure
typedef struct tagEthernetCmd
{
BYTE    RequestType;
BYTE    Request;
WORD    wValue;
WORD    wIndex;
WORD    wLength;
BYTE    bData[1492];
} ETHERNETCMD,*PETHERNETCMD;
```

下面是在 ETHERNETCMD 结构的字段的描述。

RequestType 在特定的命令使用以指示该请求是相对于该 PC 的一个输入还是一个输出指令。泰道作如下定义：VR_UPLOAD=0xC0 用于一个发送到主机的命令，VR_DOWNLOAD=0X40 用于一个发送到设备的命令。

Request 指出你从 Turbo PMAC 以太网 connection0 请求什么类型的命令。下面是当前支持的命令集的定义列表。

```
#define VR_PMAC_SENDLINE          0xB0
#define VR_PMAC_GETLINE           0xB1
#define VR_PMAC_FLUSH             0xB3
#define VR_PMAC_GETMEM            0xB4
#define VR_PMAC_SETMEM            0xB5
#define VR_PMAC_SETBIT            0xBA
#define VR_PMAC_SETBITS           0xBB
#define VR_PMAC_PORT              0xBE
#define VR_PMAC_GETRESPONSE       0xBF
#define VR_PMAC_READREADY         0xC2
#define VR_CTRL_RESPONSE          0xC4
#define VR_PMAC_GETBUFFER         0xC5
#define VR_PMAC_WRITEBUFFER       0xC6
#define VR_PMAC_WRITEERROR        0xC7
#define VR_FWDOWNLOAD             0xCB
#define VR_IPADDRESS              0xE0
```

wValue 是特定的请求，它的使用是表示在每个命令的说明下。

wINDEX 是特定的请求，它的使用是表示在每个命令的说明下。

wLength 表示 bDATA 字段的长度。

bDATA 是发送给 PMAC 的有意义的数据。

通过一个以太网端口发送到 Turbo PMAC 的每一个命令的开始时使用

ETHERNETCMD 数据包结构，并通过一个 PC send 命令启动。然后每一个命令必须发出一个 recv 命令，要么接收到一个确认字符或接收有意义的数据。

```
#define ETHERNETCMDSIZE 8
send(sock,(char *)&EthCmd,ETHERNETCMDSIZE,0);
recv(sock,(char *)&EthCmd,1,0);
```

Turbo PMAC 的以太网协议的命令集

本节介绍了以太网通信协议的重要命令，以及他们如何必须执行。

VR_PMAC_FLUSH

此数据包会导致一个<Ctrl-X>命令字符（齐平通信缓冲区）发送到 Turbo PMAC，并会等待长达 10 毫秒，等待 Turbo PMAC 响应一个确认<CTRL-X>的回应字符。要发送的数据包应该被设置如下。命令成功完成后将返回一个字节。

```
EthCmd.RequestType = VR_DOWNLOAD;
EthCmd.Request = VR_PMAC_FLUSH;
EthCmd.wValue = 0;
EthCmd.wIndex = 0;
EthCmd.wLength = 0;
EthCmd.bData – not used for this command
```

例子

```
int CALLBACK PmacSockFlush()
{
    ETHERNETCMD EthCmd;
    Int                     rc,iTimeout;
    EthCmd.RequestType = VR_DOWNLOAD;
    EthCmd.Request      = VR_PMAC_FLUSH;
    EthCmd.wValue       = htons(FLUSH_TIMEOUT);
    EthCmd.wIndex       = 0;
    EthCmd.wLength      = 0;

    send(sock,
         (char *)&EthCmd,
          ETHERNETCMDSIZE ,
          0);
    recv(sock,
          (char *)&EthCmd,
          1,
          0);
 }
```

上面的例子以及本文档中所有的示例都不执行错误检查和超时检查。它是应用程序开发者的责任进行错误检查和超时检查，以确保应用程序不会挂起。

VR_PMAC_SENDLINE

此数据包会导致EthCmd.bData中以NULL结尾的一个字符串被发送到Turbo PMAC。该字符串不应该被以回车符终止，因为这是由固件完成。命令成功完成后将返回一个字节。

```
EthCmd.RequestType = VR_DOWNLOAD;
EthCmd.Request      = VR_PMAC_SENDLINE;
EthCmd.wValue       = 0;
EthCmd.wIndex       = 0;
EthCmd.wLength      = htons( (WORD)strlen(outstr));
strncpy((char *)&EthCmd.bData[0],
        outstr
        ,(WORD)strlen(outstr));
```

例子

```
int CALLBACK PmacSockSendLine(char *outstr)
{
    EthCmd.RequestType = VR_DOWNLOAD;
    EthCmd.Request      = VR_PMAC_SENDLINE;
    EthCmd.wValue       = 0;
    EthCmd.wIndex       = 0;
    EthCmd.wLength      = htons( (WORD)strlen(outstr));
    strncpy((char *)&EthCmd.bData[0],outstr,(WORD)strlen(outstr));

    send(sock,
          (char *)&EthCmd,
          ETHERNETCMDSIZE + strlen(outstr),
         0);
    recv(sock,(char *)&EthCmd,1 ,0);
   }
```

VR_PMAC_GETLINE

此数据包导致以太网连接返回Turbo PMAC中任何可用的字符串。<CR>，<ACK>，或<LF>之前的所有字符都被返回。PMAC的可用字符串被返回并应该通过一个以太网的recv命令被捕获。建议不使用此功能。使用VR_PMAC_GETBUFFER代替，因为该功能将检索多个行并拒绝使用VR_PMAC_GETLINE的多次调用而大大提高了性能。

```
EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request      = VR_PMAC_GETLINE;
EthCmd.wValue       = 0;
EthCmd.wIndex       = 0;
EthCmd.wLength      Not used
```

例子

```
int CALLBACK PmacSockGetLine(char *instr)
{
```

```
    EthCmd.RequestType = VR_DOWNLOAD;
    EthCmd.Request      = VR_PMAC_GETLINE;
    EthCmd.wValue       = 0;
    EthCmd.wIndex       = 0;
    EthCmd.wLength      = htons( (WORD)strlen(outstr));
    strncpy((char *)&EthCmd.bData[0],outstr,(WORD)strlen(outstr));

    send(sock,(char *)&EthCmd,ETHERNETCMDSIZE,0);
    recv(sock,(char *)&instr,255 ,0);
}
```

VR_PMAC_GETBUFFER

此数据包导致以太网连接返回Turbo PMAC的任何可用的字符串。<ACK>或<LF>之前的所有字符都被返回。如果检测到有一个<BEL>或<STX>字符，那么只返回下一个<CR>之前的数据。返回的数据最高限额为1400字节。它是调用者的责任在逻辑上确定是否有更多的数据需要跟随以及是否必须再次调用VR_PMAC_GETBUFFER来检索所有可用的数据。

```
EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request      = VR_PMAC_GETBUFFER;
EthCmd.wValue       = 0;
EthCmd.wIndex       = 0;
EthCmd.wLength      = htons( (WORD)strlen(outstr));
EthCmd.bData -       Not Used
```

例子

```
int CALLBACK PmacSockGetBuffer(char *instr)
{
    EthCmd.RequestType = VR_DOWNLOAD;
    EthCmd.Request      = VR_PMAC_GETBUFFER;
    EthCmd.wValue       = 0;
    EthCmd.wIndex       = 0;
    EthCmd.wLength      = htons( (WORD)strlen(outstr));

    send(sock,(char *)&EthCmd,ETHERNETCMDSIZE,0);
    recv(sock,(char *)&instr,1400,0);
}
```

VR_ IPADDRESS

此数据包允许在以太网接口设置或检索当前IP地址。

当设置IP地址到一个新的值时，新地址生效需要Turbo PMAC断电。

```
EthCmd.RequestType = VR_UPLOAD to retrieve the IP address
        or
EthCmd.RequestType = VR_DOWNLOAD to set the IP address
```

EthCmd.Request = VR_IPADDRESS;
EthCmd.wValue = 0;
EthCmd.wIndex = 0;
EthCmd.wLength = htons(4);
EthCmd.bData = contains 4 bytes of data indicating the IP
address set on the send command.

对于接收的命令，返回四个字节的数据，表示IP地址。

VR_PMAC_SENDCTRLCHAR

此数据包允许发送单个字符或控制字符到Turbo PMAC以太网端口。下面的数据包是被发送的内容。接收数据是无关紧要的；其目的是为了确保接收到发送者的命令。

EthCmd.RequestType = VR_DOWNLOAD;
EthCmd.Request = VR_PMAC_SENDCTRLCHAR;
EthCmd.wValue = htons(outch); // the character to write
EthCmd.wIndex = 0;
EthCmd.bData - Not Used

VR_PMAC_PORT

此数据包允许通过以太网端口发送一个单字节到CPU的主机端口或从其中接收一个单字节。

要发送数据到主机端口，设置数据包如下所示。在发送数据包之后，程序员必须等待通过recv函数接收1个字节，然后再继续。接收数据是无关紧要的，其目的是为了确保接收到发送者的命令。

EthCmd.RequestType = VR_DOWNLOAD;
EthCmd.Request = VR_PMAC_PORT;
EthCmd.wValue = htons((WORD)offset);
EthCmd.wIndex = htons((WORD)outch);
EthCmd.wLength = 0;

为了从主机端口接收数据，设置数据包如下所示。在发送数据包后，程序员将收到1个字节，这是以太网端口从Turbo PMAC的CPU主机端口读取的值。

EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request = VR_PMAC_PORT;
EthCmd.wValue = htons(offset);
EthCmd.wIndex = 0;
EthCmd.wLength = 0;

VR_PMAC_READREADY

此数据包允许确定是否在Turbo PMAC CPU有准备好被读取的数据。

将返回两个字节。第一个字节，如果不为零，表示有数据被读取；如果为0，表示没有数据被读取。该数据包将被设置如下：

EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request = VR_PMAC_READREADY;
EthCmd.wValue = 0;

```
EthCmd.wIndex        = 0;
EthCmd.wLength       = htons(2);
```

例子

```
ETHERNETCMD EthCmd;
char                data[2];
int                 rc;

EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request       = VR_PMAC_READREADY;
EthCmd.wValue        = 0;
EthCmd.wIndex        = 0;
EthCmd.wLength       = htons(2);

rc = send(sock,((char *)&EthCmd),ETHERNETCMDSIZE,0);
rc = recv(*((SOCKET *)vh[dwDevice].hDriver),data,2,0);
return data[0];
```

VR_CTRL_REPONSE

此数据包允许在发送一个控制字符后获得响应。数据包被设置如下。所接收的数据是对所发送的控制字符的响应。下列控制字符<CTRL-B>、<CTRL-C>、<CTRL-F>、<CTRL-G>、<CTRL-P>和<CTRL-V>将返回有意义的数据。所有其他数据控制字符不返回有意义的数据。

```
EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request       = VR_CTRL_RESPONSE;
EthCmd.wValue    = htons(outchar)       ;//outchar=ctrl char to send out
EthCmd.wIndex    = 0;
EthCmd.wLength = htons(len);

rc = send(sock,((char *)&EthCmd),ETHERNETCMDSIZE,0);
rc = recv(sock,outstr,len,0)         ; // returned data appears
```

VR_PMAC_WRITEBUFFER

此数据包允许只用一个数据包写多行到Turbo PMAC。数据包被设置如下。所接收的数据是对所发送的控制字符的响应。它通常是用于下载一个文件。数据的形式应该是在其中的每一行由空字节分隔。

例如，下面的多行字符串可以发送在单个数据包中：OPEN PLC1 CLEAR<00> P1= P1+1<00> CLOSE<00>，其中<00>表示一个空字节。最大数据长度为1024字节；多于这个数值的数据必须分隔开为VR_PMAC_WRITEBUFFER的多个调用。

在接收到该数据包时，Turbo PMAC以太网接口发送回4个字节的数据。第3个字节表示是否有一个错误下载。如果该字节的值是0x80，表示在下载过程中出现一个错误。如果它是0，则在下载过程中没有错误。字节2表示如果有一个下载错误，Turbo PMAC的错误类型。咨询Turbo PMAC软件参考汇总或变量I6的错误列表。字节0和字节1一起形成了一个WORD，表示导致发生错误的行号。字节1

为这个字的MSB，字节0是LSB。

例子

```
char errcode[4];
EthCmd.RequestType = VR_DOWNLOAD;
EthCmd.Request      = VR_PMAC_WRITEBUFFER;
EthCmd.wValue       = 0;
EthCmd.wIndex       = 0;
EthCmd.wLength      = htons(len) ;
memcpy(EthCmd.bData,data, len);
send(sock,(char *)&EthCmd,ETHERNETCMDSIZE + len,0);
recv(sock,(char *)errcode,4 ,0);
```

VR_FWDOWNLOAD

此数据包允许写入原始数据到Turbo PMAC主机端口用于固件下载。以太网固件拾取数据流，然后写入到Turbo PMAC CPU主机端口在地址{基址+5}、{基址+6}和{基址+7}。数据包包括wValue的参数以命令从主机端口地址{基址+5}开始下载。此数据包允许1个数据包写多行到Turbo PMAC。

数据包被设置如下。所接收的数据是对所发送的控制字符的响应。它通常是用于下载一个文件。数据的形式应该是每一行由空字节分开。在发送数据包之后，程序员必须等待通过recv函数接收1个字节，然后再继续。接收数据是无关紧要的，其目的是为了确保接收到发送方的命令

```
EthCmd.RequestType = VR_DOWNLOAD;
EthCmd.Request= VR_FWDOWNLOAD;
EthCmd.wValue = htons((WORD)bRestart)          ; //bRestart = 1 on start
EthCmd.wIndex = 0;
EthCmd.wLength = htons((WORD)len) ;

memcpy(EthCmd.bData,data, len);
send(sock,(char *)&EthCmd,ETHERNETCMDSIZE + len,0);
recv(sock,(char *)&errcode,1 ,0);
```

VR_PMAC_GETRESPONSE

此数据包导致以太网连接发送一个字符串到Turbo PMAC，然后返回Turbo PMAC的任何可用的字符串。<ACK>或<LF>之前的所有字符被返回。如果检测到有<BEL>或<STX>字符，只返回下一个<CR>之前的数据。返回数据的最高限额为1400字节。它是调用者的责任，在逻辑上确定是否有更多的数据需要跟随，以及是否需要再次调用VR_PMAC_GETBUFFER以检索所有可用的数据。

```
EthCmd.RequestType = VR_DOWNLOAD;
EthCmd.Request      = VR_PMAC_GETRESPONSE;
EthCmd.wValue       = 0;
EthCmd.wIndex       = 0;
EthCmd.wLength      = htons( (WORD)strlen(outstr));
strncpy((char *)&EthCmd.bData[0],outstr,(WORD)strlen(outstr));
```

```
send(sock,(char*)&EthCmd,ETHERNETCMDSIZE + strlen(outstr),0);
recv(sock, szPmacData,1400,0);
```

VR_PMAC_GETMEM

此数据包导致以太网连接来读取Turbo PMAC CPU和以太网微控制器之间共享的双口RAM中的数据。在单个数据包可以接收多达1400个字节。该wValue的字段包含了检索数据的字节偏移量，而wLength参数表示接收多少字节。

例子

```
EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request      = VR_PMAC_GETMEM;
EthCmd.wValue       = htons(offset); //
EthCmd.wIndex       = 0;
EthCmd.wLength      = htons(length);

send(sock,(char *)&EthCmd,ETHERNETCMDSIZE ,0);
recv(sock,(char *)data,1400,0);
```

VR_PMAC_SETMEM

此数据包导致以太网连接将数据写入到Turbo PMAC CPU和以太网微控制器之间共享的双口RAM中。一个数据包中可以写入多达1400字节。该wValue的字段包含了数据写入的字节偏移量，而wLength参数表示写多少字节。在发送数据包之后，程序员必须等待通过recv函数接收到1个字节，然后再继续。接收的数据是无关紧要的，其目的是为了确保接收到发送者的命令。

例子

```
EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request      = VR_PMAC_SETMEM;
EthCmd.wValue       = htons(offset);
EthCmd.wIndex       = 0;
EthCmd.wLength      = htons(length);
```

VR_PMAC_SETBIT

此数据包将导致以太网连接向Turbo PMAC CPU和以太网微控制器之间共享的DPRAM中写入值，可以置位一个32位字的位，或清除一个32位的字的位。如果wIndex参数是1，那么执行一个逻辑或，即将位置位。如果它是0，执行一个逻辑AND，它清除位。这是程序员的责任，利用适当的掩码设置或清除位。wValue的字段包含检索数据的字节偏移量。在发送数据包之后，程序员必须等待通过recv函数接收到1个字节，然后再继续。接收的数据是无关紧要的，其目的是为了确保接收到发送者的命令。

例子

```
DWORD       mask = 0x00000001;

EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request      = VR_PMAC_SETBIT;
EthCmd.wValue       = htons((WORD)offset);
```

```
EthCmd.wIndex        = htons((WORD)on);
EthCmd.wLength       = htons(len);

// generate the mask
mask <<= bitno; // zero based
// If clearing a bit complement mask to prepare firmware for AND
if(!on)
        mask = ~mask;
memcpy(EthCmd.bData,&mask,len);

// Send command request
send(sock,(char *)&EthCmd,ETHERNETCMDSIZE+len,0);
recv(sock,(char *)&errcode,1,0);
```

VR_PMAC_SETBITS

此数据包将导致以太网连接向Turbo PMAC CPU和以太网微控制器之间共享的DPRAM中来执行一个写入动作，将一个32位的字的位设置为一个新值。wValue的字段包含检索数据的字节偏移量。以太网命令数据包的bData字段必须包括一个掩模，表示四个字节中设置哪些位，后面的四个字节用于指示一个32位字中要清除的位。在发送数据包之后，程序员必须等待通过recv函数接收到1个字节，然后再继续。接收的数据是无关紧要的，其目的是为了确保接收到发送者的命令。

例子

```
EthCmd.RequestType = VR_UPLOAD;
EthCmd.Request       = VR_PMAC_SETBITS;
EthCmd.wValue        = htons((WORD)offset);
EthCmd.wIndex        = 0;
EthCmd.wLength       = htons(2*sizeof(DWORD));

temp = 0xFF03FFFF ;
memcpy(EthCmd.bData,&temp,sizeof(DWORD));
temp = 0x00030000 ;
memcpy(EthCmd.bData + 4,&temp,sizeof(DWORD));
// Send command request
send(sock,(char *)&EthCmd,ETHERNETCMDSIZE + 2*sizeof(DWORD),0);
recv(sock,(char *)&errcode,1,0);
```

19-8 数据采集

Turbo PMAC有一个通用数据采集功能，用于重复的即时存储实时数据。在这个功能中，Turbo PMAC可以在不超出伺服中断频率指定的时间间隔内存储多达48个内存位置的内容。这些数据被存储在打开的Turbo PMAC内存的一个缓冲区中供稍后传输到主机。此功能可用于滤波器调谐和解决运动问题。

执行程序数据采集

大多数用户会使用PC上的PMAC执行程序利用此功能，因为它会自动处理它的细节。详细信息参考执行程序的手册。可以（尽管不普遍）写一个自定义的主机程序来使用这个功能。

采集 I-变量

变量I5000控制数据采集缓冲区的位置和模式。I5000的1位控制采集的数据是被存储在Turbo PMAC的主存储器（1位=0），还是存储在可选的双端口RAM（1位= 1）。该PMAC执行程序仅支持使用采集到Turbo PMAC的主内存中的数据。

I5000的0位控制当到达缓冲区的末尾时停止采集，或者是将环绕并继续采集在缓冲区的开头，覆盖以前采集的数据。设置0位为0会导致采集停止在末尾。这是最常见的模式，用于在已知操作序列的过程中采集。

设置I5000的0位为1将启用环绕。这种模式需要用于采集到双口RAM，并实时上传；它可以用于在试图追赶一个间歇性问题时，采集到主内存中。在这种模式下采集时，采集在X：$000000的伺服周期计数器是一个好主意，所以有可能主机计算机软件正确地解包数据。该PMAC执行程序的数据采集和绘图程序可以从一个已经环绕的缓冲区中上传和绘图数据，但它不能以适当的时间顺序解包这个数据来绘制它。

用户可用I-变量I5001到I5048指定多达48个源地址。这些变量的低19位代表字地址本身。高2位控制是使用X字、Y字或两者都使用（以定点或浮点形式）来采集数据。可在24位变量I5050和I5051中设置一个“屏蔽”指定采集这些48个地址的哪一个，并通过I5049定义伺服中断周期中的采集时间。

采集命令

Turbo PMAC的内存的缓冲区是通过在线的**DEFINE GATHER[{常数}]**命令进行设置的。如果没有指定值，那么Turbo PMAC所有打开的内存都被保留用于此缓冲区。（这意味着只要这个空间是被保留的，就没有新的运动或PLC程序可以加入到Turbo PMAC之中）。如果在Turbo PMAC的内存之中存在一个数据采集缓冲区，即使是它不占用所有的打开的内存，也没有其他的通过**DEFINE{buffer}**命令创建的类型的缓冲区（如COMP，LOOKAHEAD）被添加到内存中，直到数据采集缓冲区被删除。

实际的数据采集功能是通过**GATHER**命令开始的。当输入命令后，Turbo PMAC将以指定的速度将指定的数据存储到采集缓冲区中，直到**ENDGATHER**告知它停止，或直到空间耗尽。这些都是在线命令；当来自一个运动程序或PLC程序内时，使用**CMD“GATHER”**和**CMD“ENDGATHER”**语句。

在一个运动程序中，这些语句是在程序计算时间执行的，它远远快于移动执行时间，所以如果你不小心，很容易错过采集最后一个或两个指令运动，因为**CMD“ENDGATHER“**声明是在这些运动实际执行之前发出的。通常情况下，在运动程序中，在CMD“ENDGATHER”声明之前加上一个DWELL语句是一个好主意。DWELL语句停止程序中的任何前瞻，所以**CMD“ENDGATHER”**语句将不会发出，直到暂停的实际执行完成。甚至一个DWELL 0将确保最后的指令运动的末尾都能被采集到；一个较长的停顿可以赶上最后的暂停。

该语句**CMD“GATHER”**和**CMD“ENDGATHER”** 作为Turbo PMAC的后台任务被执行。它们的功能分别是简单地置位和清除X：$000006的位19的数据采集控制位。要做到这一点而且没有后台周期的延迟，你可以分配一个M变量到此位（如M89-> X：$000006,19），然后在程序中直接置位和清除此位。

存储的数据可以通过**LIST GATHER**命令上传到主机。这些数据被以ASCII十六进制形式发送到主机，对于单（X或Y）字，每一项是6个字符；对于双（L或D）字，每一项是12个字符。该数据是以12个字符为一组提供的。如果采样的的数据使得最后一组只有六个字符，该最后的一组将填写伺服周期计数器寄存器X：$000000的内容

它是主机程序的责任来解码并处理该数据，进行绘图、存储、分析或其他用途。

保留用于数据收集缓冲区的空间可以通过使用**DELETE GATHER**命令释放。

通过双端口 RAM 实时数据采集

使用双端口RAM，能够实时执行Turbo PMAC的数据收集功能并将采集的数据上传到主计算机。（标准的数据采集功能---PMAC执行程序使用用于产生图形---实时地执行数据采集，存储到Turbo PMAC的打开的常规内存中，然后上传到主机）。这种实时上传需要主机和Turbo PMAC之间紧密握手通讯稳定，以确保数据被可靠地和有效地传递。

有可能在一些Turbo PMAC系统，特别是在UMAC系统中，存在多个双口RAM芯片。数据收集只用于那些其Turbo PMAC的基址指定在I24中的双口RAM芯片。如果I24被设置为0，使用其Turbo PMAC基地址是$060000的DPRAM IC。

设置

DPRAM的数据采集功能设置方法与标准数据采集功能的设置方法是相同的，通过Turbo PMAC I-变量I5001 - I5051控制采集什么数据和其采集的频率。为了实现指定数据采集入DPRAM，I5000应该被设置为3（它被设置为0用于标准采集）。

临时存储采集数据的缓冲区是通过**DEFINE GATHER{constant}**命令建立的，其中{constant}是缓冲区的大小，单位是Turbo PMAC字（在双口RAM，每个Turbo PMAC字是32位）。缓冲区总是从相对DPRAM的开头偏移$0450的Turbo PMAC地址处开始的（在默认DPRAM基址$060000下，是地址$060450），它从主机侧是双口RAM的基地址加0x1140（4416十进制）个字节。每个短数据源在缓冲区中占用一个32位字，每个长数据源（定点或浮点）占用两个2个32位字。用户必须基于数据源的数量和长度，以及最坏情况下主机从双口RAM读取数据时落后的采集周期数，来设置缓冲区的大小。对于8Kx16双口RAM，此大小不能大于2500。典型大小是20至100字。

双口RAM采集功能的启动和停止与标准采集是相同的：或通过GATHER和ENDGATHER（ENDG）命令，或直接通过M变量置位和清除数据采集控制位。

获取数据

一旦采集功能已经开始，主机必须监视DPRAM中包含加载到DPRAM的数据的指针的寄存器。有两个关键寄存器，并且只有一个需要进行反复读取。在双口RAM的基地址加0x113E（4414十进制）是指向缓冲区的末端的指针。这个值是由DEFINE GATHER命令确定的，并且对于一个给定的应用它是固定的。

在双口RAM的基地址加上0x113C（4412十进制）是指向双口RAM中存入采集数据的下一个地址。正是这个寄存器，主机应反复监测，看它是否已经改变---这意味着新的数据被放置在双口RAM---以及如果它已经改变了，数据已被存入多少次。

这两个寄存器都包含了一个Turbo PMAC内存字地址---实际上是相对采集缓冲区开头的偏移量（相对双口RAM本身开头的$0450）。为翻译成一个主机内存字节地址，应使用下面的公式：

$$主机地址 = (\text{DPRAM基址} + 0x1140) + 4 * 指针值$$

当存储的指针值变成大于或等于缓冲区末端的指针的值时，它将绕回为0（Turbo PMAC地址{DPRAM基址+$0450}）。没有数据项将被存储在缓冲区末端指针所示的起始于Turbo PMAC字地址的DPRAM中，但如果一个长的数据项被开始存储在前一个双口RAM字，第二个的一半将被放置在实际的缓冲区末端字中。

数据格式

数据被以32位符号扩展的形式存储在缓冲区中。也就是说，从Turbo PMAC采集的每个短（24位）字是带符号扩展，并存储在32位的双口RAM（首先是LSByte）。符号位由第23位扩展而来，最显著的字节是全一或全部为零。每个长（48位）字被视为2个24位字，每一个短字符号扩展到32位。主机必须重新装配这些字为单个值。

要重新装配主机上的一个长定点字，取低位32位字，并屏蔽掉符号扩展位（高八位）。在C语言中，此操作可通过一个逐位AND来完成：（LSW&16777215）。把这个结果作为一个无符号整数。其次，取高位字和16,777,216相乘。最后，两个中间结果相加。

要重新装配主机的一个长浮点值，首先要拆分64位值为两部分。低地址的32位字的0到11位生成了指数。这个字的12-23位形成了36位尾数的低12位。另一个32位字的0-23位构成了尾数的高24位；24-31位是符号扩展位。下面的代码显示了完成方法之一（不是最有效）：

```
exp = first_word & 0x00000FFFL      ; // Select low 12 bits for exponent
low_mantissa = (first_word >> 12) & 0x00FFFL;
                                        // Select next 12 bits for mantissa
                                       // shift right and mask
high_mantissa = second_word;
```

然后浮点值可以重构：

```
mantissa = high_mantissa * 4096.0 + low_mantissa;
value = mantissa * pow (2.0, exp - 2047 - 35);
```

从指数中减去2047是将它从一个无符号转换为一个有符号值；减去35，从而把尾数放在0.5至1.0的范围内。

以下过程以C语言高效、稳健地进行转换：

```
double DPRFLoat (long first_word, long second word) {
        // return mantissa/2^35 * 2^(exp-2047)
double mantissa;
long int exp;
m = (double)(second_word) * 4096.0 +
            (double)((first_word>>12) & 0x00FFFL);
if (m == 0.0) return (0.0);
exp = (first_word & 0x00000FFFL) - 2082;      // 2082=2047+35
return (mantissa * pow(2.0 * (double) exp));
}
```

要在主机中重新装配一个长浮点字，对待低32位字与上述定点情况是一样的。取高32字的低12位（MSW＆4095），乘以16,777,216，并加到屏蔽后的低32字中。这形成了浮点值的尾数。现在取下一个12位（MSW＆16773120）。这是以2为底的指数，可以与尾数进行组合以形成完整的值。

